



RISC-V 指令集手册

第 1 卷

非特权架构

版本：20240411

（注：实际内容为 2025 年 5 月 1 日发布版）

目 次

目 次	I
序（中文版）	VII
序（英文版）	VIII
前 言	IX
RISC-V 指令集手册 第一卷：非特权架构	1
1 引言	1
1.1 RISC-V 硬件平台术语	1
1.2 RISC-V 软件执行环境和硬件线程	2
1.3 RISC-V ISA 概述	3
1.4 内存	5
1.5 基础指令长度编码	5
1.6 异常、陷阱和中断	7
1.7 未指定的行为和值	9
2 RV32I 基础整数指令集 V2.1	9
2.1 基础整数 ISA 的程序员模型	9
2.2 基本指令格式	11
2.3 立即数编码变体	12
2.4 整数计算指令	13
2.5 控制转移指令	16
2.6 加载和存储指令	19
2.7 内存排序指令	21
2.8 环境调用和断点	22
2.9 HINT 指令	23
3 RV32E 与 RV64E 基础整数指令集, V2.0	24
3.1 RV32E 与 RV64E 程序员模型	24
3.2 RV32E 与 RV64E 指令集编码	25
4 RV64I 基础整数指令集, V2.1	25
4.1 寄存器状态	25
4.2 整数计算指令	25
4.3 加载和存储指令	27
4.4 HINT 指令	27
5 RV128I 基础整数指令集, V1.7	28
6 Zifencei 扩展：指令获取屏障, V2.0	29
7 Zicsr 拓展：控制状态寄存器（CSR）指令, V2.0	30
7.1 控制状态寄存器指令	31
8 Zicntr 和 Zihpm 扩展：计数器, V2.0	33
8.1 Zicntr 扩展：基本计数器和定时器	34

8.2 Zihpm 扩展: 硬件性能计数器	36
9 Zihintntl 扩展: 非时间局部性提示, V1.0	36
10 Zihintpause 扩展: PAUSE 提示指令, V2.0	38
11 Zimop 扩展: 可能操作 (May-Be-Operations), V1.0	39
11.1 Zcmop 扩展: 可能操作的压缩 (Compressed May-Be-Operations)	40
12 Zicond 扩展: 整数条件操作, V1.0.0	41
12.1 介绍	41
12.2 Zicond 规范	41
12.3 指令 (按字母顺序)	42
12.4 使用示例	43
13 M 扩展: 用于整数乘法和除法, V2.0	43
13.1 乘法操作 (Multiplication Operations)	44
13.2 除法操作 (Division Operations)	44
13.3 Zmmul 扩展: 版本 1.0	45
14 A 扩展: 原子指令, V2.1	45
14.1 用于确定访问顺序的原子指令	45
14.2 Zalrsc 扩展: 用于 Load-Reserved/Store-Conditional 指令	46
14.3 存储条件指令的最终成功	48
14.4 Zaamo 扩展: 原子内存操作	49
15 Zawrs 扩展: 等待预留集指令, V1.01	51
15.1 等待预订设置说明	52
16 Zacas 扩展: 用于原子比较并交换 (CAS) 指令, V1.0.0	52
16.1 介绍	52
16.2 字/双字/四字 CAS (AMOCAS.W/D/Q)	53
16.3 额外的 AMO PMA	56
17 RVWMO 内存一致性模型, V2.0	56
17.1 RVWMO 内存模型的定义	57
17.2 CSR 依赖性跟踪粒度	60
17.3 源寄存器与目标寄存器列表	60
18 Ztso 扩展: 总存储排序扩展, V1.0	62
19 CMO 扩展: 基础缓存管理操作 ISA, V1.0.0	62
19.1 指令语义的伪代码	63
19.2 介绍	63
19.3 背景	63
19.4 一致代理和缓存	64
19.5 控制状态寄存器状态	69
19.6 扩展	71
19.7 指令	73
20 F 扩展: 单精度浮点, V2.2	76
20.1 F 寄存器状态	76
20.2 浮点控制状态寄存器	78
20.3 NaN 生成和传播	79
20.4 次正规数运算	79
20.5 单精度浮点加载和存储指令	80

20.6	单精度浮点计算指令	80
20.7	单精度浮点转换和移动指令	81
20.8	单精度浮点比较指令	83
20.9	单精度浮点分类指令	83
21	D 扩展: 双精度浮点指令, V2.2	84
21.1	D 寄存器状态	84
21.2	较窄值的 NaN 装箱 (NaN Boxing)	84
21.3	双精度加载和存储指令	85
21.4	双精度浮点计算指令	85
21.5	双精度浮点转换与移动指令	86
21.6	双精度浮点比较指令	87
21.7	双精度浮点分类指令	87
22	Q 扩展: 四精度浮点, V2.2	87
22.1	四精度加载和存储指令	87
22.2	四精度计算指令	88
22.3	四精度转换与移动指令	88
22.4	四精度浮点数比较指令	89
22.5	四精度浮点数分类指令	89
23	Zfh 和 Zfhmin 扩展: 用于半精度浮点, V1.0	89
23.1	半精度加载和存储指令	90
23.2	半精度计算指令	90
23.3	半精度转换和移动指令	90
23.4	半精度浮点比较指令	91
23.5	半精度浮点分类指令	92
23.6	Zfhmin 标准扩展: 用于最小半精度浮点	92
24	Zfa 扩展: 额外浮点指令, V1.0	92
24.1	立即数加载指令	92
24.2	最小值和最大值指令	94
24.3	四舍五入到整数指令	94
24.4	模块化向整数转换指令	94
24.5	移动指令	95
24.6	比较指令	95
25	ZfinxZdinxZhinxZhinxmin 扩展: 整数寄存器中的浮点, V1.0	95
25.1	窄位数值的处理	96
25.2	Zdinx	96
25.3	宽位数值的处理	96
25.4	Zhinx	97
25.5	hinxmin	97
25.6	特权架构的影响	97
26	C 扩展: 压缩指令, V2.0	97
26.1	概述	98
26.2	压缩指令格式	99
26.3	加载和存储指令	101
26.4	控制转移指令	103

26.5	整数计算指令	104
26.6	在 LR/SC 序列中使用 C 指令	107
26.7	HINT 指令	108
26.8	RVC 指令集列表	108
27	Zc*扩展: 代码大小缩减扩展, V1.0.0	111
27.1	Zc* 概述	111
27.2	C	111
27.3	Zce	112
27.4	MISA.C	112
27.5	Zca	112
27.6	Zcf (仅 RV32)	112
27.7	Zcd	112
27.8	Zcb	113
27.9	Zcmp	113
27.10	Zcmt	113
27.11	Zc 指令格式	113
27.12	Zcb 指令	114
27.13	PUSH/POP 寄存器指令	122
27.14	表跳转概述	144
28	B 扩展: 位操作指令, V1.0.0	148
28.1	位操作扩展 (a、b、c 和 s 扩展) 已合并进入公开评审及批准流程	148
28.2	字指令	148
28.3	指令语义伪代码规范 本规范采用类 SAIL 语法描述指令集中各条指令的语义。	148
28.4	扩展	148
28.5	指令 (按字母顺序排列)	153
28.6	软件优化指南	187
29	J 扩展: 用于动态翻译语言, V0.0	190
30	P 扩展: 打包式 SIMD 指令, V0.2	190
31	V 扩展: 向量操作, V1.0	190
31.1	介绍	191
31.2	实现定义的常量参数	191
31.3	向量扩展编程模型	191
31.4	向量元素到向量寄存器状态的映射	200
31.5	向量指令格式	204
31.6	配置设定指令 (vsetvli/vsetivli/vsetvl)	209
31.7	向量加载与存储	213
31.8	向量内存对齐约束	225
31.9	向量内存一致性模型	225
31.10	向量算术指令格式	226
31.11	向量整数算术指令	229
31.12	向量定点算术指令	242
31.13	向量浮点指令	245
31.14	向量归约操作	254
31.15	向量掩码指令	257

31.16	向量置换指令	265
31.17	异常处理	272
31.18	标准向量扩展	273
31.19	向量指令列表	276
32	加密扩展：标量和熵源指令，V1.0.1	282
32.1	更新日志	282
32.2	介绍	282
32.3	扩展概述	283
32.4	指令	287
32.5	熵源	322
32.6	数据独立执行延迟子集：Zkt	326
32.7	指令设计原理	331
32.8	熵源设计理由和建议	334
32.9	补充材料	343
32.10	支持的 Sail 代码	343
33	加密扩展：向量指令，V1.0.0	351
33.1	介绍	351
33.2	扩展概述	355
33.3	指令	364
33.4	向量加密指令	406
33.5	向量位操作和无进位乘法指令	407
33.6	支持的 Sail 代码	409
34	RV32/64G 指令集列表	418
35	扩展 RISC-V	427
35.1	扩展术语	427
35.2	RISC-V 扩展设计理念	429
35.3	固定宽度 32 位指令格式内的扩展	429
35.4	添加对齐的 64 位指令扩展	430
35.5	支持 VLIW 编码	430
36	ISA 扩展命名约定	431
36.1	大小写敏感性	431
36.2	基础整数 ISA	431
36.3	指令集扩展名称	431
36.4	版本号	432
36.5	下划线	432
36.6	其他标准扩展名称	432
36.7	监督级指令集扩展	432
36.8	虚拟机级指令集扩展	432
36.9	机器级指令集扩展	432
36.10	非标准扩展名称	432
36.11	子集命名约定	433
37	历史与致谢	433
37.1	为什么开发新的 ISA？——来自伯克利小组的理由	433
37.2	ISA 手册 1.0 版的历史	435

37.3 ISA 手册 2.0 版的历史	435
37.4 致谢	437
37.5 ISA 手册 2.1 版的历史	437
37.6 致谢	437
37.7 ISA 手册 2.2 版的历史	437
37.8 致谢	437
37.9 ISA 手册 2.3 版的历史	437
37.10 资金支持	437
附录 A: RVWMO 解释材料, V0.1	438
A.1 为什么选择 RVWMO?	438
A.2 试纸测试	438
A.3 解释 RVWMO 规则	440
A.4 超越主内存	450
A.5 代码移植和映射指南	451
A.6 实现指南	454
A.7 已知问题	456
附录 B: 正式内存模型规范, V0.1	457
B.1 Alloy 中的正式公理规范	457
B.2 Herd 中的正式公理规范	461
B.3 操作内存模型	463
附录 C: 正式内存模型规范	474
C.1 向量-向量加法示例	474
C.2 混合位宽掩码与计算示例	475
C.3 内存复制 (Memcpy) 示例	475
C.4 条件运算示例	476
C.5 SAXPY (线性代数运算) 示例	476
C.6 SGEMM (矩阵乘法) 示例	477
C.7 除法近似计算示例	482
C.8 平方根近似计算示例	482
C.9 C 标准库 strcmp 函数示例	482
C.10 分数阶向量乘法 (Fractional Lmul) 示例	483
附录 D: 向量状态调用约定 (非权威性 - 仅作占位说明)	487

序（中文版）

该版本由中国电子工业标准化技术协会 RISC-V 工作委员会技术委员会组织翻译。

该版本的主要贡献人包括：唐丹、颀靖、唐旖浓、陈振宇、陈俊岳、刘永艳、许文琪、何家桢、管思涵、安南、朱孝娟、李振杰、贺屹、路亮。

该版本的英文原版遵从 CC BY 4.0（署名 4.0 协议国际版）许可协议，中文翻译版在遵从 CC BY 4.0 许可证协议基础上，同时遵从 MulanOWL BY-SS（木兰开放作品署名-相同共享）许可协议。该版本在遵从上述许可协议的前提下，允许在全球范围内非独占的、永久的、合规的被拷贝、展示、更改、分发、演绎和共享。

序（英文版）

Contributors to all versions of the spec in alphabetical order (please contact editors to suggest corrections): Derek Atkins, Arvind, Krste Asanović, Rimas Avižienis, Jacob Bachmeyer, Christopher F. Batten, Allen J. Baum, Abel Bernabeu, Alex Bradbury, Scott Beamer, Hans Boehm, Preston Briggs, Christopher Celio, Chuanhua Chang, David Chisnall, Paul Clayton, Palmer Dabbelt, L Peter Deutsch, Ken Dockser, Paul Donahue, Aaron Durbin, Roger Espasa, Greg Favor, Andy Glew, Shaked Flur, Stefan Freudenberger, Marc Gauthier, Andy Glew, Jan Gray, Gianluca Guida, Michael Hamburg, John Hauser, John Ingalls, David Horner, Bruce Houlton, Bill Huffman, Alexandre Joannou, Olof Johansson, Ben Keller, David Kruckemyer, Tariq Kurd, Yunsup Lee, Paul Loewenstein, Daniel Lustig, Yatin Manerkar, Luc Maranget, Ben Marshall, Margaret Martonosi, Phil McCoy, Nathan Menhorn, Christoph Müllner, Joseph Myers, Vijayanand Nagarajan, Rishiyur Nikhil, Jonas Oberhauser, Stefan O'Rear, Markku-Juhani O. Saarinen, Albert Ou, John Ousterhout, Daniel Page, David Patterson, Christopher Pulte, Jose Renau, Josh Scheid, Colin Schmidt, Peter Sewell, Susmit Sarkar, Ved Shanbhogue, Brent Spinney, Brendan Sweeney, Michael Taylor, Wesley Terpstra, Matt Thomas, Tommy Thorn, Philipp Tomsich, Caroline Trippel, Ray VanDeWalker, Muralidaran Vijayaraghavan, Megan Wachs, Paul Wamsley, Andrew Waterman, Robert Watson, David Weaver, Derek Williams, Claire Wolf, Andrew Wright, Reinoud Zandijk, and Sizhuo Zhang.

This document is released under a Creative Commons Attribution 4.0 International License.

This document is a derivative of The RISC-V Instruction Set Manual, Volume I: User-Level ISA Version 2.1 released under the following license: ©2010-2017 Andrew Waterman, Yunsup Lee, David Patterson, Krste Asanović. Creative Commons Attribution 4.0 International License. Please cite as: The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 20191214-draft, Editors Andrew Waterman and Krste Asanović, RISC-V Foundation, December 2019.

前 言

本文档描述了RISC-V的非特权架构。

本文档中标记为批准的指令集架构（ISA）模块已经批准。标记为冻结的模块不希望在正式提交批准前进行重大变更。标记为起草的模块希望在批准前变更。

本文档包括以下版本的RISC-V ISA模块：

基础指令集	版本	状态
RV32I	2.1	批准
RV32E	2.0	批准
RV64E	2.0	批准
RV64I	2.1	批准
RV128I	1.7	草案
扩展指令集	版本	状态
Zifencei	2.0	批准
Zicsr	2.0	批准
Zicntr	2.0	批准
Zihintntnl	1.0	批准
Zihintpause	2.0	批准
Zimop	1.0	批准
Zicond	1.0	批准
M	2.0	批准
Zmmul	1.0	批准
A	2.1	批准
Zawrs	1.01	批准
Zacas	1.0	批准
RVWMO	2.0	批准
Ztso	1.0	批准
CMO	1.0	批准
F	2.2	批准
D	2.2	批准
Q	2.2	批准
Zfh	1.0	批准
Zfhmin	1.0	批准
Zfa	1.0	批准
Zfinx	1.0	批准
Zdinx	1.0	批准
Zhinx	1.0	批准

扩展指令集	版本	状态
Zhinxmin	1.0	批准
C	2.0	批准
*Zce	1.0	批准
B	1.0	批准
P	0.2	草案
V	1.0	批准
*Zbkb	1.0	批准
*Zbkc	1.0	批准
*Zbkx	1.0	批准
*Zk	1.0	批准
*Zks	1.0	批准
*Zvbb	1.0	批准
*Zvbc	1.0	批准
*Zvkg	1.0	批准
*Zvkned	1.0	批准
*Zvknhb	1.0	批准
*Zvksed	1.0	批准
*Zvksh	1.0	批准
*Zvkt	1.0	批准

该版本变更的内容包括：

- 纳入了截至2024年3月所有已批准的扩展内容
- 移除了Zam扩展草案，改为采用未对齐原子粒度PMA的定义方案

注：原文中涉及20191213、20190608版以及2.0、2.1和2.2版的前言未在本文件中翻译，若需要了解，请阅读原文。

RISC-V 指令集手册

第一卷：非特权架构

1 引言

RISC-V（读作“risc-five”）作为一种新型指令集架构（ISA），设计之初是为了计算机架构研究和教学，现在希望成为产业实现的免费标准和开放架构。我们定义 RISC-V 的目标包括：

- 学术界和产业界可免费获得、完全开放的 ISA。
- 具备原生硬件实现能力的实用型 ISA，不局限于仿真或二进制翻译。
- 避免对特定微架构风格（如微编码、顺序执行、解耦、乱序执行）或实现技术（如全定制、ASIC、FPGA）“过度架构”的一种 ISA，同时又能有效支持上述微架构风格和实现技术。
- 划分成精简基础整数的 ISA，既可用于定制化的加速器，也适用于教学目的，同时可进行标准扩展，以支持通用软件开发。
- 支持 2008 年修订的 IEEE-754 浮点标准。
- 支持各种扩展和专用变体的 ISA。
- 提供 32 位和 64 位地址空间变体，满足应用程序、操作系统内核及硬件实现需求。
- 支持高度并行的多核/众核集群实现（含异构多处理器）。
- 可选的可变长度指令，既可扩展指令编码空间，也能通过紧凑指令编码提升性能、减少静态代码规模并提高能效。
- 支持完全虚拟化的 ISA，便于虚拟机监视器（Hypervisor）开发。
- 支持新型特权架构设计简化实验的 ISA。



对设计决策的评论如本段所述。仅关注规范本身的读者可跳过这些非规范性内容。



选择命名为 RISC-V 是为了表示加州大学伯克利分校设计的第五代主要 RISC ISA（前四代分别是：RISC-I、RISC-II、SOAR 和 SPUR）。利用罗马数字 V 既表示变体（variations），也表示向量（vectors），目的是支持各种架构研究，包括各种数据并行加速器，这是该 ISA 设计的明确目标。

RISC-V ISA 的定义尽量避免涉及实现细节（尽管包含了关于实现驱动决策的注释）。这些定义应理解为适用于多种实现的软硬件可见接口，而不是特定硬件制品的设计。RISC-V 手册分为两卷，本卷描述了基础的非特权指令，包括可选的非特权 ISA 扩展。非特权指令是指通常情况下在所有特权架构的所有特权模式中都可用的指令，但是其行为可能会因特权模式和特权架构的不同而有所变化。第二卷则提供了第一版（经典）特权架构的设计。手册遵循 IEC 80000-13:2008 标准，一个字节等于 8 位。



在非特权 ISA 设计中，尽量去除了对特定微架构特性的依赖（例如缓存行大小），以及对特权架构细节的依赖（例如页表翻译），目的是简化设计，并为其他微架构或特权架构提供最大限度的灵活性。

1.1 RISC-V 硬件平台术语

一个 RISC-V 硬件平台可以包含一个或多个兼容 RISC-V 的处理核，以及其他 RISC-V 不兼容的核、固定功能的加速器、各种物理内存结构、I/O 设备，以及支持这些组件通信的互连结构。

如果一个组件包含独立的指令获取单元，则称其为核。一个兼容 RISC-V 的核可通过多线程支持多个与 RISC-V 兼容的硬件线程（或称为 harts）。

一个 RISC-V 核可能具有额外的专用指令集扩展或一个附加的协处理器（coprocessor）。使用协处理器这个术语指代连接到 RISC-V 核的单元，主要由 RISC-V 指令流控制，但包含额外的架构状态和指令集扩展，相对于主要的 RISC-V 指令流可能具有一定的有限自主性。

使用加速器（accelerator）一词指代不可编程的固定功能单元，或能够自主运行但专用于特定任务的核。在 RISC-V 系统中，预计许多可编程加速器是基于 RISC-V 的核，并具有专用指令集扩展和/或定制的协处理器。RISC-V 加速器中一个重要的类别是 I/O 加速器，主要功能是卸载主应用核中的 I/O 处理任务。

一个 RISC-V 硬件平台的系统级组织可以从单核微控制器扩展到拥有成千上万节点的共享内存众核服务器集群。即使是小型片上系统（SoC），也可能通过多计算机和/或多处理器的分层架构进行模块化开发，或者在子系统之间提供安全隔离。

1.2 RISC-V 软件执行环境和硬件线程

RISC-V 程序的行为取决于其运行的执行环境。RISC-V 执行环境接口（简称 EEI）定义了程序的初始状态、环境中 harts 的数量和类型（包括 harts 支持的特权模式）、内存和 I/O 区域的可访问性及属性、每个 hart 执行的所有合法指令的行为（即 ISA 是 EEI 的一个组成部分），以及在执行过程中引发的任何中断或异常（包括环境调用）的处理方式。EEI 的例子包括 Linux 应用程序二进制接口（ABI）或 RISC-V 监督二进制接口（SBI）。RISC-V 执行环境的实现可以是纯硬件、纯软件，或硬件与软件的结合。例如，可以使用操作码陷阱和软件仿真来实现硬件未提供的功能。执行环境实现的示例如下：

- * 裸硬件平台：harts 直接由物理处理器线程实现，指令可以完全访问物理地址空间。硬件平台定义了一个从上电复位（power-on reset）开始的执行环境。

- * RISC-V 操作系统：通过将用户级 harts 多路复用到可用的物理处理器线程上，并通过虚拟内存控制对内存的访问，提供多个用户级执行环境。

- * RISC-V 虚拟机监视器（Hypervisors）：虚拟机监视器为客户操作系统提供多个监督级执行环境。

- * RISC-V 仿真器：例如 Spike、QEMU 或 rv8，这些仿真器在底层 x86 系统上仿真 RISC-V 的 harts，可以提供用户级或监督级执行环境。



裸硬件平台可视为定义了一个执行环境接口（EEI），其中可访问的 harts、内存和其他设备构成了环境，其初始状态为上电复位时的状态。通常，大多数软件的设计是为了使用更抽象的硬件接口，因为更抽象的 EEI 提供了跨硬件平台的更多可移植性。通常，EEI 会层层叠加，较高层的 EEI 使用较低层的 EEI。

从运行在特定执行环境中的软件视角来看，hart 是一种资源，可以在该执行环境中自主获取并执行 RISC-V 指令。从这个意义上讲，尽管 hart 在执行环境中通过时间复用方式运行在真实硬件上，但其行为类似于硬件线程资源。一些 EEI 支持创建和销毁额外的 harts，例如通过环境调用来派生新的 harts。

执行环境负责确保每个 hart 最终都能向前推进。对于特定的 hart，当它执行某种明确等待事件的机制时，例如本规范第二卷中定义的等待中断（wait-for-interrupt）指令，这一责任会暂时中止；当 hart 被终止时，这一责任则结束。以下事件构成了向前推进的标志：

- * 一条指令的退役（retirement）。
- * 第 1.6 节中定义的陷进（trap）。
- * 由扩展定义的其他事件。

术语hart最早在Lithe的研究中引入，用来表示一种抽象的执行资源，而不是软件线程的编程抽象。

硬件线程与软件线程上下文之间的重要区别在于运行在执行环境中的软件不需要负责促使每个hart 继续运行，而是由外部执行环境负责。因此，从执行环境内软件的视角来看，环境提供的 harts 就像硬件线程一样运行。



一个执行环境的实现可能会通过时间复用（time-multiplexing）将一组客户 harts 映射到更少的由其自身执行环境提供的宿主 harts 上，但必须以一种使客户 harts 像独立硬件线程一样运行的方式进行。具体而言，如果客户 harts 的数量多于宿主 harts，则执行环境必须能够抢占(preempt) 客户 harts，而不能无限期地等待运行在某个客户 hart 上的软件释放对该客户 hart 的控制权。

1.3 RISC-V ISA 概述

RISC-V ISA 定义为一个基础整数 ISA，任何实现中都必须包含该部分，并可以根据需要增加可选的扩展。RISC-V 的基础整数 ISA 与早期的 RISC 处理器非常相似，区别在于没有分支延迟槽，并支持可选的可变长度指令编码。将基础 ISA 精心限制为一组最小化的指令集，足以为编译器、汇编器、链接器和操作系统（包括额外的特权操作）提供合适的基础，同时作为一个方便的 ISA 和软件工具链的“骨架”，可以围绕其构建更多定制化的处理器 ISA。

尽管业界通常谈论 RISC-V ISA，但实际上 RISC-V 是一组相关的 ISAs 家族，目前包含四种基础 ISAs。每个基础整数指令集的特点在于整数寄存器的宽度、对应的地址空间大小以及整数寄存器的数量。主要有两种基础整数变体：RV32I 和 RV64I，分别提供 32 位和 64 位地址空间，具体描述见第 2 章和第 4 章。这里使用术语 XLEN 指代整数寄存器的宽度（以位为单位，取值为 32 或 64）。第 6 章描述了 RV32E 和 RV64E，这些是 RV32I 和 RV64I 基础指令集的子集变体，它们的整数寄存器的数量减半，设计用于支持小型微控制器。第 8 章概述了未来的 RV128I 基础整数指令集变体，该变体支持平坦的 128 位地址空间（XLEN=128）。基础整数指令集使用二进制补码（twos complement）表示有符号整数值。



虽然64位地址空间对于更大系统十分必要，但业界认为32位地址空间在未来几十年内仍然足够满足许多嵌入式设备和客户端设备的需求，并且对降低内存流量和能耗来说更具优势。此外，32位地址空间对教学目的来说也足够。未来可能需要更大的128位平坦地址空间。因此需要确保RISC-V ISA 能够支持这一需求。

RISC-V 中的四种基础 ISA 被视为独立的基础 ISA。一个常见的问题是：为什么不设计成单一的 ISA？尤其是，为什么 RV32I 不是 RV64I 的严格子集？一些早期的 ISA 设计（例如 SPARC、MIPS）采用了严格的超集策略，以便在扩展地址空间支持新 64 位硬件时能够运行现有的32位二进制程序。



明确分离基础 ISA 的主要优势在于每个基础 ISA 都可以针对自身需求进行优化，而无需支持其他基础 ISA 所需的所有操作。例如，RV64I 可以省略仅用于应对 RV32I 中较窄寄存器的指令和 CSR，而 RV32I 变体则可以利用本来保留给宽地址空间变体指令的编码空间。

不将设计视为单一ISA的主要缺点是在一种基础ISA上模拟另一种基础 ISA（例如在 RV64I 上模拟 RV32I）会增加硬件复杂性。然而，由于地址和非法指令异常处理的差异，即使采用完整的超集指

令编码，硬件仍然需要某种模式切换。此外，不同的 RISC-V 基础 ISA 十分相似，因此支持多个版本的成本相对较低。尽管有人提出严格超集设计可以让遗留的32位库与64位代码进行链接，但实际上由于软件调用约定和系统调用接口的差异，即使编码兼容，这种方式在实践中也是不可行的。

RISC-V 特权架构在 `misa` 寄存器中提供了字段，用于控制每个级别上的非特权 ISA，从而支持在同一硬件上模拟不同的基础ISA。值得注意的是，更新版本的SPARC和 MIPS ISA 修订版已不再支持 64 位系统上运行未修改的32位代码。

一个相关的问题是为什么RV32I (ADD)和RV64I (ADDW)的32位加法有不同的编码？实际上，ADDW 操作码可以用于 RV32I 中的32位加法， ADDD操作码可以用于 RV64I中的64位加法，这样就可以替代当前的设计：在 RV32I 中使用相同的ADD操作码进行32位加法，在 RV64I中用不同的ADDW操作码进行32位加法。这样做会更加一致，类似于RV32I和RV64I中32位加载指令都使用相同的LW操作码。RISC-V ISA 的最初版本确实采用了这种替代设计，但在2011年1月，RISC-V 的设计被修改为当前的选择。重点是支持64位ISA中的32位整数，而不是提供与32位ISA的兼容性，动机是消除由于并非所有 RV32I 操作码都带有 *W 后缀（例如，ADDW，但AND没有ANDW）而产生的不对称性。事后来，这种设计可能并不完全合理，主要是因为两种ISA是同时进行设计的，而不是一个在另一个基础上添加。同时业界也认为必须将平台需求纳入ISA规范，这意味着RV64I中必须支持所有RV32I指令。现在已经为时过晚，无法更改编码，但基于上述原因，这对实际应用几乎没有影响。

有观点认为，可以将*W变体作为扩展启用到RV32I系统中，从而在RV64I和未来的RV32变体之间提供统一的编码。

RISC-V 支持定制化和专用化。每个基础整数 ISA 都可以通过一个或多个可选的指令集扩展来进行扩展。扩展的类型分为标准扩展、自定义扩展和不相容扩展。为此，将每个 RISC-V 指令集编码空间（以及相关的编码空间，例如 CSR）划分为三个不相交的类别：标准编码、保留编码和自定义编码。标准扩展和编码由RISC-V 国际协会定义；任何未由RISC-V 国际协会定义的扩展均为非标准扩展。每个基础 ISA 及其标准扩展仅使用标准编码，并且在使用这些编码时相互不冲突。保留编码当前未定义，但保留用于未来的标准扩展；一旦被使用，将成为标准编码。自定义编码永远不会用于标准扩展，而是提供给特定厂商的非标准扩展使用。非标准扩展可以是只使用自定义编码的自定义扩展，或者是使用任何标准或保留编码的不相容扩展。指令集扩展通常是共享的，但可根据基础 ISA 提供略微不同的功能。扩展部分描述了扩展 RISC-V ISA 的各种方法。此外，还制定了 RISC-V 基础指令和指令集扩展的命名规范，详细描述请参见第 37 章。

为了支持通用软件开发，定义了一组标准扩展，以提供整数乘法/除法、原子操作以及单精度和双精度浮点运算。基础整数 ISA 被命名为 I（根据整数寄存器宽度加上 RV32 或 RV64 前缀），包含整数计算指令、整数加载、整数存储和控制流指令。标准整数乘法和除法扩展命名为 M，增加了在整数寄存器中进行乘法和除法操作的指令。标准原子指令扩展记为 A，增加了用于多处理器同步的原子性读取、修改和写入内存的指令。标准单精度浮点扩展记为 F，增加了浮点寄存器、单精度计算指令以及单精度加载和存储。标准双精度浮点扩展记为 D，扩展了浮点寄存器，并增加了双精度计算指令、加载和存储。标准 C 压缩指令扩展提供了常用指令更窄的 16 位形式。

尽管新的指令可能对某个领域非常有用，但除了基础整数 ISA 及其标准扩展之外，我们认为很难对所有应用程序提供显著的优势。能效要求日益提高，促进了更大程度的专用化，简化 ISA 规范中必需的部分就非常重要。其他架构通常将其 ISA 视为一个整体，随着时间的推移不断添加新指令而发布新版本；RISC-V 力求保持基础 ISA 和每个标准扩展的稳定性，新增指令则作为额外的可选扩展逐层添加。例如，基础整数 ISA 将始终作为独立的、完全支持的 ISA 存在，不受后续扩展影响。

1.4 内存

RISC-V 的一个 hart 拥有一个单字节可寻址的地址空间，大小为 2^{XLEN} 字节，用于所有内存访问。内存中的一个字 (word) 定义为 32 位 (4 字节)。相应地，半字 (halfword) 为 16 位 (2 字节)，双字 (doubleword) 为 64 位 (8 字节)，四字 (quadword) 为 128 位 (16 字节)。内存地址空间是循环的，地址为 $2^{\text{XLEN}-1}$ 的字节与地址为 0 的字节相邻。因此，硬件执行的内存地址计算会忽略溢出，并以 2^{XLEN} 为模运算进行循环。

执行环境决定硬件资源在 hart 地址空间中的映射。hart 地址空间的不同地址范围可能包括：(1) 包含主存，或 (2) 包含一个或多个 I/O 设备。对 I/O 设备的读写可能会产生可见的副作用，而对主存的访问则不会。尽管执行环境可以将 hart 地址空间中的所有内容都视为 I/O 设备，但通常期望将一部分地址范围指定为主存。

当一个 RISC-V 平台包含多个 hart 时，任意两个 hart 的地址空间可以完全相同，也可以完全不同，或者部分不同但共享某些资源。这些资源可以映射到相同或不同的地址范围中。



对于一个纯裸机环境，所有的hart可能会看到相同的地址空间，并完全通过物理地址进行访问。然而，当执行环境包含一个使用地址转换的操作系统时，通常每个hart会被分配一个几乎完全独立的虚拟地址空间。

每条 RISC-V 机器指令的执行都涉及一个或多个内存访问，这些访问分为隐式和显式两类。对于每条被执行的指令，都会进行一次隐式内存读取（指令取值，或简称为取指），以获取要执行的编码指令。许多 RISC-V 指令在指令取值之外不再进行额外的内存访问。特定的加载 (load) 和存储 (store) 指令会显式地读取或写入由指令确定地址的内存。执行环境可能要求指令执行期间除了无特权 ISA 文档中记录的内容外，还进行其他隐式内存访问（例如地址转换的实现）。

执行环境决定了非空闲地址空间的哪些部分可用于哪种类型的内存访问。例如，指令取值可以隐式读取的地址范围可能与加载指令显式读取的地址范围完全不同；而存储指令可以显式写入的地址范围可能只是可读取地址范围的一个子集。通常，如果一条指令试图访问一个不可访问的地址，则会引发该指令的异常。地址空间中的空白位置永远不可访问。

除非另有说明，否则不会引发异常且无副作用的隐式读取可以在任意时刻提前执行和推测性发生，甚至机器在逻辑上还无法确定是否需要该读取。例如，一个合法的实现可以在尽早的时机尝试读取整个主存，将尽可能多的可取值（可执行）字节缓存起来以供后续指令取值使用，并避免为指令取值再次读取主存。为了确保某些隐式读取在写入同一内存位置之后才进行，软件必须执行为此目的定义的特定屏障 (fence) 或缓存控制指令（例如第 6 章中定义的 FENCE.I 指令）。

一个 hart 执行的内存访问(隐式或显式)在其他 hart 或可以访问相同内存的任何其他代理看来，可能呈现出不同的顺序。然而，这种感知到的内存访问重排序始终受到适用的内存一致性模型的约束。RISC-V 的默认内存一致性模型是 RISC-V 弱内存排序 (RVWMO)，其定义见第 17 章及相关附录。作为选择，一个实现可以采用更强的全存储排序模型 (Total Store Ordering)，定义见第 18 章。执行环境也可能增加额外的限制，进一步约束内存访问的感知重排序。由于 RVWMO 模型是所有 RISC-V 实现允许的最弱模型，为此模型编写的软件与所有 RISC-V 实现的实际内存一致性规则兼容。与隐式读取一样，软件必须执行屏障或缓存控制指令，以确保超出默认内存一致性模型和执行环境要求的内存访问顺序。

1.5 基础指令长度编码

基础 RISC-V ISA 使用固定长度的 32 位指令，必须自然对齐到 32 位边界。然而，标准 RISC-V 编码方案旨在支持具有可变长度指令的 ISA 扩展，其中每条指令可以由任意数量的 16 位指令段 (parcels) 组成，而这些段自然对齐到 16 位边界。第 26 章中描述的标准压缩 ISA 扩展通过提供压缩

的 16 位指令来减少代码大小，并放宽对齐约束，允许所有指令（16 位和 32 位）对齐到任意 16 位边界，从而提升代码密度。

这里使用 IALIGN（以位为单位）表示实现强制执行的指令地址对齐约束。在基础 ISA 中，IALIGN 是 32 位，但某些 ISA 扩展（包括压缩 ISA 扩展）将 IALIGN 放宽到 16 位。IALIGN 仅允许取值 16 或 32。

这里使用 ILEN（以位为单位）表示实现支持的最大指令长度，并且该值始终是 IALIGN 的倍数。对于仅支持基础指令集的实现，ILEN 是 32 位。支持更长指令的实现会有更大的 ILEN 值。

1.5.1 扩展指令长度编码

一部分 32 位指令编码空间被暂时分配用于长度超过 32 位的指令。目前，这部分空间完全保留，而以下针对超过 32 位指令的编码提案尚未冻结。

标准的指令集扩展采用超过 32 位的编码时，附加的低位会被设置为 1，其中 48 位和 64 位长度的编码约定如表 1.1 所示。指令长度在 80 位到 176 位之间的编码使用位 [14:12] 的 3 位字段来表示比初始的 5×16 位字多出的 16 位字的数量。位 [14:12] 设置为 111 的编码保留用于未来更长指令的编码。

表 1.1 展示了标准 RISC-V 指令长度编码约定。基础 ISA 中的所有 32 位指令的最低两位被设置为 11。可选的压缩 16 位指令集扩展的最低两位为 00、01 或 10。

表 1.1 RISC-V 指令长度编码

			xxxxxxxxxxxxxaa	16-bit (aa≠11)
		xxxxxxxxxxxxxxxx	xxxxxxxxxxxxbb11	32-bit (bbb≠111)
	· · · · · xxxx	xxxxxxxxxxxxxxxx	Xxxxxxxxxx011111	48-bit
	· · · · · xxxx	xxxxxxxxxxxxxxxx	xxxxxxxxx011111	64-bit
	· · · · · xxxx	xxxxxxxxxxxxxxxx	xnnnxxxxx111111	(80+16*nnn)-bit, nnn≠111
	· · · · · xxxx	xxxxxxxxxxxxxxxx	x111xxxxx111111	Reserved for $\geq$ 192-bits
Byte Address:	base+4	base+2	base	

考虑到压缩格式在代码大小和能效上的优势，我们希望将对压缩格式的支持内建到 ISA 编码方案中，而不是事后补充。但为了简化实现，我们不想强制要求使用压缩格式。我们还希望可以选择性地支持更长的指令，以便于支持实验和更大的指令集扩展。尽管我们的编码约定要求对核心 RISC-V ISA 进行更紧凑的编码，但这带来了多个有利的结果。



对于标准 IMAFD ISA 的实现，指令缓存中只需存储指令的最高 30 位（节省 6.25%）。在指令缓存重新填充时，任何遇到的低位清零的指令都应在存储到缓存之前被重新编码为非法的 30 位指令，以保留非法指令的异常行为。

更重要的是，通过将基础 ISA 压缩到 32 位指令字的一个子集，我们为非标准和自定义扩展留出了更多的编码空间。具体来说，基础 RV32I ISA 在 32 位指令字中使用的编码空间不到 1/8。对于不需要支持标准压缩指令扩展的实现，可以将 3 个额外的不符合规范的 30 位指令空间映射到 32 位固定宽度格式，同时保留对标准 ≥ 32 位指令集扩展的支持。此外，如果实现也不需要支持超过 32

位的指令长度，还可以额外恢复 4 个主操作码供不符合规范的扩展使用。

所有位 [15:0] 都为零的编码被定义为非法指令。这些指令被认为是最小长度：如果存在任何 16 位指令集扩展，则为 16 位；否则为 32 位。位 [ILEN-1:0] 全为 1 的编码同样是非法的；该指令被视为是 ILEN 位长的指令。

我们认为，任何全零位的指令长度被定义为非法是一种特性，因为它可以快速捕获跳转到全零内存区域的错误跳转。同样，我们也将全 1 位的指令编码保留为非法指令，以捕获未编程的非易失性存储设备、断开连接的内存总线或损坏的内存设备中这些常见的模式。



软件可以依赖于一个自然对齐的 32 位全零字，在所有 RISC-V 实现上将其视为非法指令，以便在需要显式非法指令的情况下使用。为全 1 位定义对应的非法值更为困难，因为存在可变长度编码。软件通常不能使用 ILEN 位全为 1 的非法值，因为软件可能无法确定最终目标机器的 ILEN（例如，当软件被编译为供多种不同机器使用的标准二进制库时）。我们曾考虑定义一个 32 位全一字为非法，因为所有机器都必须支持 32 位指令长度，但这会要求在 ILEN > 32 的机器上，指令取值单元在此类指令位于保护边界时报告非法指令异常，而不是访问错误异常，从而复杂化了可变指令长度的取值和解码。

RISC-V 基础 ISA 支持小端序 (little-endian) 或大端序 (big-endian) 内存系统，而特权架构进一步定义了双端序操作 (bi-endian)。无论内存系统的字节序如何，指令都以 16 位小端格式段的形式 (parcels) 存储在内存中。这些段组成一条指令，并按增加的半字地址存储，其中最低地址的段包含指令规格中最低编号的位。

我们最初为 RISC-V 内存系统选择小端字节序，是因为目前小端系统在商业上占主导地位（例如所有 x86 系统，以及 iOS、Android 和 Windows 的 ARM 系统）。一个小细节是，我们还发现小端序内存系统对硬件设计者来说更为自然。然而，某些应用领域（例如 IP 网络）操作的是大端序数据结构，而某些遗留代码库是基于大端序处理器构建的，因此我们为 RISC-V 定义了大端序和双端序变体。

我们必须固定指令段在内存中的存储顺序，与内存系统的字节序无关，以确保长度编码位始终在半字地址顺序中首先出现。这使得指令取值单元只需检查第一条 16 位指令段的前几位即可快速确定可变长度指令的长度。



此外，我们将指令段本身设计为小端，以完全将指令编码与内存系统的字节序解耦。这种设计对软件工具和双端硬件都有好处。例如，如果不这么做，RISC-V 的汇编器或反汇编器将始终需要知道当前活动的字节序模式，即使在双端系统中，这种模式可能会在执行期间动态改变。相比之下，通过为指令指定固定的字节序，有时可以使经过仔细编写的软件即使在二进制形式下也不依赖字节序，就像位置无关代码一样。

然而，选择仅支持小端序的指令确实会对 RISC-V 软件带来一些影响，特别是涉及对机器指令进行编码或解码的情况。例如，大端序的即时编译器 (JIT) 在将指令存储到指令内存时必须交换字节顺序。

当我们决定采用小端的指令编码时，这自然导致将长度编码位放在指令格式的最低有效位 (LSB) 位置，以避免破坏操作码字段。

1.6 异常、陷阱和中断

我们使用异常（exception）指代当前 RISC-V hart 中与指令相关联的运行时异常情况。使用中断（interrupt）指代外部的异步事件，它可能导致 RISC-V hart 发生意外的控制转移。使用陷阱（trap）指代由异常或中断引发的控制转移到陷阱处理程序的过程。

以下章节中的指令描述说明了执行过程中可能引发异常的条件。在大多数 RISC-V 执行环境接口（EEI）中，当指令上触发异常信号时，通常会触发陷阱并跳转到相应的处理程序（标准浮点扩展中的浮点异常除外，这些异常不会引发陷阱）。而中断的生成、路由和启用的方式取决于具体的 EEI 实现。



我们对异常和陷阱的使用与 IEEE-754 浮点标准中的定义兼容。

陷阱的处理方式及其对运行在 hart 上的软件的可见性取决于所处的执行环境。从运行在某个执行环境中的软件角度来看，hart 在运行时遇到的陷阱可能有以下四种效果：

受控陷阱

陷阱对运行在执行环境内的软件可见，并由其处理。例如，在一个提供 supervisor 模式和 user 模式的 EEI 中，user 模式 hart 的 ECALL 通常会导致控制转移到在同一 hart 上运行的 supervisor 模式处理程序。同样，在相同的环境中，当 hart 被中断时，中断处理程序将在 hart 的 supervisor 模式下运行。

请求陷阱

这种陷阱是一个同步异常，明确请求执行环境为执行环境内的软件执行某项操作。例如，系统调用（system call）就是一个例子。在这种情况下，请求操作完成后，执行可能会也可能不会在 hart 上恢复。例如，一个系统调用可能移除该 hart，或导致整个执行环境的有序终止。

隐式陷阱

陷阱由执行环境透明处理，处理完成后执行正常恢复。例子包括模拟缺失的指令、在按需分页虚拟内存系统中处理非驻留页面错误，或在多任务机器中处理其他作业的设备中断。在这些情况下，运行在执行环境内的软件不会意识到陷阱的发生（在这些定义中忽略了时序影响）。

致命陷阱

陷阱代表致命失败，导致执行环境终止执行。例如，未通过虚拟内存页面保护检查，或允许看门狗计时器超时。每个 EEI 应定义如何终止执行以及如何向外部环境报告。

表 1.2 陷阱特性说明

	包含	请求	不可见	灾难性的
执行终止	否	否	否	是
软件无反应	否	否	是	是
环境处理	否	是	是	是

表 1.2 陷阱特性说明：1) 执行终止可能是通过请求产生的。2) 非精确的致命陷阱可能会被软件观察到。EEI（执行环境接口）定义了每种陷阱是否需要精确处理，但建议尽可能保持精确性。在执行环境内部的软件中，受控陷阱和请求陷阱可能被观察为非精确的。根据定义，隐式陷阱无法被执行环境中的软件观察为精确或非精确的。如果已知错误的指令不会导致立即终止，执行环境内的软件可能观察到致命陷阱是非精确的。

由于本文件描述的是非特权指令，因此很少提及陷阱。用于处理受控陷阱的架构方法在特权架构手册中定义，同时还包括支持更丰富 EEI 的其他特性。专门为了引发请求陷阱而定义的非特权指令会在本文档中记录。隐式陷阱根据其性质不在本文档范围内。未在本文件中定义或通过其他方式未定义的指令编码可能会引发致命陷阱。

1.7 未指定的行为和值

架构全面描述了实现中必须遵守的内容以及任何可能的约束。在某些情况下，架构有意不约束实现，此时会明确使用术语未指定（UNSPECIFIED）。

术语未指定指的是有意不加约束的行为或数值。这些行为或数值的定义可以由扩展、平台标准或具体实现来确定。扩展、平台标准或实现文档可以提供规范性内容，以进一步约束基本架构定义为未指定的情况。

与基本架构类似，扩展应完全描述允许的行为和数值，并在有意不约束的情况下使用术语未指定。这些未约束的情况可以由其他扩展、平台标准或具体实现进一步约束或定义。

2 RV32I 基础整数指令集 V2.1

本章描述了 RV32I 基础整数指令集。

RV32I 的设计旨在足以作为编译器的目标架构，并支持现代操作系统环境。该指令集架构（ISA）还专门设计为减少最小实现所需的硬件资源。RV32I 包含 40 条独特的指令，尽管在一个简单的实现中，可能通过一个始终触发陷阱的 SYSTEM 硬件指令来替代 ECALL/EBREAK 指令，并且可能将 FENCE 指令实现为 NOP（无操作指令），从而将基础指令总数减少到 38 条。RV32I 可以模拟几乎所有其他 ISA 扩展（除了 A 扩展，它需要额外的硬件支持以保证原子性）。



实际上，包括机器模式特权架构的硬件实现还需要额外的 6 条 CSR 指令。

基础整数 ISA 的子集可能在教学中有所帮助，但由于基础架构的设计，实际硬件实现中几乎没有必要进一步精简指令集，除非是去掉对不对齐内存访问的支持，或者将所有 SYSTEM 指令统一视为一个陷阱指令。



标准的 RISC-V 汇编语言语法已在汇编程序员手册中进行详细说明。



大部分关于 RV32I 的说明同样适用于 RV64I 基础指令集。

2.1 基础整数 ISA 的程序员模型

表 2.1 描述了基础整数 ISA 的非特权状态。对于 RV32I，32 个 x 寄存器每个宽度为 32 位，即 XLEN=32。寄存器 x0 硬连接为全零。通用寄存器 x1-x31 存储不同指令解释的值，这些值可以是布尔值的集合、二进制补码有符号整数或二进制无符号整数。

还有一个额外的非特权寄存器：程序计数器 pc 存储当前指令的地址。

表 2.1 RISC-V 基本非特权整数寄存器状态

XLEN-1	0
X0	
X1	
X2	
X3	
X4	
X5	
X6	
X7	
X8	
X9	
X10	
X11	
X12	
X13	
X14	
X15	
X16	
X17	
X18	
X19	
X20	
X21	
X22	
X23	
X24	
X25	
X26	
X27	
X28	
X29	
X30	
X31	
XLEN	
31	0
Xcsr	
32	

在基础整数 ISA 中，没有专门的栈指针或子程序返回地址链接寄存器；指令编码允许任何 x 寄存器用于这些目的。然而，标准的软件调用约定使用寄存器 x1 来保存调用的返回地址，同时寄存器 x5 可作为备用的链接寄存器。标准调用约定还使用寄存器 x2 作为栈指针。

硬件可以选择使用 x1 或 x5 来加速函数调用和返回的操作。具体请参见 JAL 和 JALR 指令的描述。

可选的压缩 16 位指令格式是基于假设 x1 是返回地址寄存器，x2 是栈指针寄存器来设计的。使用其他约定的软件仍然可以正常运行，但可能会导致更大的代码大小。



可用的体系结构寄存器数量对代码大小、性能和能耗有很大影响。虽然 16 个寄存器可以足够支持运行已编译代码的整数 ISA，但在 16 位指令中使用三地址格式来编码完整的 ISA 是不可能的。虽然两地址格式是可行的，但会增加指令数量并降低效率。我们希望避免中间指令大小（如 Xtensa 的 24 位指令），以简化基础硬件实现，并且一旦采用了 32 位指令大小，支持 32 个整数寄存器变得相对容易实现。更多的整数寄存器还有助于高性能代码的执行，在这类代码中，循环展开、软件流水线和缓存切片的使用非常广泛。

出于这些考虑，我们为 RV32I 选择了 32 个整数寄存器的传统配置。动态寄存器的使用通常由少数频繁访问的寄存器主导，因此寄存器文件实现可以通过优化，减少频繁访问寄存器的访问能耗。可选的压缩 16 位指令格式主要访问 8 个寄存器，因此可以提供高效的指令编码；而如果需要，额外的指令集扩展可以支持更大规模的寄存器空间（无论是扁平式还是分层式）。

对于资源受限的嵌入式应用，我们定义了 RV32E 子集，该子集仅包含 16 个寄存器。

2.2 基本指令格式

在基础的 RV32I ISA 中，有四种核心指令格式（R/I/S/U）。所有指令的长度都是固定的 32 位。基本 ISA 的 IALIGN=32，这意味着指令必须在内存中按照四字节边界对齐。如果目标地址未按 IALIGN-bit 对齐，在发生分支或无条件跳转时，将生成指令地址未对齐异常。此异常会在分支或跳转指令上报告，而不是在目标指令上报告。对于未被执行的条件分支，不会生成指令地址未对齐异常。



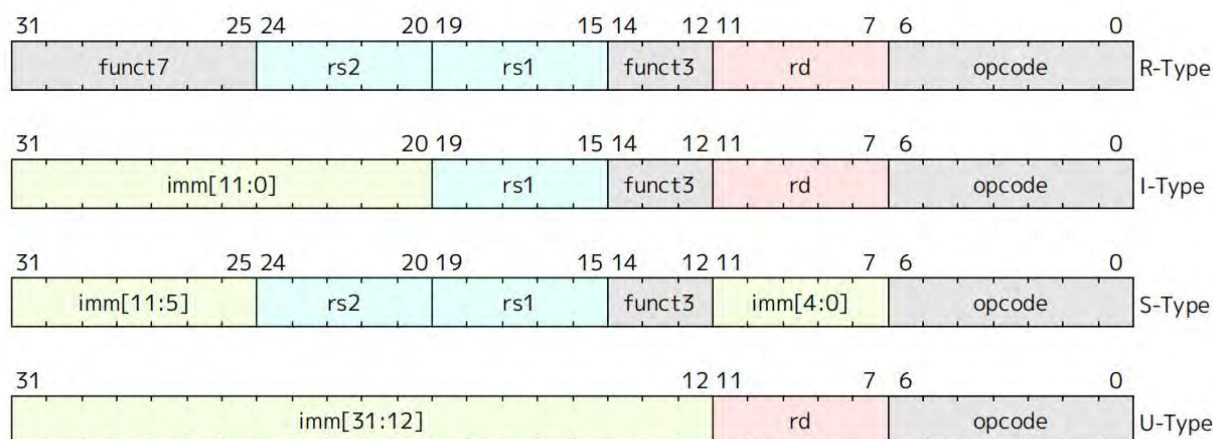
当添加了长度为 16 位的指令扩展或其他 16 位倍数的扩展（即 IALIGN=16）时，基础指令集的对齐约束放宽为两字节边界。

指令地址未对齐异常会在导致指令未对齐的分支或跳转指令上报告，以帮助调试，并简化硬件设计，特别是对于 IALIGN=32 的系统，在这些系统中只有在分支或跳转指令处才可能发生未对齐情况。

解码保留指令时的行为是 UNSPECIFIED 的。



有些平台可能要求为标准用途保留的操作码触发非法指令异常。其他平台则可能允许保留的操作码空间用于非符合规范的扩展。



RISC-V ISA 在所有格式中保持源 (`rs1` 和 `_rs2`) 和目标 (`rd`) 寄存器在同一个位置以简化解码。除了在 CSR 指令中使用的 5 位立即数外，其他立即数均采用符号扩展，并且通常被紧凑地放置在指令中最左侧可用的比特位置，以降低硬件复杂性。特别地，所有立即数的符号位总是在指令的第 31 位，以加速符号扩展电路。

RISC-V 基本指令格式：每个立即数字字段的标注方式是使用其在最终生成的立即数值中的比特位置 (`imm[x]`)，而不是通常采用的指令中立即数字段内的比特位置。

在处理器实现中，寄存器标识符的解码通常位于关键路径上，因此 RISC-V 选择在所有指令格式中保持寄存器标识符的位置一致，尽管这需要在不同指令格式之间重新排列立即数的比特位置（这一特性与 RISC-IV 相同，即 SPUR）。



实际上，大多数立即数要么很小，要么需要所有 `XLEN` 位。我们选择了一种非对称的立即数拆分方案（指令使用 12 位立即数，另外提供一条特殊的加载高位立即数指令，支持 20 位立即数）以增加常规指令可用的操作码空间。

所有立即数均采用符号扩展，因为我们未发现像 MIPS ISA 那样对某些立即数使用零扩展的优势，并希望尽可能简化指令集架构 (ISA)。

2.3 立即数编码变体

根据立即数的处理方式，指令格式还有另外两种变体 (B 型和 J 型)。S 型格式和 B 型格式之间的唯一区别在于，在 B 型格式中，12 位立即数字段用于编码以 2 为倍数的分支偏移量。与传统做法（在硬件中将指令编码的立即数整体左移一位）不同，在 B 型格式中，中间比特位 (`imm[10:1]`) 和符号位保持在固定位置，同时 S 型格式的最低位 (`inst[7]`) 被用作 B 型格式立即数的高位比特。

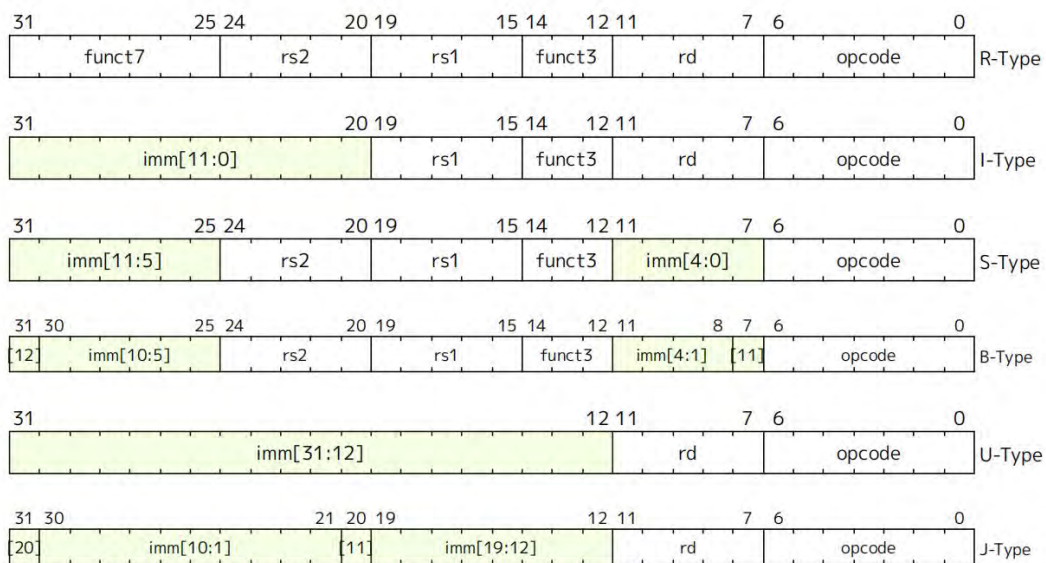
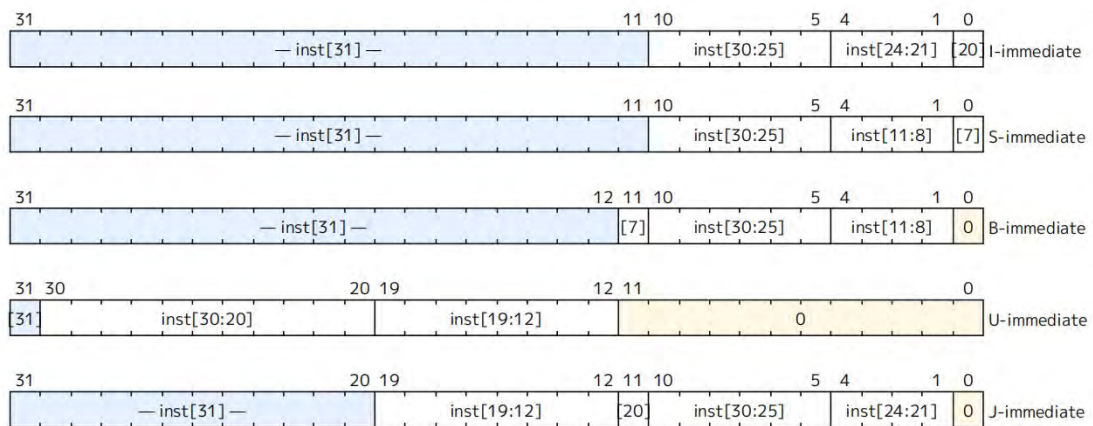


图 2.1 基础指令格式的立即数编码变体

类似地，U 型格式和 J 型格式的唯一区别在于，20 位立即数在 U 型格式中左移 12 位形成 U 型立即数，而在 J 型格式中左移 1 位形成 J 型立即数。U 型和 J 型格式中的指令比特位置经过优化，以最大程度地与其他格式对齐，并提高它们之间的一致性。下图描述了每个基本指令格式生成的立即数，并标注了每个位的来源，即指令中的哪个比特位（ $inst[y]$ ）对应于立即数的哪一位。



符号扩展是立即数处理中最关键的操作之一（尤其是在 $XLEN > 32$ 的情况下）。在 RISC-V 中，所有立即数的符号位始终固定在指令的第 31 位，以便在指令解码的同时并行执行符号扩展。



尽管更复杂的实现可能会为分支和跳转计算分别使用独立的加法器，因此不需要在所有指令类型中保持立即数字段位置一致，但我们希望降低最简单实现的硬件成本。通过在 B 型和 J 型立即数的指令编码中旋转比特，而不是使用动态硬件多路复用器（MUX）将立即数乘以 2，我们成功将指令信号的扇出（fanout）和立即数 MUX 的成本降低了大约 2 倍。这种重新排列的立即数编码对静态编译或提前编译（ahead-of-time compilation）的影响可以忽略不计。对于动态指令生成，可能会引入一些额外的开销，但最常见的短距离前向分支具有直观的立即数编码，因此额外开销很小。

2.4 整数计算指令

大多数整数计算指令操作整数寄存器堆中 XLEN 位的值。整数计算指令要么被编码为使用 I 类型格式的寄存器-立即数操作，要么被编码为使用 R 类型格式的寄存器-寄存器操作。寄存器-立即数和寄存器-寄存器指令的目标都是寄存器 rd。整数计算指令不会引发算术异常。大多数整数计算指令对 XLEN 位的值进行操作，这些值存储在整数寄存器文件中。整数计算指令要么采用 I 型格式（寄存器-立即数运算），要么采用 R 型格式（寄存器-寄存器运算）。无论是寄存器-立即数指令还是寄存器-寄存器指令，目标寄存器均为 rd。所有整数计算指令都不会引发算术异常。

我们在基础指令集中未包含专门用于整数算术运算溢出检查的指令支持，因为在 RISC-V 中，许多溢出检查可以通过分支指令以较低成本实现。对于无符号加法，溢出检查仅需在加法运算后添加一条额外的分支指令：add t0, t1, t2; bltu t0, t1, overflow。

对于有符号加法，如果其中一个操作数的符号已知，溢出检查仅需在加法后添加一条分支指令：addi t0, t1, +imm; blt t0, t1, overflow。这涵盖了加立即数这一常见情况。

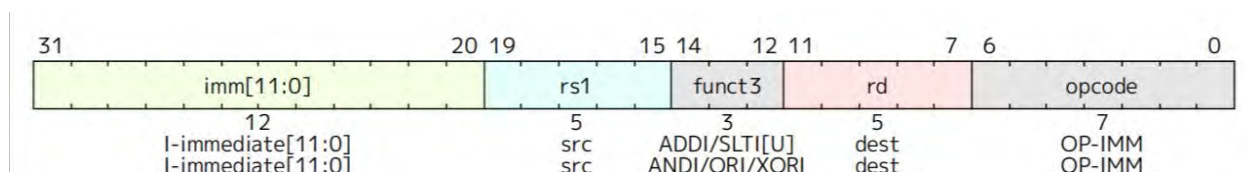
对于一般的有符号加法，溢出检查需要在加法后额外执行三条指令，利用的原理是：当且仅当另一个操作数为负数时，和应小于其中一个操作数。



```
add t0, t1, t2
slti t3, t2, 0
slt t4, t0, t1
bne t3, t4, overflow
```

在 RV64I 中，可以通过比较操作数在 ADD 和 ADDW 指令下的结果，进一步优化对 32 位有符号加法的溢出检查。

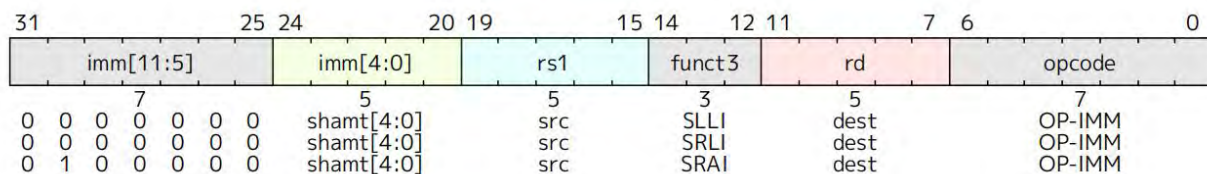
2.4.1 整数寄存器-立即数指令



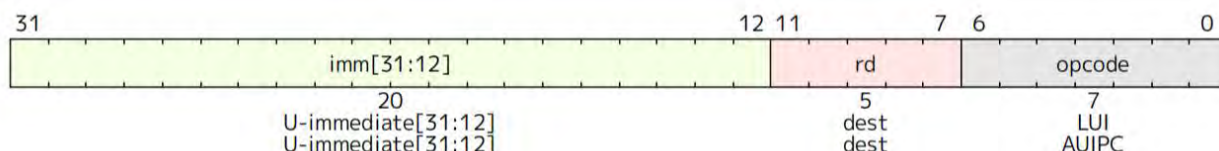
ADDI 指令对寄存器 rs1 的值与符号扩展的 12 位立即数进行加法运算。算术溢出被忽略，结果仅保留低 XLEN 位。ADDI rd, rs1, 0 用于实现汇编伪指令 MV rd, rs1。

SLTI (Set Less Than Immediate) 指令比较寄存器 rs1 与符号扩展的立即数（均视为有符号数），如果 rs1 小于该立即数，则将 1 写入 rd，否则写入 0。SLTIU (Set Less Than Immediate Unsigned) 类似，但将两者视为无符号数进行比较（即立即数先符号扩展为 XLEN 位，然后按无符号数处理）。需要注意的是，SLTIU rd, rs1, 1 的作用是：当 rs1 为 0 时，rd 置为 1，否则置为 0（汇编伪指令 SEQZ rd, rs）。

ANDI、ORI、XORI 是逻辑运算指令，分别对寄存器 rs1 的值和符号扩展的 12 位立即数执行按位 AND、OR 和 XOR 运算，并将结果存入 rd。需要注意的是，XORI rd, rs1, -1 的作用是对 rs1 进行按位取反（汇编伪指令 NOT rd, rs）。



按常数位移的指令是 I 型格式的一种特化形式。待移位的操作数存储在 rs1，移位量编码在 I 型立即数字段的低 5 位。右移的类型由指令的第 30 位进行编码。SLLI 是逻辑左移（低位补 0）；SRLI 是逻辑右移（高位补 0）；SRAI 是算术右移（高位填充原符号位）。



LUI (Load Upper Immediate) 用于构造 32 位常数，采用 U 型格式。LUI 将 32 位 U-立即数载入目标寄存器 rd，并在最低 12 位填充 0。

AUIPC (Add Upper Immediate to pc) 用于构造 pc 相对地址，采用 U 型格式。AUIPC 通过 U-立即数构造一个 32 位偏移量，最低 12 位填充 0，然后将该偏移量加到 AUIPC 指令所在的地址，并将结果存入寄存器 rd。

lui 和 auipc 指令的汇编语法不会显示 U-立即数的最低 12 位，因为这些位始终为 0。

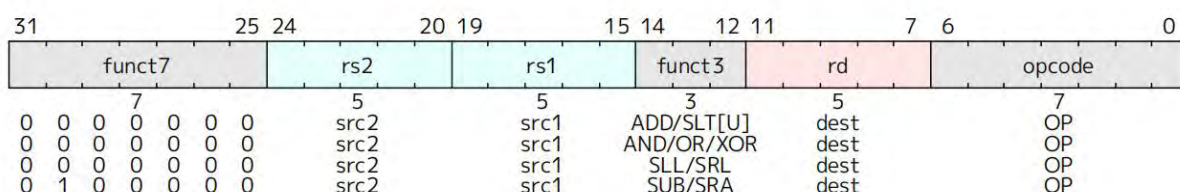
AUIPC 指令支持两条指令的组合序列，以访问 PC 相对的任意偏移量，可用于控制流跳转和数据访问。通过 AUIPC 与 JALR 指令中的 12 位立即数组合，可以跳转到任意 32 位 PC 相对地址。类似地，AUIPC 加上普通加载 (load) 或存储 (store) 指令中的 12 位立即数偏移，可以访问任意 32 位 PC 相对数据地址。



当前 PC 的值可以通过将 AUIPC 的 U 型立即数设为 0 来获取。虽然 JAL +4 也可以用于获取本地 PC（即 JAL 指令后续指令的地址），但在简单的微架构中可能会导致流水线断裂，而在复杂的微架构中可能会污染 BTB（分支目标缓冲区）。

2.4.2 整数寄存器-寄存器运算

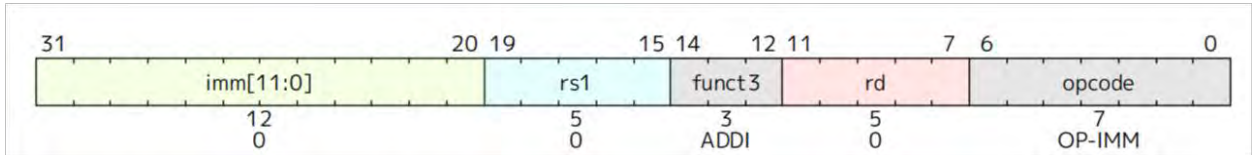
RV32I 定义了多种算术 R 型操作。所有操作均以寄存器 rs1 和 rs2 作为源操作数，并将结果写入寄存器 rd。操作类型由 funct7 和 funct3 字段决定。



ADD 对 rs1 和 rs2 执行加法运算。SUB 计算 rs1 减 rs2 的结果。溢出被忽略，计算结果的低 XLEN 位被写入目标寄存器 rd。SLT 和 SLTU 分别执行有符号和无符号比较，如果 rs1 < rs2，则向 rd 写入 1，否则写入 0。需要注意的是，SLTU rd, x0, rs2 指令会在 rs2 ≠ 0 时将 rd 置为 1，否则置为 0（对应汇编伪指令 SNEZ rd, rs）。AND、OR 和 XOR 分别执行按位与（AND）、或（OR）和异或（XOR）运算。

SLL、SRL 和 SRA 分别对 rs1 进行逻辑左移（SLL）、逻辑右移（SRL）和算术右移（SRA），移位量由 rs2 的低 5 位指定。

2.4.3 NOP 指令



NOP 指令不会改变任何架构可见的状态，但会推进 pc 并增加任何适用的性能计数器的值。NOP 指令的编码为 ADDI x0, x0, 0。

NOP 指令可用于将代码段对齐到微架构上重要的地址边界，或预留空间以便进行内联代码修改。尽管有多种可能的 NOP 编码方式，我们定义了一种标准 NOP 编码，以便支持微架构优化，同时使反汇编输出更易阅读。其他 NOP 编码可用于 HINT 指令。



我们选择 ADDI 作为 NOP 的编码方式，因为它在各种系统上最可能以最少的资源执行（如果在解码阶段未被优化掉）。尤其是，该指令只需要读取一个寄存器。此外，在超标量设计中，ADDI 功能单元更有可能可用，因为加法是最常见的操作。特别地，地址生成单元可以使用相同的硬件执行 ADDI 指令，而寄存器-寄存器 ADD 或逻辑/移位操作则需要额外的硬件支持。

2.5 控制转移指令

RV32I 提供了两类控制转移指令：无条件跳转 和 条件分支。在 RV32I 中，控制转移指令没有架构可见的延迟槽（delay slot）。

如果跳转目标或被执行的分支目标发生指令访问错误或指令页错误异常，该异常会报告在目标指令上，而不是报告在跳转或分支指令上。

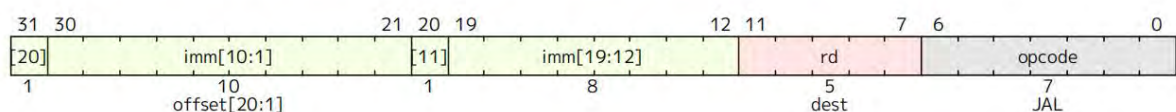
2.5.1 无条件跳转

跳转并链接（JAL: Jump and Link）指令采用 J 型格式，其中 J 型立即数表示一个以 2 字节为单位的有符号偏移量。该偏移量经过符号扩展后，与跳转指令的地址相加，以形成跳转目标地址。因此，JAL 指令的跳转范围为 ± 1 MiB。JAL 指令会将跳转指令后续指令的地址（pc+4）存入寄存器 rd。标准的软件调用约定使用 x1 作为返回地址寄存器，并使用 x5 作为备用链接寄存器。



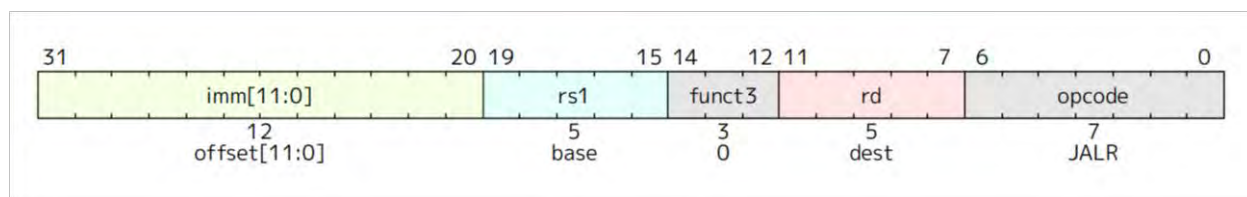
注：备用链接寄存器用于调用微代码（millicode）例程（例如用于在压缩代码中保存和恢复寄存器的例程），同时保留常规返回地址寄存器。寄存器 x5 被选作备用链接寄存器，因为它在标准调用约定中对应于一个临时寄存器，并且它的编码与常规链接寄存器仅相差 1 位。

普通的无条件跳转（汇编伪指令 J）被编码为 JAL，其中 rd=x0。



间接跳转指令（JALR: Jump and Link Register）采用 I 型格式进行编码。目标地址由寄存器 rs1

与符号扩展的 12 位 I 型立即数相加得到，并将计算结果的最低有效位清零。跳转指令后续指令的地址（pc+4）被写入寄存器 rd。如果不需要保存返回地址，可以使用寄存器 x0 作为目标寄存器。



无条件跳转指令全部采用 PC 相对寻址，以支持位置无关代码（PIC）。JALR 指令 被设计为支持两条指令的组合序列，从而能够跳转到 32 位绝对地址范围 的任意位置。一条 LUI 指令可以先将目标地址的高 20 位加载到 rs1，然后 JALR 可以将低位加上去。同样，AUIPC + JALR 可以跳转到 32 位 PC 相对地址范围内的任意位置。



需要注意的是，JALR 指令的 12 位立即数并不像条件分支指令那样以 2 字节为单位，这避免了在硬件中增加额外的立即数格式。在实际应用中，大多数 JALR 指令的立即数要么为 0，要么与 LUI 或 AUIPC 组合使用，因此这一点对跳转范围的影响不大。

在计算 JALR 目标地址 时，将最低有效位（LSB）清零，这样既可以简化硬件，也可以利用指针的最低位存储附加信息。尽管这种方式可能会略微降低错误检测能力，但在实际应用中，如果跳转到错误的指令地址，通常会迅速触发异常。

当 JALR 的基地址寄存器 rs1=x0 时，它可以实现单条指令的子程序调用，从地址空间的任意位置跳转到最低或最高地址区域，这可以用于快速调用小型运行时库。或者，某个 ABI（应用二进制接口）也可以专门保留一个通用寄存器，用于指向地址空间中的库。

如果 JAL 或 JALR 指令的目标地址未对齐至 4 字节边界，则会触发指令地址未对齐异常。



在支持 16 位对齐指令（例如压缩指令集扩展 C）的机器上，不会发生指令地址未对齐（instruction-address-misaligned）异常。

返回地址预测栈（Return-Address Prediction Stacks, RAS）是高性能指令取指单元中的常见功能，但要有效工作，必须准确检测用于过程调用和返回的指令。在 RISC-V 中，指令的用途提示（HINTS）是通过使用的寄存器编号来隐式编码的。JAL 指令仅在 rd 为 x1 或 x5 时，才应将返回地址压入 返回地址栈（RAS）。JALR 指令的 RAS 推入/弹出 规则如图所示。返回编码在 JALR 指令寄存器操作数中的地址堆栈预测值。

表2.3 返回编码在JALR指令寄存器操作数中的地址堆栈预测提示

<i>rd is x1/x5</i>	<i>rs1 is x1/x5</i>	<i>rd=rs1</i>	RAS action
No	No	—	None
No	Yes	—	Pop
Yes	No	—	Push
Yes	Yes	No	Pop, then push
Yes	Yes	Yes	Push

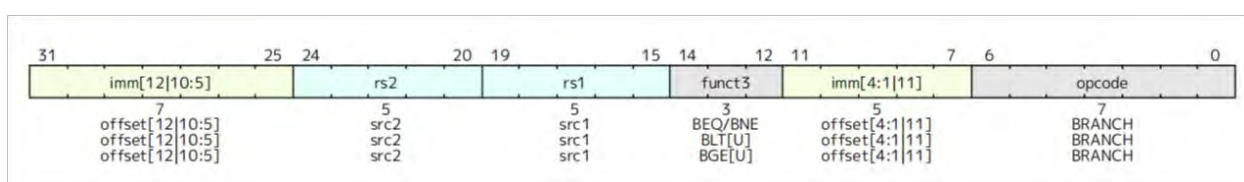


某些其他 ISA 在它们的间接跳转指令中添加了显式提示位，用于引导返回地址栈（RAS）的操作。而在 RISC-V 中，我们采用了基于寄存器编号和调用约定的隐式提示机制，从而减少了编码空间的占用。

当两个不同的链接寄存器（x1 和 x5）分别用于 rs1 和 rd 时，RAS 会同时执行弹出（pop）和压入（push）操作，以支持协程（coroutines）。如果 rs1 和 rd 是相同的链接寄存器（x1 或 x5），则 RAS 仅执行压入（push）操作，以支持宏指令融合（macro-op fusion），如以下指令序列：lui ra, imm20; jalr ra, imm12(ra) 和 auipc ra, imm20; jalr ra, imm12(ra)。

2.5.2 条件分支

所有分支指令均采用 B 型指令格式。12 位 B 型立即数以 2 字节为单位编码有符号偏移量。该偏移量经过符号扩展后，与分支指令的地址相加，以计算目标地址。条件分支的跳转范围为±4 KiB。



分支指令用于比较两个寄存器。BEQ 和 BNE 分别在寄存器 rs1 和 rs2 相等或不等时跳转。BLT 和 BLTU 分别在有符号和无符号比较中，当 rs1 小于 rs2 时跳转。BGE 和 BGEU 分别在有符号和无符号比较中，当 rs1 大于或等于 rs2 时跳转。注意，BGT、BGTU、BLE 和 BLEU 可以通过交换 BLT、BLTU、BGE 和 BGEU 的操作数来合成。



带符号的数组边界检查可以通过单条 BLTU 指令完成，因为任何负索引都会被判定为大于任意非负边界。

软件应优化顺序执行路径，使其成为最常见的执行路径，并将较少执行的代码路径放置在线代码之外。此外，软件应假设后向分支（backward branches）默认会被预测为执行，而前向分支（forward branches）默认在首次遇到时被预测为不跳转。动态预测器应该能够快速学习任何可预测的分支行为。

与某些其他架构不同，RISC-V 无条件跳转应始终使用跳转指令（JAL，且 rd=x0），而不是使用条件分支指令并设定一个始终为真的条件。RISC-V 跳转指令也是 PC 相对寻址，支持的偏移范围比分支指令更广，并且不会污染条件分支预测表。



条件分支指令被设计旨在包含两个寄存器之间的算数比较操作（与 PA-RISC、Xtensa 和 MIPS R6 类似），而不是使用条件码（condition codes）（如 x86、ARM、SPARC、PowerPC），也不是仅仅将一个寄存器与零进行比较（如 Alpha、MIPS），或者仅比较两个寄存器是否相等（如 MIPS）。这一设计的动机在于，合并比较与分支的指令（compare-and-branch）可以很好地适配常规流水线，避免引入额外的条件码状态或使用临时寄存器，同时减少静态代码体积以及动态指令取指流量。另一个考虑点是，与零进行比较所需的电路延迟不可忽视（尤其在先进工艺转向静态逻辑后更是如此），其延迟几乎与算术大小比较相当。融合比较与分支指令的另一优点在于，分支能够在前端指令流中更早地被识别，从而更早地进行预测。虽然在基于相同条件码可以采取多个分支的情况下，使用条件码的设计可能具有一定优势，但我们认为这种情况相对较少。

我们曾考虑在指令编码中加入静态分支提示（static branch hints），但最终未被采纳。这些提示虽

然可以降低动态预测器的压力，但会占用更多的指令编码空间，并且需要软件进行性能分析以达到最佳效果；如果实际生产运行与性能分析的环境不匹配，反而可能导致性能下降。

我们曾考虑过但并未采用条件移动（conditional moves）或谓词指令（predicated instructions），这两者都能有效替代不可预测的短程前向分支。条件移动较为简单，但在涉及可能引发异常的条件代码（如内存访问和浮点运算）时使用起来较为困难。谓词化则需要为系统增加额外的标志状态，增加设置和清除这些标志的指令，并且每条指令还会因此增加额外的编码开销。无论是条件移动还是谓词指令，都给乱序微架构带来额外复杂性，因为如果谓词为假，就需要将目标架构寄存器的原始值复制到重命名后的目标物理寄存器中，从而隐含地引入了第三个源操作数。此外，在编译时静态决定使用谓词化而非分支，可能会在那些不包含在编译器训练集中的输入上导致性能下降，尤其是在不可预测分支本就稀少且随着分支预测技术的改进而愈加罕见的情况下。

我们注意到，存在各种微架构技术，可以动态地将不可预测的短程前向分支转换为内部谓词化代码，以避免因分支预测错误而刷新流水线的开销，并且这些技术已在商用处理器中得到应用。最简单的技术仅通过只刷新分支影区（branch shadow，或称为分支预测区）中的指令而非整个取指流水线，或利用宽取指（wide instruction fetch）或空闲取指槽（idle instruction fetch slots）同时从两个方向取指，从而减少从预测错误的短程前向分支中恢复的代价。对于乱序执行核（out-of-order cores），更复杂的技术则在分支影区中的指令上增加内部谓词，该内部谓词的值由分支指令写入，从而使该分支及其后续指令能够相对于其他代码进行投机性和乱序执行。

如果条件分支指令的目标地址未对齐至 4 字节边界，并且分支条件计算结果为真，则会触发指令地址未对齐异常。如果分支条件计算结果为假，则不会触发该异常。



在支持 16 位对齐指令（例如压缩指令集扩展 C）的机器上，不会发生指令地址未对齐异常。

2.6 加载和存储指令

RV32I 是一种加载-存储（load-store）架构，其中只有加载和存储指令可以访问内存，而算术指令仅能对 CPU 寄存器进行操作。RV32I 提供了 32 位地址空间，并采用字节寻址。EEI（Execution Environment Interface，执行环境接口）将定义地址空间的哪些部分可以被哪些指令访问（例如，某些地址可能仅可读，或仅支持字（word）访问）。如果加载指令的目标寄存器是 x0，尽管加载的值会被丢弃，但仍然必须触发异常，并引发所有相关的副作用。

EEI 还将定义内存系统的字节序（小端或大端）。在 RISC-V 中，无论采用大端还是小端格式，内存中每个字节的地址分布始终保持不变。

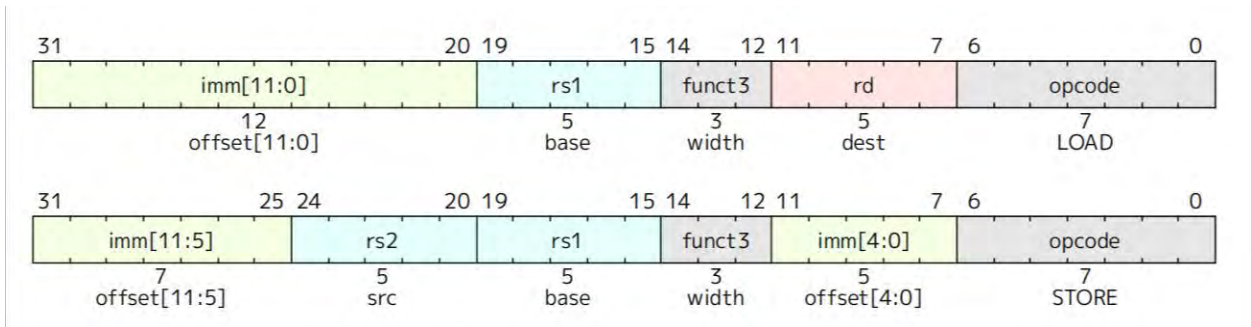
在字节序与字节地址无关的系统中，以下性质成立：如果以某种字节序将一个字节存储到某个内存地址，那么无论采用何种字节序，从该地址加载一个字节时，都应返回存储的值。



在小端序（little-endian）配置下，多字节存储操作将寄存器中最低有效字节存放在最低的内存字节地址，其余寄存器字节按有效性从低到高的顺序依次存放。加载操作也会相应地将较低内存字节地址中的内容传送到寄存器中较低有效性的字节位置。

在大端序（big-endian）配置下，多字节存储操作将寄存器中最高有效字节存放在最低的内存字节地址，其余寄存器字节按有效性从高到低的顺序依次存放。加载操作则将较高内存字节地址中的内容传

送到寄存器中较低有效性的字节位置。



加载和存储指令用于在寄存器和内存之间传递数值。加载指令采用 I 型格式进行编码，而存储指令采用 S 型格式。有效地址由寄存器 rs1 与经过符号扩展的 12 位偏移量相加得到。加载指令从内存读取值并存入寄存器 rd，存储指令则将寄存器 rs2 的值写入内存。

LW 指令将一个 32 位的值从内存加载到寄存器 rd 中。LH 指令从内存加载一个 16 位的值，并在存入 rd 前进行符号扩展至 32 位。LHU 指令从内存加载 16 位的值，但在存入 rd 前进行零扩展至 32 位。LB 和 LBU 指令对 8 位值的加载方式类似。SW、SH 和 SB 指令分别从寄存器 rs2 的低位中取出 32 位、16 位和 8 位数据存入内存。

无论 EEI 如何定义，对于有效地址自然对齐的加载和存储，都不应引发地址未对齐异常。而那些有效地址没有自然对齐到所引用数据类型（即有效地址不能被访问大小（以字节为单位）整除）的加载和存储，其行为则依赖于 EEI 的规定。

一个 EEI 可能保证对未对齐的加载和存储操作提供完全支持，因此在执行环境中运行的软件永远不会遭遇受控或致命的地址未对齐陷阱。在这种情况下，未对齐的加载和存储可以由硬件直接处理，或通过对执行环境实现的隐式陷阱（invisible trap）处理，亦或根据地址采用硬件和隐式陷阱相结合的方式。

另一方面，一个 EEI 可能不保证对未对齐的加载和存储操作进行隐式处理。在这种情况下，未自然对齐的加载和存储操作可能会成功完成执行，也可能引发异常。引发的异常可以是地址未对齐异常，也可以是访问错误异常。对于原本能够完成的内存访问，如果仅因为未对齐而无法完成，并且该未对齐访问不应被模拟（例如，因为对该内存区域的访问具有副作用），则可以引发访问错误异常而不是地址未对齐异常。当 EEI 不保证对未对齐加载和存储操作进行隐式处理时，EEI 必须定义因地址未对齐而引发的异常是产生受控陷阱（允许执行环境内的软件处理该陷阱）还是致命陷阱（导致执行终止）。



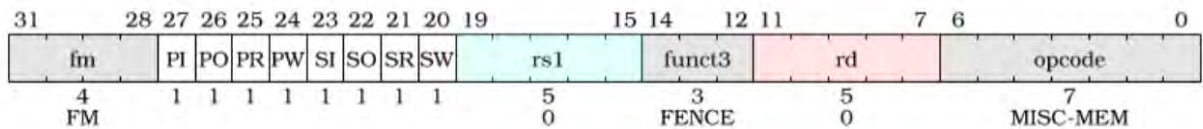
在移植遗留代码时，有时需要进行未对齐访问；而在使用任何形式的 packeted-SIMD 扩展或处理外部打包数据结构时，未对齐访问也有助于提高性能。我们允许 EEI 选择通过常规加载和存储指令来支持未对齐访问，其理由在于简化增加未对齐硬件支持的过程。一种选择本来是禁止在基础 ISA 中进行未对齐访问，然后为未对齐访问提供单独的 ISA 支持——无论是通过特殊指令来帮助软件处理未对齐访问，还是通过引入新的未对齐访问硬件寻址模式。然而，特殊指令使用起来较为困难，不仅会增加 ISA 的复杂性，还常常引入新的处理器状态（例如 SPARC VIS 对齐地址偏移寄存器），或者使得对现有处理器状态的访问变得更为复杂（例如 MIPS LWL/LWR 部分寄存器写操作）。此外，对于面向循环的 packeted-SIMD 代码，操作数未对齐所产生的额外开销，会促使软件根据操作数的对齐情况提供多种循环形式，这不仅使代码生成过程变得复杂，也增加了循环启动时的开销。新的未对齐硬件寻址模式在指令编码中要么占用较多空间，要么要求采用非常简化的寻址方式（例如，仅支持寄存器间接寻址）。

即使未对齐的加载和存储操作成功完成，这些访问的速度仍可能非常慢，具体取决于其实现方式（例如，当通过隐式陷阱实现时）。此外，虽然自然对齐的加载和存储操作保证原子执行，但未对齐的加载和存储操作可能不具备原子性，因此需要额外的同步措施以确保原子性。



我们不强制要求未对齐访问具备原子性，因此执行环境的实现可以使用隐式机器陷阱和软件处理程序来处理部分或全部的未对齐访问。如果提供了硬件对未对齐访问的支持，软件只需简单地使用常规加载和存储指令即可利用这一特性。硬件则可以根据运行时地址是否对齐自动优化访问。

2.7 内存排序指令



FENCE 指令用于对设备 I/O 和内存访问进行排序，使其他 RISC-V hart 以及外部设备或协处理器看到的访问顺序符合预期。设备输入（I）、设备输出（O）、内存读取（R）和内存写入（W）的任何组合，都可以彼此之间建立顺序关系。换句话说，其他 RISC-V hart 或外部设备不会在观察到 FENCE 前面的操作（前驱集合，predecessor）完成之前，就看到 FENCE 后面的操作（后继集合，successor）。第 17 章提供了 RISC-V 内存一致性模型的精确定义。

FENCE 指令还会对 hart 执行的内存读写操作进行排序，使其在外部设备进行的内存读写操作中得到一致的观察。然而，FENCE 不会对外部设备使用其他信号机制所观察到的事件进行排序。



设备可能通过某种外部通信机制观察到对内存地址的访问，例如，通过内存映射的控制寄存器来驱动中断控制器的中断信号。这种通信超出了 FENCE 排序机制的范围，因此 FENCE 指令无法保证中断信号的变化何时能被中断控制器观察到。特定设备可能会提供额外的排序保证以降低软件开销，但这些内容超出了 RISC-V 内存模型的定义范围。

EEI 将定义哪些 I/O 操作是可能的，特别是哪些内存地址在被加载和存储指令访问时，会分别被视为设备输入和设备输出操作，而不是内存读取和写入。例如，内存映射 I/O 设备通常会通过未缓存的加载和存储进行访问，并且按照 FENCE 指令中的 I 和 O 位而非 R 和 W 位进行排序。指令集扩展还可能描述新的 I/O 指令，这些指令同样会使用 FENCE 指令中的 I 和 O 位进行排序。

表2.4 围栏模式编码

fm field	Mnemonic	Meaning
0000	none	Normal Fence
0000	none	Normal Fence
other		Reserved for future use

模式字段 fm 定义了 FENCE 的语义。对于 fm=0000 的 FENCE 指令，其会将前驱集合中的所有内存操作排序在后继集合中的所有内存操作之前。FENCE.TSO 指令编码为 fm=1000 的 FENCE 指令，同时其前驱字段设置为 RW，后继字段也设置为 RW。FENCE.TSO 会将其前驱集合中的所有加载操作排序在后继集合中的所有内存操作之前，并将其前驱集合中的所有存储操作排序在后继集合中的所有存储操作之前。这使得 FENCE.TSO 的前驱集合中的非原子内存操作（non-AMO store）与后继集合中的非原子内存操作（non-AMO load）之间的执行顺序不受约束。



由于 FENCE RW, RW 所施加的顺序约束是 FENCE.TSO 所施加的顺序约束的超集，因此忽略 fm 字段，将 FENCE.TSO 实现为 FENCE RW, RW 是正确的。

FENCE 指令中未使用的字段 rs1 和 rd 保留用于未来扩展中更细粒度的栅栏。为实现向前兼容，基础实现应忽略这些字段，而标准软件应将这些字段置零。同样，fm 中许多 fm 以及前驱/后继集合的设置也保留以供将来使用。基础实现应将所有此类保留配置视为 fm=0000 的普通栅栏，而标准软件则应仅使用非保留配置。



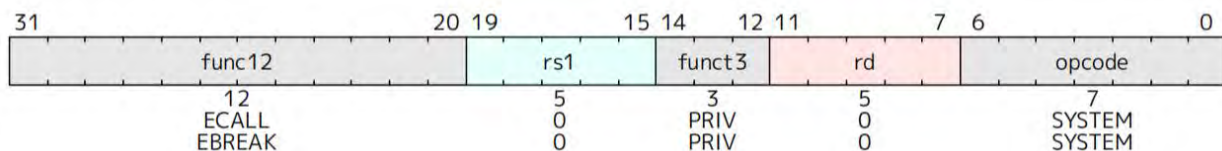
我们选择了宽松的内存模型，以便从简单的机器实现以及未来可能的协处理器或加速器扩展中获得高性能。我们将 I/O 排序与内存读写排序分离，以避免在设备驱动 hart 内部产生不必要的串行化，同时也支持通过非内存路径控制附加的协处理器或 I/O 设备。简单的实现甚至可以忽略 predecessor 和 successor 字段，总是对所有操作执行一个保守的 fence 操作。

2.8 环境调用和断点

SYSTEM 指令用于访问可能需要特权访问的系统功能，并采用 I 型指令格式编码。这些指令主要分为两大类：一类是对控制和状态寄存器（CSR）进行原子读-修改-写操作的指令，另一类是所有其他可能需要特权的指令。CSR 指令的详细说明见第 8 章，而基础的非特权指令将在下一节中介绍。



SYSTEM 指令的设计允许简单实现总是转入一个单一的软件陷阱处理程序。更复杂的实现则可能在硬件上执行更多的 SYSTEM 指令操作。



这两条指令会导致向支持它们的执行环境发起一个精确的请求陷阱。ECALL 指令用于向执行环境发起服务请求。EEI 将定义服务请求参数的传递方式，但通常这些参数会存放在整数寄存器文件的预定位置。

EBREAK 指令用于将控制权交还给调试环境。

ECALL 和 EBREAK 以前分别称为 SCALL 和 SBREAK。这两条指令的功能和编码保持不变，但改名后的名称反映了它们不仅限于调用超级用户级操作系统或调试器，而是可以更广泛地使用。



EBREAK 最初主要设计用于由调试器调用，使执行停止并退回到调试器中。标准的 gcc 编译器也使用 EBREAK 来标记不应执行的代码路径。

另外，EBREAK 还用于支持半主机（semihosting）模式，此模式下执行环境包含一个调试器，能够通过围绕 EBREAK 指令构建的另一种系统调用接口提供服务。由于 RISC-V 基础 ISA 只提供一种 EBREAK 指令，RISC-V 半主机模式采用了一组特殊的指令序列，以区分半主机模式下的 EBREAK 与调试器插入的 EBREAK。


```
slli x0, x0, 0x1f # Entry NOP
ebreak           # Break to debugger
srai x0, x0, 7    # NOP encoding the semihosting call number 7
```



请注意，这三条指令必须是 32 位宽的指令，也就是说，它们不能属于第26章中描述的 16 位压缩指令。

移位 NOP 指令仍然可用作 HINTs。半主机是一种服务调用方式，更自然的编码方式是使用现有 ABI 的 ECALL，但这要求调试器能够截获 ECALL，而这一功能是调试标准的较新扩展。我们打算转而使用带有标准 ABI 的 ECALL，这样半主机就可以与现有标准共享服务 ABI。我们注意到，ARM 处理器在较新设计中也已经从使用 BKPT 改为使用 SVC 来进行半主机调用。

2.9 HINT 指令

RV32I 为 HINT 指令保留了大量的编码空间，这些指令通常用于向微架构传递性能提示。与 NOP 指令类似，HINT 指令不会改变任何架构可见的状态，除非用于更新程序计数器（pc）和任何适用的性能计数器。实现可以始终忽略这些编码的提示。

大多数 RV32I 的 HINT 指令被编码为 rd=x0 的整数计算指令。其余的 RV32I HINT 指令则被编码为前驱和后继集合为空或 fm=0 的 FENCE 指令。



这些 HINT 指令的编码方式被设计成简单的实现可以完全忽略 HINT，而将其当作普通指令执行，只不过该指令不会改变架构可见的状态。例如，当目标寄存器为 x0 时，ADD 指令就被视为 HINT；而 rs1 和 rs2 的 5 位字段则用于编码 HINT 的参数。然而，简单的实现可以直接将该 HINT 当作一条 ADD 指令执行，即将 rs1 和 rs2 相加并将结果写入 x0，从而不会产生任何架构可见的影响。

另一个例子是：一个 pred 字段和 fm 字段均为零的 FENCE 指令被视为一条 HINT；而 succ、rs1 和 rd 字段则对 HINT 的参数进行编码。简单的实现可以直接将该 HINT 当作一条 FENCE 指令执行，其效果是将空的先前内存访问集合排序在 succ 字段所编码的任意后续内存访问之前。由于先前集合与后续集合的交集为空，该指令不会施加任何内存排序，从而不会产生任何架构可见的影响。



我们预计未来的标准 HINT 将涵盖内存系统的空间和时间局部性提示、分支预测提示、线程调度提示、安全标签，以及用于仿真/模拟的插桩标志。

表2.5 RV32I提示指令

指令	约束	目的
LUI	rd=x0	指定用于将来的标准使用
AUIPC	rd=x0	
ADDI	rd=x0, and either rs1≠x0 or imm≠0	
ANDI	rd=x0	
ORI	rd=x0	

XORI	rd=x0	
ADD	rd=x0, rs1≠x0	
ADD	rd=x0, rs1=x0, rs2≠x2-x5	
ADD	rd=x0, rs1=x0, rs2=x2-x5	(rs2=x2) NTL. P1 (rs2=x3) NTL. PALL (rs2=x4) NTL. S1 (rs2=x5) NTL. ALL
SUB	rd=x0	指定用于将来的标准使用
AND	rd=x0	
OR	rd=x0	
XOR	rd=x0	
SLL	rd=x0	
SRL	rd=x0	
SRA	rd=x0	
FENCE	rd=x0, rs1≠x0, fm=0, and either pred=0 or succ=0	
FENCE	rd≠x0, rs1=x0, fm=0, and either pred=0 or succ=0	
FENCE	rd=rs1=x0, fm=0, pred=0, succ≠0	
FENCE	rd=rs1=x0, fm=0, pred≠W, succ=0	
FENCE	rd=rs1=x0, fm=0, pred=W, succ=0	终止
SLTI	rd=x0	指定自定义使用
SLTIU	rd=x0	
SLLI	rd=x0	
SRLI	rd=x0	
SRAI	rd=x0	
SLT	rd=x0	
SLTU	rd=x0	

3 RV32E 与 RV64E 基础整数指令集, V2.0

本章介绍了为嵌入式系统中的微控制器设计的 RV32E 与 RV64E 基础整数指令集提案。RV32E 和 RV64E 分别是 RV32I 和 RV64I 的简化版, 具体来说, 它们唯一的变化是将整数寄存器的数量减少到 16 个。本章仅概述了 RV32E/RV64E 与 RV32I/RV64I 之间的差异, 因此建议在阅读第 2 章 与第 4 章 之后再行阅读本章。



RV32E 旨在为嵌入式微控制器提供更小的基本核; 得益于它有助于降低高度并发 64 位处理器的上下文状态开销, RV64E 在大型 Soc 设计中的微控制器上也得到了关注。

除非另有说明, 兼容 RV32I 和 RV64I 标准的扩展同样适用于 RV32E 与 RV64E。

3.1 RV32E 与 RV64E 程序员模型

RV32E 与 RV64E 将整数寄存器数量缩减为 16 个通用寄存器 (x0-x15)，其中 x0 为专用的零寄存器。



我们发现，在小型 RV32I 核实现中，后半部分的 16 个寄存器约占整个核（不包含存储器）的四分之一面积，因此，去除这些寄存器能节省约 25% 的核面积，同时相应的降低核功耗。

3.2 RV32E 与 RV64E 指令集编码

RV32E 和 RV64E 采用与 RV32I 和 RV64I 一致的指令集编码方案，不同之处仅在于仅提供寄存器 x0-x15。所有指定其他寄存器 (x16-x31) 的编码均被保留。



本章先前的草案将所有使用 x16-x31 寄存器的编码标记为可用的自定义编码；而本版本则采取了更为保守的策略，将这些编码保留，以便将来可以在自定义扩展与新的标准编码之间进行分配。

4 RV64I 基础整数指令集, V2.1

本章描述了 RV64I 基础整数指令集，它基于第 2 章中描述的 RV32I 变体构建。本章仅介绍与 RV32I 的区别，因此应结合前一章一起阅读。

4.1 寄存器状态

RV64I 将整数寄存器和支持的用户地址空间扩展到 64 位（表 2.1 中的 XLEN 等于 64）。

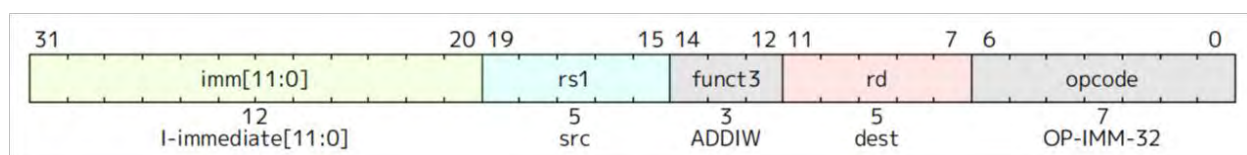
4.2 整数计算指令

大多数整数计算指令在 XLEN 位值上进行操作。RV64I 提供了额外的指令变体来操作 32 位值，这些指令通过操作码的后缀 W 表示。这些 *W 指令忽略输入的高 32 位，并始终生成 32 位有符号值，将其符号扩展到 64 位，即 XLEN-1 到 31 位相等。



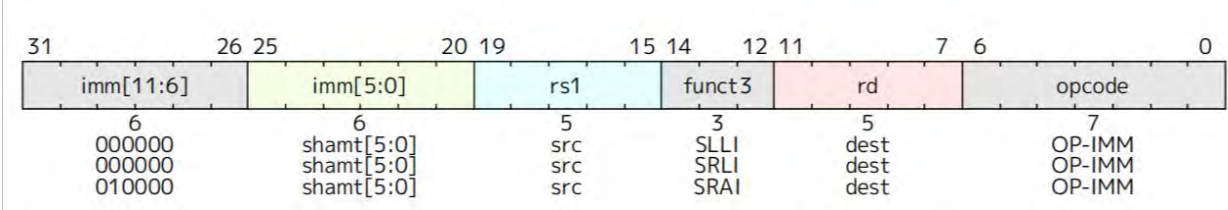
编译器和调用约定保持一个不变式：所有 32 位值都以符号位扩展格式储存在 64 位寄存器中。因此，即使是 32 位无符号整数，也会将第 31 位扩展到第 63 到 32 位。因此，符号扩展后的 32 位无符号整数与有符号整数之间的转换是一个空操作，从有符号 32 位整数转换到有符号 64 位整数也是如此。在这一不变式下，现有的 64 位宽的 SLTU 和无符号分支比较仍然能正确操作无符号 32 位整数。同样，现有的 64 位宽逻辑操作在处理符号扩展的 32 位整数时，会保持符号位扩展的属性。为了确保 32 位值的合理性能，需要增加一些新指令 (ADD[I]W/SUBW/SxxW) 用于加法和移位操作。

4.2.1 整数寄存器-立即数指令

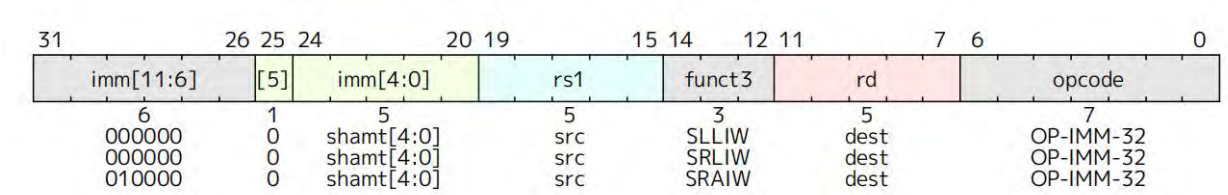


ADDIW (Add Immediate Word) 是一个 RV64I 指令，它将符号位扩展的 12 位立即数与寄存器 rs1 的值相加，得到一个 32 位结果，并将这一 32 位结果的符号位扩展写入到 rd 中。计算出来的 32 位

结果的溢出应当被忽略，最终写入的结果应当是符号位扩展到 64 位的 32 位结果。注意，ADDIW rd, rs1, 0 将寄存器 rs1 的低 32 位的符号位扩展写入寄存器 rd（汇编伪指令 SEXT.W）。



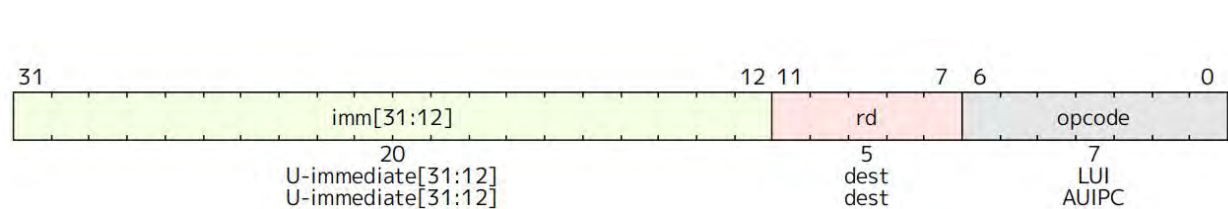
依靠常量进行的移位操作被特化编码为 I 型格式，使用与 RV32I 相同的指令操作码。要移位的操作数在 rs1 中，移位量编码在 RV64I 的 I 型立即数字段的低 6 位中。右移类型编码在第 30 位。SLLI 是逻辑左移（低位补零）；SRLI 是逻辑右移（高位补零）；SRAI 是算术右移（初始符号位被复制到空出来的高位）。



SLLIW (Shift Left Logical Immediate Word)、SRLIW (Shift Right Logical Immediate Word) 和 SRAIW (Shift Right Arithmetic Immediate Word) 是 RV64I 的专用指令，它们的定义与前述指令类似，但对 32 位值进行操作，并将其 32 位结果符号位扩展到 64 位。具有 $imm[5] \neq 0$ 的 SLLIW、SRLIW 和 SRAIW 编码是保留的。



此前， $imm[5] \neq 0$ 时的 SLLIW、SRLIW 和 SRAIW 指令会引发非法指令异常，而现在它们被标记为保留。这是一个向后兼容的更改。



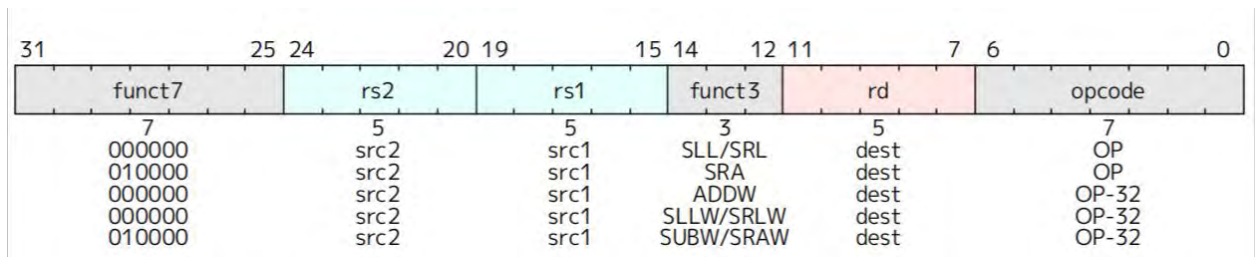
LUI (Load Upper Immediate) 使用与 RV32I 相同的操作码。LUI 将 32 位 U 型立即数放入寄存器 rd，最低 12 位填充为零。然后再将 32 位结果符号位扩展到 64 位。

AUIPC (Add Upper Immediate to PC) 使用与 RV32I 相同的操作码。AUIPC 用于构建 pc 相对地址，并使用 U 型指令格式。AUIPC 从 U 型立即数形成 32 位偏移量，最低 12 位填充为零，同时符号位扩展到 64 位，然后与 AUIPC 指令的地址相加，将结果放入寄存器 rd。



在 RV64I 中，通过将 LUI 与 LD 配对、AUIPC 与 JALR 配对等方式形成的地址偏移量集。

4.2.2 整数寄存器-寄存器操作



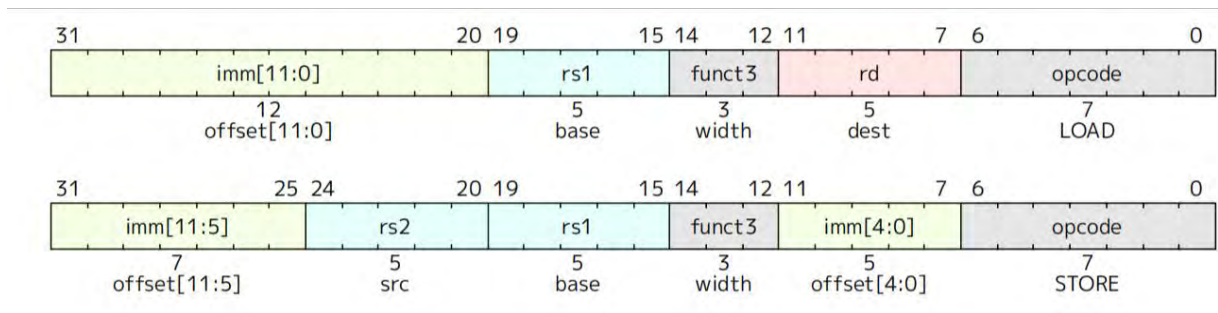
ADDW (Add Word) 和 SUBW (Subtract Word) 是 RV64I 专用指令，它们的定义类似于 ADD 和 SUB，但对 32 位值进行操作并生成有符号的 32 位结果。溢出被忽略，结果会被符号位扩展到 64 位并写入目标寄存器。

SLL、SRL 和 SRA 对寄存器 rs1 中的值按寄存器 rs2 中的移位数执行逻辑左移、逻辑右移和算术右移操作。在 RV64I 中，仅考虑 rs2 的低 6 位作为移位数。

SLLW (Shift Left Logical Word)、SRLW (Shift Right Logical Word) 和 SRAW (Shift Right Arithmetic Word) 是 RV64I 专用指令，定义与上述指令类似，但对 32 位值进行操作，并将其 32 位结果符号位扩展到 64 位。移位数由 rs2[4:0] 给出。

4.3 加载和存储指令

RV64I 将地址空间扩展到 64 位。执行环境将定义地址空间的哪些部分是可以合法访问的。LD 指令从内存加载 64 位值到寄存器 rd，适用于 RV64I。



LW 指令从内存加载一个 32 位值，并将其符号位扩展到 64 位，然后存储在寄存器 rd 中（适用于 RV64I）。另一方面，LWU 指令将内存中的 32 位值零扩展（适用于 RV64I）。LH 和 LHU 类似地定义为对 16 位值的操作，LB 和 LBU 类似地定义为对 8 位值的操作。SD、SW、SH 和 SB 指令分别将寄存器 rs2 的低 64 位、32 位、16 位和 8 位值存储到内存中。

4.4 HINT 指令

RV32I 中的所有微架构提示 (HINT) 指令（见第 2 章）在 RV64I 中也是 HINT 指令。RV64I 中的额外计算指令扩展了标准 HINT 和自定义 HINT 编码空间。

表 4.1 列出了所有 RV64I HINT 代码点。91% 的 HINT 空间保留用于标准 HINT，但目前没有定义。其余的 HINT 空间被指定用于自定义 HINT；在此子空间中永远不会定义标准 HINT。

表 4.1 RV64I HINT 指令

指令	约束	代码点	目的
LUI	rd=x0	2^{20}	指定用于将来的标准使用
AUIPC	rd=x0	2^{20}	
ADDI	rd=x0, and either rs1≠x0 or imm≠0	$2^{17}-1$	

ANDI	rd=x0	2^{17}	
ORI	rd=x0	2^{17}	
XORI	rd=x0	2^{17}	
ADDIW	rd=x0	2^{17}	
ADD	rd=x0, rs1≠x0	$2^{10}-32$	
ADD	rd=x0, rs1=x0, rs2≠x2-x5	28	
ADD	rd=x0, rs1=x0, rs2=x2-x5	4	(rs2=x2) NTL. P1 (rs2=x3) NTL. PALL (rs2=x4) NTL. S1 (rs2=x5) NTL. ALL
SUB	rd=x0	2^{10}	指定用于将来的标准使用
AND	rd=x0	2^{10}	
OR	rd=x0	2^{10}	
XOR	rd=x0	2^{10}	
SLL	rd=x0	2^{10}	
SRL	rd=x0	2^{10}	
SRA	rd=x0	2^{10}	
ADDW	rd=x0	2^{10}	
SUBW	rd=x0	2^{10}	
SLLW	rd=x0	2^{10}	
SRLW	rd=x0	2^{10}	
SRAW	rd=x0	2^{10}	
FENCE	rd=x0, rs1≠x0, fm=0, pred=0, succ=0	$2^{10}-63$	
FENCE	rd≠x0, rs1≠x0, fm=0, pred=0, succ=0	$2^{10}-63$	
FENCE	rd=rs1=x0, fm=0, pred=0, succ=0	15	
FENCE	rd=rs1=x0, fm=0, pred=W, succ=0	15	
FENCE	rd=rs1=x0, fm=0, pred=W, succ=0	1	终止
SLTI	rd=x0	2^{17}	指定自定义使用
SLTIU	rd=x0	2^{17}	
SLLI	rd=x0	2^{17}	

5 RV128I 基础整数指令集, V1.7

在计算机设计中，只有一个错误是难以纠正的——没有足够的地址位用于内存寻址和内存管理。

贝尔和斯特雷克, ISCA-3, 1976 年

本章描述了 RV128I，这是一个支持平面 128 位地址空间的 RISC-V ISA 变体。该变体是现有 RV32I 和 RV64I 设计的直接扩展。



扩展整数寄存器宽度的主要目的是支持更大的地址空间。目前尚不清楚何时会需要超过 64 位的平面地址空间。截至本文撰写时，世界上最快的超级计算机在 Top500 基准测试中排名第一，拥有超过 1PB 的 DRAM，如果所有 DRAM 都驻留在单一地址空间中，所需的地址空间就会超过 50 位。一些仓库级计算机已经包含更大容量的 DRAM，新的高密度固态非易失性存储器 and 高速互连技术可能会推动对更

大内存空间的需求。Exascale 系统研究的目标是实现 100PB 的内存系统,这将需要 57 位地址空间。按照历史增长速度,可能在 2030 年之前会需要超过 64 位的地址空间。历史表明,每当明确需要超过 64 位的地址空间时,架构师会反复讨论扩展地址空间的替代方案,包括分段、96 位地址空间和软件解决方法,最终往往选择平面 128 位地址空间,因为这是最简单且最佳的解决方案。目前我们尚未冻结 RV128 的规范,可能会根据实际的 128 位地址空间使用情况进一步调整设计。

RV128I 是在 RV64I 基础上构建的,正像 RV64I 是在 RV32I 上构建的一样,整数寄存器扩展为 128 位(即 XLEN=128)。大多数整数计算指令保持不变,因为它们都是被定义为在 XLEN 位宽度上进行的操作。RV64I 中的 *W 整数指令仍然保留,它们在寄存器的低 32 位上操作,但现在将其结果从第 31 位符号位扩展到第 127 位。新增了一组 *D 整数指令,它们在 128 位整数寄存器的低 64 位上操作,并将其结果从第 63 位符号位扩展到第 127 位。这些 *D 指令在标准 32 位编码中占用了两个主要操作码(OP-IMM-64 和 OP-64)。



为了提高 RV64 的兼容性,我们可能会反向调整解码方式,就像 RV32 到 RV64 的处理方式一样,将 RV64I 的 ADD 指令重命名为 64 位 ADDD,并在之前的 OP-64 主要操作码中添加 128 位 ADDQ(现已重命名为 OP-128 主要操作码)。

立即数移位(SLLI/SRLI/SRAI)现在使用 I 型立即数的低 7 位进行编码,而变量移位(SLL/SRL/SRA)则使用移位源寄存器的低 7 位。

新增了一条 LDU(Load Unsigned: 加载无符号双字)指令,使用现有的 LD 主操作码,并新增了 LQ 和 SQ 指令来加载和存储四倍字值。SQ 被添加到 STORE 主操作码中,而 LQ 被添加到 MISC-MEM 主操作码中。

浮点指令集保持不变,尽管 128 位 Q 浮点扩展现在可以支持 FMV.X.Q 和 FMV.Q.X 指令,并新增了 FCVT 指令,用于与 T(128 位)整数格式之间的转换。

6 Zifencei 扩展: 指令获取屏障, V2.0

本章定义了 Zifencei 扩展,该扩展包含 FENCE.I 指令,用于在同一硬件线程上实现对指令内存写入操作与指令获取操作之间的显式同步。目前,FENCE.I 指令是能够确保对某一内核可见的存储操作也能对这一内核的指令获取可见的唯一标准机制。



我们曾考虑过,但最终未采纳。JIT 编译器可能会在执行单条 FENCE.I 指令前,先生成大量的指令序列,并通过将翻译后的指令写入已知不在 I-cache 中的内存区域,来分摊指令缓存监听/失效操作的开销。



FENCE.I 指令的设计旨在支持多种不同的实现方式。一种简单的实现方案是在执行 FENCE.I 指令时,刷新本地指令缓存和指令流水线。更复杂的实现方案可能包括:在每次数据(或指令)缓存缺失时,监听指令(或数据)缓存;或者使用包含式统一私有二级缓存,以便在本地存储指令写入时,使主指令缓存中的缓存行失效。如果指令缓存和数据缓存以这种方式保持一致性,或者内存系统完全由非缓

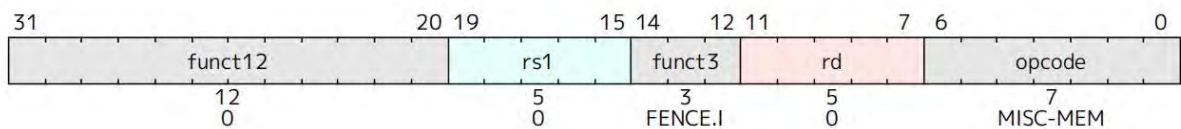
存 RAM 构成，那么在执行 FENCE.I 指令时，仅需刷新取指流水线即可。

FENCE.I 指令曾是基本 I 指令集的一部分。虽然在撰写本文时，它仍然是维护指令获取一致性的唯一标准方法。但是有两个主要问题促使我们将其从强制性基础指令集中移除。

首先，人们逐渐认识到，在某些系统上，FENCE.I 指令的实现成本可能很高，内存模型工作组也在探讨替代机制。特别是对于那些指令缓存和数据缓存不一致的设计，或者指令缓存的重填操作未监听一致性数据缓存的设计，当遇到 FENCE.I 指令时，必须完全刷新指令缓存和数据缓存。当统一缓存或外部内存系统前端存在多级指令缓存和数据缓存时，这个问题会变得尤为突出。

其次，在类 Unix 操作系统环境中，单凭这条指令还不足以在用户级别提供指令获取一致性保障。FENCE.I 指令仅同步本地硬件线程的指令获取行为。操作系统可能在 FENCE.I 指令执行完毕后，将用户硬件线程重新调度到不同的物理硬件线程上。这将要求操作系统在每次上下文切换时都执行额外的 FENCE.I 指令。因此，标准的 Linux ABI 已经将 FENCE.I 指令从用户级别移除，转而需要通过系统调用来维护指令获取一致性。这样做既能让操作系统在现有系统上尽可能减少 FENCE.I 指令的执行次数，又能为未来改进的指令获取一致性机制提供前向兼容性。

目前正在讨论的未来指令获取一致性方法包括：提供更受限的 FENCE.I 指令版本，例如仅针对 rs1 寄存器中指定地址的 FENCE.I 指令；和/或允许软件使用依赖于机器模式缓存维护操作的 ABI。



FENCE.I 指令用于同步指令流和数据流。RISC-V 架构不保证对指令内存的存储操作在 RISC-V 硬件线程上对指令的获取操作立即可见，除非该硬件线程执行 FENCE.I 指令。FENCE.I 指令确保在 RISC-V 硬件线程上，后续的指令获取操作能够观察到任何先前的数据存储操作，这些数据存储操作对于同一 RISC-V 硬件线程而言已经是可见的。FENCE.I 指令不能确保在多处理器系统中，其他 RISC-V 硬件线程的指令获取操作会观察到本地硬件线程的存储操作。为了使对指令内存的存储操作对所有 RISC-V 硬件线程都可见，执行写入的硬件线程还必须先执行数据 FENCE 指令，然后再请求所有远程 RISC-V 硬件线程执行 FENCE.I 指令。

FENCE.I 指令中未使用的字段 imm[11:0]、rs1 和 rd 保留给未来扩展中更细粒度的屏障指令使用。为了保持前向兼容性，基本实现应忽略这些字段，标准软件应将这些字段置零。



由于 FENCE.I 指令仅对单个硬件线程自身的指令获取操作进行排序，因此，仅当应用程序线程不会迁移到不同的硬件线程上时，应用程序代码才应依赖 FENCE.I 指令。执行环境接口（EEI）可以提供用于高效多处理器指令流同步的机制。

7 Zicsr 拓展：控制状态寄存器（CSR）指令，V2.0

RISC-V 定义了一个与每个硬件线程相关联的 4096 个控制状态寄存器（CSR）的独立地址空间。本章定义了操作这些 CSR 的完整 CSR 指令集。

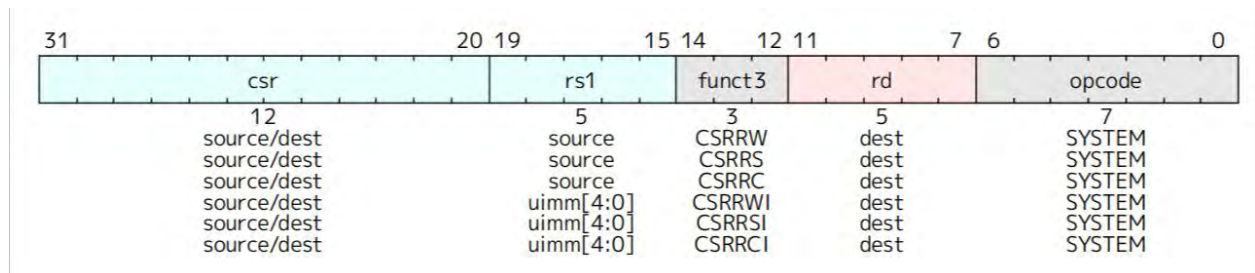


虽然控制状态寄存器（CSR）主要用于特权架构，但在非特权代码中也有多种应用，包括计数器、定时器以及浮点状态。

计数器和定时器不再是标准基本指令集（ISA）中的必需部分，因此访问这些寄存器的相关 CSR 指令已从第2章中移出，并单独归入本章。

7.1 控制状态寄存器指令

所有控制状态寄存器指令都原子性地读-修改-写单个控制状态寄存器，其控制状态寄存器指定符编码在指令的 12 位 `csr` 字段中，位于 31-20 位。立即数格式采用 `rs1` 字段中编码的 5 位零扩展立即数。



CSRRW（原子读/写控制状态寄存器）指令原子性地交换控制状态寄存器和整数寄存器中的值。CSRRW 首先读取控制状态寄存器的旧值，将其零扩展到 XLEN 位，然后将扩展后的值写入整数寄存器 `rd`。紧接着，`rs1` 中的初始值被写入控制状态寄存器。如果 `rd=x0`，则该指令不会读取控制状态寄存器，也不会引发任何可能在读取控制状态寄存器时产生的副作用。

CSRRS（原子读并设置控制状态寄存器中的位）指令读取控制状态寄存器的值，将其零扩展到 XLEN 位，并将结果写入整数寄存器 `rd`。整数寄存器 `rs1` 中的初始值被视为位掩码，指定要设置的控制状态寄存器中的位位置。`rs1` 中的任何高位都会导致相应的控制状态寄存器位被设置，前提是该位是可写的。

CSRRC（原子读并清除控制状态寄存器中的位）指令读取控制状态寄存器的值，将其零扩展到 XLEN 位，并将结果写入整数寄存器 `rd`。整数寄存器 `rs1` 的初始值作为位掩码，指定在控制状态寄存器中需要清除的位。`rs1` 中的任何非零位会清除相应控制寄存器中的位，前提是该位可写。

对于 CSRRS 和 CSRRC，如果 `rs1=x0`，则指令不会写入控制状态寄存器，因此不会引起任何可能在控制状态寄存器写入时发生的副作用，也不会访问只读控制寄存器时引发非法指令异常。无论 `rs1` 和 `rd` 的值如何，CSRRS 和 CSRRC 总是读取指定的控制状态寄存器，并引起任何读取时的副作用。注意，如果 `rs1` 指定的是一个非 `x0` 的寄存器，且该寄存器的值为零，则指令不会触发任何字段级副作用，但会执行写入整个控制寄存器时产生的副作用。CSRRW 与 `rs1=x0` 将尝试将零写入目标控制状态寄存器。

CSRRWI、CSRRSI 和 CSRRCI 变体类似于 CSRRW、CSRRS 和 CSRRC，但它们使用通过零扩展 `rs1` 字段中编码的 5 位无符号立即数（`uimm[4:0]`）得到的 XLEN 位值来更新控制状态寄存器，而非来自整数寄存器的值。对于 CSRRSI 和 CSRRCI，如果 `uimm[4:0]` 字段为零，这些指令将不写入控制寄存器，也不会引发任何副作用，也不会访问只读控制状态寄存器时触发非法指令异常。对于 CSRRWI，如果 `rd=x0`，则指令不会读取状态控制寄存器，也不会引发任何与读取控制寄存器相关的副作用。无论 `rd` 和 `rs1` 字段如何，CSRRSI 和 CSRRCI 始终会读取控制状态寄存器并引发读取副作用。

表 7.1 确定控制状态寄存器指令是否读取或写入指定控制状态寄存器的条件

寄存器操作数

介绍	rd is x0	rs1 is x0	CSR 读取	CSR 写入
CSRRW	是	–	否	是
CSRRW	否	–	是	是
CSRRS/CSRRC	–	是	是	否
CSRRS/CSRRC	–	否	是	是
立即操作数				
介绍	rd is x0	uimm 0	CSR 读取	CSR 写入
CSRRWI	是	–	否	是
CSRRWI	否	–	是	是
CSRRSI/CSRRCI	–	是	是	否
CSRRSI/CSRRCI	–	否	是	是

表 7.1 总结了控制状态寄存器指令在读取和/或写入控制状态寄存器方面的行为。除了由于读取或写入控制状态寄存器而发生的副作用外，控制状态寄存器中的各个字段在写入时可能会产生副作用。CSRRW[I] 指令对写入的控制状态寄存器中的所有此类字段执行副作用。CSRRS[I] 和 CSRRC[I] 指令仅对 rs1 或 uimm 参数中至少有一个位设置对应字段的字段执行副作用。



截至本文撰写时，没有标准控制状态寄存器在字段被写入的时候有副作用。因此，标准控制状态寄存器访问是否有任何副作用可以仅从操作码确定。

不推荐定义在字段写入时会引发副作用的控制状态寄存器。

对于由于控制状态寄存器具有特定值而发生的任何事件或结果，如果通过写入控制状态寄存器使其获得该值，那么所产生的事件或后果被称为该写入的间接效果。控制状态寄存器写入的间接效果被 RISC-V ISA 视为该写入的副作用。

控制状态寄存器访问副作用的一个例子是，如果从特定控制状态寄存器读取会导致灯泡亮起，而写入奇数值会导致灯泡熄灭，假设写入偶数值没有效果。在这种情况下，读取和写入都会控制灯泡是否点亮，因此这两个操作都有副作用，因为灯泡的状态不仅由控制状态寄存器的值决定。（注意，在写入奇数值使灯泡熄灭后，读取控制状态寄存器会使灯泡重新点亮，再次写入相同的奇数值会导致灯泡再次熄灭。因此，最后一次写入时，灯泡熄灭并非由于控制状态寄存器值的变化。）



另一方面，如果灯泡的亮灭与特定控制状态寄存器值是否为奇数相关，则点亮和熄灭灯泡不被视为写入控制状态寄存器的副作用，而仅仅是这种写入的间接效果。

更具体地说，RISC-V 特权架构在第二卷中规定，某些控制状态寄存器值组合会导致陷阱触发。当显式写入控制状态寄存器触发陷阱时，这个陷阱不被视为写入的副作用，而仅仅是该写入的间接效果。

标准控制状态寄存器在读取时没有副作用。标准控制状态寄存器在写入时可能有副作用。自定义扩展可能会添加在读取或写入时会引发副作用的控制状态寄存器。

某些控制状态寄存器，例如指令退役计数器 instret，可能会作为指令执行的副作用而被修改。在这种情况下，控制状态寄存器访问指令读取时会获取执行前的值。如果写入此类控制状态寄存器，写入操作会替代递增。特别地，一条指令写入的值将成为下一条指令读取的值。

汇编伪指令读取控制状态寄存器 CSRR rd, csr 编码为 CSRRS rd, csr, x0。汇编伪指令写入控制状态寄存器 CSRW csr, rs1 编码为 CSRRW x0, csr, rs1, 而 CSRWI csr, uimm 编码为 CSRRWI x0, csr, uimm。

进一步的汇编伪指令定义为在不需要旧值时设置和清除控制状态寄存器中的位：CSRS/CSRC csr, rs1; CSRSI/CSRCI csr, uimm。

7.1.1 控制状态寄存器访问顺序

每个 RISC-V 硬件线程通常按程序顺序观察其自己的控制状态寄存器访问，包括隐式的控制状态寄存器访问。特别是，除非另有规定，控制状态寄存器访问会在程序顺序中，先执行任何修改或受控状态寄存器状态的指令，然后再执行任何修改或受控状态寄存器状态的后续指令。此外，显式控制状态寄存器读取会返回指令执行前的控制状态寄存器状态，而显式控制状态寄存器写入则会抑制并覆盖同一指令对同一控制状态寄存器的任何隐式写入或修改。

同样，显式控制状态寄存器访问的副作用通常是按程序顺序同步观察的。除非另有规定，任何此类副作用的全部后果将在下一条指令中可观察到，且前面的指令不会观察到任何乱序的后果。（注意前面提到的控制状态寄存器写入的副作用和间接效果之间的区别。）

对于 RVWMO 内存一致性模型（第 17 章），控制状态寄存器访问默认是弱序的，因此其他硬件线程或设备可能会以不同于程序顺序的顺序观察控制状态寄存器访问。此外，控制状态寄存器访问与显式内存访问之间默认没有排序，除非控制状态寄存器访问修改执行显式内存访问的指令行为，或控制状态寄存器访问与显式内存访问由内存模型定义的语法依赖关系，或本手册第二卷中定义的内存排序 PMA 要求排序。为了在其他情况下强制排序，软件应在相关访问之间执行 FENCE 指令。对于 FENCE 指令的目的，控制状态寄存器读取访问被分类为设备输入（I），控制状态寄存器写入访问被分类为设备输出（O）。

非正式地，控制状态寄存器空间被视为弱排序的内存映射 I/O 区域，具体定义参见本手册第二卷中的内存排序 PMA 部分。因此，控制状态寄存器访问的顺序与所有其他访问的顺序受到与内存映射 I/O 区域访问顺序相同的机制约束。



这些控制状态寄存器排序限制旨在支持与设备或其他硬件线程可见的控制状态寄存器访问相关的主内存和内存映射 I/O 访问顺序。例如 time、cycle 和 mcycle 控制状态寄存器，以及反映挂起中断的控制状态寄存器，如 mip 和 sip。注意，此类控制状态寄存器的隐式读取（例如，由于 mip 的变化而中断）也按设备输入排序。

大多数控制状态寄存器（如 fcsr）对其他硬件线程不可见；它们的访问可以在全局内存顺序中自由重排序，而不会违反本规范。

硬件平台可以定义某些控制状态寄存器为强排序的，具体定义见本手册第二卷中的内存排序 PMA 部分。对强排序控制状态寄存器的访问，具有比弱排序控制状态寄存器和内存映射 I/O 区域访问更严格的排序约束。



控制状态寄存器在全局内存顺序中的重新排序规则，应移至 第17章中与 RVWMO 内存一致性模型相关的部分。

8 Zicntr 和 Zihpm 扩展：计数器，V2.0

RISC-V ISA 提供了一组最多 32 个 64 位的性能计数器和定时器，用户可以通过非特权的 XLEN 位只读 CSR 寄存器 0xC00 - 0xC1F 进行访问（当 XLEN=32 时，上 32 位可通过 CSR 寄存器 0xC80 - 0xC9F 访问）。这些计数器被划分为 Zicntr 和 Zihpm 扩展。

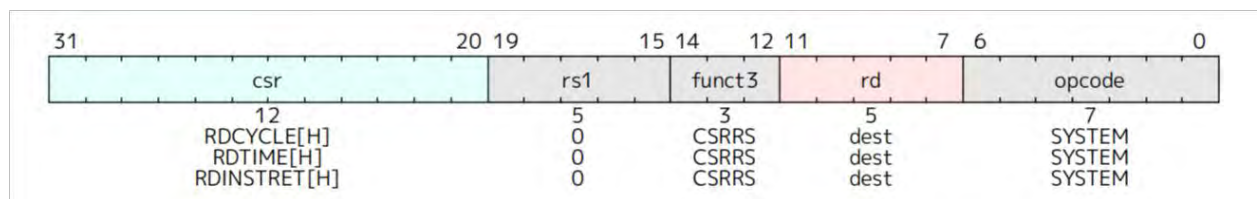
8.1 Zicntr 扩展：基本计数器和定时器

Zicntr 标准扩展包括前三个基本计数器（CYCLE、TIME 和 INSTRET），分别用于 CPU 周期计数、实时时钟和已提交指令数。Zicntr 扩展依赖于 Zicsr 扩展。



我们建议在实现中提供这些基本计数器，因为它们对于基本性能分析、自适应和动态优化至关重要，并且允许应用程序处理实时的数据流。Zihpm 扩展中的额外计数器有助于诊断性能问题，且应以低开销的方式向用户级应用程序代码开放访问。

某些执行环境可能会禁止访问计数器，例如，防止时间侧信道攻击。



对于 $XLEN \geq 64$ 的基本 ISA，CSR 指令可直接访问完整的 64 位 CSR。特别地，RDCYCLE、RDTIME 和 RDINSTRET 伪指令可读取 cycle、time 和 instret 计数器的完整 64 位数据。



计数器伪指令映射到只读的 csrrs rd, counter, x0 规范形式，但其他只读 CSR 指令形式（基于 CSRRS/CSRRSI/CSRRCI）也是读取这些 CSR 的合法方式。

对于 $XLEN=32$ 的基本 ISA，Zicntr 扩展使得这三个 64 位只读计数器可以以 32 位片段进行访问。RDCYCLE、RDTIME 和 RDINSTRET 伪指令提供低 32 位，而 RDCYCLEH、RDTIMEH 和 RDINSTRETH 伪指令提供各自计数器的高 32 位。



我们要求计数器在 $XLEN=32$ 时仍需保持 64 位宽度，否则软件难以判断是否溢出。对于轻量级实现，每个计数器的高 32 位可以通过由低 32 位溢出触发的陷阱处理程序递增的软件计数器来实现。下面的示例代码展示了如何使用单个 32 位宽度的伪指令安全地读取完整的 64 位值。

RDCYCLE 伪指令读取 cycle CSR 的低 XLEN 位，该 CSR 保存处理器核从某个任意起始时间开始执行的时钟周期数。当 $XLEN=32$ 时，RDCYCLEH 仅存在并读取相同周期计数器的 63-32 位。基础 64 位计数器不应溢出。周期计数器的递增速率将取决于实现和操作环境。执行环境应提供一种方法来确定周期计数器递增的当前速率（周期/秒）。



RDCYCLE 的目的是返回处理器核执行的周期数，而非硬件线程的周期数。由于某些实现（例如 AMD Bulldozer），精确定义核是困难的。精确定义时钟周期同样具有挑战性，因为实现方式差异较大（包括软件仿真）。然而，RDCYCLE 的主要目的是与其他性能计数器一起用于性能监控。特别地，当只有一个硬件线程/核时，周期计数/指令退役可用来衡量硬件线程的 CPI。

核不必完全暴露给软件，实施者可能会选择将一个物理核上的多个硬件线程伪装成运行在独立核上的线程，每个硬件线程/核提供独立的周期计数器。这在某些简单的桶式处理器（例如 CDC 6600 外围处理器）中可能有意义，在这种架构中，硬件线程之间的时间交互要么不存在，要么非常小。

当有多个硬件线程/核以及动态多线程时，通常无法准确区分每个硬件线程的周期数（尤其是在 SMT 中）。可能会定义一个单独的性能计数器来捕获特定硬件线程执行的周期数，但这种定义必须非常模糊，以适应所有可能的线程实现。例如，我们是否仅计算该硬件线程执行的任何指令所经历的周期，或者包括指令退役时的周期，抑或是包括该硬件线程因其他硬件线程执行而无法执行的周期？可能需要考虑以上所有情况才能提供可理解的性能统计数据。这种定义每个硬件线程周期计数的复杂性，以及调整多线程代码时对每核周期计数的需求，导致标准化每核周期计数成为最佳选择，这对于常见的单硬件线程/核情况也同样适用。

标准化睡眠期间发生的情况并不现实，因为睡眠在不同的执行环境中并没有统一定义。但如果整个核进入暂停状态（完全时钟门控或深度睡眠中断电），则核不会执行时钟周期，周期计数应按照规定保持不变。细节部分，如是否应计数唤醒后处理器恢复所需的时钟周期等，视为特定执行环境的细节。

尽管没有适用于所有平台的精确定义，但对大多数平台而言，这仍然是一个有用的功能。一个不精确、常见的、通常正确的标准比没有标准要好。RDCYCLE 的设计初衷是用于性能监控和调优，规范的编写目标正是围绕这一点展开的。

RDTIME 伪指令读取 time CSR 的低 XLEN 位，该 CSR 计算从过去某个任意起始时间起经过的实际时间。当 XLEN=32 时，RDTIMEH 仅存在并读取相同实时计数器的 63-32 位。基础 64 位计数器随着每个实时时钟的节拍递增，对于实际的实时时钟频率，实际上不应溢出。执行环境应提供一种方法来确定计数器节拍的周期（秒/节拍）。该周期应在一个小的误差范围内保持恒定。环境应提供一种方法来确定时钟的准确性（即标称和实际实时时钟周期之间的最大相对误差）。

在一些简单的平台上，周期计数可能是 RDTIME 的有效实现，在这种情况下，RDTIME 和 RDCYCLE 可能返回相同的结果。



鉴于可能的实现平台种类繁多，很难对时钟周期进行严格的规定。最大误差范围应根据平台的要求设定。

所有硬件线程的实时时钟必须同步到实时时钟的一个节拍内。



与其他架构类似，只要硬件线程看起来好像同步到实时时钟的一个节拍内即可，即软件无法观察到两个硬件线程上观察到的实时时钟值之间存在更大的差异。

RDINSTRET 伪指令读取 instret CSR 的低 XLEN 位，该 CSR 计算从过去某个任意起始点起该硬件线程退役的指令数。当 XLEN=32 时，RDINSTRETH 仅存在并读取相同指令计数器的 63-32 位。基础 64 位计数器在实践中不应溢出。



导致同步异常的指令，包括 ECALL 和 EBREAK，不被视为退役，因此不会增加 instret CSR。

以下代码序列将读取一个有效的 64 位周期计数器值到 x3:x2 中，即使计数器在读取其高半部分和低半部分之间溢出。

again:

```
rdcycleh    x3
rdcycle     x2
```

rdcycleh	x4
bne	x3, x4, again

8.2 Zihpm 扩展：硬件性能计数器

Zihpm 扩展包括多达 29 个额外的非特权 64 位硬件性能计数器，hpmcounter3-hpmcounter31。当 XLEN=32 时，这些性能计数器的高 32 位可以通过额外的 CSR hpmcounter3h-hpmcounter31h 访问。Zihpm 扩展依赖于 Zicsr 扩展。



在某些应用中，能够在同一时间读取多个计数器是很重要的。当在多任务环境下运行时，用户线程在尝试读取计数器时可能会遭遇上下文切换。解决方案之一是用户线程在读取其他计数器之前和之后读取实时计数器，以确定在读取序列中是否发生了上下文切换，在这种情况下可以重试读取。我们曾考虑添加输出锁存器以允许用户线程原子地快照计数器值，但这会增加用户上下文的大小，特别是对于具有更多计数器集的实现。

这些额外计数器的实现数量与宽度，以及它们计数的事件集，是平台相关的。访问未实现或配置错误的计数器可能会导致非法指令异常或返回常数值。

执行环境应提供一种方法来确定已实现计数器的数量和宽度，以及配置每个计数器要计数的事件的接口。



对于在 RISC-V 特权平台上实现的执行环境，特权架构手册描述了控制低特权模式访问这些计数器的特权 CSR，并设置要计数的事件。

替代执行环境（例如：仅用户级的软件性能模型）可能提供替代机制来配置性能计数器计数的事件。

最终标准化事件设置在计数 ISA 级别的指标方面非常重要，例如执行的浮点指令数量，以及一些可能常见的微架构指标，例如 L1 指令缓存未命中。

9 Zihintntl 扩展：非时间局部性提示，V1.0

NTL 指令是一种提示（HINT），用于指示紧随其后的目标指令的显式内存访问可能表现出较差的时间局部性。NTL 指令不会更改架构状态，也不会影响目标指令的架构可见效果。提供了四种变体：

NTL.P1 指令用于提示目标指令在内存层次结构中最内层的私有缓存范围内缺乏时间局部性。其编码方式为 ADD x0, x0, x2。

NTL.PALL 指令用于提示目标指令在内存层次结构中的任何级别的私有缓存范围内缺乏时间局部性。其编码方式为 ADD x0, x0, x3。

NTL.S1 指令用于提示目标指令在内存层次结构中最内层的共享缓存范围内缺乏时间局部性。其编码方式为 ADD x0, x0, x4。

NTL.ALL 指令用于提示目标指令在内存层次结构中的任何级别的缓存范围内缺乏时间局部性。其编码方式为 ADD x0, x0, x5。



NTL 指令可用于避免流式数据处理或遍历大型数据结构时的缓存污染，或在生产者-消费者交互场景中降低延迟。

在微架构层面，NTL 指令可以用于指导缓存替换策略，决定数据应分配至哪个缓存级别，甚至完全避免缓存分配。例如，NTL.P1 可指示处理器不要在私有 L1 缓存中分配缓存行，而应在 L2（无论是私

有还是共享)中分配。另一些实现可能仍会在 L1 缓存中分配缓存行,但将其设为最近最少使用状态。

NTL. ALL 通常用于指示处理器避免在整个缓存层次结构中进行分配。程序员应在无法利用时间局部性的内存访问(例如流式传输)时使用 NTL. ALL。

与所有提示指令一样,NTL 指令可以被自由忽略。因此,尽管本文档以基于缓存的内存层次结构为背景描述这些指令,但它们并不强制要求提供缓存机制。

部分处理器可能会选择性地应用这些提示,例如某些通过 L1 缓存独占状态获取缓存行以实现 LR/SC 的处理器,可能会忽略 LR 和 SC 指令前的 NTL 指令,但仍可能对 AMO 及常规加载/存储指令前的 NTL 指令加以应用。

表 9.1 列出了多个软件使用场景,并提供了可移植软件(即未针对特定实现的内存层次结构进行调优的软件)应采用的推荐 NTL 变体。

表 9.1 推荐便携式软件在不同场景中采用的 NTL 变体

场景	推荐的NTL变体
访问工作集规模在小和中之间	NTL. P1
访问工作集规模在中和大之间	NTL. PALL
访问工作集规模大于	NTL. S1
没有可利用时间局部性的访问(例如,流式传输)	NTL. ALL
访问争用的同步变量	NTL. PALL

 表9.1中列出的工作集大小并非旨在约束实现者的缓存大小决策。缓存大小在不同实现之间显然会有所不同,因此程序员应仅将这些工作集大小作为粗略的指导方针。

9.1 列出了几个示例内存层次结构,并推荐了每种 NTL 变体如何映射到每个缓存级别。该表还推荐了实现调整的软件应使用哪种 NTL 变体来避免在特定缓存级别进行分配。例如,对于具有私有 L1 和共享 L2 的系统,建议 NTL. P1 和 NTL. PALL 指示 L1 无法利用时间局部性,NTL. S1 和 NTL. ALL 指示 L2 无法利用时间局部性。此外,针对此类系统调整的软件应使用 NTL. P1 来指示 L1 无法利用的时间局部性,或应使用 NTL. ALL 来指示 L2 无法利用的时间局部性。

如果提供了 C 扩展,则还提供了这些 HINT(提示)的压缩变体:C. NTL. P1 的编码为 C. ADD x0, x2; C. NTL. PALL 的编码为 C. ADD x0, x3; C. NTL. S1 的编码为 C. ADD x0, x4; C. NTL. ALL 的编码为 C. ADD x0, x5。NTL 指令影响除 Zicbom 扩展中的缓存管理指令外的所有内存访问指令。

 截至本文撰写时,没有其他例外情况,因此 NTL 指令影响基本 ISA 和 A、F、D、Q、C 和 V 标准扩展中定义的所有内存访问指令,以及第二卷中的虚拟机监视器扩展中定义的那些指令。

 NTL 指令可以影响 Zicbom 扩展之外的缓存管理操作。例如,NTL. PALL 后跟 CBO. ZERO 可能表示该行应在 L3 中分配并清零,但不应在 L1 或 L2 中分配。

当 NTL 指令被应用于 Zicbop 扩展的预取提示时,它所指示的是,应该将缓存行预取到相对于 NTL 所指定级别更外部的缓存层级中。

例如，在具有私有 L1 和共享 L2 的系统中，NTL.P1 后跟 PREFETCH.R 可能会将数据预取到具有读取意图的 L2 中。



要预取到最内层缓存，请不要在预取指令前添加 NTL 指令。

在某些系统中，NTL.ALL 后跟预取指令可能会预取到内存控制器内部的缓存或预取缓冲区中。

我们不建议软件在 NTL 指令之后跟随一条不显式访问内存的指令。如果不遵循此建议，可能会降低性能，但在架构层面不会产生任何可见影响。

如果在目标指令上发生陷阱事件，我们建议处理器实现不要将 NTL 提示应用于陷阱处理程序的第一条指令。相反，我们更建议在这种情况下忽略该提示。

假定在执行 NTL 指令与其目标指令之间发生中断，程序执行通常依然会在目标指令处恢复。NTL 指令是否重新执行并不会改变程序的语义。



考虑到某些处理器实现可能倾向于在确认目标指令之后再处理 NTL 指令（例如：为了使 NTL 能够与其修饰的内存访问操作相融合）。因此，这类实现可能更倾向于在 NTL 指令之前响应中断，而不是在 NTL 指令和内存访问操作之间响应中断。

正因如此，NTL 指令才被编码为 ADD 指令，这样一来，它们就可以在 LR/SC 循环中使用，而不会破坏持续进展保证。然而，又由于在 LR/SC 循环中使用其他加载和存储确实会破坏持续进展保证（forward-progress guarantee），故而，在此类循环中使用 NTL 的唯一目的是为了修改 LR 或 SC 指令。



10 Zihintpause 扩展：PAUSE 提示指令，V2.0

PAUSE 指令是一条硬件提示指令（HINT），用于表明当前硬件线程的指令执行速度应被临时降低或暂停。其影响的持续时间必须有界，且可能为零。

软件可以利用 PAUSE 指令来降低执行自旋等待循环时的功耗。在多线程处理器核上，当执行 PAUSE 指令时，硬件线程或许会暂时将执行资源让渡给其他硬件线程。通常建议在自旋等待循环的代码中包含 PAUSE 指令。

未来的扩展可能会添加类似于 x86 架构的 MONITOR/MWAIT 指令，这些指令提供了更高效的机制，用于等待对特定内存位置的写入。然而，这些指令并不会替代 PAUSE 指令。PAUSE 指令更适用于轮询非内存事件、轮询多个事件，或者当软件无法准确得知其正在轮询何种事件时。



PAUSE 指令的效果持续时间在不同实现之间以及同一实现内部可能存在显著差异。在典型实现中，该持续时间应远小于上下文切换所需的时间，大致与片上缓存未命中延迟或无缓存访问主存的延迟相当。

一系列连续执行的 PAUSE 指令可以产生累积的延迟效果，总延迟时间与 PAUSE 指令的数量大致成正比。然而，在可移植代码的自旋等待循环中，在重新评估循环条件之前务必仅使用一条 PAUSE 指令；否则在某些实现中，硬件线程可能会非必要地停滞过长时间，从而导致系统性能下降。

PAUSE 指令采用 FENCE 指令的编码格式，并定义其字段为：pred=W、succ=0、fm=0、rd=x0 以及 rs1=x0。



注：PAUSE 指令之所以被编码为 FENCE 操作码中的一种提示，是因为部分实现可能会刻意使得 PAUSE 指令停顿，直至所有未完成的内存事务处理完毕。然而，鉴于其后续指令集为空，PAUSE 指令并不强制规定任何特定的内存访问顺序。因此，它的确只是一个硬件提示。

注：与其他的 FENCE 指令相同，PAUSE 指令也不能在 LR/SC 序列中使用，否则将破坏持续进展。

注：由于其后继集为空，PAUSE 指令可以任意选择前驱指令集 W。与 PAUSE 指令类似的其他提示指令可能会使用其他前驱集进行编码。

11 Zimop 扩展：可能操作（May-Be-Operations），V1.0

本章定义了 Zimop 扩展，它引入了可能操作（MOPs）的概念。MOPs 最初被定义为简单地将零写入 $x[rd]$ ，但设计为可以通过后续扩展重新定义，以执行其他操作。Zimop 扩展为 40 个 MOP 定义了编码空间。

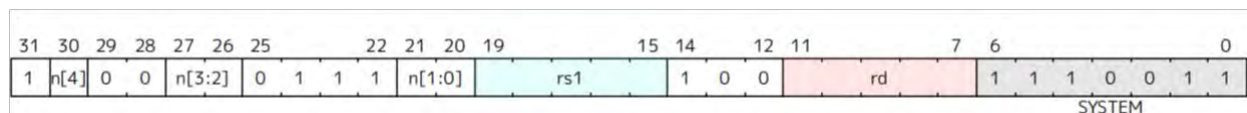


有时，我们希望定义一种指令集扩展：当这一指令集扩展中的指令未被实现时，这些指令不会引发非法指令异常，而是执行无实际作用的操作（除了写入 $x[rd]$ 以外）。例如，具有控制流完整性检查功能的程序可以在没有相应扩展的实现上正确运行，前提是检查被简单地忽略。将这些检查实现为 MOPs 允许相同的程序在有或没有相应扩展的实现上运行。

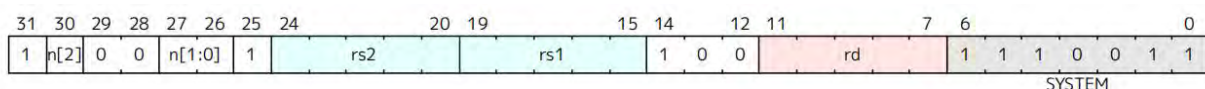
尽管在某些方面与 HINTs 类似，但 MOPs 不能编码为 HINTs，因为与 HINTs 不同，MOPs 允许改变架构状态。

由于 MOPs 可能会被后续扩展重新定义，标准软件不应执行 MOP，除非它明确地面向已重新定义该 MOP 的扩展。

Zimop 扩展定义了 32 个名为 MOP.R.n 的 MOP 指令，其中 n 是 0 到 31 之间的整数（包括 0 和 31）。除非被其他扩展重新定义，否则这些指令只是将 0 写入 $x[rd]$ 。它们的编码允许未来的扩展将它们定义为读取 $x[rs1]$ 并且写入 $x[rd]$ 。



Zimop 扩展还定义了 8 个名为 MOP.RR.n 的 MOP 指令，其中 n 是 0 到 7 之间的整数（包括 0 和 7）。



除非被其他扩展重新定义，否则这些指令只是将 0 写入 $x[rd]$ 。它们的编码允许未来的扩展将它们定义为读取 $x[rs1]$ 和 $x[rs2]$ 并且写入 $x[rd]$ 。



MOP.R.n 的推荐汇编语法是 MOP.R.n rd, rs1，任何 x 寄存器指定符对任一参数都是有效的。类似地，MOP.RR.n 的推荐语法是 MOP.RR.n rd, rs1, rs2。重新定义 MOP 的扩展可以定义替代的汇编助记符。



这些 MOP 被编码在 SYSTEM 主操作码中，部分原因是预计它们的行为将会被特权控制状态寄存器状态调节。



这些 MOP 被定义为将零写入 $x[rd]$ ，而不是不执行任何操作，从而简化指令解码，并允许通过判断结果是否为零来测试功能是否存在。

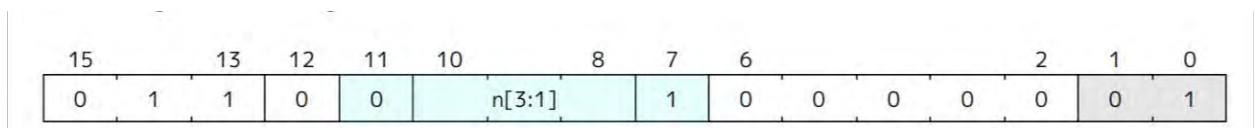
Zimop 扩展中定义的 MOP 不在语法上要求 $x[rs1]$ 或 $x[rs2]$ 到 $x[rd]$ 的依赖关系，尽管重新定义该 MOP 的扩展可能会添加此类要求。



不包含语法依赖性的简单实现可以不读取 $x[rs1]$ 和 $x[rs2]$ 。

11.1 Zcmop 扩展：可能操作的压缩（Compressed May-Be-Operations）

本节定义了 Zcmop 扩展，它定义了八个 16 位 MOP 指令，名为 C.MOP.n，其中 n 是 1 到 15 之间的奇数（包括 1 和 15）。C.MOP.n 被编码在对应于 C.LUI $xn, 0$ 的保留编码空间中，如表 11.1 所示。与 Zimop 扩展中定义的 MOP 不同，C.MOP.n 指令被定义为不写入任何寄存器。它们的编码允许未来的扩展将它们定义为读取寄存器 $x[n]$ 。



合适的 16 位编码空间非常少。选择此空间是因为它已经在 $rd/rs1$ 字段方面具有不寻常的行为——当字段包含 $x2$ 时，它编码为 `c.addi16sp`——因此对于大多数用途来说价值较低。

表 11.1 C.MOP.n 指令编码

助记符	编码	可重新定义以读取寄存器
C.MOP.1	0110000010000001	X1
C.MOP.3	0110000110000001	X3
C.MOP.5	0110001010000001	X5
C.MOP.7	0110001110000001	X7
C.MOP.9	0110010010000001	X9
C.MOP.11	0110010110000001	X11
C.MOP.13	0110011010000001	X13
C.MOP.15	0110011110000001	X15



C.MOP.n的推荐汇编语法为无参数的简单形式 C.MOP.n。可能访问的寄存器是隐式的 xn。



预期每条 Zcmop 指令都等效于某条 Zimop 指令，但扩展（如果有）的选择留给重新定义 MOP 的扩展。

提示：不写入值的 Zcmop 指令可以扩展为写入 x0。

12 Zicond 扩展：整数条件操作，V1.0.0

12.1 介绍

Zicond 扩展定义了一种简单的符合 RISC-V 设计理念的解决方案，用于支持条件算术和条件选择/移动操作。这一方案提供了许多好处和高度的灵活性。这些指令遵循 R 型指令的格式，具有 3 个操作数（即 2 个源操作数和 1 个目标操作数）。使用这些指令，可以实现无分支序列（通常是两条指令的序列），无需指令融合、在解码架构指令时的特殊规定或其他微架构规定。

相较于竞品指令集架构，RISC-V 的一个不足之处是缺乏支持无分支代码生成的条件操作：这包括条件算术运算、条件选择和条件移动操作。RISC-V 的设计原则（例如不支持包含三个源寄存器和一个输出寄存器的指令格式）使得直接引入竞品指令的等价形式可能性较低。

然而，低成本的条件指令是极具价值的，因为它们可以在广泛的场景下替换分支（无论分支是否不可预测或可预测），以减少 BTB 和分支预测器的容量和别名压力，并允许更长的基本块（供硬件和编译器使用）。

12.2 Zicond 规范

条件操作扩展定义了一种简单的符合 RISC-V 设计理念的解决方案，用于支持条件算术和条件选择/移动操作。这一方案提供了许多好处和高度的灵活性。这些指令遵循 R 型指令的格式，具有 3 个操作数（即 2 个源操作数和 1 个目标操作数）。使用这些指令，可以实现无分支序列（通常是两条指令的序列），无需指令融合、在解码架构指令时的特殊规定或其他微架构规定。

以下指令包含 Zicond 扩展：

RV32	RV64	Mnemonic	Instruction
✓	✓	<i>czero.eqz rd, rs1, rs2</i>	Conditional zero, if condition is equal to zero
✓	✓	<i>czero.nez rd, rs1, rs2</i>	Conditional zero, if condition is nonzero



架构说明：由于缺乏立即数或额外的寄存器操作数进行比较，定义额外的比较（除了等于零和不等于零之外）没有好处。

基于这两条指令，支持以下条件算术操作的合成指令（即短指令序列）：

- * 条件加法，如果为零
- * 条件加法，如果非零
- * 条件减法，如果为零
- * 条件减法，如果非零

- * 条件按位与，如果为零
- * 条件按位与，如果非零
- * 条件按位或，如果为零
- * 条件按位或，如果非零
- * 条件按位异或，如果为零
- * 条件按位异或，如果非零

此外，还支持以下条件选择指令：

- * 条件选择，如果为零
- * 条件选择，如果非零

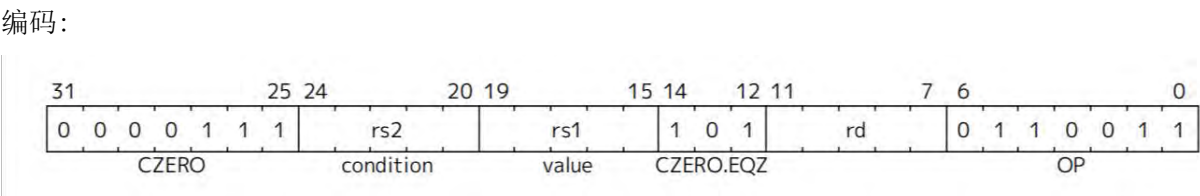
更复杂的条件（如与立即数比较、寄存器比较、单比特测试、范围比较等）可通过组合这些新指令与现有指令实现。

12.3 指令（按字母顺序）

12.3.1 czero.eqz

简述：
如果条件 rs2 等于零，则将零移动到寄存器 rd，否则将 rs1 移动到 rd。

助记符：
czero.eqz rd, rs1, rs2



描述：
如果 rs2 包含值零，则此指令将值零写入 rd。否则，此指令将 rs1 的内容复制到 rd。
此指令在语法上存在 rs1 和 rs2 到 rd 之间的依赖关系。此外，如果实现了 Zkt 扩展，则此指令的时序与 rs1 和 rs2 中的数据值无关。

SAIL 代码：

```

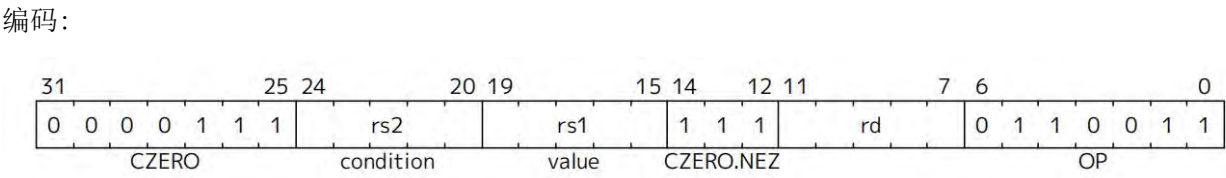
let condition = X(rs2);
result : xlenbits = if (condition == zeros()) then zeros()
                                     else X(rs1);

X(rd) = result;
```

12.3.2 czero.nez

简述：
如果条件 rs2 不等于零，则将零移动到寄存器 rd，否则将 rs1 移动到 rd。

助记符:
czero.nez rd, rs1, rs2



描述:

如果 rs2 包含非零值, 则此指令将值零写入 rd。否则, 此指令将 rs1 的内容复制到 rd。

此指令在语法上存在 rs1 和 rs2 到 rd 之间的依赖关系。此外, 如果实现了 Zkt 扩展, 则此指令的时间与 rs1 和 rs2 中的数据值无关。

SAIL 代码:

```
let condition = X(rs2);
result : xlenbits = if (condition != zeros()) then zeros()
                                     else X(rs1);

X(rd) = result;
```

12.4 使用示例

表 12.1 指令序列

操作	指令序列
rd = (rc == 0) ? (rs1 + rs2) : rs1	czero.nez rd, rs2, rc add rd, rs1, rd
rd = (rc != 0) ? (rs1 + rs2) : rs1	czero.eqz rd, rs2, rc add rd, rs1, rd
rd = (rc == 0) ? (rs1 - rs2) : rs1	czero.nez rd, rs2, rc sub rd, rs1, rd
rd = (rc != 0) ? (rs1 - rs2) : rs1	czero.eqz rd, rs2, rc sub rd, rs1, rd
rd = (rc == 0) ? (rs1 rs2) : rs1	czero.nez rd, rs2, rc or rd, rs1, rd

此扩展中的指令可用于构建执行条件算术、条件按位逻辑和条件选择操作的序列。

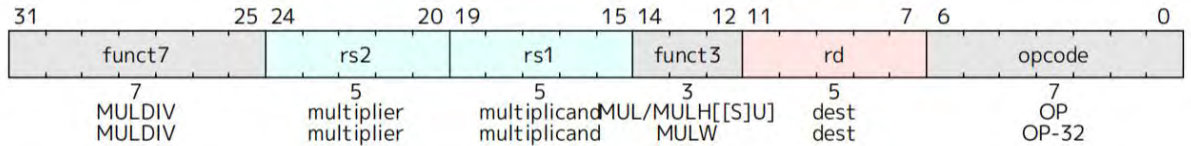
13 M 扩展：用于整数乘法和除法，V2.0

本章描述了标准的整数乘法和除法指令扩展, 名为 M 扩展, 包含用于对两个整数寄存器中保存的值进行乘法或除法运算的指令。



我们将整数乘法和除法从基础架构中分离出来，以简化轻量级实现，或适用于那些整数乘法和除法操作较为罕见或更适合由附加加速器处理的应用。

13.1 乘法操作 (Multiplication Operations)



MUL 执行 $rs1$ 乘以 $rs2$ 的 $XLEN$ 位 $\times$ $XLEN$ 位乘法，并将低 $XLEN$ 位放入目标寄存器。MULH、MULHU 和 MULHSU 执行相同的乘法，但返回完整 $2 \times XLEN$ 位乘积的高 $XLEN$ 位，分别用于有符号 $\times$ 有符号、无符号 $\times$ 无符号和 $rs1 \times$ 无符号 $rs2$ 乘法。如果需要同一乘积的高位和低位，则推荐的代码序列是：MULH[S]U rdh, rs1, rs2; MUL rdl, rs1, rs2（源寄存器说明符必须相同顺序，且 rdh 不能与 $rs1$ 或 $rs2$ 相同）。微架构可以将这些融合为单个乘法操作，而不是执行两个独立的乘法。



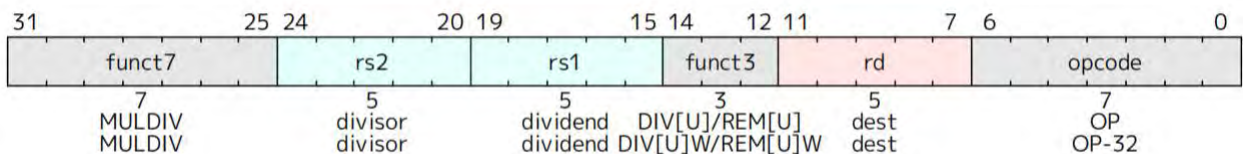
MULHSU 用于多字有符号乘法，将一个乘数的最高有效字（包含符号位）与一个乘数的低有效字（无符号）相乘。

MULW 是一个 RV64 指令，它将源寄存器的低 32 位相乘，并将结果的低 32 位符号扩展到目标寄存器。



在 RV64 中，MUL 可用于获取 64 位乘积的高 32 位，但有符号参数必须是正确的 32 位有符号值，而无符号参数的高 32 位必须清零。如果参数不确定是否已符号或零扩展，可以将两个参数都左移 32 位，然后使用 MULH[S]U。

13.2 除法操作 (Division Operations)



DIV 和 DIVU 执行 $rs1$ 除以 $rs2$ 的 $XLEN$ 位有符号和无符号整数除法，向零舍入。REM 和 REMU 提供相应除法操作的余数。对于 REM，非零结果的符号等于被除数的符号。



对于有符号和无符号除法，除溢出情况外，均满足 $dividend = divisor \times quotient + remainder$ 。

如果需要同一除法的商和余数，则推荐的代码序列是：DIV[U] rdq, rs1, rs2; REM[U] rdr, rs1, rs2（需要保证 rdq 不能与 $rs1$ 或 $rs2$ 相同）。在这一序列下，微架构可以将这些融合为单个除法操作，而不是执行两个独立的除法。

DIVW 和 DIVUW 是 RV64 指令，它们将 $rs1$ 的低 32 位除以 $rs2$ 的低 32 位，分别将它们视为有符号和无符号整数，并将 32 位商放入 rd ，符号扩展到 64 位。REMW 和 REMUW 是 RV64 指令，分别

提供相应的有符号和无符号余数操作。REMW 和 REMUW 总是将 32 位结果符号扩展到 64 位，包括在除以零时。

除以零和除法溢出的语义总结在表 13.1 中。除以零的商所有位都设置为 1，除以零的余数等于被除数。有符号除法溢出仅在最小负整数除以 -1 时发生。有符号除法溢出的商等于被除数，余数为零。无符号除法溢出不会发生。

表 13.1 除零和除溢出的语义

Condition	Dividend	Divisor	DIVU[W]	REMU[W]	DIV[W]	REM[W]
Division by zero	X	0	2^{L-1}	X	-1	X
Overflow (signed only)	-2^{L-1}	-1	-	-	-2^{L-1}	0



我们曾考虑在整数除以零时引发异常，这些异常在大多数执行环境中会触发陷阱。然而，这将是标准 ISA 中唯一的算术陷阱（浮点异常通过设置标志并写入默认值来处理，但不会触发陷阱），并且这会要求语言实现者与执行环境的陷阱处理程序进行交互。进一步来说，当语言标准要求除以零异常必须立即导致控制流变化时，只需在每个除法操作中添加一个分支指令，该指令可插入在除法操作后，并且通常预期不会被采取，从而对运行时的开销影响极小。

为了简化除法器电路，无论是无符号除法还是有符号除法，都会返回所有位都设置为 1 的值。所有 1 的值既是无符号除法返回的自然值，表示最大的无符号数，也是简单无符号除法器实现的自然结果。有符号除法通常使用无符号除法电路实现，指定相同的溢出结果从而简化了硬件设计。

13.3 Zmmul 扩展：版本 1.0

Zmmul 扩展实现了 M 扩展的乘法子集。它添加了第 13.1 章中定义的所有指令，即：MUL、MULH、MULHU、MULHSU 和（仅适用于 RV64）MULW。编码与相应的 M 扩展指令相同。M 包含 Zmmul。



Zmmul 扩展使需要乘法操作但不需要除法的低成本实现成为可能。对于许多微控制器应用，除法操作不够频繁，不足以证明除法器硬件的成本是合理的。相比之下，乘法操作更频繁，使得乘法器硬件的成本更合理。简单的 FPGA 软核特别受益于消除除法但保留乘法，因为许多 FPGA 提供硬连线乘法器，但需要在软逻辑中实现除法器。

14 A 扩展：原子指令，V2.1

A 扩展，即原子指令扩展，包含了原子性地读-改-写内存的指令，以支持在同一内存空间中运行的多个 RISC-V 硬件线程之间的同步。提供的两种原子指令形式是加载保留/存储条件指令和原子取值并操作内存指令。这两种类型的原子指令支持多种内存一致性排序，包括无序、获取、释放和顺序一致语义。这些指令使 RISC-V 能够支持 RCsc 内存一致性模型。



经过深入的探讨，语言社区和架构社区似乎最终倾向于将释放一致性模型作为标准的内存一致性模型。因此，RISC-V 的原子操作支持也是围绕此模型构建的。

A 扩展包括 Zaamo 扩展和 Zalrsc 扩展所提供的指令。

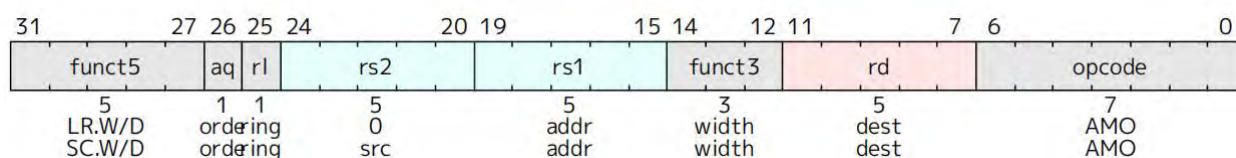
14.1 用于确定访问顺序的原子指令

基准 RISC-V ISA 采用弱内存模型，使用 FENCE 指令来施加额外的顺序约束。地址空间由执行环境划分为内存域和 I/O 域，FENCE 指令允许对这两个地址域的访问进行顺序控制。

为了更高效地支持释放一致性，每条原子指令都配备了 aq 和 rl 位，用于在其他 RISC-V 硬件线程的视角下，指定额外的内存顺序约束。这两位用于控制原子指令所访问的地址域（内存或 I/O）的访问顺序。对于访问另一域的操作，则不施加任何顺序约束。若需要在不同域之间强制顺序，则应使用 FENCE 指令。

如果两个位都未设置，则不会对原子内存操作施加额外的顺序约束。如果仅设置了 aq 位，则原子内存操作被视为获取（acquire）访问，即在此 RISC-V 硬件线程上，任何后续内存操作都不能被观察到发生在获取内存操作之前。如果仅设置了 rl 位，则原子内存操作被视为释放（release）访问，即释放内存操作不能被观察到发生在此 RISC-V 硬件线程上的任何先前内存操作之前。如果同时设置了 aq 和 rl 位，则原子内存操作是顺序一致（sequentially consistent）的，并且不能被观察到发生在任何先前内存操作之前或任何后续内存操作之后，且作用于相同地址域的内存操作。

14.2 Zalrsc 扩展：用于 Load-Reserved/Store-Conditional 指令



对单个内存字或双字的复杂原子内存操作，RISC-V 提供了加载预留（LR）和条件存储（SC）指令来实现。LR.W 从 rs1 指定的内存地址加载一个字，将符号扩展后的值放入 rd，并注册一个预留集——该集合包含了被寻址字所覆盖的所有字节。SC.W 条件性地将 rs2 中的字写入 rs1 中的地址：只有当预留仍然有效且预留集包含正在写入的字节时，SC.W 才会成功。如果 SC.W 成功，指令将 rs2 中的字写入内存，并将零写入 rd。如果 SC.W 失败，指令不写入内存，并将非零值写入 rd。出于保护内存的考虑，执行失败的 SC.W 指令可以被视为一次存储操作。无论执行成功与否，执行 SC.W 指令都会使该硬件线程持有的任何预留失效。LR.D 和 SC.D 对双字执行类似操作，但仅在 RV64 上可用。对于 RV64，LR.W 和 SC.W 指令均会对其置入 rd 寄存器的值进行符号扩展。



比较并交换（CAS）和 LR/SC 都可以用来构建无锁数据结构。经过广泛讨论，我们选择了 LR/SC，原因有以下几点：1）CAS 存在 ABA 问题，而 LR/SC 通过监控对地址的所有写入来避免这个问题，而不仅仅检查数据值的变化；2）CAS 还需要一种新的整数指令格式来支持三个源操作数（地址、比较值、交换值），以及不同的内存系统消息格式，这会使微架构变得更加复杂；3）此外，为了避免 ABA 问题，其他系统提供了双字 CAS（DW-CAS），允许在测试和增加计数器的同时操作数据字。这需要在一条指令中读取五个寄存器并写入两个寄存器，还需要一个新的更大的内存系统消息类型，进一步增加了实现的复杂性；4）LR/SC 提供了更高效的许多原语的实现，因为它只需要一个加载，而 CAS 需要两个加载（在 CAS 指令前进行一次加载以获取用于投机计算的值，然后在 CAS 指令中进行第二次加载，检查值是否未更改，然后再进行更新）。

LR/SC 相比于 CAS 的主要缺点是活锁（livelock），我们在特定情况下通过架构层面保证最终的持续进展来避免活锁，具体内容如下。另一个问题是当前 x86 架构的影响，尤其是其对双字 DW-CAS 的依赖，是否会使同步库和假定 DW-CAS 为基本机器原语的其他软件的移植工作变得复杂。一个可能的缓解因素是最近 x86 添加了事务性内存指令，这可能会导致从 DW-CAS 向其他方法的转变。

一般来说，多字原子操作是理想的，但关于其应采取何种形式仍存在相当大的争议，而且保证持续进

展会增加系统的复杂性。

值为 1 的错误代码表示 UNSPECIFIED 的错误。目前，其他错误代码被保留。可移植软件应仅假设错误代码将是非零值。



我们保留了错误代码 1 表示 UNSPECIFIED，以便简单的实现可以使用现有的多路复用器（mux）返回该值，该多路复用器是 SLT/SLTU 指令所需的。未来版本或 ISA 扩展中可能会定义更具体的错误代码。

对于 LR 和 SC，Zalrsc 扩展要求 rs1 中的地址与操作数的大小自然对齐（即，对于双字，需要 8 字节对齐，对于字，需要 4 字节对齐）。如果地址没有自然对齐，将会生成地址未对齐异常或访问故障异常。如果内存访问因对齐问题无法完成，而该未对齐的访问不应被模拟，则会生成访问故障异常。



在大多数系统中，模拟未对齐的 LR/SC 序列是不可行的。

未对齐的 LR/SC 序列还会带来同时访问多个保留集的可能性，而目前的定义并未考虑这种情况。

实现可以在每个 LR 上注册一个任意大的保留集，前提是保留集包含所访问的数据字或双字的所有字节。SC 只能与程序顺序中的最近 LR 配对。SC 只有在以下条件下才可能成功：没有观察到来自其他硬件线程的存储操作发生在 LR 和 SC 之间，并且在 LR 和 SC 之间没有其他 SC 指令。SC 只有在以下条件下才可能成功：没有观察到来自硬件线程以外的设备对 LR 指令访问的字节的写入发生在 LR 和 SC 之间。请注意，这个 LR 可能具有不同的有效地址和数据大小，但仍然将 SC 的地址作为保留集的一部分进行保留。



遵循此模型，在具有内存地址转换的系统中，如果先前执行的 LR 指令使用具有不同虚拟地址的别名保留了相同的位置，则 SC 允许成功；但如果虚拟地址不同，SC 也允许失败。

为了兼容旧设备和总线，除了 RISC-V 硬件线程之外的设备写入仅在它们访问的字节与 LR 访问的字节重叠时才需要使保留无效。当它们访问保留集中的其他字节时，这些写入不需要使保留无效。

如果目标地址不在程序顺序中最近的 LR 的预留集内，则 SC 必须执行失败。如果在 LR 和 SC 指令之间，可以观测到来自另一硬件线程对预留集的存储操作，SC 指令必须执行失败。如果在 LR 和 SC 指令之间，可以观测到来自其他设备对 LR 指令访问字节的写操作，SC 指令必须执行失败。（如果该设备写入了预留集，但并未写入 LR 指令访问的字节，则 SC 可能成功，也可能失败。）如果在程序顺序中，LR 和 SC 指令之间存在另一条 SC 指令（无论目标地址为何），SC 指令必须执行失败。成功完成的 LR/SC 指令序列的原子性需求，其精确定义见第 17.1 章中的原子性公理（Atomicity Axiom）。

平台应提供一种机制来确定预留集的大小和形状。

平台规范可能会限制预留集的大小和形状。

应使用 SC 指令对内存中的空闲字进行操作，以强制撤销任何现有的加载预留：



- * 在抢占式上下文切换时

- * 必要时，在更改虚拟地址到物理地址的映射时，例如，当迁移可能包含活动预留的页面时。

当硬件线程执行 LR 或 SC 指令时，预留的无效化意味着硬件线程一次只能持有一个预留，并且 SC 只能与最近的 LR 配对，LR 只能与紧随其后的 SC 配对，按程序顺序排列。这是对第 17.1 章中原子性公理（Atomicity Axiom）的一项约束，旨在确保软件在符合手册预期的常见实现上能够正确运行

在另一个 RISC-V 硬件线程观察到 SC 指令之前，必须先执行建立预留的 LR 指令。



通过在 LR 指令上设置 `aq` 位，可以使 LR/SC 序列具有获取语义 (acquire semantics)。通过在 SC 指令上设置 `rl` 位，可以使 LR/SC 序列具有释放语义 (release semantics)。假设其他原子操作有合适的映射，在 LR 指令上设置 `aq` 位，并在 SC 指令上设置 `rl` 位，使得 LR/SC 序列在 Cpp 的 `memory_order_seq_cst` 意义下是顺序一致的 (sequentially consistent)。这样的序列不会充当屏障 (fence) 来排序序列前后普通的加载和存储指令。对于其他 Cpp 原子操作的具体指令映射，或者更强的顺序一致性概念，可能要求在 LR 或 SC 指令的任意一条或两条上都设置这两个位。

如果在 LR 或 SC 上都没有设置 `aq` 和 `rl` 位，则可以观测到 LR/SC 序列与同一 RISC-V 硬件线程发出的周围内存操作，在执行顺序上互不约束。当 LR/SC 序列用于实现并行归约操作时，这是一种合理的选择。

软件不应在 LR 指令上设置 `rl` 位，除非同时设置 `aq` 位；同样，软件也不应在 SC 指令上设置 `aq` 位，除非同时设置 `rl` 位。LR.`rl` 和 SC.`aq` 指令并不保证提供比两个位都未设置时更强的排序，这可能会导致性能降低。

[source, asm]

. 使用 LR/SC 的比较并交换函数示例代码。

```
# a0 寄存器存储内存地址
# a1 寄存器存储期望值
# a2 寄存器存储目标值
# a0 寄存器存储返回值, 0 表示成功, 非 0 表示失败

cas:
    lr.w t0, (a0)      # 加载初始值。
    bne t0, a1, fail    # 值不匹配, 因此失败。
    sc.w t0, a2, (a0)   # 尝试更新。
    bnez t0, cas        # 如果条件存储失败, 则重试。
    li a0, 0            # 设置返回值为成功。
    jr ra               # 返回。

fail:
    li a0, 1            # 设置返回值为失败。
    jr ra               # 返回。
```

LR/SC 可以用于构造无锁数据结构。一个使用 LR/SC 实现比较并交换 (compare-and-swap) 功能的示例显示在 `cas` 中。如果内联实现，比较并交换功能仅需四条指令。

14.3 存储条件指令的最终成功

`Zalrsc` 扩展定义了受限的 LR/SC 循环，其具有以下特点：

- * 循环仅包含一个 LR/SC 序列和在失败时重试该序列的代码，且最多只能包含 16 条指令，这些指令必须按顺序放置在内存中。

- * 一个 LR/SC 序列以 LR 指令开始，以 SC 指令结束。在 LR 和 SC 指令之间执行的动态代码只能包含基准 I 指令集中的指令，不包括加载、存储、向后跳转、被执行的向后分支、JALR、FENCE 和 SYSTEM 指令。如果支持 C 扩展，则也允许上述 I 指令的压缩形式。

- * 重试失败的 LR/SC 序列的代码可以包含向后跳转和/或分支以重复执行 LR/SC 序列，但除此之外，其约束与 LR 和 SC 之间的代码相同。

* LR 和 SC 的地址必须位于具有 LR/SC 最终性属性的内存区域内。执行环境负责通知哪些区域具有该属性。

* SC 必须针对相同的有效地址，并且数据大小与同一硬件线程执行的最新 LR 相同。

不位于受限 LR/SC 循环中的 LR/SC 序列是不受约束的。不受约束的 LR/SC 序列可能在某些实现上某些尝试成功，但在其他实现上可能永远不会成功。



我们将 LR/SC 循环的长度限制为在基础 ISA 中适配 64 字节连续指令，以避免对指令缓存和 TLB 大小及关联性的过度限制。同样，我们禁止在循环内执行其他加载和存储操作，以避免对简单实现中私有缓存中跟踪保留集的数据缓存关联性产生限制。对分支和跳转的限制则限制了序列中可以花费的时间。为了简化操作系统对缺乏适当硬件支持的实现的指令仿真，浮点运算和整数乘法/除法被禁止。

软件并不禁止使用不受限的 LR/SC 序列，但可移植的软件必须检测到该序列反复失败的情况，然后回退到不依赖于不受约束的 LR/SC 序列的替代代码序列。实现允许在任何不受限的 LR/SC 序列上无条件失败。

如果一个硬件线程 H 进入一个受限的 LR/SC 循环，执行环境必须保证最终发生以下事件之一：

* H 或其他硬件线程执行一个成功的 SC 指令，且该 SC 指令操作的是 H 的受限 LR/SC 循环中的预留集。

* 其他硬件线程执行一个无条件的存储或原子内存操作（AMO）指令，且该指令操作的是 H 的受限 LR/SC 循环中的预留集合，或者系统中的其他设备向该预留集写入数据。

* H 执行一个跳转或分支，退出受限 LR/SC 循环。

* H 触发陷阱。

请注意，这些定义允许在实现中 SC 指令因任何原因偶尔失败，只要上述保证没有被违反。

由于要保证最终的结构，如果某些硬件线程在执行受限的 LR/SC 循环，并且在执行环境中没有其他硬件线程或设备对该保留集执行无条件存储或 AMO 操作，那么至少有一个硬件线程最终会退出其受限的 LR/SC 循环。相反，如果其他硬件线程或设备继续写入该保留集，则无法保证任何硬件线程会退出其 LR/SC 循环。



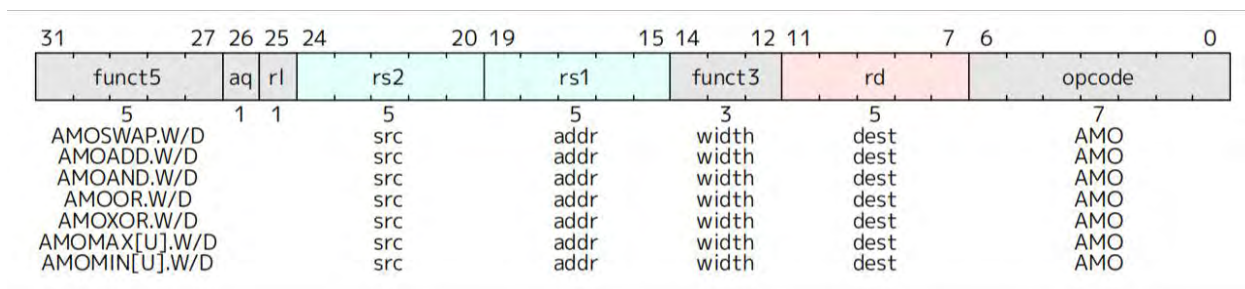
加载和 LR 指令本身不会妨碍其他硬件线程的 LR/SC 序列进展。请注意，这一约束意味着，加载和 LR 指令的执行（可能发生在同一个核内的其他硬件线程中）不会无限期地妨碍 LR/SC 的进展。例如，由于另一个硬件线程共享缓存而导致的缓存驱逐，不能无限期地妨碍 LR/SC 的进展。通常，这意味着预留操作会独立于任何共享缓存的驱逐进行跟踪。同样，由硬件线程内部的推测执行引起的缓存未命中，也不能无限期地妨碍 LR/SC 的进展。

这些定义允许 SC 指令因实现原因而偶尔虚假的发生了失败，只要最终能够继续进行下去。

CAS 的一个优势是，它保证某个硬件线程最终会取得进展，而 LR/SC 原子序列在某些系统上可能会无限期地发生活锁。为了避免这个问题，我们为某些 LR/SC 序列添加了活锁自由的架构保证。

早期版本的规范强制要求提供更严格的饥饿自由（starvation-freedom）保证。然而，较弱的无活锁（livelock-freedom）保证已足以支持 C11 和 C++11 语言的实现，并且在某些微架构风格中提供这一保证要简单得多。

14.4 Zaamo 扩展：原子内存操作



原子内存操作（AMO）指令执行用于多处理器同步的读-改-写操作，并采用 R 型指令格式编码。这些 AMO 指令会以原子方式从寄存器 rs1 指定的地址加载数据值，将该值存入寄存器 rd 中。随后，对刚加载的值与 rs2 中的初始值应用二进制运算符，并将运算结果存回 rs1 指定的初始内存地址。AMO 指令可以操作内存中的双字（仅限 RV64）或单字。对于 RV64，32 位 AMO 总是对其置入 rd 的值进行符号扩展，并忽略 rs2 初始值的高 32 位。

对于 AMO 指令，Zaamo 扩展要求寄存器 rs1 中保存的地址，必须按照操作数大小进行自然对齐（例如，双字需 8 字节对齐，单字需 4 字节对齐）。如果地址未按自然边界对齐，则会触发地址未对齐异常或访问故障异常。当内存访问原本可以成功完成，但因未对齐而无法完成，且该未对齐访问不应被模拟时，便会触发访问故障异常。

本手册第二卷定义的未对齐原子性粒度 PMA 机制，可选择性地放宽上述对对齐的要求。一旦启用 PMA，它将指定一个未对齐原子性粒度的大小，该粒度大小必须是 2 的幂次方字节数。未对齐原子性粒度 PMA 机制仅适用于以下指令集：AMO 指令、基本 ISA 中定义的加载和存储指令，以及 F、D 和 Q 扩展中定义的、操作数大小不超过 XLEN 位的加载和存储指令。对于属于上述指令集的指令，如果其所有访问的字节都落在同一个未对齐原子性粒度所覆盖的范围内，则该指令将不会因地址对齐问题而触发异常，并且在 RVWMO 内存模型下，该指令将仅产生一次内存操作——换言之，它将以原子方式执行。

支持的操作包括交换、整数加法、按位与、按位或、按位异或，以及有符号和无符号整数的最大值和最小值。在没有顺序约束的情况下，这些 AMO 操作可以用于实现并行归约操作，在这种情况下，返回值通常会通过写入 x0 来丢弃。

我们提供了取值并操作（fetch-and-op）风格的原子操作（AMO），因为它们比 LR/SC 或 CAS 更适用于高并发系统。一个简单的微架构可以通过使用 LR/SC 原语来实现 AMO，前提是该实现能够保证 AMO 最终完成。更复杂的实现可能还会在内存控制器中实现 AMO，并可以在目标是 x0 时优化掉获取初始值的操作。



AMO 的操作集被选择为高效支持 C11/C++11 的原子内存操作，并且能够支持内存中的并行归约操作。AMO 的另一个用途是在 I/O 空间内为内存映射的设备寄存器提供原子更新（例如，设置、清除或切换位）。

Zaamo 扩展使得微控制器类的实现能够利用 A 扩展中的 AMO 子集提供的原子原语。通常，这类实现没有缓存，因此可能无法自然地支持 Zalrsc 扩展提供的 LR/SC 指令。

为了帮助实现多处理器同步，AMO 可选地提供释放一致性语义。如果设置了 aq 位，则在此 RISC-V 内核中，任何后续的内存操作都不能在 AMO 之前被观察到。相反，如果设置了 rl 位，则其他 RISC-V 内核不会在此内核的 AMO 之前观察到 AMO，且 AMO 之前的内存访问会先于 AMO 被观察到。将 aq 和 rl 位都设置在 AMO 上，使得该序列具有顺序一致性，意味着它不能与同一内核的早期或晚期内存操作重排序。



AMO 被设计为高效地实现 C11 和 C++11 的内存模型。尽管 FENCE R、RW 指令足以实现 acquire 操作，FENCE RW、W 指令足以实现 release 操作，但与设置了相应 aq 或 rl 位的 AMO 相比，这两者都会引入额外的不必要的排序。

```

li          t0, 1          # 初始化交换值为 1。
again:
lw          t1, (a0)        # 检查锁是否被持有。
bnez        t1, again       # 如果已被持有，则重试。
amoswap.w.aq t1, t0, (a0)   # 尝试获取锁。
bnez        t1, again       # 如果已被持有，则重试。
# ...
# 临界区。
# ...
amoswap.w.rl x0, x0, (a0)   # 通过存储 0 来释放锁。

```



我们建议使用上面展示的 AMO Swap 范式来实现锁的获取和释放，以简化推测性锁消除（speculative lock elision）的实现。



A 扩展中的指令可以用于提供顺序一致性的加载和存储，但这会比必要的更严格地限制硬件对内存访问的重排序。C++ 顺序一致性的加载可以通过设置 aq 位的 LR 指令来实现。然而，LR/SC 的最终成功保证可能会减慢来自相同有效地址的并发加载。顺序一致性的存储可以通过一个 AMOSWAP 指令来实现，该指令将旧值写入 x0 并设置 rl 位。然而，不必要的加载可能会引入不必要的排序约束，在这个例子中并不需要。特定的编译约定可能要求在 LR 和 AMOSWAP 指令中设置 aq 和 rl 位，或者两者都设置。

15 Zawrs 扩展：等待预留集指令，V1.01

Zawrs 扩展定义了一对用于轮询的指令，进入低功耗的暂停执行状态并等待对内存位置的存储操作。等待某一内存的数据被更新是很常见的情况，例如：

- 某个尝试获取锁的线程正在等待锁变量的更新。
- 消费者正在等待生产者将工作/数据加入一个空的等待队列。在 RISC-V 硬件线程中，生产者可能是硬件线程上正在执行的代码，加速器设备或者外部 I/O 代理等。
- 代码正在等待内存中的标志位被设置，以指示某个事件的发生。例如，RISC-V 处理器硬件线程上的软件可能等待加速器设备在内存中设置完成标志，以指示之前提交给设备的作业已完成。

这些例子涉及对内存位置的轮询，在这些情况下采用忙等待循环可能会浪费大量能源。为了减轻此类用法中的浪费循环，提供了 WRS.NTO（无超时的 WRS）指令。与轮询特定内存位置的存储不同，软件会使用 LR 指令注册一个包含某一内存位置所有字节的预留集。然后，WRS.NTO 指令会使得硬件线程暂时停止执行并进入低功耗状态，直到发生对预留集的存储操作或观察到中断。

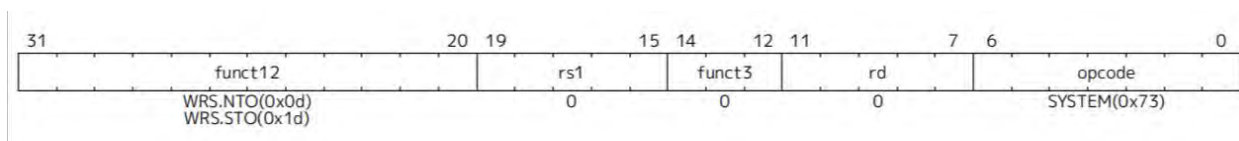
有时，等待内存更新的程序可能还需要在未来执行任务或对等待时间设置上限。为了支持这类情况，该扩展还提供了另一条指令 WRS.STO（带短超时的 WRS），其工作方式类似于 WRS.NTO，但将停顿持续时间限制在一个由指令具体实现定义的短超时时间，使得如果没有其他终止停顿的条件，停顿将在超时时终止。使用此指令的程序可以随后确定是否已达到其截止时间。



Zawrs 扩展中的指令只有在配合 LR 指令时才能起作用，后者由 A 扩展的 Zalrsc 部分提供。

15.1 等待预订设置说明

WRS.NTO 和 WRS.STO 指令会使得硬件线程在预留集有效且未观察到待处理中断（即使是中断已经被禁用）的情况下，暂时停止执行并进入低功耗状态。对于 WRS.STO 指令而言，停顿持续时间被一个由指令具体的实现定义的短超时时间限制。这些指令在所有特权模式下都可用。这些指令在受限的 LR/SC 循环中不被支持。



当以下所有条件同时成立时，硬件线程的执行可能进入暂停状态：

- 预留集有效
- 对于 WRS.STO 而言，从停顿到现在尚未超过短超时时间的限制
- 未观察到待处理的中断（具体规则见下）

在停顿期间，允许实现偶尔终止停顿，并因任何原因完成执行。

WRS.NTO 和 WRS.STO 指令遵循 WFI 指令的规则，当有挂起的中断时，恢复执行。当 mstatus 中的 TW（超时等待）位为 1，而 WRS.NTO 正在 M 模式以外的任何特权模式下执行，并且在实现所特别规定的有界时间限制内未完成，WRS.NTO 指令将导致非法指令异常。

在 VS 或 VU 模式下执行时，如果 hstatus 寄存器中的 VTW 位被设置，mstatus 寄存器中的 TW 位值为 0，且 WRS.NTO 在实现特定的有界时间限制内未完成，WRS.NTO 指令将导致虚拟指令异常。

由于 WRS.STO 和 WRS.NTO 指令可能由于除预留集存储以外的原因完成执行，软件可能需要一种循环方法，直到所需的存储操作已发生。



WRS.STO 指令超时的持续时间可能在不同实现中以及同一实现的不同情况下有显著变化。在典型的实现中，这个持续时间应该大致处于片上缓存未命中延迟或无缓存访问主内存延迟的 10 到 100 倍范围。

与 WFI 指令不同，即使在 U 模式下执行且对应的 TW 位为 0 时，WRS.NTO 指令也不会被定义为引发非法指令异常。WFI 指令通常不适用于 U 模式，在许多系统中，于 U 模式下使用 WFI 指令可能会立即导致非法指令异常。与 WFI 不同，WRS.NTO 更适用于软件在 U 模式下进行无截止时间的内存等待。

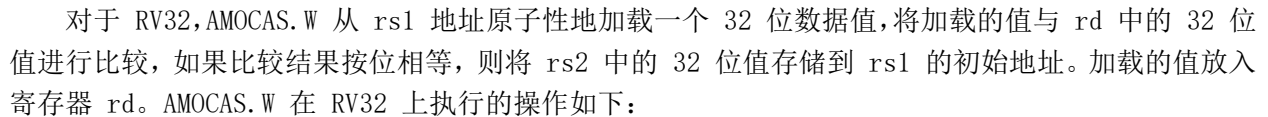
16 Zacas 扩展：用于原子比较并交换（CAS）指令，V1.0.0

16.1 介绍

比较并交换（CAS）指令提供了一种简单且通常更快的方式来执行线程同步操作，当作为硬件指令支持时。CAS 通常用于无锁（lock-free）和无等待算法（wait-free algorithms）中。本扩展提议 CAS 指令操作 32 位、64 位和 128 位（仅限 RV64）数据值。CAS 指令支持 C++11 原子比较和交换操作。

虽然可以使用 LR/SC 实现 XLEN 宽数据的比较并交换，但 CAS 原子指令在高度并行系统中比 LR/SC 更具扩展性。许多无锁算法，例如无锁队列，需要操作指针变量。简单的 CAS 操作可能不足以

16.2 字/双字/四字 CAS (AMOCAS.W/D/Q)



AMOCAS.D 类似于 AMOCAS.W，但操作 64 位数据值。对于 RV32，AMOCAS.D 从 rs1 地址原子性地加载 64 位数据值，将加载的值与由 rd 和 rd+1 组成的寄存器对中的 64 位值进行比较，如果比较结果按位相等，则将 rs2 和 rs2+1 寄存器对中的 64 位值存储到 rs1 初始地址。加载的值放入寄存器对 rd 和 rd+1。指令要求寄存器对中的第一个寄存器为偶数；在 rs2 和 rd 中指定奇数寄存器的编码是保留的。当源寄存器对中的第一个寄存器为 x0 时，寄存器对的两半都读取为零。当目标寄存器对中的第一个寄存器为 x0 时，整个寄存器结果被丢弃，且不写入任何目标寄存器。AMOCAS.D 在 RV32 上执行的操作如下：

53


```

temp[31:0] = mem[X(rs1)]
if ( temp[31:0] == X(rd)[31:0] )
    mem[X(rs1)] = X(rs2)[31:0]
X(rd) = SignExtend(temp[31:0])

```

对于 RV64, AMOCAS.D 从 rs1 地址原子性地加载 64 位数据值, 将加载的值与 rd 中的 64 位值进行比较, 如果比较结果按位相等, 则将 rs2 中的 64 位值存储到 rs1 的初始地址。加载的值放入寄存器 rd。AMOCAS.D 在 RV64 上执行的操作如下:

```

temp = mem[X(rs1)]
if ( temp == X(rd) )
    mem[X(rs1)] = X(rs2)
X(rd) = temp

```

AMOCAS.Q (仅 RV64) 从 rs1 地址原子性地加载 128 位数据值, 将加载的值与由 rd 和 rd+1 组成的寄存器对中的 128 位值进行比较, 如果比较结果按位相等, 则将 rs2 和 rs2+1 寄存器对中的 128 位值存储到 rs1 的初始地址。加载的值放入寄存器对 rd 和 rd+1。指令要求寄存器对中的第一个寄存器为偶数; 在 rs2 和 rd 中指定奇数寄存器的编码是保留的。当源寄存器对中的第一个寄存器为 x0 时, 寄存器对的两半都读取为零。当目标寄存器对中的第一个寄存器为 x0 时, 整个寄存器结果被丢弃, 且不写入任何目标寄存器。AMOCAS.Q 执行的操作如下:

[listing]

```

temp0 = mem[X(rs1)+0]
temp1 = mem[X(rs1)+8]
comp0 = (rd == x0) ? 0 : X(rd)
comp1 = (rd == x0) ? 0 : X(rd+1)
swap0 = (rs2 == x0) ? 0 : X(rs2)
swap1 = (rs2 == x0) ? 0 : X(rs2+1)
if ( temp0 == comp0 ) && ( temp1 == comp1 )
    mem[X(rs1)+0] = swap0
    mem[X(rs1)+8] = swap1
endif
if ( rd != x0 )
    X(rd) = temp0
    X(rd+1) = temp1
Endif

```

对于未来的 RV128 扩展, AMOCAS.Q 将在 rs2 和 rd 中编码一个 XLEN=128 的单寄存器。



一些算法可能会将内存位置的先前数据值加载到用于 Zacas 指令的比较数据值源寄存器中。当使用 Zacas 指令时, 使用寄存器对来提供比较值, 可以使用两个单独的加载来加载这两个寄存器。两个单独的加载可能会读取不一致的值对, 但这不是问题, 因为 AMOCAS 操作本身使用原子加载对从内存中获取数据值进行比较。

以下示例代码序列说明了在 RV32 实现中使用 AMOCAS.D 原子性地递增 64 位计数器。



```
# a0 - address of the counter.
increment:
    lw  a2, (a0)      # Load current counter value using
    lw  a3, 4(a0)     # two individual loads.
retry:
    mv  a6, a2        # Save the low 32 bits of the current value.
    mv  a7, a3        # Save the high 32 bits of the current value.
    addi a4, a2, 1     # Increment the low 32 bits.
    sltu a1, a4, a2    # Determine if there is a carry out.
    add  a5, a3, a1    # Add the carry if any to high 32 bits.
    amocas.d.aqrl a2, a4, (a0)
    bne  a2, a6, retry # If amocas.d failed then retry
    bne  a3, a7, retry # using current values loaded by amocas.d.
    ret
```

与 A 扩展中的 AMO 一样，AMOCAS.W/D/Q 要求 rs1 中的地址自然对齐到操作数的大小（即，四字对齐为 16 字节，双字对齐为 8 字节，字对齐为 4 字节）。如果地址未自然对齐，则适用相同的异常选项。

与 A 扩展中的 AMO 一样，AMOCAS.W/D/Q 可选地提供释放一致性语义，使用 aq 和 rl 位，以帮助实现多处理器同步。当 aq 位为 1 时，AMOCAS.W/D/Q 成功执行的内存操作具有获取语义；当 rl 位为 1 时，具有释放语义。当 AMOCAS.W/D/Q 不成功时，内存操作具有获取语义（如果 aq 位为 1），但不具有释放语义（无论 rl 位如何）。

可以使用 FENCE 指令来排序 AMOCAS.W/D/Q 指令产生的内存读访问和（如果生成的）内存写访问。



不成功的 AMOCAS.W/D/Q 可以不执行内存写入，也可以写回从内存加载的旧值。如果产生内存写入，则不具有释放语义（无论 rl 位如何）。

AMOCAS.W/D/Q 指令始终需要写权限。



以下示例代码序列说明了使用 AMOCAS.Q 实现无阻塞并发队列的 enqueue 操作。该算法使用 AMOCAS.Q 指令原子性地操作指针及其关联的修改计数器（modification counter），以避免 ABA 问题。

```

# Enqueue operation of a non-blocking concurrent queue.
# Data structures used by the queue:
#  structure pointer_t {ptr:  node_t *, count: uint64_t}
#  structure node_t   {next: pointer_t, value: data type}
#  structure queue_t  {Head: pointer_t, Tail: pointer_t}
# Inputs to the procedure:
#  a0 - address of Tail variable
#  a4 - address of a new node to insert at tail
enqueue:
    ld  a6, (a0)          # a6 = Tail.ptr
    ld  a7, 8(a0)         # a7 = Tail.count
    ld  a2, (a6)          # a2 = Tail.ptr->next.ptr
    ld  a3, 8(a6)         # a3 = Tail.ptr->next.count
    ld  t1, (a0)
    ld  t2, 8(a0)
    bne a6, t1, enqueue   # Retry if Tail & next are not consistent
    bne a7, t2, enqueue   # Retry if Tail & next are not consistent
    bne a2, x0, move_tail # Was tail pointing to the last node?
    mv  t1, a2            # Save Tail.ptr->next.ptr
    mv  t2, a3            # Save Tail.ptr->next.count
    addi a5, a3, 1        # Link the node at the end of the list
    amocas.q.aqrl a2, a4, (a6)
    bne a2, t1, enqueue   # Retry if CAS failed
    bne a3, t2, enqueue   # Retry if CAS failed
    addi a5, a7, 1        # Update Tail to the inserted node
    amocas.q.aqrl a6, a4, (a0)
    ret                  # Enqueue done
move_tail:
    addi a3, a7, 1        # Try to swing Tail to the next node
    amocas.q.aqrl a6, a2, (a0)
    j    enqueue          # Retry

```

16.3 额外的 AMO PMA

A 扩展中定义了四个级别的 AMO 支持。Zacas 定义了三个额外的支持级别：AMOCASW、AMOCASD 和 AMOCASQ。

AMOCASW 表示除了 AMOArithmetic 级别支持的指令外，还支持 AMOCAS.W 指令。AMOCASD 表示除了 AMOCASW 级别支持的指令外，还支持 AMOCAS.D 指令。AMOCASQ 表示除了 AMOCASD 级别支持的指令外，还支持 AMOCAS.Q 指令。



AMOCASW/D/Q 需要 AMOArithmetic 级别支持，因为 AMOCAS.W/D/Q 指令需要执行算术比较和交换操作的能力。

17 RVWMO 内存一致性模型，V2.0

本章定义了 RISC-V 内存一致性模型。内存一致性模型是一系列规则，用于规范内存加载操作可能返回的值。RISC-V 采用了一种名为 RVWMO（RISC-V 弱内存序）的内存模型。RVWMO 的设计目标是为架构师提供具有高性能、可扩展设计的灵活性，并同时兼顾易于理解和使用的编程模型。

在 RVWMO 下，单个硬件线程上运行的代码从同一硬件线程的其他内存指令的角度来看，似乎是按顺序执行的，但来自其他硬件线程的内存指令可能会以不同的顺序观察到第一个硬件线程的内存指令。因此，多线程代码可能需要显式的同步机制，以确保来自不同硬件线程的内存指令之间的顺序。基础的

RISC-V ISA 提供了一个 FENCE 指令用于此目的，具体描述见 第 2.7 章，而原子扩展 A 还定义了加载保留/存储条件和原子读-修改-写指令。

用于总存储排序的标准 ISA 扩展 Ztso (第 18 章) 通过特定于这些扩展的附加规则增强了 RVWMO。本规范的附录提供了内存一致性模型的公理和操作形式化 (formalizations) 以及其他解释材料。



本章定义了常规主内存操作的内存模型。内存模型与 I/O 内存、指令获取、FENCE.I、页表遍历和 SFENCE.VMA 的交互尚未形式化。上述部分或全部内容可能会在本规范的未来修订版中形式化。RV128 基础 ISA 和未来的 ISA 扩展 (如 V 向量和 J JIT 扩展) 将需要纳入未来的修订版。

支持不同宽度重叠内存访问的内存一致性模型仍然是学术研究的活跃领域，尚未被完全理解。不同大小的内存访问在 RVWMO 下如何交互的具体细节已根据我们当前的能力进行了规定，但如果发现新问题，可能会进行修订。

17.1 RVWMO 内存模型的定义

RVWMO 内存模型是根据全局内存顺序定义的，这是所有硬件线程产生的内存操作的总顺序。通常，多线程程序有许多不同的可能执行，每个执行都有其对应的全局内存顺序。

全局内存顺序是根据内存指令生成的初始加载和存储操作定义的。然后，它受到本章其余部分定义的约束。任何满足所有内存模型约束的执行都是合法执行 (就内存模型而言)。

17.1.1 内存模型原语

内存操作的程序顺序反映了指令在其所在硬件线程的动态指令流中的逻辑排列次序，这些指令负责生成每一次加载和存储操作。换句话说，程序顺序就是在一个简单的顺序处理器上硬件线程执行指令的次序。

内存访问指令会产生内存操作。内存操作可以是加载操作、存储操作，或者同时包含两者。所有内存操作都具备单副本原子性：它们永远不会被观察到处于部分完成的状态。

在 RV32GC 和 RV64GC 指令集中，通常每个对齐的内存指令都会产生且仅产生一个内存操作，但存在以下两个例外情况：第一，执行未成功的 SC 指令不会产生任何内存操作；第二，如果 XLEN 小于 64 位，FLD 和 FSD 指令可能会产生多个内存操作，具体细节如第 21.3 章章节所述，并在下文进一步阐明。对于对齐的 AMO 指令，它会产生单个内存操作，该操作同时包含加载和存储两种行为。



RV128 基础指令集和未来的 ISA 扩展 (如 *V* (vector) 和 *P* (SIMD)) 中的指令可能会产生多个内存操作。然而，这些扩展的内存模型尚未形式化。

未对齐的加载或存储指令可以分解为任意粒度的组件内存操作。如果 XLEN 小于 64，则 FLD 或 FSD 指令也可以分解为任意粒度的组件内存操作。这些指令生成的内存操作在程序顺序中彼此之间没有顺序，但它们在程序顺序中与前面的和后续的指令生成的内存操作之间有正常的顺序。原子扩展 A 不要求执行环境支持未对齐的原子指令。然而，如果通过未对齐原子粒度 PMA 支持未对齐原子，则 AMO 在原子粒度内不分解，基础 ISA 中定义的加载和存储也不分解，F、D 和 Q 扩展中定义的不超过 XLEN 位的加载和存储也不分解。



将未对齐内存操作分解到字节粒度有助于在非原生支持非对齐访问的实现上进行模拟。例如，这些实现可能只是逐字节迭代未对齐访问。

LR 指令和 SC 指令如果在程序顺序中 LR 先于 SC 且中间没有其他 LR 或 SC 指令,则称为配对;相应的内存操作也称为配对(除非 SC 失败,在这种情况下不会生成存储操作)。确定 SC 必须成功、可能成功或必须失败的完整条件列表在第 14.2 章 中定义。

加载和存储操作还可以携带以下集合中的一个或多个排序注释: acquire-RCpc、acquire-RCsc、release-RCpc 和 release-RCsc。设置了 `aq` 的 AMO 或 LR 指令具有 acquire-RCsc 注释。设置了 `rl` 的 AMO 或 SC 指令具有 release-RCsc 注释。设置了 `aq` 和 `rl` 的 AMO、LR 或 SC 指令同时具有 acquire-RCsc 和 release-RCsc 注释。

为了方便起见,我们使用术语 `acquire` 注释 来指代 acquire-RCpc 注释或 acquire-RCsc 注释。同样, `release` 注释 指代 release-RCpc 注释或 release-RCsc 注释。RCpc 注释 指代 acquire-RCpc 注释或 release-RCpc 注释。RCsc 注释 指代 acquire-RCsc 注释或 release-RCsc 注释。

在内存模型文献中,术语 RCpc 代表具有处理器一致同步操作 (processor-consistent synchronization operations) 的释放一致性 (release consistency),术语 RCsc 代表具有顺序一致同步操作的释放一致性。



虽然在文献中有许多不同的获取和释放注释定义,但在 RVWMO 的上下文中,这些术语由 ppo, 保留程序顺序 规则 5-7 进行简洁且完全地定义。

RCpc 注释目前仅在标准扩展 Ztso (第18章) 中隐式分配给每个内存访问时使用。此外,尽管 ISA 当前不包含原生加载-获取或存储-释放指令,也不包含其 RCpc 变体,但 RVWMO 模型本身已经设计为向前兼容,以便未来可以在扩展中加入上述任何或所有内容。

17.1.2 语法依赖性

RVWMO 内存模型的定义部分依赖于语法依赖性的概念,定义如下。

在定义依赖性的上下文中,寄存器指的是整个通用寄存器、CSR 的某些部分或整个 CSR。通过 CSR 跟踪依赖性的粒度特定于每个 CSR,并在第 17.2 章 中定义。

语法依赖性的定义基于指令的源寄存器、指令的目标寄存器,以及指令如何从其源寄存器传递依赖到目标寄存器的方式。本节提供了所有这些术语的通用定义;然而,第 17.3 章提供了每个指令的具体细节的完整列表。

通常,如果满足以下任一条件,则寄存器 `r` (除 `x0` 外)是指令 `i` 的源寄存器:

- * 在 `i` 的操作码中, `rs1`、`rs2` 或 `rs3` 设置为 `r`

- * `i` 是 CSR 指令,并且在 `i` 的操作码中, `csr` 设置为 `r`,除非 `i` 是 CSRRW 或 CSRRWI 且 `rd` 设置为 `x0`

- * `r` 是 CSR 并且是 `i` 的隐式源寄存器,如第 17.3 章 中定义

- * `r` 是与 `i` 的另一个源寄存器别名的 CSR

内存指令还进一步指定哪些源寄存器是地址源寄存器,哪些是数据源寄存器。

通常,如果满足以下任一条件,则寄存器 `r` (除 `x0` 外)是指令 `i` 的目标寄存器:

- * 在 `i` 的操作码中, `rd` 设置为 `r`

- * `i` 是 CSR 指令,并且在 `i` 的操作码中, `csr` 设置为 `r`,除非 `i` 是 CSRRS 或 CSRRCI 且 `rs1` 设置为 `x0`,或 `i` 是 CSRRSI 或 CSRRCI 且 `uimm[4:0]` 设置为零。

- * `r` 是 `i` 的隐式目标寄存器的 CSR,如第 17.3 章 中定义

- * `r` 是与 `i` 的另一个目标寄存器别名的 CSR

大多数非内存指令携带依赖性从其每个源寄存器到其每个目标寄存器。然而,有一些例外;请参见第 17.3 章。

如果满足以下任一条件,则指令 j 通过 i 的目标寄存器 s 和 j 的源寄存器 r 对指令 i 具有语法依赖性:

- * s 与 r 相同,并且在程序顺序中没有指令在 i 和 j 之间将 r 作为目标寄存器
- * 在程序顺序中 i 和 j 之间存在指令 m , 满足以下所有条件:
 - . j 通过目标寄存器 q 和源寄存器 r 对 m 具有语法依赖性
 - . m 通过目标寄存器 s 和源寄存器 p 对 i 具有语法依赖性
 - . m 携带依赖性从 p 到 q

最后,在以下定义中,令 a 和 b 为两个内存操作, i 和 j 为生成 a 和 b 的指令。

如果 r 是 j 的地址源寄存器,并且 j 通过源寄存器 r 对 i 具有语法依赖性,则 b 对 a 具有语法地址依赖性

如果 b 是存储操作, r 是 j 的数据源寄存器,并且 j 通过源寄存器 r 对 i 具有语法依赖性,则 b 对 a 具有语法数据依赖性

如果在程序顺序中 i 和 j 之间存在指令 m , 并且 m 是分支或间接跳转, 并且 m 对 i 具有语法依赖性, 则 b 对 a 具有语法控制依赖性



一般来说,非 AMO 加载指令没有数据源寄存器,无条件非 AMO 存储指令没有目标寄存器。然而,成功的 SC 指令被认为在 rd 中指定的寄存器是目标寄存器,因此可能存在指令对程序顺序中先于它的成功 SC 指令具有语法依赖性。

17.1.3 保留程序顺序

任何给定程序执行的全局内存顺序尊重每个硬件线程的程序顺序的一部分,但不是全部。全局内存顺序必须遵守的程序顺序子集,被称为保留程序顺序。

保留程序顺序的完整定义如下(注意,AMO 同时是加载和存储):如果 a 在程序顺序中先于 b , a 和 b 都访问常规主内存(而不是 I/O 区域),并且满足以下任一条件,则内存操作 a 在保留程序顺序中先于内存操作 b (因此也在全局内存顺序中):

* 重叠地址排序:

- . b 是存储操作,并且 a 和 b 访问重叠的内存地址
- . a 和 b 均为加载操作,且都读取了字节 x ; 在程序顺序中, a 和 b 之间没有对字节 x 的存储操作; 并且 a 和 b 读取到的字节 x 的值是由不同的内存操作写入的。
- . a 由 AMO 或 SC 指令生成, b 是加载操作, 并且 b 返回由 a 写入的值

* 显式同步

- . 有一个 FENCE 指令将 a 排在 b 之前
- . a 具有获取注释
- . b 具有释放注释
- . a 和 b 都具有 RCsc 注释
- . a 与 b 配对

* 语法依赖性

- . b 对 a 具有语法地址依赖性
- . b 对 a 具有语法数据依赖性
- . b 是存储操作, 并且 b 对 a 具有语法控制依赖性

* 管道依赖性

- . b 是加载操作, 并且在程序顺序中 a 和 b 之间存在一些存储 m , m 对 a 具有地址或数据依赖性, 并且 b 返回由 m 写入的值
- . b 是存储操作, 并且在程序顺序中 a 和 b 之间存在一些指令 m , m 对 a 具有地址依赖性

17.1.4 内存模型公理

只有当存在一个全局内存顺序，既符合保留程序顺序，又满足加载值公理、原子性公理和进展公理时，RISC-V 程序的执行才算遵守 RVWMO 内存一致性模型。

加载值公理

- 每个加载 i 的每个字节返回由以下存储写入该字节的值，这些存储在全局内存顺序中是最新的：
- 在全局内存顺序中，写入该字节并且位于 i 之前的存储
 - 在程序顺序中，写入该字节并且位于 i 之前的存储

原子性公理

如果 r 和 w 是由硬件线程 h 中对齐的 LR 和 SC 指令生成的配对加载和存储操作， s 是对字节 x 的存储，并且 r 返回由 s 写入的值，则 s 必须在全局内存顺序中先于 w ，并且在全局内存顺序中 s 和 w 之间不能有来自 h 以外的硬件线程对字节 x 的存储。



ax-atom, 原子性公理 理论上支持不同宽度和不匹配地址的 LR/SC 对，因为实现允许 SC 操作在这种情况下成功。然而，实际上，我们预计这种模式很少见，并且不鼓励使用。

进展公理

全局内存顺序中，任何内存操作之前，都不能存在无限多的其他内存操作。

17.2 CSR 依赖性跟踪粒度

表 17.1 通过 csr 跟踪语法依赖关系的粒度

Name	Portions Tracked as Independent Units	Aliases
fflags	Bits4, 3, 2, 1, 0	fcsr
frm	entire CSR	fcsr
fcsr	Bits7-5, 4, 3, 2, 1, 0	fflags, frm

注：只读 CSR 未列出，因为它们不参与语法依赖性的定义。

17.3 源寄存器与目标寄存器列表

本节提供了每个指令的源寄存器和目标寄存器的具体列表。这些列表用于定义第 17.1.2 章中的语法依赖性。

术语 累积型 CSR (accumulating CSR) 用于描述既是源寄存器又是目标寄存器的 CSR，但它仅对自身存在依赖关系，即只有从自身到自身的依赖。

指令在 源寄存器 列中的每个源寄存器到 目标寄存器 列中的每个目标寄存器之间、从 源寄存器 列中的每个源寄存器到 累积 CSR 列中的每个 CSR 之间，以及从 累积 CSR 列中的每个 CSR 到其自身之间存在依赖关系，除非另有注释说明。

说明：

- $\hat{A}$ 地址源寄存器
- $\hat{D}$ 数据源寄存器
- $\dagger$ 指令不从任何源寄存器到任何目标寄存器携带依赖性
- $\ddagger$ 指令按指定从源寄存器携带依赖性到目标寄存器

表 17.2 RV32M 标准扩展

	源寄存器	目标寄存器	累积 CSR
MUL	rs1, rs2	rd	
MULH	rs1, rs2	rd	
MULHSU	rs1, rs2	rd	
MULHU	rs1, rs2	rd	
DIV	rs1, rs2	rd	
DIVU	rs1, rs2	rd	
REM	rs1, rs2	rd	
REMU	rs1, rs2	rd	

表 17.3 RV64M 标准扩展

	源寄存器	目标寄存器	累积 CSR
MULW	rs1, rs2	rd	
DIVW	rs1, rs2	rd	
DIVUW	rs1, rs2	rd	
REMW	rs1, rs2	rd	
REMUW	rs1, rs2	rd	

表 17.4 RV32A 标准扩展

	源寄存器	目标寄存器	累积 CSR	
LR. W+	rS1A	rd		
SC. W+	rs1 A. rs2 D	rd*		* if successful
AMOSWAP. W+	rs1 A. rs2 D	rd		
AMOADD. W+	rs1 A. rs2 D	rd		
AMOXOR. W+	rs1 A. rs2 D	rd		
AMOAND. W+	rs1 A. rs2 D	rd		
AMOOR. W+	rs1 A. rs2 D	rd		
AMOMIN. W+	rs1 A. rs2 D	rd		
AMOMAX. W+	rs1 A. rs2 D	rd		
AMOMINU. W+	rs1 A. rs2 D	rd		
AMOMAXU. W+	rs1 A. rs2 D	rd		

表 17.5 RV64A 标准扩展

	源寄存器	目标寄存器	累积 CSR	
LR. D+	rS7A	rd		
SC. D+	rs1 A. rs2 D	rd*		* if successful
AMOSWAP. D+	rs1 A. rs2 D	rd		
AMOADD. D+	rs1 A. rs2 D	rd		
AMOXOR. D+	rs1 A. rs2 D	rd		

AMOAND. D+	rs1 A.rs2 D	rd		
AMOOR. D+	rs1 A.rs2 D	rd		
AMOMIN. D+	rs1 A.rs2 D	rd		
AMOMAX. D+	rs1 A.rs2 D	rd		
AMOMINU. D+	rs1 A.rs2 D	rd		
AMOMAXU. D+	rs1 A.rs2 D	rd		

表 17.6 RV64F 标准扩展

	源寄存器	目标寄存器	累积 CSR	
FCVT. L. S	rs1, frm*	rd	NV, NX	* if rm=11
FCVT. LU. S	rs1, frm*	rd	NV, NX	* if rm=11
FCVT. S. L	rs1, frm*	rd	NX	* if rm=11
FCVT. S. LU	rs1, frm*	rd	NX	* if rm=11

18 Ztso 扩展：总存储排序扩展，V1.0

本节定义了用于 RISC-V 总存储序 (RVTSO) 内存一致性模型的 Ztso 扩展。RVTSO 被定义为 RVWMO 模型的增量式扩展，RVWMO 的具体定义见第 17.1 章。



Ztso 扩展旨在促进最初为 x86 或 SPARC 架构编写的代码的移植，这两种架构都默认采用 TSO 内存模型。此外，对于那些本质上就具备 RVTSO 特性，并希望向软件层面暴露这一特性的硬件实现，本扩展也提供了支持。

RVTSO 对 RVWMO 进行了以下调整：

- * 所有加载操作的行为，都如同带有 acquire-RCpc 注释。
- * 所有存储操作的行为，都如同带有 release-RCpc 注释。
- * 所有原子内存操作 (AMO) 的行为，都如同同时带有 acquire-RCsc 和 release-RCsc 注释。

这些规则使得除 4-7 之外的所有 PPO 规则都变得冗余。同样，任何未同时设置 PW 和 SR 标志的非 I/O fences 指令也变得不再必要。最后，这些规则还意味着，任何内存操作都不会与原子内存操作 (AMO) 指令发生重排序，无论重排序方向如何。



与 RVWMO 类似，在 RVTSO 内存模型中，存储顺序的约束也由 PPO 规则 5-7 简洁而完整地定义。在这两种内存模型下，第 17.4.1 章规则都允许硬件线程将数据从自身的存储缓冲区转发到后续的（程序顺序上的）加载操作。这意味着，存储操作可以在对其他硬件线程可见之前，先在本地被转发。

此外，如果实现了 Ztso 扩展，V 扩展以及 Zve 系列扩展中的向量内存指令，在指令层面也将遵循 RVTSO 内存模型。但 Ztso 扩展并不会增强单条向量指令内部元素访问操作的顺序性。

尽管 Ztso 扩展本身并未在指令集架构 (ISA) 中引入任何新指令，但为 RVTSO 内存模型编写的代码，在未实现 Ztso 扩展的硬件平台上将无法正确执行。因此，建议仅在 Ztso 环境下运行的二进制文件，应通过某种二进制标志进行标识，以便那些未实现 Ztso 扩展的平台能够直接拒绝执行这些程序。

19 CM0 扩展：基础缓存管理操作 ISA，V1.0.0

19.1 指令语义的伪代码

在指令章节中，每条指令的语义均以类 SAIL 语法描述。

19.2 介绍

缓存管理操作（或 CMO）指令对内存层次结构中的数据副本执行操作。通常，CMO 指令操作的是数据的缓存副本，但在某些情况下，CMO 指令可能直接对内存位置进行操作。此外，CMO 指令按操作分为以下几类：

- * 管理指令根据可以访问数据的一组代理来操作缓存的数据副本
- * 清零指令将一范围内的内存位置清零，可能在一个或多个缓存中分配数据的缓存副本
- * 预取指令向硬件指示给定内存位置的数据可能在不久之后被访问，可能在一个或多个缓存中分配数据的缓存副本

本文档介绍了一组专门对缓存块或与缓存块对应的内存位置进行操作的 CMO ISA 扩展；这些称为缓存块操作（或 CBO）指令。上述每类指令在本规范中都代表一个扩展：

- * Zicbom 扩展定义了一组缓存块管理指令：CBO.INVALID、CBO.CLEAN 和 CBO.FLUSH
- * Zicboz 扩展定义了一条缓存块清零指令：CBO.ZERO
- * Zicbop 扩展定义了一组缓存块预取指令：PREFETCH.R、PREFETCH.W 和 PREFETCH.I

上述指令的执行行为也受本规范添加的控制状态寄存器状态的影响。

本文档的其余部分提供了有关 CMO 指令的一般背景信息，并描述了上述每个 ISA 扩展。



CMO 一词包含了对缓存或与缓存相关的资源的所有操作。CBO 一词代表仅对缓存块进行操作的 CMO 子集。第一个 CMO 扩展仅定义了 CBO。

19.3 背景

本章提供了所有 CMO 扩展的通用信息。

19.3.1 内存和缓存

内存位置是系统中由物理地址唯一标识的物理资源。代理是一个逻辑块，例如 RISC-V 硬件线程、加速器、I/O 设备等，可以访问给定的内存位置。



某个代理可能无法访问系统中的所有内存位置，两个不同的代理可能会也可能不会访问同一组内存位置。

加载操作（或存储操作）由代理执行，用于获取（或修改）给定内存位置的数据。加载和存储操作是通过对该内存位置的显式内存访问而执行的。此外，读取传输从内存中获取该内存位置的数据，而写入传输则更新该内存位置的数据。

缓存是一种结构，用于缓冲数据副本以减少平均内存延迟。在代理和内存位置之间可以插入任意数量的缓存，并且代理的加载和存储操作可以由缓存而不是内存位置来满足。



加载和存储操作通过缓存实现了与物理读写传输的解耦。例如：加载操作可能仅通过缓存即可完成（无需执行来自内存的读取传输）；而存储操作可能需要先通过缓存执行来自内存的读取传输，之后才能完成该存储操作。

缓存将数据副本组织成缓存块，每个缓存块代表一组连续的、自然对齐的二的幂（或 NAPOT）的内存位置范围。缓存块由对应于底层内存位置的任何物理地址标识。缓存的容量和组织以及缓存块的大小都是实现特定的，执行环境为软件提供了一种方法来发现系统中缓存和缓存块的信息。在最初的 CMO 扩展中，缓存块的大小在整个系统中应是统一的。



在未来的 CMO 扩展中，对统一缓存块大小的要求可能会放宽。

诸如推测执行或硬件预取等实现技术可能会导致在缓存任何时间分配或释放缓存块的副本，前提是相应的物理地址可以根据支持的访问类型物理内存属性（PMA）访问，并且根据缓存可缓存能力物理内存属性是可缓存的。分配缓存块的副本会导致从另一个缓存或内存进行读取传输，而释放缓存块的副本可能会导致写入传输到另一个缓存或内存，具体取决于副本中的数据是否已被存储操作修改。更多详细信息请参见 nCoherent Agents and Caches。

19.3.2 缓存块操作

CBO 指令会对指令标识的缓存块执行一个或多个操作。通常，CBO 指令可以标识一个或多个缓存块；然而，在最初的 CMO 扩展中，CBO 指令仅标识单个缓存块。

缓存块管理指令相对于分配在给定缓存中的给定缓存块的副本执行以下操作之一：

- * 使无效操作释放缓存块的副本
- * 清理操作如果缓存块副本中的数据已被存储操作修改，则执行写入传输到另一个缓存或内存
- * 刷新操作原子性地执行清理操作，然后执行使无效操作

更多详细信息，包括给定缓存块管理指令执行的实际操作，请参见 Cache-Block Management Instructions。

缓存块清零指令执行一组存储操作，将零写入与缓存块对应的一组字节。除非另有说明，否则缓存块清零指令生成的存储操作具有与架构中的其他存储指令相同的一般属性和行为。实现可以选择是否使用单个存储操作原子性地更新整个字节集。更多详细信息请参见 Cache-Block Zero Instructions。

缓存块预取指令是对硬件的提示，表明软件预计在不久的将来会执行特定类型的内存访问。更多详细信息请参见 Cache-Block Prefetch Instructions。

19.4 一致代理和缓存

对于给定的内存位置，一组一致代理由满足以下所有条件的代理组成：

- * 集合中所有代理的存储操作相对于彼此呈现为串行化
- * 集合中所有代理的存储操作最终对集合中的所有其他代理可见
- * 集合中某代理的加载操作返回集合中某代理的存储操作的数据（或内存中的初始数据）

这样的集合中的一致性代理应使用相同的物理地址和相同的物理内存属性访问给定的内存位置；但是，如果给定代理的一致性 PMA 表明给定的内存位置不一致，则该代理不应成为该内存位置与任何其他代理的一致代理集合的成员，并且应成为由其自身组成的一致代理集合的唯一成员。

成为一致代理集合成员的代理被称为相对于集合中的其他代理是一致的。另一方面，不是成员的代理被称为相对于集合中的代理是不一致的。

缓存引入了在系统中同时存在给定缓存块的多个副本的可能性。一个实现特定的机制使这些副本相对于一致代理集合中的代理的加载和存储操作保持一致。此外，如果集合中的一致代理执行指定缓存块的 CBO 指令，则结果操作应适用于可以通过一致代理的加载和存储操作访问的缓存中的所有副本。



来自 CBO 指令的操作被定义为仅对由一致代理集合执行的显式内存访问可访问的缓存中缓存的缓存块副本进行操作。这包括仅通过加载和存储操作间接访问的缓存中的缓存块副本，例如一致性指令缓存。

受上述机制约束的缓存集合形成一个一致性缓存集合，并且每个一致性缓存具有以下行为，假设所有操作均由一致性代理集合中的代理执行：

- * 允许一致缓存分配和释放缓存块的副本，并执行 Memory and Cachs 中描述的读写传输
- * 只要自最近一次对缓存块使无效（invalidate）、清理或刷新操作以来，存储操作已经修改了缓存块中的数据，缓存是允许执行写入传输到内存的操作的
- * 至少一个一致缓存负责执行写入传输到内存，一旦存储操作修改了缓存块中的数据，直到下一次对缓存块的使无效、清理或刷新操作为止，在此之后，直到下一次存储操作修改缓存块中的数据，所有一致性缓存都不负责（或不允许）执行写传输到内存。
- * 如果存储操作自最近一次对缓存块的使无效、清理或刷新操作以来修改了缓存块中的数据，并且下一次清理或刷新操作需要写入传输到内存，则一致缓存必须执行写入传输到内存。



上述限制确保了通过从内存读取传输能够获取一个清洁的缓存块副本，并且没有被存储操作修改，之后不会覆盖由非一致性代理通过写传输更新到内存的缓存块副本。

非一致性代理可以启动一个缓存块操作，该操作作用于由一致性代理集访问的一致性缓存集。执行此类操作的机制是特定于实现的。

19.4.1 内存排序

保留的程序顺序

保留的程序顺序（缩写为 PPO）规则由 RVWMO 内存排序模型定义。以下描述了由 CMO 指令产生的操作如何适应这些规则。

对于缓存块管理指令，产生的使无效、清理和刷新操作在 PPO 规则中表现为存储操作，但受一个附加的重叠地址规则的约束。具体来说，如果 a 在程序顺序中先于 b，则如果满足以下条件，a 将在全局内存顺序中先于 b：

- a 是使无效、清理或刷新操作，b 是加载操作，并且 a 和 b 访问相同的内存地址。



上述规则确保了程序顺序中的后续加载操作在全局内存顺序中永远不会出现在先前的使无效、清除或刷新操作之前，尤其是当它们访问相同地址时。

此外，使无效、清理和刷新操作在 FENCE 指令的前驱和后继集中被分类为 W 或 O（取决于相应物理地址的物理内存属性）。这些操作不会被其他排序存储的指令排序，例如 FENCE.I 和 SFENCE.VMA。

对于缓存块清零指令，生成的存储操作在 PPO 规则中表现为存储，并会被其他排序存储的指令排序。

最后，对于缓存块预取指令，产生的操作不受 PPO 规则的排序，也不受任何其他排序指令的排序。

加载值

使无效操作可能会改变加载可以返回的值集。特别地，加载值公理中新增了一个附加条件：

· 如果一个使无效操作 *i* 在加载操作 *r* 之前，并且该操作作用于 *r* 返回的字节 *x*，并且在 *i* 和 *r* 之间的程序顺序或全局内存顺序中没有对 *x* 的存储操作，那么 *r* 可以返回以下任意值：

1. 如果在全局内存顺序中没有清除或刷新操作在 *i* 之前操作 *x*，则返回 *x* 的初始值或任何在 *i* 之前对 *x* 进行的存储操作的值。

2. 如果在全局内存顺序中没有存储操作在清除或刷新操作之前操作 *x*，并且 *x* 上的清除或刷新操作在 *i* 之前操作 *x*，则返回 *x* 的初始值或任何在 *i* 之前对 *x* 进行的存储操作的值。

3. 如果在全局内存顺序中，存储操作在清除或刷新操作之前操作 *x*，并且 *x* 上的清除或刷新操作在 *i* 之前操作 *x*，则返回 *x* 的最新存储操作的值（该操作在最新的清除或刷新操作之前），或者返回任何对 *x* 的存储操作，该操作既在 *i* 之前，又在最新的清除或刷新操作之后。

4. 返回由非一致代理对 *x* 的任何存储的值，无论上述条件如何



前三个要点描述了在全局内存顺序中相对于清理或刷新操作，不同位置的可能加载值。最后一个要点意味着加载值可能在任何时候由非一致性代理产生。

19.4.2 陷阱

执行某些 CMO 指令可能会由于控制状态寄存器的状态（在控制和状态寄存器状态部分中描述）或由于地址转换和保护机制而触发陷阱。CMO 指令的陷阱行为在以下部分描述。

非法指令和虚拟指令异常

缓存块管理指令和缓存块清零指令可能会根据当前特权模式和控制 and 状态寄存器状态部分中描述的 CMO 控制寄存器的状态引发非法指令异常或虚拟指令异常。

缓存块预取指令不会引发非法指令异常或虚拟指令异常。

缺页异常、客户机缺页异常和访问故障异常

与加载和存储指令类似，CMO 指令是计算有效地址的显式内存访问指令。有效地址最终根据特权模式和启用的转换机制转换为物理地址，CMO 扩展对给定缓存块中的物理地址施加以下约束：

· 物理内存保护（PMP）访问控制位对于缓存块中的所有物理地址应相同，并且如果物理内存保护访问控制位授予写权限，则还应授予读权限

· 物理内存属性（PMA）对于缓存块中的所有物理地址应相同，并且如果支持的访问类型物理内存属性授予写权限，则还应授予读权限

如果上述约束未得到满足，则 CBO 指令的行为是 UNSPECIFIED 的。



本规范假定上述约束通常会在主存区域满足，并且可能在某些 I/O 区域满足。

Zicboz 扩展引入了一个额外的支持访问类型物理内存属性，用于缓存块清零指令。主存区域需要支持缓存块清零指令的访问；然而，I/O 区域可以指定是否支持缓存块清零指令的访问。

当加载指令或存储指令被允许访问相应的物理地址时，缓存块管理指令被允许访问指定的缓存块。如果加载指令和存储指令都不允许访问物理地址，但指令获取被允许访问物理地址，则缓存块管理指令是否被允许访问缓存块是 UNSPECIFIED 的。如果不允许访问缓存块，则缓存块管理指令会在地址转换不允许任何访问时引发存储页面错误（store page fault）或存储访客页面错误异常（store guest-page fault exception），或者在其他情况下引发存储访问错误异常。在地址转换过程中，指令还会检查访问位，并根据需要抛出异常或设置该位。



缓存块管理指令与指令获取之间的交互将在未来的扩展中指定。

这意味着，缓存块管理指令不会检查脏位，也不会引发异常或设置该位。

只要存储指令被允许访问相应的物理地址，并且 PMA 指示缓存块清零指令是支持的访问类型，就允许缓存块清零指令访问指定的缓存块。如果不允许访问缓存块，则缓存块清零指令会在地址转换不允许写访问时引发存储页故障或存储访客页故障异常，否则会引发存储访问故障异常。在地址转换期间，指令还会检查访问位和脏位，并可能根据需要引发异常或设置这些位。

只要加载指令、存储指令或指令获取被允许访问相应的物理地址，就允许缓存块预取指令访问指定的缓存块。如果不允许访问缓存块，则缓存块预取指令不会引发任何异常，也不会访问任何缓存或内存。在地址转换期间，指令不会检查访问位和脏位，也不会引发异常或设置这些位。

当发生页故障、访客页故障或访问故障异常时，相关的 `*tval` 控制状态寄存器会写入故障的有效地址（即与这些异常的其他原因相同的故障地址值）。



像加载或存储指令一样，CMO 指令是否被允许访问缓存块取决于 `mstatus` 中的 `MPRV`、`MPV` 和 `MPP` 位以及 `mstatus`、`sstatus` 和 `vsstatus` 中的 `SUM` 和 `MXR` 位的状态。

本规范预计实现将像处理存储/AMO 指令一样处理缓存块管理指令，因此，无论所需的权限如何，存储/AMO异常适用于这些指令。

地址未对齐异常

CMO 指令不会生成地址未对齐异常。

断点异常和调试模式进入

除非调试架构规范另有定义，否则触发模块（`trigger modules`）相对于 CMO 指令的行为是 UNSPECIFIED 的。

对于 Zicbom、Zicboz 和 Zicbop 扩展，本规范建议以下通用触发模块行为：

- * 应支持类型 6 地址匹配触发器（`match triggers`），即 `tdatal.type=6` 和 `mcontrol6.select=0`
- * 不应支持类型 2 地址/数据匹配触发器，即 `tdatal.type=2`
- * 内存访问的大小等于访问的缓存块的大小，比较值来自包含有效地址的 NAPOT 内存区域的地址
- * 除非在 `mcontrol6.size` 字段中添加了缓存块的编码，否则地址触发器仅在 `mcontrol6.size=0` 时匹配来自 CBO 指令的内存访问



如果实现了 Zicbom 扩展，本规范建议以下附加触发模块行为：

- * 实现地址匹配触发器应为可选项
- * 不应支持类型 6 数据匹配触发器，即 `tdatal.type=6` 和 `mcontrol6.select=1`
- * 内存访问被视为存储，即地址触发器仅在 `mcontrol6.store=1` 时匹配

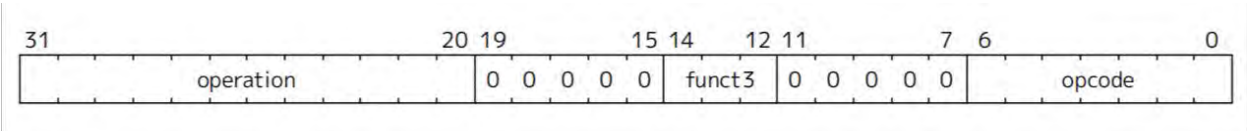
如果实现了 Zicboz 扩展，本规范建议以下附加触发模块行为：

- * 实现地址匹配触发器应为强制项

- * 应支持类型 6 数据匹配触发器，即 `tdata1.type=6` 和 `mcontrol6.select=1`，并且实现这些触发器应为可选
 - * 内存访问被视为存储，即地址触发器仅在 `mcontrol6.store=1` 时匹配
- 如果实现了 Zicbop 扩展，本规范建议以下附加触发模块行为：
- * 实现地址匹配触发器应为可选项
 - * 不应支持类型 6 数据匹配触发器，即 `tdata1.type=6` 和 `mcontrol6.select=1`
 - * 内存访问可以视为加载或存储，具体取决于实现，即当 `mcontrol6.load=1` 或 `mcontrol6.store=1` 时，这些指令上的地址触发器是否匹配是实现特定的
- 本规范还建议，Zicboz 扩展的触发模块行为应在调试架构规范的 1.0 版本中定义。Zicbom 和 Zicbop 扩展的触发模块行为预计将在未来的扩展中定义。

虚拟机监视器扩展

为了在陷阱时写入 `mtinst` 或 `htinst` 寄存器，定义了以下标准转换用于缓存块管理指令和缓存块清零指令：



`operation` 字段对应于陷阱指令的 12 个最高有效位。



如虚拟机监视器扩展中所述，可以将零写入 `mtinst` 或 `htinst`，而不是上述标准转换。

19.4.3 对受限 LR/SC 循环的影响

以下事件被添加到满足 A 扩展中定义的受约束 LR/SC 循环提供的最终保证的事件列表中：

- * 其他硬件线程对 H 的受约束 LR/SC 循环中的 LR 指令的预留集执行缓存块管理指令或缓存块清零指令。

添加上述事件是为了适应无法区分存储失效和缓存块管理操作失效的缓存一致性协议。



除了上述事件外，CMO 指令既不会改变受约束 LR/SC 循环的属性，也不会修改它们提供的最终性保证。例如，执行 CMO 指令可能会导致任何硬件线程上的受约束 LR/SC 循环周期性失败，或者可能导致同一硬件线程上的非受约束 LR/SC 序列始终失败。此外，执行缓存块预取指令不会影响任何硬件线程上执行的受约束 LR/SC 循环提供的最终保证。

19.4.4 软件探测

最初的 CMO 扩展要求软件发现以下信息：

- * 管理和预取指令的缓存块大小
- * 清零指令的缓存块大小
- * 每个特权级别的 CBIE 支持

探测机制中还可以指定其他一般缓存特性。

19.5 控制状态寄存器状态



CMO 扩展依赖于将在未来的特权架构更新中定义的 {csrname} 控制状态寄存器的状态。如果此控制状态寄存器更新未被批准，CMO 扩展将定义其自己的控制状态寄存器。

三种 CSR 控制 CMO 指令的执行：

- mencvfg
- sencvfg
- hencfg

寄存器由所有监督模式使用，包括 VS 模式。虚拟机监视器负责在客户上下文切换时保存和恢复 sencvfg。hencfg 寄存器仅在实现并启用了 H 扩展时存在。

每个 xencvfg 寄存器（其中 x 是 m、s 或 h）具有以下通用格式：

xencvfg CSRs 的通用格式

能够在较低特权模式下执行缓存块使无效指令 CBO.INVALID：

- * 00：指令引发非法指令或虚拟指令异常
- * 01：指令执行并执行刷新操作
- * 10：保留
- * 11：指令执行并执行使无效操作

能够在较低特权模式下执行缓存块清理指令 CBO.CLEAN 和缓存块刷新指令 CBO.FLUSH：

- * 0：指令引发非法指令或虚拟指令异常
- * 1：指令执行

能够在较低特权模式下执行缓存块清零指令 CBO.ZERO：

- * 0：指令引发非法指令或虚拟指令异常
- * 1：指令执行

xencvfg 寄存器根据当前特权模式和适当的控制状态寄存器状态控制 CBO 指令的执行，具体如下所述。

CBO.INVALID 指令根据 xencvfg.CBIE 字段的状态执行或引发非法指令异常或虚拟指令异常：
[source, sail, subs=attributes+]

```
if (((priv_mode != M) && (mencvfg.CBIE == 00)) ||
    ((priv_mode == U) && (sencvfg.CBIE == 00)))
{
    <触发非法指令异常>
}
// 虚拟指令异常
else if (((priv_mode == VS) && (hencfg.CBIE == 00)) ||
         ((priv_mode == VU) && (hencfg.CBIE == 00) || (sencvfg.CBIE == 00)))
{
    <触发虚拟指令异常>
}
// 执行指令
```

```

else
{
    if (((priv_mode != M) && (mencvfg.CBIE == 01)) ||
        ((priv_mode == U) && (sencvfg.CBIE == 01)) ||
        ((priv_mode == VS) && (hencfg.CBIE == 01)) ||
        ((priv_mode == VU) && (hencfg.CBIE == 01) || (sencvfg.CBIE == 01)))
    {
        <执行 CBO. INVALID 并执行刷新操作>
    }
    else
    {
        <执行 CBO. INVALID 并执行使无效操作>
    }
}

```

在修改后的缓存块更新内存之前，如果控制状态寄存器被编程为执行使无效操作，则 CBO. INVALID 指令可能会在内存中暴露过时的数据值。如果较低特权级别的软件执行使无效操作并访问内存中的敏感信息，这种行为可能会导致安全漏洞。



为了避免此类漏洞，较高特权级别的软件必须在允许较低特权级别的软件对缓存块执行使无效操作之前，对缓存块执行清理或刷新操作。或者，较高特权级别的软件可以编程控制状态寄存器，使得 CBO. INVALID 在较低特权级别下要么触发陷阱，要么执行刷新操作。

CBO.CLEAN 或 CBO.FLUSH 指令根据 xencvfg.CBCFE 位的状态决定是否能够成功执行或是引发非法指令异常或是引发虚拟指令异常：

```

// 非法指令异常
if (((priv_mode != M) && !mencvfg.CBCFE) ||
    ((priv_mode == U) && !sencvfg.CBCFE))
{
    <触发非法指令异常>
}
// 虚拟指令异常
else if (((priv_mode == VS) && !hencfg.CBCFE) ||
    ((priv_mode == VU) && !(hencfg.CBCFE && sencvfg.CBCFE)))
{
    <触发虚拟指令异常>
}
// 执行指令
else
{
    <执行 CBO.CLEAN 或 CBO.FLUSH 指令>
}

```

最后, CBO.ZERO 指令根据 xencvfg.CBZE 位的状态决定是否能够成功执行或是引发非法指令异常或是引发虚拟指令异常:

[source, sail, subs=attributes+]

```
// 非法指令异常
if (((priv_mode != M) && !mencvfg.CBZE) ||
    ((priv_mode == U) && !sencvfg.CBZE))
{
    <触发非法指令异常>
}
// 虚拟指令异常
else if (((priv_mode == VS) && !hencfg.CBZE) ||
    ((priv_mode == VU) && !(hencfg.CBZE && sencvfg.CBZE)))
{
    <触发虚拟指令异常>
}
// 执行指令
else
{
    <执行 CBO.ZERO 指令>
}
```

每个 xencvfg 寄存器都是 WARL (Write Any, Read Legal) 的; 然而, 软件应从执行环境发现机制来确定合法值。

[#extensions, reftext=Extensions]

19.6 扩展

CMO 指令在以下扩展中定义:

- 缓存块管理说明
- 缓存块零指令
- 缓存块预取指令

19.6.1 缓存块管理指令

缓存块管理指令使运行在一致性代理集 (coherent agents) 上的软件能够通过执行以下操作之一与非一致性代理集 (non-coherent agents) 进行通信:

* 使无效操作通过从一致性缓存集合中释放所有缓存块副本, 使得由一组非一致性代理执行的存储操作的数据在两个集合的共同点上对一致性代理集合可见

* 清理操作通过执行缓存块副本的写入传输, 使得由一致代理集合执行的存储操作的数据在两个集合的共同点上对非一致代理集合可见, 前提是自上次对缓存块的使无效、清理或刷新操作以来, 一致代理执行了修改缓存块中数据的存储操作

* 刷新操作原子性地执行清理操作, 然后执行使无效操作

在 Zicbom 扩展中, 指令在系统中所有代理的共同点上操作。换句话说, 使无效操作确保所有非一致性代理的存储操作对一致性代理集合中的代理可见, 而清理操作确保一致性代理的存储操作对所有非一致性代理可见。



Zicbom 扩展不禁止超出上述架构定义的代理; 然而, 软件不能依赖定义的缓存操作对这些代理产生预期的效果。

未来的扩展可能会定义不同的代理集合以进行性能优化。

这些指令对其有效地址在 `rs1` 中指定的缓存块进行操作。有效地址通过适当的转换机制转换为相应的物理地址。

以下指令组成了 Zicbom 扩展:

RV32	RV64	Mnemonic	Instruction
✓	✓	<code>cbo.clean base</code>	Cache Block Clean
✓	✓	<code>cbo.flush base</code>	Cache Block Flush
✓	✓	<code>cbo.inval base</code>	Cache Block Invalidate

19.6.2 缓存块归零指令

缓存块归零指令将零值存储到与缓存块对应的一组字节中。实现可以以任意顺序和粒度、原子性地更新字节, 包括单个字节。



缓存块归零指令将零值存储到字节中, 而不考虑底层内存位置的数据是否可缓存。此外, 本规范不限制字节写入的方式。

这些指令作用于缓存块或与缓存块对应的内存位置, 其有效地址由 `rs1` 指定。有效地址通过适当的转换机制转换为相应的物理地址。

以下指令组成了 Zicboz 扩展:

RV32	RV64	Mnemonic	Instruction
✓	✓	<code>cbo.clean base</code>	Cache Block Zero

19.6.3 缓存块预取指令

缓存块预取指令是对硬件的提示, 表明软件打算在不久的将来执行特定类型的内存访问。内存访问的类型包括指令获取、数据读取 (即加载) 和数据写入 (即存储)。

这些指令对其有效地址是 `rs1` 中指定的基地址与 `imm[11:0]` 中编码的符号扩展偏移量之和的缓存块进行操作, 其中 `imm[4:0]` 应等于 `0b00000`。有效地址通过适当的转换机制转换为相应的物理地址。



缓存块预取指令被编码为 `rd` 等于 `0b00000` 的 ORI 指令; 然而, 为了有效地址计算的目的, 该字段也被解释为 `imm[4:0]`, 类似于存储指令。

以下指令组成了 Zicbop 扩展:

RV32	RV64	Mnemonic	Instruction
✓	✓	<code>prefetch.i offset(base)</code>	Cache Block Prefetch for Instruction Fetch
✓	✓	<code>prefetch.r offset(base)</code>	Cache Block Prefetch for Data Read
✓	✓	<code>prefetch.w offset(base)</code>	Cache Block Prefetch for Data Write

19.7 指令

19.7.1 缓存块清除指令

cbo.clean

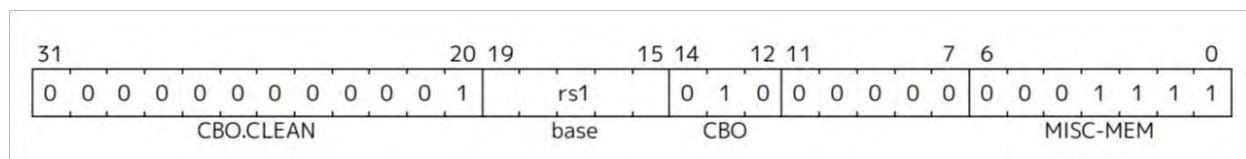
简述:

对缓存块执行清除操作

助记符:

cbo.clean offset(base)

编码:



描述:

cbo.clean 指令对有效地址为 rs1 中指定的基地址的缓存块执行清除操作。偏移操作数可以省略; 否则, 任何计算偏移量的表达式都应计算为零。该指令作用于执行指令的代理访问的缓存一致性缓存集。

操作:

TODO

19.7.2 缓存块刷新指令

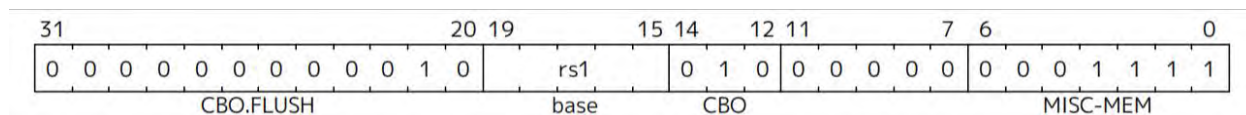
简述:

对缓存块执行刷新操作

助记符:

cbo.flush offset(base)

编码:



描述:

cbo.flush 指令对有效地址为 rs1 中指定的基地址的缓存块执行刷新操作。偏移操作数可以省略; 否则, 任何计算偏移量的表达式都应计算为零。该指令作用于执行指令的代理访问的缓存一致性缓存集。

操作:

TODO

19.7.3 缓存块无效指令

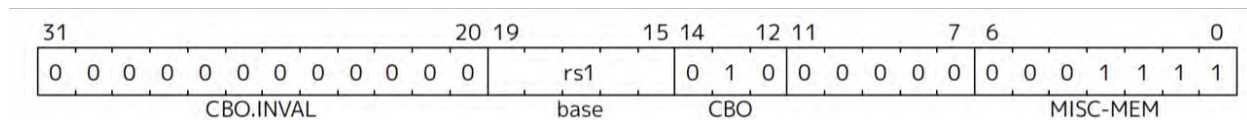
简述:

对缓存块执行使无效操作

助记符:

cbo.inval offset(base)

编码:



描述:

cbo.inval 指令对有效地址为 rs1 中指定的基地址的缓存块执行使无效操作。偏移操作数可以省略; 否则, 任何计算偏移量的表达式都应计算为零。该指令作用于执行指令的代理访问的缓存一致性缓存集。根据控制状态寄存器的编程, 指令可能会执行刷新操作, 而不是使无效操作。

操作:

TODO

19.7.4 缓存块清零指令

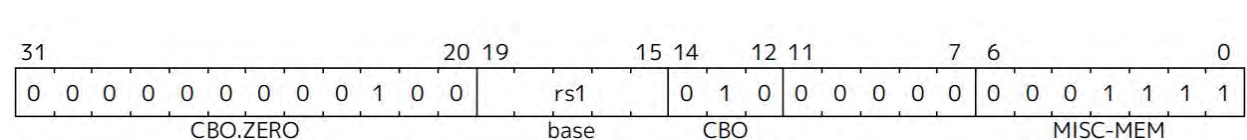
简述:

将零存储到与缓存块对应的完整字节集

助记符:

cbo.zero offset(base)

编码:



描述:

cbo.zero 指令将零值存储到与有效地址为 rs1 中指定的基地址的缓存块对应的所有字节中。偏移操作数可以省略; 否则, 任何计算偏移量的表达式都应计算为零。实现可以选择是否以原子方式更新整个字节集。

操作:

TODO

19.7.5 指令取值的缓存块预取

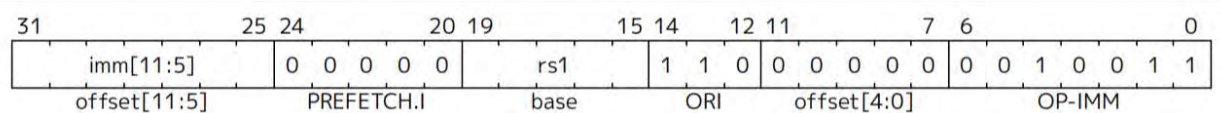
简述:

向硬件提供一个 HINT, 表明某个缓存块在不久的将来可能会被指令取值访问

助记符:

`prefetch.i offset(base)`

编码:



描述:

`prefetch.i` 指令向硬件指示, 有效地址为 `rs1` 中指定的基地址与 `imm[11:0]` 中编码的符号扩展偏移之和的缓存块, 可能在不久的将来被指令获取访问, 其中 `imm[4:0]` 等于 `0b00000`。



某些实现可能会选择在指令取值访问的缓存中缓存该缓存块的副本, 以降低内存访问延迟, 但这种行为并非强制要求的。

操作:

TODO

19.7.6 数据取值的缓存块预读取

`prefetch.r`

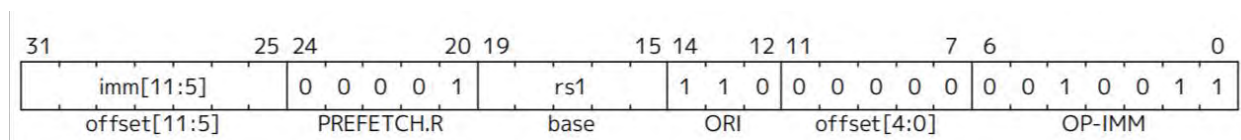
简述:

向硬件提供一个 HINT, 表明某个缓存块在不久的将来可能会被数据读取访问

助记符:

`prefetch.r offset(base)`

编码:



描述:

`prefetch.r` 指令向硬件指示, 有效地址为 `rs1` 中指定的基地址与 `imm[11:0]` 中编码的符号扩展偏移之和的缓存块, 可能会在不久的将来通过数据读取 (即加载) 被访问, 其中 `imm[4:0]` 等于 `0b00000`。



某些实现可能会选择在数据读取访问的缓存中缓存该缓存块的副本，以改善内存访问延迟，但这种行为并非强制要求。

操作:

TODO

`prefetch.w`

19.7.7 数据取值的缓存块预写入

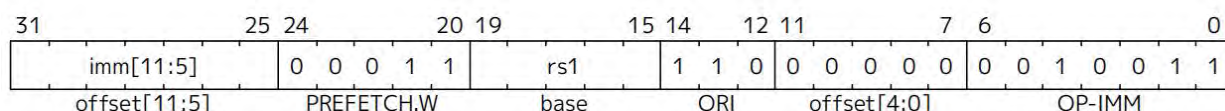
简述:

向硬件提供一个 HINT，表明某个缓存块在不久的将来可能会被数据写入访问

助记符:

`prefetch.w offset(base)`

编码:



描述:

*`prefetch.w` 指令向硬件指示，有效地址为 `rs1` 中指定的基地址与 `imm[11:0]` 中编码的符号扩展偏移之和的缓存块，可能会在不久的将来通过数据写入（即存储）被访问，其中 `imm[4:0]` 等于 `0b00000`。



某些实现可能会选择在数据写入访问的缓存中缓存该缓存块的副本，以改善内存访问延迟，但这种行为并非强制要求。

20 F 扩展：单精度浮点，V2.2

本章描述了用于单精度浮点的标准指令集扩展，称为 F。该扩展增加了符合 IEEE 754-2008 算术标准的单精度浮点计算指令。F 扩展依赖于 Zicsr 扩展，用于访问控制状态寄存器。

20.1 F 寄存器状态

F 扩展新增了 32 个浮点寄存器 (`f0-f31`)，每个寄存器宽度为 32 位，并增加了一个浮点控制状态寄存器 `fcsr`，用于存储浮点单元的运行模式和异常状态。这些附加状态如表 20.1 所示。我们使用术语 `FLEN` 来表示 RISC-V ISA 中浮点寄存器的位宽，对于 F 单精度浮点扩展，`FLEN=32`。大多数浮点指令在浮点寄存器中的数值上执行操作。浮点加载和存储指令用于在浮点寄存器与内存之间传输浮点值，同时也提供了用于在浮点寄存器与整数寄存器之间传输数据的指令。



我们曾考虑将整数和浮点数统一存储在同一个寄存器文件中，这样可以简化软件寄存器分配和调用约定，同时减少用户状态的总量。然而，采用分离的寄存器组织方式，能够在相同的指令宽度下增加可访问的寄存器数量，简化宽超标量执行（superscalar issue）所需的寄存器端口设计，支持解耦的浮点单元架构，并且简化内部浮点编码技术的使用。对于分离式寄存器文件架构，编译器支持和调用约定已经非常成熟。此外，在浮点寄存器文件上使用脏位（dirty bits），可以降低上下文切换的开销。

表 20.1 RISC-V 标准 F 扩展单精度浮点状态

FLEN-1	0
F0	
F1	
F2	
F3	
F4	
F5	
F6	
F7	
F8	
F9	
F10	
F11	
F12	
F13	
F14	
F15	
F16	
F17	
F18	
F19	
F20	
F21	
F22	
F23	
F24	
F25	
F26	
F27	
F28	
F29	
F30	
F31	

FLEN	
31	0
Fcsr	
32	

20.2 浮点控制状态寄存器

浮点控制和状态寄存器 `fcsr` 是 RISC-V 的控制和状态寄存器 (CSR)。它是一个 32 位的可读写寄存器，用于选择浮点算术运算的动态舍入模式，并存储累积的异常标志，如表 20.2 所示。

`fcsr` 寄存器可以通过 `FRCSR` 和 `FSCSR` 指令进行读写，这些指令是基于底层 CSR 访问指令的汇编伪指令。`FRCSR` 通过将 `fcsr` 复制到整数寄存器 `rd` 来读取 `fcsr`。`FSCSR` 通过将 `fcsr` 的初始值复制到整数寄存器 `rd`，然后将从整数寄存器 `rs1` 获取的新值写入 `fcsr`，从而交换 `fcsr` 的值。



图 20.1 浮点控制和状态寄存器

`fcsr` 内的各个字段也可以通过不同的 CSR 地址单独访问，并且为这些访问定义了单独的汇编伪指令。`FRRM` 指令读取舍入模式字段 `frm` (`fcsr` 的第 7 至 5 位)，并将其复制到整数寄存器 `rd` 的最低三位，其他位填充为零。`FSRM` 交换 `frm` 的值，它先将 `frm` 的初始值复制到整数寄存器 `rd`，然后将整数寄存器 `rs1` 的最低三位写入 `frm`。`FRFLAGS` 和 `FSFLAGS` 的定义方式与累积异常标志字段 `fflags` (`fcsr` 的第 4 至 0 位) 类似。

`fcsr` 的第 31 至 8 位保留用于其他标准扩展。如果这些扩展不存在，实现应忽略对这些位的写入，并在读取时返回零值。标准软件应当保留这些位的内容。

浮点运算可以使用指令中编码的静态舍入模式，或存储在 `frm` 中的动态舍入模式。舍入模式的编码方式如表 20.2 所示。当指令的 `rm` 字段取值为 111 时，表示选择 `frm` 中存储的动态舍入模式。当执行依赖舍入模式的浮点指令时，如果使用了保留的舍入模式，其行为是 UNSPECIFIED 的，包括静态保留舍入模式 (101-110) 和动态保留舍入模式 (101-111)。某些指令 (例如宽度扩展转换指令) 虽然具有 `rm` 字段，但其数学计算实际上不受舍入模式影响。软件应当将这些指令的 `rm` 字段设置为 RNE (000)，但硬件实现必须按通常方式处理 `rm` 字段 (特别是在解码合法编码与保留编码时)。

表 20.2 舍入模式编码

舍入模式	助记符	含义
000	RNE	最接近偶数舍入 (Round to Nearestties to Even)
001	RTZ	向零舍入 (Round towards Zero)
010	RDN	向下舍入 (RoundDown, 朝-8 方向)
011	RUP	向上舍入 (RoundUp, 朝+8 方向)
100	RMM	最接近最大幅值舍入 (Round to Nearestties to Max Magnitude)
101		保留供未来使用 (Reserved for future use.)
110		保留供未来使用 (Reserved for future use.)
111	DYN	在指令的 <code>rm</code> 字段中，表示选择动态舍入模式；在舍入模式寄存器 (<code>frm</code>) 中，该值是保留的。

C99 语言标准实际上要求提供一个动态舍入模式寄存器。在典型的实现中，对动态舍入模式 CSR 状态的写入会使流水线串行化。静态舍入模式通常用于实现特殊的算术运算，这些运算往往需要频繁切换不同的舍入模式。



已批准版本的 F 规范要求当指令使用了保留的动态舍入模式时，必须引发非法指令异常。这一要求已被弱化为保留（reserved），使其行为与静态舍入模式指令的处理方式一致。当遇到保留的编码时，引发非法指令异常仍然是合法的行为，因此与已批准规范兼容的实现也兼容该弱化版本的规范。

累积异常标志（Accrued Exception Flags）表示自该字段上次被软件重置以来，任意浮点算术指令发生的异常情况，如表 20.3 所示。基本的 RISC-V ISA 不支持在设置浮点异常标志时触发陷阱。

表 20.3 累积异常标志编码

标志助记符	标志含义
NV	非法操作
DZ	被零除
OF	溢出
UF	下溢
NX	不精确



按照标准的允许范围，我们在 F 扩展中不支持浮点异常的陷阱，而是要求由软件显式检查异常标志。我们曾考虑添加由浮点累积异常标志直接控制的分支指令，但最终选择省略这些指令，以保持 ISA 的简洁性。

20.3 NaN 生成和传播

除非另有说明，如果浮点运算的结果是 NaN，则该 NaN 为规范 NaN（canonical NaN）。规范 NaN 具有正号，并且所有尾数位（significand bits）均为 0，除最高位（又称 quiet bit）外。对于单精度浮点数，规范 NaN 对应的位模式为 0x7fc00000。



我们曾考虑按照标准的建议传播 NaN 负载（NaN payloads），但这一决定会增加硬件成本。此外，由于该功能在标准中是可选的，因此它无法用于可移植代码。

实现者可以自由地提供一种非标准扩展，在非标准操作模式下启用 NaN 负载传播方案。然而，上述规范 NaN 方案 必须始终得到支持，并且应作为默认模式。



我们要求实现返回标准规定的默认值，以应对异常情况，而无需用户级软件进行任何进一步的干预（与 Alpha ISA 浮点陷阱屏障不同）。我们相信，完全的硬件处理异常情况将变得更加普遍，因此希望避免复杂化用户级 ISA 以优化其他方法。实现始终可以陷入机器模式软件处理程序，以提供异常默认值。

20.4 次正规数运算

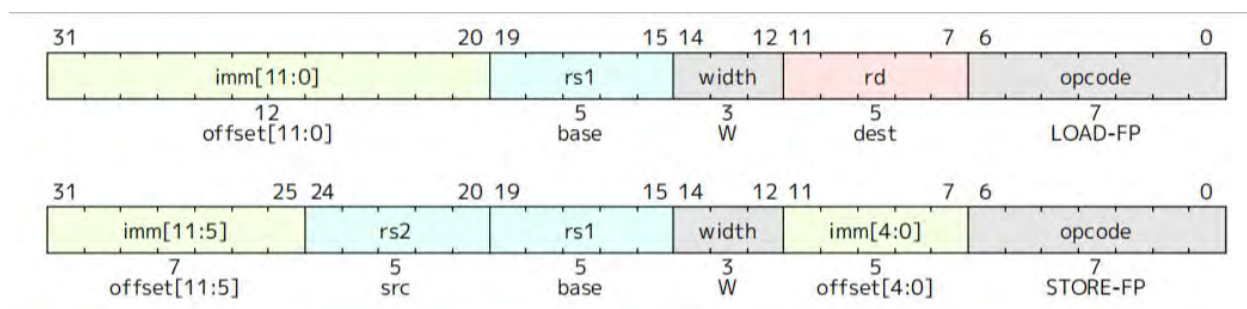
对次正规数（subnormal numbers）的运算遵循 IEEE 754-2008 标准。按照 IEEE 标准的术语，微小性（tininess）在舍入后检测。



在舍入后检测微小性可以减少不必要的下溢（underflow）信号。

20.5 单精度浮点加载和存储指令

浮点加载和存储使用与整数基本 ISA 相同的基址+偏移寻址模式，基址地址存储在寄存器 `rs1` 中，并使用 12 位有符号字节偏移量。FLW 指令将单精度浮点值从内存加载到浮点寄存器 `rd` 中。FSW 指令将单精度浮点值从浮点寄存器 `rs2` 存储到内存中。



FLW 和 FSW 仅在有效地址自然对齐时才能保证原子性执行。FLW 和 FSW 在传输数据时不会修改任何位，特别是会完整保留非规范 NaN（Non-Canonical NaN）的有效负载。如第 2.6 章所述，执行环境决定未对齐的浮点加载和存储是透明处理，还是触发受限或致命的异常。

20.6 单精度浮点计算指令

具有一个或两个源操作数的浮点算术指令使用 R 型格式，并以 OP-FP 作为主操作码。FADD.S 和 FMUL.S 分别执行 `rs1` 和 `rs2` 之间的单精度浮点加法和乘法。FSUB.S 计算 `rs1` 减去 `rs2` 的单精度浮点减法。FDIV.S 计算 `rs1` 除以 `rs2` 的单精度浮点除法。FSQRT.S 计算 `rs1` 的平方根。在每种情况下，计算结果都会写入 `rd` 寄存器。2 位浮点格式字段 `fmt` 的编码如表 20.4 所示。对于 F 扩展中的所有指令，该字段的值均设为 S（00）。

表 20.4 格式字段编码

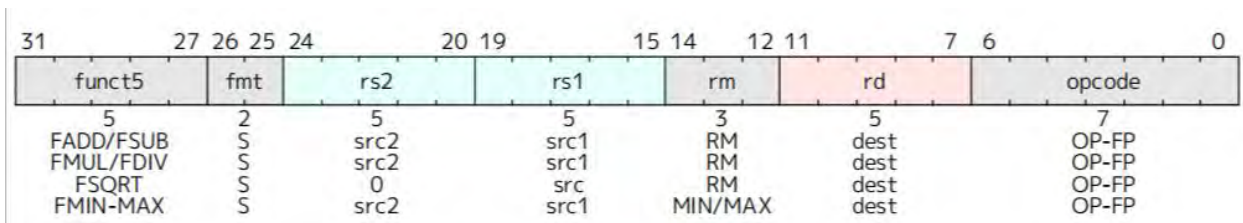
Fmt 字段	助记符	含义
00	S	32 位 单精度
01	D	64 位 双精度
10	H	16 位 半精度
11	Q	128 位 四精度

所有执行舍入的浮点运算都可以使用 `rm` 字段来选择舍入模式，其编码如表 20.2 所示。

浮点最小数和最大数指令 FMIN.S 和 FMAX.S 分别将 `rs1` 和 `rs2` 中较小或较大的值写入 `rd`。仅针对这些指令，值 -0.0 被认为小于值 +0.0。如果两个输入都是 NaN，则结果为规范 NaN（Canonical NaN）。如果只有一个操作数是 NaN，则结果为非 NaN 的那个操作数。如果输入的是信号 NaN（Signaling NaN），即使计算结果不是 NaN，也会触发无效操作异常标志（Invalid Operation Exception Flag）。



需要注意的是，在 F 扩展 2.2 版本中，FMIN.S 和 FMAX.S 指令被修改为实现 IEEE 754-201x 提议的 minimumNumber 和 maximumNumber 操作，而不再采用 IEEE 754-2008 标准中的 minNum 和 maxNum 操作。这些操作在处理信号 NaN (Signaling NaN) 时的行为有所不同。



浮点融合乘加 (Fused Multiply-Add, FMA) 指令采用了一种新的标准指令格式。R4 型指令指定三个源寄存器 (rs1、rs2 和 rs3) 以及一个目标寄存器 (rd)。这种格式仅用于浮点融合乘加指令。

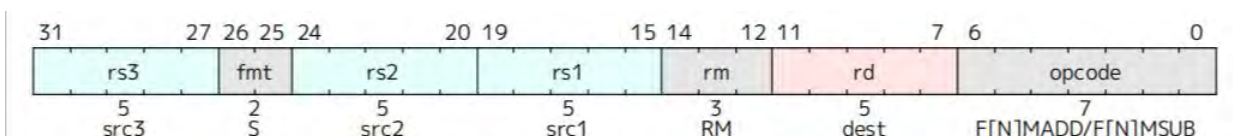
FMADD.S 计算 rs1 和 rs2 的乘积，然后加上 rs3，并将最终结果写入 rd。计算公式： $(rs1 \times rs2) + rs3$ 。

FMADD.S 计算 rs1 和 rs2 的乘积，然后减去 rs3，并将最终结果写入 rd。计算公式： $(rs1 \times rs2) - rs3$ 。

FMADD.S 计算 rs1 和 rs2 的乘积，取反后加上 rs3，并将最终结果写入 rd。计算公式： $-(rs1 \times rs2) + rs3$ 。

FMADD.S 计算 rs1 和 rs2 的乘积，取反后减去 rs3，并将最终结果写入 rd。计算公式： $-(rs1 \times rs2) - rs3$ 。

FNMSUB 和 FNMADD 指令的命名方式可能会让人感到困惑，这与 MIPS-IV 架构中对应指令的命名方式有关。在 MIPS 指令集中，这些指令的定义是对求和结果取反，而 RISC-V 指令的定义则是对乘积取反，因此在当时 MIPS 的命名方式更具合理性。这两种定义在带符号零 (signed-zero) 结果的处理上存在不同之处。RISC-V 的定义采用了与 x86 和 ARM 的融合乘加 (FMA) 指令一致的行为，但不幸的是，RISC-V 的 FNMSUB 和 FNMADD 指令的命名方式与 x86 和 ARM 含义相反。



融合乘加 (FMA) 指令占用了 32 位指令编码空间的很大一部分。曾考虑过一些替代方案，例如限制 FMA 仅使用动态舍入模式，但静态舍入模式在利用积不舍入的代码中仍然有用。另一种方案是使用 rd 作为 rs3，但这在某些常见序列中需要额外的移动指令。当前的设计仍然保留了 32 位编码空间中的大部分，同时避免了 FMA 指令的非正交性。

融合乘加指令在乘数为无穷大且被乘数为零时，必须设置无效操作异常标志，即使加数是静默 NaN。



注：IEEE 754-2008 标准允许，但不强制要求，在执行 $\infty \times 0 + \text{NaN}$ 这一运算时触发无效操作异常。

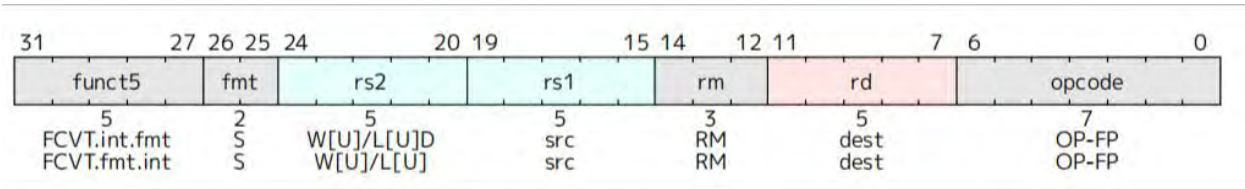
20.7 单精度浮点转换和移动指令

浮点数与整数之间的转换指令在 OP-FP 主操作码空间中进行编码。FCVT.W.S 或 FCVT.L.S 将浮点寄存器 rs1 中的浮点数分别转换为有符号 32 位或 64 位整数，并存入整数寄存器 rd。FCVT.S.W 或 FCVT.S.L 将整数寄存器 rs1 中的 32 位或 64 位有符号整数分别转换为浮点寄存器 rd 中的浮点数。

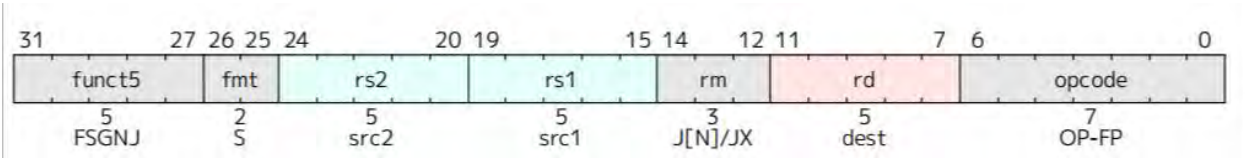
FCVT.WU.S、FCVT.LU.S、FCVT.S.WU 和 FCVT.S.LU 用于无符号整数值的转换。对于 XLEN>32 的情况，FCVT.W[U].S 会对 32 位结果进行符号扩展，以匹配目标寄存器的位宽。FCVT.L[U].S 和 FCVT.S.L[U] 仅适用于 RV64 指令集。如果舍入后的结果无法用目标格式表示，则会将其截断到最接近的值，并设置无效操作异常标志。表 20.5 表示 FCVT.int.S 指令的有效输入范围，以及对无效输入的处理方式。

所有浮点数到整数以及整数到浮点数的转换指令都根据 rm 字段进行舍入。使用 FCVT.S.W rd, x0 可以将浮点寄存器初始化为浮点正零（+0.0），并且不会触发任何异常标志。

所有浮点转换指令在舍入后的结果与操作数值不同且未设置无效操作异常标志时，会设置不精确（Inexact）异常标志。



浮点到浮点的符号注入指令 FSGNJ.S、FSGNJN.S 和 FSGNJX.S 产生的结果从 rs1 中获取除符号位以外的所有位。对于 FSGNJ，结果的符号位是 rs2 的符号位；对于 FSGNJN，结果的符号位是 rs2 符号位的相反位；对于 FSGNJX，符号位是 rs1 和 rs2 符号位的异或。符号注入指令不会设置浮点异常标志，也不会规范化 NaN。注意，FSGNJ.S rx, ry, ry 将 ry 移动到 rx（汇编伪指令 FMV.S rx, ry）；FSGNJN.S rx, ry, ry 将 ry 的相反数移动到 rx（汇编伪指令 FNEG.S rx, ry）；FSGNJX.S rx, ry, ry 将 ry 的绝对值移动到 rx（汇编伪指令 FABS.S rx, ry）。





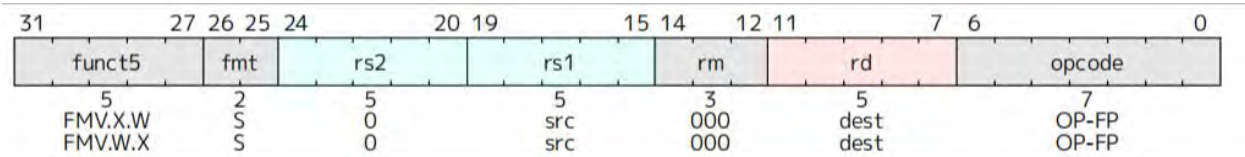
符号注入指令提供了浮点 MV、ABS 和 NEG 操作，并支持其他一些操作，包括 IEEE copySign 操作和超越数学函数库中的符号操作。虽然 MV、ABS 和 NEG 只需要一个寄存器操作数，而 FSGNJ 指令需要两个操作数，但大多数微架构不太可能添加优化，以便从减少寄存器读取次数中受益，因为这些指令相对不常用。即使如此，微架构可以通过检测 FSGNJ 指令的两个源寄存器是否相同，来简化操作，仅读取一个副本。

提供了将位模式在浮点寄存器和整数寄存器之间移动的指令。FMV.X.W 将浮点寄存器 rs1 中的单精度值（按照 IEEE 754-2008 编码）移动到整数寄存器 rd 的低 32 位。传输过程中的位不会修改，特别是非规范 NaN 的有效负载会被保留。对于 RV64，目标寄存器的高 32 位填充为浮点数符号位的副本。

FMV.W.X 将按照 IEEE 754-2008 标准编码的单精度值，从整数寄存器 rs1 的低 32 位移动到浮点寄存器 rd。传输过程中的位不会修改，特别是非规范 NaN 的有效负载会被保留。



FMV.W.X 和 FMV.X.W 指令之前被称为 FMV.S.X 和 FMV.X.S。使用 W 更符合它们作为一种指令的语义，即移动 32 位数据而不对其进行解释。随着 NaN装箱（NaN-boxing）定义的清晰，这一点变得更加明确。为了避免影响现有代码，工具将同时支持 W 版和 S 版指令。



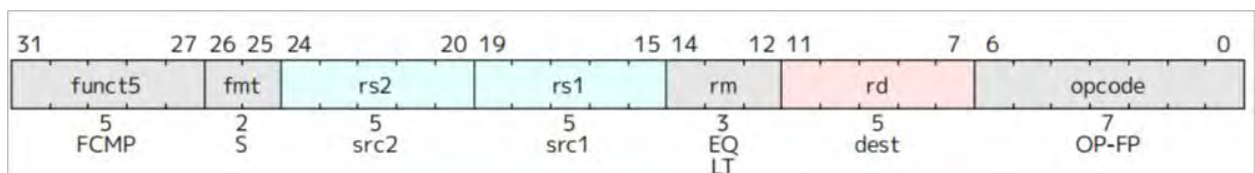


基础浮点 ISA 的定义允许实现采用浮点格式的内部重新编码，以便在寄存器中简化次正规值的处理，并可能减少功能单元的延迟。为此，F 扩展避免在浮点寄存器中表示整数值，而是通过定义转换和比较操作，使其直接读写整数寄存器文件。这也减少了许多常见情况下整数寄存器和浮点寄存器之间显式移动数据的需求，从而降低指令数量，并减少混合格式代码序列的关键路径。

20.8 单精度浮点比较指令

浮点比较指令（FEQ.S、FLT.S、FLE.S）在浮点寄存器（=，<，≤）之间执行指定的比较，如果条件成立，则向整数寄存器 rd 写入 1，否则写入 0。

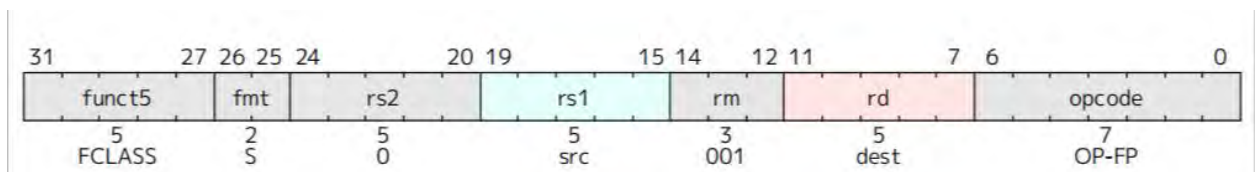
FLT.S 和 FLE.S 执行 IEEE 754-2008 标准所称的信号比较（signaling comparison），即如果任一输入为 NaN，则设置无效操作异常标志。FEQ.S 执行静默比较（quiet comparison），仅当任一输入为信号 NaN 时才会设置无效操作异常标志。对于这三条指令，如果任一操作数为 NaN，则结果为 0。



F 扩展提供了 ≤ 比较，而基础 ISA 提供的是 ≥ 分支比较。由于 ≤ 可以由 ≥ 合成，反之亦然，这种不一致性不会影响性能，但它仍然是 ISA 设计中一个令人遗憾的不协调之处。

20.9 单精度浮点分类指令

FCLASS.S 指令检查浮点寄存器 rs1 中的值，并将一个 10 位掩码写入整数寄存器 rd，该掩码指示浮点数的类别。掩码的格式如表 20.6 所述。如果属性为真，则 rd 中相应的位将被设置，否则将被清除。rd 中的所有其他位都被清除。注意，rd 中将恰好设置一个位。FCLASS.S 不会设置浮点异常标志。



FCLASS.S 指令检查浮点寄存器 rs1 中的值，并向整数寄存器 rd 写入一个 10 位掩码，该掩码指示浮点数的类别。掩码的格式在表 20.6 中描述。如果某个属性成立，则 rd 中对应的位会被置 1，否则清零。rd 中的所有其他位都会被清零。需要注意的是，rd 中恰好只有一个位会被置 1，FCLASS.S 不会设置浮点异常标志。

表 20.6 FCLASS 指令结果的格式

rd bit	Meaning
0	rs1 is $-\infty$.
1	rs1 is a negative normal number.
2	rs1 is a negative subnormal number.
3	rs1 is -0 .
4	rs1 is $+0$.
5	rs1 is a positive subnormal number.
6	rs1 is a positive normal number.
7	rs1 is $+\infty$.
8	rs1 is a signaling NaN.
9	rs1 is a quiet NaN.

21 D 扩展：双精度浮点指令，V2.2

本章描述了标准的双精度浮点指令集扩展，称为 D 扩展。该扩展添加了符合 IEEE 754-2008 算术标准的双精度浮点计算指令。D 扩展依赖于基础的单精度指令子集 F。

21.1 D 寄存器状态

D 扩展将 32 个浮点寄存器 f0-f31 扩展到 64 位表 20.1 中的 FLEN=64)。这些 f 寄存器现在可以保存 32 位或 64 位浮点值，如下文第 21.2 章中所描述。



FLEN 可以是 32、64 或 128，具体取决于支持的 F、D 和 Q 扩展。最多可以支持四种不同的浮点精度，包括 H、F、D 和 Q。

21.2 较窄值的 NaN 装箱 (NaN Boxing)

当支持多种浮点精度时，较窄的 n 位类型 ($n < \text{FLEN}$) 的有效值会以 NaN 装箱的方式表示在 FLEN 位的 NaN 值的低 n 位中。有效的 NaN 装箱值 (NaN-boxed) 的高位必须全部为 1。因此，当以任意更宽的 m 位值 ($n < m \leq \text{FLEN}$) 查看时，合法的 NaN 包装 n 位值会显示为负静默 NaN (qNaN)。任何将较窄结果写入 f 寄存器的操作，必须将所有 1 写入最高的 $\text{FLEN}-n$ 位，以产生合法的 NaN 包装值。



软件可能无法知道浮点寄存器中当前存储的数据类型，但必须能够保存和恢复寄存器的值。因此，使用更宽操作传输更窄值的结果必须是明确定义的。一个常见的情况是对于被调用者保存的寄存器，但对于包括可变参数、用户级线程库、虚拟机迁移和调试等功能，标准约定也是必要的。

浮点 n 位传输操作将以 IEEE 标准格式保存的外部值传入和传出 f 寄存器，包括浮点加载和存储操作 (FLn/FSn) 以及浮点移动指令 (FMV.n.X/FMV.X.n)。将较窄的 n 位值 ($n < \text{FLEN}$) 传输移入 f 寄存器时，将创建一个有效的 NaN 装箱值。将较窄的 n 位值从浮点寄存器传出时，将传输寄存器的低 n 位，忽略高 $\text{FLEN}-n$ 位。

除了前述的传输操作外，所有其他浮点操作在处理较窄的 n 位操作 ($n < \text{FLEN}$) 时，会检查输入操作数是否正确地进行了 NaN 装箱，即所有高于 $\text{FLEN}-n$ 的位是否为 1。如果是，输入的 n 低位将作为输入值，否则输入值将被视为 n 位标准 NaN。

该文档的早期版本没有定义将较窄或较宽操作数的结果输入到操作中的行为，除了要求较宽的操作数保存和恢复时会保留较窄操作数的值。新的定义移除了这一特定实现的行为，同时仍然适应浮点单元的非重新编码和重新编码实现。新的定义还通过在值被错误使用时传播 NaN 来帮助捕捉软件错误。

非重新编码实现会在每次浮点操作的输入和输出时，将操作数解包并打包为 IEEE 标准格式。对于非重新编码实现，NaN 装箱的代价主要在于检查较窄操作的高位是否表示一个合法的 NaN 装箱值，并在结果的高位写入所有 1。



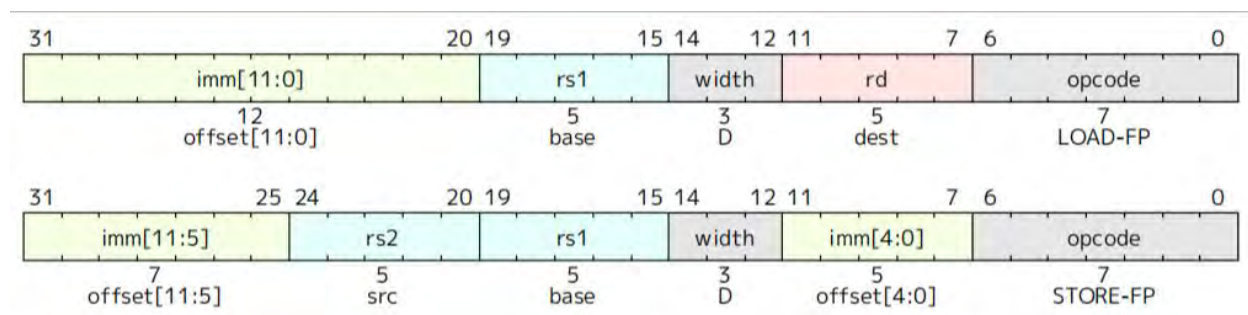
重新编码实现使用更方便的内部格式来表示浮点值，并增加了一个指数位以允许所有值保持标准化。重新编码实现的代价主要是需要额外的标签来跟踪内部类型和符号位，但这可以通过在指数字段内部重新编码 NaN 来完成，而不需要增加新的状态位。传输值进出重新编码格式的流水线需要做一些小的修改，但数据通路和延迟代价最小。无论如何，重新编码过程必须处理宽操作数的输入次正规值的移位，而提取 NaN 装箱值的过程与标准化类似，只是跳过前导 1 位而不是跳过前导 0 位，从而允许共享数据通路的多路选择器。

21.3 双精度加载和存储指令

FLD 指令将双精度浮点值从内存加载到浮点寄存器 rd 中。FSD 指令将浮点寄存器中的双精度值存储到内存。



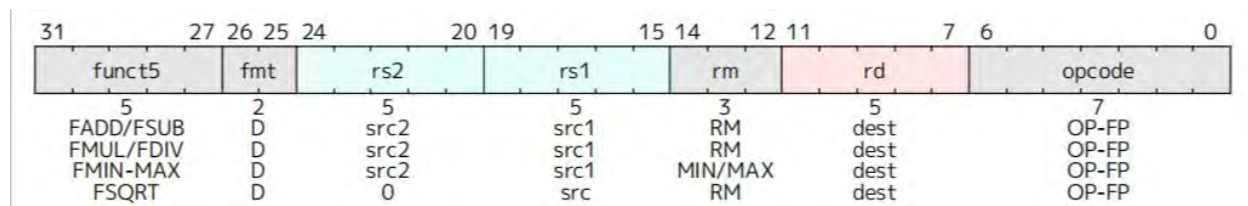
双精度值可以是一个 NaN 装箱的单精度值。

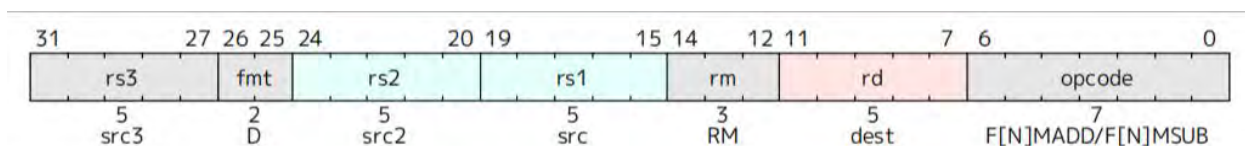


只有当有效地址自然对齐且 $XLEN \geq 64$ 时，FLD 和 FSD 才能保证原子性执行。FLD 和 FSD 不会修改被传输的位；特别地，非正规 NaN 的有效载荷会被保留。

21.4 双精度浮点计算指令

双精度浮点数计算指令的定义与其单精度对应指令类似，但它们操作的是双精度操作数，并生成双精度结果。

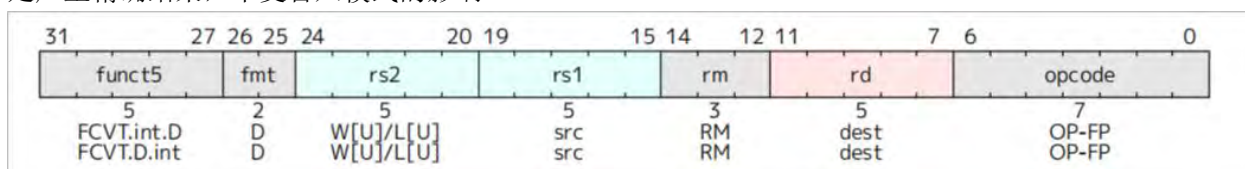




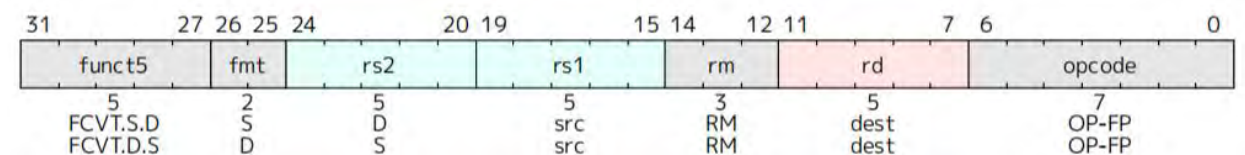
21.5 双精度浮点转换与移动指令

浮点数到整数和整数到浮点数的转换指令被编码在 OP-FP 主操作码空间中。FCVT.W.D 或 FCVT.L.D 分别将浮点寄存器 rs1 中的双精度浮点数转换为有符号 32 位或 64 位整数，并存储在整数寄存器 rd 中。FCVT.D.W 或 FCVT.D.L 分别将整数寄存器 rs1 中的 32 位或 64 位有符号整数转换为双精度浮点数，并存储在浮点寄存器 rd 中。FCVT.W.U.D、FCVT.L.U.D、FCVT.D.W.U 和 FCVT.D.L.U 变体用于在无符号整数值之间进行转换。对于 RV64，FCVT.W[U].D 对 32 位结果进行符号扩展。FCVT.L[U].D 和 FCVT.D.L[U] 是 RV64 专用指令。FCVT.int.D 的有效输入范围和无效输入的行为与 FCVT.int.S 相同。

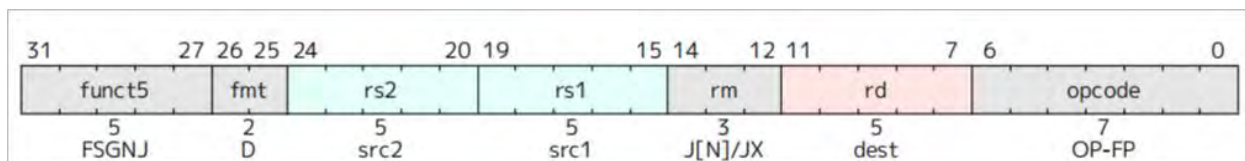
所有浮点数到整数和整数到浮点数的转换指令都根据 rm 字段进行舍入。请注意，FCVT.D.W[U] 总是产生精确结果，不受舍入模式的影响。



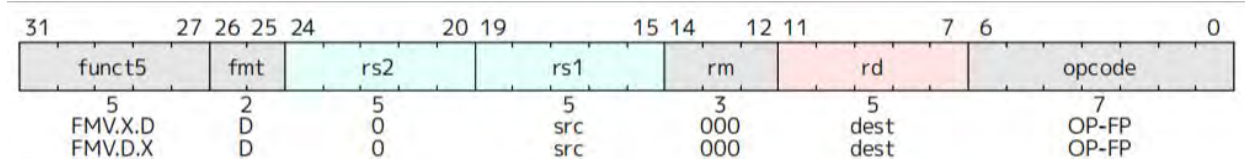
双精度到单精度和单精度到双精度的转换指令，FCVT.S.D 和 FCVT.D.S，编码在 OP-FP 主操作码空间中，源操作数和目标操作数都是浮点寄存器。rs2 字段编码源操作数的数据类型，fmt 字段编码目标操作数的数据类型。FCVT.S.D 根据 RM 字段进行舍入；FCVT.D.S 则从不进行舍入。



浮点数到浮点数的符号注入指令，FSGNJ.D、FSGNJN.D 和 FSGNJX.D，定义方式类似于单精度符号注入指令。



仅对于 XLEN≥64，提供了将位模式在浮点寄存器和整数寄存器之间移动的指令。FMV.X.D 将浮点寄存器 rs1 中的双精度值移动到整数寄存器 rd 中，采用 IEEE 754-2008 标准编码表示。FMV.D.X 将整数寄存器 rs1 中采用 IEEE 754-2008 标准编码的双精度值移动到浮点寄存器 rd 中。FMV.X.D 和 FMV.D.X 不会修改被传输的位；特别地，非标准 NaN 的有效载荷会被保留。



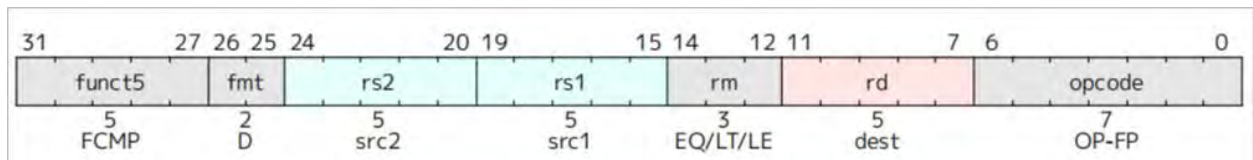


早期版本的 RISC-V 指令集架构 (ISA) 包含了额外的指令, 以允许 RV32 系统在 64 位浮点寄存器的上半部分和下半部分与整数寄存器之间进行传输。然而, 这些指令将是唯一具有部分寄存器写入的指令, 并且会在实现了重新编码浮点数或寄存器重命名的系统中增加复杂性, 需要一个流水线读取-修改-写入的序列。如果它们遵循这种模式, 扩展到处理四倍精度的 RV32 和 RV64 系统也需要额外的指令。该 ISA 的定义旨在减少显式的整数与浮点寄存器之间的传输, 通过让转换和比较指令将结果写入适当的寄存器文件, 因此我们预期这些指令的效益低于其他 ISA。

我们注意到, 对于实现了 64 位浮点单元, 包括融合乘加 (FMA) 支持以及 64 位浮点加载和存储的系统, 将数据路径从 32 位转换为 64 位整数的数据路径的边际硬件成本较低, 并且可以使用支持 32 位宽地址空间和指针的软件 ABI, 以避免静态数据和动态内存流量的增长。

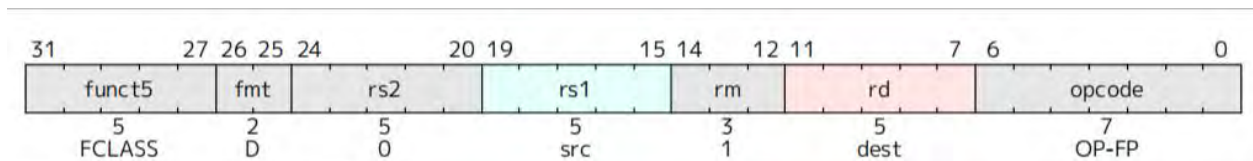
21.6 双精度浮点比较指令

双精度浮点数比较指令的定义与其单精度对应指令类似, 但它们操作的是双精度操作数。



21.7 双精度浮点分类指令

双精度浮点数分类指令 FCLASS.D 的定义与其单精度对应指令类似, 但它操作的是双精度操作数。

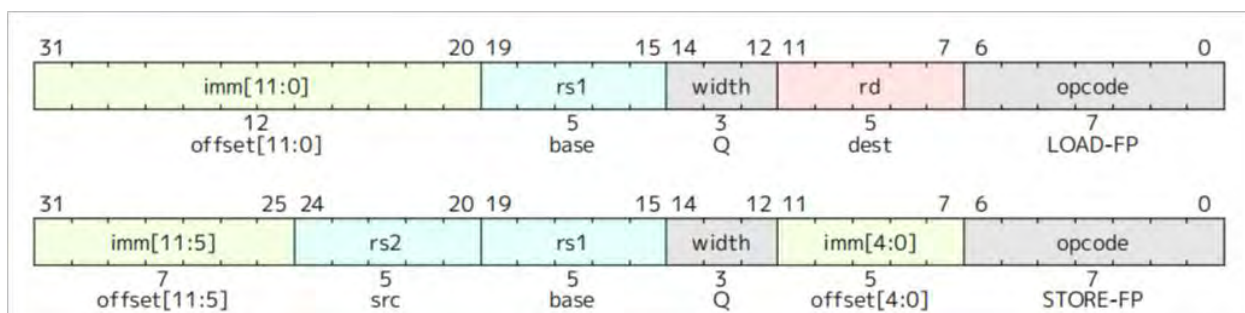


22 Q 扩展: 四精度浮点, V2.2

本章节描述了符合 IEEE 754-2008 算术标准的 128 位四精度二进制浮点指令的 Q 标准扩展。四精度二进制浮点指令集扩展被命名为 Q; 它依赖于双精度浮点扩展 D。浮点寄存器现在被扩展为可以存储单精度、双精度或四精度浮点值 (FLEN=128)。第 21.2 章中描述的 NaN 装箱方案现在递归扩展, 允许将单精度值嵌套在双精度值中的 NaN 装箱内, 而双精度值又嵌套在四精度值中的 NaN 装箱内。

22.1 四精度加载和存储指令

新增了 128 位版本的 LOAD-FP 和 STORE-FP 指令, 使用新的 funct3 宽度字段值进行编码。只有有效地址自然对齐且 XLEN=128 时, FLQ 和 FSQ 才能保证原子性执行。FLQ 和 FSQ 不会修改被传输的位; 特别地, 非标准 NaN 的有效载荷会被保留。



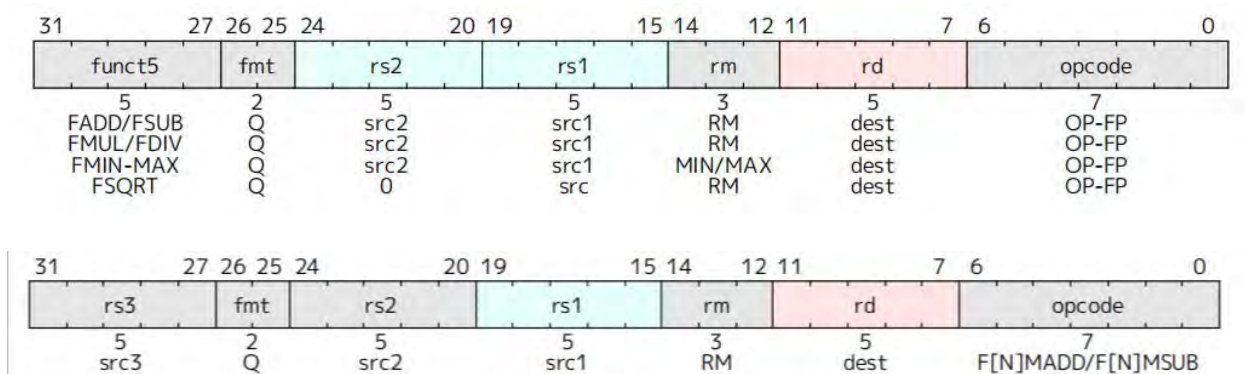
22.2 四精度计算指令

大多数指令的格式字段中新增了一种支持的新格式，如 22.1 所示。

表 22.1 格式字段编码

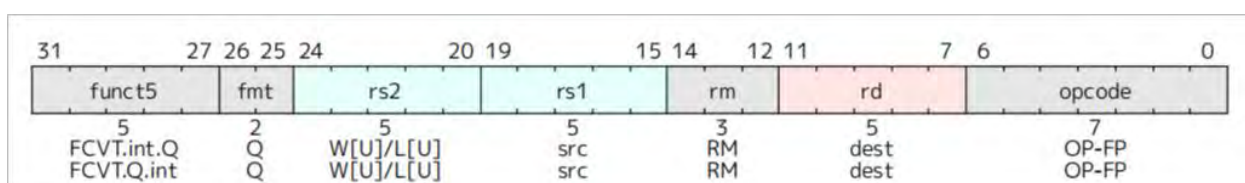
fmt field	助记符	意义
00	S	32 位单精度
01	D	64 位双精度
10	H	16 位半精度单位
11	Q	128 位四精度单位

四精度浮点数计算指令的定义与其双精度对应指令类似，但它们操作的是四精度操作数，并生成四精度结果。



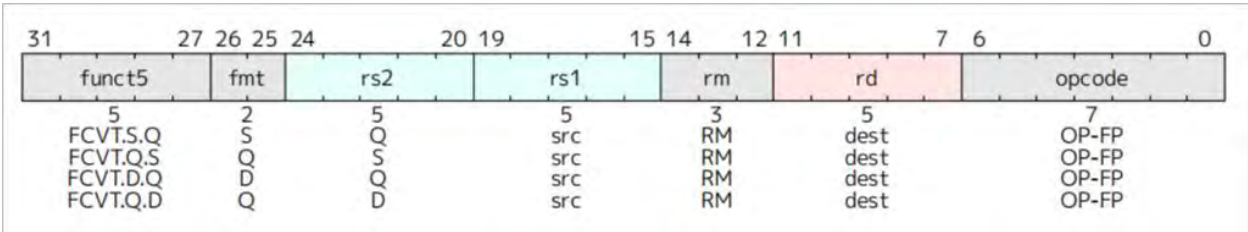
22.3 四精度转换与移动指令

新增了浮点数到整数和整数到浮点数的转换指令。这些指令的定义与双精度到整数和整数到双精度转换指令类似。FCVT.W.Q 或 FCVT.L.Q 分别将四精度浮点数转换为有符号 32 位或 64 位整数。FCVT.Q.W 或 FCVT.Q.L 分别将 32 位或 64 位有符号整数转换为四精度浮点数。FCVT.WU.Q、FCVT.LU.Q、FCVT.Q.WU 和 FCVT.Q.LU 变体用于在无符号整数值之间进行转换。FCVT.L[U].Q 和 FCVT.Q.L[U] 是 RV64 专用指令。请注意，FCVT.Q.L[U] 总是产生精确结果，不受舍入模式的影响。



新增了浮点数到浮点数的转换指令。这些指令的定义与双精度浮点数到浮点数转换指令类似。FCVT.S.Q 或 FCVT.Q.S 分别将四精度浮点数转换为单精度浮点数，或将单精度浮点数转换为四精度浮点数。FCVT.D.Q 或 FCVT.Q.D 分别将四精度浮点数转换为双精度浮点数，或将双精度浮点数转换为四精度浮点数。

浮点数到浮点数的符号注入指令 FSGNJ.Q、FSGNJN.Q 和 FSGNJX.Q 的定义方式与双精度符号注入指令类似。



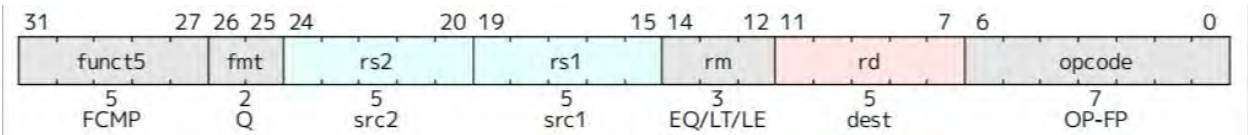
RV32 和 RV64 中均未提供 FMV.X.Q 和 FMV.Q.X 指令，因此必须通过内存将四精度位模式传输到整数寄存器中。



RV128 将在 Q 扩展中支持 FMV.X.Q 和 FMV.Q.X。

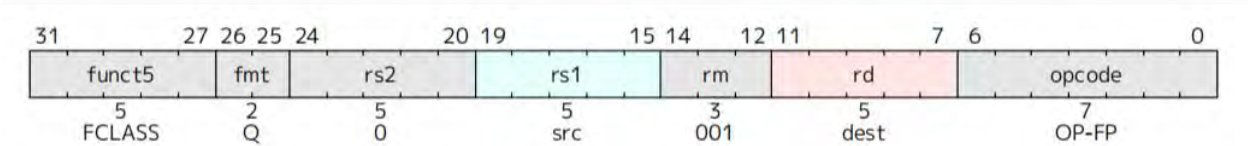
22.4 四精度浮点数比较指令

四精度浮点数比较指令的定义与其双精度对应指令类似，但它们操作的是四精度操作数。



22.5 四精度浮点数分类指令

四精度浮点数分类指令 FCLASS.Q 的定义方式与其双精度对应指令类似，但它们操作的是四精度操作数。



23 Zfh 和 Zfhmin 扩展：用于半精度浮点，V1.0

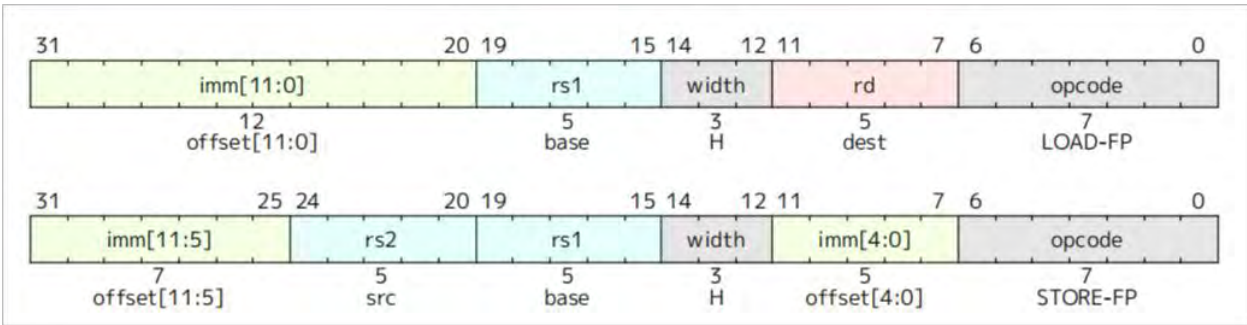
本章描述了 Zfh 标准扩展，用于支持符合 IEEE 754-2008 算术标准的 16 位半精度二进制浮点指令。Zfh 扩展依赖于单精度浮点扩展 F。第 21. 章中描述的 NaN-装箱 (NaN-boxing) 方案得到了扩展，以允许将半精度值使用 NaN-装箱在单精度值内（当 D 或 Q 扩展可用时，该值还可以递归地作为 NaN-装箱在双精度或四精度值中）。



此扩展主要提供处理半精度操作数并产生半精度结果的指令。然而，使用更高精度的中间值来计算半精度数据也是常见的做法。虽然该扩展提供了显式的转换指令，可以实现这种模式，但未来的扩展可能会通过额外的指令进一步加速此类计算，这些指令可以隐式地拓宽操作数。

23.1 半精度加载和存储指令

新增了 16 位版本的 LOAD-FP 和 STORE-FP 指令，这些指令使用了 funct3 宽度字段的新值进行编码。只有当有效地址自然对齐时，FLH 和 FSH 才保证原子性执行。FLH 和 FSH 不修改被传输的位；特别地，非规范 NaN 的有效载荷被保留。FLH 将结果写入 rd 时会进行 NaN-装箱，而 FSH 则忽略 rs2 中除了最低 16 位之外的所有位。



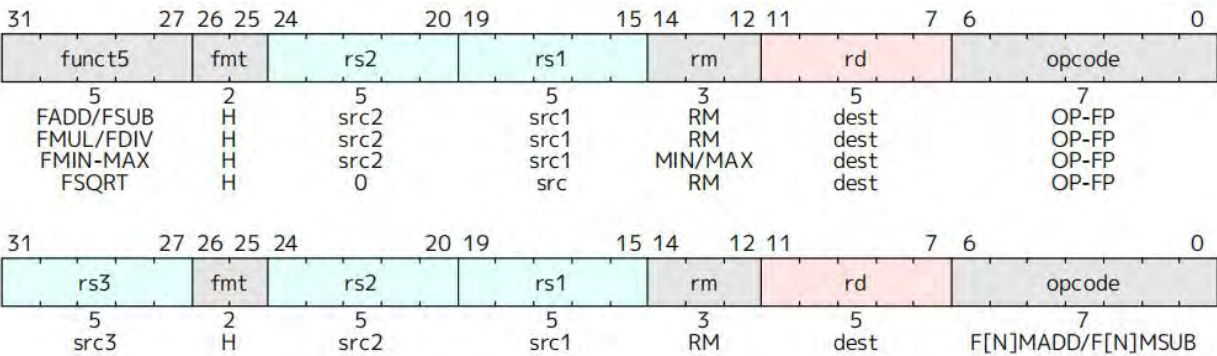
23.2 半精度计算指令

在大多数指令的格式字段中添加了一个新的支持格式，如表 23.1 所示。

表 23.1 格式字段编码

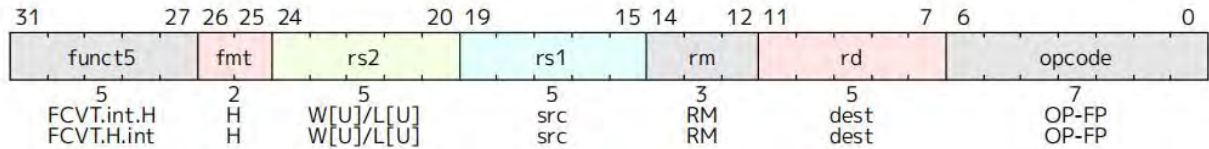
fmt field	简称	含义
00	S	32位单精度
01	D	64位双精度
10	H	16位半精度
11	Q	128位四精度

半精度浮点计算指令的定义类似于它们的单精度对应指令，但操作的是半精度操作数并产生半精度结果。

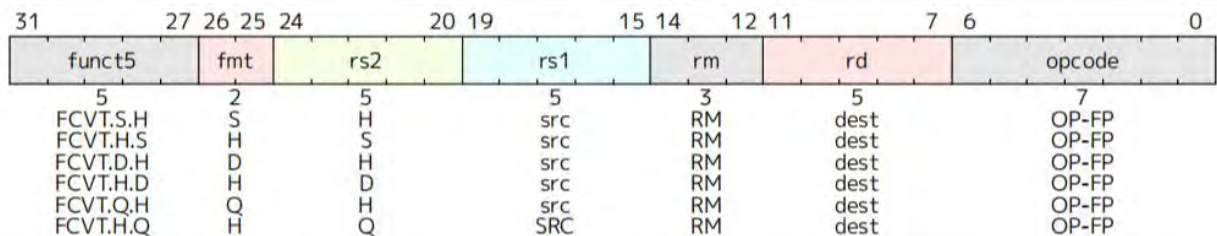


23.3 半精度转换和移动指令

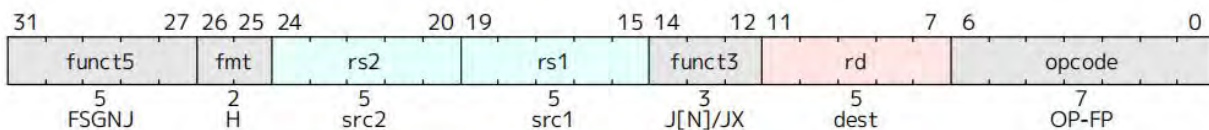
新增了浮点数与整数之间的转换指令。这些指令的定义类似于单精度到整数和整数到单精度的转换指令。FCVT.W.H 或 FCVT.L.H 分别将半精度浮点数转换为有符号 32 位或 64 位整数。FCVT.H.W 或 FCVT.H.L 分别将 32 位或 64 位有符号整数转换为半精度浮点数。FCVT.W.U.H、FCVT.L.U.H、FCVT.H.W.U 和 FCVT.H.L.U 变体用于转换无符号整数值。FCVT.L[U].H 和 FCVT.H.L[U] 是仅限 RV64 的指令。



新增了浮点数到浮点数的转换指令。这些指令的定义类似于双精度浮点数到浮点数的转换指令。FCVT.S.H 或 FCVT.H.S 分别将半精度浮点数转换为单精度浮点数，反之亦然。如果 D 扩展可用，FCVT.D.H 或 FCVT.H.D 将半精度浮点数转换为双精度浮点数，反之亦然。如果 Q 扩展可用，FCVT.Q.H 或 FCVT.H.Q 将半精度浮点数转换为四精度浮点数，反之亦然。



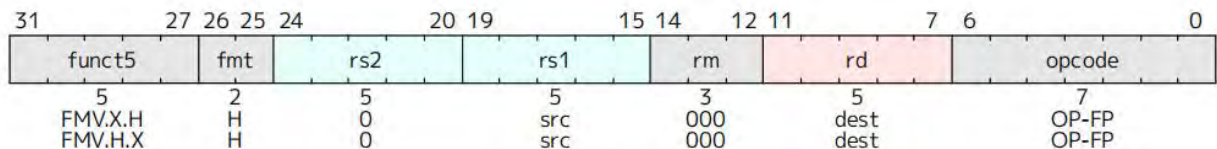
浮点到浮点的符号注入指令 FSGNJ.H、FSGNJN.H 和 FSGNJX.H 的定义类似于单精度符号注入指令。



提供了指令用于在浮点寄存器和整数寄存器之间移动位模式。FMV.X.H 将浮点寄存器 rs1 中的半精度值移动到整数寄存器 rd 中，并以 IEEE 754-2008 标准编码表示，填充高 XLEN-16 位为浮点数符号位的副本。

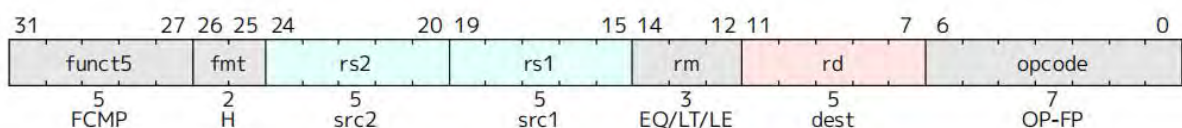
FMV.H.X 将整数寄存器 rs1 中低 16 位以 IEEE 754-2008 标准编码表示的半精度值移动到浮点寄存器 rd 中，并对结果进行 NaN-装箱。

FMV.X.H 和 FMV.H.X 不修改被传输的位；特别地，非规范 NaN 的有效载荷被保留。



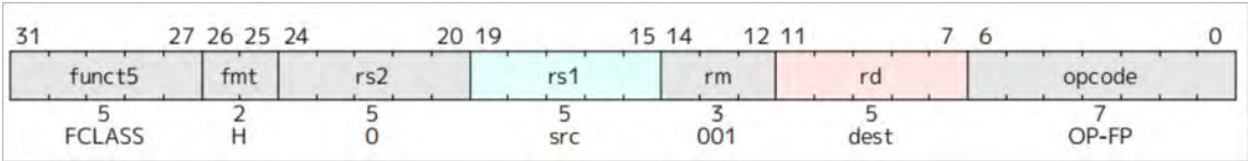
23.4 半精度浮点比较指令

半精度浮点比较指令的定义类似于它们的单精度对应指令，但操作的是半精度操作数。



23.5 半精度浮点分类指令

半精度浮点分类指令 FCLASS.H 的定义类似于其单精度对应指令，但操作的是半精度操作数。



23.6 Zfhmin 标准扩展：用于最小半精度浮点

本节描述了 Zfhmin 标准扩展，它为 16 位半精度二进制浮点指令提供了最小的支持。Zfhmin 扩展是 Zfh 扩展的一个子集，仅包括数据传输和转换指令。与 Zfh 一样，Zfhmin 扩展依赖于单精度浮点扩展 F。预期 Zfhmin 软件主要使用半精度格式进行存储，并在更高精度下执行大多数计算。

Zfhmin 扩展包括来自 Zfh 扩展的以下指令：FLH、FSH、FMV.X.H、FMV.H.X、FCVT.S.H 和 FCVT.H.S。如果 D 扩展可用，FCVT.D.H 和 FCVT.H.D 指令也包括在内。如果 Q 扩展可用，FCVT.Q.H 和 FCVT.H.Q 指令也会被包含在内。

Zfhmin 不包括 FSGNJ.H 指令，因为使用 FSGNJ.S 指令即可满足将半精度值在浮点寄存器之间移动的需求。



半精度加法、减法、乘法、除法和平方根操作可以通过将半精度操作数转换为单精度，使用单精度算术执行操作后再转换回半精度来精确地模拟。使用这种方法执行半精度融合乘加操作时，对于某些输入，RNE 和 RMM 舍入模式会引入 1-ulp 的误差。

从 8 位或 16 位整数到半精度的转换可以通过先转换为单精度，然后再转换为半精度来模拟。从 32 位整数的转换可以通过先转换为双精度来模拟。如果没有 D 扩展，并且在 RNE 或 RMM 模式下 1-ulp 的误差是可以接受的，则 32 位整数也可以先转换为单精度。没有 Q 扩展时，从 64 位整数的转换也适用相同的说明。

24 Zfa 扩展：额外浮点指令，V1.0

本章描述了 Zfa 标准扩展，该扩展增加了立即数加载指令、IEEE 754-2019 最小值和最大值操作、舍入到整数操作（round-to-integer operations）以及静默浮点数比较指令（quiet floating-point comparisons）。对于 RV32D，Zfa 扩展还增加了将双精度浮点值与整数寄存器之间传输的指令；对于 RV64Q，则增加了类似的指令，用于四精度浮点值。Zfa 扩展依赖于 F 扩展。

24.1 立即数加载指令

FLI.S 指令将编码在 rs1 字段中的 32 个单精度浮点常量之一加载到浮点寄存器 rd 中。rs1 字段值与单精度浮点值的对应关系如表 24.1 所示。FLI.S 的编码方式类似于 FMV.W.X，但 rs2=1。

表 24.1 FLI.S 指令加载的立即数值

Rs1	值	标志	指数
0	-1.0	1	01111111
1	最小值	0	00000001
2	1.0×2^{-16}	0	01101111

3	1.0×2^{-15}	0	01110000
4	1.0×2^{-8}	0	01110111
5	1.0×2^{-7}	0	01111000
6	0.0625 (2-42-4)	0	01111011
7	0.125 (2-32-3)	0	01111100
8	0.25	0	01111101
9	0.3125	0	01111101
10	0.375	0	01111101
11	0.4375	0	01111101
12	0.5	0	01111110
13	0.625	0	01111110
14	0.75	0	01111110
15	0.875	0	01111110
16	1.0	0	01111111
17	1.25	0	01111111
18	1.5	0	01111111
19	1.75	0	01111111
20	2.0	0	10000000
21	2.5	0	10000000
22	3	0	10000000
23	4	0	10000001
24	8	0	10000010
25	16	0	10000011
26	128 (2727)	0	10000110
27	256 (2828)	0	10000111
28	215	0	10001110
29	216	0	10001111
30	$+\infty$	0	11111111
31	Canonical NaN	0	11111111



条目 1、30 和 31 的首选汇编语法分别是 `min`、`inf` 和 `nan`。对于条目 0 到 29（包括条目 1），汇编器将接受 C 类语法中的十进制常量。



这 32 个常量的集合是通过检查浮点库（包括 C 标准数学库）并优化定点到浮点数的转换后选定的。条目 8 到 22 遵循一个规律的编码模式。没有任何条目会设置除最高有效位之外的尾数位。

如果实现了 D 扩展，FLI.D 执行类似的操作，但将双精度浮点值加载到浮点寄存器 `rd` 中。请注意，条目 1（对应最小正规范值）在双精度中与单精度中的数值不同。FLI.D 的编码方式与 FLI.S 相似，但 `fmt=D`。

如果实现了 Q 扩展, FLI.Q 执行类似的操作, 但将四精度浮点值加载到浮点寄存器 rd 中。请注意, 条目 1 (对应最小正规值) 在四精度中与其他精度中的数值不同。FLI.Q 的编码方式与 FLI.S 相似, 但 $\text{fmt}=\text{Q}$ 。

如果实现了 Zfh 或 Zvfh 扩展, FLI.H 执行类似的操作, 但将半精度浮点值加载到寄存器 rd 中。请注意, 条目 1 (对应最小正规值) 在半精度中与其他精度中的数值不同。此外, 由于 2^{-16} 不能在半精度浮点数中表示, 表中的条目 29 改为加载正无穷——即, 它与条目 30 是冗余的。FLI.H 的编码方式与 FLI.S 相似, 但 $\text{fmt}=\text{H}$ 。



另外, 由于 2^{-16} 和 2^{-15} 在半精度下是次正规数, 因此在 FLI.H 中, 第 1 项的数值大于第 2 项和第 3 项。

FLI.fmt 指令永远不会设置任何浮点异常标志。

24.2 最小值和最大值指令

FMINM.S 和 FMAXM.S 指令的定义与 FMIN.S 和 FMAX.S 指令类似, 不同之处在于, 如果任一输入是 NaN, 结果将是规范的 NaN。如果实现了 D 扩展, FMINM.D 和 FMAXM.D 指令也类似地定义为作用于双精度数。如果实现了 Zfh 扩展, FMINM.H 和 FMAXM.H 指令也类似地定义为作用于半精度数。如果实现了 Q 扩展, FMINM.Q 和 FMAXM.Q 指令也类似地定义为作用于四倍精度数。这些指令的编码方式与 FMIN 和 FMAX 指令相同, 只是指令的第 13 位设置为 1。



这些指令实现了 IEEE 754-2019 的最小值和最大值操作。

24.3 四舍五入到整数指令

FROUND.S 指令将浮点寄存器 rs1 中的单精度浮点数根据指令的 rm 字段指定的舍入模式四舍五入为一个整数。然后, 它将该整数作为单精度浮点数写入浮点寄存器 rd。零和无穷大的输入会原样复制到 rd 中。信号 NaN 输入会导致设置无效操作异常标志; 不会设置其他异常标志。FROUND.S 的编码方式与 FCVT.S.D 相似, 但 $\text{rs2} = 4$ 。

FROUNDX.S 指令的定义类似, 但如果输入与舍入结果不同并且不是 NaN, 它还会设置不精确异常标志。FROUNDX.S 的编码方式与 FCVT.S.D 相似, 但 $\text{rs2} = 5$ 。

如果实现了 D 扩展, FROUND.D 和 FROUNDX.D 指令类似地定义为操作双精度浮点数。它们的编码方式与 FCVT.D.S 相似, 但 rs2 分别为 4 和 5。

如果实现了 Zfh 扩展, FROUND.H 和 FROUNDX.H 指令类似地定义为操作半精度浮点数。它们的编码方式与 FCVT.H.S 相似, 但 rs2 分别为 4 和 5。

如果实现了 Q 扩展, FROUND.Q 和 FROUNDX.Q 指令类似地定义为操作四精度浮点数。它们的编码方式与 FCVT.Q.S 相似, 但 rs2 分别为 4 和 5。



FROUNDX.fmt 指令实现了 IEEE 754-2019 的 roundToIntegralExact 操作, 而 FROUND.fmt 指令实现了 roundToIntegral 系列中的其他操作。

24.4 模块化向整数转换指令

FCVTMOD.W.D 指令的定义类似于 FCVT.W.D 指令,具有以下不同之处。FCVTMOD.W.D 总是向零舍入。位 31:0 来自舍入后的无界二进制补码结果,然后进行符号扩展至 XLEN 位,并写入整数寄存器 rd。infty 和 NaN 会被转换为零。浮点异常标志的设置与 FCVT.W.D 指令对相同输入操作数时的设置相同。只有在实现了 D 扩展时,才提供此指令。

其编码方式与 FCVT.W.D 相似,但 rs2 字段设置为 8,rm 字段设置为 1(RTZ)。其他 rm 值是保留的。



汇编语法要求显式指定 RTZ 舍入模式,即 fcvtmod.w.d rd, rs1, rtz。FCVTMOD.W.D 指令的添加主要是为了加速 JavaScript 数字的处理。数字是双精度值,但某些运算符会隐式将其截断为符号整数,模 $2^{\{32\}}$ 。

24.5 移动指令

仅对于 RV32,如果实现了 D 扩展,FMVH.X.D 指令将浮点寄存器 rs1 的 63:32 位移动到整数寄存器 rd。它在 OP-FP 主操作码中进行编码,funct3=0,rs2=1,funct7=1110001。



FMVH.X.D 指令与现有的 FMV.X.W 指令配合使用,将双精度浮点数移动到一对 x 寄存器中。

仅对于 RV32,如果实现了 D 扩展,FMVP.D.X 指令将一对整数寄存器中的双精度数移动到浮点寄存器中。整数寄存器 rs1 和 rs2 分别提供 31:0 位和 63:32 位,结果写入浮点寄存器 rd。FMVP.D.X 在 OP-FP 主操作码中进行编码,funct3=0,funct7=1011001。

仅对于 RV64,如果实现了 Q 扩展,FMVH.X.Q 指令将浮点寄存器 rs1 的 127:64 位移动到整数寄存器 rd。它在 OP-FP 主操作码中进行编码,funct3=0,rs2=1,funct7=1110011。



FMVH.X.Q 指令与现有的 FMV.X.D 指令配合使用,将四倍精度浮点数移动到一对 x 寄存器中。

仅对于 RV64,如果实现了 Q 扩展,FMVP.Q.X 指令将一对整数寄存器中的双精度数移动到浮点寄存器中。整数寄存器 rs1 和 rs2 分别提供 63:0 位和 127:64 位,结果写入浮点寄存器 rd。FMVP.Q.X 在 OP-FP 主操作码中进行编码,funct3=0,funct7=1011011。

24.6 比较指令

FLEQ.S 和 FLTQ.S 指令的定义类似于 FLE.S 和 FLT.S 指令,不同之处在于,静默 NaN 输入不会导致设置无效操作异常标志。如果实现了 D 扩展,FLEQ.D 和 FLTQ.D 指令类似地定义为操作双精度浮点数。如果实现了 Zfh 扩展,FLEQ.H 和 FLTQ.H 指令类似地定义为操作半精度浮点数。如果实现了 Q 扩展,FLEQ.Q 和 FLTQ.Q 指令类似地定义为操作四精度浮点数。这些指令的编码方式与它们的 FLE 和 FLT 对应指令相似,但指令位 14 设置为 1。



我们不预计会向矢量 ISA 添加类似的比较指令,因为它们可以通过掩码操作合理高效地模拟。

25 ZfinxZdinxZhinxZhinxmin 扩展: 整数寄存器中的浮点, V1.0

本章定义了 Zfinx 扩展（发音为 z-f-in-x）。该扩展提供了一系列指令，其功能与标准单精度浮点 F 扩展中的指令类似，但操作对象为通用整数寄存器 x，而非浮点寄存器 f。本章还定义了 Zdinx、Zhinx 和 Zhinxmin 扩展，它们为其他浮点精度提供了类似的指令支持。



F 扩展采用独立的 f 寄存器进行浮点运算，旨在减轻寄存器压力，并简化对宽超标量处理器提供寄存器堆端口的设计。然而，引入的额外架构会增加最低实现成本。Zfinx 扩展通过移除 f 寄存器，显著降低了支持浮点指令集的简单 RISC-V 实现的成本，同时也减少了上下文切换的开销。

一般而言，假定 F 扩展存在的软件与假定 Zfinx 扩展存在的软件互不兼容。

Zfinx 扩展添加了 F 扩展添加的所有指令，除了传输指令 FLW、FSW、FMV.W.X、FMV.X.W、C.FLW[SP] 和 C.FSW[SP]。



Zfinx 软件使用整数加载和存储将浮点值从内存传输到内存或从内存传输到寄存器。寄存器之间的传输使用整数算术操作或浮点符号注入指令。

这些 F 扩展指令的 Zfinx 变体具有相同的语义，只是当该指令本应访问 f 寄存器时，它改为访问具有相同编号的 x 寄存器。

Zfinx 扩展依赖于 Zicsr 扩展来访问控制状态寄存器。

25.1 窄位数值的处理

对于位宽为 w 且小于 XLEN 的浮点操作数，其值将占据 x 寄存器的第 0 到 w-1 位。针对 w 位操作数进行的浮点运算，将忽略操作数在 XLEN-1 到 w 位的数值。

产生 w 位（w < XLEN）浮点结果的操作，会将结果的符号位（即第 w-1 位）复制填充到 x 寄存器的 XLEN-1 到 w 位。



f 寄存器中采用的 NaN-装箱方案（NaN-boxing）旨在高效支持重编码浮点格式。然而，由于 Zfinx 扩展的寄存器需要同时存储浮点数和整数，重新编码对于 Zfinx 而言实用性降低，因此 NaN-装箱的需求也随之减弱。

在 RV64 架构中，将 32 位浮点数存储于 x 寄存器时进行符号扩展，这与现有的 RV64 调用约定相符——当通过 x 寄存器传递 32 位浮点值时，调用约定明确指出寄存器的第 32-63 位为 UNSPECIFIED 状态。为了保持架构设计的一致性，我们将此模式推广至 RV32 和 RV64 架构中的 16 位浮点数。

25.2 Zdinx

Zdinx 扩展提供了类似的双精度浮点指令。Zdinx 依赖于 Zfinx 扩展。Zdinx 扩展添加了 D 扩展添加的所有指令，除了传输指令 FLD、FSD、FMV.D.X、FMV.X.D、C.FLD[SP] 和 C.FSD[SP]。

这些 D 扩展指令的 Zdinx 变体具有相同的语义，除了每当这样的指令访问 f 寄存器时，它改为访问具有相同编号的 x 寄存器。

25.3 宽位数值的处理

在 RV32Zdinx 扩展中，双精度操作数需存储于对齐的 x 寄存器对中，即寄存器编号必须为偶数。使用未对齐的（奇数编号）寄存器存储双精度浮点操作数的行为是保留的。

无论采用何种字节序，寄存器编号较小的寄存器存储低位数据，编号较大的寄存器存储高位数据。例如，在 RV32Zdinx 中，双精度操作数的 0-31 位可能存储在 x14 寄存器中，而 32-63 位则存储在 x15 寄存器中。

当双精度浮点结果写入 x0 寄存器时，整个写入操作将无效。例如，在 RV32Zdinx 架构下，将双精度结果写入 x0 不会影响 x1 寄存器的内容。

当 x0 寄存器被用作双精度浮点操作数时，其值将被视为零，且指令执行过程中不会访问 x1 寄存器。



注：扩展规范中未提供成对加载/存储指令。因此，在 RV32Zdinx 扩展中，从内存加载或向内存存储双精度操作数，需要执行两次独立的加载或存储操作。然而，寄存器之间的数据移动仅需一条 FSGNJ.D 指令即可完成。

25.4 Zhinx

Zhinx 扩展提供了与之类似的半精度浮点指令集，且 Zhinx 扩展依赖于 Zfinx 扩展。Zhinx 扩展添加了 Zfh 扩展添加的所有指令，除了传输指令 FLH、FSH、FMV.H.X 和 FMV.X.H。

Zfh 扩展指令的 Zhinx 版本，在指令语义上与 Zfh 扩展保持一致，不同之处仅在于，当指令原本需要访问 f 寄存器时，Zhinx 版本会转而访问相同编号的 x 寄存器。

25.5 hinxmin

Zhinxmin 扩展旨在为在 x 寄存器上进行的 16 位半精度浮点运算提供最基本的支持。Zhinxmin 扩展依赖于 Zfinx 扩展。

Zhinxmin 扩展仅包含 Zhinx 扩展中的 FCVT.S.H 和 FCVT.H.S 指令。如果系统中同时存在 Zdinx 扩展，则 Zhinxmin 扩展还会额外包含 FCVT.D.H 和 FCVT.H.D 指令。



未来，可以参照 RV32Zdinx 扩展的模式，定义 RV64Zqinx 四精度扩展。此外，亦可考虑定义 RV32Zqinx 扩展，但这将需要采用四倍字寄存器组合来存储四精度浮点数。

25.6 特权架构的影响

在卷 II 定义的标准特权架构中，若实现了 Zfinx 扩展，则 mstatus 寄存器的 FS 字段将被硬连接为 0。此时，FS 字段将不再对浮点指令或 fcsr 访问的陷阱行为产生影响。

当 Zfinx 扩展被实现后，misa 寄存器中的 F、D 和 Q 位也将被硬连接置 0。



未来或将引入某种探测机制，用于检测 Zfinx、Zhinx 和 Zdinx 扩展是否存在于系统中。

26 C 扩展：压缩指令，V2.0

本章描述了 RISC-V 标准压缩指令集扩展，称为 C，通过为常见操作添加简短的 16 位指令编码来减少静态和动态代码大小。C 扩展可以添加到任何基本 ISA（RV32、RV64、RV128），我们使用通用术语 RVC 来涵盖这些基础架构。通常，程序中 50%-60% 的 RISC-V 指令可以替换为 RVC 指令，从而实现减少 25%-30% 的代码量。

26.1 概述

RVC 使用了一种简单的压缩方案,当满足以下条件时,提供常见 32 位 RISC-V 指令的简短 16 位版本:

- * 立即数或地址偏移量较小
- * 其中一个寄存器是零寄存器 (x0)、ABI 链接寄存器 (x1) 或 ABI 堆栈指针 (x2)
- * 目标寄存器和第一个源寄存器相同
- * 使用的寄存器是最常用的 8 个寄存器之一

C 扩展与所有其他标准指令扩展兼容。C 扩展允许 16 位指令与 32 位指令自由混合使用,且后者现在可以在任何 16 位边界上开始,即 IALIGN=16。随着 C 扩展的加入,不再有指令可以引发指令地址未对齐异常 (instruction-address-misaligned exceptions)。



取消对初始 32 位指令的 32 位对齐约束可以显著提高代码密度。

压缩指令编码在 RV32C、RV64C 和 RV128C 之间大多是通用的,但如 rvc-instr-table0, Table 34 所示,一些操作码根据基本 ISA 的不同而有不同的用途。例如,具有更宽地址空间的 RV64C 和 RV128C 变体需要额外的操作码来压缩 64 位整数值的加载和存储,而 RV32C 使用相同的操作码来压缩单精度浮点值的加载和存储。同样,RV128C 需要额外的操作码来处理 128 位整数值的加载和存储,而这些相同的操作码在 RV32C 和 RV64C 中用于加载和存储双精度浮点值。如果实现了 C 扩展,则必须在实现相关标准浮点扩展 (F 和/或 D) 时提供适当的压缩浮点加载和存储指令。此外, RV32C 包括一个压缩跳转和链接指令,用于压缩短距离子程序调用,而相同的操作码在 RV64C 和 RV128C 中用于压缩 ADDIW 指令。

双精度加载和存储是静态和动态指令的重要组成部分,因此有必要将它们包含于 RV32C 和 RV64C 编码中。



虽然单精度加载和存储指令在目前支持的 ABI 下编译的基准测试中不是静态或动态压缩的重要来源,但对于仅提供硬件单精度浮点单元并且具有仅支持单精度浮点数的 ABI 的微控制器而言,单精度加载和存储指令的使用频率将至少与双精度加载和存储指令相当。因此,有必要在 RV32C 中提供对这些指令的压缩支持。

短距离子程序调用在微控制器的小型二进制文件中更常见,因此有必要将它们包含于 RV32C 中。

尽管为不同的基础 ISA 重用操作码会增加一些文档的复杂性,但即使是支持多个基本 ISA 的设计,实施复杂性的影响也很小。压缩的浮点加载和存储变体与宽整数加载和存储指令使用相同的指令格式,并具有相同的寄存器指定符。

RVC 是在以下约束下设计的: 每条 RVC 指令扩展为基础 ISA (RV32I/E、RV64I/E 或 RV128I) 或存在的 F 和 D 标准扩展中的单个 32 位指令。采用这一约束有两个主要好处:

- * 硬件设计可以在解码阶段简单地扩展 RVC 指令,从而简化验证并最小化对现有微架构的修改。
- * 编译器可以对 RVC 扩展不知情,将代码压缩留给汇编器和链接器来处理,尽管具有压缩感知功能的编译器通常能够产生更好的结果。



我们认为 C 和基础 IFD 指令之间简单的一对一映射所带来的多重复杂性减少,远远超过了仅在 C 扩展中支持的额外指令,或允许在一个 C 指令中编码多个 IFD 指令的稍微更密集编码的潜在收益。

需要注意的是, C 扩展并不是一个在设计上独立的 ISA,而是与基本 ISA 一起使用。

长期以来,可变长度指令集一直用于提高代码密度。例如,IBM Stretch 于 1950 年代后期开发具有 32 位和 64 位指令集,其中一些 32 位指令是完整 64 位指令的压缩版本。Stretch 还采用了限制某些较短指令格式中可寻址寄存器集的概念,使用短跳转指令,这些指令只能引用其中一个索引寄存器。后来的 IBM 360 架构支持简单的可变长度指令编码,具有 16 位、32 位或 48 位指令格式。

1963 年, CDC 推出了 Cray 设计的 CDC 6600,这是 RISC 架构的前身,引入了一个寄存器丰富的加载-存储架构,具有两种长度的指令,15 位和 30 位。后来的 Cray-1 设计使用了非常相似的指令格式,具有 16 位和 32 位指令长度。

1980 年代的初始 RISC ISA 都选择了性能优先于代码大小,这对于工作站环境是合理的,但对于嵌入式系统则不然。因此,ARM 和 MIPS 随后推出了版本,通过提供一种 16 位宽的替代指令集,而不是标准的 32 位宽指令,以实现更小的代码大小。这些压缩 RISC ISA 相对于其起点减少了大约 25%-30% 的代码大小,生成的代码显著小于 80x86。这一结果让一些人感到惊讶,因为他们的直觉是可变长度的 CISC ISA 应该比仅提供 16 位和 32 位格式的 RISC ISA 更小。



由于初始 RISC ISA 没有留下足够的操作码空间来包含这些未计划的压缩指令,因此它们被开发为完整的新 ISA。这意味着编译器需要为不同的压缩 ISA 编写不同的代码生成器。第一代压缩 RISC ISA 扩展(例如 ARM Thumb 和 MIPS16)仅使用固定的 16 位指令大小,这虽然很好的减少了静态代码的大小,但导致动态指令数的增加,从而导致性能低于初始固定宽度的 32 位指令大小。这导致了第二代压缩 RISC ISA 设计的发展,采用具有混合的 16 位和 32 位指令长度(例如 ARM Thumb2、microMIPS、PowerPC VLE),这样性能与纯 32 位指令相似,但显著地节省代码大小。不幸的是,这些不同代的压缩 ISA 彼此不兼容,也不兼容初始的未压缩 ISA,导致文档、实现和软件工具支持的复杂性显著增加。

在常用的 64 位 ISA 中,只有 PowerPC 和 microMIPS 目前支持压缩指令格式。令人惊讶的是,移动平台上最流行的 64 位 ISA (ARM v8) 不包括压缩指令格式,尽管静态代码大小和动态指令获取带宽是重要的指标。虽然静态代码大小在较大的系统中不是主要关注点,但在运行商业工作负载的服务器中,指令获取带宽可能是一个主要瓶颈,这些服务器通常具有较大的指令工作集。

受益于 25 年后的视角,RISC-V 设计时支持压缩指令,从一开始就留下足够的操作码空间,使 RVC 可以作为简单扩展添加到基本 ISA 上(以及许多其他扩展)。RVC 的理念是减少嵌入式应用的代码大小,并通过减少指令缓存未命中次数来提高所有应用的性能和能效。Waterman 表明,RVC 获取的指令位数减少了 25%-30%,从而减少了 20%-25% 的指令缓存未命中,或大致相当于将指令缓存大小加倍的性能影响。

26.2 压缩指令格式

表 26.1 显示了九种压缩指令格式。CR、CI 和 CSS 可以使用任何 32 个 RVI 寄存器,但 CIW、CL、CS、CA 和 CB 仅限于其中的 8 个。

表 26.2 列出了这些常用寄存器，对应于寄存器 x8 到 x15。请注意，有一个单独的加载和存储指令版本使用堆栈指针作为基地址寄存器，因为保存到堆栈和从堆栈恢复非常普遍，并且它们使用 CI 和 CSS 格式以允许访问所有 32 个数据寄存器。CIW 为 ADDI4SPN 指令提供了一个 8 位立即数。



RISC-V ABI 已更改，使得常用寄存器映射到寄存器 x8-x15。这通过使用一组连续自然对齐的寄存器编号来简化解压缩解码器，并且与仅有 16 个整数寄存器的 RV32E 和 RV64E 基本 ISA 兼容。

基于压缩寄存器的浮点加载和存储也分别使用 CL 和 CS 格式，八个寄存器映射到 f8 到 f15。



标准 RISC-V 调用约定将最常用的浮点寄存器映射到寄存器 f8 到 f15，这允许与整数寄存器编号相同的寄存器解压缩解码。

这些格式的设计目的是在所有指令中将两个寄存器源说明符的位保持在相同位置，而目标寄存器字段可以移动。当存在完整的 5 位目标寄存器说明符时，它位于与 32 位 RISC-V 编码相同的位置。当立即数是符号扩展时，符号扩展始终来自第 12 位。立即数字段已被打乱，如基本规范中所述，以减少所需的立即数多路复用器数量。



立即数字段在指令格式中被打乱而不是按顺序排列，以便尽可能多的位在每条指令中都处于相同位置，从而简化实现。

对于许多 RVC 指令，不允许零值立即数，并且 x0 不是有效的 5 位寄存器说明符。这些限制为其他需要较少操作数位的指令腾出编码空间。

表 26.1 压缩的 16 位 RVC 指令格式

Format	Meaning	15 14 13 12		11 10 9 8 7			6 5 4 3 2			1 0
CR	Register	funct4		rd/rs1			rs2			op
CI	Immediate	funct3	imm	rd/rs1			imm			op
CSS	Stack-relative Store	funct3	imm				rs2			op
CIW	Wide Immediate	funct3	imm					rd'	op	
CL	Load	funct3	imm		rs1'	imm	rd'	op		
CS	Store	funct3	imm		rs1'	imm	rs2'	op		
CA	Arithmetic	funct6			rd'/rs1'	funct2	rs2'	op		
CB	Branch/Arithmetic	funct3	offset		rd'/rs1'	offset			op	
CJ	Jump	funct3	jump target						op	

表 26.2 由 CIW、CL、CS、CA 和 CB 格式的三位 rs1'、rs2' 和 rd' 字段指定的寄存器

RVC Register Number
Integer Register Number
Integer Register ABI Name
Floating-Point Register Number
Floating-Point Register ABI Name

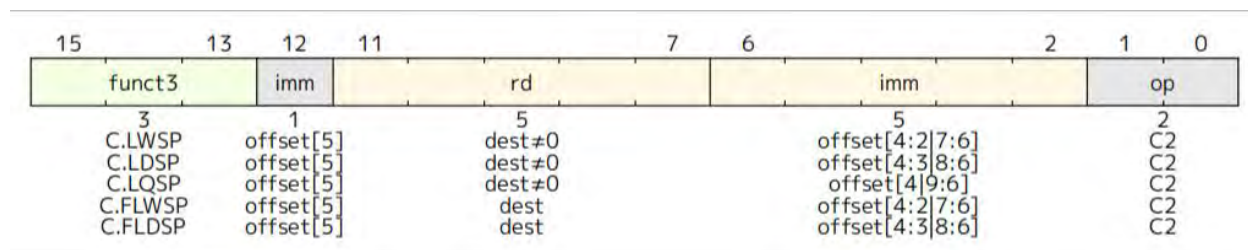
000	001	010	011	100	101	110	111
x8	x9	x10	x11	x12	x13	x14	x15
s0	s1	a0	a1	a2	a3	a4	a5
f8	f9	f10	f11	f12	f13	f14	f15
fs0	fs1	fa0	fa1	fa2	fa3	fa4	fa5

26.3 加载和存储指令

为了增加 16 位指令的覆盖范围,数据传输指令对以字节为单位的数据大小的零扩展立即数进行缩放: $\times 4$ 用于字, $\times 8$ 用于双字, $\times 16$ 用于四字。

RVC 提供了两种加载和存储变体。一种使用 ABI 堆栈指针 x2 作为基地址,并且可以针对任何数据寄存器。另一种可以引用 8 个基地址寄存器之一和 8 个数据寄存器之一。

26.3.1 基于堆栈指针的加载和存储



这些指令使用 CI 格式。

C.LWSP 从内存加载一个 32 位值到寄存器 rd。它通过将乘 4 之后的零扩展偏移量与堆栈指针 x2 相加来计算有效地址。它扩展为 `lw rd, offset(x2)`。当 `rd=x0` 时, C.LWSP 才有效, `rd=x0` 的代码点保留。

C.LDSP 是 RV64C/RV128C 专用指令,从内存加载一个 64 位值到寄存器 rd。它通过将乘 8 之后的零扩展偏移量与堆栈指针 x2 相加来计算有效地址。它扩展为 `ld rd, offset(x2)`。当 `rd=x0` 时, C.LDSP 才有效, `rd=x0` 的代码点保留。

C.LQSP 是 RV128C 专用指令,从内存加载一个 128 位值到寄存器 rd。它通过乘 16 之后的零扩展偏移量与堆栈指针 x2 相加来计算有效地址。它扩展为 `lq rd, offset(x2)`。当 `rd $\neq$ x0` 时, C.LQSP 才有效, `rd=x0` 的代码点保留。

C.FLWSP 是 RV32FC 专用指令,从内存加载一个单精度浮点值到浮点寄存器 rd。它通过将乘 4 之后的零扩展偏移量与堆栈指针 x2 相加来计算有效地址。它扩展为 `flw rd, offset(x2)`。

C.FLDSP 是 RV32DC/RV64DC 专用指令,从内存加载一个双精度浮点值到浮点寄存器 rd。它通过将乘 8 之后的零扩展偏移量与堆栈指针 x2 相加来计算有效地址。它扩展为 `fld rd, offset(x2)`。

这些指令使用 CSS 格式。

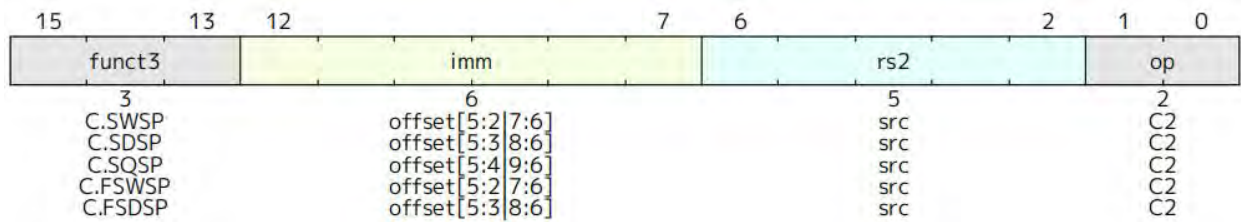
C.SWSP 将寄存器 rs2 中的 32 位值存储到内存中。它通过将乘 4 之后的零扩展偏移量与堆栈指针 x2 相加来计算有效地址。它扩展为 `sw rs2, offset(x2)`。

C.SDSP 是 RV64C/RV128C 专用指令,将寄存器 rs2 中的 64 位值存储到内存中。它通过将乘 8 之后的零扩展偏移量与堆栈指针 x2 相加来计算有效地址。它扩展为 `sd rs2, offset(x2)`。

C.SQSP 是 RV128C 专用指令,将寄存器 rs2 中的 128 位值存储到内存中。它通过将乘 16 之后的零扩展偏移量与堆栈指针 x2 相加来计算有效地址。它扩展为 `sq rs2, offset(x2)`。

C.FSWSP 是 RV32FC 专用指令,将浮点寄存器 rs2 中的单精度浮点值存储到内存中。它通过将乘 4 之后的零扩展偏移量与堆栈指针 x2 相加来计算有效地址。它扩展为 `fsw rs2, offset(x2)`。

C.FSDSP 是 RV32DC/RV64DC 专用指令，将浮点寄存器 rs2 中的双精度浮点值存储到内存中。它通过将乘 8 之后的零扩展偏移量与堆栈指针 x2 相加来计算有效地址。它扩展为 fsd rs2, offset(x2)。



在函数入口/出口处，寄存器保存/恢复代码占静态代码大小的很大一部分。RVC 中基于堆栈指针的压缩加载和存储操作在减少保存/恢复静态代码大小方面非常有效，能够减少代码大小到原来的二分之一，同时通过减少动态指令带宽来提高性能。

其他 ISA 中常用的一种机制是并行加载指令和并行存储指令，以进一步减少保存/恢复代码大小。我们考虑过将这些指令用于 RISC-V，但注意到以下缺点：

这些指令使处理器实现复杂化。

对于虚拟内存系统，某些数据访问可能驻留在物理内存中，而某些则不在，这需要一种新的重启机制来处理部分执行的指令。



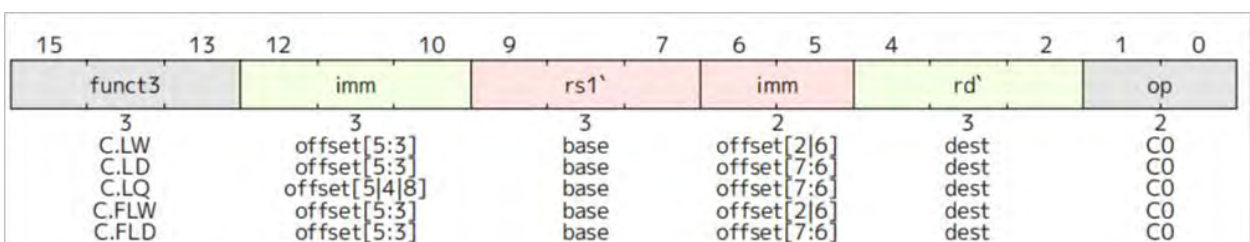
与 RVC 的其他指令不同，没有 IFD 等效的加载多个和存储多个指令。与 RVC 的其他指令不同，编译器必须知道这些指令，以便生成指令并按顺序分配寄存器，以最大化它们被保存和存储的机会，因为它们将按顺序保存和恢复。

简单的微架构实现将限制其他指令如何在加载和存储多个指令周围调度，导致潜在的性能损失。

顺序寄存器分配的需求可能与 CIW、CL、CS、CA 和 CB 格式中选择的特征寄存器冲突。

此外，通过用子程序调用常见的序言和尾声代码替换序言和尾声代码，可以在软件中实现大部分收益，这种技术在第 5.6 节中描述。

26.3.2 基于寄存器的加载和存储



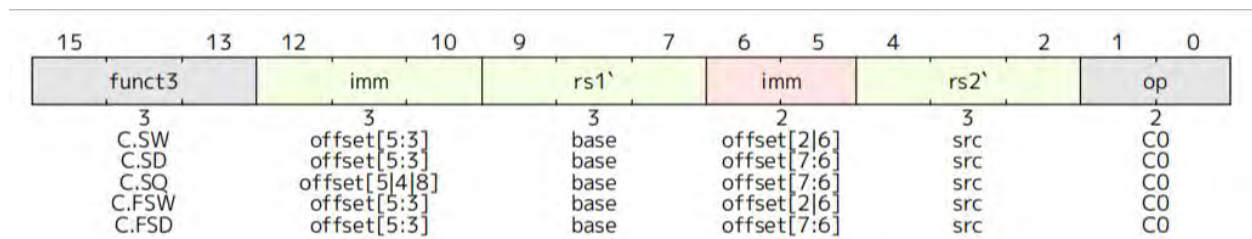
这些指令使用 CL 格式。C.LW 从内存加载一个 32 位值到寄存器 rd'。它通过将乘 4 之后的零扩展偏移量与寄存器 rs1' 中的基地址相加来计算有效地址。它扩展为 lw rd', offset(rs1')。

C.LD 是 RV64C/RV128C 专用指令，从内存加载一个 64 位值到寄存器 rd'。它通过将乘 8 之后的零扩展偏移量与寄存器 rs1' 中的基地址相加来计算有效地址。它扩展为 ld rd', offset(rs1')。

C.LQ 是 RV128C 专用指令，从内存加载一个 128 位值到寄存器 rd'。它通过将乘 16 之后的零扩展偏移量与寄存器 rs1' 中的基地址相加来计算有效地址。它扩展为 lq rd', offset(rs1')。

C.FLW 是 RV32FC 专用指令，从内存加载一个单精度浮点值到浮点寄存器 rd'。它通过将乘 4 之后的零扩展偏移量与寄存器 rs1' 中的基地址相加来计算有效地址。它扩展为 flw rd', offset(rs1')。

C.FLD 是 RV32DC/RV64DC 专用指令，从内存加载一个双精度浮点值到浮点寄存器 rd'。它通过将乘 8 之后的零扩展偏移量与寄存器 rs1' 中的基地址相加来计算有效地址。它扩展为 fld rd' , offset(rs1')。



这些指令使用 CS 格式。

C.SW 将寄存器 rs2' 中的 32 位值存储到内存中。它通过将乘 4 之后的零扩展偏移量与寄存器 rs1' 中的基地址相加来计算有效地址。它扩展为 sw rs2' , offset(rs1')。

C.SD 是 RV64C/RV128C 专用指令，将寄存器 rs2' 中的 64 位值存储到内存中。它通过将乘 8 之后的零扩展偏移量与寄存器 rs1' 中的基地址相加来计算有效地址。它扩展为 sd rs2' , offset(rs1')。

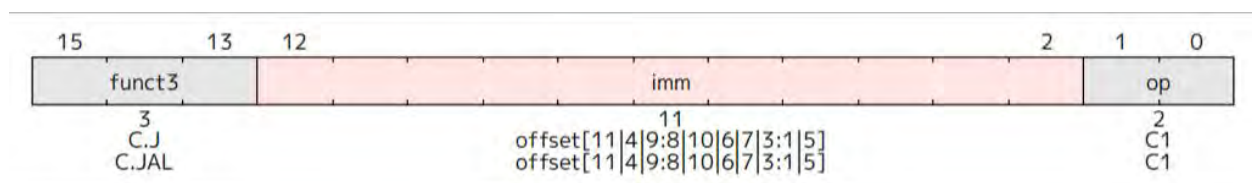
C.SQ 是 RV128C 专用指令，将寄存器 rs2' 中的 128 位值存储到内存中。它通过将乘 16 之后的零扩展偏移量与寄存器 rs1' 中的基地址相加来计算有效地址。它扩展为 sq rs2' , offset(rs1')。

C.FSW 是 RV32FC 专用指令，将浮点寄存器 rs2' 中的单精度浮点值存储到内存中。它通过将乘 4 之后的零扩展偏移量与寄存器 rs1' 中的基地址相加来计算有效地址。它扩展为 fsw rs2' , offset(rs1')。

C.FSD 是 RV32DC/RV64DC 专用指令，将浮点寄存器 rs2' 中的双精度浮点值存储到内存中。它通过将乘 8 之后的零扩展偏移量与寄存器 rs1' 中的基地址相加来计算有效地址。它扩展为 fsd rs2' , offset(rs1')。

26.4 控制转移指令

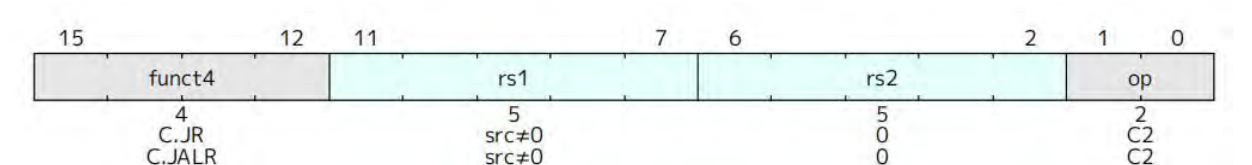
RVC 提供无条件跳转指令和条件分支指令。与基本 RVI 指令一样，所有 RVC 控制转移指令的偏移量都是 2 字节的倍数。



这些指令使用 CJ 格式。

C.J 执行无条件控制转移。偏移量是符号扩展的，并添加到 pc 以形成跳转目标地址。因此，C.J 可以定位 ± 2 KiB 范围。C.J 扩展为 jal x0, offset。

C.JAL 是 RV32C 专用指令，执行与 C.J 相同的操作，但另外将跳转后指令的地址 (pc+2) 写入链接寄存器 x1。C.JAL 扩展为 jal x1, offset。



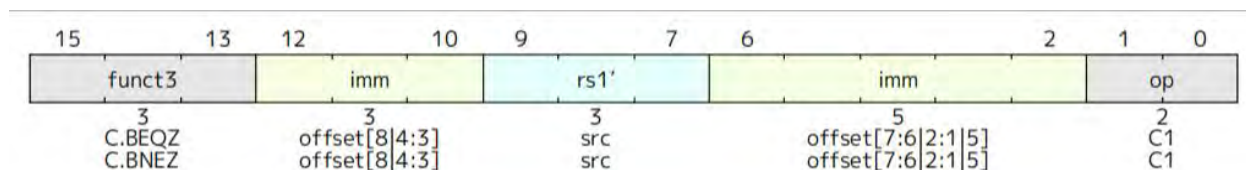
这些指令使用 CR 格式。

C. JR (跳转寄存器) 执行无条件控制转移到寄存器 $rs1$ 中的地址。C. JR 扩展为 `jalr x0, 0(rs1)`。当 $rs1 \neq x0$ 时, C. JR 才有效; 代码点 $rs1=x0$ 保留。

C. JALR (跳转并链接寄存器) 执行与 C. JR 相同的操作, 但另外将跳转后指令的地址 ($pc+2$) 写入链接寄存器 $x1$ 。C. JALR 扩展为 `jalr x1, 0(rs1)`。当 $rs1 \neq x0$ 时, C. JALR 才有效; 代码点 $rs1=x0$ 对应于 C. EBREAK 指令。



严格来说, C. JALR 并不完全扩展为基本 RVI 指令, 因为用于形成链接地址的 PC 增量值是 2 而不是基本 ISA 中的 4, 但支持 2 和 4 字节的偏移量对基本微架构来说只是一个非常小的变化。



这些指令使用 CB 格式。

C. BEQZ 执行条件控制转移。偏移量是符号扩展的, 并添加到 pc 以形成分支目标地址。因此, C. BEQZ 可以定位 $\&\#177;256$ B 范围。如果寄存器 $rs1'$ 中的值为零, 则 C. BEQZ 进行分支。它扩展为 `beq rs1', x0, offset`。

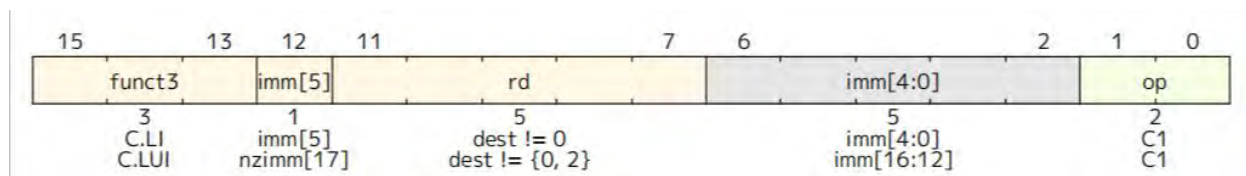
C. BNEZ 的定义类似, 但如果 $rs1'$ 包含非零值, 则进行分支。它扩展为 `bne rs1', x0, offset`。

26.5 整数计算指令

RVC 提供了几条用于整数算术和常量生成的指令。

26.5.1 整数常量生成指令

这两条常量生成指令都使用 CI 指令格式, 可以针对任何整数寄存器。

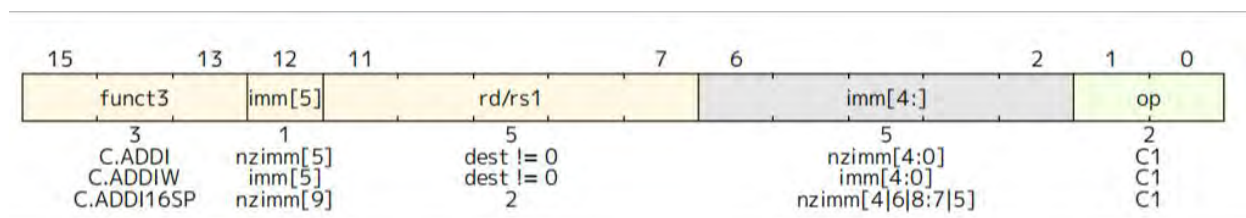


C. LI 将符号扩展的 6 位立即数 imm 加载到寄存器 rd 中。C. LI 扩展为 `addi rd, x0, imm`。当 $rd \neq x0$ 时, C. LI 才有效; $rd=x0$ 的代码点编码为 HINTs。

C. LUI 将非零 6 位立即数字段加载到目标寄存器的第 17-12 位, 清除底部 12 位, 并将第 17 位符号扩展到目标寄存器的所有更高位。C. LUI 扩展为 `lui rd, imm`。C. LUI 仅在 $rd \neq \{x0, x2\}$ 时有效, 并且立即数不等于零。 $imm=0$ 的代码点保留; $rd=x0$ 的剩余代码点为 HINTs; $rd=x2$ 的剩余代码点对应于 C. ADDI16SP 指令。

26.5.2 整数寄存器-立即数操作

这些整数寄存器-立即数操作在 CI 格式中编码, 并对整数寄存器和 6 位立即数执行操作。



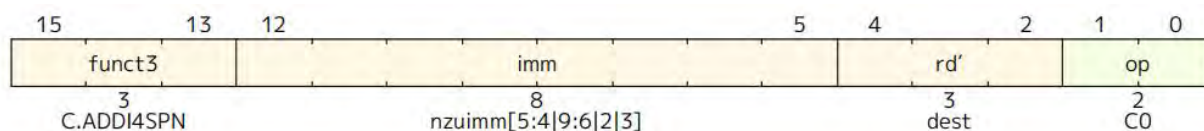
C.ADDI 将非零符号扩展的 6 位立即数添加到寄存器 rd 中的值，然后将结果写入 rd。C.ADDI 扩展为 `addi rd, rd, imm`。当 $rd \neq x0$ 且 $imm \neq 0$ 时，C.ADDI 才有效。 $rd=x0$ 的代码点编码为 C.NOP 指令； $imm=0$ 的剩余代码点编码为 HINTs。

C.ADDIW 是 RV64C/RV128C 专用指令，执行相同的计算，但生成 32 位结果，然后将结果符号扩展到 64 位。C.ADDIW 扩展为 `addiwr rd, rd, imm`。对于 C.ADDIW，立即数可以为零，此时对应于 `sext.w rd`。C.ADDIW 仅在 $rd \neq x0$ 时有效； $rd=x0$ 的代码点保留。

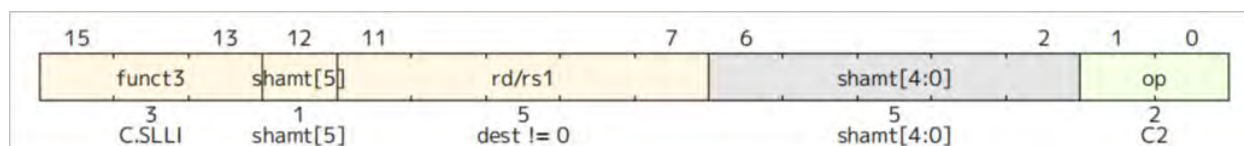
C.ADDI16SP 与 C.LUI 共享操作码，但目标字段为 x2。C.ADDI16SP 将非零符号扩展的 6 位立即数添加到堆栈指针 ($sp=x2$) 的值中，其中立即数按比例表示 (-512,496) 范围内的 16 的倍数。C.ADDI16SP 用于在程序头和尾声中调整堆栈指针。它扩展为 `addi x2, x2, nzimm[9:4]`。当 $nzimm \neq 0$ 时，C.ADDI16SP 才有效； $nzimm=0$ 的代码点保留。



在标准 RISC-V 调用约定中，堆栈指针 sp 始终是 16 字节对齐的。

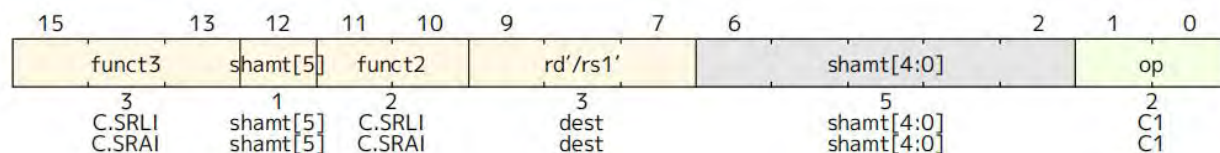


C.ADDI4SPN 是 CIW 格式指令，将零扩展的非零立即数（按 4 缩放）与堆栈指针 x2 相加，并将结果写入 rd'。此指令用于生成指向堆栈分配变量的指针，并扩展为 `addi rd', x2, nzuimm[9:2]`。当 $nzuimm \neq 0$ 时，C.ADDI4SPN 才有效； $nzuimm=0$ 的代码点保留。



C.SLLI 是 CI 格式指令，对寄存器 rd 中的值执行逻辑左移，然后将结果写入 rd。移位量编码在 shamt 字段中。对于 RV128C，移位量为零用于编码 64 的移位。C.SLLI 扩展为 `slli rd, rd, shamt[5:0]`，除了 RV128C 中 $shamt=0$ ，扩展为 `slli rd, rd, 64`。

对于 RV32C， $shamt[5]$ 必须为零； $shamt[5]=1$ 的代码点指定用于自定义扩展。对于 RV32C 和 RV64C，移位量必须为非零； $shamt=0$ 的代码点为 HINTs。对于所有基本 ISA， $rd=x0$ 的代码点为 HINTs，除了 RV32C 中 $shamt[5]=1$ 的代码点。



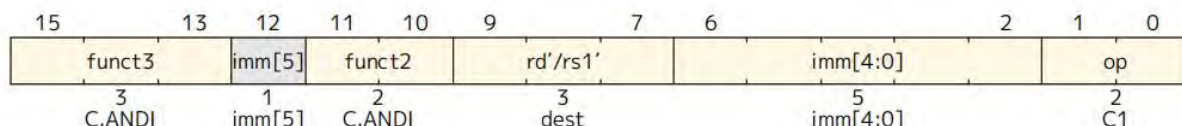
C.SRLI 是 CB 格式指令，对寄存器 rd' 中的值执行逻辑右移，然后将结果写入 rd'。移位量编码在 shamt 字段中。对于 RV128C，移位量为零用于编码 64 的移位。此外，移位量对于 RV128C 是符号扩展的，因此合法的移位量为 1-31、64 和 96-127。C.SRLI 扩展为 `srli rd', rd', shamt`，除了 RV128C 中 $shamt=0$ ，扩展为 `srli rd', rd', 64`。

对于 RV32C， $shamt[5]$ 必须为零； $shamt[5]=1$ 的代码点指定用于自定义扩展。对于 RV32C 和 RV64C，移位量必须为非零； $shamt=0$ 的代码点为 HINTs。

C.SRAI 的定义类似于 C.SRLI，但执行算术右移。C.SRAI 扩展为 `srai rd', rd', shamt`。

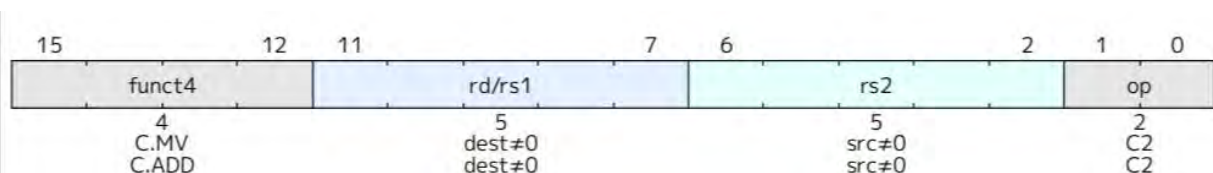


左移通常比右移更频繁，因为左移常用于缩放地址值。因此，右移分配了较少的编码空间，并放置在其他立即数符号扩展的编码象限中。对于 RV128，决定将 6 位移位量立即数也符号扩展。除了减少解码复杂性外，我们认为 96-127 的右移量比 64-95 更有用，以允许提取位于 128 位地址指针高位部分的标签。我们注意到 RV128C 不会与 RV32C 和 RV64C 同时冻结，以便评估 128 位地址空间代码的典型使用情况。



C.ANDI 是 CB 格式指令，计算寄存器 rd' 中的值与符号扩展的 6 位立即数的按位与，然后将结果写入 rd' 。C.ANDI 扩展为 `andi rd', rd', imm`。

26.5.3 整数寄存器-寄存器操作

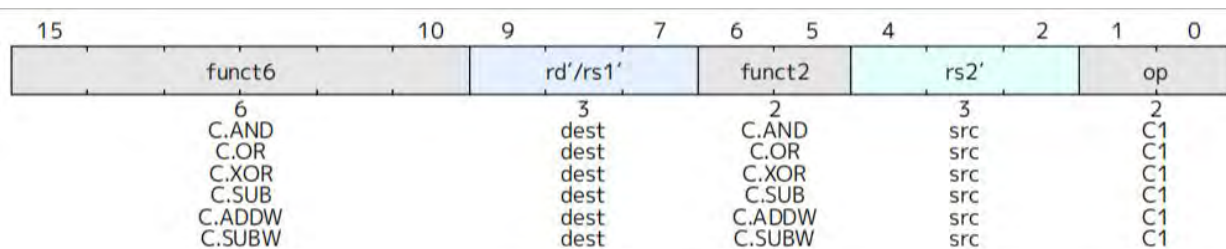


这些指令使用 CR 格式。C.MV 将寄存器 $rs2$ 中的值复制到寄存器 rd 中。C.MV 扩展为 `add rd, x0, rs2`。当 $rs2 \neq x0$ 时，C.MV 才有效； $rs2 = x0$ 的代码点对应于 C.JR 指令。 $rs2 \neq x0$ 且 $rd = x0$ 的代码点为 HINTs。



C.MV 扩展为与规范的 MV 伪指令不同的指令，后者使用 ADDI。处理 MV 特殊情况的实现，例如使用寄存器重命名硬件，可能会发现将 C.MV 扩展为 MV 而不是 ADD 更方便，尽管硬件成本略有增加。

D. ADD 将寄存器 rd 和 $rs2$ 中的值相加，并将结果写入寄存器 rd 。C.ADD 扩展为 `add rd, rd, rs2`。当 $rs2 \neq x0$ 时，C.ADD 才有效； $rs2 = x0$ 的代码点对应于 C.JALR 和 C.EBREAK 指令。 $rs2 \neq x0$ 且 $rd = x0$ 的代码点为 HINTs。



这些指令使用 CA 格式。

C.AND 计算寄存器 rd' 和 $rs2'$ 中值的按位与，然后将结果写入寄存器 rd' 。C.AND 扩展为 `*and rd', rd', rs2' *`。

C.OR 计算寄存器 rd' 和 $rs2'$ 中值的按位或，然后将结果写入寄存器 rd' 。C.OR 扩展为 `*or rd', rd', rs2' *`。

C.XOR 计算寄存器 rd' 和 $rs2'$ 中值的按位异或，然后将结果写入寄存器 rd' 。C.XOR 扩展为 `*xor rd', rd', rs2' *`。

C.SUB 将寄存器 $rs2'$ 中的值从寄存器 rd' 中的值中减去,然后将结果写入寄存器 rd' 。C.SUB 扩展为 $*sub\ rd',\ rd',\ rs2' *$ 。

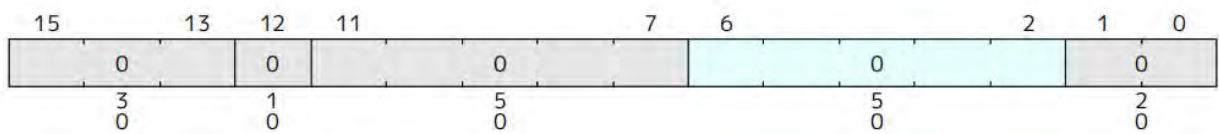
C.ADDW 是 RV64C/RV128C 专用指令,将寄存器 rd' 和 $rs2'$ 中的值相加,然后将和的低 32 位符号扩展后写入寄存器 rd' 。C.ADDW 扩展为 $*addw\ rd',\ rd',\ rs2' *$ 。

C.SUBW 是 RV64C/RV128C 专用指令,将寄存器 $rs2'$ 中的值从寄存器 rd' 中的值中减去,然后将差的低 32 位符号扩展后写入寄存器 rd' 。C.SUBW 扩展为 $*subw\ rd',\ rd',\ rs2' *$ 。



这组六条指令单独提供的节省不大,但占用的编码空间不多,且实现简单,作为一个整体在静态和动态压缩方面提供了有价值的改进。

26.5.4 定义的非指令

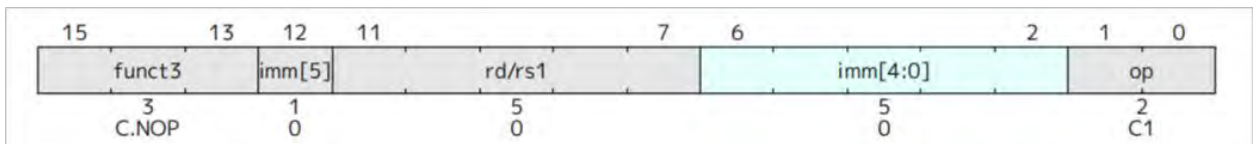


所有位都为零的 16 位指令永久保留为非法指令。



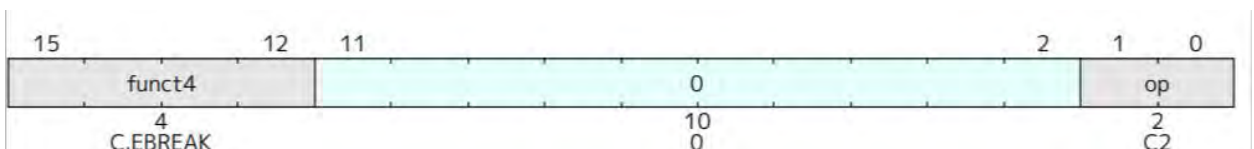
我们保留所有全零指令作为非法指令,以帮助捕获尝试执行全零或不存在的内存空间部分的操作。全零值不应在任何非标准扩展中重新定义。同样,我们保留所有位都设置为 1 的指令(对应于 RISC-V 可变量长度编码方案中的非常长的指令)作为非法指令,以捕获在不存在的内存区域中看到的另一种常见值。

26.5.5 NOP 指令



C.NOP 是一种 CI 格式指令,不会更改任何用户可见的状态,除了推进 pc 和增加任何适用的性能计数器。C.NOP 扩展为 nop。当 $imm=0$ 时, C.NOP 才有效; $imm \neq 0$ 的代码点编码为 HINTs。

26.5.6 断点指令



调试器可以使用 C.EBREAK 指令,该指令扩展为 ebreak,以使控制转移回调试环境。C.EBREAK 与 C.ADD 指令共享操作码,但 rd 和 rs2 都为零,因此也可以使用 CR 格式。

26.6 在 LR/SC 序列中使用 C 指令

在支持 C 扩展的实现中,允许在受限 LR/SC 序列中使用描述在第 14.3 章中的 I 指令的压缩形式。



这意味着任何声称支持 A 和 C 扩展的实现必须确保包含有效 C 指令的 LR/SC 序列最终会完成。

26.7 HINT 指令

RVC 编码空间的一部分保留用于微架构 HINTs。与 RV32I 基本 ISA 中的 HINTs 类似（见 HINT 指令），这些指令不会修改任何架构状态，除了推进 pc 和增加任何适用的性能计数器。对于忽略它们的实现，HINTs 作为无操作执行。

RVC HINTs 编码为不修改架构状态的计算指令，因为 $rd=x0$ （例如 C.ADD $x0, t0$ ），或因为 rd 被覆盖为自身的副本（例如 C.ADDI $t0, 0$ ）。



选择这种 HINT 编码是为了使简单的实现可以完全忽略 HINTs，而是将 HINT 作为常规计算指令执行，恰好不改变架构状态。

RVC HINTs 不一定扩展为其 RVI HINT 对应物。例如，C.ADD $x0, a0$ 可能不会编码为与 ADD $x0, x0, a0$ 相同的 HINT。



不要求 RVC HINT 扩展为 RVI HINT 的主要原因是 HINT 不太可能以与基础计算指令相同的方式压缩。此外，解耦 RVC 和 RVI HINT 映射允许将稀缺的 RVC HINT 空间分配给最流行的 HINT，特别是那些适合宏操作融合的 HINT。

表 24.1 列出了所有 RVC HINT 代码点。对于 RV32C，78% 的 HINT 空间保留用于标准 HINTs。剩余的 HINT 空间指定用于自定义 HINTs；在此子空间中不会定义任何标准 HINTs。

表 26.1 RVC 提示指令

介绍	约束	代码点	目的
C.NOP	$imm \neq 0$	63	指定用于将来的标准使用
C.ADDI	$rd \neq x0, imm = 0$	31	
C.LI	$rd = x0$	64	
C.LUI	$rd = x0, imm \neq 0$	63	
C.MV	$rd = x0, rs2 \neq x0$	31	
C.ADD	$rd = x0, rs2 \neq x0, rs2 \neq x2-x5$	27	
C.ADD	$rd = x0, rs2 \neq x2-x5$	4	($rs2=x2$)C.NTL.P1 ($rs2=x3$)C.NTL.PALL ($rs2=x4$)C.NTL.S1 ($rs2=x5$)C.NTL.ALL
C.SLLI	$rd = x0, imm \neq 0$	31 (RV32), 63 (RV64/128)	指定自定义使用
C.SLLI64	$rd = x0$	1	
C.SLLI64	$rd \neq x0$, RV32 and RV64 only	31	
C.SRLI64	RV32 and RV64 only	8	
C.SRAI64	RV32 and RV64 only	8	

26.8 RVC 指令集列表

表 26.2 显示了 RVC 的主操作码映射。表的每一行对应于编码空间的一个象限。最后一个象限具有两个最低有效位设置，对应于宽于 16 位的指令，包括基本 ISA 中的指令。某些指令仅对某些操作数有效；当无效时，它们标记为 RES，表示该操作码保留用于未来的标准扩展；Custom 表示该操作码指定用于自定义扩展；或 HINT 表示该操作码保留用于微架构提示。

表 26.2 RVC 操作码映射指令

inst[15:13] inst[1:0]	000	001	010	011	100	101	110	111	
00	ADDI4 SPN	FLD FLD LQ	LW	FLW LD LD	Reserved	FSD FSD SQ	SW	FSW SD SD	RV32 RV64 RV128
01	ADDI	JAL ADDIW ADDIW	LI	LUI/ADDI 16SP	MISC-ALU	J	BEQZ	BNEZ	RV32 RV64 RV128
10	SLLI	FLDSP FLDSP LQSP	LWSP	FLWSP LDSP LDSP	J[AL]R/MV /ADD	FSDSP FSDSP SQSP	SWSP	FSWSP SDSP SDSP	RV32 RV64 RV128
11	>16b								

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
000	0										0	00	Illegal instruction <i>C.ADDI4SPN</i> <i>RES, uimm=0</i> <i>C.FLD</i> <i>(RV32/64)</i> <i>C.LQ</i> <i>(RV128)</i> <i>C.LW</i> <i>C.FLW</i> <i>(RV32)</i> <i>C.LD</i> <i>(RV64/128)</i> Reserved <i>C.FSD</i> <i>(RV32/64)</i> <i>C.SQ</i> <i>(RV128)</i> <i>C.SW</i> <i>C.FSW</i> <i>(RV32)</i> <i>C.SD</i> <i>(RV64/128)</i>			
000	uimm[5:4 9:6 2 3]										rd'	00				
001	uimm[5:3]			rs1'			uimm[7:6]			rd'	00					
001	uimm[5:4 8]			rs1'			uimm[7:6]			rd'	00					
010	uimm[5:3]			rs1'			uimm[2 6]			rd'	00					
011	uimm[5:3]			rs1'			uimm[2 6]			rd'	00					
011	uimm[5:3]			rs1'			uimm[7:6]			rd'	00					
100	---											00				
101	uimm[5:3]			rs1'			uimm[7:6]			rs2'	00					
101	uimm[5:4 8]			rs1'			uimm[7:6]			rs2'	00					
110	uimm[5:3]			rs1'			uimm[2 6]			rs2'	00					
111	uimm[5:3]			rs1'			uimm[2 6]			rs2'	00					
111	uimm[5:3]			rs1'			uimm[7:6]			rs2'	00					

图26.1 RVC指令列表，象限0

000	imm[5]	0		imm[4:0]		01	C.NOP (R31, imm=0)
000	imm[5]	rs1/rd≠0		imm[4:0]		01	C.ADDI (R31, imm=0)
001	imm[11 4 9:8 10 6 7 3:1 5]					01	C.JAL (R32)
001	imm[5]	rs1/rd≠0		imm[4:0]		01	C.ADDIW (R31, rd=0; R32, rd=0)
010	imm[5]	rd≠0		imm[4:0]		01	C.LI (R31, rd=0)
011	imm[9]	2		imm[4 6 8:7 5]		01	C.ADDI16SP (R32, imm=0)
011	imm[17]	rd≠{0, 2}		imm[16:12]		01	C.LUI (R32, imm=0; R31, rd=0)
100	uimm[5]	00	rs1'/rd'	uimm[4:0]		01	C.SRLI (R32 Custom, uimm[3]=0)
100	0	00	rs1'/rd'	0		01	C.SRLI64 (R32, R32R4 R31)
100	uimm[5]	01	rs1'/rd'	uimm[4:0]		01	C.SRAI (R32 Custom, uimm[3]=0)
100	0	01	rs1'/rd'	0		01	C.SRAI64 (R32, R32R4 R31)
100	imm[5]	10	rs1'/rd'	imm[4:0]		01	C.ANDI
100	0	11	rs1'/rd'	00	rs2'	01	C.SUB
100	0	11	rs1'/rd'	01	rs2'	01	C.XOR
100	0	11	rs1'/rd'	10	rs2'	01	C.OR
100	0	11	rs1'/rd'	11	rs2'	01	C.AND
100	1	11	rs1'/rd'	00	rs2'	01	C.SUBW (R31, R32, R32, R32)
100	1	11	rs1'/rd'	01	rs2'	01	C.ADDW (R31, R32, R32, R32)
100	1	11	---	10	---	01	Reserved
100	1	11	---	11	---	01	Reserved
101	imm[11 4 9:8 10 6 7 3:1 5]					01	C.J
110	imm[8 4:3]		rs1'	imm[7:6 2:1 5]		01	C.BEQZ
111	imm[8 4:3]		rs1'	imm[7:6 2:1 5]		01	C.BNEZ

图26.2 RVC指令列表，象限1

000	uimm[5]	rs1/rd≠0	uimm[4:0]	10	C.SLLI (R31, rd=0; R32 Custom, uimm[3]=0)
000	0	rs1/rd≠0	0	10	C.SLLI64 (R32, R32R4 R31; R31, rd=0)
001	uimm[5]	rd	uimm[4:3 8:6]	10	C.FLDSP (R32R4)
001	uimm[5]	rd≠0	uimm[4 9:6]	10	C.LQSP (R32, R32, rd=0)
010	uimm[5]	rd≠0	uimm[4:2 7:6]	10	C.LWSP (R32, rd=0)
011	uimm[5]	rd	uimm[4:2 7:6]	10	C.FLWSP (R32)
011	uimm[5]	rd≠0	uimm[4:3 8:6]	10	C.LDSP (R32, R32R4 R31; R31, rd=0)
100	0	rs1≠0	0	10	C.JR (R32, rd=0)
100	0	rd≠0	rs2≠0	10	C.MV (R31, rd=0)
100	1	0	0	10	C.EBREAK
100	1	rs1≠0	0	10	C.JALR
100	1	rs1/rd≠0	rs2≠0	10	C.ADD (R31, rd=0)
101		uimm[5:3 8:6]	rs2	10	C.FSDSP (R32R4)
101		uimm[5:4 9:6]	rs2	10	C.SQSP (R32)
110		uimm[5:2 7:6]	rs2	10	C.SWSP
111		uimm[5:2 7:6]	rs2	10	C.FSWSP (R32)
111		uimm[5:3 8:6]	rs2	10	C.SDSP (R32, R32R4)

图26.2 RVC指令列表，象限2

27 Zc*扩展：代码大小缩减扩展，V1.0.0

27.1 Zc* 概述

Zc* 是一组定义了现有 C 扩展（Zca、Zcd、Zcf）的子集和仅包含 16 位编码的新扩展。Zcm* 重用了 c.fld、c.fsd、c.fldsp、c.fsdsp 的编码。

表 27.1 zc 扩展概述

介绍	Zca	Zcf	Zcd	Zcb	Zcmp	Zcmt
c.flw		rv32				
c.flwsp		rv32				
c.fsw		rv32				
c.fswsp		rv32				
c.fld			是			
c.fldsp			是			
c.fsd			是			
c.fsdsp			是			
c.lbu				是		
c.lh				是		
c.lhu				是		
c.sb				是		
c.sh				是		
c.zext.b				是		
c.sext.b				是		
c.zext.h				是		
c.sext.h				是		
c.zext.w				是		
c.mul				是		
c.not				是		
cm.push					是	
cm.pop					是	
cm.popret					是	
cm.popretz					是	
cm.jt						是
cm.jalt						是

27.2 C

C 扩展是以下扩展的超集：

- * Zca
- * 如果指定 F，则为 Zcf（仅 RV32）
- * 如果指定 D，则为 Zcd

由于 C 定义了与 Zca、Zcf 和 Zcd 相同的指令，规则是：

- * C 总是意味着 Zca
- * C+F 意味着 Zcf (仅 RV32)
- * C+D 意味着 Zcd

27.3 Zce

Zce 扩展旨在用于微控制器，并包括所有相关的 Zc 扩展。

- * 在不支持 F 扩展的 RV32 上的 Zce 包括 Zca、Zcb、Zcmp、Zcmt
- * 在支持 F 扩展的 RV32 上的 Zce 包括 Zca、Zcb、Zcmp、Zcmt 和 Zcf
- * 在 RV64 上的 Zce 总是包括 Zca、Zcb、Zcmp、Zcmt
- ** Zcf 不存在于 RV64

因此，常见的 ISA 字符串可以更新为包含相关的 Zc 扩展，例如：

- * RV32IMC 变为 RV32IMZce
- * RV32IMCF 变为 RV32IMFZce

27.4 MISA.C

如果支持以下扩展，则设置 MISA.C：

- * 支持 Zca 并且不支持 F
- * 同时支持 Zca、Zcf 和 F (仅 RV32)
- * 在支持 D 扩展的情况下支持 Zca、Zcf 和 Zcd (仅 RV32)
- ** 此配置排除 Zcmp、Zcmt
- * 在支持 D 扩展的情况下支持 Zca、Zcd (仅 RV64)
- ** 此配置排除 Zcmp、Zcmt

27.5 Zca

Zca 扩展被添加为引用 C 扩展中不包括浮点加载和存储的指令。

因此，它不包括所有 16 位浮点加载和存储：c.flw、c.flwsp、c.fsw、c.fswsp、c.fld、c.fldsp、c.fsd、c.fsdsp。



C 扩展仅在同时支持 D 扩展和 F 扩展时包括 F/D 指令

27.6 Zcf (仅 RV32)

Zcf 是现有的压缩单精度浮点加载和存储指令集合：c.flw、c.flwsp、c.fsw、c.fswsp。

Zcf 仅与 RV32 相关，不能在 RV64 上指定。

Zcf 扩展依赖于 Zca 和 F 扩展。

27.7 Zcd

Zcd 是现有的压缩双精度浮点加载和存储指令集合：c.fld、c.fldsp、c.fsd、c.fsdsp。

Zcd 扩展依赖于 Zca 和 D 扩展。

27.8 Zcb

Zcb 具有简单的代码大小节省指令，这些指令易于在所有 CPU 上实现。所有编码目前都为所有架构保留，并且与任何现有扩展没有冲突。



Zcb 可以在任何 CPU 上实现，因为这些指令是应用类配置文件中现有 32 位指令的 16 位版本。

Zcb 扩展依赖于 Zca 扩展。如单独指令页所示，Zcb 中的许多指令依赖于另一个扩展的实现。例如，`c.mul` 仅在实现 M 或 Zmmul 时实现，`c.sext.b` 仅在实现 Zbb 时实现。

`c.mul` 编码使用 CA 寄存器格式以及其他指令，例如 `c.sub`、`c.xor` 等。

`c.sext.w` 是 `c.addiw rd, 0 (RV64)` 的伪指令

27.9 Zcmp

Zcmp 扩展是一组指令，这些指令可以作为一系列现有的 32 位 RISC-V 指令执行。该扩展重用了来自 `c.fsdsp` 的一些编码。因此，它与 Zcd 不兼容，而 Zcd 是在同时包含 C 扩展和 D 扩展时使用的。



由于实现复杂性，Zcmp 主要面向嵌入式 CPU。此外，它与架构类配置文件不兼容。

Zcmp 扩展依赖于 Zca 扩展。

PUSH/POP 汇编语法使用几个变量，其含义如下：

* `reglist` 是包含 1 到 13 个寄存器（`ra` 和 0 到 12 个 `s` 寄存器）的列表

** 有效值：{`ra`}、{`ra, s0`}、{`ra, s0-s1`}、{`ra, s0-s2`}、...、{`ra, s0-s8`}、{`ra, s0-s9`}、{`ra, s0-s11`}

** 注意 {`ra, s0-s10`} 是无效的，给出 12 个列表而不是 13 个，以便更好地编码

* `stackadj` 是堆栈帧的总大小。

** 有效值随寄存器列表长度和特定编码而变化，详见指令页面。

27.10 Zcmt

Zcmt 添加了表跳转指令，并添加了 `jvt` 控制状态寄存器。如果实现了 `Smstateen`，则 `jvt` 控制状态寄存器需要进入启用状态。

该扩展重用了来自 `c.fsdsp` 的一些编码。因此，它与 Zcd 不兼容，而 Zcd 是在同时包含 C 扩展和 D 扩展时使用的。



由于实现复杂性，Zcmt 主要面向嵌入式类别的 CPU。此外，它与 RVA 配置文件不兼容。

Zcmt 扩展依赖于 Zca 和 Zicsr 扩展。

27.11 Zc 指令格式

本规范中的几个指令使用以下新的指令格式。

Format	instructions	15:10	9	8	7	6	5	4	3	2	1	0	
CLB	c.lbu	funct6	rs1'			uimm		rd'			op		
CSB	c.sb	funct6	rs1'			uimm		rs2'			op		
CLH	c.lhu, c.lh	funct6	rs1'			funct1	uimm	rd'			op		
CSH	c.sh	funct6	rs1'			funct1	uimm	rs2'			op		
CU	c.[sz]ext.*, c.not	funct6	rd'/rs1'			funct5					op		
CMMV	cm.mvsa01 cm.mva01s	funct6	r1s'			funct2		r2s'			op		
CMJT	cm.jt cm.jalt	funct6	index									op	
CMPP	cm.push*, cm.pop*	funct6	funct2		urlist				spimm		op		



c.mul 使用现有的 CA 格式。

27.12 Zcb 指令

27.12.1 c.lbu

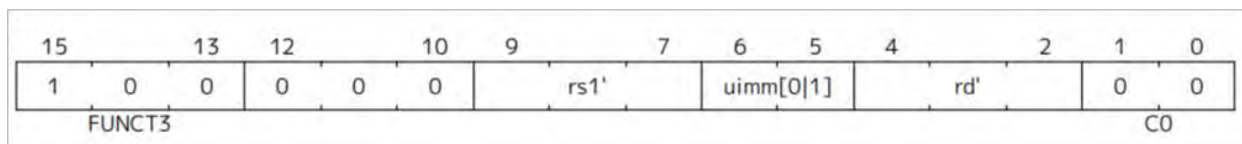
简述:

加载无符号字节, 16 位编码

助记符:

c.lbu rd, uimm (rs1)

编码 (RV32, RV64):



立即数偏移量的形成方式如下:

```

uimm[31:2] = 0;
uimm[1]    = encoding[5];
uimm[0]    = encoding[6];

```

描述:

该指令从 rs1 加上零扩展的立即数 uimm 形成的内存地址处加载一个字节。结果字节被零扩展到 XLEN 位, 并写入 rd。



rd 和 rs1 来自标准的 8-寄存器集 x8-x15。

操作:

//这不是 SAIL, 这是伪代码。SAIL 尚未编写。

```
X(rdc) = EXTZ(mem[X(rs1c)+EXTZ(uimm)][7..0]);
```

27.12.2 c.lhu

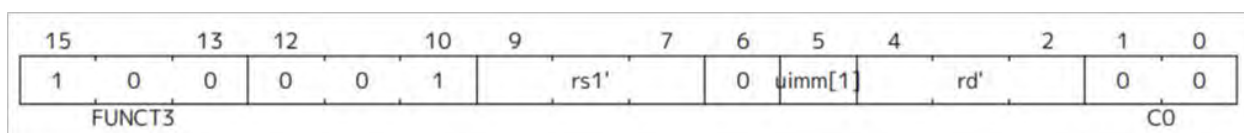
简述:

加载无符号半字, 16 位编码

助记符:

c.lhu rd, uimm(rs1)

编码 (RV32, RV64):



立即数偏移量的形成方式如下所示:

```
uimm[31:2] = 0;  
uimm[1]     = encoding[5];  
uimm[0]     = 0;
```

描述:

此指令从由 rs1 和零扩展的立即数 uimm 形成的内存地址处加载一个半字。结果半字被零扩展到 XLEN 位, 并写入 rd。



rd 和 rs1 来自标准的 8-寄存器集 x8-x15。

先决条件:

无

操作:

//这不是 SAIL, 这是伪代码。SAIL 尚未编写。

```
X(rdc) = EXTZ(loadmem[X(rs1c)+EXTZ(uimm)][15..0]);
```

27.12.3 c.lh

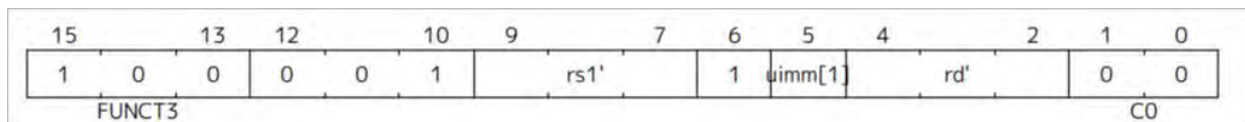
简述:

加载有符号半字, 16 位编码

助记符:

c.lh rd, uimm(rs1)

编码 (RV32, RV64):



立即数偏移按照如下规则形成:

```
uimm[31:2] = 0;  
uimm[1]     = encoding[5];  
uimm[0]     = 0;
```

描述:

该指令从 rs1 加上零扩展的立即数 uimm 形成的内存地址处加载一个半字。结果半字被符号扩展到 XLEN 位, 并写入 rd。



rd 和 rs1 来自标准的 8-寄存器集 x8-x15。

先决条件:

无

操作:

//这不是 SAIL, 这是伪代码。SAIL 尚未编写。

```
X(rdc) = EXTS(loadmem[X(rs1c)+EXTZ(uimm)][15..0]);
```

27.12.4 c.sb

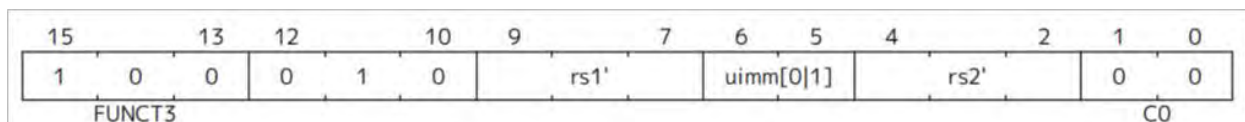
简述:

存储字节, 16 位编码

助记符:

c.sb rs2, uimm(rs1)

编码 (RV32, RV64):



立即数偏移按照如下规则形成：

```
uimm[31:2] = 0;
uimm[1]     = encoding[5];
uimm[0]     = encoding[6];
```

描述：
该指令将 rs2 中的最低有效字节存储到 rs1 加上零扩展的立即数 uimm 所形成的内存地址中。



rs1 和 rs2 来自标准的 8-寄存器集 x8-x15。

先决条件：
无

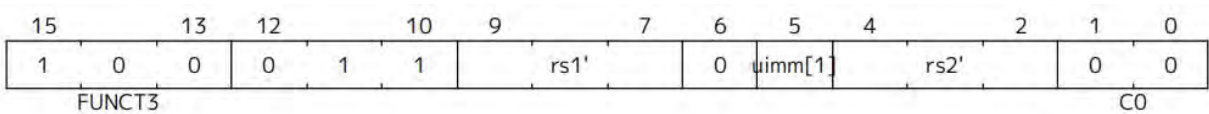
操作：
//这不是 SAIL，这是伪代码。SAIL 尚未编写。
mem[X(rs1c)+EXTZ(uimm)][7..0] = X(rs2c)

27.12.5 c.sh

简述：
存储半字，16 位编码

助记符：
c.sh rs2, uimm(rs1)

编码（RV32，RV64）：



立即数偏移按照如下规则形成：

```
uimm[31:2] = 0;
uimm[1]     = encoding[5];
uimm[0]     = 0;
```

描述：
该指令将 rs2 中的最低有效半字节存储到 rs1 加上零扩展的立即数 uimm 所形成的内存地址中。



rs1 和 rs2 来自标准的 8-寄存器集 x8-x15

先决条件:

无

操作:

//这不是 SAIL, 这是伪代码。SAIL 尚未编写。

`mem[X(rs1c)+EXTZ(uimm)][15..0] = X(rs2c)`

27.12.6 c.zext.b

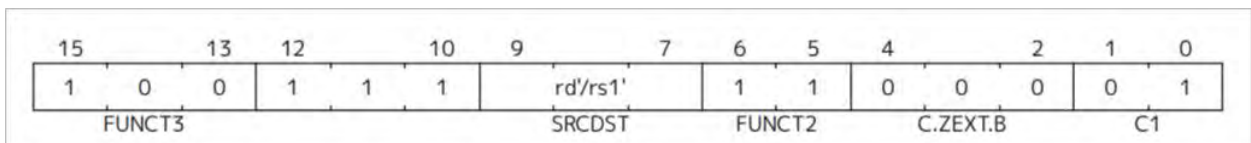
简述:

零扩展字节, 16 位编码

助记符:

`c.zext.b rd/rs1`

编码 (RV32, RV64):



描述:

该指令使用一个源/目的操作数。它将操作数的最低有效字节零扩展到 XLEN 位, 通过在高于第 7 位的所有位中插入零来实现。



rd/rs1 来自标准的 8-寄存器集 x8-x15。

先决条件:

无

32 位等效:

`andi rd'/rs1', rd'/rs1', 0xff`

操作:

`X(rsdc) = EXTZ(X(rsdc)[7..0]);`

27.12.7 c.sext.b

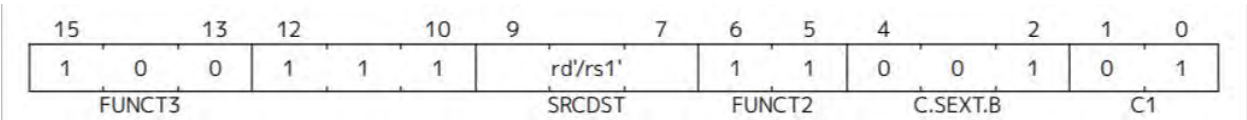
简述:

符号扩展字节，16 位编码

助记符：

c.sext.b rd/rs1

编码（RV32，RV64）：



描述：

该指令使用一个源/目的操作数。它将操作数中最低有效字节的符号扩展到 XLEN 位，通过将字节中的最高有效位（即第 7 位）复制到所有更高位来实现。



rd/rs1 来自标准的 8-寄存器集 x8-x15。

先决条件：

Zbb



SAIL 模块变量 rd/rs1 称为 rsdc。

操作：

$X(rsdc) = EXTZ(X(rsdc)[15..0]);$

27.12.8 c.zext.h

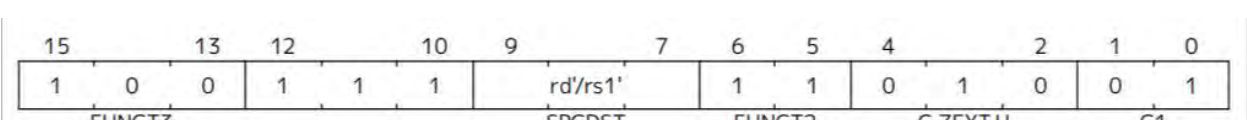
简述：

零扩展半字，16 位编码

助记符：

c.zext.h rd/rs1

编码（RV32，RV64）：



描述：

该指令使用一个源/目的操作数。它将操作数中最低有效半字零扩展到 XLEN 位，通过在高于第 15 位的所有位中插入零来实现。



rd/rs1 来自标准的 8-寄存器集 x8-x15。

先决条件：
Zbb



SAIL 模块变量 rd/rs1 称为 rsdc。

操作：

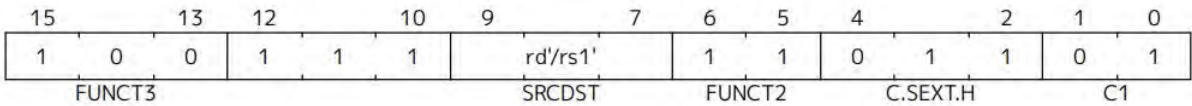
$X(rsdc) = EXTZ(X(rsdc)[15..0]);$

27.12.9 c.sext.h

简述：
符号扩展半字，16 位编码

助记符：
c.sext.h rd/rs1

编码（RV32，RV64）：



描述：
该指令使用一个源/目的操作数。它将操作数中最低有效半字的符号扩展到 XLEN 位，通过将半字中的最高有效位（即第 15 位）复制到所有更高位来实现。



rd/rs1 来自标准的 8-寄存器集 x8-x15。

先决条件：
Zbb



SAIL 模块变量 rd/rs1 称为 rsdc。

操作：

$X(rsdc) = EXTZ(X(rsdc)[15..0]);$

27.12.10 c.zext.w

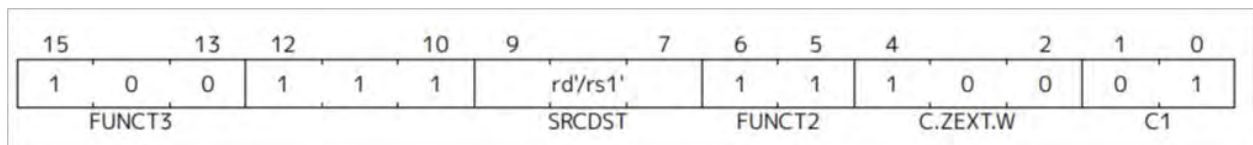
简述:

零扩展字, 16 位编码

助记符:

`c.zext.w rd/rs1`

编码 (RV64):



描述:

该指令使用一个源/目的操作数。它将操作数中最低有效字的零扩展到 XLEN 位, 通过在高于第 31 位的所有位中插入零来实现。



rd/rs1 来自标准的 8-寄存器集 x8-x15。

先决条件:

需要实现 Zba。

32 位等效:

`add.uw rd'/rs1', rd'/rs1', zero`

操作:

$X(rsdc) = \text{EXTZ}(X(rsdc)[31..0]);$

27.12.11 c.not

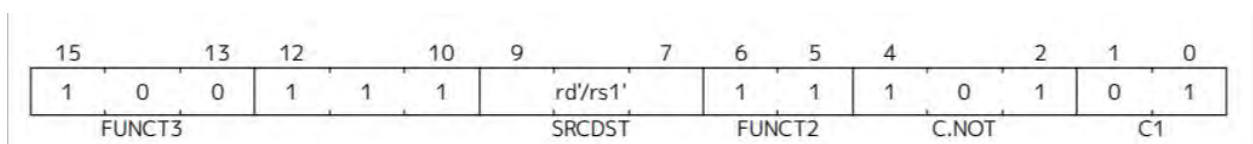
简述:

按位取反, 16 位编码

助记符:

`c.not rd/rs1`

编码 (RV32, RV64):



描述:

此指令对 rd/rs1 进行按位取反, 并将结果写入同一寄存器。



rd/rs1 来自标准的 8-寄存器集 x8-x15。

先决条件：
无

32 位等效：

```
xori rd'/rs1', rd'/rs1', -1
```

操作：

```
X(rsdc) = X(rsdc) XOR -1;
```

27.12.12 c.mul

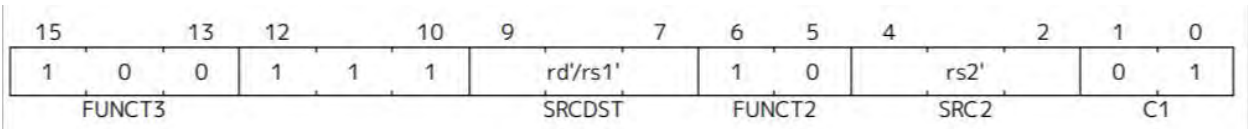
简述：

乘法，16 位编码

助记符：

```
c.mul rsd, rs2
```

编码（RV32，RV64）：



描述：

此指令将 rsd 和 rs2 的源操作数的 XLEN 位相乘，并将结果的最低 XLEN 位写入 rsd。



rd/rs1 和 rs2 来自标准的 8-寄存器集 x8-x15。

先决条件：
必须配置 M 或 Zmmul。



SAIL 模块变量 rd/rs1 称为 rsdc，rs2 称为 rs2c。

操作：

```
let result_wide = to_bits(2 * sizeof(xlen), signed(X(rsdc)) * signed(X(rs2c)));
X(rsdc) = result_wide[(sizeof(xlen) - 1) .. 0];
```

27.13 PUSH/POP 寄存器指令

这些指令统称为 PUSH/POP:

- cm.push
- cm.pop
- cm.popret
- cm.popretz

术语 PUSH 指的是 cm.push。

术语 POP 指的是 cm.pop。

术语 POPRET 指的是 cm.popret 和 cm.popretz。

这些指令的共同细节在本节中。

27.13.1 PUSH/POP 功能概述

PUSH、POP、POPRET 用于减少函数序言和尾声的大小。

. PUSH 指令

** 调整堆栈指针以创建堆栈帧

** 将寄存器列表中指定的寄存器推送（存储）到堆栈帧

. POP 指令

** 从堆栈帧中弹出（加载）寄存器列表中的寄存器

** 调整堆栈指针以销毁堆栈帧

. POPRET 指令

** 从堆栈帧中弹出（加载）寄存器列表中的寄存器

** cm.popretz 还将零移动到 a0 作为返回值

** 调整堆栈指针以销毁堆栈帧

** 执行 ret 指令以从函数返回

27.13.2 示例用法

此示例说明了 PUSH 和 POPRET 的使用。

EMBench 基准测试 picojpeg 中的函数 processMarkers 在 GitHub 上的以下文件中:

Libpicojpeg.c。

序言和尾声使用 GCC10 编译为:

```
0001098a <processMarkers>:
1098a:      711d          addi    sp, sp, -96 ;#cm.push(1)
1098c:      c8ca          sw      s2, 80(sp) ;#cm.push(2)
1098e:      c6ce          sw      s3, 76(sp) ;#cm.push(3)
10990:      c4d2          sw      s4, 72(sp) ;#cm.push(4)
10992:      ce86          sw      ra, 92(sp) ;#cm.push(5)
10994:      cca2          sw      s0, 88(sp) ;#cm.push(6)
10996:      caa6          sw      s1, 84(sp) ;#cm.push(7)
10998:      c2d6          sw      s5, 68(sp) ;#cm.push(8)
1099a:      c0da          sw      s6, 64(sp) ;#cm.push(9)
1099c:      de5e          sw      s7, 60(sp) ;#cm.push(10)
1099e:      dc62          sw      s8, 56(sp) ;#cm.push(11)
```

```

109a0:      da66                sw      s9, 52(sp) ;#cm. push(12)
109a2:      d86a                sw      s10, 48(sp) ;#cm. push(13)
109a4:      d66e                sw      s11, 44(sp) ;#cm. push(14)
...
109f4:      4501                li      a0, 0 ;#cm. popretz(1)
109f6:      40f6                lw      ra, 92(sp) ;#cm. popretz(2)
109f8:      4466                lw      s0, 88(sp) ;#cm. popretz(3)
109fa:      44d6                lw      s1, 84(sp) ;#cm. popretz(4)
109fc:      4946                lw      s2, 80(sp) ;#cm. popretz(5)
109fe:      49b6                lw      s3, 76(sp) ;#cm. popretz(6)
10a00:      4a26                lw      s4, 72(sp) ;#cm. popretz(7)
10a02:      4a96                lw      s5, 68(sp) ;#cm. popretz(8)
10a04:      4b06                lw      s6, 64(sp) ;#cm. popretz(9)
10a06:      5bf2                lw      s7, 60(sp) ;#cm. popretz(10)
10a08:      5c62                lw      s8, 56(sp) ;#cm. popretz(11)
10a0a:      5cd2                lw      s9, 52(sp) ;#cm. popretz(12)
10a0c:      5d42                lw      s10, 48(sp) ;#cm. popretz(13)
10a0e:      5db2                lw      s11, 44(sp) ;#cm. popretz(14)
10a10:      6125                addi    sp, sp, 96 ;#cm. popretz(15)
10a12:      8082                ret                    ;#cm. popretz(16)

```

使用 GCC 选项 `-msave-restore` 的输出如下：
SAIL 代码

```

0001080e <processMarkers>:
1080e:      73a012ef            jal      t0, 11f48 <riscvsave12>
10812:      1101                addi     sp, sp, -32
...
10862:      4501                li      a0, 0
10864:      6105                addi     sp, sp, 32
10866:      71e0106f            j        11f84 <riscvrestore12>

```

使用 PUSH/POPRET 后会将上面的指令缩减为
SAIL 代码

```

0001080e <processMarkers>:
1080e:      b8fa                cm. push {ra, s0-s11}, -96
...
10866:      bcfa                cm. popretz {ra, s0-s11}, 96

```

序言/尾声从原始代码中的 60 字节减少到 14 字节，使用 `-msave-restore`，
并使用 PUSH 和 POPRET 减少到 4 字节。

除了减少代码大小外，PUSH 和 POPRET 还消除了调用 `millicode save/restore` 例程的分支，因

此可能表现更好。

当目标函数超出 $\pm 1\text{MB}$ 范围时，对 `<riscvsave0>/<riscvrestore0>` 的调用变为 64 位，增加了序言/尾声的大小到 22 字节。



POP 通常用于尾调用序列中，在销毁堆栈帧后，`ret` 不用于返回到 `ra`。

所有指令会自动调整堆栈指针，以足够覆盖保存或恢复的寄存器所需的内存。此外，编码中的 `spimm` 字段允许堆栈指针按额外的 16 字节增量进行调整。编码中可用的范围较小；如果该范围不足，则可以使用单独的 `c.addi16sp` 指令来增加该范围。

如果不同时添加 `s11`，则不支持 `{ra, s0-s10}` 寄存器列表。因此，在这种情况下，必须使用 `{ra, s0-s11}` 寄存器列表。

27.13.3 PUSH/POP 故障处理

正确的执行要求 `sp` 引用幂等内存，因为核必须能够处理在序列执行过程中检测到的陷阱。整个 PUSH/POP 序列会在从陷阱处理程序返回后重新执行，并且在序列执行过程中可能发生多个陷阱。

如果在序列执行过程中发生陷阱，`xEPC` 将更新为指令的 `PC`，`xTVAL`（如果不是只读零）将更新为错误地址（如果是访问异常），并且 `xCAUSE` 将更新为陷阱类型。



是否可以在序列执行过程中处理中断是由实现定义的。

27.13.4 软件执行试图

PUSH 序列的软件视图

从软件的角度来看，PUSH 序列表现为：

- * 一系列存储写入伪代码所需的字节
- ** 字节可以以任何顺序写入。
- ** 字节可以被组合成更大的访问操作。
- ** 任何字节可以被写入多次。
- * 堆栈指针的调整



如果一个实现允许在序列执行期间处理中断，并且中断处理程序使用 `sp` 来分配堆栈内存，那么在中断之前执行的任何存储操作可能会被处理程序覆盖。这是安全的，因为内存是幂等的，且当执行恢复时，存储操作将会重新执行。

堆栈指针的调整只有在确认整个 PUSH 指令会提交时才能提交。存储操作也可能会从总线返回不精确的故障。是否在继续执行序列的最后阶段之前等待总线响应，或在完成 PUSH 指令后处理错误响应，取决于平台的定义。

```
cm.push {ra, s0-s5}, -64
```

在软件中显示为：


```

# 在指令完成之前，sp-1 到 sp-28 之间的任何字节都可能被多次写入，
# 因此这些更新可能在堆栈指针下的中断/异常处理程序中可见
sw  s5, -4(sp)
sw  s4, -8(sp)
sw  s3, -12(sp)
sw  s2, -16(sp)
sw  s1, -20(sp)
sw  s0, -24(sp)
sw  ra, -28(sp)
# 这必须只执行一次，并且只有在所有存储完成且没有任何精确错误后才会执行。
# 因此，只有在 cm.push 执行完成后，才能在中断/异常处理程序中看到此更新。
addi sp, sp, -64

```

POP/POPRET 序列的软件视图

从软件的角度来看，POP/POPRET 序列如下：

- * 一系列加载读取伪代码所需的字节。
- ** 字节可以以任何顺序加载。
- ** 字节可以分组为更大的访问。
- ** 任何字节都可以多次加载。
- * 堆栈指针调整
- * 可选的 li a0, 0
- * 可选的 ret

如果在序列执行期间发生陷阱，则在陷阱发生之前执行的任何加载操作可能会更新架构状态。加载操作将在陷阱处理程序完成后重新执行，因此值将被覆盖。因此，实现可以在发生故障之前更新部分目的寄存器。

可选的 li a0, 0、堆栈指针调整和可选的 ret 只有在确认整个 POP/POPRET 指令会提交时才能提交。

对于 POPRET，一旦堆栈指针调整已经提交，ret 必须执行。

例如：

```
cm.popretz {ra, s0-s3}, 32;
```

在软件中显示为：

```

# 可能会多次执行这些加载指令中的一些甚至全部，因此这些更新可能在中断/异常处
# 理程序中可见
lw  s3, 28(sp)
lw  s2, 24(sp)
lw  s1, 20(sp)
lw  s0, 16(sp)
lw  ra, 12(sp)
# 这些必须只执行一次，并且只有在所有加载成功完成后才会执行，所有指令必须原子执
# 行，因此这些更新在中断/异常处理程序中不可见

```

```
li a0, 0
addi sp, sp, 32
Ret
```

27.13.5 非幂等内存处理

某些实现可能需要将 PUSH/POP 指令发往非幂等内存 (non-idempotent memory)。

如果核实现不支持将 PUSH/POP 发往非幂等内存, 则核可以使用幂等性 (idempotency) PMA 来检测, 并触发加载 (POP/POPRET) 或存储 (PUSH) 访问故障异常, 以避免不可预测的结果。

当软件可以容忍在发生异常时反复发出所需的内存访问时, 软件应仅在非幂等内存区域使用这些指令。

27.13.6 RV32I 推送/POP 序列示例

以下提供的示例展示了加载/存储序列的展开和堆栈调整。由于在所有情况下, cm.popret 和 cm.popretz 的展开序列与 cm.pop 的差异都很小, 因此未包含 cm.popret 和 cm.popretz 的示例。

编码: rlist=7, spimm=3

SAIL 代码

```
sw s2, -4(sp);
sw s1, -8(sp);
sw s0, -12(sp);
sw ra, -16(sp);
addi sp, sp, -64;
```

```
cm.push {ra, s0-s11}, -112
```

编码: rlist=15, spimm=3

扩展为:

SAIL 代码

```
sw s11, -4(sp);
sw s10, -8(sp);
sw s9, -12(sp);
sw s8, -16(sp);
sw s7, -20(sp);
sw s6, -24(sp);
sw s5, -28(sp);
sw s4, -32(sp);
sw s3, -36(sp);
sw s2, -40(sp);
sw s1, -44(sp);
sw s0, -48(sp);
sw ra, -52(sp);
```

```
addi sp, sp, -112;
```

```
cm.pop {ra}, 16
```

编码: rlist=4, spimm=0

扩展为:

```
lw ra, 12(sp);
```

```
addi sp, sp, 16;
```

```
cm.pop {ra, s0-s3}, 48
```

编码: rlist=8, spimm=1

扩展为:

SAIL 代码

```
lw s3, 44(sp);
```

```
lw s2, 40(sp);
```

```
lw s1, 36(sp);
```

```
lw s0, 32(sp);
```

```
lw ra, 28(sp);
```

```
addi sp, sp, 48;
```

```
cm.pop {ra, s0-s4}, 64
```

编码: rlist=9, spimm=2

扩展为:

SAIL 代码

```
lw s4, 60(sp);
```

```
lw s3, 56(sp);
```

```
lw s2, 52(sp);
```

```
lw s1, 48(sp);
```

```
lw s0, 44(sp);
```

```
lw ra, 40(sp);
```

```
addi sp, sp, 64;
```

27.13.7 cm.push

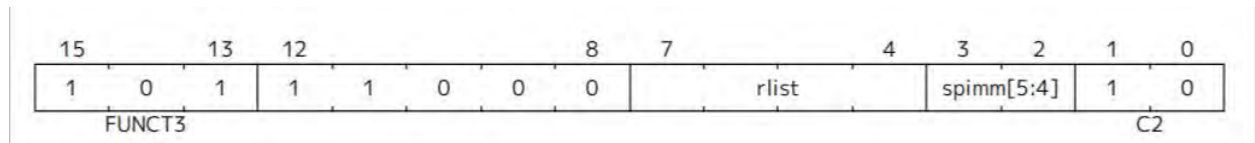
简述:

创建堆栈帧: 将返回地址 ra 和 0 到 12 个被保存的寄存器存储到栈帧中, 并可选择性地分配额外的栈空间。

助记符:

```
cm.push {reglist}, -stackadj
```

编码 (RV32, RV64) :



rlist 值 0 到 3 保留给未来的 EABI 变体, 称为 cm.push.e

汇编语法:

```
cm.push {reglist}, -stackadj
```

```
cm.push {xreglist}, -stackadj
```

汇编语法中使用的变量定义如下。

RV32E:

```
switch (rlist){
  case 4: {reg_list="ra";      xreg_list="x1";}
  case 5: {reg_list="ra, s0";  xreg_list="x1, x8";}
  case 6: {reg_list="ra, s0-s1"; xreg_list="x1, x8-x9";}
  default: reserved();
}
stack_adj      = stack_adj_base + spimm[5:4] * 16;
```

RV32I, RV64:

```
switch (rlist){
  case 4: {reg_list="ra";      xreg_list="x1";}
  case 5: {reg_list="ra, s0";  xreg_list="x1, x8";}
  case 6: {reg_list="ra, s0-s1"; xreg_list="x1, x8-x9";}
  case 7: {reg_list="ra, s0-s2"; xreg_list="x1, x8-x9, x18";}
  case 8: {reg_list="ra, s0-s3"; xreg_list="x1, x8-x9, x18-x19";}
  case 9: {reg_list="ra, s0-s4"; xreg_list="x1, x8-x9, x18-x20";}
  case 10: {reg_list="ra, s0-s5"; xreg_list="x1, x8-x9, x18-x21";}
  case 11: {reg_list="ra, s0-s6"; xreg_list="x1, x8-x9, x18-x22";}
  case 12: {reg_list="ra, s0-s7"; xreg_list="x1, x8-x9, x18-x23";}
  case 13: {reg_list="ra, s0-s8"; xreg_list="x1, x8-x9, x18-x24";}
  case 14: {reg_list="ra, s0-s9"; xreg_list="x1, x8-x9, x18-x25";}
  //note - to include s10, s11 must also be included

  case 15: {reg_list="ra, s0-s11"; xreg_list="x1, x8-x9, x18-x27";}
  default: reserved();
}
stack_adj      = stack_adj_base + spimm[5:4] * 16;
```

RV32E:

```
stack_adj_base = 16;  
Valid values:  
stack_adj      = [16|32|48|64];
```

RV32I:

```
switch (rlist) {  
    case 4..7: stack_adj_base = 16;  
    case 8..11: stack_adj_base = 32;  
    case 12..14: stack_adj_base = 48;  
    case 15: stack_adj_base = 64;  
}
```

```
Valid values:  
switch (rlist) {  
    case 4..7: stack_adj = [16|32|48| 64];  
    case 8..11: stack_adj = [32|48|64| 80];  
    case 12..14: stack_adj = [48|64|80| 96];  
    case 15: stack_adj = [64|80|96|112];  
}
```

RV64:

```
switch (rlist) {  
    case 4..5: stack_adj_base = 16;  
    case 6..7: stack_adj_base = 32;  
    case 8..9: stack_adj_base = 48;  
    case 10..11: stack_adj_base = 64;  
    case 12..13: stack_adj_base = 80;  
    case 14: stack_adj_base = 96;  
    case 15: stack_adj_base = 112;  
}
```

```
Valid values:  
switch (rlist) {  
    case 4..5: stack_adj = [ 16| 32| 48| 64];  
    case 6..7: stack_adj = [ 32| 48| 64| 80];  
    case 8..9: stack_adj = [ 48| 64| 80| 96];  
    case 10..11: stack_adj = [ 64| 80| 96|112];  
    case 12..13: stack_adj = [ 80| 96|112|128];  
    case 14: stack_adj = [ 96|112|128|144];  
    case 15: stack_adj = [112|128|144|160];  
}
```

描述:

将这条指令会将 `reglist` 中指定的寄存器压入（存储）到栈指针下方的内存中，然后通过将栈指针递减 `stackadj` 的值来创建栈帧，其中也包括了由 `spimm` 的值所请求的任何额外的栈空间。



所有 ABI 寄存器映射均为 UABI。计划在 EABI 冻结后推出 EABI 版本。

有关更多信息，请参见 PUSH/POP 注册说明。

堆栈调整计算：

stackadjbase 是覆盖寄存器列表中寄存器所需的最小字节数，以 16 字节地址增量为单位向上取整。spimm 是为堆栈帧分配的额外 16 字节地址增量的数量。总堆栈调整量表示堆栈帧的总大小，它是 stackadjbase 加上 spimm 乘以 16 的结果，如上所述。

先决条件：

无

32 位等效：

不存在直接等效编码

操作：

伪代码的第一部分可能会在指令成功完成之前多次执行。

SAIL 代码

//这不是 SAIL，这是伪代码。SAIL 尚未编写。

```
if (XLEN==32) bytes=4; else bytes=8;
```

```
addr=sp-bytes;
```

```
for(i in 27,26,25,24,23,22,21,20,19,18,9,8,1) {
    //如果寄存器 i 在 xreglist 中
    if (xreglist[i]) {
        switch(bytes) {
            4: asm(sw x[i], 0(addr));
            8: asm(sd x[i], 0(addr));
        }
        addr-=bytes;
    }
}
```

伪代码的最后部分是原子执行的，并且仅在上述部分完成且没有任何异常或中断时执行。

SAIL 代码

//这不是 SAIL，这是伪代码。SAIL 尚未编写。

```
sp-=stackadj;
```


27.13.8 cm.pop

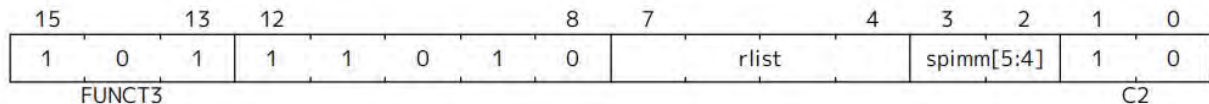
简述:

销毁堆栈帧: 从栈帧加载返回地址 ra 和 0 到 12 个保存的寄存器, 释放堆栈帧。

助记符:

cm.pop {reglist}, stackadj

编码 (RV32, RV64):



rlist 值 0 到 3 保留给未来的 EABI 变体, 称为 cm.pop.e

汇编语法:

cm.pop {reglist}, stackadj

cm.pop {xreglist}, stackadj

汇编语法中使用的变量定义如下。

RV32E:

```
switch (rlist){
  case 4: {reg_list="ra";      xreg_list="x1";}
  case 5: {reg_list="ra, s0";  xreg_list="x1, x8";}
  case 6: {reg_list="ra, s0-s1"; xreg_list="x1, x8-x9";}
  default: reserved();
}
stack_adj      = stack_adj_base + spimm[5:4] * 16;
```

RV32I, RV64:

```
switch (rlist){
  case 4: {reg_list="ra";      xreg_list="x1";}
  case 5: {reg_list="ra, s0";  xreg_list="x1, x8";}
  case 6: {reg_list="ra, s0-s1"; xreg_list="x1, x8-x9";}
  case 7: {reg_list="ra, s0-s2"; xreg_list="x1, x8-x9, x18";}
  case 8: {reg_list="ra, s0-s3"; xreg_list="x1, x8-x9, x18-x19";}
  case 9: {reg_list="ra, s0-s4"; xreg_list="x1, x8-x9, x18-x20";}
  case 10: {reg_list="ra, s0-s5"; xreg_list="x1, x8-x9, x18-x21";}
  case 11: {reg_list="ra, s0-s6"; xreg_list="x1, x8-x9, x18-x22";}
  case 12: {reg_list="ra, s0-s7"; xreg_list="x1, x8-x9, x18-x23";}
  case 13: {reg_list="ra, s0-s8"; xreg_list="x1, x8-x9, x18-x24";}
  case 14: {reg_list="ra, s0-s9"; xreg_list="x1, x8-x9, x18-x25";}
  //note - to include s10, s11 must also be included
```

```

    case 15: {reg_list="ra, s0-s11"; xreg_list="x1, x8-x9, x18-x27";}
    default: reserved();
}
stack_adj      = stack_adj_base + spimm[5:4] * 16;

```

RV32E:

```

stack_adj_base = 16;
Valid values:
stack_adj      = [16|32|48|64];

```

RV32I:

```

switch (rlist) {
    case 4..7: stack_adj_base = 16;
    case 8..11: stack_adj_base = 32;
    case 12..14: stack_adj_base = 48;
    case 15: stack_adj_base = 64;
}

```

Valid values:

```

switch (rlist) {
    case 4..7: stack_adj = [16|32|48| 64];
    case 8..11: stack_adj = [32|48|64| 80];
    case 12..14: stack_adj = [48|64|80| 96];
    case 15: stack_adj = [64|80|96|112];
}

```

RV64:

```
switch (rlist) {
    case 4.. 5: stack_adj_base = 16;
    case 6.. 7: stack_adj_base = 32;
    case 8.. 9: stack_adj_base = 48;
    case 10..11: stack_adj_base = 64;
    case 12..13: stack_adj_base = 80;
    case 14: stack_adj_base = 96;
    case 15: stack_adj_base = 112;
}
```

Valid values:

```
switch (rlist) {
    case 4.. 5: stack_adj = [ 16| 32| 48| 64];
    case 6.. 7: stack_adj = [ 32| 48| 64| 80];
    case 8.. 9: stack_adj = [ 48| 64| 80| 96];
    case 10..11: stack_adj = [ 64| 80| 96|112];
    case 12..13: stack_adj = [ 80| 96|112|128];
    case 14: stack_adj = [ 96|112|128|144];
    case 15: stack_adj = [112|128|144|160];
}
```

描述:

此指令从堆栈内存中弹出（加载）reglist 中的寄存器，然后通过 stackadj 调整堆栈指针。



注：所有 ABI 寄存器映射均为 UABI。EABI 版本将在 EABI 冻结之后发布。

堆栈调整计算

stackadjbase 是覆盖列表中寄存器所需的最小字节数，以 16 字节地址增量的倍数表示。spimm 是为堆栈帧分配的额外 16 字节地址增量的数量。

总堆栈调整表示堆栈帧的总大小，即 stackadjbase 加上 spimm 按 16 缩放，如上所定义。

先决条件:

无

32 位等效:

不存在直接等效编码

操作:

伪代码的第一部分可能会在指令成功完成之前多次执行。

SAIL 代码

//这不是 SAIL，这是伪代码。SAIL 尚未编写。

```
if (XLEN==32) bytes=4; else bytes=8;
```

```

addr=sp+stackadj-bytes;
for(i in 27,26,25,24,23,22,21,20,19,18,9,8,1) {
    //如果寄存器 i 在 xreglist 中
    if (xreglist[i]) {
        switch(bytes) {
            4:  asm(lw x[i], 0(addr));
            8:  asm(ld x[i], 0(addr));
        }
        addr-=bytes;
    }
}

```

伪代码的最后部分是原子执行的，并且仅在上述部分完成且没有任何异常或中断时执行。
SAIL 代码

```

//这不是 SAIL，这是伪代码。SAIL 尚未编写。
sp+=stackadj;

```

27.13.9 cm.popretz

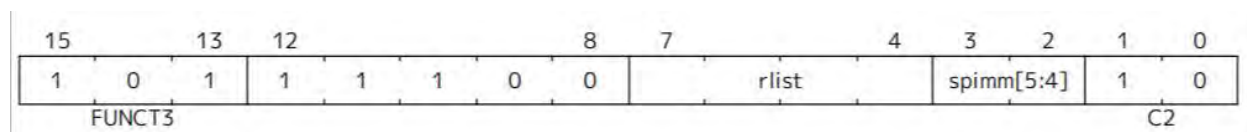
简述：

销毁堆栈帧：从堆栈帧加载返回地址 ra 和 0 到 12 个保存的寄存器，释放堆栈帧，将零移动到 a0，返回到 ra。

助记符：

cm.popretz {reglist}, stackadj

编码（RV32，RV64）：



汇编语法：

```

cm.popretz {reglist}, stackadj
cm.popretz {xreglist}, stackadj

```

RV32E:

```
switch (rlist){
  case 4: {reg_list="ra";      xreg_list="x1";}
  case 5: {reg_list="ra, s0";  xreg_list="x1, x8";}
  case 6: {reg_list="ra, s0-s1"; xreg_list="x1, x8-x9";}
  default: reserved();
}
stack_adj      = stack_adj_base + spimm[5:4] * 16;
```

RV32I, RV64:

```
switch (rlist){
  case 4: {reg_list="ra";      xreg_list="x1";}
  case 5: {reg_list="ra, s0";  xreg_list="x1, x8";}
  case 6: {reg_list="ra, s0-s1"; xreg_list="x1, x8-x9";}
  case 7: {reg_list="ra, s0-s2"; xreg_list="x1, x8-x9, x18";}
  case 8: {reg_list="ra, s0-s3"; xreg_list="x1, x8-x9, x18-x19";}
  case 9: {reg_list="ra, s0-s4"; xreg_list="x1, x8-x9, x18-x20";}
  case 10: {reg_list="ra, s0-s5"; xreg_list="x1, x8-x9, x18-x21";}
  case 11: {reg_list="ra, s0-s6"; xreg_list="x1, x8-x9, x18-x22";}
  case 12: {reg_list="ra, s0-s7"; xreg_list="x1, x8-x9, x18-x23";}
  case 13: {reg_list="ra, s0-s8"; xreg_list="x1, x8-x9, x18-x24";}
  case 14: {reg_list="ra, s0-s9"; xreg_list="x1, x8-x9, x18-x25";}
  //note - to include s10, s11 must also be included
  case 15: {reg_list="ra, s0-s11"; xreg_list="x1, x8-x9, x18-x27";}
  default: reserved();
}
stack_adj      = stack_adj_base + spimm[5:4] * 16;
```

RV32E:

```
stack_adj_base = 16;
Valid values:
stack_adj      = [16|32|48|64];
```


RV32I:

```
switch (rlist) {
    case 4.. 7: stack_adj_base = 16;
    case 8..11: stack_adj_base = 32;
    case 12..14: stack_adj_base = 48;
    case      15: stack_adj_base = 64;
}
```

Valid values:

```
switch (rlist) {
    case 4.. 7: stack_adj = [16|32|48| 64];
    case 8..11: stack_adj = [32|48|64| 80];
    case 12..14: stack_adj = [48|64|80| 96];
    case      15: stack_adj = [64|80|96|112];
}
```

RV64:

```
switch (rlist) {
    case 4.. 5: stack_adj_base = 16;
    case 6.. 7: stack_adj_base = 32;
    case 8.. 9: stack_adj_base = 48;
    case 10..11: stack_adj_base = 64;
    case 12..13: stack_adj_base = 80;
    case      14: stack_adj_base = 96;
    case      15: stack_adj_base = 112;
}
```

Valid values:

```
switch (rlist) {
    case 4.. 5: stack_adj = [ 16| 32| 48| 64];
    case 6.. 7: stack_adj = [ 32| 48| 64| 80];
    case 8.. 9: stack_adj = [ 48| 64| 80| 96];
    case 10..11: stack_adj = [ 64| 80| 96|112];
    case 12..13: stack_adj = [ 80| 96|112|128];
    case      14: stack_adj = [ 96|112|128|144];
    case      15: stack_adj = [112|128|144|160];
}
```

描述:

此指令从堆栈内存中弹出(加载) `reglist` 中的寄存器, 调整堆栈指针 `stackadj`, 将零移动到 `a0`, 然后返回到 `ra`。

有关更多信息, 请参见 `insns-pushpop`。

堆栈调整计算:

`stackadjbase` 是覆盖列表中寄存器所需的最小字节数, 以 16 字节地址增量的倍数表示。`spimm` 是为堆栈帧分配的额外 16 字节地址增量的数量。

总堆栈调整表示堆栈帧的总大小，即 `stackadjbase` 加上 `spimm` 按 16 缩放，如上所定义。

先决条件：

无

32 位等效：

不存在直接等效编码

操作：

伪代码的第一部分可能会在指令成功完成之前多次执行。

SAIL 代码

```
// 这不是 SAIL，这是伪代码。SAIL 尚未编写。
if (XLEN==32) bytes=4; else bytes=8;
addr=sp+stackadj-bytes;
for(i in 27,26,25,24,23,22,21,20,19,18,9,8,1) {
    // 如果寄存器 i 在 xreglist 中
    if (xreglist[i]) {
        switch(bytes) {
            4: asm(lw x[i], 0(addr));
            8: asm(ld x[i], 0(addr));
        }
        addr-=bytes;
    }
}
```

伪代码的最后一部分是原子执行的，并且仅在上述部分完成且没有任何异常或中断时执行。



注： `li a0, 0` *可能* 会多次执行，但为了方便起见包含于原子部分中。

SAIL 代码

```
// 这不是 SAIL，这是伪代码。SAIL 尚未编写
asm(li a0, 0);
sp+=stackadj;
asm(ret);
```

27.13.10 `cm.popret`

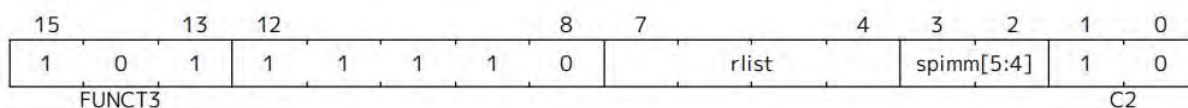
简述：

销毁堆栈帧：从堆栈帧加载 `ra` 和 0 到 12 个保存的寄存器，释放堆栈帧，返回到 `ra`。

助记符:

```
cm.popret {reglist}, stackadj
```

编码 (RV32, RV64):



rlist 值 0 到 3 保留给未来的 EABI 变体, 称为 cm.popret.e

汇编语法:

```
cm.popret {reglist}, stackadj
```

```
cm.popret {xreglist}, stackadj
```

汇编语法中使用的变量定义如下。

RV32E:

```
switch (rlist){
  case 4: {reg_list="ra";      xreg_list="x1";}
  case 5: {reg_list="ra, s0";  xreg_list="x1, x8";}
  case 6: {reg_list="ra, s0-s1"; xreg_list="x1, x8-x9";}
  default: reserved();
}
stack_adj      = stack_adj_base + spimm[5:4] * 16;
```

RV32I, RV64:

```
switch (rlist){
  case 4: {reg_list="ra";      xreg_list="x1";}
  case 5: {reg_list="ra, s0";  xreg_list="x1, x8";}
  case 6: {reg_list="ra, s0-s1"; xreg_list="x1, x8-x9";}
  case 7: {reg_list="ra, s0-s2"; xreg_list="x1, x8-x9, x18";}
  case 8: {reg_list="ra, s0-s3"; xreg_list="x1, x8-x9, x18-x19";}
  case 9: {reg_list="ra, s0-s4"; xreg_list="x1, x8-x9, x18-x20";}
  case 10: {reg_list="ra, s0-s5"; xreg_list="x1, x8-x9, x18-x21";}
  case 11: {reg_list="ra, s0-s6"; xreg_list="x1, x8-x9, x18-x22";}
  case 12: {reg_list="ra, s0-s7"; xreg_list="x1, x8-x9, x18-x23";}
  case 13: {reg_list="ra, s0-s8"; xreg_list="x1, x8-x9, x18-x24";}
  case 14: {reg_list="ra, s0-s9"; xreg_list="x1, x8-x9, x18-x25";}
  //note - to include s10, s11 must also be included

  case 15: {reg_list="ra, s0-s11"; xreg_list="x1, x8-x9, x18-x27";}
  default: reserved();
}
stack_adj      = stack_adj_base + spimm[5:4] * 16;
```

RV32E:

```
stack_adj_base = 16;  
Valid values:  
stack_adj      = [16|32|48|64];
```

RV32I:

```
switch (rlist) {  
    case 4.. 7: stack_adj_base = 16;  
    case 8..11: stack_adj_base = 32;  
    case 12..14: stack_adj_base = 48;  
    case      15: stack_adj_base = 64;  
}
```

Valid values:

```
switch (rlist) {  
    case 4.. 7: stack_adj = [16|32|48| 64];  
    case 8..11: stack_adj = [32|48|64| 80];  
    case 12..14: stack_adj = [48|64|80| 96];  
    case      15: stack_adj = [64|80|96|112];  
}
```

RV64:

```
switch (rlist) {  
    case 4.. 5: stack_adj_base = 16;  
    case 6.. 7: stack_adj_base = 32;  
    case 8.. 9: stack_adj_base = 48;  
    case 10..11: stack_adj_base = 64;  
    case 12..13: stack_adj_base = 80;  
    case      14: stack_adj_base = 96;  
    case      15: stack_adj_base = 112;  
}
```

Valid values:

```
switch (rlist) {  
    case 4.. 5: stack_adj = [ 16| 32| 48| 64];  
    case 6.. 7: stack_adj = [ 32| 48| 64| 80];  
    case 8.. 9: stack_adj = [ 48| 64| 80| 96];  
    case 10..11: stack_adj = [ 64| 80| 96|112];  
    case 12..13: stack_adj = [ 80| 96|112|128];  
    case      14: stack_adj = [ 96|112|128|144];  
    case      15: stack_adj = [112|128|144|160];  
}
```

描述:

此指令从堆栈内存中弹出(加载)reglist 中的寄存器,调整堆栈指针 stackadj,然后返回到 ra。



所有 ABI 寄存器映射均为 UABI。计划在 EABI 冻结后推出 EABI 版本。

有关更多信息，请参见 `insns-pushpop`。

堆栈调整计算：

`stackadjbase` 是覆盖列表中寄存器所需的最小字节数，以 16 字节地址增量的倍数表示。`spimm` 是为堆栈帧分配的额外 16 字节地址增量的数量。

总堆栈调整表示堆栈帧的总大小，即 `stackadjbase` 加上 `spimm` 按 16 缩放，如上所定义。

先决条件：

无

32 位等效：

不存在直接等效编码

操作：

伪代码的第一部分可能会在指令成功完成之前多次执行。

SAIL 代码

// 这不是 SAIL，这是伪代码。SAIL 尚未编写。

```
if (XLEN==32) bytes=4; else bytes=8;
addr=sp+stackadj-bytes;
for(i in 27,26,25,24,23,22,21,20,19,18,9,8,1) {
    // 如果寄存器 i 在 xreglist 中
    if (xreglist[i]) {
        switch(bytes) {
            4: asm(lw x[i], 0(addr));
            8: asm(ld x[i], 0(addr));
        }
        addr-=bytes;
    }
}
```

伪代码的最后一部分是原子执行的，并且仅在上述部分完成且没有任何异常或中断时执行。

SAIL 代码

// 这不是 SAIL，这是伪代码。SAIL 尚未编写。

```
sp+=stackadj;
asm(ret);
```

<

27.13.11 cm.mvsa01

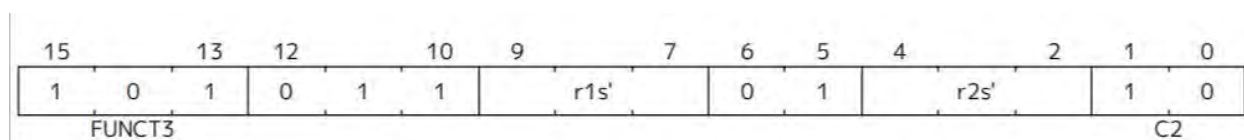
简述:

将 a0-a1 移动到 s0-s7 的两个寄存器

助记符:

cm.mvsa01 r1s, r2s

编码 (RV32, RV64):



编码合法时 r1s != r2s。

汇编语法:

cm.mvsa01 r1s, r2s

描述:

该指令将 a0 移动到 r1s, 并将 a1 移动到 r2s。r1s 和 r2s 必须不同。

执行是原子性地, 因此不可能观察到仅更新了 r1s 或 r2s 之一的状态。

编码使用 sreg 编号说明符而不是 xreg 编号说明符以节省编码空间。

它们之间的映射在下面的伪代码中指定。



s 寄存器映射取自 UABI, 可能与当前未批准的 EABI 不匹配。cm.mvsa01.e 可能会在未来包含。

先决条件:

无

32 位等效:

不存在直接等效编码。

操作:

```
//This is not SAIL, it's pseudo-code. The SAIL hasn't been written yet.
if (RV32E && (r1sc>1 || r2sc>1)) {
    reserved();
}
xreg1 = {r1sc[2:1]>0,r1sc[2:1]==0,r1sc[2:0]};
xreg2 = {r2sc[2:1]>0,r2sc[2:1]==0,r2sc[2:0]};
X[xreg1] = X[10];
X[xreg2] = X[11];
```

27.13.12 cm.mva01s

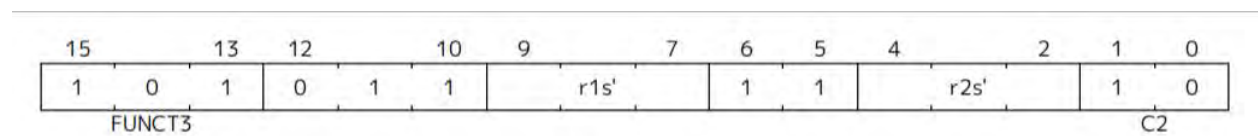
简述:

将两个 s0-s7 寄存器移动到 a0-a1

助记符:

cm.mva01s r1s, r2s

编码 (RV32, RV64):



汇编语法:

cm.mva01s r1s, r2s

描述:

该指令将 r1s 移动到 a0,并将 r2s 移动到 a1。执行是原子的,因此不可能观察到仅更新了 a0 或 a1 之一的状态。

编码使用 sreg 编号说明符而不是 xreg 编号说明符以节省编码空间。

它们之间的映射在下面的伪代码中指定。



注: s 寄存器映射取自 UABI, 可能与当前未批准的 EABI 不匹配。cm.mva01s.e 可能会在未来包含。

先决条件:

无

32 位等效:

不存在直接等效编码。

操作:


```
//This is not SAIL, it's pseudo-code. The SAIL hasn't been written yet.
if (RV32E && (r1sc>1 || r2sc>1)) {
    reserved();
}
xreg1 = {r1sc[2:1]>0,r1sc[2:1]==0,r1sc[2:0]};
xreg2 = {r2sc[2:1]>0,r2sc[2:1]==0,r2sc[2:0]};
X[10] = X[xreg1];
X[11] = X[xreg2];
```

27.14 表跳转概述

cm.jt (Jump via table) 和 cm.jalt (Jump and link via table) 被称为表跳转。表跳转使用指令存储器中的 256 项的 XLEN 宽表来包含函数地址。该表必须至少对齐 64 字节。表项遵循当前的数据字节序。这与始终为小端字节序的正常指令获取不同。

cm.jt 和 cm.jalt 编码索引表，允许访问整个 XLEN 宽地址空间内的函数。这被用作一种字典压缩形式，以减少 jal / auipc+jalr / jr / auipc+jr 指令的代码大小。

表跳转允许链接器用 cm.jt 或 cm.jalt 编码和表中的一个条目替换以下指令序列：

- * 32 位 j 调用
- * 32 位 jal ra 调用
- * 64 位 auipc+jr 调用到固定位置
- * 64 位 auipc+jalr ra 调用到固定位置
- ** 使用 auipc+jr/jalr 序列是因为 PC 的偏移量超出了 $\pm 1\text{MB}$ 范围。

如果实现了返回地址堆栈，则由于 cm.jalt 等效于 jal ra，它会推送到堆栈。

27.14.1 jvt

表的基地址在 jvt 控制状态寄存器中（见 jvt-CSR、表跳转基向量和控制寄存器），每个表项为 XLEN 位。如果同一个函数在有和没有链接的情况下被调用，则它必须在表中有两个条目。这通常是由于同一个函数在有和没有尾调用的情况下被调用造成的。

27.14.2 表跳转故障处理

对于表跳转指令，指令选择的表项被视为指令本身的扩展。因此，执行表跳转指令涉及两次指令获取，第一次读取指令（cm.jt/cm.jalt），第二次从跳转向量表（JVT）读取。这两个指令获取都是隐式读取，并且都需要执行权限；读取权限无关紧要。建议忽略第二次获取的硬件触发器和断点。对跳转向量表的内存写入需要指令屏障（fence.i）以确保它们对指令获取可见。

多个上下文可能有不同的跳转向量表。如果自上次 fence.i 以来表在内存中没有更新，则可以在它们之间切换 JVT 而无需指令屏障。如果在任一指令获取上发生异常，xEPC 设置为表跳转指令的 PC，xCAUSE 设置为预期的故障类型，如果 xTVAL 未设置为零，则包含导致故障的获取地址。

27.14.3 jvt 控制状态寄存器

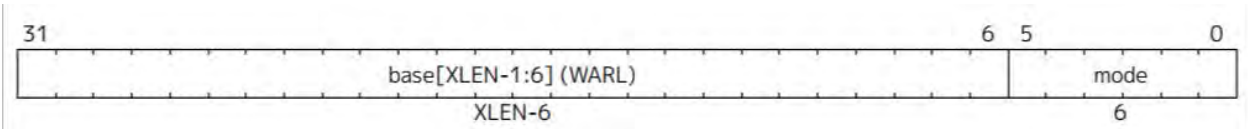
简述：

表跳转基向量和控制寄存器

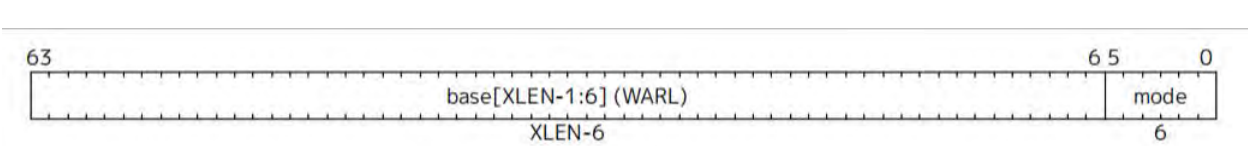
地址：
0x0017

权限：
URW

格式 (RV32):



格式 (RV64):



描述：

`jvt` 寄存器是一个 `XLEN` 位 *WARL* 读/写寄存器, 包含跳转表配置, 包括跳转表基地址 (BASE) 和跳转表模式 (MODE)。

如果实现了 第 27.10 章, 则 `jvt` 也必须实现, 但可以包含只读值。如果 `jvt` 是可写的, 则寄存器可能持有的值集可能因实现而异。BASE 字段中的值必须始终在 64 字节边界上对齐。

如果虚拟内存被启用, 则 `jvt.base` 是一个虚拟地址。`jvt.base` 指向的内存被视为指令内存, 用于执行表跳转指令, 这意味着执行访问权限。

`jvt.mode` 是一个 *WARL* 字段, 因此只能编程为已实现的模式。因此, 发现机制是尝试编程不同的模式并读回值以查看哪些可用。必须实现跳转表模式。



注：将来, RISC-V 统一发现方法将报告可用的模式。

架构状态：

`jvt` 控制状态寄存器向系统软件上下文（如操作系统进程）添加架构状态, 因此必须在上下文切换时保存/恢复。

状态启用：

如果实现了 `Smstateen` 扩展, 则在 `mstateen0`、`sstateen0` 和 `hstateen0` 中的第 2 位也会被实现。如果控制 `stateen0` CSR 的第 2 位为零, 则较低特权级别访问 `jvt` CSR 和执行 `cm.jalt` 或 `cm.jt` 指令会导致非法指令陷阱（或者, 如果适用, 则是虚拟指令陷阱）。

27.14.4 cm.jt

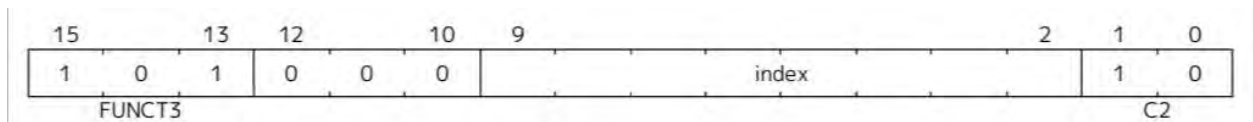
简述：

通过表跳转

助记符:

cm.jt index

编码 (RV32, RV64):



对于此编码解码为 cm.jt, index<32, 否则解码为 cm.jalt, 见 [通过表格跳转和链接](#)。



如果 jvt.mode = 0 (跳转表模式), 则 cm.jt 按此处指定的行为。如果 jvt.mode 是保留值, 则 cm.jt 也是保留的。将来, jvt.mode 的其他定义值可能会改变 cm.jt 的行为。

汇编语法:

SAIL 代码

cm.jt index

描述:

cm.jt 从内存中的跳转向量表读取一个条目并跳转到读取的地址。
有关更多信息, 请参见表格跳转概述。

先决条件:

无

32 位等效:

不存在直接等效编码。

操作:

SAIL 代码

//这不是 SAIL, 这是伪代码。SAIL 尚未编写。

targetaddress 是临时内部状态, 不代表真实寄存器

InstMemory 是字节索引

```
switch(XLEN) {
  32: table_address[XLEN-1:0] = jvt.base + (index2);
  64: table_address[XLEN-1:0] = jvt.base + (index3);
}
```

```
//从跳转表获取
target_address[XLEN-1:0] = InstMemory[table_address][XLEN-1:0];

j target_address[XLEN-1:0]&~0x1;
```

27.14.5 cm.jalt

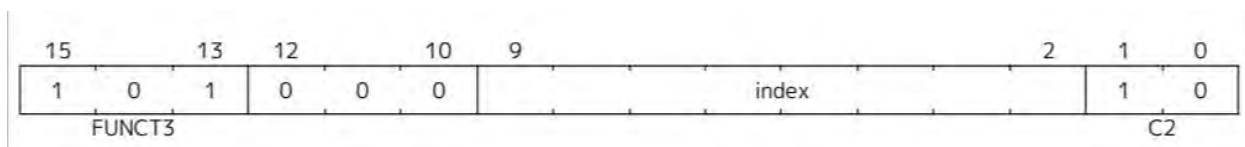
简述:

带有可选链接的通过表跳转

助记符:

cm.jalt index

编码 (RV32, RV64):



注: 对于此编码应解码为 cm.jalt, index>=32, 否则解码为 cm.jt, 见 insns-cmjt。



注: 如果 jvt.mode = 0 (跳转表模式), 则 cm.jalt 按此处指定的行为。如果 jvt.mode 是保留值, 则 cm.jalt 也是保留的。将来, jvt.mode 的其他定义值可能会改变 cm.jalt 的行为。

汇编语法:

SAIL 代码

cm.jalt index

描述:

cm.jalt 从内存中的跳转向量表读取一个条目并跳转到读取的地址, 链接到 ra。

有关更多信息, 请参见 insns-tablejump。

先决条件:

无

32 位等效:

不存在直接等效编码。

[#insns-cm_jalt-SAIL, reftext=cm.jalt SAIL 代码]

操作:

SAIL 代码

```
//这不是 SAIL，这是伪代码。SAIL 尚未编写。
# target_address 是临时内部状态，不代表真实寄存器
# InstMemory 是字节索引

switch(XLEN) {
    32: table_address[XLEN-1:0] = jvt.base + (index2);
    64: table_address[XLEN-1:0] = jvt.base + (index3);
}

//从跳转表获取
target_address[XLEN-1:0] = InstMemory[table_address][XLEN-1:0];
jal ra, target_address[XLEN-1:0]&~0x1;
```

28 B 扩展：位操作指令，V1.0.0

B 标准扩展包括由 Zba、Zbb 和 Zbs 扩展提供的指令。位操作 a、b、c 和 s 扩展组合（供公众审查和批准）

28.1 位操作扩展（a、b、c 和 s 扩展）已合并进入公开评审及批准流程

位操作（bitmanip）扩展集由几个基本 RISC-V 架构的扩展组成，旨在提供代码大小减少、性能提升和功耗减少的组合。虽然这些指令旨在具有通用用途，但某些指令在某些领域比其他领域更有用。因此，提供的是几个较小的位操作扩展，而不是一个大的扩展。每个较小的扩展根据共同的功能和使用场景进行分组，每个扩展都有其独立的 Zb*-<扩展名称>。

每个位操作扩展都包括一组功能相似的位操作指令，这些指令通常可以共享相同的逻辑。某些指令仅在一个扩展中可用，而其他指令则在多个扩展中可用。这些指令的助记符和编码独立于它们所在的扩展。因此，在实现具有重复指令的扩展时，逻辑和编码不会产生冗余。

位操作扩展适用于 RV32 和 RV64。大多数指令预计与 RV128 向前兼容。虽然移位立即数指令的定义最多支持 6 位立即数字段，但如果 RV128 需要，可以在编码空间中提供第 7 位。

28.2 字指令

位操作扩展遵循 RV64 中的约定，带有 w 后缀的指令（w 前没有点）会忽略其输入的高 32 位，操作低 32 位作为有符号值，并生成一个符号位扩展到 XLEN 的 32 位有符号结果。

带有 .uw 后缀的位操作指令则将操作数视为从指定寄存器的最低 32 位提取的无符号 32 位值。除此之外，这些指令执行完整的 XLEN 操作。

带有 .b、.h 和 .w 后缀的位操作指令仅查看输入的最低 8 位、16 位和 32 位（分别），并生成符号扩展或零扩展的 XLEN 位宽结果，具体取决于特定指令。

#insns 中每条指令的语义以类似 SAIL 的语法表示。

28.3 指令语义伪代码规范

本规范采用类 SAIL 语法描述指令集中各条指令的语义。

28.4 扩展

第一组发布供公众审查的位操作扩展是：

- 地址生成说明
- 基本位操作
- 减少乘法运算
- 单比特指令

以下是这些扩展中包含的所有指令及其特定映射的列表：

表 28.1 扩展中包含的所有指令及其特定映射的列表

RV32	RV64	助记符	指令	Zba	Zbb	Zbc	Zbs
	✓	add.uw rd, rs1, rs2		✓			
✓	✓	andn rd, rs1, rs2			✓		
✓	✓	clmul rd, rs1, rs2				✓	
✓	✓	clmulh rd, rs1, rs2				✓	
✓	✓	clmulr rd, rs1, rs2				✓	
✓	✓	clz rd, rs			✓		
	✓	clzw rd, rs			✓		
✓	✓	cpop rd, rs			✓		
	✓	cpopw rd, rs			✓		
✓	✓	ctz rd, rs			✓		
	✓	ctzw rd, rs			✓		
✓	✓	max rd, rs1, rs2			✓		
✓	✓	maxu rd, rs1, rs2			✓		
✓	✓	min rd, rs1, rs2			✓		
✓	✓	minu rd, rs1, rs2			✓		
✓	✓	orc.b rd, rs			✓		
✓	✓	orn rd, rs1, rs2			✓		
✓	✓	rev8 rd, rs			✓		
✓	✓	rol rd, rs1, rs2			✓		
	✓	rolw rd, rs1, rs2			✓		
✓	✓	ror rd, rs1, rs2			✓		
✓	✓	rori rd, rs1, shamt			✓		
	✓	roriw rd, rs1, shamt			✓		
✓	✓	rorw rd, rs1, rs2			✓		
✓	✓	bclr rd, rs1, rs2					✓
✓	✓	bclri rd, rs1, imm					✓
✓	✓	bext rd, rs1, rs2					✓
✓	✓	bexti rd, rs1, imm					✓
✓	✓	binv rd, rs1, rs2					✓
✓	✓	binvi rd, rs1, imm					✓
✓	✓	bset rd, rs1, rs2					✓

✓	✓	bseti rd, rs1, imm					✓
✓	✓	sext.b rd, rs			✓		
✓	✓	sext.h rd, rs			✓		
	✓	shladd rd, rs1, rs2		✓			
✓	✓	shladd.uw rd, rs1, rs2		✓			
	✓	sh2add rd, rs1, rs2		✓			
✓	✓	sh2add.uw rd, rs1, rs2		✓			
	✓	sh3add rd, rs1, rs2		✓			
✓	✓	sh3add.uw rd, rs1, rs2		✓			
	✓	slli.uw rd, rs1, imm		✓			
✓	✓	xnor rd, rs1, rs2			✓		
✓	✓	zext.h rd, rs			✓		

28.4.1 Zba: 地址生成

Zba 指令用于加速生成指向基本类型（半字、字、双字）数组的索引地址，使用无符号字大小和 XLEN 大小的索引。通过将移位后的索引加到基地址上来实现。

移位和加法指令将左移 1、2 或 3 位，因为这些操作在实际代码中较为常见，且可以通过最少的额外硬件实现，超出简单加法器的范围。这可以避免延长实现中的关键路径。

虽然移位和加法指令的左移限制为最多 3 位，但可以使用 slli 指令（来自基本 ISA）对更宽元素的数组进行类似的移位操作。slli.uw 在此扩展中新增，可以当索引被解释为无符号字时使用。

表 28.2 指令（和伪指令）组成 Zba 扩展

RV32	RV64	助记符	介绍
✓	✓	add.uw rd, rs1, rs2	添加无符号单词
	✓	shladd rd, rs1, rs2	左移 1，然后相加
✓	✓	shladd.uw rd, rs1, rs2	将无符号字左移 1，然后相加
	✓	sh2add rd, rs1, rs2	左移 2，然后相加
✓	✓	sh2add.uw rd, rs1, rs2	将无符号字左移 2，然后相加
	✓	sh3add rd, rs1, rs2	左移 3，然后相加
	✓	sh3add.uw rd, rs1, rs2	将无符号字左移 3，然后相加
	✓	slli.uw rd, rs1, imm	左移无符号单词
	✓	zext.w rd, rs	添加无符号单词

28.4.2 Zbb: 基本位操做

表 28.3 带否定的逻辑操作

RV32	RV64	助记符	介绍
✓	✓	clz rd, rs	最大

	✓	clzw rd, rs	无符号最大
✓	✓	ctz rd, rs	最低
	✓	sh2add rd, rs1, rs2	无符号最低



逻辑非指令可以通过对基础所需的与、或和异或逻辑指令的 rs2 输入进行反转来实现。在某些实现中，用于减法的 rs2 反相器可以被用于此目的。

表 28.4 计数前导/后导零位

RV32	RV64	助记符	介绍
✓	✓	clz rd, rs	计数前导零位
	✓	clzw rd, rs	对 word 中的前导零位进行计数
✓	✓	ctz rd, rs	计数尾零位
	✓	ctzw rd, rs	对 word 中的尾零位进行计数

这些指令用于计算设置位（1 位）的数量。这个操作也通常称为位计数。

表 28.5 位计数

RV32	RV64	助记符	介绍
✓	✓	cpop rd, rs	计数设置位
	✓	cpopw rd, rs	以 word 计算集合位

符号位扩展和零扩展

这些指令对源寄存器的最低 8 位或 16 位执行符号扩展或零扩展。

这些指令替代了习惯用法 slli rd, rs, (XLEN-<size>) + srli（用于零扩展）或 slli + srli（用于符号扩展），用于对 8 位和 16 位量进行符号扩展，以及对 16 位量进行零扩展。

位旋转

位旋转指令类似于基本规范中的逻辑移位操作。然而，与逻辑移位指令将零移入不同，旋转指令将从值的另一侧移出的位移入。这种操作也被称为循环移位（circular shifts）。

表 28.6 位旋转表格图

RV32	RV64	助记符	介绍
✓	✓	rol rd, rs1, rs2	向左旋转（寄存器）
	✓	rolw rd, rs1, rs2	左旋字（寄存器）
✓	✓	ror rd, rs1, rs2	右转（寄存器）
✓	✓	rori rd, rs1, shamt	右转（立即）
	✓	roriw rd, rs1, shamt	右转单词（即时）
	✓	rorw rd, rs1, rs2	右转字（寄存器）



加入旋转指令是为了替换一个常见的四指令序列，以实现相同的效果（neg; sll/srl; srl/sll; or）

如果 rs 中相应字节的任何位都没有被设置，*orc.b* 将结果寄存器 rd 中每个字节的位设置为全零；如果 rs 中相应字节的任何一位被设置，则将结果字节的位设置为全一。

一个典型的应用场景是字符串处理函数，如 *strlen* 和 *strcpy*，它们可以使用 *orc.b* 通过计算字中的前导非零字节中的设置位，来测试终止零字节。

28.4.3 Zbc：无进位乘法指令

无进位乘法是在 GF(2) 上的多项式环中的乘法。

clmul 生成无进位乘积的下半部分，*clmulh* 生成 $2 \times XLEN$ 无进位乘积的上半部分。

clmulr 生成 $2 \times XLEN - 2 : XLEN - 1$ 的 $2 \times XLEN$ 无进位乘积。

表 28.7 无进位乘法指令

RV32	RV64	助记符	介绍
√	√	clmul rd, rs1, rs2	无进位乘法（低部）
√	√	clmulh rd, rs1, rs2	无进位乘法（高部）
√	√	clmulr rd, rs1, rs2	无进位乘法（反向）

28.4.4 Zbs：单比特指令

单比特指令提供了一种机制，用于在寄存器中设置、清除、反转或提取单比特位。该位由其索引指定。

表 28.8 单比特指令表

RV32	RV64	助记符	介绍
√	√	bclr rd, rs1, rs2	单位清除（寄存器）
√	√	bclri rd, rs1, imm	单位清除（即时）
√	√	bext rd, rs1, rs2	单位提取（寄存器）
√	√	bexti rd, rs1, imm	单位提取（即时）
√	√	binv rd, rs1, rs2	单位反转（寄存器）
√	√	binvi rd, rs1, imm	单位反转（即时）
√	√	bset rd, rs1, rs2	单位集（寄存器）
√	√	bseti rd, rs1, imm	单位集（即时）

28.4.5 Zbkb：加密中的位操作

表 28.9 Zbkb 扩展

RV32	RV64	助记符	介绍
√	√	rol	向左旋转（寄存器）

	✓	rolw	左旋字（寄存器）
✓	✓	ror	右转（寄存器）
✓	✓	rori	右转（立即）
	✓	roriw	右转单词（即时）
	✓	rorw	右转字（寄存器）
✓	✓	andn	带反向操作数的 AND
✓	✓	orn	与反向操作数的或
✓	✓	xnor	错误
✓	✓	pack	包装寄存器的低半部分
✓	✓	packh	封装寄存器的低字节数
	✓	packw	封装低 16 位寄存器（RV64）
✓	✓	brev8	反转位（字节）
✓	✓	rev8	Byte-reverse 注册
✓		zip	位交错
✓		unzip	deinterleave 位

28.4.6 Zbkc：用于加密的无进位乘法



Zbkc 已被冻结。

无进位乘法是在 $GF(2)$ 上的多项式环中的乘法。这是一些加密工作负载中的关键操作，特别是在 AES-GCM 认证加密方案中。此扩展仅提供高效实现 GHASH 操作所需的指令，GHASH 是该工作负载的一部分。

表 28.10 Zbkc 扩展

RV32	RV64	Mnemonic	Instruction
✓	✓	clmul rd, rs1, rs2	Carry-less multiply (low-part)
✓	✓	clmulh rd, rs1, rs2	Carry-less multiply (high-part)

28.4.7 Zbkx：交叉开关置换（Crossbar permutations）

这些指令实现了一个用于通用寄存器中的 4 位和 8 位元素的查找表。rs1 被用作一个 N 位字的向量，rs2 被用作中一个 N 位的索引向量，指向 rs1 中的元素。rs1 中的元素将被 rs2 中对应索引的元素替换，如果索引超出 rs2 的范围，则替换为零。

这些指令对于表达 N 位到 N 位的布尔操作非常有用，并且可以实现具有依赖于密钥的内存访问（特别是 SBox）的加密代码，使得执行延迟不依赖于被操作的（密钥）数据。

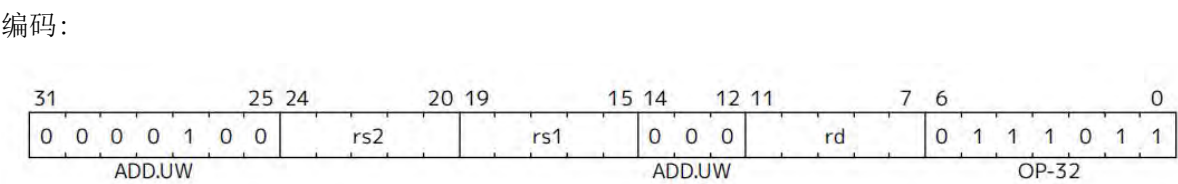
28.5 指令（按字母顺序排列）

28.5.1 add.uw

简述：
添加无符号字

助记符：
add.uw rd, rs1, rs2

伪指令：
zext.w rd, rs1 → add.uw rd, rs1, zero



描述：
此指令执行 rs2 与 rs1 的最低有效字零扩展后的 XLEN 宽度加法。

操作：

```
let base = X(rs2);
let index = EXTZ(X(rs1)[31..0]);

X(rd) = base + index;
```

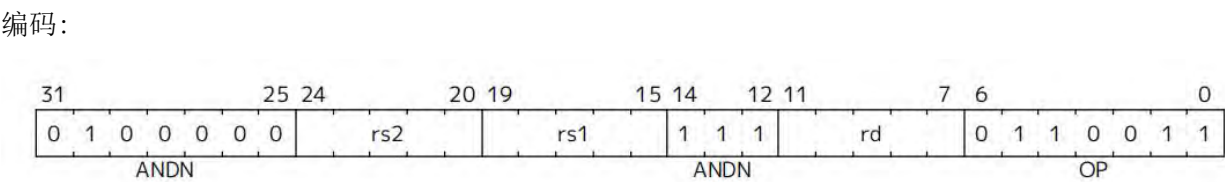
包含于

扩展	最低版本	生命周期阶段
Zba	0.93	冻结

28.5.2 andn

简述：
与反转操作数的按位与

助记符：
andn rd, rs1, rs2



描述：
此指令在 rs1 和 rs2 的按位反转之间执行按位逻辑与操作。

操作：

$X(rd) = X(rs1) \& \sim X(rs2);$

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结
Zbkb	V0.9.4	冻结

28.5.3 bclr

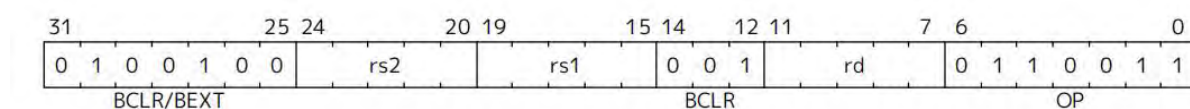
简述:

单比特清除（寄存器）

助记符:

bclr rd, rs1, rs2

编码:



描述:

此指令返回在 rs2 指定的索引处清除了单比特位的 rs1 的值。索引从 rs2 的低 $\log_2(XLEN)$ 位读取。

操作:

```
let index = X(rs2) & (XLEN - 1);
X(rd) = X(rs1) & ~(1 << index)
```

包含于

扩展	最低版本	生命周期阶段
Zbs	0.93	冻结

28.5.4 bclri

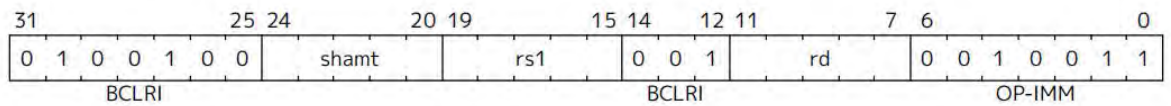
简述:

单比特清除（立即数）

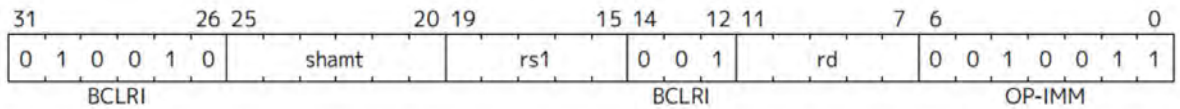
助记符:

bclri rd, rs1, shamt

编码（RV32）:



编码 (RV64):



描述:
 此指令返回在 shamt 指定的索引处清除单比特位的 rs1 的值。索引从 shamt 的低 log2(XLEN) 位读取。对于 RV32, 对应于 shamt[5]=1 的编码是保留的。

操作:
`let index = shamt & (XLEN - 1);`
`X(rd) = X(rs1) & ~(1 << index)`

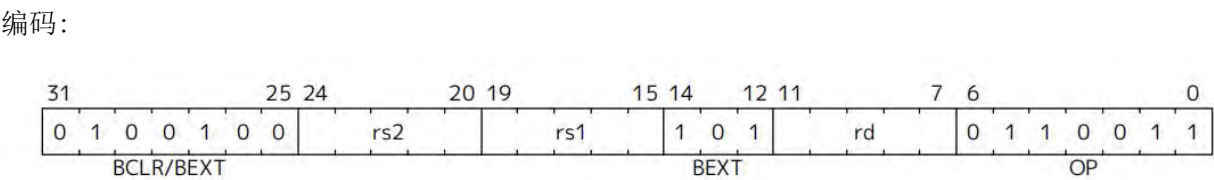
包含于

扩展	最低版本	生命周期阶段
Zbs	0.93	冻结

28.5.5 bext

简述:
 单比特位提取 (寄存器)
 // 我们是否应该将其描述为如果位被设置则设置?

助记符:
 bext rd, rs1, rs2



描述:
 此指令返回从 rs1 中提取的单比特位,索引位置由 rs2 指定。索引值从 rs2 的低 log2(XLEN) 位读取。

操作:
`let index = X(rs2) & (XLEN - 1);`
`X(rd) = (X(rs1) >> index) & 1;`

包含于

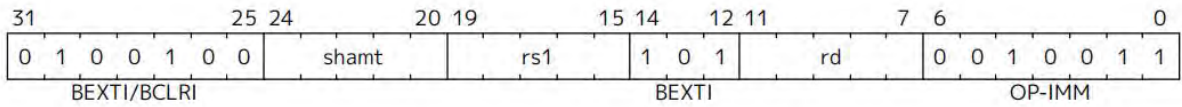
扩展	最低版本	生命周期阶段
Zbs	0.93	冻结

28.5.6 bexti

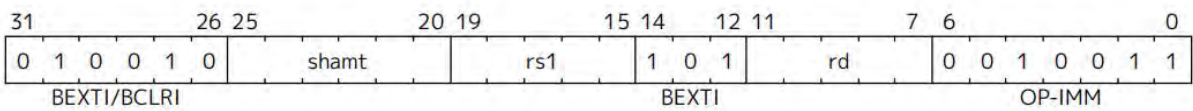
简述：
单比特位提取（立即数）

助记符：
bexti rd, rs1, shamt

编码（RV32）：



编码（RV64）：



描述：
此指令从 rs1 中提取一个比特位，该比特位的位置由 rs2 中指定的索引确定。索引值从 shamt 的低 $\log_2(XLEN)$ 位读取。对于 RV32，shamt[5]=1 对应的编码是保留的。

操作：

```
let index = shamt & (XLEN - 1);
X(rd) = (X(rs1) >> index) & 1;
```

包含于

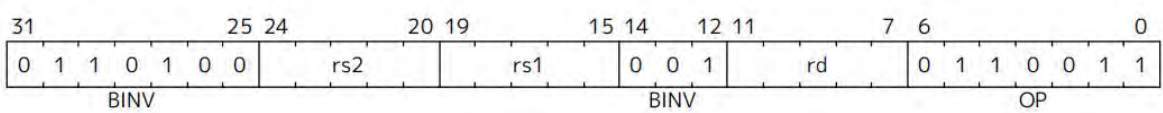
扩展	最低版本	生命周期阶段
Zbs	0.93	冻结

28.5.7 binv

简述：
单比特位翻转（寄存器）

助记符：
binv rd, rs1, rs2

编码：



描述

此指令返回 `rs1`，其在 `rs2` 中指定的索引位置的单比特位被翻转。索引值从 `rs2` 的低 $\log_2(\text{XLEN})$ 位读取。

操作

```
let index = X(rs2) & (XLEN - 1);
X(rd) = X(rs1) ^ (1 << index)
```

包含于

扩展	最低版本	生命周期阶段
Zbs	0.93	冻结

28.5.8 binvi

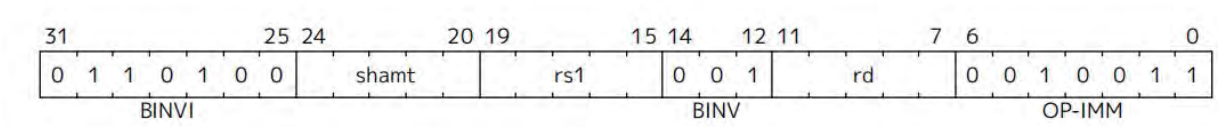
简述

单比特位翻转（立即数）

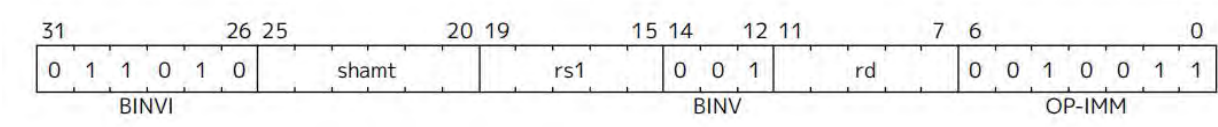
助记符

```
binvi rd, rs1, shamt
```

编码（RV32）



编码（RV64）



描述

此指令返回 `rs1`，其中在 `shamt` 中指定的索引位置的单比特位被翻转。索引从 `shamt` 的低 $\log_2(\text{XLEN})$ 位中读取。对于 RV32，`shamt[5]=1` 对应的编码是保留的。

操作：

```
let index = shamt & (XLEN - 1);
X(rd) = X(rs1) ^ (1 << index)
```

包含于

扩展	最低版本	生命周期阶段
Zbs	0.93	冻结

28.5.9 bset

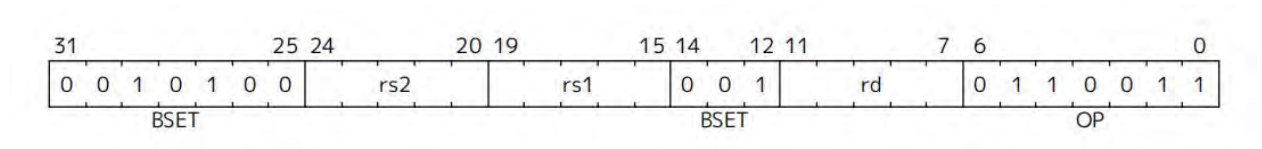
简述

单比特位设置（寄存器）

助记符

bset rd, rs1,rs2

编码:



描述:

此指令返回 rs1，其中在 rs2 中指定的索引位置设置单比特位。索引从 rs2 的低 $\log_2(XLEN)$ 位读取。

操作:

```
let index = X(rs2) & (XLEN - 1);  
X(rd) = X(rs1) | (1 << index)
```

包含于

扩展	最低版本	生命周期阶段
Zbs	0.93	冻结

28.5.10 bseti

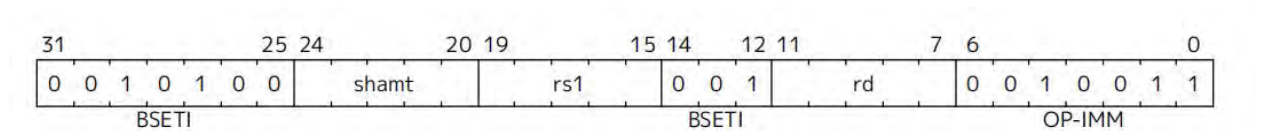
简述:

单比特位设置（立即数）

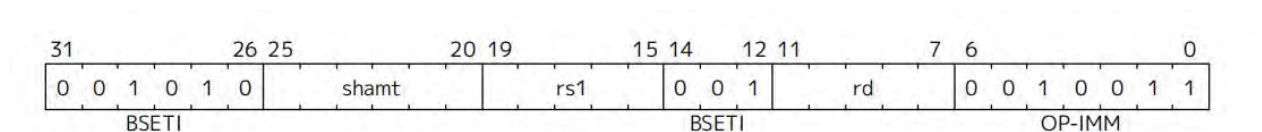
助记符:

bseti rd, rs1,shamt

编码 (RV32):



编码 (RV64):



描述:

此指令返回 rs1，并在 shamt 指定的索引位置设置单比特位。索引从 shamt 的低 $\log_2(XLEN)$ 位

读取。对于 RV32，shamt[5]=1 的编码是保留的。

操作：

```
let index = shamt & (XLEN - 1);
X(rd) = X(rs1) | (1 << index)
```

包含于

扩展	最低版本	生命周期阶段
Zbs	0.93	冻结

28.5.11 clmul

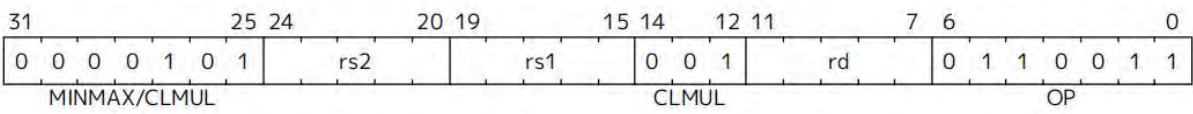
简述：

无进位乘法（低位部分）

助记符：

```
clmul rd, rs1, rs2
```

编码：



描述：

clmul 生成 2 · XLEN 无进位乘积的低半部分。

操作：

```
let rs1_val = X(rs1);
let rs2_val = X(rs2);
let output : xlenbits = 0;

foreach (i from 0 to (xlen - 1) by 1) {
    output = if ((rs2_val >> i) & 1)
               then output ^ (rs1_val << i);
               else output;
}

X[rd] = output
```

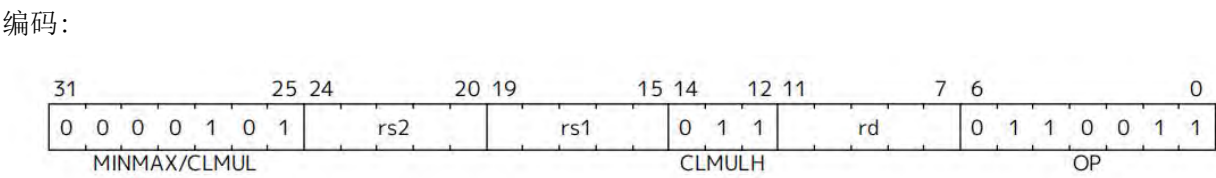
包含于

扩展	最低版本	生命周期阶段
Zbc	0.93	冻结
Zbkc	v0.9.4	冻结

28.5.12 clmulh

简述：
无进位乘法（高位部分）

助记符：
clmulh rd, rs1, rs2



描述：
clmulh 生成 $2 \cdot \text{XLEN}$ 无进位乘积的高半部分。

操作：

```
let rs1_val = X(rs1);
let rs2_val = X(rs2);
let output : xlenbits = 0;

foreach (i from 1 to xlen by 1) {
    output = if ((rs2_val >> i) & 1)
        then output ^ (rs1_val >> (xlen - i));
        else output;
}

X[rd] = output
```

包含于

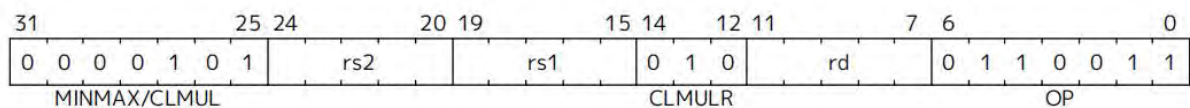
扩展	最低版本	生命周期阶段
Zbc	0.93	冻结
Zbkc	v0.9.4	冻结

28.5.13 clmulr

简述：
无进位乘法（反向）

助记符：
clmulr rd, rs1, rs2

编码：



描述:

clmulr 生成 2 • XLEN 无进位乘积的 2 • XLEN-2:XLEN-1 位。

```
操作:

let rs1_val = X(rs1);
let rs2_val = X(rs2);
let output : xlenbits = 0;

foreach (i from 0 to (xlen - 1) by 1) {
    output = if ((rs2_val >> i) & 1)
        then output ^ (rs1_val >> (xlen - i - 1));
        else output;
}

X[rd] = output
```

包含于

扩展	最低版本	生命周期阶段
Zbc	0.93	冻结

28.5.14 clz

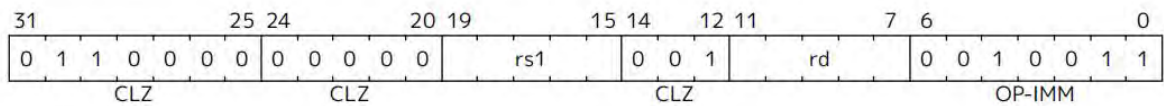
简述:

计数前导零位

助记符:

clz rd, rs

编码:



描述:

此指令从最高有效位（即 XLEN-1 位）开始，向最低有效位（即位 0）逐位计算第一个 1 之前的 0 的数量。因此，如果输入为 0，输出为 XLEN；如果输入的最高有效位为 1，输出为 0。

操作:

```
val HighestSetBit : forall ('N : Int), 'N >= 0. bits('N) -> int

function HighestSetBit x = {
  foreach (i from (xlen - 1) to 0 by 1 in dec)
    if [x[i]] == 0b1 then return(i) else ();
  return -1;
}

let rs = X(rs);
X[rd] = (xlen - 1) - HighestSetBit(rs);
```

包含于

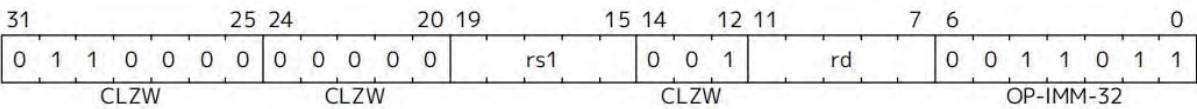
扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.15 clzw

简述：
计数字中的前导零位

助记符：
clzw rd, rs

编码：



描述：
此指令计算从位 31 开始到第一个 1 之前 0 的数量。因此，如果最低有效字为 0，输出为 32；如果字的最高有效位（即位 31）为 1，输出为 0。

操作：

```
val HighestSetBit32 : forall ('N : Int), 'N >= 0. bits('N) -> int

function HighestSetBit32 x = {
  foreach (i from 31 to 0 by 1 in dec)
    if [x[i]] == 0b1 then return(i) else ();
  return -1;
}

let rs = X(rs);
X[rd] = 31 - HighestSetBit(rs);
```

包含于

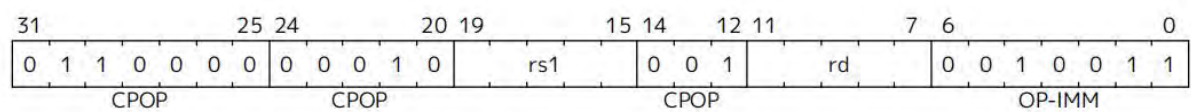
扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.16 cpop

简述：
计数设置位

助记符：
cpop rd, rs

编码：



描述：
此指令计算源寄存器中 1 的数量（即设置位）。

操作：

```
let bitcount = 0;
let rs = X(rs);

foreach (i from 0 to (xlen - 1) in inc)
    if rs[i] == 0b1 then bitcount = bitcount + 1 else ();

X[rd] = bitcount
```

此操作被称为人口计数、popcount、横向求和、位求和或汉明权重。

GCC 内建函数 `+builtinpopcount (unsigned int x)+` 在 RV32 上由 cpop 实现，在 RV64 上由 `*cpopw*` 实现。

GCC 内建函数 `+builtinpopcountl (unsigned long x)+` 在 LP64 上由 `*cpop*` 在 RV64 上实现。

包含于

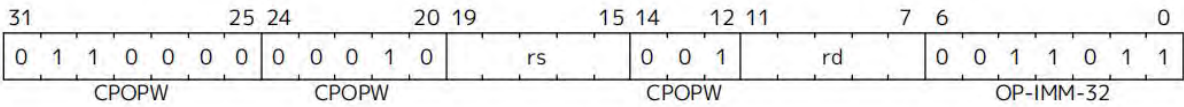
扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.17 cpopw

简述：
计数字中的设置位

助记符：
cpopw rd, rs

编码:



描述:

此指令计算源寄存器中最低有效字中的 1 的数量（即设置位）。

操作:

```
let bitcount = 0;
let val = X(rs);

foreach (i from 0 to 31 in inc)
    if val[i] == 0b1 then bitcount = bitcount + 1 else ();

X[rd] = bitcount
```

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.18 ctz

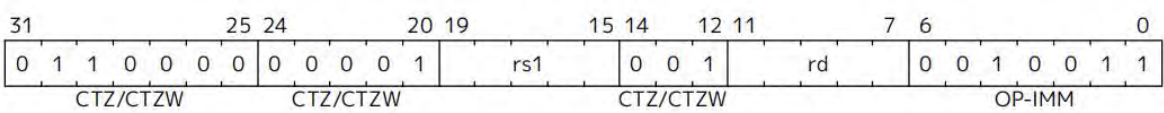
简述:

计数后导零位

助记符:

ctz rd, rs

编码:



描述:

此指令从最低有效位（即位 0）开始，逐位计算第一个 1 之前的 0 的数量，直到最高有效位（即位 XLEN-1）。因此，如果输入为 0，输出为 XLEN；如果输入的最低有效位为 1，输出为 0。

操作:

```

val LowestSetBit : forall ('N : Int), 'N >= 0. bits('N) -> int

function LowestSetBit x = {
  foreach (i from 0 to (xlen - 1) by 1 in dec)
    if [x[i]] == 0b1 then return(i) else ();
  return xlen;
}

let rs = X(rs);
X[rd] = LowestSetBit(rs);

```

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.19 ctzw

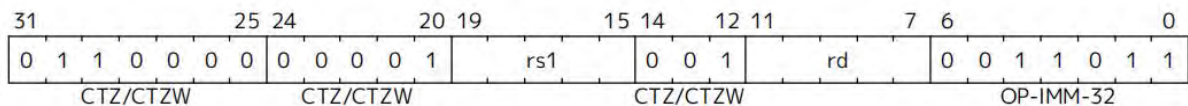
简述:

计数字中的后导零位

助记符:

ctzw rd, rs

编码:



描述:

此指令从最低有效位（即位 0）开始，逐位计算第一个 1 之前的 0 的数量，直到最低有效字的最高有效位（即位 31）。因此，如果最低有效字为 0，输出为 32；如果输入的最低有效位为 1，输出为 0。

操作:

```

val LowestSetBit32 : forall ('N : Int), 'N >= 0. bits('N) -> int

function LowestSetBit32 x = {
  foreach (i from 0 to 31 by 1 in dec)
    if [x[i]] == 0b1 then return(i) else ();
  return 32;
}

let rs = X(rs);
X[rd] = LowestSetBit32(rs);

```

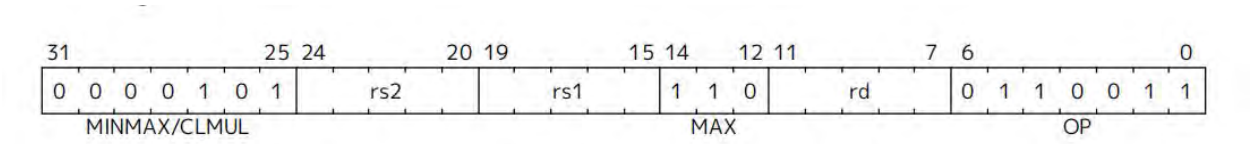
包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.20 max

助记符：
max rd, rs1, rs2

编码：



描述：
此指令返回两个有符号整数中较大的一个。

```

操作：
let rs1_val = X(rs1);
let rs2_val = X(rs2);

let result = if rs1_val <_s rs2_val
               then rs2_val
               else rs1_val;

X(rd) = result;

```



计算有符号整数的绝对值可以通过以下序列进行：`*neg rd, rS*`，然后是 `*max rd, rS, rD*`。在使用此常见序列时，建议将它们安排在没有其它指令插入的情况下，以便经过优化的实现可以将它们合并在一起。

包含于

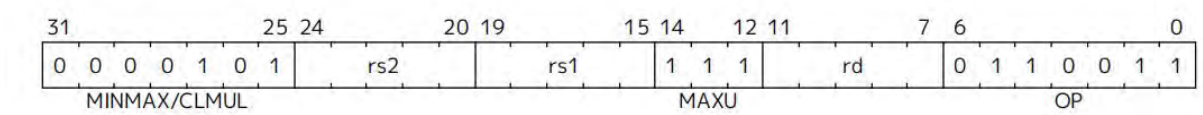
扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.21 maxu

简述：
无符号最大值

助记符：
maxu rd, rs1, rs2

编码：



描述:

此指令返回两个无符号整数中较大的一个。

操作:

```
let rs1_val = X(rs1);
let rs2_val = X(rs2);

let result = if rs1_val <_u rs2_val
               then rs2_val
               else rs1_val;

X(rd) = result;
```

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.22 min

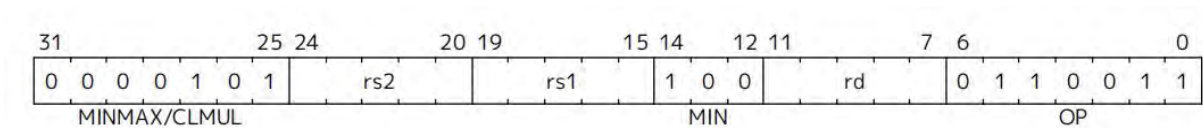
简述:

最小值

助记符:

min rd, rs1, rs2

编码:



描述:

此指令返回两个有符号整数中较小的一个。

操作:

```

let rs1_val = X(rs1);
let rs2_val = X(rs2);

let result = if  rs1_val <_s rs2_val
                then rs1_val
                else rs2_val;

X(rd) = result;

```

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.23 minu

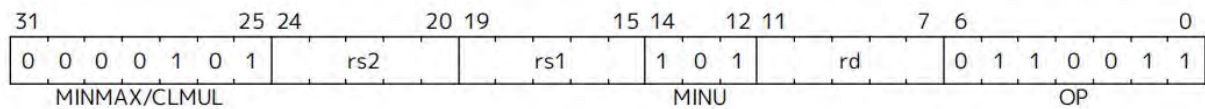
简述:

无符号最小值

助记符:

minu rd, rs1, rs2

编码:



描述:

此指令返回两个无符号整数中较小的一个。

操作:

```

let rs1_val = X(rs1);
let rs2_val = X(rs2);

let result = if  rs1_val <_u rs2_val
                then rs1_val
                else rs2_val;

X(rd) = result;

```

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.24 orc.b

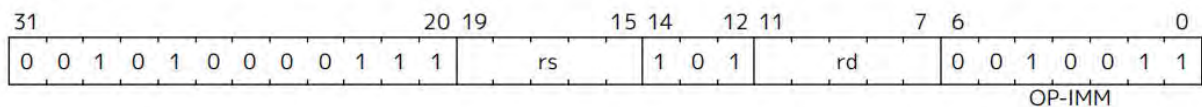
简述:

按字节进行按位或组合

助记符:

orc.b rd, rs

编码:



描述:

通过按位逻辑或将每个字节内的位进行合并。如果 rs 的对应字节内没有任何位被设置，则将结果寄存器 rd 中该字节的所有位都设置为零；如果 rs 中对应字节内有任何位被设置，则结果寄存器 rd 中该字节的所有位都为 1。

操作:

```
let input = X(rs);
let output : xlenbits = 0;

foreach (i from 0 to (xlen - 8) by 8) {
    output[(i + 7)..i] = if input[(i + 7)..i] == 0
                        then 0b00000000
                        else 0b11111111;
}

X[rd] = output;
```

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.25 orn

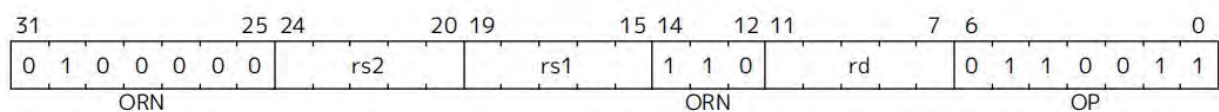
简述:

对反转操作数进行按位或

助记符:

orn rd, rs1, rs2

编码:



描述:

此指令对 rs1 和 rs2 的按位反结果执行按位或操作。

操作:

$X(rd) = X(rs1) \mid \sim X(rs2);$

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结
Zbkb	v0.9.4	冻结

28.5.26 pack

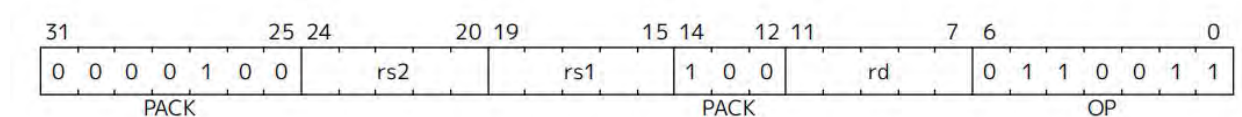
简述:

将 rs1 和 rs2 的低半部分打包到 rd 中。

助记符:

pack rd, rs1, rs2

编码:



描述:

pack 指令将 rs1 和 rs2 的 $XLEN/2$ 位低半部分打包到 rd 中, rs1 位于低半部分, rs2 位于高半部分。

操作:

```
let lo_half : bits(xlen/2) = X(rs1)[xlen/2-1..0];
let hi_half : bits(xlen/2) = X(rs2)[xlen/2-1..0];
X(rd) = EXTZ(hi_half @ lo_half);
```

包含于

扩展	最低版本	生命周期阶段
Zbkb	v0.9.4	冻结

28.5.27 packh

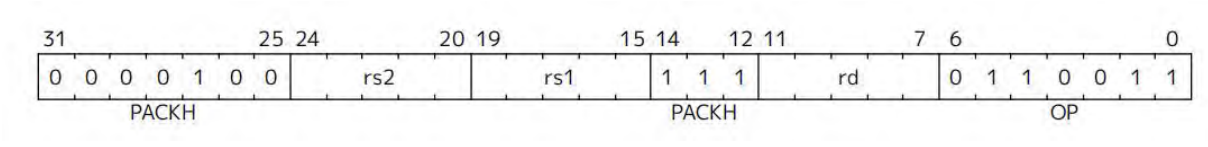
简述:

将 rs1 和 rs2 的低字节打包到 rd 中。

助记符:

packh rd, rs1, rs2

编码:



描述:

packh 指令将 rs1 和 rs2 的最低有效字节打包到 rd 的 16 个最低有效位中, 零扩展 rd 的其余部分。

操作:

```
let lo_half : bits(8) = X(rs1)[7..0];
let hi_half : bits(8) = X(rs2)[7..0];
X(rd) = EXTZ(hi_half @ lo_half);
```

包含于

扩展	最低版本	生命周期阶段
Zbkb	v0.9.4	冻结

28.5.28 packw

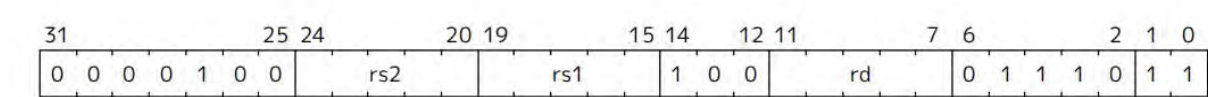
简述:

在 RV64 上将 rs1 和 rs2 的低 16 位打包到 rd 中。

助记符:

```
packw rd, rs1, rs2
```

编码:



描述:

此指令将 rs1 和 rs2 的低 16 位打包到 rd 的 32 个最低有效位中, 将 32 位结果符号位扩展到 rd 的其余部分。此指令仅存在于基于 RV64 的系统上。

操作:

```
let lo_half : bits(16) = X(rs1)[15..0];
let hi_half : bits(16) = X(rs2)[15..0];
X(rd) = EXT(S(hi_half @ lo_half));
```

包含于

扩展	最低版本	生命周期阶段
Zbkb	v0.9.4	冻结

28.5.29 rev8

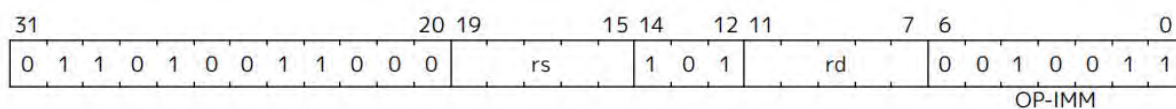
简述:

字节反转寄存器

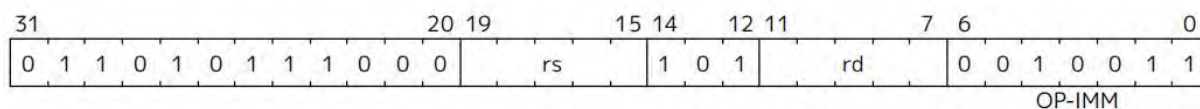
助记符:

rev8 rd, rs

编码 (RV32):



编码 (RV64):



描述:

此指令反转 rs 中的字节顺序。

操作:

```
let input = X(rs);
let output : xlenbits = 0;
let j = xlen - 1;

foreach (i from 0 to (xlen - 8) by 8) {
    output[i..(i + 7)] = input[(j - 7)..j];
    j = j - 8;
}
```

X[rd] = output



rev8 助记符在 RV32 和 RV64 中对应不同的指令编码。



字节反转操作仅适用于整个寄存器宽度。为了模拟单字大小和半字大小的字节反转，可以执行 rev8 rd,rs ，然后执行 srai rd,rd,K ，其中 K 分别为 XLEN-32 和 XLEN-16。

包含于

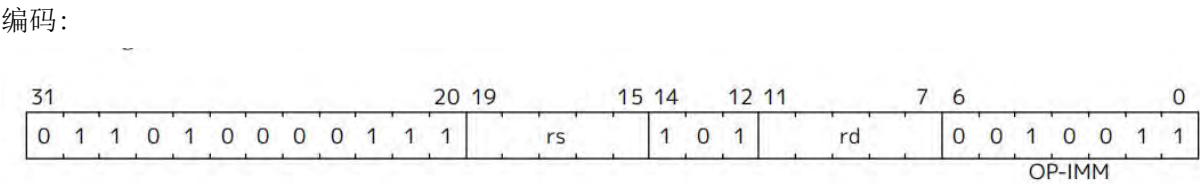
扩展	最低版本	生命周期阶段
----	------	--------

Zbb	0. 93	冻结
Zbkb	v0. 9. 4	冻结

28. 5. 30 rev. b

简述：
反转源寄存器中每个字节的比特位。

助记符：
rev. b rd, rs



描述：
此指令反转寄存器中每个字节的位顺序。

操作：

```
result : xlenbits = EXTZ(0b0);
foreach (i from 0 to sizeof(xlen) by 8) {
    result[i+7..i] = reverse_bits_in_byte(X(rs1)[i+7..i]);
};
X(rd) = result;
```

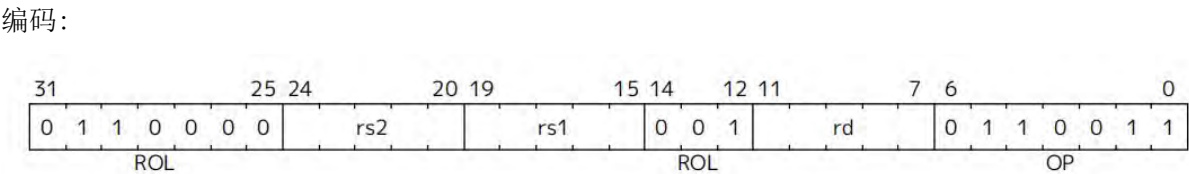
包含于

扩展	最低版本	生命周期阶段
Zbkb	v0. 9. 4	冻结

28. 5. 31 rol

简述：
左旋转（寄存器）

助记符：
rol rd, rs1, rs2



描述:

此指令执行 rs1 的左旋转, 旋转的位数由 rs2 中最低有效的 log2(XLEN) 位决定。

操作:

```
let shamt = if xlen == 32
    then X(rs2)[4..0]
    else X(rs2)[5..0];
let result = (X(rs1) << shamt) | (X(rs1) >> (xlen - shamt));

X(rd) = result;
```

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结
Zbkb	v0.9.4	冻结

28.5.32 rolw

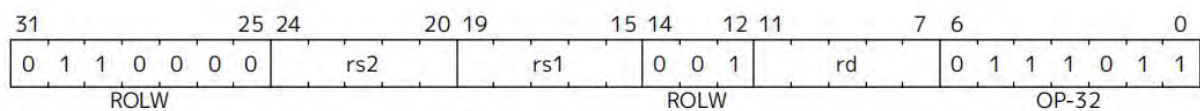
简述:

左旋转字 (寄存器)

助记符:

rolw rd, rs1, rs2

编码:



描述:

此指令对 rs1 的最低有效字进行左旋转, 选择的位数由 rs2 中最低有效的 5 位决定。结果字值通过将第 31 位复制到所有更高有效位来进行符号扩展。

操作:

```
let rs1 = EXTZ(X(rs1)[31..0])
let shamt = X(rs2)[4..0];
let result = (rs1 << shamt) | (rs1 >> (32 - shamt));
X(rd) = EXTZ(result[31..0]);
```

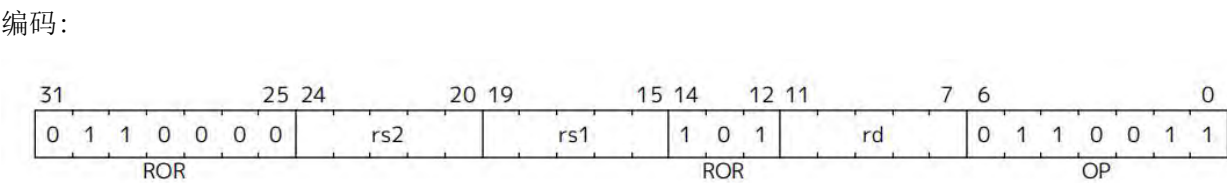
包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结
Zbkb	v0.9.4	冻结

28.5.33 ror

简述：
右旋转

助记符：
ror rd, rs1, rs2



描述：
此指令对 rs1 进行右旋转，旋转的位数由 rs2 的最低有效的 log2(XLEN) 位决定。

操作：

```
let shamt = if xlen == 32
    then X(rs2)[4..0]
    else X(rs2)[5..0];
let result = (X(rs1) >> shamt) | (X(rs1) << (xlen - shamt));
X(rd) = result;
```

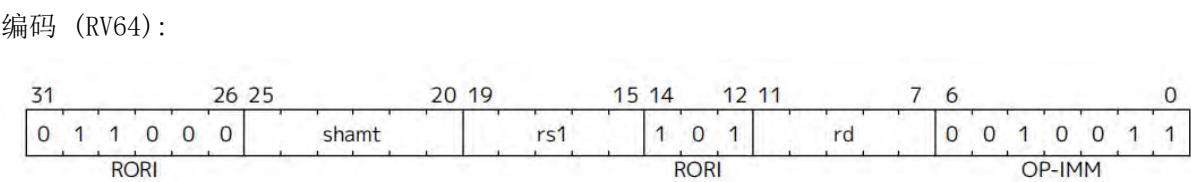
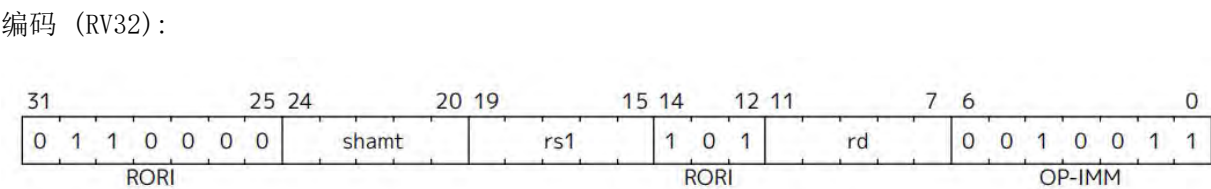
包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结
Zbkb	v0.9.4	冻结

28.5.34 rori

简述：
右旋转（立即数）

助记符：
rori rd, rs1, shamt



描述:

此指令对 rs1 进行右旋转,旋转的位数由 shamt 的最低有效的 log2(XLEN) 位决定。对于 RV32 , shamt[5]=1 对应的编码是保留的。

操作:

```
let shamt = if xlen == 32
              then shamt[4..0]
              else shamt[5..0];
let result = (X(rs1) >> shamt) | (X(rs1) << (xlen - shamt));

X(rd) = result;
```

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结
Zbkb	v0.9.4	冻结

28.5.35 roriw

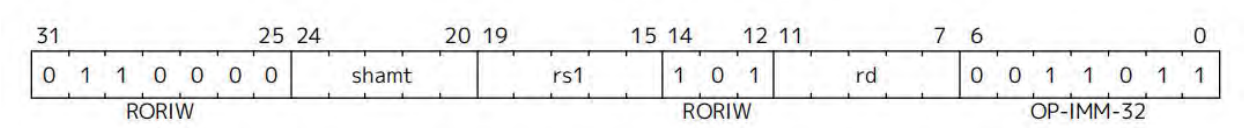
简述:

右旋转字（立即数）

助记符:

roriw rd, rs1, shamt

编码:



描述:

此指令对 rs1 的最低有效字进行右旋转，旋转的位数由 shamt 中最低有效的 log2(XLEN) 位决定。结果字值通过将第 31 位复制到所有更高有效位来进行符号扩展。

操作:

```
let rs1_data = EXTZ(X(rs1)[31..0]);
let result = (rs1_data >> shamt) | (rs1_data << (32 - shamt));
X(rd) = EXTZ(result[31..0]);
```

包含于

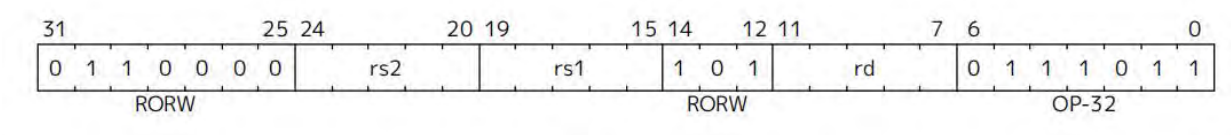
扩展	最低版本	生命周期阶段
Zbb	0.93	冻结
Zbkb	v0.9.4	冻结

28.5.36 rorw

简述：
右旋转字（寄存器）

助记符：
rorw rd, rs1, rs2

编码：



描述：
此指令对 rs1 的最低有效字执行右旋操作，旋转的位数由 rs2 的最低有效 5 位决定。
结果字的值会通过将位 31 复制到所有更高位来进行符号扩展。

操作：

```
let rs1 = EXTZ(X(rs1)[31..0])
let shamt = X(rs2)[4..0];
let result = (rs1 >> shamt) | (rs1 << (32 - shamt));
X(rd) = EXT5(result);
```

包含于

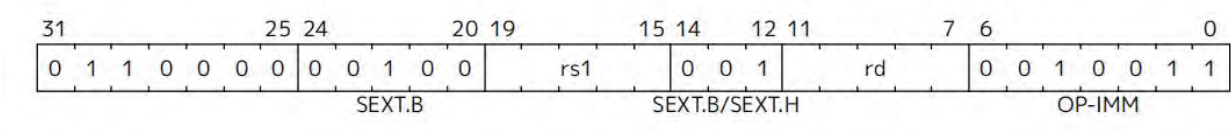
扩展	最低版本	生命周期阶段
Zbb	0.93	冻结
Zbkb	v0.9.4	冻结

28.5.37 sext.b

简述：
符号位扩展字节

助记符：
sext.b rd, rs

编码：



描述：
此指令通过将字节中的最高有效位（即第 7 位）复制到所有更高有效位来将源操作数中的最低有效字节符号位扩展到 XLEN 位。

操作:

```
X(rd) = EXT5(X(rs)[7..0]);
```

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.38 sext.h

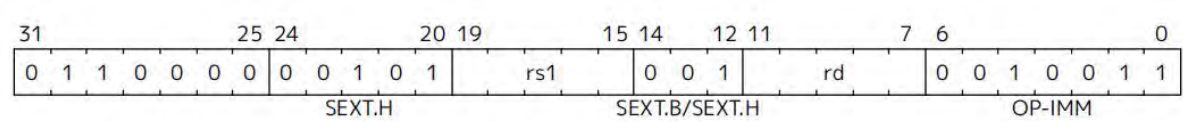
简述:

符号扩展半字

助记符:

sext.h rd, rs

编码:



描述:

此指令通过将半字中的最高有效位（即第 15 位）复制到所有更高有效位，来将 rs 中的最低有效半字符号位扩展到 XLEN 位。

操作:

```
X(rd) = EXT5(X(rs)[15..0]);
```

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.39 sh1add

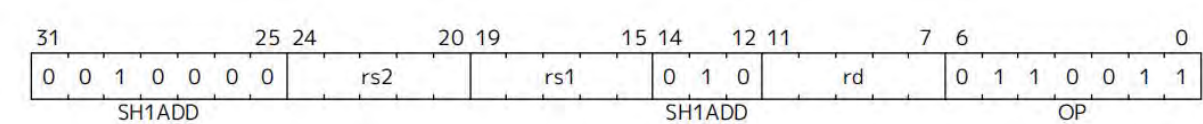
简述:

左移 1 位并且相加

助记符:

sh1add rd, rs1, rs2

编码:



描述：
此指令将 rs1 左移 1 位，并将其加到 rs2 中。

操作：
$$X(rd) = X(rs2) + (X(rs1) \ll 1);$$

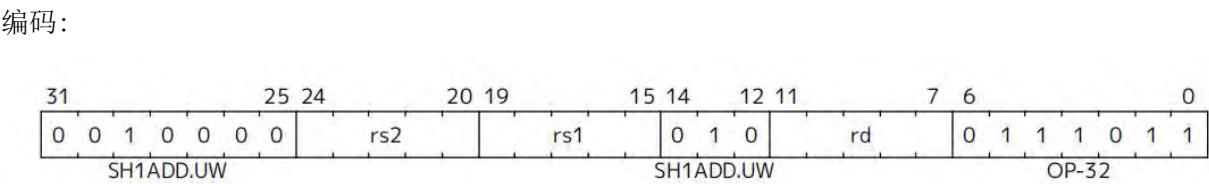
包含于

扩展	最低版本	生命周期阶段
Zba	0.93	冻结

28.5.40 sh1add.uw

简述：
与左移 1 位的无符号字相加

助记符：
shladd.uw rd, rs1, rs2



描述：
此指令执行两个加数的 XLEN 位宽加法。第一个加数是 rs2。第二个加数是通过提取 rs1 中的最低有效字并将其左移 1 位形成的无符号值。

操作：
$$\begin{aligned} \text{let } \text{base} &= X(rs2); \\ \text{let } \text{index} &= \text{EXTZ}(X(rs1)[31..0]); \\ X(rd) &= \text{base} + (\text{index} \ll 1); \end{aligned}$$

包含于

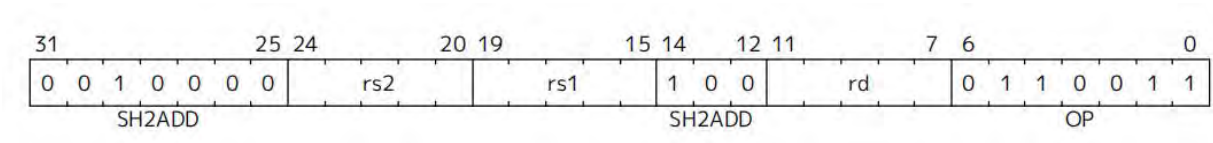
扩展	最低版本	生命周期阶段
Zba	0.93	冻结

28.5.41 sh2add

简述：
左移 2 位并且相加

助记符：
sh2add rd, rs1, rs2

编码:



描述:

此指令将 rs1 左移 2 位，并将其与 rs2 相加。

操作:

$$X(rd) = X(rs2) + (X(rs1) \ll 2);$$

包含于

扩展	最低版本	生命周期阶段
Zba	0.93	冻结

28.5.42 sh2add.uw

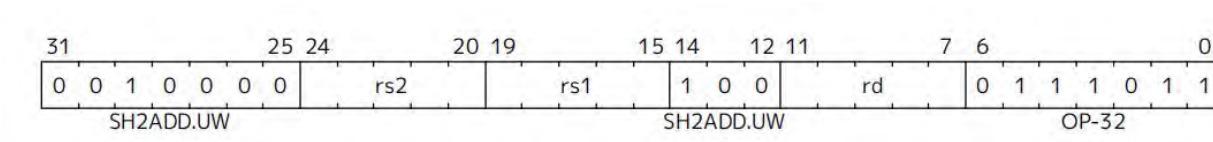
简述:

与左移 2 位的无符号字相加

助记符:

sh2add.uw rd, rs1, rs2

编码:



描述:

此指令执行一个 XLEN 位宽的加法运算。第一个加数是 rs2。第二个加数是通过提取 rs1 的最低有效字并将其左移 2 位得到的无符号值。

操作:

```
let base = X(rs2);
let index = EXTZ(X(rs1)[31..0]);

X(rd) = base + (index << 2);
```

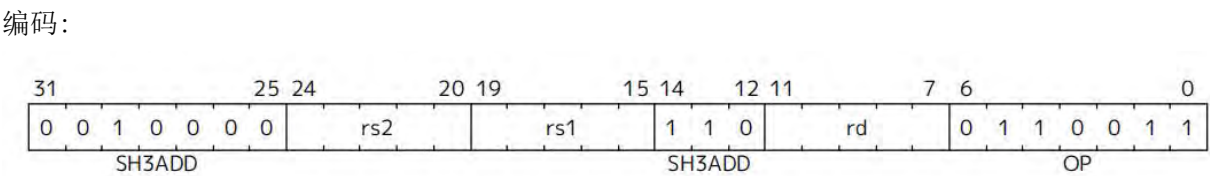
包含于

扩展	最低版本	生命周期阶段
Zba	0.93	冻结

28. 5. 43 sh3add

简述：
左移 3 位并且相加

助记符：
sh3add rd, rs1, rs2



描述：
此指令将 rs1 左移 3 位，并将其加到 rs2。

操作：
 $X(rd) = X(rs2) + (X(rs1) \ll 3);$

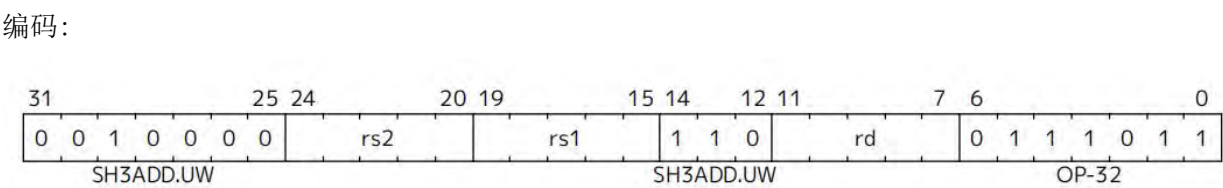
包含于

扩展	最低版本	生命周期阶段
Zba	0.93	冻结

28. 5. 44 sh3add.uw

简述：
与左移 3 位的无符号字相加

助记符：
sh3add.uw rd, rs1, rs2



描述：
此指令执行两个加数的 XLEN 位宽加法。第一个加数是 rs2。第二个加数是通过提取 rs1 的最低有效字并将其左移 3 位得到的无符号值。

操作：

```
let base = X(rs2);
let index = EXTZ(X(rs1)[31..0]);
```

```
X(rd) = base + (index << 3);
```

包含于

扩展	最低版本	生命周期阶段
Zba	0.93	冻结

28.5.45 slli.uw

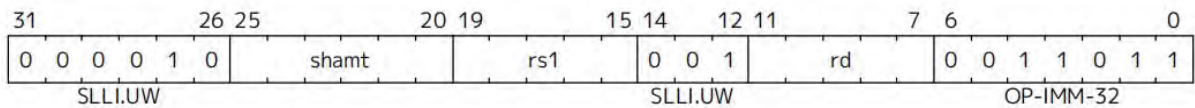
简述:

左移无符号字（立即数）

助记符:

slli.uw rd, rs1, shamt

编码:



描述:

此指令将 rs1 的最低有效字提取出来，进行零扩展，并按立即数左移。

操作:

```
X(rd) = (EXTZ(X(rs)[31..0]) << shamt);
```

包含于

扩展	最低版本	生命周期阶段
Zba	0.93	冻结

28.5.46 unzip

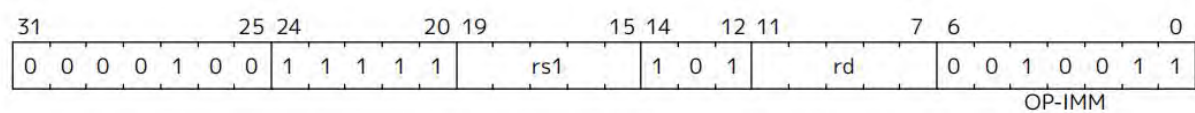
简述:

实现 zip 指令的逆操作。

助记符:

unzip rd, rs

编码:



描述：
此指令将源操作字的高半部分和低半部分中的位收集到目标字的奇偶位位置中。它是 zip 指令的逆操作。此指令仅在 RV32 上可用。

操作：

```
foreach (i from 0 to xlen/2-1) {
    X(rd)[i] = X(rs1)[2*i]
    X(rd)[i+xlen/2] = X(rs1)[2*i+1]
}
```

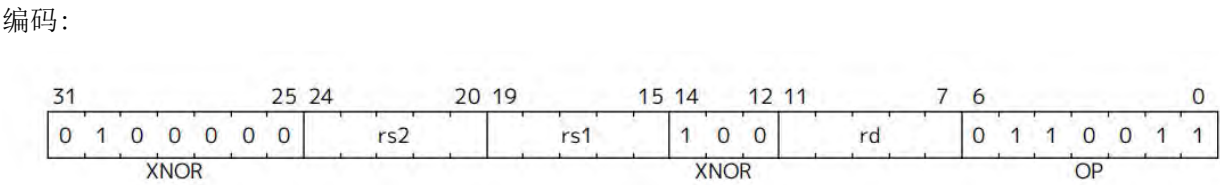
包含于

扩展	最低版本	生命周期阶段
Zbkb	v0.9.4	冻结

28.5.47 xnor

简述：
异或非

助记符：
xnor rd, rs1, rs2



描述：
此指令对 rs1 和 rs2 执行按位异或非操作。

操作：

```
X(rd) = ~(X(rs1) ^ X(rs2));
```

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结
Zbkb	v0.9.4	冻结

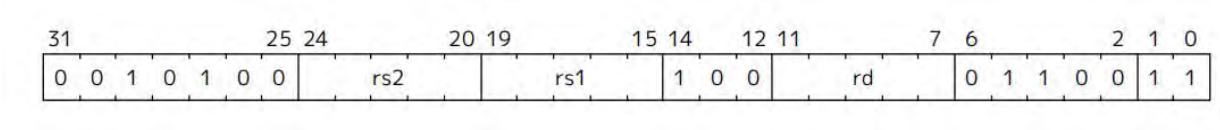
28.5.48 xperm.b

简述：
按字节方式查找寄存器中向量的索引。

助记符：

xperm.b rd, rs1, rs2

编码:



描述:

xperm.b 指令作用于字节。rs1 寄存器包含一个 XLEN/8 大小的 8 位元素向量。rs2 寄存器包含一个 XLEN/8 大小的 8 位索引向量。结果是，rs2 中的每个元素将被替换为 rs1 中对应索引的元素，如果 rs2 中的索引越界，则该元素为零。

操作:

```
val xpermb_lookup : (bits(8), xlenbits) -> bits(8)
function xpermb_lookup (idx, lut) = {
    (lut >> (idx @ 0b000))[7..0]
}

function clause execute ( XPERM_B (rs2,rs1,rd)) = {
    result : xlenbits = EXTZ(0b0);
    foreach(i from 0 to xlen by 8) {
        result[i+7..i] = xpermb_lookup(X(rs2)[i+7..i], X(rs1));
    };
    X(rd) = result;
    RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zbkx	v0.9.4	冻结

28.5.49 xperm.n

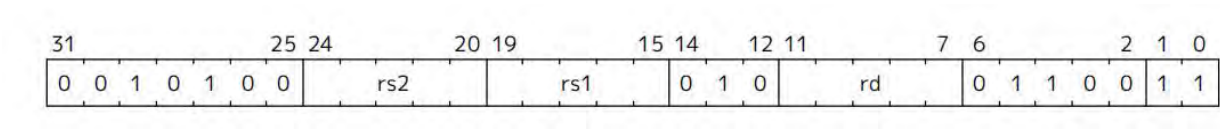
简述:

在向量中按半字节查找索引。

助记符:

xperm.n rd, rs1, rs2

编码:



描述:

xperm.n 指令作用于半字节。rs1 寄存器包含一个 XLEN/4 大小的 4 位元素向量。rs2 寄存器包

含一个 $XLEN/4$ 大小的 4 位索引向量。结果是， $rs2$ 中的每个元素将被替换为 $rs1$ 中对应索引的元素，如果 $rs2$ 中的索引越界，则该元素为零。

操作:

```
val xpermn_lookup : (bits(4), xlenbits) -> bits(4)
function xpermn_lookup (idx, lut) = {
    (lut >> (idx @ 0b00))[3..0]
}

function clause execute ( XPERM_N (rs2,rs1,rd)) = {
    result : xlenbits = EXTZ(0b0);
    foreach(i from 0 to xlen by 4) {
        result[i+3..i] = xpermn_lookup(X(rs2)[i+3..i], X(rs1));
    };
    X(rd) = result;
    RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zbkx	v0.9.4	冻结

28.5.50 zext.h

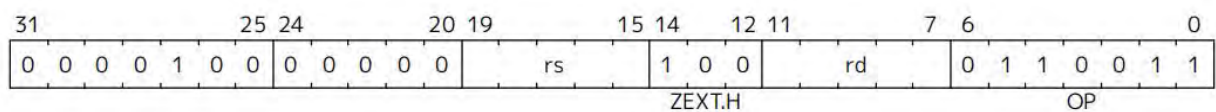
简述:

零扩展半字

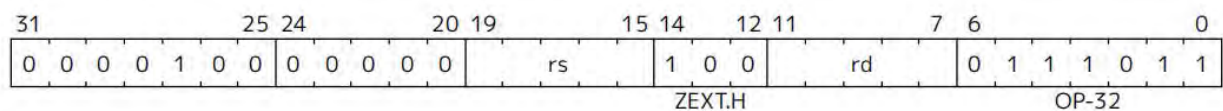
助记符:

zext.h rd, rs

编码 (RV32):



编码 (RV64):



描述:

此指令通过在高于第 15 位的所有位插入 0，将源操作数的最低有效半字零扩展到 $XLEN$ 。

操作:

```
X(rd) = EXTZ(X(rs)[15..0]);
```

包含于

扩展	最低版本	生命周期阶段
Zbb	0.93	冻结

28.5.51 zip

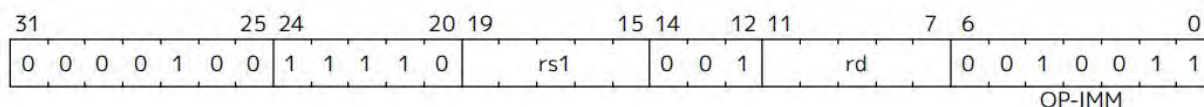
简述:

将源操作字的奇偶位收集到目标的上半部分和下半部分。

助记符:

zip rd, rs

编码:



描述:

此指令将源操作字的所有奇偶位分散到目标操作数的高半部分和低半部分。它是 unzip 指令的逆操作。此指令仅在 RV32 上可用。

操作:

```
foreach (i from 0 to xlen/2-1) {  
    X(rd)[2*i] = X(rs1)[i]  
    X(rd)[2*i+1] = X(rs1)[i+xlen/2]  
}
```



此指令对于在32位架构上实现SHA3加密哈希函数非常有用，因为它实现了用于直接加速64位旋转的位交织操作。

包含于

扩展	最低版本	生命周期阶段
Zbkb (RV32)	v0.9.4	冻结

28.6 软件优化指南

28.6.1 strlen

orc.b 指令允许高效检测 XLEN 大小数据块中的 *NUL* 字节:

- * 对于不包含任何 *NUL* 字节的数据块, *orc.b* 的结果将是全为 1,
- * 在对 *orc.b* 的结果进行按位取反后, 可以通过 *ctz*/*clz* (取决于数据的字节序) 检测出第一个 *NUL* 字节之前的数据字节数 (如果有的话)。

下面一个完整的 *strlen* 函数示例, 它使用了这些技术, 并演示了它在未对齐/部分数据情况下的使用:

[source, asm]

```
#include <sys/asm.h>

.text
.globl strlen
.type  strlen, @function
strlen:
    andi    a3, a0, (SZREG-1)    // 偏移量
    andi    a1, a0, -SZREG       // 对齐指针
.Lprologue:
    li      a4, SZREG
    sub     a4, a4, a3           // XLEN - 偏移量
    slli    a3, a3, 3           // 偏移量 * 8
    REGL    a2, 0(a1)           // 数据块
    /*
     * 将我们加载的部分或未对齐的数据块进行移位，以去除字符串开始之前的字节，并在末尾添加
    NUL 字节。
     */
    #if BYTEORDER == ORDERLITTLEENDIAN
        srl a2, a2, a3           // 数据块 (偏移量 * 8)
    #else
        sll  a2, a2, a3
    #endif
    orc.b   a2, a2
    not a2, a2
    /*
     * 字符串中的非 NUL 字节已扩展为 0x00，而 NUL 字节已变为 0xff。搜索第一个设置的位
     * （对应于初始数据块中的 NUL 字节）。
     */
    #if BYTEORDER == ORDERLITTLEENDIAN
        ctz    a2, a2
    #else
        clz    a2, a2
    #endif
    /*
     * 第一个数据块是特殊的：与此数据块中的有效字节数进行比较。
     */
    srli     a0, a2, 3
    bgtu     a4, a0, .Ldone
    addi     a3, a1, SZREG
    li       a4, -1
    .align 2
    /*
```

```

    * 我们的关键循环是 4 条指令，并以 4 字节或 8 字节的数据块处理数据。
    */
.Lloop:
    REGL    a2, SZREG(a1)
    addi    a1, a1, SZREG
    orc.b   a2, a2
    beq     a2, a4, .Lloop

.Lepilogue:
    not     a2, a2
#if BYTEORDER == ORDERLITTLEENDIAN
    ctz     a2, a2
#else
    clz     a2, a2
#endif
    sub     a1, a1, a3
    add a0, a0, a1
    srli    a2, a2, 3
    add     a0, a0, a2
.Ldone:
    ret

```

28.6.2 strcmp

[source, asm]

```

#include <sys/asm.h>

.text
.globl strcmp
.type  strcmp, @function
strcmp:
    or    a4, a0, a1
    li    t2, -1
    and   a4, a4, SZREG-1
    bnez  a4, .Lsimpleloop

    # 对齐字符串的主循环
.Lloop:
    REGL a2, 0(a0)
    REGL a3, 0(a1)
    orc.b t0, a2
    bne   t0, t2, .Lfoundnull
    addi  a0, a0, SZREG

```

```

addi a1, a1, SZREG
beq a2, a3, .Lloop

# 字不匹配，且第一个字中没有空字节。
# 获取大端字节顺序的字节并进行比较。
#if BYTEORDER == ORDERLITTLEENDIAN
    rev8 a2, a2
    rev8 a3, a3
#endif
# 在无分支序列中合成 (a2 >= a3) ? 1 : -1。
sltu a0, a2, a3
neg a0, a0
ori a0, a0, 1
ret

.Lfoundnull:
# 找到一个空字节。
# 如果字不匹配，则回退到简单循环。
bne a2, a3, .Lsimpleloop

```

29 J 扩展：用于动态翻译语言，V0.0

本章节为未来标准扩展占位，旨在支持动态翻译语言。



许多流行的编程语言通常通过动态翻译实现，包括 Java 和 JavaScript。这些语言可以从额外的 ISA 支持中受益，特别是在动态检查和垃圾回收方面。

30 P 扩展：打包式 SIMD 指令，V0.2



在第五届 RISC-V 研讨会上的讨论表明，与会者倾向于放弃这个针对浮点寄存器的打包式 SIMD 提案，转而支持将 V 扩展标准化，用于大规模浮点 SIMD 运算。不过，与会者对在小型 RISC-V 实现中使用整数寄存器进行打包式 SIMD 定点运算表示了兴趣。目前有一个工作组正在着手定义这个新的 P 扩展。

31 V 扩展：向量操作，V1.0



基础向量扩展旨在提供在 32 位指令编码空间内对数据并行执行的通用支持，后续的向量扩展将为特定领域提供更丰富的功能。

31.1 介绍

本文档是 RISC-V 向量扩展的版本 1.1 草案。



此版本包含自公开评审开始后收集的更新内容。规范将在批准时最终更新为版本 2.0。

本规范包括当前已冻结的完整向量指令集。其他在开发过程中已考虑但未包含于本文档中的指令不在评审和批准范围内，可能会被完全修订或放弃。31.18 节列出了标准向量扩展，以及每个扩展支持的指令和元素宽度。

31.2 实现定义的常量参数

每个支持向量扩展的硬件线程定义两个参数：

- 任意操作可以产生或使用的向量元素的最大位数， $ELEN \geq 8$ ，且必须是 2 的幂。
- 单个向量寄存器的位数， $VLEN \geq ELEN$ ，且必须是 2 的幂，且不大于 2^{16} 。

标准向量扩展（第 31.18 节）和架构配置文件可能会对 $ELEN$ 和 $VLEN$ 设置进一步的约束。

未来的扩展可能允许 $ELEN > VLEN$ ，通过使用多个向量寄存器中的位来保存一个元素，但当前提案不包括这一选项。



$VLEN$ 的上限使得软件可以知道索引将适用于 16 位（最大的 $VLMAX$ 为 65,536，发生在 $LMUL=8$ 和 $SEW=8$ 时，且 $VLEN=65,536$ ）。任何超过 64Kib 的向量寄存器扩展将需要新的配置指令，以确保使用旧配置指令的软件不会观察到更大的向量长度。

向量扩展支持在特定约束下编写二进制代码，该代码可以在具有不同 $VLEN$ 参数值的硬件线程上可移植执行，前提是这些硬件线程支持所需的元素类型和指令。

可以编写代码来暴露实现参数的差异。



通常情况下，具有激活向量状态的线程上下文在执行过程中不能在具有不同 $VLEN$ 或 $ELEN$ 参数的硬件线程之间迁移。

31.3 向量扩展编程模型

向量扩展向基础的标量 RISC-V ISA 添加了 32 个向量寄存器，以及七个非特权控制和状态寄存器（CSRs）：`vstart`、`vxsat`、`vxrm`、`vcsr`、`vtype`、`vl` 和 `vlenb`。

表31.1新载体csr

地址	特权	名称	描述
0x008	URW	vstart	矢量起始位置
0x009	URW	vxsat	定点饱和标志
0x00A	URW	vxrm	定点舍入模式
0x00F	URW	vcsr	矢量控制和状态寄存器
0xC20	URO	vl	向量的长度
0xC21	URO	vtype	矢量数据类型寄存器
0xC22	URO	vlenb	以字节为单位的向量寄存器长度



四个控制和状态寄存器（CSR）编号 0x00B–0x00E 暂时保留给未来的向量，其中一些可能会镜像到 vcsr。

31.3.1 向量寄存器

向量扩展向基础的标量 RISC-V ISA 添加了 32 个架构向量寄存器（v0–v31）。每个向量寄存器具有固定的 VLEN 位状态。

31.3.2 向量上下文状态在 mstatus 中

向 mstatus[10:9] 中添加了一个向量上下文状态字段 VS，并在 sstatus[10:9] 中进行了影射。它的定义方式类似于浮点上下文状态字段 FS。当 mstatus.VS 设置为 Off 时，尝试执行任何向量指令或访问向量控制状态寄存器时会引发非法指令异常。当 mstatus.VS 设置为初始（initial）或清洁（clean）时，执行任何改变向量状态的指令（包括向量控制状态寄存器）将使 mstatus.VS 更改为脏（dirty）。实现也可以在任何时候将 mstatus.VS 从初始或清洁更改为脏，即使向量状态没有发生变化。



准确设置 mstatus.VS 是一种优化。软件通常会利用 VS 来减少上下文交换的开销。

如果 mstatus.VS 为脏，则 mstatus.SD 设置为 1；否则，mstatus.SD 将根据现有规范进行设置。实现可能具有可写的 misa.V 字段。类似于浮点单元的处理方式，mstatus.VS 字段可能存在，即使 misa.V 被清除。



允许在 `misa.V` 清除时存在 `mstatus.VS`，这样可以支持向量仿真，并简化在具有可写 `misa.V` 的系统中对 `mstatus.VS` 的处理。

31.3.3 向量上下文状态在 `vsstatus` 中

当存在虚拟化扩展时，`vsstatus[10:9]` 中会添加一个向量上下文状态字段 `VS`。它的定义方式类似于浮点上下文状态字段 `FS`。

当 `V=1` 时，`vsstatus.VS` 和 `mstatus.VS` 都生效：如果任一字段设置为关闭，尝试执行任何向量指令或访问向量都将引发非法指令异常。

当 `V=1` 并且 `vsstatus.VS` 和 `mstatus.VS` 都没有设置为关闭时，执行任何改变向量状态的指令（包括向量）将使 `mstatus.VS` 和 `vsstatus.VS` 都变为脏。实现也可以在任何时候将 `mstatus.VS` 或 `vsstatus.VS` 从初始或清洁状态变为脏，即使没有发生向量状态的变化。

如果 `vsstatus.VS` 为脏，则 `vsstatus.SD` 为 1；否则，`vsstatus.SD` 将根据现有规范进行设置。如果 `mstatus.VS` 为脏，则 `mstatus.SD` 为 1；否则，`mstatus.SD` 将根据现有规范进行设置。对于具有可写 `misa.V` 字段的实现，即使 `misa.V` 被清除，`vsstatus.VS` 字段仍然可以存在。

31.3.4 向量类型寄存器，`vtype`

只读的 `XLEN` 宽的向量类型控制状态寄存器，`vtype` 提供了用于解释向量寄存器文件内容的默认类型，并且只能通过 `vset{i}vl{i}` 指令更新。向量类型决定了每个向量寄存器中元素的组织方式，以及如何将多个向量寄存器分组。`vtype` 寄存器还指示如何处理被掩蔽掉的元素以及超出当前向量长度的元素。



仅允许通过 `vset{i}vl{i}` 指令更新 `vtype` 寄存器，有助于简化 `vtype` 寄存器状态的维护。

`vtype` 寄存器包含五个字段：`vill`、`vma`、`vta`、`vsew[2:0]` 和 `vlmul[2:0]`。位 `vtype[XLEN-2:8]` 应该被写入零，且该字段中的非零值保留供未来使用。

支持 `ELEN=32` 的小型实现只需要在 `vtype` 中存储七个位：`ma` 和 `ta` 各两个位，`vsew[1:0]` 两个位，以及 `vlmul[2:0]` 三个位。`vill` 表示的非法值可以使用 `vsew[1:0]` 中的非法 64 位组合进行内部编码，而无需额外的存储位来保存 `vill`。



未来的标准和自定义向量扩展可能会扩展这些字段，以支持更多的数据类型。

`vtype` 的主要功能是使向量指令集能够适配 32 位的指令编码空间。在执行向量指令之前，可以使用单独的 `vset{i}vl{i}` 指令来设置 `vl` 和/或 `vtype` 字段，且实现可以选择将这两条指令融合成一个内部的向量微操作。在许多情况下，`vl` 和 `vtype` 的值可以在多个指令之间复用，从而减少来自 `vset{i}vl{i}` 指令的静态和动态指令开销。预计未来扩展的 64 位指令编码将允许在指令编码中静态指定这些字段。

向量选择的元素宽度 `vsew[2:0]`

vsew 中的值设置了动态选定的元素宽度（SEW）。默认情况下，向量寄存器被视为被划分为 VLEN/SEW 个元素。

表31.2 选定元素宽度

vsew[2:0]			SEW
0	0	0	8
0	0	1	16
0	1	0	32
0	1	1	64
1	X	X	Reserved



虽然预计较大的 vsew[2:0] 编码（100-111）将用于编码更大的 SEW，但这些编码目前是保留的。

表31.3 VLEN = 128 bits

SEW	每个向量寄存器的元素数
64	2
32	4
16	8
8	16

支持的元素宽度可能会随着 LMUL 的变化而变化。



当前的标准向量扩展集不随着 LMUL 变化而变化支持的元素宽度。一些未来的扩展可能仅在使用 LMUL 将多个向量寄存器的位组合时，支持更大的 SEW。在这种情况下，依赖于大 SEW 的软件应尝试使用最大的 LMUL，因此使用最少的向量寄存器组，以增加代码能够运行的实现数量。在设置 vtype 后，应检查 vtype 中的 vill 位，以查看配置是否受支持，如果不支持，则应提供备用代码路径。或者，可以在每个 LMUL 设置下强制规定最小的 SEW。

向量寄存器分组

多个向量寄存器可以组合在一起，使得单个向量指令可以操作多个向量寄存器。这里使用向量寄存器组这一术语，指代作为单个操作数传递给向量指令的一个或多个向量寄存器。向量寄存器组可以用于提高较长应用向量的执行效率，但其主要目的是允许双宽或更大元素与单宽元素具有相同的向量长度进行操作。当 LMUL 大于 1 时，它表示将组合多个向量寄存器形成一个向量寄存器组的默认数量。实现必须支持 LMUL 的整数值 1、2、4 和 8。



向量架构包括需要多个源和目标向量操作数的指令，这些操作数具有不同的元素宽度，但元素数量相同。每个向量操作数的有效 LMUL（EMUL）由所需的寄存器数量来确定，以容纳这些元素。例如，对于扩展加法操作（如将 32 位值相加以产生 64 位结果），双宽结果需要是单宽输入 LMUL 的两倍。

LMUL 还可以是一个分数值，从而减少单个向量寄存器中使用的位数。分数 LMUL 用于在处理混合宽度值时增加有效可用的向量寄存器组的数量



仅使用整数 LMUL 值时，操作一个范围内不同大小的循环需要为最窄的数据类型分配至少一个完整的向量寄存器（LMUL=1），然后为每个更宽的向量操作数消耗多个向量寄存器（LMUL>1）以形成一个向量寄存器组。这会限制可用的向量寄存器组数量。使用分数 LMUL 时，最宽的值仅需要占用一个单独的向量寄存器，而较窄的值可以占用一个向量寄存器的一部分，从而允许在处理混合宽度值时，即使在向量循环中，也可以使用所有 32 个架构向量寄存器名称。分数 LMUL 意味着部分向量寄存器未被使用，但在某些情况下，使用更多较短的寄存器驻留向量相对于使用更少较长的寄存器驻留向量可以提高效率。

实现必须提供分数 LMUL 设置，允许最窄支持的类型占用一个向量寄存器的一个分数部分，该分数部分对应于最窄支持的类型的宽度与最宽支持的类型的宽度之间的比例。一般来说，要求支持 $LMUL \geq SEW_{min}/ELEN$ ，其中 SEW_{min} 是最窄支持的 SEW 值，ELEN 是最宽支持的 SEW 值。在标准扩展中， $SEW_{min} = 8$ 。对于 ELEN=32 的标准向量扩展，必须支持分数 LMUL 值 1/2 和 1/4。对于 ELEN=64 的标准向量扩展，必须支持分数 LMUL 值 1/2、1/4 和 1/8。



当 $LMUL < SEW_{min}/ELEN$ 时，不能保证实现会在分数向量寄存器中有足够的位来存储至少一个元素，因为 $VLEN=ELEN$ 是有效的实现选择。例如，对于 $VLEN=ELEN=32$ ， $SEW_{min}=8$ ，LMUL 为 1/8 时，只会向量寄存器中提供四个位的存储空间。

对于给定支持的分数 LMUL 设置，实现必须支持 SEW 设置，范围在 SEW_{min} 和 $LMUL * ELEN$ 之间（包括这两个值）。使用 vtype 编码时， $LMUL < SEW_{min}/ELEN$ 是保留的，但如果实现不支持这些配置，则可以设置 `vill`。

要求所有实现设置 `vill` 的做法会禁止未来在扩展中使用这种情况，因此为了允许未来定义 $LMUL < SEW_{min}/ELEN$ 行为，我们将此情况视为保留。



如果 `vsetvli` 指令尝试写入 $LMUL < SEW_{min}/ELEN$ ，建议汇编器提供警告（而不是错误）。

要求所有实现设置 `vill` 的做法会禁止未来在扩展中使用这种情况，因此为了允许未来定义 $LMUL < SEW_{min}/ELEN$ 行为，我们将此情况视为保留。

LMUL 由 vtype 中的带符号 `vlmul` 字段设置（即 $LMUL = 2^{v1mul[2:0]}$ ）。衍生值 $VLMAX = LMUL * VLEN / SEW$ 表示在当前 SEW 和 LMUL 设置下，单个向量指令可以操作的最大元素数量，如下表所示。

表31.4 SEW 和 LMUL 设置对应表

v1mul[2:0]			LMUL	groups	VLMAX	按寄存器n分组的寄存器
1	0	1	1/8	32	$VLEN/SEW/8$	$v \quad n$
1	1	0	1/4	32	$VLEN/SEW/4$	$v \quad n$
1	1	1	1/2	32	$VLEN/SEW/2$	$v \quad n$
0	0	0	1	32	$VLEN/SEW$	$v \quad n$

0	0	1	2	16	2*VLEN/SEW	v_n, v_{n+1}
0	1	0	4	8	4*VLEN/SEW	$v_n, \dots, v_{n+3}$
0	1	1	8	4	8*VLEN/SEW	$v_n, \dots, v_{n+7}$

当 LMUL=2 时，向量寄存器组包含向量寄存器 v_n 和向量寄存器 v_{n+1} ，提供两倍的向量长度（以位为单位）。指定 LMUL=2 向量寄存器组且使用奇数编号的向量寄存器的指令是保留的。

当 LMUL=4 时，向量寄存器组包含四个向量寄存器，且指定 LMUL=4 向量寄存器组并使用非四的倍数的寄存器编号的指令是保留的。

当 LMUL=8 时，向量寄存器组包含八个向量寄存器，且指定 LMUL=8 向量寄存器组并使用非八的倍数的寄存器编号的指令是保留的。

无论 LMUL 为何值，掩码寄存器总是包含于单个向量寄存器中。

vta: 向量尾部无关 (Vector Tail Agnostic) 和 vma: 向量掩码无关 (Vector Mask Agnostic) 这两个位分别修改了目标尾部元素和目标未激活的被屏蔽元素在执行向量指令时的行为。尾部和未激活集包含在向量操作期间不会接收新结果的元素位置。

表31.5系统选项表

Vta	Vma	尾部元素	不活跃元素
0	0	不受干扰的	不受干扰的
0	1	不受干扰的	不可预测的
1	0	不可预测的	不受干扰的
1	1	不可预测的	不可预测的

掩码目标尾元素总是被视为与尾部无关的，无论 vta 的设置如何。当一个集合被标记为未干扰时，相应的向量寄存器组中的目标元素保持其先前持有的值。当一个集合被标记为无关时，任何向量目标操作数中的相应目标元素可以保留其先前持有的值，或者被 1 覆盖。在单个向量指令内，每个目标元素可以是未干扰的，也可以被 1 覆盖，且当指令使用相同的输入执行时未干扰或被 1 覆盖的模式在执行时不需要是确定性的。

无关策略是为了适应具有向量寄存器重命名的机器。使用未干扰策略时，所有元素都必须从旧的物理目标向量寄存器读取，并复制到新的物理目标向量寄存器中。当这些未激活值或尾部值不再用于后续计算时，这会导致效率低下。



选择用全 1 而非全 0 的值进行覆盖，是为了避免软件开发人员依赖于写入的值。

一个简单的顺序执行实现可以忽略设置，并简单地使用未干扰策略执行所有向量指令。vta和vma状态位仍然必须在vtype中提供，以确保兼容性并支持线程迁移。

一个乱序执行实现可以选择使用尾部无关 + 掩码无关，利用尾部无关 + 掩码未干扰来减少实现的复

杂度。

无关结果策略的定义较为宽松，以便支持在小型顺序核之间迁移应用线程（这种情况可能会保持无关区域未干扰），以及在具有寄存器重命名的大型乱序核上迁移（这种情况下可能会将无关元素覆盖为1）。由于可能需要在中途重新启动，因此我们允许在单个向量指令中任意混合无关策略。这种允许的策略混合也支持一些实现，它们可能会针对向量寄存器的不同粒度改变策略。例如，在一个正在积极操作的粒度中使用未干扰策略，而对于尾部的粒度则使用重命名为全1的策略。

此外，除了掩码加载指令外，掩码结果尾部中的任何元素也可以被写入该掩码生成操作在 $v1=VLMAX$ 时计算的值。此外，对于掩码逻辑指令和 `vmsbf.m`、`vmsif.m`、`vmsof.m` 等掩码操作指令，结果尾部中的任何元素也可以被写入该掩码生成操作在 $v1=VLEN$ ， $SEW=8$ ， $LMUL=8$ 时计算的值（即掩码寄存器的所有位都可以被覆盖）。



为了简化掩码数据管理，掩码尾部始终被视为无关的，可以按位粒度进行写入。对于掩码寄存器值，似乎没有太多软件需求支持尾部未干扰。允许掩码生成指令写回指令的结果避免了需要逻辑来屏蔽尾部，除了掩码加载指令不能将内存值写入目标掩码尾部，因为这将意味着访问超出软件意图的内存。

汇编语法为 `vsetvli` 指令添加了两个必要的标志：

```
ta    # 尾部无关
tu    # 尾部未干扰
ma    # 掩码无关
mu    # 掩码未干扰
```

```
vsetvli t0, a0, e32, m4, ta, ma    # 尾部无关    , 掩码无关
vsetvli t0, a0, e32, m4, tu, ma    # 尾部未干扰    , 掩码无关
vsetvli t0, a0, e32, m4, ta, mu    # 尾部无关    , 掩码未干扰
vsetvli t0, a0, e32, m4, tu, mu    # 尾部未干扰    , 掩码未干扰
```



在 v0.9 之前，当 `vsetvli` 指令未指定这些标志时，它们默认为掩码未干扰/尾部未干扰。然而，现在已经不建议使用没有这些标志的 `vsetvli`，并且指定标志设置现在是强制性的。默认值可能应该是尾部无关/掩码无关，这样软件就必须在关心非参与元素时明确指定。但是，考虑到这些标志引入之前指令的历史意义，我们决定在未来的汇编代码中始终要求指定这些标志。

向量类型非法 `vill`

`vill` 位用于编码先前的 `vset{i}vl{i}` 指令尝试将不支持的值写入 `vtype`。



`vill` 位位于控制状态寄存器的 $XLEN-1$ 位，用于支持通过分支到符号位检查非法值。

如果 `vill` 位被设置，则任何依赖于 `vtype` 的向量指令执行尝试将引发非法指令异常。



`vset{i} vl{i}` 以及整个寄存器的加载和存储操作不依赖于 `vtype`。

当 `vll` 位被设置时，`vtype` 中的其他 $XLEN-1$ 位应为零。

31.3.5 向量长度寄存器 `vl`

`vl` 是一个 $XLEN$ 位宽的只读控制状态寄存器，只能通过 `vset{i} vl{i}` 指令以及仅故障首次 (fault-only-first) 向量加载指令变体进行更新。`vl` 寄存器保存一个无符号整数，指定要通过向量指令更新的元素数量。



`vl` 中实现的位数取决于实现支持的最小类型的最大向量长度。例如，最小的支持 $SEW=8$ 且 $VLEN=32$ 的向量实现，需要至少六个位来存储值 $0-32$ ($VLEN=32$ ， $LMUL=8$ 和 $SEW=8$ 时， $VLMAX=32$)。

31.3.6 向量字节长度 `vlenb`

该 $XLEN$ 位宽的只读控制状态寄存器 `vlenb` 保存值 $VLEN/8$ ，即向量寄存器的字节长度。



`vlenb` 中的值在任何实现中都是设计时确定的常量。

如果没有这个控制状态寄存器，则需要使用多个指令来计算以字节为单位的 $VLEN$ ，这会干扰当前的 `vl` 和 `vtype` 设置，要求它们被保存和恢复。

31.3.7 向量起始索引控制状态寄存器 `vstart`

该 $XLEN$ 位宽的可读写 `vstart` 寄存器指定了向量指令执行的第一个元素的索引。通常，`vstart` 只有在向量指令发生陷阱时由硬件写入，`vstart` 的值表示触发陷阱的元素（无论是同步异常还是异步中断），并在可恢复陷阱处理后，执行将从该元素继续。所有向量指令定义为从 `vstart` 寄存器中给出的元素编号开始执行，保持目标向量中早期的元素不变，并在执行结束时将 `vstart` 寄存器重置为零。



所有向量指令，包括 `vset{i} vl{i}`，都会将 `vstart` 控制状态寄存器重置为零。

`vstart` 不会被触发非法指令异常的向量指令修改。

`vstart` 控制状态寄存器被定义为仅包含足够的可写位来保存最大元素索引（即最大 VL_{MAX} 减一）。



最大向量长度是通过最大的 LMUL 设置（8）和最小的 SEW 设置（8）得到的，所以 $VLMAX_max = 8 * VLEN / 8 = VLEN$ 。例如，对于 $VLEN=256$ ，`vstart` 将需要 8 位来表示从 0 到 255 的索引。

使用大于当前 `vtype` 设置的最大元素索引的 `vstart` 值是保留的。



如果 `vstart` 超出边界，建议实现应触发陷阱。虽然不要求触发，但未来 `vstart` 高位的潜在用途是存储不精确的陷阱信息。

`vstart` 控制状态寄存器对非特权代码可写，但非零的 `vstart` 值可能导致一些实现上的向量指令运行变慢，因此不推荐应用程序开发者使用 `vstart`。某些向量指令无法在非零 `vstart` 值下执行，并将根据下文定义引发非法指令异常。



让非特权代码访问 `vstart` 可以支持用户级线程库。

实现可以在尝试执行 `vstart` 值在该实现执行相同指令时永远无法产生的向量指令时，抛出非法指令异常，尤其是当执行该指令时具有相同的 `vstart` 设置时。



例如，一些实现永远不会在执行向量算术指令期间进行中断，而是等到指令执行完成后才进行中断。这类实现允许在尝试执行向量算术指令时，当 `vstart` 非零时抛出非法指令异常。



在两个具有不同微架构的硬件线程之间迁移软件线程时，新的硬件线程微架构可能不支持当前的 `vstart` 值。在接收硬件线程上的运行时可能需要模拟指令执行，直到下一个支持的 `vstart` 元素位置为止。或者，迁移事件可以限制为仅发生在相互支持的 `vstart` 位置。

31.3.8 向量定点舍入模式寄存器 `vxrm`

向量定点舍入模式寄存器保存一个两位的读写舍入模式字段，位于最低有效位（`vxrm[1:0]`）。高位 `vxrm[XLEN-1:2]` 应该写为零。向量定点舍入模式有一个单独的控制状态寄存器地址，以允许独立访问，但它也作为字段反映在 `vcsr` 中。



可以通过单个 `csrwi` 指令在保存初始舍入模式的同时设置新的舍入模式。

定点舍入算法如下所示。假设舍入前的结果为 v ，并且要舍去 d 位结果。则舍入后的结果为 $(v \gg d) + r$ ，其中 r 取决于舍入模式，具体由下表指定。

表31.6 vxrm编码

vxrm[1:0]		缩写	舍入模式	舍入增量, r
0	0	rnu	四舍五入	v[d-1]
0	1	rne	截断	截断
1	0	rdn	截断	0
1	1	rod	四舍五入到奇数	!v[d]&v[d-1:0]≠0

取整功能:

`roundoff_unsigned(v, d) = (unsigned(v) d) + r`
`roundoff_signed(v, d) = (signed(v) d) + r`
在下面的指令描述中用于表示此操作。

31.3.9 向量定点饱和标志 vxsat

vxsat 控制状态寄存器具有一个可读写的最低有效位(vxsat[0])，用于指示定点指令是否需要输出值进行饱和，以适应目标格式。位 vxsat[XLEN-1:1]应该被写为零。vxsat 位在 vcsr 控制状态寄存器中有一个镜像。

31.3.10 向量控制和状态寄存器 vcsr

vxrm 和 vxsat 这两个独立的控制状态寄存器也可以通过 XLEN 位宽的向量控制状态寄存器 vcsr 中的字段进行访问。

31.3.11 复位时的向量扩展状态

向量扩展在复位时必须处于一致的状态。特别是，vtype 和 vl 必须具有可以通过单条 vsetvl 指令读取并恢复的值。



建议在复位时，将vtype.vill置为 1，vtype的其余位设为零，并将vl设为零。

大多数向量单元的使用场景都需要执行初始的vset{i}vl{i}，这将重置vstart。vxrm和vxsat字段在使用前应通过软件显式复位。

vstart、vxrm 和 vxsat 控制状态寄存器在复位时可以具有任意值。
向量寄存器在复位时可以具有任意值。

31.4 向量元素到向量寄存器状态的映射

下列示意图展示了在不同的 SEW（元素宽度）和 LMUL（向量寄存器分组）设置下，向量寄存器如何存储不同宽度的元素。此外，映射方式也取决于具体实现的 VLEN（向量寄存器长度）。元素存储到向量寄存器时，最低有效字节（LSB）位于最低编号的比特位置。

该映射的选择旨在为软件提供最简单和最具移植性的模型，但在某些操作上，对于较宽的向量数据通路，可能会导致较大的布线成本。向量指令集特别设计以支持内部重新排列向量数据以适应不同 SEW 的实现，从而减少数据通路布线成本，同时在外部分保持简单的软件模型。



例如，某些微架构可以跟踪向量寄存器的EEW（有效元素宽度），如果使用不同的EEW访问该寄存器，则可以插入额外的数据重排操作。

31.4.1 LMUL = 1 时的映射方式

当 LMUL=1 时，元素按顺序从向量寄存器的最低有效位（LSB）到最高有效位（MSB）进行存储。



为了提高可读性，向量寄存器的布局以从右到左递增字节地址的方式绘制。元素内部的比特编号采用小端格式，从右到左递增，对应于数值大小的增加。

LMUL=1 示例

元素索引以十六进制表示，并显示在存储元素的最低有效字节位置。

VLEN=32b

Byte	3	2	1	0
SEW=8b	3	2	1	0
SEW=16b	1	0		
SEW=32b	0			

VLEN=64b

Byte	7	6	5	4	3	2	1	0
SEW=8b	7	6	5	4	3	2	1	0
SEW=16b	3	2	1	0				
SEW=32b	1	0						
SEW=64b	0							

VLEN=128b

Byte	F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0
SEW=8b	F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0
SEW=16b	7	6	5	4	3	2	1	0								
SEW=32b	3	2	1	0												
SEW=64b	1	0														

VLEN=256b

Byte 1F1E1D1C1B1A19181716151413121110 F E D C B A 9 8 7 6 5 4 3 2 1 0

SEW=8b 1F1E1D1C1B1A19181716151413121110 F E D C B A 9 8 7 6 5 4 3 2 1 0

SEW=16b F E D C B A 9 8 7 6 5 4 3 2 1 0

SEW=32b 7 6 5 4 3 2 1 0

SEW=64b 3 2 1 0

31.4.2 LMUL < 1 的映射方式

当 $LMUL < 1$ 时，向量寄存器中仅使用前 $LMUL * VLEN / SEW$ 个元素。
寄存器中的其余空间被视为向量尾部的一部分，因此必须遵循 *vta*（向量尾部操作）设置。

例如，VLEN=128b，LMUL=1/4

Byte F E D C B A 9 8 7 6 5 4 3 2 1 0

SEW=8b - - - - - - - - - - - 3 2 1 0

SEW=16b - - - - - - 1 0

SEW=32b - - - 0

31.4.3 LMUL > 1 的映射方式

当向量寄存器被分组时，向量寄存器组中的元素按照元素顺序连续存储。存储从编号最小的向量寄存器开始，当一个寄存器填满后，依次存入组内编号更高的寄存器。

LMUL > 1 时

VLEN=32b, SEW=8b, LMUL=2

Byte 3 2 1 0

v2*n 3 2 1 0

v2*n+1 7 6 5 4

VLEN=32b, SEW=16b, LMUL=2

Byte 3 2 1 0

v2*n 1 0

v2*n+1 3 2

VLEN=32b, SEW=16b, LMUL=4

Byte 3 2 1 0

v4*n 1 0

v4*n+1	3	2
v4*n+2	5	4
v4*n+3	7	6

VLEN=32b, SEW=32b, LMUL=4

Byte	3	2	1	0
v4*n				0
v4*n+1				1
v4*n+2				2
v4*n+3				3

VLEN=64b, SEW=32b, LMUL=2

Byte	7	6	5	4	3	2	1	0
v2*n					1			0
v2*n+1					3			2

VLEN=64b, SEW=32b, LMUL=4

Byte	7	6	5	4	3	2	1	0
v4*n					1			0
v4*n+1					3			2
v4*n+2					5			4
v4*n+3					7			6

VLEN=128b, SEW=32b, LMUL=2

Byte	F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0
v2*n							3		2				1			0
v2*n+1							7		6				5			4

VLEN=128b, SEW=32b, LMUL=4

Byte	F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0
v4*n							3			2				1		0
v4*n+1							7			6				5		4
v4*n+2							B			A				9		8
v4*n+3							F			E				D		C

31.4.4 混合宽度操作的映射

向量 ISA 设计支持混合宽度操作，而无需额外的显式重排指令。在处理具有不同精度值的多个向量时，推荐的软件策略是动态修改 vtype，以保持 SEW/LMUL 不变（从而保持 VLMAX 不变）。

以下示例展示了在 VLEN=128b 的实现中，四种不同的打包元素宽度（8b、16b、32b、64b）。向量

寄存器分组因子（LMUL）根据相对元素大小增加，以确保每个组能够容纳相同数量的向量元素（此示例中 VLMAX=8），从而简化循环剥离代码。

当 VLEN=128b，而 SEW/LMUL=16 时

Byte	F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0	
vn	-	-	-	-	-	-	-	7	6	5	4	3	2	1	0		SEW=8b, LMUL=1/2
vn								7	6	5	4	3	2	1	0		SEW=16b, LMUL=1
v2*n																	SEW=32b, LMUL=2
v2*n+1																	
v4*n																	SEW=64b, LMUL=4
v4*n+1																	
v4*n+2																	
v4*n+3																	

表 31.7 展示了在包含混合宽度操作的循环中，每种可能的固定 SEW/LMUL 操作点。每一列代表一个固定的 SEW/LMUL 操作点。表中的条目是 LMUL 值，这些 LMUL 值使该行的数据宽度对应于该列的 SEW/LMUL 值。在同一列中，某个数据宽度的 LMUL 设置表明它可以与该列中其他具有 LMUL 设置的数据宽度对齐，从而确保它们都具有相同的 VLMAX。

表31.7 SEW/LMUL

SEW= 8	8	4	2	1	1/2	1/4	1/8
SEW= 16		8	4	2	1	1/2	1/4
SEW= 32			8	4	2	1	1/2
SEW= 64				8	4	2	1

较大的 LMUL 设置还可以用于简单地增加向量长度，以减少指令获取和调度开销，适用于需要较少向量寄存器组的情况。

31.4.5 掩码寄存器布局

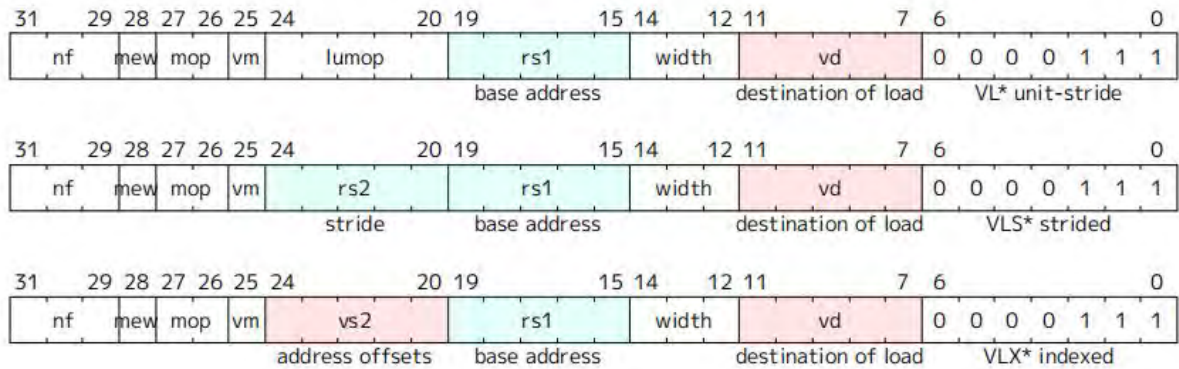
无论 SEW 和 LMUL 为何，向量掩码只占用一个向量寄存器。在掩码向量寄存器中，每个元素对应一个单独的掩码位。元素 i 的掩码位位于掩码寄存器的第 i 位，而不受 SEW 或 LMUL 影响。

31.5 向量指令格式

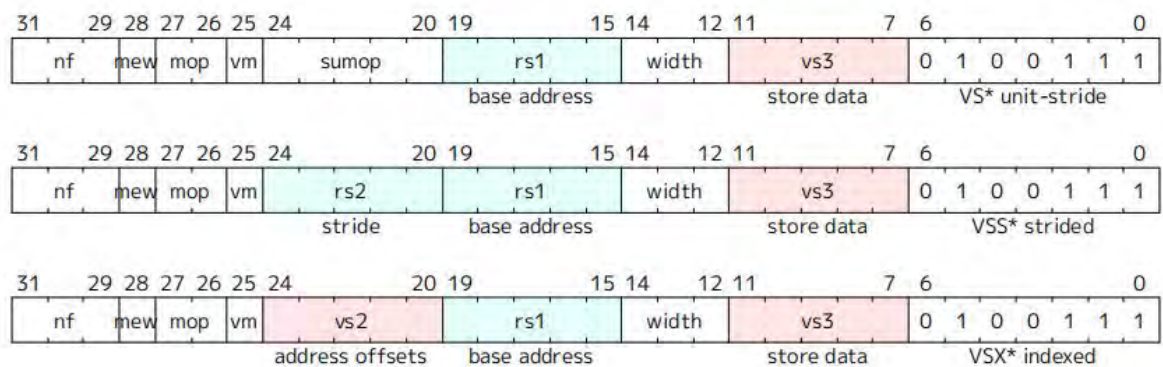
向量扩展的指令分布在两个现有的主要操作码（LOAD-FP 和 STORE-FP）以及一个新的主要操作码（OP-V）下。

向量加载和存储指令被编码在标量浮点加载和存储的主要操作码（LOAD-FP/STORE-FP）内。向量加载和存储的编码重新利用了标准标量浮点加载/存储指令的 12 位立即数字段的一部分，以提供额外的向量指令编码，其中第 25 位存储标准向量掩码位（详见 31.5.3.1）。

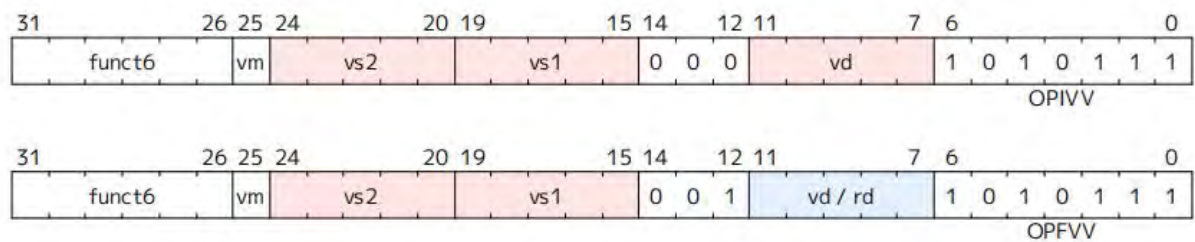
LOAD-FP 主操作码下的向量加载指令格式

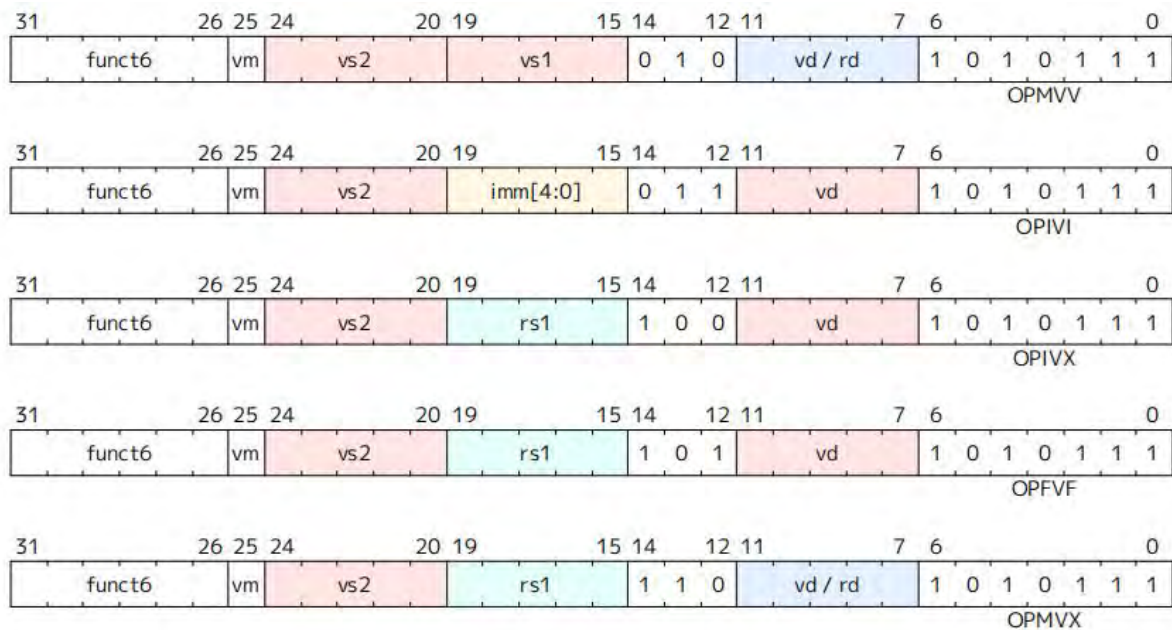


STORE-FP 主操作码下的向量存储指令格式

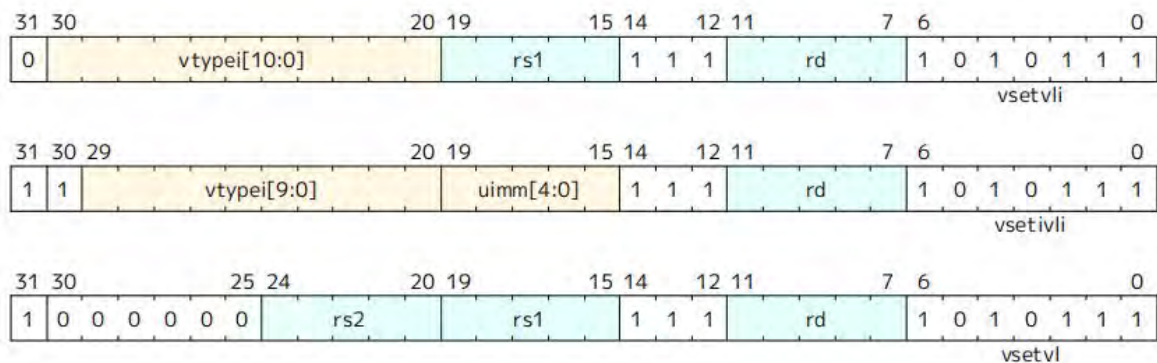


OP-V 主操作码下的向量算术指令格式





OP-V 主操作码下的向量配置指令格式



向量指令可以使用标量或向量作为源操作数，并生成标量或向量结果。大多数向量指令可以无条件执行，或者在掩码控制下有条件执行。

向量加载和存储指令在向量寄存器元素和内存之间传输比特模式。向量算术指令在向量寄存器元素中进行运算。

31.5.1 标量操作数

标量操作数可以是立即数，也可以来自 x 寄存器、f 寄存器或某个向量寄存器的元素 0。标量结果可以写入 x 或 f 寄存器，也可以写入某个向量寄存器的元素 0。无论当前 LMUL 设置如何，任何向量寄存器都可以用于存储标量值。



实Zfinx (F in X) 是一个新的 ISA 扩展，其中浮点指令的参数来自整数寄存器文件。向量扩展也兼容 Zfinx，在 Zfinx 向量扩展中，向量-标量浮点指令从x寄存器获取其标量参数。

我们曾考虑将f寄存器与v寄存器进行重叠，但最终未采用这一方案。当前方案减少了向量寄存器的压力，避免了与标准调用约定的冲突，简化了高性能标量浮点设计，并与 Zfinx ISA 选项兼容。尽管f和v寄存器的重叠可以减少某些实现中的状态位数，但会增加高性能设计的复杂性，并阻碍与 Zfinx

ISA 选项的兼容性。

31.5.2 向量操作数

每个向量操作数都有一个有效元素宽度 (EEW) 和一个有效 LMUL (EMUL)，用于确定向量寄存器组中所有元素的大小和位置。默认情况下，对于大多数指令的大多数操作数， $EEW=SEW$ ， $EMUL=LMUL$ 。

某些向量指令的源和目标向量操作数具有相同数量的元素，但宽度不同，因此 EEW 和 EMUL 可能与 SEW 和 LMUL 不同，但满足 $EEW/EMUL = SEW/LMUL$ 。例如，大多数加宽算术指令的源操作数组具有 $EEW=SEW$ 和 $EMUL=LMUL$ ，而目标操作数组的 $EEW=2 \times SEW$ ， $EMUL=2 \times LMUL$ 。缩小指令的源操作数具有 $EEW=2 \times SEW$ 和 $EMUL=2 \times LMUL$ ，而目标操作数的 $EEW=SEW$ ， $EMUL=LMUL$ 。

向量操作数或结果可能占用一个或多个向量寄存器，具体取决于 EMUL，但始终使用编号最小的向量寄存器来指定寄存器组。使用除编号最小的寄存器以外的寄存器来指定寄存器组的行为目前是保留的。

一个向量寄存器不能在同一条指令中以不同的 EEW 作为源操作数。掩码寄存器被视为 $EEW=1$ 。如果某个编码导致同一个向量寄存器在一次指令中以两个或多个不同 EEW 被读取（包括当该寄存器在两个或多个寄存器组的不同位置出现时），则该编码是保留的。



实际上，在同一指令中以不同 EEW 读取同一寄存器没有软件上的好处，这一约束减少了需要根据 EEW 内部重新排列数据的实现的复杂性。

如果目标向量寄存器组与源向量寄存器组重叠，必须满足以下条件之一：

- 目标 EEW 等于源 EEW。
- 目标 EEW 小于源 EEW，并且重叠发生在源寄存器组编号最小的部分（例如，当 $LMUL=1$ 时，`vnsrl.wi v0, v0, 3` 是合法的，但目标为 `v1` 则非法）。
- 目标 EEW 大于源 EEW，源 EMUL 至少为 1，并且重叠发生在目标寄存器组编号最大的部分（例如，当 $LMUL=8$ 时，`vzext.vf4 v0, v6` 是合法的，但源为 `v0`、`v2` 或 `v4` 则非法）。

对于确定寄存器组重叠约束的目的，掩码元素的 EEW 被视为 1。



重叠约束旨在支持没有寄存器重命名的架构中的可恢复异常。

任何违反重叠约束的指令编码都是保留的。当源和目标寄存器重叠且 EEW 不同时，该指令在执行时不会受掩码或尾部控制，无论 `vtype` 中 `vta` 和 `vma` 位的设置如何。

指令使用的最大向量寄存器组不能超过 8 个向量寄存器（即 $EMUL \leq 8$ ）。如果某条向量指令要求寄存器组超过 8 个向量寄存器，则该指令编码保留。例如， $LMUL=8$ 时的加宽操作会生成加宽的目标向量寄存器组，这将导致 $EMUL=16$ ，因此该操作是保留的。

加宽的标量值（例如加宽归约操作的输入和输出）存储在向量寄存器的第一个元素中，并且 $EMUL=1$ 。

31.5.3 向量掩码

许多向量指令支持掩码。被掩码屏蔽的元素操作不会产生异常。对于目标向量寄存器中的被屏蔽元素，根据 `vtype` 中 `vma` 位的设置（详见 31.3.4.3），这些元素的处理方式可以是掩码未干扰或掩码无关。

用于控制掩码向量指令执行的掩码值始终由向量寄存器 v0 提供。



掩码存储在向量寄存器中，而不是单独的掩码寄存器文件，这样可以减少整体的架构状态并简化指令集架构。

未来的向量扩展可能会提供更长的指令编码，以支持完整的掩码寄存器指定。

对于掩码向量指令，目标向量寄存器组不能与源掩码寄存器（v0）重叠，除非该目标寄存器用于存储掩码值（例如比较操作），或者用于存储归约运算的标量结果。任何违反此规则的指令编码均为保留编码。



该约束旨在支持使用非零 vstart 值进行指令重启。

其他向量寄存器可以用于存储工作掩码值，指令集也提供了掩码向量逻辑操作来执行谓词计算。根据第 31.3.4.3 章节的规范，掩码目标值始终被视为尾部无关（tail-agnostic），无论 vtype 中 vta 位的设置如何。

掩码编码

在支持掩码的情况下，掩码信息被编码在指令（inst[25]）的单比特 vm 字段中。

在汇编代码中，向量掩码被表示为另一个向量操作数，其中 .t 表示操作仅在 v0.mask[i] = 1 时执行（t 代表 true）。如果未指定掩码操作数，则默认假定为无掩码的向量执行（vm=1）。

```
vop.v*    v1, v2, v3, v0.t # 当 v0.mask[i]=1, vm=0 时有效
vop.v*    v1, v2, v3       # 无掩码向量操作, vm=1
```



尽管当前的向量扩展仅支持一个向量掩码寄存器 v0 且只支持谓词的真值形式，但汇编语法会将其完全写出，以兼容未来可能添加掩码寄存器说明符并支持真值和补码掩码值的扩展。掩码操作数上的 .t 后缀也有助于直观地

.mask 后缀不是汇编语法的一部分。我们仅在涉及掩码向量的下标表示时添加，例如 v0.mask[i]。

31.5.4 预启动、激活、未激活、主体和尾部元素的定义

在执行向量指令时，目标元素索引可分为三个不相交的子集：

- * 预启动元素是指索引小于 vstart 寄存器初始值的元素。预启动元素不会引发异常，也不会更新目标向量寄存器。

- * 主体元素是指索引大于或等于 vstart 寄存器初始值，并且小于当前 vl（向量长度）设置的元素。主体元素可以进一步分为两个不相交的子集：

- ** 激活元素是在向量指令执行期间，处于主体范围内且当前掩码在该元素位置启用的元素。激活元素可以引发异常，并更新目标向量寄存器组。

**** 未激活元素**是在主体范围内但当前掩码在该元素位置被禁用的元素。未激活元素不会引发异常，也不会更新目标向量寄存器组，除非掩码无关模式被指定（`vtype.vma=1`），在这种情况下，未激活元素可能会被覆盖为 1。

*** 尾部元素**是指超出 `v1`（向量长度）设置的范围的元素。尾部元素不会引发异常，也不会更新目标向量寄存器组，除非尾部无关模式被指定（`vtype.vta=1`）。在尾部无关模式下，尾部元素可能会被覆盖为 1，或者在掩码生成指令（除掩码加载指令外）的情况下，被指令的计算结果覆盖。当 `LMUL < 1` 时，尾部还包括超出 `VLMAX`（最大向量长度）但仍存储在同一向量寄存器中的元素。

```
for element index x
  prestart(x) = (0 <= x < vstart)
  body(x)     = (vstart <= x < v1)
  tail(x)     = (v1 <= x < max(VLMAX, VLEN/SEW))
  mask(x)     = unmasked || v0.mask[x] == 1
  active(x)   = body(x) && mask(x)
  inactive(x) = body(x) && !mask(x)
```

当 `vstart ≥ v1` 时，不存在主体元素，并且任何目标向量寄存器组中的元素都不会被更新，包括尾部元素也不会被无关值覆盖。



由此可得，当 `v1=0` 时，无论 `vstart` 的值如何，目标向量寄存器组中的任何元素（包括无关元素）都不会被更新。

如果指令写入 `x` 寄存器或 `f` 寄存器，即使 `vstart ≥ v1`，包括 `v1=0` 的情况，这些寄存器仍会被写入。



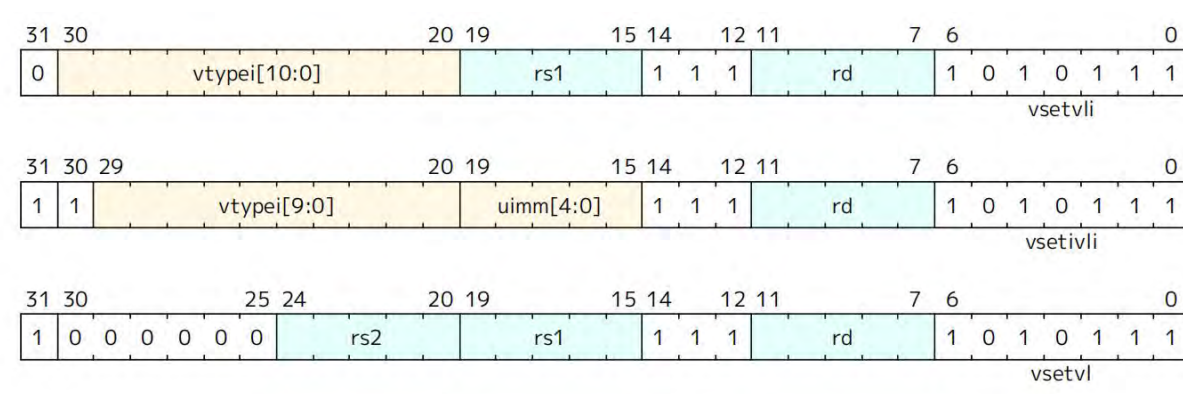
某些指令（如 `vslidtdown` 和 `vrgather`）可能会在源向量寄存器组中读取超出 `v1` 甚至 `VLMAX` 之后的索引。一般策略是，当索引大于源向量寄存器组的 `VLMAX` 时，返回值 0。

31.6 配置设定指令（`vsetvli/vsetivli/vsetvl`）

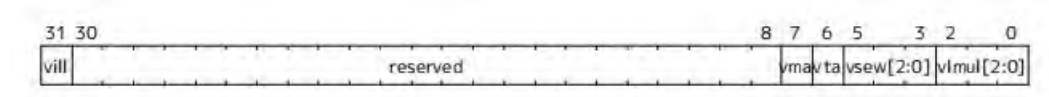
在处理大量元素时，常见的一种方法是分块处理，其中每次循环迭代处理一定数量的元素，并持续迭代，直到所有元素都被处理完。RISC-V 向量规范提供了对这种方法的直接且可移植的支持。应用程序指定要处理的总元素数（应用向量长度，AVL），作为 `v1` 的候选值，硬件则会通过通用寄存器返回每次迭代中硬件将实际处理的元素数量（存储在 `v1` 中）。这一数量通常比 AVL 小，具体取决于微架构实现和 `vtype` 设置。一个简单的循环结构（详见 31.6.4）展示了代码如何轻松跟踪剩余元素数量以及每次迭代由硬件处理的元素数量。

下面提供了一组指令用于以快速配置 `v1` 和 `vtype` 的值，使其匹配应用需求。`vset{i}v1{i}` 指令根据其参数设置 `vtype` 和 `v1` 控制状态寄存器，并将 `v1` 的新值写入 `rd`。


```
vsetvli rd, rs1, vtypei    # rd = new vl, rs1 = AVL, vtypei = new vtype setting
vsetivli rd, uimm, vtypei  # rd = new vl, uimm = AVL, vtypei = new vtype setting
vsetvl  rd, rs1, rs2       # rd = new vl, rs1 = AVL, rs2 = new vtype value
```



31.6.1 vtype 编码



 该图展示的是 RV32 系统的布局，而通常 vill 位应位于 XLEN-1 比特处

表 31.8 vtype 寄存器布局

位	名称	描述
XLEN-1	vill	Illegal value if set
XLEN-2:8	o	Reserved if non-zero
7	vma	Vector mask agnostic
6	vta	Vector tail agnostic
5: 3	Vsew[2:0]	Selected element width (SEW) setting
2:0	Vlmul[2:0]	Vector register group multiplier (LMUL) setting

新的 vtype 值在 vsetvli 和 vsetivli 指令的立即数字段中进行编码，而在 vsetvl 指令中则存储在 rs2 寄存器中

推荐的 vset{i}vli 类型立即数汇编名称

```
e8    # SEW=8b
e16   # SEW=16b
e32   # SEW=32b
e64   # SEW=64b

mf8   # LMUL=1/8
mf4   # LMUL=1/4
mf2   # LMUL=1/2
m1    # LMUL=1, 如果缺少 m 设置, 则假定为
m2    # LMUL=2
m4    # LMUL=4
m8    # LMUL=8
```

Examples:

```
vsetvli t0, a0, e8, ta, ma      # SEW= 8, LMUL=1
vsetvli t0, a0, e8, m2, ta, ma  # SEW= 8, LMUL=2
vsetvli t0, a0, e32, mf2, ta, ma # SEW=32, LMUL=1/2
```

vsetvl 变体的操作方式类似于 vsetvli，不同之处在于它从 rs2 获取 vtype 值，并且可用于上下文恢复。

不支持的 vtype 值
如果 vtype 值不受实现支持，则 vtype 中的 vill 位将被设置，其余位被清零，同时 vl 寄存器也被设置为零。



早期草案要求在将vtype设为非法值时触发异常。然而，这将导致指令集在控制状态寄存器写入时出现首个数据相关的异常。实现可以选择在向vtype写入非法值时触发异常，而不是设置vill，以便通过仿真支持新的配置，实现前向兼容性。当前方案支持通过检查vill是否被清除来进行轻量级运行时查询，以确定支持的向量单元配置。

vtype 值如果 vill 被设置，则视为不受支持的配置。实现必须检查 vtype 值的所有位，以确定配置是否受支持。如果 vtype 值的任何位置包含不受支持的值，则必须设置 vill。



特别地，vsetvl指令的vtype参数中所有XLEN 位都必须进行检查。实现不能忽略未实现的字段。必须检查所有位，以确保假设vtype中不受支持的向量特性的代码不会在较旧的实现上错误执行，而是会触发异常。

31.6.2 AVL 编码

新的向量长度设置基于 AVL，对于 vsetvli 和 vsetvl，AVL 在 rs1 和 rd 字段中编码，如表 31.9 下所示：

表31.9 AVL在vsetvli和vsetvl指令中的使用

Rd	Rs1	AVL 值	对 vl 的影响
-	!x0	Value in x[rs1]	Normal stripmining
!x0	x0	~ 0	Set vl to VLMAX
x0	x0	Value in vl register	Keep existing vl (of course, vtype may change)

当 rs1 不为 x0 时，AVL 由 rs1 指定的 x 寄存器中的无符号整数表示，并且新的 vl 值也会写入 rd 指定的 x 寄存器。

当 rs1=x0 但 rd $\neq$ x0 时，AVL 取最大无符号整数值(~ 0)，计算出的 VLMAX 被写入 vl，同时也写入 rd 指定的 x 寄存器。

当 rs1=x0 且 rd=x0 时，指令的操作就像当前 vl 中的向量长度作为 AVL 使用一样，结果值会写入 vl，但不会写入目标寄存器。只有在新的 SEW/LMUL 比率不会实际改变 VLMAX 和 vl 时，这种形式才可以使用。如果新的 SEW/LMUL 比例会导致 VLMAX 发生变化，则该指令的使用是保留的。此外，如果 vill 先前已经被设置为 1，则该指令的使用也是保留的。在这两种情况下，实现可以选择设置 vill。

该指令的最后一种形式允许在不降低 VLMAX 的情况下更改 vtype 寄存器，同时保持当前 vl。这一设计是为了确保 vl 总是持有当前 vtype 设置的合法值。当前 vl 值可以从 vl 控制状态寄存器中读取。如果新的 SEW/LMUL 比率导致 VLMAX 缩小，这条指令可能会减小 vl 的值，因此这一情况被保留，因为不清楚这种操作是否普遍有用，且实现可以假设该指令不会更改 vl，以优化其微架构。



对于 vsetivli 指令，AVL 在 rs1 段中编码为一个 5 位零扩展立即数（范围 031）。

vsetivli 的 AVL 编码方式与常规控制状态寄存器立即数值的编码方式相同。

当向量的尺寸较小且已知能够适应向量寄存器时，vsetivli 指令可以生成更紧凑的代码，在这种情况下，不会有分块处理开销。

31.6.3 设置 vl 的约束

vset{i}vl{i} 指令首先根据其 vtype 参数设置 VLMAX，然后按照以下约束设置 vl：

- 如果 $AVL \leq VLMAX$ ，则 $vl = AVL$
- 如果 $AVL < (2 * VLMAX)$ ，则 $\text{ceil}(AVL / 2) \leq vl \leq VLMAX$
- 如果 $AVL \geq (2 * VLMAX)$ ，则 $vl = VLMAX$
- 在相同的输入 AVL 和 VLMAX 值的情况下，实现应保证 vl 取值是确定性的
- 以下特性是上述规则的推论：
 - .. $AVL = 0$ 则 $vl = 0$
 - .. $AVL > 0$ 则 $vl > 0$
 - .. vl {1e} VLMAX
 - .. vl {1e} AVL
- .. 读取 vl 作为 vset{i}vl{i} 指令的 AVL 参数时，如果生成的 VLMAX 与 vl 被读取时的 VLMAX 相等，则 vl 保持不变



vl 的设定规则被设计得足够严格，以确保在 $AVL \leq VLMAX$ 的情况下，vl 的行为在寄存器溢出和上下文切换时保持一致，同时又足够灵活，使得在 $AVL > VLMAX$ 的情况下，能够优化向量通道的利用率。

例如，当 $VLMAX < AVL < 2 * VLMAX$ 时，实现可以选择 $vl = \text{ceil}(AVL/2)$ 以便在分块处理循环的最后两次迭代中均匀分配工作量。约束 2 确保即使在 $AVL < 2 * VLMAX$ 的情况下，归约循环的第一轮分块处理迭代也

能使用所有迭代中最大的向量长度。这样，软件就不需要显式计算分开处理循环过程中观察到的最大向量长度。约束 2 还允许在 $VLMAX < AVL < 2*VLMAX$ 的情况下，将 $v1$ 设为 $VLMAX$ 。

31.6.4 分块处理与 SEW 变化示例

SEW 和 LMUL 设置可以动态更改，以在单个循环中为混合宽度的操作提供高吞吐量。

示例：加载 16 位值，将其扩展并乘法得到 32 位结果，再将 32 位结果右移 3 位，然后存储 32 位值。

```
# 入口：
# a0 存储要处理的总元素数量
# a1 存储源数组的地址
# a2 存储目标数组的地址
loop:
    vsetvli a3, a0, e16, m4, ta, ma # vtype = 16 位整数向量;
                                     # 同时更新 a3 为 v1 (本次迭代的元素数)
    vle16.v v4, (a1)                # 获取 16 位向量
    slli t1, a3, 1                    # 将本次迭代的元素数乘以 2 字节/源元素
    add a1, a1, t1                    # 增加指针
    vwmul.vx v8, v4, x10              # 在 <v8-v15> 中进行 32 位宽度乘法

    vsetvli x0, x0, e32, m8, ta, ma # 操作 32 位值
    vsrl.vi v8, v8, 3
    vse32.v v8, (a2)                  # 存储 32 位元素向量
    slli t1, a3, 2                    # 将本次迭代的元素数乘以 4 字节/目标元素
    add a2, a2, t1                    # 增加指针
    sub a0, a0, a3                    # 将计数减去 v1
    bnez a0, loop                     # 根据情况决定是否继续迭代
```

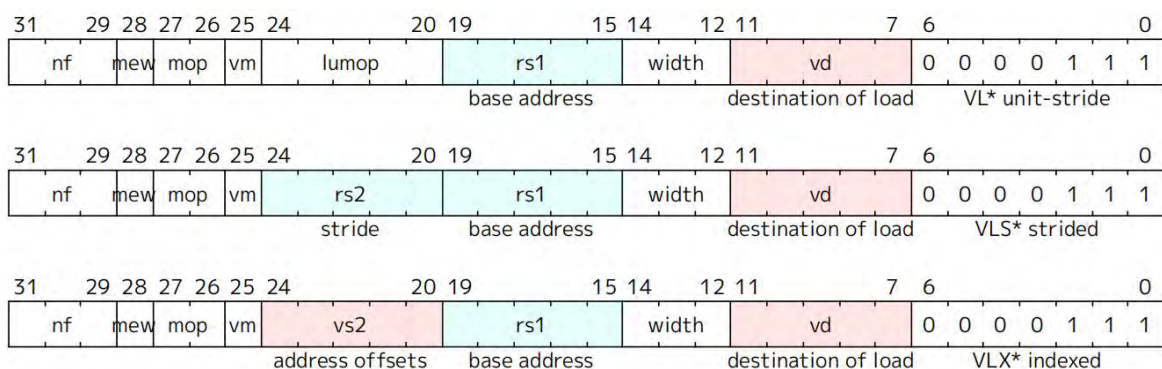
31.7 向量加载与存储

向量加载和存储用于在向量寄存器和内存之间传输数据。向量加载和存储可以使用掩码，并且仅对激活元素访问内存或触发异常。带掩码的向量加载不会更新目标向量寄存器组中的未激活元素，除非指定了掩码无关模式。所有向量加载和存储指令都可以接受并生成非零的 $vstart$ 值。

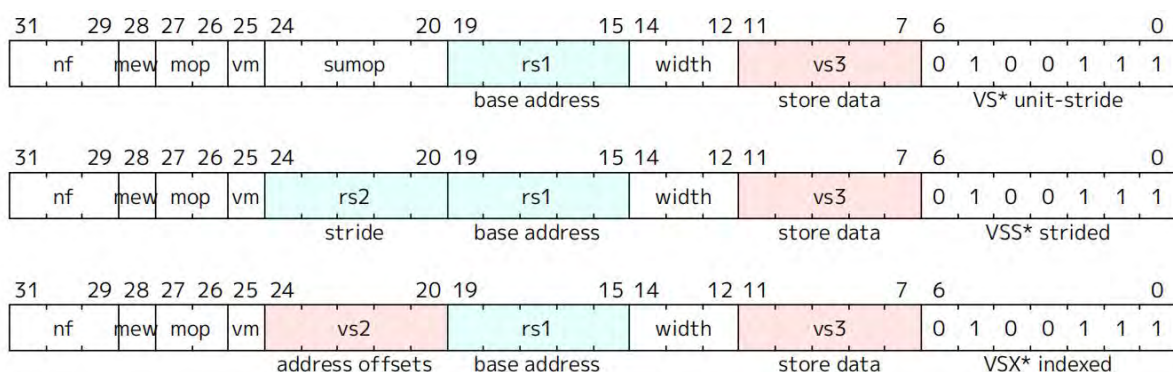
31.7.1 向量加载/存储指令编码

向量加载和存储指令编码在标量浮点加载和存储的主操作码范围内。这些指令复用了标量浮点加载/存储指令的 12 位立即数字段的一部分，以提供额外的向量指令编码。其中，第 25 位存储标准向量掩码位。（详见第 31.5.3.1）。

向量加载指令格式（位于 LOAD-FP 主操作码下）



向量存储指令格式（位于 STORE-FP 主操作码下）



向量内存的单位步长和常量步长操作会在指令中静态编码数据传输的 EEW（元素有效宽度），以减少在混合宽度程序中访问内存时 vtype 变更的次数。索引操作使用指令中的显式 EEW 编码来设置用于索引的元素大小，并使用 SEW/LMUL 来指定数据宽度。

31.7.2 向量加载/存储的寻址模式

向量扩展支持以下三种寻址模式：单位步长（unit-stride）、步长（strided）、索引。向量加载/存储的基址寄存器和步长源于通用寄存器 x 组。

基址计算所有向量访问的基址有效地址由 rs1 指定的 x 寄存器内容提供。向量单位步长操作：访问的元素在内存中是连续存储的，并从基址有效地址开始存取。

向量常量步长操作：第一个元素存取基址有效地址，后续元素按照 rs2 指定的 x 寄存器中的字节偏移量进行地址递增。

向量索引操作计算每个元素的有效地址时，采用如下方式：由 vs2 指定的向量偏移操作数的每个元素值加上基址有效地址，计算得到每个元素的最终地址。数据向量寄存器组的 EEW=SEW，EMUL=LMUL。偏移向量寄存器组的 EEW 由指令编码决定， $EMUL=(EEW/SEW) \times LMUL$ 。向量偏移操作数被视为字节地址偏移量的向量。



索引操作也可用于访问结构体数组中的字段。其中 vs2 向量保存指向对象基址的指针，标量 x 寄存器保存每个对象中成员字段的偏移量。支持这种情况是为什么索引操作没有被定义为按数据 EEW 缩放元素索引的原因。

若偏移向量元素宽度小于 XLEN，则零扩展至 XLEN 后再加上基址地址。若偏移向量元素宽度大于 XLEN，则仅取低 XLEN 位用于地址计算。若 EEW（元素有效宽度）不受支持，实现必须引发非法指令异常。



某些硬件可能会对索引EEW施加上限（例如仅支持最大 XLEN，小于 ELEN）。

向量寻址模式由 2-bit mop[1:0] 字段进行编码。

表31.10 加载编码

mop [1:0]		描述	操作码
0	0	unit-stride	VLE<EEW>
0	1	indexed-unordered	VLUXEI<EEW>
1	0	strided	VLSE<EEW>
1	1	indexed-ordered	VLOXEI<EEW>

向量单位步长和常量步长的内存访问不保证各个元素访问的顺序。向量索引加载和存储指令提供两种形式：有序和无序。索引有序指令在内存访问时会保留元素的顺序。

对于无序指令（mop[1:0] ≠ 11），元素访问的顺序不受保证。如果访问的是强序的 IO 区域，元素访问可以以任意顺序发起。



如果要对强顺序性 IO 区域提供有序的向量访问，则应使用索引有序指令。

对于支持精确向量异常的实现：索引无序存储的异常必须是精确的。

单位步长加载和存储指令中，使用 5-bit lumop 和 sumop 字段对额外的单位步长向量寻址模式进行编码。

Lumop[4:0]					描述
0	0	0	0	0	unit-stride load
0	1	0	0	0	unit-stride, whole register load
0	1	0	1	1	unit-stride, mask load, EEW=8
1	0	0	0	0	unit-stride fault-only-first
x	x	x	x	x	other encodings reserved

sumop[4:0]					描述
0	0	0	0	0	unit-stride store
0	1	0	0	0	unit-stride, whole register store
0	1	0	1	1	unit-stride, mask store, EEW=8
0	1	0	1	1	unit-stride fault-only-first
x	x	x	x	x	other encodings reserved

nf[2:0]字段用于编码每个段中的字段数。对于常规向量加载和存储，nf=0，表示每个元素位置在向量寄存器组和内存之间仅移动一个值。更大的nf值用于访问段内多个连续字段。此外，nf[2:0]字段还用于编码整个向量寄存器加载/存储指令所需传输的完整向量寄存器数量。

31.7.3 向量加载/存储宽度编码

向量加载和存储指令的EEW（元素有效宽度）直接编码在指令中。计算EMUL的公式为 $EMUL = (EEW/SEW) * LMUL$ 。如果EMUL超出范围（即 $EMUL > 8$ 或 $EMUL < 1/8$ ），该指令编码会被保留。选定的EMUL必须具有合法的寄存器组编号，否则指令编码会被保留。

向量单位步长和常量步长使用指令中编码的EEW/EMUL来处理数据值，而向量索引加载和存储则使用指令中编码的EEW/EMUL来处理索引值，使用vtype中的SEW/LMUL来处理数据值。向量加载和存储使用的编码宽度是标准标量浮点加载和存储未占用的宽度值。实现必须提供与所有支持的SEW设置对应的EEW的向量加载和存储。对于不支持的EEW宽度的向量加载/存储编码，必须触发非法指令异常。

内存位是内存中每个元素的大小。数据寄存器位是寄存器中每个数据元素的大小。索引位是寄存器中每个索引的大小。当mew位（inst[28]）被设置时，预计用于编码128位及以上的扩展内存大小，但这些编码当前是保留的。

31.7.4 向量单位步长指令

向量单位步长加载和存储

vd 目标寄存器，rs1 基地址，vm 为掩码编码（v0.t 或 <缺失>）

vle8.v vd, (rs1), vm # 8 位单位步长加载

vle16.v vd, (rs1), vm # 16 位单位步长加载

vle32.v vd, (rs1), vm # 32 位单位步长加载

vle64.v vd, (rs1), vm # 64 位单位步长加载

vs3 存储数据，rs1 基地址，vm 为掩码编码（v0.t 或 <缺失>）

vse8.v vs3, (rs1), vm # 8 位单位步长存储

vse16.v vs3, (rs1), vm # 16 位单位步长存储

vse32.v vs3, (rs1), vm # 32 位单位步长存储

vse64.v vs3, (rs1), vm # 64 位单位步长存储

额外的单位步长掩码加载和存储指令被提供，用于将掩码值传输到内存或从内存传输。这些指令的操作与未掩码字节加载或存储（EEW=8）类似，唯一的区别是有效向量长度为 $evl = \text{ceil}(vl/8)$ （即 $EMUL=1$ ），并且目标寄存器始终使用与尾部无关的策略进行写入。

向量单位步长掩码加载

vlm.v vd, (rs1) # 加载长度为 $\text{ceil}(vl/8)$ 的字节向量

向量单位步长掩码存储

vsm.v vs3, (rs1) # 存储长度为 $\text{ceil}(vl/8)$ 的字节向量

vlm.v 和 vsm.v 与 vle8.v 和 vse8.v 使用相同的width[2:0]=0 编码，但通过不同的 lumop 和 sumop 编码加以区分。由于vlm.v 和 vsm.v 作为字节加载和存储指令工作，因此这些指令的vstart是按字节

为单位的。

`vlm.v`和`vsm.v`会遵循`vtype`中的`vill`字段，因为它们通过`vtype`间接依赖于`vl`的约束。



之前的汇编助记符`vlel.v`和`vsel.v`由于处理长度的方式与其他元素加载/存储指令不同，容易引起混淆。为了避免软件更新带来的麻烦，这些旧的汇编助记符被保留为别名。

提供掩码加载和存储的主要动机是支持那些内部重新排列数据以减少跨数据通路布线的机器。然而，这些指令也为在内存中使用打包的位向量作为掩码值提供了便捷的机制，并通过减少对`vl`的修改，降低了掩码溢出/填充的成本。

31.7.5 向量步长指令

向量步长加载和存储

```
# vd 为目标寄存器, rs1 为基地址, rs2 为字节步长
vlse8.v    vd, (rs1), rs2, vm # 8 位步长加载
vlse16.v   vd, (rs1), rs2, vm # 16 位步长加载
vlse32.v   vd, (rs1), rs2, vm # 32 位步长加载
vlse64.v   vd, (rs1), rs2, vm # 64 位步长加载
```

```
# vs3 为存储数据, rs1 为基地址, rs2 为字节步长
vsse8.v    vs3, (rs1), rs2, vm # 8 位步长存储
vsse16.v   vs3, (rs1), rs2, vm # 16 位步长存储
vsse32.v   vs3, (rs1), rs2, vm # 32 位步长存储
vsse64.v   vs3, (rs1), rs2, vm # 64 位步长存储
```

支持负步长和零步长。在步长指令中，元素访问是无序的，元素之间没有顺序关系。当 `rs2=x0` 时，允许但不要求实现执行的内存操作次数少于激活元素的数量，并且可能在同一静态指令的不同动态执行中执行不同数量的内存操作。



编译器必须注意，当立即步幅为0且意图要求执行所有内存访问时，不能使用`x0`形式作为 `rs2`。

且 $x[rs2] = 0$ 时，实施必须为每个激活元素执行一次内存访问（但这些访问是无序的）。



与其他架构规定一样，实施必须看起来进行了每一次内存访问。微架构可以优化掉那些不会被其他代理观察到的访问，例如，在遵守RVWMO的幂等内存区域中。对于非幂等内存区域，由于定义上每次访问都可以被设备观察到，因此无法进行这种优化。

当需要对同一内存地址进行重复有序的向量访问时，可以使用有序索引操作。

31.7.6 向量索引指令

向量索引加载和存储

向量索引-无序加载指令

vd 目标寄存器, rs1 基地址, vs2 字节偏移

vluxei8.v vd, (rs1), vs2, vm # 无序 8 位索引加载 SEW 数据

vluxei16.v vd, (rs1), vs2, vm # 无序 16 位索引加载 SEW 数据

vluxei32.v vd, (rs1), vs2, vm # 无序 32 位索引加载 SEW 数据

vluxei64.v vd, (rs1), vs2, vm # 无序 64 位索引加载 SEW 数据

向量索引-有序加载指令

vd 目标寄存器, rs1 基地址, vs2 字节偏移

vloxei8.v vd, (rs1), vs2, vm # 有序 8 位索引加载 SEW 数据

vloxei16.v vd, (rs1), vs2, vm # 有序 16 位索引加载 SEW 数据

vloxei32.v vd, (rs1), vs2, vm # 有序 32 位索引加载 SEW 数据

vloxei64.v vd, (rs1), vs2, vm # 有序 64 位索引加载 SEW 数据

向量索引-无序存储指令

vs3 存储数据, rs1 基地址, vs2 字节偏移

vsuxei8.v vs3, (rs1), vs2, vm # 无序 8 位索引存储 SEW 数据

vsuxei16.v vs3, (rs1), vs2, vm # 无序 16 位索引存储 SEW 数据

vsuxei32.v vs3, (rs1), vs2, vm # 无序 32 位索引存储 SEW 数据

vsuxei64.v vs3, (rs1), vs2, vm # 无序 64 位索引存储 SEW 数据

向量索引-有序存储指令

vs3 存储数据, rs1 基地址, vs2 字节偏移

vsoxei8.v vs3, (rs1), vs2, vm # 有序 8 位索引存储 SEW 数据

vsoxei16.v vs3, (rs1), vs2, vm # 有序 16 位索引存储 SEW 数据

vsoxei32.v vs3, (rs1), vs2, vm # 有序 32 位索引存储 SEW 数据

vsoxei64.v vs3, (rs1), vs2, vm # 有序 64 位索引存储 SEW 数据

索引加载和存储的汇编语法使用 `eix` 而不是 `ex` 以表示静态编码的 EEW 是索引的，而不是数据的。



索引操作的助记符使用了 `U` 或 `O` 来区分无序和有序，而其他向量寻址模式没有这个字符。虽然这种做法可能稍显不一致，但这种方式最小化了对现有软件的干扰，因为之前的 `VSXEI` 表示有序——在过渡期间，可以将操作码保留为别名，帮助减少软件的变动。

31.7.7 单位步幅仅首次故障加载 (Fault-Only-First Loads)

单位步幅仅首次故障加载指令用于向量化具有数据依赖退出条件的循环 (while 循环)。这些指令作为常规加载执行, 唯一的不同是它们只会在元素 0 上遇到同步异常时才会触发陷阱。如果元素 0 引发异常, `v1` 不会修改, 并且陷阱会被触发。如果元素 > 0 引发异常, 则不会触发相应的陷阱, 且向量长度 `v1` 会减少到本应引发异常的元素的索引。

加载指令可能会覆盖在报告陷阱的元素索引之后的活跃目标向量寄存器组元素。类似地, 故障只对首个元素生效的加载指令也可能更新超出导致向量长度裁剪的元素的目标元素 (但不会超过初始向量长度)。这些虚假更新的值不必与内存中对应地址位置的值一致。非幂等内存位置只能在知道相应的元素加载操作不会因陷阱或向量长度裁剪而重新启动时访问。

向量单位步长故障只对首个元素生效的加载指令

`vd` 目标寄存器, `rs1` 基地址, `vm` 是掩码编码 (`v0.t` 或 `<缺失>`)

`vle8ff.v` `vd, (rs1), vm` # 8 位单位步长故障只对首个元素生效的加载

`vle16ff.v` `vd, (rs1), vm` # 16 位单位步长故障只对首个元素生效的加载

`vle32ff.v` `vd, (rs1), vm` # 32 位单位步长故障只对首个元素生效的加载

`vle64ff.v` `vd, (rs1), vm` # 64 位单位步长故障只对首个元素生效的加载



故障只对首个元素生效的加载指令存在安全隐患, 因为它们可以用来探测有效的地址。单位步长版本只允许探测与已知区域相邻的区域, 因此在非特权代码中使用, 减少了安全风险。然而, 运行在 S 模式的代码可以建立任意的页翻译, 从而允许探测由虚拟机监控程序提供的随机访客物理地址。由于编码空间的限制, 幅和散射/聚集故障仅首次指令未被提供, 但它们也可能代表一个更大的安全漏洞, 允许即使是未特权软件也能轻松检查多个随机页面的可访问性, 而不会触发陷阱。本标准未涉及故障仅首次指令可能的安全缓解措施。

即使未引发异常, 实施仍然可以处理少于 `v1` 个元素, 并相应地减少 `v1`, 但如果 `vstart=0` 且 `v1>0`, 则必须处理至少一个元素。

当故障只对首个元素生效的指令由于中断而触发陷阱时, 实施不应减少 `v1`, 而应设置 `vstart` 值。



当故障只对首个元素生效的指令在处理第一个元素之后触发调试数据观察点陷阱时, 实施不应减少 `v1`, 而应触发调试陷阱, 否则该事件可能会丢失。

31.7.8 向量加载/存储分段指令

向量加载/存储分段指令用于在多个连续编号的向量寄存器之间移动内存中的多个连续字段。



分段这个名称表明移动的项是包含同类元素的子数组。这些操作可以用于在内存和寄存器之间转置数组, 也可以通过将结构体中的每个字段解包到一个单独的向量寄存器中, 支持对结构体数组数据类型的操作。

向量指令编码中的三位 `nf` 字段是一个无符号整数, 包含每个分段中字段数减去 1 的值, 表示字段的数量 (即 `NFIELDS`)。

表31. 11 NFIELDS编码

nf[2:0]			NFIELDS
0	0	0	1
0	0	1	2
0	1	0	3
0	1	1	4
1	0	0	5
1	0	1	6
1	1	0	7
1	1	1	8

EMUL 设置必须满足 $EMUL * NFIELDS \leq 8$ ，否则该指令编码是被保留的。



$\text{ceil}(EMUL) * NFIELDS$ 的乘积表示将由分段加载或存储指令访问的底层向量寄存器的数量。这个约束确保总数不超过架构寄存器文件的1/4，并且与常规操作（ $EMUL=8$ ）一致。

每个字段将被保存在依次编号的向量寄存器组中。当 $EMUL > 1$ 时，每个字段将占据多个依次编号的向量寄存器组，并且每个字段的向量寄存器组必须遵循常规的向量寄存器对齐约束（例如，当 $EMUL=2$ 且 $NFIELDS=4$ 时，每个字段的向量寄存器组必须从偶数编号的向量寄存器开始，但不必从 8 的倍数开始）。如果分段加载或存储指令访问的向量寄存器编号超过 31，则指令编码将被保留。



这个约束有助于为可能的未来更长指令编码提供向前兼容性，这种编码可以访问更多的向量寄存器。

v1 寄存器给出了要移动的分段数，该数值等于每个向量寄存器组传输的元素数。掩码也应用于整个分段级别。

对于分段加载和存储，在每个分段内访问字段的单独内存访问是无序的，即使是有序的索引分段加载和存储指令也是如此。

vstart 值以整个分段为单位。如果在访问分段期间发生陷阱，是否执行分段内某些故障字段的访问（在触发陷阱之前）由实现定义。

向量单位步长加载和存储分段指令将打包的连续分段移动到多个目标向量寄存器组中。



当分段包含具有异质大小字段的结构时，软件可以在分段加载将数据带入向量寄存器后，使用额外的指令拆包各个结构字段。

汇编器前缀 vlseg/vsseg 分别用于单位步长分段加载和存储。

格式

vlseg<nf>e<eew>.v vd, (rs1), vm # 单位步长分段加载模板

vsseg<nf>e<eew>.v vs3, (rs1), vm # 单位步长分段存储模板

示例

vlseg8e8.v vd, (rs1), vm # 使用八字节字段加载八个向量寄存器。

vsseg3e32.v vs3, (rs1), vm # 从内存中将 3*4 字节的打包向量 (vs3, vs3+1, vs3+2) 存储到内存

对于加载操作，vd 寄存器将保存从分段中加载的第一个字段。对于存储操作，vs3 寄存器被读取，用于提供每个分段要存储的第一个字段。

示例 1

内存结构保存打包的 RGB 像素 (24 位数据结构, 8bpp)

vsetvli a1, t0, e8, ta, ma

vlseg3e8.v v8, (a0), vm

v8 保存红色像素

v9 保存绿色像素

v10 保存蓝色像素

示例 2

内存结构保存复数值, 32 位为实部, 32 位为虚部

vsetvli a1, t0, e32, ta, ma

vlseg2e32.v v8, (a0), vm

v8 保存实部

v9 保存虚部

还存在单位步长指令的故障只对首个元素生效版本。

向量故障只对首个元素生效的单位步长分段加载模板

vlseg<nf>e<eew>ff.v vd, (rs1), vm # 单位步长故障只对首个元素生效的分段加载

对于故障只对首个元素生效的分段加载，如果在访问分段过程中检测到异常，无论元素索引是否为零，都由实现定义是否加载分段的子集。

这些指令可能会覆盖在报告陷阱或向量长度裁剪点之后的目标向量寄存器组元素。

向量跨步分段加载和存储

向量跨步分段加载和存储将连续的分段移动，其中每个分段由 rs2 寄存器指定的字节跨步偏移量分隔。



支持负步长和零步长。

```
# 格式
vlsseg<nf>e<eew>.v vd, (rs1), rs2, vm      # 跨步分段加载
vssseg<nf>e<eew>.v vs3, (rs1), rs2, vm      # 跨步分段存储

# 示例
vsetvli a1, t0, e8, ta, ma
vlsseg3e8.v v4, (x5), x6      # 将地址 x5+i*x6 处的字节加载到 v4[i],
                                # 并将地址 x5+i*x6+1 处的字节加载到 v5[i],
                                # 将地址 x5+i*x6+2 处的字节加载到 v6[i]。

# 示例
vsetvli a1, t0, e32, ta, ma
vssseg2e32.v v2, (x5), x6     # 将 v2[i] 中的字存储到地址 x5+i*x6,
                                # 并将 v3[i] 中的字存储到地址 x5+i*x6+4
```

每个分段内字段的访问顺序可以是任意的，包括字节跨步导致分段在内存中重叠的情况。

向量索引分段加载和存储

向量索引分段加载和存储将连续的分段移动，每个分段的位置由将标量基地址（rs1 字段中的地址）与向量寄存器 vs2 中的字节偏移量相加来确定。提供了有序和无序两种形式，其中有序形式按照元素顺序访问分段。然而，即使是有序形式，对于单个分段内字段的访问也是无序的。

数据向量寄存器组的 EEW=SEW，EMUL=LMUL，而索引向量寄存器组的 EEW 在指令中编码， $EMUL = (EEW/SEW) * LMUL$ 。EMUL * NFIELD {1e} 8 这一约束适用于数据向量寄存器组。

```
# 格式
vluxseg<nf>ei<eew>.v vd, (rs1), vs2, vm    # 索引无序分段加载
vloxsseg<nf>ei<eew>.v vd, (rs1), vs2, vm    # 索引有序分段加载
vsuxseg<nf>ei<eew>.v vs3, (rs1), vs2, vm    # 索引无序分段存储
vsoxsseg<nf>ei<eew>.v vs3, (rs1), vs2, vm    # 索引有序分段存储

# 示例
vsetvli a1, t0, e8, ta, ma
vluxseg3ei8.v v4, (x5), v3      # 将地址 x5+v3[i] 处的字节加载到 v4[i],
                                # 并将地址 x5+v3[i]+1 处的字节加载到 v5[i],
                                # 将地址 x5+v3[i]+2 处的字节加载到 v6[i]。

# 示例
vsetvli a1, t0, e32, ta, ma
vsuxseg2ei32.v v2, (x5), v5     # 将 v2[i] 中的字存储到地址 x5+v5[i],
                                # 并将 v3[i] 中的字存储到地址 x5+v5[i]+4
```

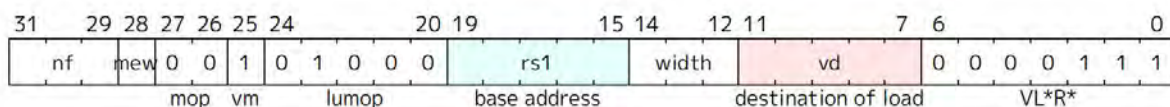
对于向量索引分段加载，目标向量寄存器组不能与源向量寄存器组（由 vs2 指定）重叠，否则指令编码将被保留。



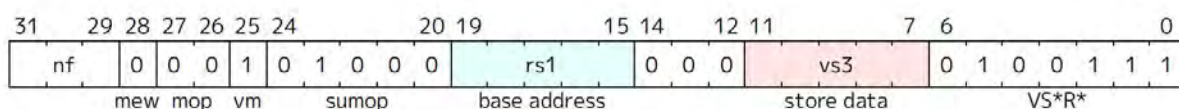
这个约束支持重新启动索引分段加载，当加载结构体的过程中部分发生异常时，能够重新开始加载。

31.7.9 向量加载/存储全寄存器指令

在 LOAD-FP 主要操作码下，向量加载整个寄存器指令的格式



在 STORE-FP 主要操作码下，向量存储全寄存器指令的格式



这些指令的设计目的是在当前向量寄存器的内容类型或长度未知，或者修改 vl 和 vtype 会非常耗时的情况下，保存和恢复向量寄存器。例如，这些情况包括编译器寄存器溢出、通过向量寄存器传递值的向量函数调用、处理中断的中断处理程序和操作系统上下文切换等。软件可以通过读取 vlenb 寄存器来确定传输的字节数。

加载指令的 EEW 编码位于 mew 和 width 字段中，遵循常规单元步长加载的模式。



由于寄存器内字节布局与内存中的字节布局相同，因此无论 EEW 是什么，写入目标寄存器组的数据都是相同的。因此，实际上只提供 EEW=8 的变体就足够了。提供完整的 EEW 变体是为了使编码的 EEW 可以作为一个提示，指示目标寄存器组接下来将使用这个 EEW 进行访问，这有助于重新排列数据的实现。

向量全寄存器存储指令的编码类似于未屏蔽的单元步长元素存储，EEW=8。



nf 字段的含义是使用 NFIELDs 编码加载和存储多少个向量寄存器（见表 31.11）。编码的寄存器数目必须是 2 的幂，且向量寄存器编号必须像向量寄存器组那样对齐，否则该指令编码是保留的。NFIELDs 表示要传输的向量寄存器的数量，这些寄存器按顺序编号，从基地址开始。仅支持 NFIELDs 值为 1、2、4、8，其他值是保留的。当传输多个寄存器时，编号最小的向量寄存器将存放在内存地址中最小的地方，随后编号的向量寄存器依次排列在内存中。

这些指令使用有效向量长度 $evl = NFIELDs * VLEN / EEW$ 进行操作，而不受当前 vtype 和 vl 设置的影响。通常的特性是，如果 $vstart \geq vl$ 则不写入任何元素，但对于这些指令，该属性不适用。相

反，只有当 $vstart \geq evl$ 时，才不会写入任何元素。

这些指令的操作方式类似于未屏蔽的单元步长加载和存储指令，基地址通过由 $rs1$ 指定的标量 x 寄存器传递。

如果基地址没有按自然对齐到编码的 EEW 字节大小 ($EEW/8$) 或实现支持的最小 SEW 字节大小 ($SEW_{MIN}/8$)，实现可以允许在整个寄存器加载和存储时触发地址不对齐异常。



允许根据与编码的 EEW 不对齐来抛出不对齐异常，可以简化这些指令的实现。一些子集实现可能不支持较小的 SEW 宽度，因此即使大于编码的 EEW ，也允许报告不对齐异常。极端的非标准实现可能有 $SEW_{MIN} > XLEN$ 。例如，软件环境可以要求最低对齐要求，以支持 ABI。

整个寄存器加载和存储指令的格式 $vllr.v\ v3, (a0)$ # 等效于 $vllre8.v$ 的伪指令

$vllre8.v\ v3, (a0)$ # 将 $v3$ 加载为位于 $a0$ 地址处的 $VLEN/8$ 字节数据
 $vllre16.v\ v3, (a0)$ # 将 $v3$ 加载为位于 $a0$ 地址处的 $VLEN/16$ 半字数据
 $vllre32.v\ v3, (a0)$ # 将 $v3$ 加载为位于 $a0$ 地址处的 $VLEN/32$ 字数据
 $vllre64.v\ v3, (a0)$ # 将 $v3$ 加载为位于 $a0$ 地址处的 $VLEN/64$ 双字数据

$vl2r.v\ v2, (a0)$ # 等效于 $vl2re8.v$ 的伪指令

$vl2re8.v\ v2, (a0)$ # 将 $v2-v3$ 加载为位于 $a0$ 地址处的 $2VLEN/8$ 字节数据
 $vl2re16.v\ v2, (a0)$ # 将 $v2-v3$ 加载为位于 $a0$ 地址处的 $2VLEN/16$ 半字数据
 $vl2re32.v\ v2, (a0)$ # 将 $v2-v3$ 加载为位于 $a0$ 地址处的 $2VLEN/32$ 字数据
 $vl2re64.v\ v2, (a0)$ # 将 $v2-v3$ 加载为位于 $a0$ 地址处的 $2VLEN/64$ 双字数据

$vl4r.v\ v4, (a0)$ # 等效于 $vl4re8.v$ 的伪指令

$vl4re8.v\ v4, (a0)$ # 将 $v4-v7$ 加载为位于 $a0$ 地址处的 $4VLEN/8$ 字节数据
 $vl4re16.v\ v4, (a0)$ # 将 $v4-v7$ 加载为位于 $a0$ 地址处的 $4VLEN/16$ 半字数据
 $vl4re32.v\ v4, (a0)$ # 将 $v4-v7$ 加载为位于 $a0$ 地址处的 $4VLEN/32$ 字数据
 $vl4re64.v\ v4, (a0)$ # 将 $v4-v7$ 加载为位于 $a0$ 地址处的 $4VLEN/64$ 双字数据

$vl8r.v\ v8, (a0)$ # 等效于 $vl8re8.v$ 的伪指令

$vl8re8.v\ v8, (a0)$ # 将 $v8-v15$ 加载为位于 $a0$ 地址处的 $8VLEN/8$ 字节数据
 $vl8re16.v\ v8, (a0)$ # 将 $v8-v15$ 加载为位于 $a0$ 地址处的 $8VLEN/16$ 半字数据
 $vl8re32.v\ v8, (a0)$ # 将 $v8-v15$ 加载为位于 $a0$ 地址处的 $8VLEN/32$ 字数据
 $vl8re64.v\ v8, (a0)$ # 将 $v8-v15$ 加载为位于 $a0$ 地址处的 $8VLEN/64$ 双字数据

$vs1r.v\ v3, (a1)$ # 将 $v3$ 存储到 $a1$ 地址处
 $vs2r.v\ v2, (a1)$ # 将 $v2-v3$ 存储到 $a1$ 地址处
 $vs4r.v\ v4, (a1)$ # 将 $v4-v7$ 存储到 $a1$ 地址处
 $vs8r.v\ v8, (a1)$ # 将 $v8-v15$ 存储到 $a1$ 地址处

实现应当在不支持的 EEW 值下，对于 `vl<nf>r` 指令触发非法指令异常。



我们曾考虑添加一个整个寄存器掩码加载指令 (`vllrm.v`)，但决定在初始扩展中省略该指令。其主要目的是通知微架构数据将作为掩码使用。可以通过以下代码序列实现相同的效果，其成本最多为四条指令。在这些指令中，第一条指令可能不需要，因为 `v1` 通常已经在标量寄存器中，最后一条指令在接下来的向量指令需要新的 SEW/LMUL 的情况下可能已经存在。因此，在最佳情况下，只需要两条指令（其中只有一条执行向量操作）即可合成专用指令的效果：

```
csrr t0, v1                # 保存当前的 v1（可能不需要）
vsetvli t1, x0, e8, m8, ta, ma # 设置最大 VLMAX
vlm.v v0, (a0)             # 加载掩码寄存器
vsetvli x0, t0, <new type>    # 恢复 v1（可能已经存在）
```

31.8 向量内存对齐约束

如果向量内存指令访问的元素没有按元素的自然大小对齐，元素要么成功传输，要么在该元素上触发地址未对齐异常。对未对齐的向量内存访问的支持独立于实现对未对齐标量内存访问的支持。



一个实现可以没有、一个，或者两个标量和向量内存访问，硬件支持部分或全部的未对齐访问。应该定义一个单独的 PMA 来确定是否在相关地址范围内支持向量未对齐访问。

向量未对齐的内存访问遵循与标量未对齐内存访问相同的原子性规则。

31.9 向量内存一致性模型

对外部观察者而言，向量内存指令在本地硬件线程上按程序顺序执行。向量内存指令在指令级别遵循 RVWMO。如果实现了 Ztso 扩展，向量内存指令还会在指令级别遵循 RVTSO。除向量索引有序加载和存储外，元素操作在指令内是无序的。向量索引有序的加载和存储按元素顺序从内存中加载和写入元素，遵循 RVWMO 在元素级别的规则。



Ztso 仅在指令级别施加 RVTSO；无论 Ztso 是否实现，指令内的顺序遵循 RVWMO。



需要更多的正式定义。

受向量长度寄存器 `v1` 影响的指令对 `v1` 具有控制依赖关系，而不是数据依赖关系。类似地，受掩码

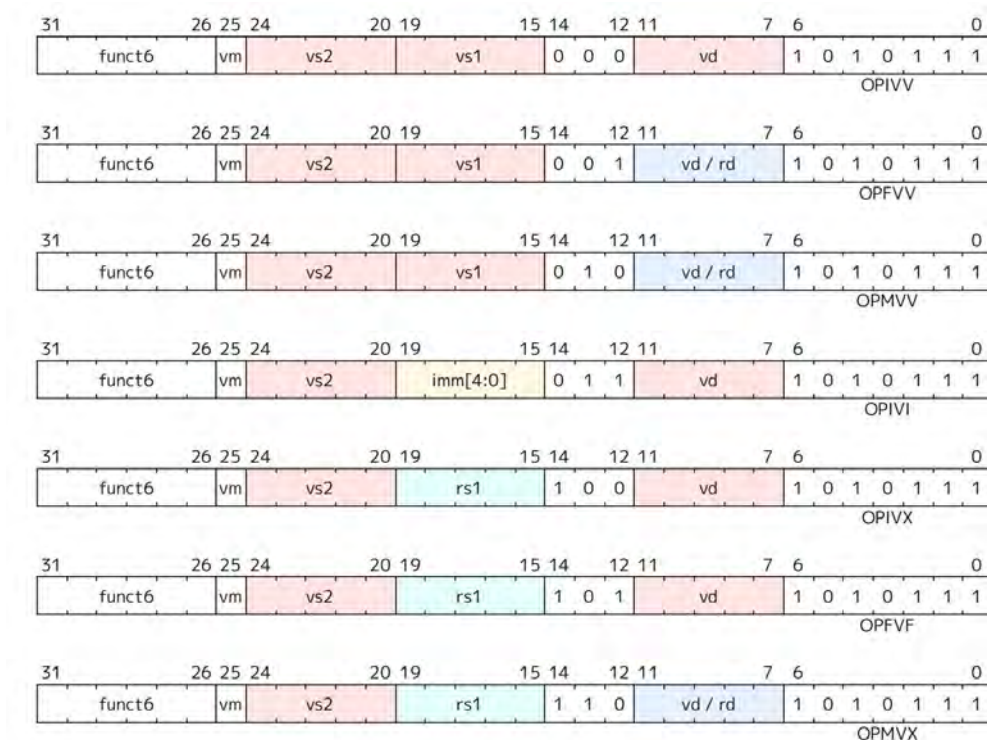
影响的向量指令对源掩码寄存器有控制依赖关系，而不是数据依赖关系。



将向量长度和掩码视为控制而非数据，通常与相应的标量代码的语义一致，在标量代码中通常会使用分支指令。将掩码视为控制使得掩码向量加载指令能够在掩码值尚未确定之前访问内存，无需错失恢复机制。

31.10 向量算术指令格式

向量算术指令使用一个新的主操作码 (OP-V = 1010111₂)，该操作码与 OP-FP 相邻。三位的 funct3 字段用于定义向量指令的子类别。



31.10.1 向量算术指令编码

funct3 字段编码了操作数类型和源位置。

表31.12 funct3

funct3[2:0]			种类	操作数	标量操作数的类型
0	0	0	OPIVV	向量-向量	N/A
0	0	1	OPFVV	向量-向量	N/A
0	1	0	OPMVV	向量-向量	N/A
0	1	1	OPIVI	向量-立即数	imm[4:0]
1	0	0	OPIVX	向量-标量	GPR x register rs1
1	0	1	OPFVF	向量-标量	FP f register rs1
1	1	0	OPMVX	向量-标量	GPR x register r

					sl
1	1	1	OPCFG	标量-立即数	GPR x register r sl & rs2/imm

整数操作使用无符号或二进制补码的有符号整数算术进行，具体取决于操作码。



在本讨论中，定点运算被视为整数运算。

所有标准的向量浮点算术操作遵循 IEEE-754/2008 标准。所有向量浮点操作使用 frm 寄存器中的动态舍入模式。如果 frm 字段包含无效的舍入模式，任何向量浮点指令（即使是那些不依赖于舍入模式的指令，或者当 vl=0，或者当 vstart $\geq$ vl 时）都会保留。



所有向量浮点代码将依赖于 frm 中的有效值。实现可以使所有向量浮点指令在舍入模式无效时报告异常，以简化控制逻辑。

向量-向量操作从分别由 vs2 和 vs1 指定的向量寄存器组中获取两个操作数。向量-标量操作可以有三种可能的形式。在这三种形式中，向量寄存器组操作数由 vs2 指定。第二个标量源操作数来自以下三种选择之一：

- 对于整数操作，标量可以是编码在 rs1 字段中的 5 位立即数 imm[4:0]，该值将符号扩展到 SEW 位，除非另有说明。
- 对于整数操作，标量可以来自由 rs1 指定的标量 x 寄存器。如果 XLEN > SEW，则使用 x 寄存器的最低 SEW 位，除非另有说明。如果 XLEN < SEW，则将 x 寄存器的值符号扩展到 SEW 位。
- 对于浮点操作，标量可以来自标量 f 寄存器。如果 FLEN > SEW，则会检查 f 寄存器中的值是否为有效的 NaN 值，如果是，则使用 f 寄存器的最低 SEW 位，否则使用标准的 NaN 值。如果任何浮点向量操作数的 EEW 不是支持的浮点类型宽度（包括当 FLEN < SEW 时），则保留该指令。



某些指令对 5 位立即数进行零扩展，并在汇编语法中将此立即数称为 uimm。



当向 Zfinx/Zdinx/Zhinx 扩展中添加向量扩展时，浮点标量参数来自 x 寄存器。这些扩展不支持 NaN-装箱，因此，向量浮点标量值的生成规则与整数标量操作数相同（即当 XLEN > SEW 时，使用最低 SEW 位；当 XLEN < SEW 时，使用符号扩展的值）。

向量算术指令受 vm 字段的控制进行掩码操作。

向量二元算术指令的汇编语法模式

返回向量结果的操作，按 *vm* 掩码 (*v0.t*, <无>)

```
vop.vv  vd, vs2, vs1, vm # 整数向量-向量      vd[i] = vs2[i] op vs1[i]
vop.vx  vd, vs2, rs1, vm # 整数向量-标量      vd[i] = vs2[i] op x[rs1]
vop.vi  vd, vs2, imm, vm # 整数向量-立即数    vd[i] = vs2[i] op imm

vfop.vv vd, vs2, vs1, vm # 浮点向量-向量操作  vd[i] = vs2[i] fop vs1[i]
vfop.vf vd, vs2, rs1, vm # 浮点向量-标量操作  vd[i] = vs2[i] fop f[rs1]
```



在编码中，*vs2*是第一个操作数，而*rs1/imm*是第二个操作数。这与标准的标量操作顺序相反。此排列保留了现有的编码约定：只读取一个标量寄存器的指令从*rs1*中读取它，5 位立即数来自*rs1*字段。

向量三元算术指令的汇编语法模式（乘加）

覆盖求和输入的整数操作

```
vop.vv vd, vs1, vs2, vm # vd[i] = vs1[i] * vs2[i] + vd[i]
vop.vx vd, rs1, vs2, vm # vd[i] = x[rs1] * vs2[i] + vd[i]
```

覆盖乘积输入的整数操作

```
vop.vv vd, vs1, vs2, vm # vd[i] = vs1[i] * vd[i] + vs2[i]
vop.vx vd, rs1, vs2, vm # vd[i] = x[rs1] * vd[i] + vs2[i]
```

覆盖求和输入的浮点操作

```
vfop.vv vd, vs1, vs2, vm # vd[i] = vs1[i] * vs2[i] + vd[i]
vfop.vf vd, rs1, vs2, vm # vd[i] = f[rs1] * vs2[i] + vd[i]
```

覆盖乘积输入的浮点操作

```
vfop.vv vd, vs1, vs2, vm # vd[i] = vs1[i] * vd[i] + vs2[i]
vfop.vf vd, rs1, vs2, vm # vd[i] = f[rs1] * vd[i] + vs2[i]
```



对于三元乘加操作，汇编语法总是首先放置目标向量寄存器，其次是 *rs1*或*vs1*，最后是*vs2*。这种顺序为这些三元操作提供了更自然的阅读方式，因为乘法操作数总是放在一起。

31.10.2 扩宽向量算术指令

一些向量算术指令被定义为扩宽操作，其中目标向量寄存器组的 $EEW = 2SEW$ 且 $EMUL = 2LMUL$ 。这些指令通常会在操作码前加上 **vw* 前缀，或者对于向量浮点指令使用 **vfw* 前缀。第一个向量寄存器组操作数可以是单宽度或双宽度。

向量扩宽算术指令的汇编语法模式

```
# 双宽度结果，两个单宽度源：2*SEW = SEW op SEW
vwop.vv vd, vs2, vs1, vm # 整数向量-向量    vd[i] = vs2[i] op vs1[i]
vwop.vx vd, vs2, rs1, vm # 整数向量-标量    vd[i] = vs2[i] op x[rs1]

# 双宽度结果，第一个源为双宽度，第二个源为单宽度：2SEW = 2SEW op SEW
vwop.wv vd, vs2, vs1, vm # 整数向量-向量    vd[i] = vs2[i] op vs1[i]
vwop.wx vd, vs2, rs1, vm # 整数向量-标量    vd[i] = vs2[i] op x[rs1]
```



最初操作码上使用了 `w` 后缀，但由于它可能与在双字整数中表示字操作的 `w` 后缀混淆，因此将 `w` 移动到了前缀。

浮点扩宽操作已更改为 `vfw`，而不是原来的 `vwf*`，以便与将来可能写成 `fw*` 的标量扩宽浮点操作保持一致。

扩宽指令的编码必须遵循第 31.5.2 的约束。

31.10.3 缩窄向量算术指令

提供了一些指令，用于将双宽度源向量转换为单宽度目标向量。这些指令将由 `vs2` 指定的向量寄存器组（`EEW/EMUL = 2SEW/2LMUL`）转换为具有当前 `SEW/LMUL` 设置的向量寄存器组。如果有第二个源向量寄存器组（由 `vs1` 指定），则它的宽度与结果相同（即 `EEW = SEW`）。



另一种设计选择是将 `SEW/LMUL` 视为定义源向量寄存器组大小的标准。这里选择的方法是基于认为这种方法会减少 `vtype` 更改的需求。

设置掩码寄存器的比较操作也隐式地执行缩窄操作。

操作码上使用 `vn*` 前缀来区分这些指令，或者对于缩窄的浮点操作码，使用 `vfn*` 前缀。双宽度源向量寄存器组通过源操作数后缀中的 `w` 来表示（例如，`vnsra.wv`）。

向量缩窄算术指令的汇编语法模式

```
# 单宽度结果，双宽度源 vs2，单宽度源 vs1/rs1
# SEW = 2*SEW op SEW
vnop.wv vd, vs2, vs1, vm # 整数向量-向量    vd[i] = vs2[i] op vs1[i]
vnop.wx vd, vs2, rs1, vm # 整数向量-标量    vd[i] = vs2[i] op x[rs1]
```

缩窄指令的编码必须遵循第 31.5.2 节的约束。

31.11 向量整数算术指令

提供了一组向量整数算术指令。除非另有说明，整数操作在溢出时会回绕。

31.11.1 向量单宽度整数加法与减法

提供了向量整数加法和减法指令。对于向量-标量形式，也提供了反向减法指令。

整数加法

```
vadd.vv vd, vs2, vs1, vm # 向量-向量
vadd.vx vd, vs2, rs1, vm # 向量-标量
vadd.vi vd, vs2, imm, vm # 向量-立即数
```

整数减法

```
vsub.vv vd, vs2, vs1, vm # 向量-向量
vsub.vx vd, vs2, rs1, vm # 向量-标量
```

整数反向减法

```
vrsb.vx vd, vs2, rs1, vm # vd[i] = x[rs1] - vs2[i]
vrsb.vi vd, vs2, imm, vm # vd[i] = imm - vs2[i]
```



可以使用反向减法指令与标量操作数x0来对整数值取反。提供了汇编伪指令vneg.v vd, vs =vrsb.vx vd, vs, x0。

31.11.2 向量扩宽整数加法/减法

扩宽加法/减法指令提供了有符号和无符号两种变体，具体取决于在形成双宽和之前，较窄的源操作数是否首先进行符号扩展或零扩展。

无符号扩宽整数加法/减法， $2*SEW = SEW +/ - SEW$

```
vwaddu.vv vd, vs2, vs1, vm # 向量-向量
vwaddu.vx vd, vs2, rs1, vm # 向量-标量
vwsubu.vv vd, vs2, vs1, vm # 向量-向量
vwsubu.vx vd, vs2, rs1, vm # 向量-标量
```

有符号扩宽整数加法/减法， $2*SEW = SEW +/ - SEW$

```
vwadd.vv vd, vs2, vs1, vm # 向量-向量
vwadd.vx vd, vs2, rs1, vm # 向量-标量
vwsub.vv vd, vs2, vs1, vm # 向量-向量
vwsub.vx vd, vs2, rs1, vm # 向量-标量
```

无符号扩宽整数加法/减法， $2SEW = 2SEW +/ - SEW$

```
vwaddu.wv vd, vs2, vs1, vm # 向量-向量
vwaddu.wx vd, vs2, rs1, vm # 向量-标量
vwsubu.wv vd, vs2, vs1, vm # 向量-向量
vwsubu.wx vd, vs2, rs1, vm # 向量-标量
```

```
# 有符号扩宽整数加法/减法, 2SEW = 2SEW +/- SEW
vwadd.wv vd, vs2, vs1, vm # 向量-向量
vwadd.wx vd, vs2, rs1, vm # 向量-标量
vwsb.wv vd, vs2, vs1, vm # 向量-向量
vwsb.wx vd, vs2, rs1, vm # 向量-标量
```



可以使用扩宽加法指令与标量操作数x0将整数值宽度加倍。提供了汇编伪指令vwcvt. x. x. v vd, vs, vm=
vwadd.vx vd, vs, x0, vm和vwcvtu. x. x. v vd, vs, vm =vwaddu.vx vd, vs, x0, vm。

31.11.3 向量整数扩展

向量整数扩展指令通过零扩展或符号扩展源向量整数操作数 (EEW 小于 SEW) 以填充目标中大小为 SEW 的元素。源的 EEW 是 SEW 的 1/2、1/4 或 1/8, 而源的 EMUL 为 (EEW/SEW)*LMUL。目标的 EEW 等于 SEW, EMUL 等于 LMUL。

```
vzext.vf2 vd, vs2, vm # 将 SEW/2 的源零扩展到 SEW 目标
vsxt.vf2 vd, vs2, vm # 将 SEW/2 的源符号扩展到 SEW 目标
vzext.vf4 vd, vs2, vm # 将 SEW/4 的源零扩展到 SEW 目标
vsxt.vf4 vd, vs2, vm # 将 SEW/4 的源符号扩展到 SEW 目标
vzext.vf8 vd, vs2, vm # 将 SEW/8 的源零扩展到 SEW 目标
vsxt.vf8 vd, vs2, vm # 将 SEW/8 的源符号扩展到 SEW 目标
```

如果源的 EEW 不支持, 或者源的 EMUL 小于最小合法的 LMUL, 则该指令编码被保留。



标准的向量加载指令访问的内存值与目标寄存器元素的大小相同。一些应用程序代码需要在更宽的元素范围内操作操作数的宽度, 例如, 从内存加载一个字节并加到一个八字节元素上。为了避免提供跨向量加载指令与数据类型 (字节、字、半字, 及其有符号/无符号变种) 数量的交叉积, 我们改为添加了显式的扩展指令, 如果没有可用的适当扩展算术指令, 则可以使用这些指令。

31.11.4 向量整数加法带进位 / 减法带借位指令

为了支持多字整数算术, 提供了处理进位的指令。对于每个操作 (加法或减法), 提供了两条指令: 一条提供结果 (SEW 宽度), 另一条生成进位输出 (单个位, 编码为掩码布尔值)。

进位输入和输出通过掩码寄存器布局表示, 如第 31.5.4 节所述。由于编码约束, 进位输入必须来自隐式的 v0 寄存器, 但进位输出可以写入任何遵守源/目标重叠限制的向量寄存器。

vadc 和 vsbc 分别将源操作数和进位输入或借位输入相加或相减, 并将结果写入向量寄存器 vd。这些指令被编码为掩码指令 (vm=0), 但它们作用于并写回所有主体元素。对应未掩码版本的编码 (vm=1) 被保留。

vmadc 和 vmsbc 分别将源操作数相加或相减, 若掩码 (vm=0), 还可选择加上进位输入或减去借位输入, 并将结果写回掩码寄存器 vd。如果未掩码 (vm=1), 则没有进位输入或借位输入。这些指令作用于并写回所有体元素, 即使被掩码。因为这些指令产生一个掩码值, 所以它们总是采用与尾部无关的

策略。

计算带进位的和

```
# vd[i] = vs2[i] + vs1[i] + v0.mask[i]
vadc.vvm    vd, vs2, vs1, v0 # 向量-向量
```

```
# vd[i] = vs2[i] + x[rs1] + v0.mask[i]
vadc.vxm    vd, vs2, rs1, v0 # 向量-标量
```

```
# vd[i] = vs2[i] + imm + v0.mask[i]
vadc.vim    vd, vs2, imm, v0 # 向量-立即数
```

计算掩码寄存器格式的进位输出

```
# vd.mask[i] = carryout(vs2[i] + vs1[i] + v0.mask[i])
vmadc.vvm   vd, vs2, vs1, v0 # 向量-向量
```

```
# vd.mask[i] = carryout(vs2[i] + x[rs1] + v0.mask[i])
vmadc.vxm   vd, vs2, rs1, v0 # 向量-标量
```

```
# vd.mask[i] = carryout(vs2[i] + imm + v0.mask[i])
vmadc.vim   vd, vs2, imm, v0 # 向量-立即数
```

```
# vd.mask[i] = carryout(vs2[i] + vs1[i])
vmadc.vv    vd, vs2, vs1      # 向量-向量, 无进位输入
```

```
# vd.mask[i] = carryout(vs2[i] + x[rs1])
vmadc.vx    vd, vs2, rs1      # 向量-标量, 无进位输入
```

```
# vd.mask[i] = carryout(vs2[i] + imm)
vmadc.vi    vd, vs2, imm      # 向量-立即数, 无进位输入
```

因为实现进位传播需要执行两条指令且输入不变,所以破坏性累加将需要额外的移动操作才能获得正确的结果。

```
# 示例: 多字算术序列, 将结果累加到 v4
vmadc.vvm v1, v4, v8, v0 # 将进位保存到临时寄存器 v1
vadc.vvm v4, v4, v8, v0  # 计算新的和
vmmv.m v0, v1           # 将临时进位移入 v0, 为下一个字准备
```

减法带借位指令 `vsbc` 执行等效操作, 以支持长字减法算术。没有带立即数的减法指令。

计算带借位的差值。

```

# vd[i] = vs2[i] - vs1[i] - v0.mask[i]
vsbc.vvm    vd, vs2, vs1, v0 # 向量-向量

# vd[i] = vs2[i] - x[rs1] - v0.mask[i]
vsbc.vxm    vd, vs2, rs1, v0 # 向量-标量

# Produce borrow out in mask register format

# vd.mask[i] = borrowout(vs2[i] - vs1[i] - v0.mask[i])
vmsbc.vvm   vd, vs2, vs1, v0 # 向量-向量

# vd.mask[i] = borrowout(vs2[i] - x[rs1] - v0.mask[i])
vmsbc.vxm   vd, vs2, rs1, v0 # 向量-标量

# vd.mask[i] = borrowout(vs2[i] - vs1[i])
vmsbc.vv    vd, vs2, vs1      # 向量-向量, 无借位

# vd.mask[i] = borrowout(vs2[i] - x[rs1])
vmsbc.vx    vd, vs2, rs1      # 向量-标量, 无借位

```

对于 vmsbc, 当差值 (截断前) 为负时, 借位被定义为 1。

对于 vadc 和 vsbc, 如果目标向量寄存器是 v0, 则指令编码被保留。



此约束与掩码向量操作的约束相对应, 掩码寄存器被重写。

31.11.5 向量按位逻辑操作指令

```

# 按位逻辑操作
vand.vv vd, vs2, vs1, vm # 向量-向量
vand.vx vd, vs2, rs1, vm # 向量-标量
vand.vi vd, vs2, imm, vm # 向量-立即数

vor.vv vd, vs2, vs1, vm # 向量-向量
vor.vx vd, vs2, rs1, vm # 向量-标量
vor.vi vd, vs2, imm, vm # 向量-立即数

vxor.vv vd, vs2, vs1, vm # 向量-向量
vxor.vx vd, vs2, rs1, vm # 向量-标量
vxor.vi vd, vs2, imm, vm # 向量-立即数

```




当立即数为 -1 时，`vxor` 指令的标量立即数形式提供按位取反操作。可以使用汇编伪指令 `vnot.v vd, vs, vm` 来表示，即 `vxor.vi vd, vs, -1, vm`。

31.11.6 向量单宽度移位指令

提供了一整套向量移位指令，包括逻辑左移（`sll`），逻辑右移（零扩展 `srl`）和算术右移（符号位扩展 `sra`）。需要移位的数据在由 `vs2` 指定的向量寄存器组中，移位量可以来自向量寄存器组 `vs1`、标量整数寄存器 `rs1`，或零扩展的 5 位立即数。只有移位量的低 $\lg 2(\text{SEW})$ 位用于控制移位量。

按位移位操作

```
vsll.vv vd, vs2, vs1, vm # 向量-向量
vsll.vx vd, vs2, rs1, vm # 向量-标量
vsll.vi vd, vs2, uimm, vm # 向量-立即数
```

```
vsrl.vv vd, vs2, vs1, vm # 向量-向量
vsrl.vx vd, vs2, rs1, vm # 向量-标量
vsrl.vi vd, vs2, uimm, vm # 向量-立即数
```

```
vsra.vv vd, vs2, vs1, vm # 向量-向量
vsra.vx vd, vs2, rs1, vm # 向量-标量
vsra.vi vd, vs2, uimm, vm # 向量-立即数
```

31.11.7 向量缩小整数右移指令

缩小右移操作从宽操作数中提取较小的字段，并具有零扩展（`srl`）和符号扩展（`sra`）形式。移位量可以来自向量寄存器组、标量 `x` 寄存器，或零扩展的 5 位立即数。移位量的低 $\lg 2(2*\text{SEW})$ 位用于控制移位量（例如，对于 `SEW=64` 位到 `SEW=32` 位的缩小操作，使用低 6 位）。

缩小右移逻辑操作， $\text{SEW} = (2*\text{SEW})$ `SEW`

```
vnsrl.wv vd, vs2, vs1, vm # 向量-向量
vnsrl.wx vd, vs2, rs1, vm # 向量-标量
vnsrl.wi vd, vs2, uimm, vm # 向量-立即数
```

缩小右移算术操作， $\text{SEW} = (2*\text{SEW})$ `SEW`

```
vnsra.wv vd, vs2, vs1, vm # 向量-向量
vnsra.wx vd, vs2, rs1, vm # 向量-标量
vnsra.wi vd, vs2, uimm, vm # 向量-立即数
```



未来扩展可能会添加支持将目标缩小到源的 1/4 宽度的版本。



可以使用带标量操作数x0的缩小整数移位指令将整数值宽度减半。提供了一个汇编伪指令 `vncvt.x.x.w vd,vs,vm`，等同于 `vnsrl.wx vd,vs,x0,vm`。

31.11.8 向量整数比较指令

以下整数比较指令会在比较为真时将 1 写入目标掩码寄存器元素，否则写入 0。目标掩码向量总是保存在单个向量寄存器中，元素布局如第 31.5.4 节所述。目标掩码向量寄存器可以与源向量掩码寄存器（v0）相同。

相等时置 1

```
vmseq.vv vd, vs2, vs1, vm # 向量-向量
vmseq.vx vd, vs2, rs1, vm # 向量-标量
vmseq.vi vd, vs2, imm, vm # 向量-立即数
```

不相等时置 1

```
vmsne.vv vd, vs2, vs1, vm # 向量-向量
vmsne.vx vd, vs2, rs1, vm # 向量-标量
vmsne.vi vd, vs2, imm, vm # 向量-立即数
```

小于时置 1，无符号

```
vmsltu.vv vd, vs2, vs1, vm # 向量-向量
vmsltu.vx vd, vs2, rs1, vm # 向量-标量
```

小于时置 1，有符号

```
vmslt.vv vd, vs2, vs1, vm # 向量-向量
vmslt.vx vd, vs2, rs1, vm # 向量-标量
```

小于或等于时置 1，无符号

```
vmsleu.vv vd, vs2, vs1, vm # 向量-向量
vmsleu.vx vd, vs2, rs1, vm # 向量-标量
vmsleu.vi vd, vs2, imm, vm # 向量-立即数
```

小于或等于时置 1，有符号

```
vmsle.vv vd, vs2, vs1, vm # 向量-向量
vmsle.vx vd, vs2, rs1, vm # 向量-标量
vmsle.vi vd, vs2, imm, vm # 向量-立即数
```

```
# 大于时置 1，无符号
vmsgtu.vx vd, vs2, rs1, vm # 向量-标量
vmsgtu.vi vd, vs2, imm, vm # 向量-立即数
```

```
# 大于时置 1，有符号
vmsgt.vx vd, vs2, rs1, vm # 向量-标量
vmsgt.vi vd, vs2, imm, vm # 向量-立即数
```

```
# 以下两条指令不直接提供
# 大于或等于时置 1，无符号
# vmsgtu.vx vd, vs2, rs1, vm # 向量-标量
# 大于或等于时置 1，有符号
# vmsgt.vx vd, vs2, rs1, vm # 向量-标量
```

以下表格展示了所有比较在本机机器代码中的实现方式。

比较	汇编映射	汇编伪指令
va < vb	vmslt{u}.vv vd, va, vb, vm	
va <= vb	vmsle{u}.vv vd, va, vb, vm	
va > vb	vmslt{u}.vv vd, vb, va, vm	vmsgt{u}.vv vd, va, vb, vm
va >= vb	vmsle{u}.vv vd, vb, va, vm	vmsge{u}.vv vd, va, vb, vm
va < x	vmslt{u}.vx vd, va, x, vm	
va <= x	vmsle{u}.vx vd, va, x, vm	
va > x	vmsgt{u}.vx vd, va, x, vm	
va >= x	see below	
va < i	vmsle{u}.vi vd, va, i-1, vm	vmslt{u}.vi vd, va, i, vm
va <= i	vmsle{u}.vi vd, va, i, vm	
va > i	vmsgt{u}.vi vd, va, i, vm	
va >= i	vmsgt{u}.vi vd, va, i-1, vm	vmsge{u}.vi vd, va, i, vm

va, vb 向量寄存器组
x 标量整数寄存器
i 立即数



vmslt{u}.vi 的立即数形式未提供，因为立即数可以减去 1，改用 vmsle{u}.vi 变体。vmsle.vi 的范围是 -16 到 15，因此有效的 vmslt.vi 范围是 -15 到 16。vmsleu.vi 的范围是 0 到 15，给出了有效的 vmsltu.vi 范围是 1 到 16（注意，vmsltu.vi 的立即数为 0 时没有用处，因为它总是为假）。



由于 5 位向量立即数总是符号位扩展的，当 `simm5` 立即数的高位被设置时，`vmsleu.vi` 也支持无符号立即数值，范围是 $2^{\text{SEW}}-16$ 到 $2^{\text{SEW}}-1$ ，因此相应的 `vmsltu.vi` 可以与范围 $2^{\text{SEW}}-15$ 到 2^{SEW} 的无符号立即数进行比较。注意，`vmsltu.vi` 使用立即数 2^{SEW} 是没有意义的，因为它总是为真。

类似地，`vmsge{u}.vi` 并没有被提供，这一比较通过使用 `vmsgt{u}.vi` 并将立即数减去 1 来实现。结果有效的 `vmsge.vi` 范围是 -15 到 16，结果有效的 `vmsgeu.vi` 范围是 1 到 16（注意，`vmsgeu.vi` 的立即数为 0 时没有用处，因为它总是为真）。



提供了寄存器标量和立即数形式的 `vmsgt`，允许通过单个比较指令提供正确的掩码值极性，而无需使用额外的掩码逻辑指令。

为了减少编码空间，`vmsge{u}.vx` 形式未直接提供，因此 $va \geq x$ 情况需要特殊处理。



`vmsge{u}.vx` 可能会通过在不使用的 `OPIVI` 变体下，使用 `vmslt{u}` 以非正交的方式进行编码。这些将是 `OPIVI` 中唯一使用标量 `x` 寄存器的指令。另一种方法是使用额外的两个 `funct6` 编码，但它们的操作数格式（写入掩码寄存器）与同一组 8 个 `funct6` 编码中的其他指令不同。目前的 PoR 是省略这些指令，并根据需要进行合成，如下所述。

`vmsge{u}.vx` 操作可以通过将 `x` 的值减去 1 并使用 `vmsgt{u}.vx` 指令来合成，当已知这不会导致 `x` 中的表示下溢时。

合成 `vmsge{u}.vx` 指令的序列：

```
va >= x,  x > minimum
addi t0, x, -1; vmsgt{u}.vx vd, va, t0, vm
```

上述序列通常是最有效的实现，但在 `x` 的范围未知的情况下，可以提供汇编伪指令。

未掩码 $va \geq x$

```
伪指令: vmsge{u}.vx vd, va, x
展开: vmslt{u}.vx vd, va, x; vmnand.mm vd, vd, vd
```

掩码 $va \geq x, vd \neq v0$

```
伪指令: vmsge{u}.vx vd, va, x, v0.t
展开: vmslt{u}.vx vd, va, x, v0.t; vmxor.mm vd, vd, v0
```

掩码 $va \geq x, vd == v0$

```
伪指令: vmsge{u}.vx vd, va, x, v0.t, vt
展开: vmslt{u}.vx vt, va, x; vmandn.mm vd, vd, vt
```

掩码 $va \geq x$, 任意 vd

伪指令: `vmsge{u}.vx vd, va, x, v0.t, vt`

展开: `vmslt{u}.vx vt, va, x; vmandn.mm vt, v0, vt; vmandn.mm vd, vd, v0; vmor.mm vd, vt, vd`

伪指令中的 `vt` 参数必须命名为一个临时向量寄存器, 该寄存器不同于 `vd`, 并且会被伪指令破坏。在掩码未干扰的策略下, 比较实际上是与掩码进行按位与操作, 如果目标寄存器是 `v0`, 例如:

$(a < b) \ \&\& \ (b < c)$ 在两条指令中进行, 当掩码未干扰时

`vmslt.vv v0, va, vb` # 所有元素写入

`vmslt.vv v0, vb, vc, v0.t` # 仅在设置掩码时更新

比较写入掩码寄存器, 因此始终在尾部无关的策略下操作。

31.11.9 向量整数最小值/最大值指令

支持有符号和无符号整数的最小值和最大值指令。

无符号最小值

`vminu.vv vd, vs2, vs1, vm` # 向量-向量

`vminu.vx vd, vs2, rs1, vm` # 向量-标量

有符号最小值

`vmin.vv vd, vs2, vs1, vm` # 向量-向量

`vmin.vx vd, vs2, rs1, vm` # 向量-标量

无符号最大值

`vmaxu.vv vd, vs2, vs1, vm` # 向量-向量

`vmaxu.vx vd, vs2, rs1, vm` # 向量-标量

有符号最大值

`vmax.vv vd, vs2, vs1, vm` # 向量-向量

`vmax.vx vd, vs2, rs1, vm` # 向量-标量

31.11.10 向量单宽度整数乘法指令

单宽度乘法指令执行 SEW 位 * SEW 位的乘法, 生成一个 $2*SEW$ 位的结果, 然后将乘积的一个部分返回到 SEW 位宽的目标寄存器。`*mul*`版本将乘积的低字写入目标寄存器, 而 `*mulh*`版本将乘积的高字写入目标寄存器。

有符号乘法, 返回乘积的低位

`vmul.vv vd, vs2, vs1, vm` # 向量-向量

`vmul.vx vd, vs2, rs1, vm` # 向量-标量

有符号乘法，返回乘积的高位
`vmulh.vv vd, vs2, vs1, vm` # 向量-向量
`vmulh.vx vd, vs2, rs1, vm` # 向量-标量

无符号乘法，返回乘积的高位
`vmulhu.vv vd, vs2, vs1, vm` # 向量-向量
`vmulhu.vx vd, vs2, rs1, vm` # 向量-标量

有符号 (vs2) - 无符号乘法，返回乘积的高位
`vmulhsu.vv vd, vs2, vs1, vm` # 向量-向量
`vmulhsu.vx vd, vs2, rs1, vm` # 向量-标量

没有 `vmulhus.vx` 操作码返回无符号向量 * 有符号标量乘积的高半部分。标量可以扩展为向量，然后使用 `vmulhsu.vv`。



当前的 `vmulh*` 操作码执行简单的分数乘法，但没有选项来缩放、舍入和/或饱和结果。未来的扩展可能会考虑 `vmulh`、`vmulhu`、`vmulhsu` 的变体，在丢弃乘积的低半部分时使用 `vxxrm` 舍入模式。在这些情况下，不会发生溢出。

31.11.11 向量整数除法指令

除法和余数指令与 RISC-V 标准标量整数乘法/除法等效，对于极端输入产生相同的结果。

无符号除法
`vdivu.vv vd, vs2, vs1, vm` # 向量-向量
`vdivu.vx vd, vs2, rs1, vm` # 向量-标量

有符号除法
`vdiv.vv vd, vs2, vs1, vm` # 向量-向量
`vdiv.vx vd, vs2, rs1, vm` # 向量-标量

无符号余数
`vremu.vv vd, vs2, vs1, vm` # 向量-向量
`vremu.vx vd, vs2, rs1, vm` # 向量-标量

有符号余数
`vrem.vv vd, vs2, vs1, vm` # 向量-向量
`vrem.vx vd, vs2, rs1, vm` # 向量-标量



是否包含整数除法和余数指令存在争议。支持这一观点的理由是，如果没有标准指令，软件将不得不选择某些算法来执行该操作，这可能在某些微架构上表现较差。

没有执行标量除法对向量操作的指令。

31.11.12 向量宽整数乘法指令

宽整数乘法指令返回一个 SEW 位 * SEW 位乘法的完整 2*SEW 位乘积。

宽度有符号整数乘法

`vwmul.vv vd, vs2, vs1, vm` # 向量-向量

`vwmul.vx vd, vs2, rs1, vm` # 向量-标量

宽度无符号整数乘法

`vwmulu.vv vd, vs2, vs1, vm` # 向量-向量

`vwmulu.vx vd, vs2, rs1, vm` # 向量-标量

宽度有符号(vs2)-无符号整数乘法

`vwmulsu.vv vd, vs2, vs1, vm` # 向量-向量

`vwmulsu.vx vd, vs2, rs1, vm` # 向量-标量

31.11.13 向量单宽度整数乘加指令

整数乘加指令是破坏性的，提供了两种形式，一种覆盖加数或减数（`vmacc`, `vnmsac`），另一种覆盖第一个乘数（`vmadd`, `vnmsub`）。乘积的低半部分会被加到或减去第三个操作数。



`sac` 应理解为从累加器中减去。操作码是 `vnmsac`，以匹配（不幸的是直觉上相反的）浮点 `fnmsub` 指令定义。`vnmsub` 操作码同样如此。

整数乘加，覆盖加数

`vmacc.vv vd, vs1, vs2, vm` # $vd[i] = +(vs1[i] * vs2[i]) + vd[i]$

`vmacc.vx vd, rs1, vs2, vm` # $vd[i] = +(x[rs1] * vs2[i]) + vd[i]$

整数乘减，覆盖减数

`vnmsac.vv vd, vs1, vs2, vm` # $vd[i] = -(vs1[i] * vs2[i]) + vd[i]$

`vnmsac.vx vd, rs1, vs2, vm` # $vd[i] = -(x[rs1] * vs2[i]) + vd[i]$

整数乘加，覆盖乘数

`vmadd.vv vd, vs1, vs2, vm` # $vd[i] = (vs1[i] * vd[i]) + vs2[i]$

`vmadd.vx vd, rs1, vs2, vm` # $vd[i] = (x[rs1] * vd[i]) + vs2[i]$

整数乘减，覆盖乘数

`vnmsub.vv vd, vs1, vs2, vm` # $vd[i] = -(vs1[i] * vd[i]) + vs2[i]$

`vnmsub.vx vd, rs1, vs2, vm` # $vd[i] = -(x[rs1] * vd[i]) + vs2[i]$

31.11.14 向量宽整数乘加指令

宽整数乘加指令将 SEW 位 * SEW 位乘法的完整 2*SEW 位乘积加到 2*SEW 位值上，产生 2*SEW 位的结果。支持所有有符号和无符号乘法操作数的组合。

拓宽无符号整数乘加，覆盖加数

```
vwmaccu.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) + vd[i]
vwmaccu.vx vd, rs1, vs2, vm # vd[i] = +(x[rs1] * vs2[i]) + vd[i]
```

拓宽有符号整数乘加，覆盖加数

```
vwmacc.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) + vd[i]
vwmacc.vx vd, rs1, vs2, vm # vd[i] = +(x[rs1] * vs2[i]) + vd[i]
```

拓宽有符号-无符号整数乘加，覆盖加数

```
vwmaccsu.vv vd, vs1, vs2, vm # vd[i] = +(signed(vs1[i]) * unsigned(vs2[i])) + vd[i]
vwmaccsu.vx vd, rs1, vs2, vm # vd[i] = +(signed(x[rs1]) * unsigned(vs2[i])) + vd[i]
```

拓宽无符号-有符号整数乘加，覆盖加数

```
vwmaccus.vx vd, rs1, vs2, vm # vd[i] = +(unsigned(x[rs1]) * signed(vs2[i])) + vd[i]
```

31.11.15 向量整数合并指令

向量整数合并指令根据掩码将两个源操作数组合。与常规算术指令不同，合并操作作用于所有元素（即从 vstart 到当前向量长度 vl 之间的元素集合）。

vmerge 指令被编码为掩码指令（vm=0）。指令按以下方式合并两个源操作数：在掩码值为零的元素处，第一个操作数被复制到目标元素，否则第二个操作数被复制到目标元素。第一个操作数始终是由 vs2 指定的向量寄存器组。第二个操作数是由 vs1 指定的向量寄存器组、由 rs1 指定的标量 x 寄存器，或一个 5 位符号扩展的立即数。

```
vmerge.vvm vd, vs2, vs1, v0 # vd[i] = v0.mask[i] ? vs1[i] : vs2[i]
vmerge.vxm vd, vs2, rs1, v0 # vd[i] = v0.mask[i] ? x[rs1] : vs2[i]
vmerge.vim vd, vs2, imm, v0 # vd[i] = v0.mask[i] ? imm : vs2[i]
```

31.11.16 向量整数移动指令

向量整数移动指令将源操作数复制到向量寄存器组中。vmv.v.v 变体复制一个向量寄存器组，而 vmv.v.x 和 vmv.v.i 变体将标量寄存器或立即数扩展到目标向量寄存器组的所有激活元素中。这些指令被编码为未掩码指令（vm=1）。第一个操作数说明符（vs2）必须包含 v0，并且 vs2 中的任何其他向量寄存器编号都是保留的。

```
vmv.v.v vd, vs1 # vd[i] = vs1[i]
```

```
vmv.v.x vd, rs1 # vd[i] = x[rs1]
vmv.v.i vd, imm # vd[i] = imm
```



掩码值可以通过序列 `vmv.v.i vd, 0; vmerge.vim vd, vd, 1, v0` 扩展为 SEW 宽度的元素。

向量整数移动指令与向量合并指令共享编码，但 `vm=1` 且 `vs2=v0`。

形式 `vmv.v.v vd, vd`，它保持主体元素不变，可以用来表示该寄存器接下来将以与 SEW 相等的 EEW 一起使用。



实现内部按 EEW 重组数据的，可以根据 SEW 重新排列内部表示。对于不进行数据内部重组的实现，可以动态省略此指令，将其视为 NOP。

`vmv.v.v vd, vd` 指令不是 RISC-V 提示 (HINT)，因为尾部无关的设置可能会在某些实现中引起架构状态变化。

31.12 向量定点算术指令

之前的整数算术指令集已扩展以支持定点算术。定点数是一个二进制补码的有符号或无符号整数，解释为一个分数的分子，且分母隐含。定点指令旨在应用于分子；分母的管理由软件负责。一个 N 位元素可以存储二进制补码的有符号整数，范围为 $-2^{(N-1)}$ 到 $+2^{(N-1)}-1$ ，以及无符号整数，范围为 0 到 $+2^N-1$ 。定点指令通过支持缩放和舍入来帮助保持窄操作数的精度，并通过将溢出结果饱和到目标格式范围来处理溢出。



上述描述的拓宽整数操作也可以用来避免溢出。

31.12.1 向量单宽度饱和加法与减法

提供了有符号和无符号整数的饱和加法和饱和减法形式。如果结果会溢出目标范围，结果将被替换为最接近的可表示值，并且 `vxsat` 位会被设置。

无符号整数的饱和加法

```
vsaddu.vv vd, vs2, vs1, vm # 向量-向量
vsaddu.vx vd, vs2, rs1, vm # 向量-标量
vsaddu.vi vd, vs2, imm, vm # 向量-立即数
```

有符号整数的饱和加法

```
vsadd.vv vd, vs2, vs1, vm # 向量-向量
vsadd.vx vd, vs2, rs1, vm # 向量-标量
vsadd.vi vd, vs2, imm, vm # 向量-立即数
```

无符号整数的饱和减法

```
vssubu.vv vd, vs2, vs1, vm # 向量-向量
vssubu.vx vd, vs2, rs1, vm # 向量-标量
```

有符号整数的饱和减法

```
vssub.vv vd, vs2, vs1, vm # 向量-向量
vssub.vx vd, vs2, rs1, vm # 向量-标量
```

31.12.2 向量单宽度平均加法与减法

均加法与减法指令会将结果右移一个比特，并根据 `vxrm` 中的设置对结果进行舍入。提供了有符号和无符号版本。对于 `vaaddu` 和 `vaadd`，结果中不会发生溢出。对于 `vasub` 和 `vasubu`，溢出会被忽略，结果会发生环绕。



对于 `vasub`，只有在使用 `rnu` 或 `rne` 舍入模式时，从最小数减去最大数才会发生溢出。

无符号整数的平均加法

```
vaaddu.vv vd, vs2, vs1, vm # roundoffunsigned(vs2[i] + vs1[i], 1)
vaaddu.vx vd, vs2, rs1, vm # roundoffunsigned(vs2[i] + x[rs1], 1)
```

有符号整数的平均加法

```
vaadd.vv vd, vs2, vs1, vm # roundoffsigned(vs2[i] + vs1[i], 1)
vaadd.vx vd, vs2, rs1, vm # roundoffsigned(vs2[i] + x[rs1], 1)
```

无符号整数的平均减法

```
vasubu.vv vd, vs2, vs1, vm # roundoffunsigned(vs2[i] - vs1[i], 1)
vasubu.vx vd, vs2, rs1, vm # roundoffunsigned(vs2[i] - x[rs1], 1)
```

有符号整数的平均减法

```
vasub.vv vd, vs2, vs1, vm # roundoffsigned(vs2[i] - vs1[i], 1)
vasub.vx vd, vs2, rs1, vm # roundoffsigned(vs2[i] - x[rs1], 1)
```

31.12.3 向量单宽度带舍入与饱和的分数乘法

有符号分数乘法指令将两个 `SEW` 位输入的乘积扩展为 $2*SEW$ 位，然后将结果右移 $SEW-1$ 位，根据 `vxrm` 舍入这些位，最后将结果饱和到 `SEW` 位。如果结果发生饱和，`vxsat` 位会被设置。

有符号饱和并舍入的分数乘法

请参见 `vxrm` 描述，了解舍入计算

```
vsmul.vv vd, vs2, vs1, vm # vd[i] = clip(roundoffsigned(vs2[i]*vs1[i], SEW-1))
vsmul.vx vd, vs2, rs1, vm # vd[i] = clip(roundoffsigned(vs2[i]*x[rs1], SEW-1))
```



当乘以两个 N 位有符号数时，最大幅度是通过 $-2^{(N-1)} * -2^{(N-1)}$ 得到的，结果为 $+2^{(2N-2)}$ ，该结果在 $2N$ 位中有一个（零）符号位。所有其他乘积在 $2N$ 位中将有二个符号位。为了保持 N 位结果的更高精度，乘积将右移一个比 N 少的位数，从而饱和最大的幅度结果，但对于所有其他乘积增加一个结果精度位。

我们不提供等效的分数乘法，其中一个输入是无符号的，因为这些操作会保留所有上位 SEW 位，并且无需饱和。此操作部分由 `vmulhu` 和 `vmulhsu` 指令覆盖，适用于舍入仅为截断（`rdn`）的情况。

31.12.4 向量单宽度缩放移位指令

这些指令将输入值右移，并根据 `vxrm` 对移出的比特进行舍入。缩放右移有两种形式：零扩展（`vssrl`）和符号扩展（`vssra`）。需要移位的数据在由 `vs2` 指定的向量寄存器组中，移位量的值可以来自向量寄存器组 `vs1`、标量整数寄存器 `rs1` 或零扩展的 5 位立即数。移位量值的低 $\lg 2(\text{SEW})$ 位用于控制移位量。

缩放右移（逻辑）

```
vssrl.vv vd, vs2, vs1, vm # vd[i] = roundoffunsigned(vs2[i], vs1[i])
vssrl.vx vd, vs2, rs1, vm # vd[i] = roundoffunsigned(vs2[i], x[rs1])
vssrl.vi vd, vs2, uimm, vm # vd[i] = roundoffunsigned(vs2[i], uimm)
```

缩放右移（算术）

```
vssra.vv vd, vs2, vs1, vm # vd[i] = roundoffsigned(vs2[i], vs1[i])
vssra.vx vd, vs2, rs1, vm # vd[i] = roundoffsigned(vs2[i], x[rs1])
vssra.vi vd, vs2, uimm, vm # vd[i] = roundoffsigned(vs2[i], uimm)
```

31.12.5 向量定点裁剪指令（缩小位宽）

`vnclip` 指令用于将定点值打包到较窄的目标格式中。该指令支持舍入、缩放和饱和到最终目标格式。源数据位于由 `vs2` 指定的向量寄存器组中。缩放移位量的值可以来自向量寄存器组 `vs1`、标量整数寄存器 `rs1` 或零扩展的 5 位立即数。向量或标量移位量值的低 $\lg 2(2 * \text{SEW})$ 位（例如， $\text{SEW}=64$ 位到 $\text{SEW}=32$ 位缩小操作时的低 6 位）用于控制右移量，从而提供缩放。

无符号裁剪

```
#                               SEW                2*SEW    SEW
vnclipu.vv vd, vs2, vs1, vm # vd[i] = clip(roundoffunsigned(vs2[i], vs1[i]))
vnclipu.wx vd, vs2, rs1, vm # vd[i] = clip(roundoffunsigned(vs2[i], x[rs1]))
vnclipu.wi vd, vs2, uimm, vm # vd[i] = clip(roundoffunsigned(vs2[i], uimm))
```

有符号裁剪

```
vnclip.vv vd, vs2, vs1, vm # vd[i] = clip(roundoffsigned(vs2[i], vs1[i]))
vnclip.wx vd, vs2, rs1, vm # vd[i] = clip(roundoffsigned(vs2[i], x[rs1]))
vnclip.wi vd, vs2, uimm, vm # vd[i] = clip(roundoffsigned(vs2[i], uimm))
```

对于 `vnclipu` 和 `vnclip`，舍入模式在 `vxrm` 控制状态寄存器中指定。舍入发生在目标的最低有效位之前，并且发生在饱和之前。

对于 `vnclipu`，移位后舍入的源值被视为无符号整数，如果结果溢出目标（被视为无符号整数），则结果会饱和。



没有单一指令可以将有符号值饱和到无符号目标。如果不需要为负数设置 `vxsat` 值，可以使用两个向量指令的序列：首先使用 `vmax` 与 0 执行最大值操作移除负数，然后使用 `vnclipu` 将结果无符号值裁剪到目标。要在这两条指令之间改变 SEW，需要使用 `vsetvli`。

对于 `vnclip`，移位后舍入的源值被视为有符号整数，如果结果溢出目标（被视为有符号整数），则结果会饱和。

如果任何目标元素发生饱和，`vxsat` 位会在 `vxsat` 寄存器中设置。

31.13 向量浮点指令

标准的向量浮点指令将元素视为符合 IEEE-754/2008 标准的值。如果向量浮点操作数的 EEW 与支持的 IEEE 浮点类型不匹配，则该指令编码是保留的。



浮点是否支持，以及支持哪些元素宽度，取决于具体的向量扩展。目前的扩展集包括对 32 位和 64 位浮点值的支持。当支持 16 位和 128 位元素宽度时，它们也将被视为符合 IEEE-754/2008 标准的值。未来的扩展可能会支持其他浮点格式。

向量浮点指令要求具有与支持的向量浮点元素宽度相对应的基础标量浮点扩展。



特别是，未来支持 16 位半精度浮点值的向量扩展，也将要求一些标量半精度浮点支持。

如果浮点单元状态字段 `mstatus.FS` 为 `Off`，则任何尝试执行向量浮点指令的操作将引发非法指令异常。任何修改浮点扩展状态（即浮点控制状态寄存器或 `f` 寄存器）的向量浮点指令必须将 `mstatus.FS` 设置为 `Dirty`。

如果实现了虚拟化扩展且 `V=1`，则 `vsstatus.FS` 字段对于向量浮点指令也会生效。如果 `vsstatus.FS` 或 `mstatus.FS` 为 `Off`，则任何尝试执行向量浮点指令的操作将引发非法指令异常。任何修改浮点扩展状态（即浮点 CSR 或 `f` 寄存器）的向量浮点指令必须将 `mstatus.FS` 和 `vsstatus.FS` 都设置为 `Dirty`。向量浮点指令在 NaN 处理方面与标量浮点指令具有相同的行为。浮点向量-标量操作的标量值来源于第 31.10.1 所述的方式。

31.13.1 向量浮点异常标志

任何激活浮点元素上的向量浮点异常会在 `fflags` 寄存器中设置标准的浮点异常标志。未激活元素不会设置浮点异常标志。

31.13.2 向量单宽度浮点加法/减法指令

浮点加法

```
vfadd.vv vd, vs2, vs1, vm # 向量-向量
vfadd.vf vd, vs2, rs1, vm # 向量-标量
```

浮点减法

```
vfsb.vv vd, vs2, vs1, vm # 向量-向量
vfsb.vf vd, vs2, rs1, vm # 向量-标量  $vd[i] = vs2[i] - f[rs1]$ 
vfrsub.vf vd, vs2, rs1, vm # 标量-向量  $vd[i] = f[rs1] - vs2[i]$ 
```

31.13.3 向量宽度浮点加法/减法指令

宽度浮点加法/减法, $2*SEW = SEW +/- SEW$

```
vfwadd.vv vd, vs2, vs1, vm # 向量-向量
vfwadd.vf vd, vs2, rs1, vm # 向量-标量
vfwsb.vv vd, vs2, vs1, vm # 向量-向量
vfwsb.vf vd, vs2, rs1, vm # 向量-标量
```

宽度浮点加法/减法, $2SEW = 2SEW +/- SEW$

```
vfwadd.wv vd, vs2, vs1, vm # 向量-向量
vfwadd.wf vd, vs2, rs1, vm # 向量-标量
vfwsb.wv vd, vs2, vs1, vm # 向量-向量
vfwsb.wf vd, vs2, rs1, vm # 向量-标量
```

31.13.4 向量单宽度浮点乘法/除法指令

浮点乘法

```
vfmul.vv vd, vs2, vs1, vm # 向量-向量
vfmul.vf vd, vs2, rs1, vm # 向量-标量
```

浮点除法

```
vfdiv.vv vd, vs2, vs1, vm # 向量-向量
vfdiv.vf vd, vs2, rs1, vm # 向量-标量
```

反向浮点除法, 向量 = 标量 / 向量

```
vfrdiv.vf vd, vs2, rs1, vm # 标量-向量,  $vd[i] = f[rs1]/vs2[i]$ 
```

31.13.5 向量宽度浮点乘法

宽度浮点乘法

```
vfwmul.vv vd, vs2, vs1, vm # 向量-向量
vfwmul.vf vd, vs2, rs1, vm # 向量-标量
```


31.13.6 向量单宽度浮点融合乘加指令

提供了四种类型的融合乘加指令，其中两种是破坏性形式，会覆盖操作数中的一个，可能是加数或第一个乘数。

浮点乘加，覆盖加数

```
vfmacc.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) + vd[i]
vfmacc.vf vd, rs1, vs2, vm # vd[i] = +(f[rs1] * vs2[i]) + vd[i]
```

浮点否定乘加，覆盖被减数

```
vfnmacc.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vs2[i]) - vd[i]
vfnmacc.vf vd, rs1, vs2, vm # vd[i] = -(f[rs1] * vs2[i]) - vd[i]
```

浮点乘减加，覆盖被减数

```
vfmsac.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) - vd[i]
vfmsac.vf vd, rs1, vs2, vm # vd[i] = +(f[rs1] * vs2[i]) - vd[i]
```

浮点否定乘减加，覆盖减数

```
vfnmsac.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vs2[i]) + vd[i]
vfnmsac.vf vd, rs1, vs2, vm # vd[i] = -(f[rs1] * vs2[i]) + vd[i]
```

浮点乘加，覆盖乘数

```
vfmadd.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vd[i]) + vs2[i]
vfmadd.vf vd, rs1, vs2, vm # vd[i] = +(f[rs1] * vd[i]) + vs2[i]
```

浮点否定乘加，覆盖乘数

```
vfnmadd.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vd[i]) - vs2[i]
vfnmadd.vf vd, rs1, vs2, vm # vd[i] = -(f[rs1] * vd[i]) - vs2[i]
```

浮点乘减，覆盖乘数

```
vfmsub.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vd[i]) - vs2[i]
vfmsub.vf vd, rs1, vs2, vm # vd[i] = +(f[rs1] * vd[i]) - vs2[i]
```

浮点反向乘减，覆盖乘数

```
vfnmsub.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vd[i]) + vs2[i]
vfnmsub.vf vd, rs1, vs2, vm # vd[i] = -(f[rs1] * vd[i]) + vs2[i]
```

虽然我们考虑过使用标量浮点 FMA 编码中的两个未使用的舍入模式来提供一些非破坏性的 FMA，这样做会使微架构变得复杂，因为它们是唯一具有三个输入和独立输出的带掩码操作。

31.13.7 向量宽度浮点融合乘加指令

宽度浮点融合乘加指令会用结果覆盖宽度加数。乘法输入都是 SEW 宽度，而加数和目标是 2*SEW 位宽

```

# 浮点宽度乘加，覆盖加数
vfwmacc.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) + vd[i]
vfwmacc.vf vd, rs1, vs2, vm # vd[i] = +(f[rs1] * vs2[i]) + vd[i]

# 浮点宽度否定乘加，覆盖加数
vfwnmacc.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vs2[i]) - vd[i]
vfwnmacc.vf vd, rs1, vs2, vm # vd[i] = -(f[rs1] * vs2[i]) - vd[i]

# 浮点宽度乘减加，覆盖加数
vfwsac.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) - vd[i]
vfwsac.vf vd, rs1, vs2, vm # vd[i] = +(f[rs1] * vs2[i]) - vd[i]

# 浮点宽度否定乘减加，覆盖加数
vfwnmsac.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vs2[i]) + vd[i]
vfwnmsac.vf vd, rs1, vs2, vm # vd[i] = -(f[rs1] * vs2[i]) + vd[i]

```

31.13.8 向量浮点平方根指令

这是一个一元向量-向量指令。

```

# 浮点平方根
vfsqrt.v vd, vs2, vm # 向量-向量平方根

```

31.13.9 向量浮点倒数平方根估算指令

```

# 浮点倒数平方根估算到 7 位。
vfrsqrt7.v vd, vs2, vm

```

这是一个一元向量-向量指令，返回精确到 7 位的 $1/\sqrt{x}$ 估算值。



早期的草案版本使用了汇编器名称 `vfrsqрте7`，但由于该名称与元素宽度的 `ex` 表示法容易混淆，因此进行了修改。早期的名称可以作为工具链中的别名，保持向后兼容。

表 31.13 描述了该指令对于所有浮点输入类别的行为：

表31.13 浮点输入类别

输入	输出	抛出的异常
$-\{\text{inf}\} \ \{\text{le}\} \ x < -0.0$	canonical NaN	NV
-0.0	$-\{\text{inf}\}$	DZ

+0.0	+{inf}	DZ
+0.0 < x < +{inf}	estimate of 1/sqrt(x)	
+{inf}	+0.0	
qNaN	canonical NaN	
sNaN	canonical NaN	NV

所有正的标准数和非标准数输入都会产生正常的输出。



输出值与动态舍入模式无关。

对于非异常情况，指数的最低位和有效数字的六个最高位（去掉前导 1 后）被拼接起来并用于查找下表。表格中的输出结果成为结果有效数字的七个最高位（去掉前导 1）；其余部分为零。非标准输入会先被规范化，并适当调整指数后再进行查找。输出的指数选择使得结果近似于输入数的倒数平方根。

更精确地说，结果计算如下：设输入的规范化指数为输入指数（如果输入是标准数），否则为 0 减去有效数字中前导零的个数。如果输入是非标准数，则规范化后的有效数字通过将输入有效数字左移 1 减去规范化指数的位数得到，并丢弃前导的 1 位。输出的指数等于 $\text{floor}((3*B - 1 - \text{规范化输入指数}) / 2)$ ，其中 B 是指数偏置。输出符号与输入符号相同。

下表给出了输出尾数的七个最高有效位（MSB）作为规范化输入指数最低有效位（LSB）和规范化输入尾数六个最高有效位（MSB）的函数；输出尾数的其他位为零。

例如，当 SEW=32 时， $\text{vfrsqrt7}(0x00718abc)$ （约为 $1.043e-38$ ）= $0x5f080000$ （约为 $9.800e18$ ），而 $\text{vfrsqrt7}(0x7f765432)$ （约为 $3.274e38$ ）= $0x1f820000$ （约为 $5.506e-20$ ）。



选择 7 位精度是因为它需要 0、1、2、3 次牛顿-拉弗森迭代来分别收敛到接近 bfloat16、FP16、FP32、FP64 精度。未来的指令可以定义更高精度的估算。

31.13.10 向量浮点倒数估算指令

浮点倒数估算，精确到 7 位
`vfrec7.v vd, vs2, vm`

早期的草案版本使用了汇编器名称 `vfrec7`，但由于该名称容易与元素宽度的 `ex` 表示法产生混淆，因此进行了修改。早期的名称可以作为工具链中的别名，保持向后兼容。



具有大小至少为 $2^{-(B+1)}$ 的次正常输入会产生正常输出；其他次正常输入会产生无穷大输出。具有大小至少为 $2^{-(B+1)}$ 的正常输入会产生次正常输出；其他正常输入会产生正常输出。

输出值取决于在溢出异常被触发时的动态舍入模式。

对于非异常情况，尾数的七个高位（去掉首位 1 后）用于查找以下表格。表格的输出值成为结果尾数的七个高位（去掉首位 1 后）；其余的结果尾数为零。次正常输入会先进行归一化，指数也会适当调整后再进行查找。输出指数选择以使结果逼近输入的倒数，且次正常输出会相应去归一化。

更精确地说，结果的计算方式如下：假设归一化的输入指数是输入指数（若输入为正常数），或者是 0 减去尾数中的前导零的个数（否则）。归一化的输出指数等于 $(2B - 1 - \text{归一化输入指数})$ 。如果归一化输出指数超出 $[-1, 2B]$ 范围，结果将对应于上述表格中的某个特殊情况。

如果输入为次正常数，则归一化的输入尾数由将输入尾数左移 1 减去归一化输入指数的位数，去掉前导的 1 位得到。否则，归一化的输入尾数等于输入尾数。以下表格给出了归一化输出尾数的七个最高有效位（MSB）作为归一化输入尾数的七个最高有效位的函数；归一化输出尾数的其他位为零。

如果归一化输出指数为 0 或 -1，结果为次正常数：输出指数为 0，输出尾数由将 1 位连接到归一化输出尾数的左侧，然后将该值右移 1 减去归一化输出指数得到。否则，输出指数等于归一化输出指数，输出尾数等于归一化输出尾数。输出符号等于输入符号。

例如，当 $SEW=32$ 时， $\text{vfrec7}(0x00718abc \text{ (}\{\text{approx}\} \text{ } 1.043\text{e-}38\text{)}) = 0x7e900000 \text{ (}\{\text{approx}\} \text{ } 9.570\text{e}37\text{)}$ ， $\text{vfrec7}(0x7f765432 \text{ (}\{\text{approx}\} \text{ } 3.274\text{e}38\text{)}) = 0x00214000 \text{ (}\{\text{approx}\} \text{ } 3.053\text{e-}39\text{)}$ 。



选择 7 位精度是因为它需要 0、1、2、3 次牛顿-拉夫森迭代，分别收敛到接近 bfloat16、FP16、FP32、FP64 精度。未来的指令可以定义为具有更高的估算精度。

31.13.11 向量浮点最小/最大指令

向量浮点指令 `vfmin` 和 `vfmax` 的行为与 RISC-V F/D/Q 扩展版本 2.2 中的相应标量浮点指令相同：它们对激活元素执行 `minimumNumber` 或 `maximumNumber` 操作。

浮点最小值

`vfmin.vv vd, vs2, vs1, vm` # 向量-向量

`vfmin.vf vd, vs2, rs1, vm` # 向量-标量

浮点最大值

`vfmax.vv vd, vs2, vs1, vm` # 向量-向量

`vfmax.vf vd, vs2, rs1, vm` # 向量-标量

31.13.12 向量浮点符号注入指令

标量符号注入指令的向量版本。结果将从向量 `vs2` 操作数中获取除了符号位之外的所有位。

`vfsgnj.vv vd, vs2, vs1, vm` # 向量-向量

`vfsgnj.vf vd, vs2, rs1, vm` # 向量-标量

`vfsgnjn.vv vd, vs2, vs1, vm` # 向量-向量

`vfsgnjn.vf vd, vs2, rs1, vm` # 向量-标量

`vfsgnjx.vv vd, vs2, vs1, vm` # 向量-向量

`vfsgnjx.vf vd, vs2, rs1, vm` # 向量-标量



可以使用符号注入指令对浮点值的向量进行取反，将两个源操作数设置为相同的向量操作数。提供的汇编伪指令为：`vfneg.v vd, vs = vfgsnjn.vv vd, vs, vs`。

可以使用符号注入指令计算浮点元素向量的绝对值，将两个源操作数设置为相同的向量操作数。提供的汇编伪指令为：`vfabs.v vd, vs = vfgsnjx.vv vd, vs, vs`。

31.13.13 向量浮点比较指令

这些向量浮点比较指令比较两个源操作数，并将比较结果写入掩码寄存器。目标掩码向量始终保存在单个向量寄存器中，元素布局如第 31.4.5 所述。目标掩码向量寄存器可以与源向量掩码寄存器（v0）相同。比较指令会写入掩码寄存器，因此始终在与尾部无关的策略下操作。

比较指令遵循标量浮点比较指令的语义。`vmfeq` 和 `vmfne` 仅在输入为信号 NaN 时触发无效操作异常。`vmflt`、`vmfle`、`vmfgt` 和 `vmfge` 在输入为信号 NaN 或静默 NaN 时都触发无效操作异常。`vmfne` 在任一操作数为 NaN 时将 1 写入目标元素，而其他比较指令在任一操作数为 NaN 时将 0 写入目标元素。

比较相等

`vmfeq.vv vd, vs2, vs1, vm` # 向量-向量

`vmfeq.vf vd, vs2, rs1, vm` # 向量-标量

比较不相等

`vmfne.vv vd, vs2, vs1, vm` # 向量-向量

`vmfne.vf vd, vs2, rs1, vm` # 向量-标量

比较小于

`vmflt.vv vd, vs2, vs1, vm` # 向量-向量

`vmflt.vf vd, vs2, rs1, vm` # 向量-标量

比较小于或等于

`vmfle.vv vd, vs2, vs1, vm` # 向量-向量

`vmfle.vf vd, vs2, rs1, vm` # 向量-标量

比较大于

`vmfgt.vf vd, vs2, rs1, vm` # 向量-标量

比较大于或等于

`vmfge.vf vd, vs2, rs1, vm` # 向量-标量

比较	汇编器映射	汇编器伪指令
<code>va < vb</code>	<code>vmflt.vv vd, va, vb, vm</code>	
<code>va <= vb</code>	<code>vmfle.vv vd, va, vb, vm</code>	

<code>va > vb</code>	<code>vmflt.vv vd, vb, va, vm</code>	<code>vmfgt.vv vd, va, vb, vm</code>
<code>va >= vb</code>	<code>vmfle.vv vd, vb, va, vm</code>	<code>vmfge.vv vd, va, vb, vm</code>
<code>va < f</code>	<code>vmflt.vf vd, va, f, vm</code>	
<code>va <= f</code>	<code>vmfle.vf vd, va, f, vm</code>	
<code>va > f</code>	<code>vmfgt.vf vd, va, f, vm</code>	
<code>va >= f</code>	<code>vmfge.vf vd, va, f, vm</code>	

`va, vb` 向量寄存器组
`f` 标量浮点寄存器

提供所有形式的比较指令对于正确处理 NaN 的无序比较是必要的。



C99 浮点静默比较可以通过在任一输入为 NaN 时屏蔽信号比较来实现，方法如下。当比较值是非 NaN 常量时，可以省略中间的两个指令。

实现 `isgreater()` 的示例

```
vmfeq.vv v0, va, va      # 仅在 A 不是 NaN 时设置。
vmfeq.vv v1, vb, vb      # 仅在 B 不是 NaN 时设置。
vmand.mm v0, v0, v1      # 仅在 A 和 B 是有序时设置，
vmfgt.vv v0, va, vb, v0.t # 仅在有序值上设置标志。
```



在上述序列中，虽然有可能屏蔽第二条 `vmfeq` 指令并移除 `vmand` 指令，但这个更高效的序列会错误地未能在 `va` 的元素包含静默 NaN 且 `vb` 中相应元素包含信号 NaN 时触发无效异常。

31.13.14 向量浮点分类指令

这是一个一元向量-向量指令，操作方式与标量分类指令相同。

```
vfclass.v vd, vs2, vm    # 向量-向量
```

该指令生成的 10 位掩码被放置在结果元素的最低有效位。结果的高位（SEW-10 位）填充为零。该指令仅在 SEW=16b 或更高时定义，因此结果总是能够适配目标元素。

31.13.15 向量浮点合并指令

提供了一条向量-标量浮点合并指令，它对从 `vstart` 到当前向量长度 `v1` 之间的所有元素进行操作，无论掩码值如何。

`vfmerge.vfm` 指令被编码为一个带掩码的指令（`vm=0`）。在掩码值为零的元素上，第一个向量操作数被复制到目标元素；否则，将标量浮点寄存器值复制到目标元素。

```
vfmerge.vfm vd, vs2, rs1, v0 # vd[i] = v0.mask[i] ? f[rs1] : vs2[i]
```

31.13.16 向量浮点移动指令

向量浮点移动指令将浮点标量操作数扩展到一个向量寄存器组。该指令将标量 *f* 寄存器的值复制到向量寄存器组的所有激活元素中。该指令被编码为一个不带掩码的指令 (*vm*=1)。指令必须将 *vs2* 字段设置为 *v0*，并且其他 *vs2* 的值被保留。

```
vfmv.v.f vd, rs1 # vd[i] = f[rs1]
```



`vfmv.v.f`指令与`vfmerge.vfm`指令共享编码，但其`vm=1`且`vs2=v0`。

31.13.17 单宽浮点/整数类型转换指令

提供了转换操作，用于在浮点值和无符号/有符号整数之间进行转换，源和目标均为 SEW 宽度。

```
vfcvt.xu.f.v vd, vs2, vm # 将浮点数转换为无符号整数。
vfcvt.x.f.v vd, vs2, vm # 将浮点数转换为有符号整数。

vfcvt.rtz.xu.f.v vd, vs2, vm # 将浮点数转换为无符号整数（截断）。
vfcvt.rtz.x.f.v vd, vs2, vm # 将浮点数转换为有符号整数（截断）。

vfcvt.f.xu.v vd, vs2, vm # 将无符号整数转换为浮点数。
vfcvt.f.x.v vd, vs2, vm # 将有符号整数转换为浮点数。
```

这些转换遵循与标量转换指令相同的异常条件规则。转换使用 *frm* 中的动态舍入模式，*rtz* 变体除外，后者是朝零舍入。



*rtz*变体是为了加速从浮点到整数的截断转换，正如 C 和 Java 等语言中常见的做法。

31.13.18 扩宽浮点/整数类型转换指令

提供了一组转换指令，用于将较窄的整数和浮点数据类型转换为宽度是原来两倍的类型。

```
vfwcvt.xu.f.v vd, vs2, vm # 将浮点数转换为双宽无符号整数。
vfwcvt.x.f.v vd, vs2, vm # 将浮点数转换为双宽有符号整数。
```


`vfwcvt.rtz.xu.f.v vd, vs2, vm` # 将浮点数转换为双宽无符号整数（截断）。
`vfwcvt.rtz.x.f.v vd, vs2, vm` # 将浮点数转换为双宽有符号整数（截断）。

`vfwcvt.f.xu.v vd, vs2, vm` # 将无符号整数转换为双宽浮点数。
`vfwcvt.f.x.v vd, vs2, vm` # 将有符号整数转换为双宽浮点数。

`vfwcvt.f.f.v vd, vs2, vm` # 将单宽浮点数转换为双宽浮点数。

这些指令在向量寄存器重叠上的约束与其他扩展指令相同（参见第 31.10.2）。

双宽 IEEE 浮点数值总是能够准确表示单宽度整数。



双宽 IEEE 浮点数值总是能够准确表示单宽度 IEEE 浮点数值。

并未提供完整的浮点扩宽转换作为单个指令，但任何扩宽转换都可以通过多个加倍步骤来实现，且结果等效，不会引发额外的异常标志。

31.13.19 缩小浮点/整数类型转换指令

提供了一组转换指令，用于将较宽的整数和浮点数据类型转换为宽度为原来一半的类型。

`vfncvt.xu.f.w vd, vs2, vm` # 将双宽浮点数转换为无符号整数。
`vfncvt.x.f.w vd, vs2, vm` # 将双宽浮点数转换为有符号整数。

`vfncvt.rtz.xu.f.w vd, vs2, vm` # 将双宽浮点数转换为无符号整数（截断）。
`vfncvt.rtz.x.f.w vd, vs2, vm` # 将双宽浮点数转换为有符号整数（截断）。

`vfncvt.f.xu.w vd, vs2, vm` # 将双宽无符号整数转换为浮点数。
`vfncvt.f.x.w vd, vs2, vm` # 将双宽有符号整数转换为浮点数。

`vfncvt.f.f.w vd, vs2, vm` # 将双宽浮点数转换为单宽浮点数。
`vfncvt.rod.f.f.w vd, vs2, vm` # 将双宽浮点数转换为单宽浮点数，
朝奇数舍入。

这些指令在向量寄存器重叠上的约束与其他缩小指令相同（参见第 31.10.3）。

并未提供完整的浮点缩小转换作为单个指令。转换可以通过一系列减半步骤来实现。如果除最后一步外的所有减半步骤都使用朝奇数舍入（`vfncvt.rod.f.f.w`），结果将等效地舍入，且会引发相同的异常标志。只有最后一步应使用所需的舍入模式。



对于 `vfncvt.rod.f.f.w`，如果有限值超出了目标格式的范围，将被转换为目标格式的最大有限值，并保留相同符号。

31.14 向量归约操作

向量归约操作接受一个向量寄存器组的元素和一个存储在向量寄存器第 0 元素中的标量，并使用

某个二元操作符进行归约，最终将结果存储在向量寄存器的第 0 元素中作为标量结果。标量输入和输出操作数存储在单个向量寄存器的第 0 元素中，而不是在向量寄存器组中，因此，无论 LMUL 设置如何，任何向量寄存器都可以作为向量归约的标量源操作数或目标操作数。目标向量寄存器可以与源操作数重叠，包括掩码寄存器。



向量归约操作会将标量操作数和结果读取并写入到向量寄存器的第 0 元素中，而不是标量寄存器中，以避免与标量处理器的耦合丧失，并支持未来与标量单元中不支持的类型的多态使用。

源向量寄存器组中的未激活元素会被排除在归约之外，但标量操作数始终会被包含于内，无论掩码值如何。目标向量寄存器中的其他元素 ($0 < \text{索引} < \text{VLEN}/\text{SEW}$) 被视为尾部元素，并按当前的尾部不可知/不干扰策略进行管理。如果 $v1=0$ ，则不执行任何操作，目标寄存器不会更新。



对于 $v1=0$ 的行为选择简化了实现复杂性，因为它与对向量寄存器状态的其他操作保持一致。对于常见情况，即源操作数和目标标量操作数是同一个向量寄存器时，这种行为也会产生预期的结果。对于不常见的情况，即源和目标标量操作数位于不同的向量寄存器中，当 $v1=0$ 时，这条指令不会将源寄存器的内容复制到目标寄存器。然而，预计在大多数情况下会静态地知道 $v1$ 不是零。在其他情况下，必须检查 $v1$ 是否为 0，以确保源标量被复制到目标（例如，显式设置 $v1=1$ 并执行寄存器到寄存器的复制）。

向量归约指令的异常总是以 $vstart$ 为 0 进行报告。如果 $vstart$ 非零，向量归约操作将触发非法指令异常。

归约操作的汇编语法是 `vredop.vs`，其中 `.vs` 后缀表示第一个操作数是向量寄存器组，第二个操作数是存储在向量寄存器元素 0 中的标量。

31.14.1 量单宽整数归约指令

所有单宽归约指令的操作数和结果具有相同的 SEW 宽度。在算术求和时，溢出会发生环绕。

```
# Simple reductions, where [*] denotes all active elements:
vredsum.vs vd, vs2, vs1, vm # vd[0] = sum( vs1[0], vs2[*] )
vredmaxu.vs vd, vs2, vs1, vm # vd[0] = maxu( vs1[0], vs2[*] )
vredmax.vs vd, vs2, vs1, vm # vd[0] = max( vs1[0], vs2[*] )
vredminu.vs vd, vs2, vs1, vm # vd[0] = minu( vs1[0], vs2[*] )
vredmin.vs vd, vs2, vs1, vm # vd[0] = min( vs1[0], vs2[*] )
vredand.vs vd, vs2, vs1, vm # vd[0] = and( vs1[0], vs2[*] )
vredor.vs vd, vs2, vs1, vm # vd[0] = or( vs1[0], vs2[*] )
vredxor.vs vd, vs2, vs1, vm # vd[0] = xor( vs1[0], vs2[*] )
```

31.14.2 向量宽整数归约指令

无符号的 `vwredsumu.vs` 指令在求和之前会对 SEW 宽度的向量元素进行零扩展，然后将 2SEW 宽度的标量元素加到结果中，并将结果存储到一个 2SEW 宽度的标量元素中。

`vwredsum.vs` 指令在求和之前会对 SEW 宽度的向量元素进行符号扩展。对于 `vwredsumu.vs` 和 `vwredsum.vs`，溢出会发生环绕。

```
# 无符号和归约，累加到双宽度累加器
vwredsumu.vs vd, vs2, vs1, vm # 2*SEW = 2*SEW + sum(零扩展(SEW))

# 有符号和归约，累加到双宽度累加器
vwredsum.vs vd, vs2, vs1, vm # 2*SEW = 2*SEW + sum(符号扩展(SEW))
```

31.14.3 向量单宽浮点归约指令

```
# 简单归约。
vfredosum.vs vd, vs2, vs1, vm # 有序求和
vfredusum.vs vd, vs2, vs1, vm # 无序求和
vfredmax.vs vd, vs2, vs1, vm # 最大值
vfredmin.vs vd, vs2, vs1, vm # 最小值
```



旧的汇编助记符 `vfredsum` 被保留为 `vfredusum` 的别名。

`vfredosum` 指令必须按照元素顺序对浮点值进行求和，从 `vs1[0]` 中的标量开始——即它执行以下计算：

$$vd[0] = (((vs1[0] + vs2[0]) + vs2[1]) + \dots) + vs2[v1-1]$$

其中每次加法操作与标量浮点指令相同，会触发异常标志并生成或传播特殊值。



有序归约支持编译器的自动向量化，而无序浮点求和则允许更快速的实现。

当操作被掩码时（`vm=0`），被掩码的元素不会影响结果或异常标志。



如果没有元素处于激活状态，则不会执行加法操作，因此 `vs1[0]` 中的标量将直接复制到目标寄存器，而不会标准化 NaN 值，也不会设置任何异常标志。这种行为保留了在自动向量化标量求和循环时对 NaN、异常和舍入的处理。

31.14.4 向量无序单宽浮点求和归约

无序求和归约指令 `vfredusum` 提供了更大的灵活性来执行归约。实现必须生成一个结果，等同于由二元运算符节点组成的归约树，输入为源向量寄存器组（`vs2`）中的元素和源标量值（`vs1[0]`）。树中的每个运算符接受两个输入并生成一个结果。每个操作符的结果可以选择不同的浮点范围和精度。

当某个节点的一个输入仅来自被掩码掉或超出激活向量长度的元素时，可以将该输入视为适当 EEW

类型的加法单位元素，或直接将另一个输入复制到输出。

从归约树的根节点获得的舍入结果会被转换（再次进行舍入，使用动态舍入模式）为由 SEW 指定的标准浮点格式。实现可以向最终结果添加额外的加法单位元素。加法单位是 $+0.0$ ，当舍入方向为向下（接近 $-\infty$ ）时，或者对于所有其他舍入模式为 -0.0 。归约树的结构对于给定的 `vtype` 和 `vl` 必须是确定的。



根据此定义，实现不需要在没有活动元素时通过归约树传播 NaN 负载。特别是，如果没有元素处于活动状态并且标量输入为 NaN，则实现允许将 NaN 标准化，并且如果该 NaN 为信号 NaN，则设置无效异常标志。实现也可以选择直接通过初始 NaN 并且不设置任何异常标志，就像 `vfredosum` 一样。

`vfredosum` 指令是 `vfredusum` 指令的有效实现。

`vfredmin` 和 `vfredmax` 指令分别使用 `minimumNumber` 和 `maximumNumber` 操作对 `vs1[0]` 中的标量参数和 `vs2` 中的激活元素进行归约。



浮点最大值和最小值归约应返回相同的最终值，并且无论操作顺序如何，都应触发相同的异常标志。

如果没有元素处于激活状态，则 `vs1[0]` 中的标量将简单地复制到目标寄存器，而不会标准化 NaN 值，也不会设置任何异常标志。

提供了求和归约的宽度形式，这些形式会读取并写入一个双宽度的归约结果。

简单归约

`vwfredosum.vd vs2, vs1, vm` # 有序求和

`vwfredusum.vd vs2, vs1, vm` # 无序求和



旧的汇编助记符 `vwfredsum` 被保留为 `vwfredusum` 的别名。

SEW 宽度的元素归约的执行方式与单宽度归约相同，`vs2` 中的元素在加到 2SEW 位的累加器之前会被提升到 2SEW 位。



`vwfredosum.vd` 处理未激活元素和 NaN 负载的方式类似于 `vfredosum.vd`；`vwfredusum.vd` 处理方式类似于 `vfredusum.vd`。

31.15 向量掩码指令

提供了若干指令，帮助操作存储在向量寄存器中的掩码值。

31.15.1 向量掩码寄存器逻辑指令

向量掩码寄存器逻辑操作作用于掩码寄存器。掩码寄存器中的每个元素是一个单独的位，因此这些指令作用于单个向量寄存器，而与 `vtype` 中 `vlmul` 字段的设置无关。它们不会改变 `vlmul` 的值。目标向量寄存器可以是与源向量寄存器相同。

与其他向量指令类似，索引小于 `vstart` 的元素保持不变，执行后 `vstart` 会被重置为零。向量掩码逻辑指令始终是未掩码的，因此没有未激活元素，并且 `vm=0` 的编码被保留。超出 `v1` 的掩码元素，即尾部元素，始终按尾部无关策略更新。

```
vmmand.mm vd, vs2, vs1 # vd.mask[i] = vs2.mask[i] && vs1.mask[i]
vmnand.mm vd, vs2, vs1 # vd.mask[i] = !(vs2.mask[i] && vs1.mask[i])
vmandn.mm vd, vs2, vs1 # vd.mask[i] = vs2.mask[i] && !vs1.mask[i]
vmxor.mm vd, vs2, vs1 # vd.mask[i] = vs2.mask[i] ^^ vs1.mask[i]
vmor.mm vd, vs2, vs1 # vd.mask[i] = vs2.mask[i] || vs1.mask[i]
vmnor.mm vd, vs2, vs1 # vd.mask[i] = !(vs2.mask[i] || vs1.mask[i])
vmorn.mm vd, vs2, vs1 # vd.mask[i] = vs2.mask[i] || !vs1.mask[i]
vmxnor.mm vd, vs2, vs1 # vd.mask[i] = !(vs2.mask[i] ^^ vs1.mask[i])
```



之前的汇编助记符 `vmandnot` 和 `vmornot` 已经被改为 `vmandn` 和 `vmorn`，以与等效的标量指令保持一致。旧的 `vmandnot` 和 `vmornot` 助记符可以作为兼容性别名保留。

定义了几个汇编伪指令，作为常用掩码逻辑操作的简写：

```
vmmv.m vd, vs => vmmand.mm vd, vs, vs # 复制掩码寄存器
vmclr.m vd => vmxor.mm vd, vd, vd # 清除掩码寄存器
vmset.m vd => vmxnor.mm vd, vd, vd # 设置掩码寄存器
vmnot.m vd, vs => vmnand.mm vd, vs, vs # 按位取反
```



`vmmv.m` 指令以前叫做 `vmcpy.m`，但是现在根据新的布局，它更符合 `mv` 的命名，因为位被直接复制而没有解释。为了兼容性，可以保留旧的 `vmcpy.m` 汇编伪指令。对于内部根据 `EEW` 重排位的实现，可以使用源和目标相同的 `vmmv.m` 指令，作为强制内部重新格式化为掩码向量的惯用法。

下面这八条掩码逻辑指令可以生成两个输入掩码的 16 种可能的二进制逻辑函数。

表31.14 掩码逻辑生成表

输出				介绍	
0	0	0	0	<code>vmxor.mm vd, vd, vd</code>	<code>vmclr.m vd</code>
0	0	0	1	<code>vmand.mm vd, src1, src2</code>	
0	0	1	1	<code>vmand.mm vd, src1, src1</code>	<code>vmmv.m vd, src1</code>
0	1	0	0	<code>vmandn.mm vd, src2, src1</code>	
0	1	0	1	<code>vmand.mm vd, src2, src2</code>	<code>vmmv.m vd, src2</code>
0	1	1	0	<code>vmxor.mm vd, src1, src2</code>	
0	1	1	1	<code>vmor.mm vd, src1, src2</code>	
1	0	0	0	<code>vmnor.mm vd, src1, src2</code>	

1	0	0	1	vmxnor.mm vd, src1, src2	
1	0	1	0	vmnand.mm vd, src2, src2	vmnot.mvd, src2
1	0	1	1	vmorn.mm vd, src1, src2	
1	1	0	0	vmnand.mm vd, src1, src1	vmnot.mvd, src1
1	1	0	1	vmorn.mm vd, src2, src1	
1	1	1	0	vmnand.mm vd, src1, src2	
1	1	1	1	vmxnor.mm vd, vd, vd	vmset.m vd
0	0	1	0	vmandn.mm vd, src1, src2	



向量掩码逻辑指令旨在与后续的掩码向量操作轻松融合，从而通过在使用之前将值移入 v0 来有效扩展谓词寄存器的数量。

31.15.2 向量掩码中的计数操作 vcpop.m

vcpop.m rd, vs2, vm



该指令之前的汇编助记符是 vpopc.m，但为了与标量指令保持一致，已将其重命名。为了软件兼容性，汇编指令别名 vpopc.m 仍然保留。

源操作数是一个向量寄存器，存储了掩码寄存器的值，具体内容参见第 31.4.5。

vcpop.m 指令计算向量源掩码寄存器的激活元素中值为 1 的掩码元素的数量，并将结果写入标量 x 寄存器。该操作可以在掩码下执行，此时仅计数被掩码的元素。

vcpop.m rd, vs2, v0.t # x[rd] = sumi (vs2.mask[i] && v0.mask[i])

即使 vl = 0，vcpop.m 指令也会写入 x[rd]（值为 0，因为没有激活的掩码元素）。

vcpop.m 指令的陷阱始终报告 vstart 为 0。如果 vstart 非零，则该指令会触发非法指令异常。
vfirst 查找第一个设置的掩码位

31.15.3 首元素查找掩码位的首置位

vfirst.m rd, vs2, vm

vfirst 指令查找源掩码向量中值为 1 的最小编号的激活元素，并将该元素的索引写入通用寄存器（GPR）。如果没有激活元素的值为 1，则将 -1 写入 GPR。



软件可以假设任何负值（最高位被设置）表示未找到元素，因为向量长度在任何实现中都不会达到 $2^{(XLEN-1)}$ 。

即使 vl = 0，vfirst.m 指令也会写入 x[rd]（值为 -1，因为没有激活的掩码元素）。
vfirst 指令的陷阱始终报告 vstart 为 0。如果 vstart 非零，则该指令会触发非法指令异常。

31.15.4 vmsbf.m 首置位前掩码位设置

vmsbf.m 设置第一个掩码位之前的掩码位

7	6	5	4	3	2	1	0	元素编号
1	0	0	1	0	1	0	0	v3 内容
vmsbf.m v2, v3								
0	0	0	0	0	0	1	1	v2 内容
1	0	0	1	0	1	0	1	v3 内容
vmsbf.m v2, v3								
0	0	0	0	0	0	0	0	v2
0	0	0	0	0	0	0	0	v3 内容
vmsbf.m v2, v3								
1	1	1	1	1	1	1	1	v2
1	1	0	0	0	0	1	1	v0 内容
1	0	0	1	0	1	0	0	v3 内容
vmsbf.m v2, v3, v0.t								
0	1	x	x	x	x	1	1	v2 内容

vmsbf.m 指令以掩码寄存器为输入,并将结果写回掩码寄存器。该指令在第一个活跃源元素为 1 之前,将所有活跃的掩码元素写为 1,然后将该元素及其后续的所有活跃元素写为 0。如果源向量的活跃元素中没有设置的位,则目标中的所有活跃元素都会被写为 1。目标掩码寄存器中的尾部元素在尾部无关策略下更新。

vmsbf.m 指令的异常总是报告 vstart 为 0。如果 vstart 非零,vmsbf 指令会引发非法指令异常。目标寄存器不能与源寄存器重叠,并且如果被屏蔽,也不能与掩码寄存器(v0)重叠。vmsif.m 设置包括第一个掩码位向量掩码设置包括第一个指令类似于设置前第一个,只是它还包括具有设置位的元素。

31.15.5 vmsif.m 首置位包含掩码位

vmsif.m vd, vs2, vm								
7	6	5	4	3	2	1	0	元素编号
1	0	0	1	0	1	0	0	v3 内容
vmsif.m v2, v3								
0	0	0	0	0	1	1	1	v2 内容
1	0	0	1	0	1	0	1	v3 内容
vmsif.m v2, v3								
0	0	0	0	0	0	0	1	v2
1	1	0	0	0	0	1	1	v0 内容
1	0	0	1	0	1	0	0	v3 内容


```

vmsif.m v2, v3, v0.t
1 1 x x x 1 1    v2 内容

```

目标掩码寄存器中的尾部元素在尾部无关策略下更新。

vmsif.m 指令的异常总是报告 vstart 为 0。如果 vstart 非零, vmsif 指令会引发非法指令异常。目标寄存器不能与源寄存器重叠, 并且如果被屏蔽, 也不能与掩码寄存器 (v0) 重叠。vmsof.m 仅设置第一个掩码位。向量掩码仅设置第一个指令与设置前第一个指令类似, 区别在于它只设置第一个设置了位的元素 (如果有的话)。

31.15.6 vmsof.m 仅首置位掩码位

```

vmsof.m vd, vs2, vm

```

例子

```

7 6 5 4 3 2 1 0    元素编号

1 0 0 1 0 1 0 0    v3 内容
                    vmsof.m v2, v3
0 0 0 0 0 1 0 0    v2 内容

1 0 0 1 0 1 0 1    v3 内容
                    vmsof.m v2, v3
0 0 0 0 0 0 0 1    v2

1 1 0 0 0 0 1 1    v0 内容
1 1 0 1 0 1 0 0    v3 内容
                    vmsof.m v2, v3, v0.t
0 1 x x x 0 0    v2 内容

```

目标掩码寄存器中的尾部元素在尾部无关策略下更新。vmsof.m 指令的异常总是报告 vstart 为 0。如果 vstart 非零, vmsof 指令会引发非法指令异常。目标寄存器不能与源寄存器重叠, 并且如果被屏蔽, 也不能与掩码寄存器 (v0) 重叠。

31.15.7 使用向量掩码指令的示例

```

    # char* strcpy(char *dst, const char* src)
strcpy:
    mv a2, a0          # Copy dst
    li t0, -1          # Infinite AVL
loop:
    vsetvli x0, t0, e8, m8, ta, ma # Max length vectors of bytes
    vle8ff.v v8, (a1)      # Get src bytes
    csrr t1, vl          # Get number of bytes fetched
    vmseq.vi v1, v8, 0    # Flag zero bytes
    vfirst.m a3, v1       # Zero found?
    add a1, a1, t1        # Bump pointer
    vmsif.m v0, v1        # Set mask up to and including zero byte.
    vse8.v v8, (a2), v0.t # Write out bytes
    add a2, a2, t1        # Bump pointer
    bltz a3, loop         # Zero byte not found, so loop

    ret

```

```

    # char* strncpy(char *dst, const char* src, size_t n)
    strncpy:
        mv a3, a0          # Copy dst
    loop:
        vsetvli x0, a2, e8, m8, ta, ma  # Vectors of bytes.
        vle8ff.v v8, (a1)                # Get src bytes
        vmseq.vi v1, v8, 0                # Flag zero bytes
        csrr t1, vl                      # Get number of bytes fetched
        vfirst.m a4, v1                  # Zero found?
        vmsbf.m v0, v1                   # Set mask up to before zero byte.
        vse8.v v8, (a3), v0.t            # Write out non-zero bytes
        bgez a4, zero_tail               # Zero remaining bytes.
        sub a2, a2, t1                   # Decrement count.
        add a3, a3, t1                   # Bump dest pointer
        add a1, a1, t1                   # Bump src pointer
        bnez a2, loop                    # Anymore?

        ret

    zero_tail:
        sub a2, a2, a4                  # Subtract count on non-zero bytes.
        add a3, a3, a4                  # Advance past non-zero bytes.
        vsetvli t1, a2, e8, m8, ta, ma  # Vectors of bytes.
        vmv.v.i v0, 0                   # Splat zero.

    zero_loop:
        vse8.v v0, (a3)                  # Store zero.
        sub a2, a2, t1                   # Decrement count.
        add a3, a3, t1                   # Bump pointer
        vsetvli t1, a2, e8, m8, ta, ma  # Vectors of bytes.
        bnez a2, zero_loop               # Anymore?

        ret

```

31.15.8 向量iota 指令

`viota.m` 指令读取源向量掩码寄存器,并将该掩码寄存器中所有小于当前元素索引的元素的位的总和写入目标向量寄存器组的每个元素,例如,掩码值的并行前缀和。此指令可以进行掩码操作,在这种情况下,只有启用的元素才会参与计算。

```
viota.m vd, vs2, vm
```

例子

```
7 6 5 4 3 2 1 0  元素编号
```

```
1 0 0 1 0 0 0 1  v2 内容
```

```
viota.m v4, v2 # 未掩码
```

```

2 2 2 1 1 1 1 0    v4 结果

1 1 1 0 1 0 1 1    v0 内容
1 0 0 1 0 0 0 1    v2 内容
2 3 4 5 6 7 8 9    v4 内容
                        viota.m v4, v2, v0.t # 掩码, vtype.vma=0
1 1 1 5 1 7 1 0    v4 结果

```

如果 SEW 宽度大于结果宽度，则结果值会被零扩展以填充目标元素。如果结果值会溢出目标 SEW，则保留最低有效的 SEW 位。

viota.m 指令的异常总是报告 vstart 为 0，且每次恢复后从头开始执行。如果 vstart 非零，指令会引发非法指令异常。目标寄存器组不能与源寄存器重叠，并且如果被屏蔽，也不能与掩码寄存器(v0)重叠。viota.m 指令可以与内存散射指令（索引存储）结合使用，以执行向量压缩功能。

```

# 将非零元素从输入内存数组压缩到输出内存数组
#
# sizet compactnonzero(sizet n, const int* in, int* out)
# {
#     sizet i;
#     sizet count = 0;
#     int *p = out;
#
#     for (i=0; i<n; i++)
#     {
#         const int v = *in++;
#         if (v != 0)
#             *p++ = v;
#     }
#
#     return (sizet) (p - out);
# }
#
# a0 = n
# a1 = &in
# a2 = &out

```

```

compactnonzero:
    li a6, 0                # 重置非零元素计数数量
loop:
    vsetvli a5, a0, e32, m8, ta, ma    # 32 位整形
    vle32.v v8, (a1)                # 读取输入向量
    sub a0, a0, a5                # 完成数量递减
    slli a5, a5, 2                # 乘以 4 个字节
    vmsne.vi v0, v8, 0            # 查找非零值

```

```

    add a1, a1, a5          # 碰撞输入指针
    vcpop.m a5, v0          # 对 v0 中设置的元素数进行计数
    viota.m v16, v0         # 获取激活元素的目标偏移量
    add a6, a6, a5          # 累加元素数量
    vsll.vi v16, v16, 2, v0.t # 将偏移量乘以 4 个字节
    slli a5, a5, 2          # 将非零元素的数量乘以 4 个字节
    vsuxei32.v v8, (a2), v16, v0.t # 在掩码下使用缩放的 viota 结果进行散射
    add a2, a2, a5          # 碰撞输出指针
    bnez a0, loop           # 继续?

    mv a0, a6              # 返回计数数量
    ret

```

31.15.9 向量元素索引指令

`vid.v` 指令将每个元素的索引写入目标向量寄存器组，索引从 0 到 `v1-1`。

`vid.v vd, vm` # 将元素 ID 写入目标。

该指令可以被屏蔽。屏蔽不会改变写入激活元素的索引值。指令中的 `vs2` 字段必须设置为 `v0`，否则该编码为保留。如果目标 SEW 比结果宽，结果值将被零扩展以填充目标元素。如果结果值会溢出目标 SEW，则保留最低有效的 SEW 位。

注：微架构可以使用与 `viota.m` 相同的数据通路实现 `vid.v` 指令，但隐式设置掩码源。

31.16 向量置换指令

提供了一系列置换指令，用于在向量寄存器内移动元素。

31.16.1 整数标量移动指令

整数标量读/写指令在标量 `x` 寄存器和向量寄存器的元素 0 之间传输单个值。该指令忽略 `LMUL` 和向量寄存器组。

```

vmv.x.s rd, vs2 # x[rd] = vs2[0] (vs1=0)
vmv.s.x vd, rs1 # vd[0] = x[rs1] (vs2=0)

```

`vmv.x.s` 指令将源向量寄存器索引 0 处的一个 SEW 宽元素复制到目标整数寄存器。如果 `SEW > XLEN`，则传输最低有效的 `XLEN` 位，忽略上面的 `SEW-XLEN` 位。如果 `SEW < XLEN`，则该值会被符号扩展到 `XLEN` 位。



即使 `vstart {ge} v1` 或 `v1=0`，`vmv.x.s` 也会执行操作。

`vmv.s.x` 指令将标量整数寄存器的值复制到目标向量寄存器的元素 0。如果 $SEW < XLEN$ ，则复制最低有效的位，忽略上面的 $XLEN-SEW$ 位。如果 $SEW > XLEN$ ，则该值会被符号扩展到 SEW 位。目标向量寄存器中其他元素 ($0 < \text{索引} < VLEN/SEW$) 将按照当前尾部无关/未干扰策略处理。如果 `vstart {ge} vl`，则不执行任何操作，目标寄存器不会被更新。



因此，当 `vl=0` 时，无论 `vstart` 如何，目标向量寄存器组中的元素都不会被更新。

与 `vmv.x.s` 和 `vmv.s.x` 的屏蔽版本 (`vm=0`) 对应的编码是保留的。

31.16.2 浮点标量移动指令

浮点标量读/写指令在标量 `f` 寄存器和向量寄存器的元素 0 之间传输单个值。该指令忽略 `LMUL` 和向量寄存器组。

```
vfmv.f.s rd, vs2 # f[rd] = vs2[0] (rs1=0)
vfmv.s.f vd, rs1 # vd[0] = f[rs1] (vs2=0)
```

`vfmv.f.s` 指令将源向量寄存器索引 0 处的单个 SEW 宽元素复制到目标标量浮点寄存器。



即使 `vstart {ge} vl` 或 `vl=0`，`vfmv.f.s` 也会执行操作。

`vfmv.s.f` 指令将标量浮点寄存器的值复制到目标向量寄存器的元素 0。目标向量寄存器中其他元素 ($0 < \text{索引} < VLEN/SEW$) 将按照当前尾部无关/未干扰策略处理。如果 `vstart {ge} vl`，则不执行任何操作，目标寄存器不会被更新。



因此，当 `vl=0` 时，无论 `vstart` 如何，目标向量寄存器组中的元素都不会被更新。

与 `vfmv.f.s` 和 `vfmv.s.f` 的屏蔽版本 (`vm=0`) 对应的编码是保留的。

31.16.3 向量滑动指令

滑动指令将元素在向量寄存器组中上下移动。



滑动操作比使用任意寄存器聚集指令更高效实现。实现可能会针对 `vslideup` 和 `vslidedown` 优化某些偏移量。特别是，2 的幂偏移可能比其他偏移操作要快得多。

对于所有 `vslideup`、`vslidedown`、`v[f]slidelup` 和 `v[f]slidedown` 指令，如果 `vstart {ge} vl`，则指令不执行任何操作，目标向量寄存器保持不变。



因此，当 $v1=0$ 时，无论 $vstart$ 如何，目标向量寄存器组中的元素都不会被更新。

尾部元素将按照尾部无关/未干扰策略处理。滑动指令可以被掩码，掩码元素 i 控制是否写入目标元素 i 。对于未激活元素，将遵循掩码未干扰/无关策略。

向量向上滑动指令

```
vslideup.vx vd, vs2, rs1, vm      # vd[i+x[rs1]] = vs2[i]
vslideup.vi vd, vs2, uimm, vm     # vd[i+uimm] = vs2[i]
```

对于 `vslideup`， $v1$ 中的值指定了写入目标元素的最大数量。目标的起始索引（OFFSET）可以通过 $rs1$ 指定的 x 寄存器中的无符号整数来指定，也可以通过一个 5 位的立即数来指定，并且该立即数会零扩展为 $XLEN$ 位。如果 $XLEN > SEW$ ，OFFSET 不会被截断为 SEW 位。如果未屏蔽且 $OFFSET < v1$ ，则写入目标元素 OFFSET 到 $v1-1$ 。

`vslideup` 行为对于目标元素 ($vstart < v1$)
 OFFSET 是偏移量，可以来自 x 寄存器或 5 位立即数
 $0 \leq i < \min(v1, \max(vstart, OFFSET))$ 不变
 $\max(vstart, OFFSET) \leq i < v1$ 如果 $v0.mask[i]$ 使能则 $vd[i] = vs2[i-OFFSET]$
 $v1 \leq i < VLMAX$ 遵循尾部策略

`vslideup` 的目标向量寄存器组不能与源向量寄存器组重叠，否则该指令编码为保留。



此非重叠约束避免了在执行过程中输入向量上的 WAR 危害，并且支持在 $vstart$ 非零时重新启动。

向量向下滑动指令

```
vslidedown.vx vd, vs2, rs1, vm    # vd[i] = vs2[i+x[rs1]]
vslidedown.vi vd, vs2, uimm, vm   # vd[i] = vs2[i+uimm]
```

对于 `vslidedown`， $v1$ 中的值指定了写入目标元素的最大数量。超过 $v1$ 的剩余元素将根据当前的尾部策略处理（参见 `sec-agnostic`）。

源的起始索引可以通过 $rs1$ 指定的 x 寄存器中的无符号整数来指定，也可以通过一个 5 位的立即数来指定，并且该立即数会零扩展为 $XLEN$ 位。如果 $XLEN > SEW$ ，偏移量不会被截断为 SEW 位。

`vslidedown` 对于源元素的行为（对于滑动中的元素 i ， $vstart < v1$ ）
 $0 \leq i+OFFSET < VLMAX$ $src[i] = vs2[i+OFFSET]$
 $VLMAX \leq i+OFFSET$ $src[i] = 0$

vslidedown 对于目标元素 i 的行为 (对于滑动中的元素 i , $vstart < vl$)	
$0 \leq i < vstart$	不变
$vstart \leq i < vl$	如果 $v0.mask[i]$ 使能则 $vd[i] = src[i]$
$vl \leq i < VLMAX$	遵循尾部策略

向量单元素向上滑动指令

提供了一些变体的滑动指令，它们每次只滑动一个元素，但也允许在空出的位置插入标量整数值。

```
vslidelup.vx vd, vs2, rs1, vm      # vd[0]=x[rs1], vd[i+1] = vs2[i]
```

vslidelup 指令将 x 寄存器的参数放置在目标向量寄存器组的第 0 位，前提是元素 0 是激活的，否则目标元素更新将按照当前的掩码无关/未干扰策略进行。如果 $XLEN < SEW$ ，值会被符号扩展到 SEW 位。如果 $XLEN > SEW$ ，最低有效的位将被复制，高于 SEW 的 $XLEN-SEW$ 位将被忽略。

剩余的 $vl-1$ 个激活元素 (从源向量寄存器组的索引 i 到目标向量寄存器组的索引 $i+1$) 会被复制过来。

vl 寄存器指定了从源值更新目标向量寄存器元素的最大数量，超过 vl 的剩余元素将根据当前的尾部策略处理 (参见 sec-agnostic)。

vslidelup behavior when $vl > 0$

$i < vstart$	不变
$0 = i = vstart$	如果 $v0.mask[i]$ 使能则 $vd[i] = x[rs1]$
$\max(vstart, 1) \leq i < vl$	如果 $v0.mask[i]$ 使能则 $vd[i] = vs2[i-1]$
$vl \leq i < VLMAX$	遵循尾部策略

向量浮点单元素向上滑动指令

vslidelup 指令要求目标向量寄存器组与源向量寄存器组不重叠，违背这一要求的指令编码是被保留的。

```
vfslidelup.vf vd, vs2, rs1, vm      # vd[0]=f[rs1], vd[i+1] = vs2[i]
```

vfslidelup 指令的定义类似于 vslidelup，但它的标量参数来自 f 寄存器。

向量单元素向下滑动指令 vslidedown 指令将源向量寄存器组中从索引 $i+1$ 开始的前 $vl-1$ 个激活元素的值复制到目标向量寄存器组中的索引 i 位置。 vl 寄存器指定了从源值更新目标向量寄存器元素的最大数量，超过 vl 的剩余元素将根据当前的尾部策略处理。

```
vslidedown.vx vd, vs2, rs1, vm      # vd[i] = vs2[i+1], vd[vl-1]=x[rs1]
```

vslidedown 指令将 x 寄存器的参数放置在目标向量寄存器的 $vl-1$ 位置，前提是元素 $vl-1$ 是激活的，否则目标元素更新将按照当前的掩码无关/未干扰策略进行。如果 $XLEN < SEW$ ，值会被符号扩展到 SEW 位。如果 $XLEN > SEW$ ，最低有效的位将被复制，高于 SEW 的 $XLEN-SEW$ 位将被忽略。

vslide1down 行为

$i < vstart$	不变
$vstart \leq i < vl-1$	如果 $v0.mask[i]$ 使能则 $vd[i] = vs2[i+1]$
$vstart \leq i = vl-1$	如果 $v0.mask[i]$ 使能则 $vd[vl-1] = x[rs1]$
$vl \leq i < VLMAX$	遵循尾部策略



vslide1down 指令可以用来将值加载到向量寄存器中，而无需使用内存，并且不会干扰其他向量寄存器。这为调试器提供了修改向量寄存器内容的路径，尽管速度较慢，需要多次重复调用 vslide1down。

```
vslide1down.vf vd, vs2, rs1, vm    # vd[i] = vs2[i+1], vd[vl-1]=f[rs1]
```

vslide1down 指令的定义类似于 vslide1down，但它的标量参数来自 f 寄存器。

向量寄存器收集指令从第一个源向量寄存器组读取元素，这些元素的位置由第二个源向量寄存器组给出。第二个向量中的索引值被视为无符号整数。源向量可以在任何索引 $< VLMAX$ 处读取，而不考虑 vl。要写入目标寄存器的最大元素数由 vl 给出，超过 vl 的剩余元素将根据当前的尾部策略处理（参见 sec-agnostic）。该操作可以被屏蔽，并且对于未激活元素会遵循掩码无关/未干扰策略。

31.16.4 向量寄存器聚集指令

```
vrgather.vv vd, vs2, vs1, vm    # vd[i] = (vs1[i] >= VLMAX) ? 0 : vs2[vs1[i]];
vrgather.ei16.vv vd, vs2, vs1, vm # vd[i] = (vs1[i] >= VLMAX) ? 0 : vs2[vs1[i]];
```

vrgather.vv 形式同时使用 SEW/LMUL 来处理数据和索引。vrgather.ei16.vv 形式在 vs2 中使用 SEW/LMUL 来处理数据，而在 vs1 中使用 EEW=16 和 EMUL = (16/SEW)*LMUL 来处理索引。



当 SEW=8 时，vrgather.vv 只能引用向量元素 0-255。vrgather.ei16 形式可以索引 64K 个元素，并且在 SEW > 16 时，也可以减少存储索引所需的寄存器容量。

如果元素索引超出范围 ($vs1[i] \geq VLMAX$)，则返回该元素的值为零。

还提供了向量-标量和向量-立即数形式的寄存器收集指令。这些指令在给定索引处从源向量中读取一个元素，并将该值写入目标向量寄存器的激活元素中。标量寄存器中的索引值和立即数（零扩展为 XLEN 位）被视为无符号整数。如果 XLEN > SEW，索引值不会被截断为 SEW 位。



这些形式允许将任何向量元素扩展到整个向量。

```
vrgather.vx vd, vs2, rs1, vm    # vd[i] = (x[rs1] >= VLMAX) ? 0 : vs2[x[rs1]]
vrgather.vi vd, vs2, uimm, vm  # vd[i] = (uimm >= VLMAX) ? 0 : vs2[uimm]
```

对于任何 `vrgather` 指令，目标向量寄存器组不能与源向量寄存器组重叠，违背这一要求的指令编码是被保留的。

向量压缩指令允许从源向量寄存器组中，由向量掩码寄存器选中的元素被打包到目标向量寄存器组的起始位置，形成连续的元素。

31.16.5 向量压缩指令

`vcompress.vm vd, vs2, vs1 #` 将 `vs2` 中选择的元素压缩到 `vd` 中，`vs1` 启用的元素

由 `vs1` 指定的向量掩码寄存器指示应该从向量寄存器组 `vs2` 的前 `v1` 个元素中提取哪些元素，并将它们打包到目标向量寄存器 `vd` 的起始位置。目标向量寄存器 `vd` 中的剩余元素将根据当前的尾部策略处理（参见 `sec-agnostic`）。

`vcompress` 指令的示例用法

```

8 7 6 5 4 3 2 1 0    元素编号

1 1 0 1 0 0 1 0 1    v0
8 7 6 5 4 3 2 1 0    v1
1 2 3 4 5 6 7 8 9    v2

                                vsetivli    t0, 9, e8, ml, tu, ma
                                vcompress.vm v2, v1, v0

1 2 3 4 8 7 5 2 0    v2
```

`vcompress` 被编码为一个未掩码的指令（`vm=1`）。等效的掩码指令（`vm=0`）是保留的。目标向量寄存器组不能与源向量寄存器组或源掩码寄存器重叠，违背这一要求的指令编码是被保留的。

`vcompress` 指令的异常总是报告 `vstart` 为 0。如果在非零 `vstart` 的情况下执行 `vcompress` 指令，则会引发非法指令异常。



虽然可能，但 `vcompress` 是少数几个较难在非零 `vstart` 的情况下重启的指令之一，因此假设实现通常会选择从元素 0 开始重启，而不是从非零的 `vstart` 开始。这意味着，`vstart` 之后的目标寄存器中的元素已经被更新。

向量合成指令 `vdecompress` 没有提供逆向的 `vdecompress`，因为这个操作可以通过 `iota` 和一个掩码的 `vrgather` 来方便地合成：

`vdecompress` 的期望功能

```

7 6 5 4 3 2 1 0    # vid

    e d c b a      # 打包的 5 元素向量
1 0 0 1 1 1 0 1    # 掩码向量，包含 8 个元素
p q r s t u v w    # vdecompress 之前的目标寄存器

e q r d c b v a    # vdecompress 的结果
```

```

# v0 保存掩码
# v1 保存打包的数据
# v11 保存输入扩展向量和结果
viota.m v10, v0          # 计算 v0 中 mask 的 iota
vrgather.vv v11, v1, v10, v0.t # 在目标寄存器中扩展

p q r s t u v w   # v11 目标注册
    e d c b a     # v1 源向量
1 0 0 1 1 1 0 1   # v0 掩码向量

4 4 4 3 2 1 1 0   # viota.m 的 v10 结果
e q r d c b v a    # 使用 viota.m 在掩码下经过 vrgather 后的 v11 目标寄存器

```

31.16.6 整个向量寄存器移动

整个向量寄存器移动 `vmv<nr>r.v` 指令复制整个向量寄存器（即，所有 VLEN 位），并且可以复制整个向量寄存器组。操作码中的 `nr` 值指定要复制的独立向量寄存器的数量，即 NREG。指令的操作方式就像是 `EEW=SEW`，`EMUL = NREG`，实际长度 `evl= EMUL * VLEN/SEW`。

这些指令旨在帮助编译器在不需要知道或改变 `v1` 或 `vtype` 的情况下，进行向量寄存器的重新排列。



通常情况下，如果 `vstart >= v1`，则不会写入任何元素，但这些指令不适用这一规则。相反，如果 `vstart >= evl`，则不会写入任何元素。

如果 `vd` 等于 `vs2`，则该指令是一个空操作（NOP），但它作为对实现的提示，表示下一个对寄存器组的访问将使用与 SEW 相等的 EEW。

该指令被编码为 `OPIVI` 指令。要复制的向量寄存器数量编码在 `simm` 字段的最低三位（`simm[2:0]`），并且使用与内存指令的 `nf[2:0]` 字段相同的编码方式（参见表 31.11，即 `simm[2:0] = NREG-1`。NREG 的值必须为 1、2、4 或 8，并且 `simm[4:0]` 的值除了 0、1、3 和 7 之外都是保留的。

未来的扩展可能支持移动其他数量的寄存器。



该指令使用与 `vsmul` 指令相同的 `funct6` 编码，但带有立即数操作数，并且仅支持未掩码版本（`vm=1`）。此编码方式的选择是因为它接近于 `vmerge` 编码，并且不太可能 `vsmul` 指令会从立即数形式中受益。

`vmv<nr>r.v vd, vs2` # 通用形式

`vmv1r.v v1, v2` # 复制 `v1 = v2`

`vmv2r.v v10, v12` # 复制 `v10 = v12; v11 = v13`

```

vmv4r.v v4, v8    # 复制 v4 = v8; v5 = v9; v6 = v10; v7 = v11
vmv8r.v v0, v8    # 复制 v0 = v8; v1 = v9; ...; v7 = v15

```

源向量寄存器和目标向量寄存器的编号必须与向量寄存器组的大小对齐，其他向量寄存器编号的编码是保留的。



未来的扩展可能会放宽向量寄存器对齐的限制

31.17 异常处理

在向量指令的陷阱过程中（由同步异常或异步中断引起），现有的 `*epc` 控制状态寄存器会被写入指向触发陷阱的向量指令的指针，同时 `vstart` 控制状态寄存器包含发生陷阱时的元素索引。



我们选择添加一个 `vstart` 控制状态寄存器，以允许从部分执行的向量指令恢复，从而减少中断延迟并简化持续进展保证。这类似于 IBM 3090 向量功能中的方案。为了确保在没有 `vstart` 控制状态寄存器的情况下前进，实施者必须保证整个向量指令总是能够原子性地完成，而不会触发陷阱。特别是在有跨步或散播/聚集操作以及按需分页虚拟内存的情况下，确保这一点特别困难。

31.17.1 精确向量陷阱

我们假设大多数具有按需分页的监控模式环境将需要精确的向量陷阱。精确向量陷阱要求：

- 所有比触发陷阱的向量指令更老的指令必须已经提交了它们的结果。
- 所有比触发陷阱的向量指令更年轻的指令不应修改架构状态。
- 触发陷阱的向量指令中影响 `vstart` 控制状态寄存器中索引之前的元素的操作，必须已提交其结果。
- 触发陷阱的向量指令中影响 `vstart` 控制状态寄存器索引及之后的元素的操作，除非重新启动并完成受影响的向量指令仍然能够生成正确的最终状态，否则不应修改架构状态。

我们放宽了最后一条要求，允许在报告陷阱时 `vstart` 后面的元素已被更新，只要从给定的 `vstart` 重新执行该指令能正确覆盖这些元素。

在幂等内存区域中，向量存储指令可能已更新内存中超出引发同步陷阱元素的部分。对于非幂等内存区域，在向量存储指令引发同步陷阱时，索引等于或大于导致同步陷阱的元素的区域不得被更新。

除非上述特别说明，否则向量指令可以覆盖它们的输入，因此在大多数情况下，向量指令的重启必须从 `vstart` 元素索引开始。然而，也有一些情况下禁止这种覆盖，以便使向量指令的执行是幂等的，从而能够从更早的索引位置重新启动。实施者必须确保 `vstart` 报告的元素或段最终能够保证前进。

31.17.2 不精确向量陷阱

不精确向量陷阱是指那些不精确的陷阱。特别地，比 `*epc` 更新的指令可能已提交结果，而比 `*epc` 更早的指令可能未完成执行。不精确陷阱主要用于那些报告错误并终止执行是适当响应的情况。



一个配置文件可能指定中断是精确的，而其他陷阱是不精确的。我们假设许多嵌入式实现会在发生致命错误时仅为向量指令生成不精确的陷阱，因为它们不需要可恢复的陷阱。

不精确陷阱应在同步向量异常引发的陷阱中报告 `vstart` 中的出错元素。当前标准扩展中不支持不精确陷阱。

31.17.3 可选择的精确/不精确陷阱

某些配置可能选择提供一个特权模式位，用于在精确和不精确的向量陷阱之间进行选择。不精确模式将以高性能运行，但可能使错误原因的辨别变得困难，而精确模式则运行较慢，但支持调试错误，尽管有可能无法遇到与不精确模式相同的错误。该机制在当前标准扩展中没有定义。

31.17.4 可交换陷阱

另一种陷阱模式可以支持向量单元中的可交换状态，其中在触发陷阱时，特殊指令可以保存和恢复向量单元的微体系结构状态，以便在不精确陷阱周围继续正确执行。该机制在当前标准扩展中没有定义。



未来的扩展可能会定义一种标准方式来保存和恢复向量单元实现中的非透明微体系结构状态，以支持在不精确陷阱情况下的上下文切换。

31.18 标准向量扩展

本节描述了标准向量扩展。用于嵌入式用途的一组较小扩展以 `Zve` 前缀命名，而为应用处理器设计的较大向量扩展则命名为单字母的 `V` 扩展。还提供了一组带有 `Zvl` 前缀的向量长度扩展名称。初始向量扩展旨在作为各种领域（包括加密和机器学习）中其他向量扩展的基础。

31.18.1 `Zvl*`：最小向量长度标准扩展

所有标准向量扩展都有一个最低要求的 `VLEN`，如下所述。提供了一组向量长度扩展，用于增加向量扩展的最小向量长度。



向量长度扩展可以用于指定额外的软件或体系结构配置要求，或者用于展示硬件能力。

表31.15 向量长度扩展

扩展	最小长度
<code>Zvl32b</code>	32
<code>Zvl64b</code>	64
<code>Zvl128b</code>	128
<code>Zvl256b</code>	256
<code>Zvl512b</code>	512

Zv11024b	1024
----------	------

更长的向量长度扩展应遵循相同的模式。

每个向量长度扩展实际上包含所有较短的向量长度扩展。



扩展名称的语法正在修订，这些名称可能会发生变化。尾部的b将用于区分数值字段和版本号。

对于任何标准向量扩展来说，并不要求显式使用 Zv132b 扩展字符串，因为它们都有效地要求至少有此最小值，但该字符串在说明硬件功能时可能很有用。

31.18.2 Zve*: 嵌入式处理器的向量扩展

以下五个标准扩展提供了不同程度的向量支持，旨在与嵌入式处理器一起使用。任何这些扩展可以添加到 XLEN=32 或 XLEN=64 的基本 ISA 中。表中列出了每个扩展的最小 VLEN、支持的 EEW 以及支持的浮点类型。

表31.16 嵌入向量扩展

扩展	最小长度	支持	FP32	FP64
Zve32x	32	8, 16, 32	N	N
Zve32f	32	8, 16, 32	Y	N
Zve64x	64	8, 16, 32, 64	N	N
Zve64f	64	8, 16, 32, 64	Y	N
Zve64d	64	8, 16, 32, 64	Y	Y

Zve32f 和 Zve64x 扩展依赖于 Zve32x 扩展。

Zve64f 扩展依赖于 Zve32f 和 Zve64x 扩展。

Zve64d 扩展依赖于 Zve64f 扩展。

所有 Zve* 扩展都支持精确陷阱。



目前没有标准支持处理不精确的陷阱，因此标准扩展必须提供精确的陷阱。

所有 Zve* 扩展都支持 EEW 为 8、16 和 32，Zve64* 扩展还支持 EEW 为 64。

所有 Zve* 扩展都支持向量配置指令（参见第 31.6）。

所有 Zve* 扩展都支持所有向量加载和存储指令（参见第 31.7），除了在 XLEN=32 时，Zve64* 扩展不支持索引值为 EEW=64。

所有 Zve* 扩展都支持所有向量整数指令（参见第 31.11），但对于 EEW=64，Zve64* 不包括返回乘积高字的 vmulh 整数乘法变体（vmulh.vv、vmulh.vx、vmulhu.vv、vmulhu.vx、vmulhsu.vv、vmulhsu.vx）。



产生乘积的高字可能需要大量额外的逻辑，对于较大的 EEW，这可能会增加很多门电路。

所有 Zve* 扩展都支持所有向量定点运算指令（参见第 31.12），但在 Zve64* 中，EEW=64 时不包括 vsmul.vv 和 vsmul.vx。



与 vmulh 类似，vsmul 也需要大量额外的逻辑，且 64 位定点乘法相对较少见。

所有 Zve* 扩展都支持所有向量整数单宽度和扩展宽度的规约操作（参见第 31.14.1 和第 31.14.2 节）。

所有 Zve* 扩展都支持所有向量掩码指令（参见第 31.15 节）。

所有 Zve* 扩展都支持所有向量置换指令（参见第 31.16 节），但 Zve32x 和 Zve64x 不包括带有浮点操作数的置换指令，Zve64f 不包括带有 EEW=64 浮点操作数的置换指令。

Zve32x 扩展依赖于 Zicsr 扩展。Zve32f 和 Zve64f 扩展依赖于 F 扩展，并实现所有用于浮点操作数的向量浮点指令（参见第 31.13 节），支持 EEW=32 的浮点操作数。对于 EEW=32，支持向量单宽度浮点减法操作（参见第 31.14.3 节）。

Zve64d 扩展依赖于 D 扩展，并实现所有用于浮点操作数的向量浮点指令（参见第 31.13 节），支持 EEW=32 或 EEW=64 的浮点操作数（包括扩展指令以及 FP32 与 FP64 之间的转换）。支持 EEW=32 和 EEW=64 的向量单宽度浮点减法操作（参见第 31.14.3 节），以及从 FP32 到 FP64 的扩展减法操作。

31.18.3 扩展：面向应用处理器的向量扩展

V 向量扩展支持精确异常。

V 扩展依赖于 Zvl128b 和 Zve64d 扩展。NOTE: VLEN=128 是应用处理器的折中选择：更大的 VLEN 可在某些短向量情况下避免分块代码。但同时增加了最小实现的规模。使用更大的 LMUL 可避免分块，但会减少可用的向量寄存器组：例如，LMUL=8 时，最多可处理 16 个 64 位元素的向量，无需分块，但仅能使用 4 组向量寄存器。

V 扩展支持 EEW 取值 8、16、32 和 64。

V 扩展支持向量配置指令。

V 扩展支持所有向量加载/存储指令，但当 XLEN=32 时，不支持索引值的 EEW=64。

V 扩展支持所有向量整数指令。

V 扩展支持所有向量定点算术指令。

V 扩展支持所有向量整数单宽与加宽归约操作。

V 扩展支持所有向量掩码指令。

V 扩展支持所有向量置换指令。

V 扩展依赖 F 和 D 扩展，并实现所有向量浮点指令，适用于 EEW=32 或 EEW=64 的浮点操作数（包括加宽指令以及 FP32 与 FP64 之间的转换）。支持 EEW=32 和 EEW=64 的单宽浮点归约，以及 FP32 到 FP64 的加宽归约。



与其他 RISC-V 扩展一样，可以在同一 ISA 字符串中包含重叠的扩展。例如，RV64GCV 和 RV64GCV_Zve64f 都是有效且等效的 ISA 字符串，RV64GCV_Zve64f_Zve32x_Zvl128b 也是有效的。

31.18.4 Zvfhmin: 最小半精度浮点向量扩展

Zvfhmin 扩展提供对 IEEE 754-2008 binary16 向量的最小支持，添加了 binary16 和 binary32 之间的转换指令。当实现 Zvfhmin 扩展时，在 SEW=16 时 `vfwcvt.f.f.v` 和 `vfncvt.f.f.w` 指令将被定义。这些指令的 EEW=16 浮点操作数采用 binary16 格式。

31.18.5 Zvfh: 半精度浮点向量扩展

Zvfh 扩展提供对 IEEE 754-2008 binary16 向量的支持。当实现 Zvfh 扩展时，在 SEW=16 时，以下章节中的所有指令将被定义第 31.13 节、第 31.14.3 节、第 31.14.4 节、第 31.13.16 节、第 31.16.3.4 节和第 31.16.3.6 节。这些指令的 EEW=16 浮点操作数采用 binary16 格式。

此外，Zvfh 扩展提供 8 位整数与 binary16 之间的转换：

- 浮点到整数的窄化转换 (`vfncvt[.rtz].x[u].f.w`)
- 整数到浮点的加宽转换 (`vfwcvt.f.x[u].v`)
- 这些指令在 SEW=8 时被定义。

Zvfh 扩展依赖于 Zve32f 和 Zfhmin 扩展。



要求基本的标量半精度支持使得 Zvfh 的向量-标量指令更具实用性。我们曾考虑要求更完整的标量半精度支持，但我们认为，对于许多半精度向量工作负载而言，在单精度下执行标量计算已经足够。

31.19 向量指令列表

Integer					Integer					FP				
funct3					funct3					funct3				
OPIVV	V				OPMVV	V				OPFVV	V			
OPIVX		X			OPMVX		X			OPFVF		F		
OPIVI			I											

funct6					funct6					funct6				
000000	V	X	I	vadd	000000	V		vredsum		000000	V	F	vfadd	
000001					000001	V		vredand		000001	V		vfredsum	
000010	V	X		vsub	000010	V		vredor		000010	V	F	vfsub	
000011		X	I	vrsb	000011	V		vredxor		000011	V		vfredsum	
000100	V	X		vminu	000100	V		vredminu		000100	V	F	vfmin	
000101	V	X		vmin	000101	V		vredmin		000101	V		vfredmin	
000110	V	X		vmaxu	000110	V		vredmaxu		000110	V	F	vfmax	
000111	V	X		vmax	000111	V		vredmax		000111	V		vfredmax	
001000					001000	V	X	vaaddu		001000	V	F	vfsgnj	
001001	V	X	I	vand	001001	V	X	vaadd		001001	V	F	vfsgnjn	
001010	V	X	I	vor	001010	V	X	vasubu		001010	V	F	vfsgnjx	

funct6					funct6					funct6				
001011	V	X	I	vxor	001011	V	X	vasub		001011				
001100	V	X	I	vrgather	001100					001100				
001101					001101					001101				
001110		X	I	vslideup	001110		X	vslideup		001110		F	vfslideup	
001111	V			vrgather										
				ei16										
001111		X	I	vslidedown	001111		X	vslide1down		001111		F	vfslide1down	

funct6					funct6					funct6				
010000	V	X	I	vadc	010000	V		VWXUN ARYO	010000	V			VWFUN ARYO	
					010000		X	VRXUN ARYO	010000		F		VRFUN ARYO	
010001	V	X	I	vmadc	010001				010001					
010010	V	X		vsbc	010010	V		VXUNA RYO	010010	V			VFUNA RYO	
010011	V	X		vmsbc	010011				010011	V			VFUNA RY1	
010100					010100	V		VMUNA RYO	010100					
010101					010101				010101					
010110					010110				010110					
010111	V	X	I	vmerge/ vmv	010111	V		vcompr ess	010111		F		vfmerge /vfmv	
011000	V	X	I	vmseq	011000	V		vmandn	011000	V	F		vmfeq	
011001	V	X	I	vmsne	011001	V		vmand	011001	V	F		vmfle	
011010	V	X		vmsltu	011010	V		vmor	011010					
011011	V	X		vmslt	011011	V		vmxor	011011	V	F		vmflt	
011100	V	X	I	vmsleu	011100	V		vmorn	011100	V	F		vmfne	
011101	V	X	I	vmsle	011101	V		vmnand	011101		F		vmfgt	
011110		X	I	vmsgtu	011110	V		vmnor	011110					
011111		X	I	vmsgt	011111	V		vmxnor	011111		F		vmfge	

funct6					funct6				funct6			
100000	V	X	I	vsaddu	100000	V	X	vdivu	100000	V	F	vfdiv
100001	V	X	I	vsadd	100001	V	X	vdiv	100001		F	vfrdiv
100010	V	X		vssubu	100010	V	X	vremu	100010			
100011	V	X		vssub	100011	V	X	vrem	100011			
100100					100100	V	X	vmulhu	100100	V	F	vfmul
100101	V	X	I	vsll	100101	V	X	vmul	100101			
100110					100110	V	X	vmulhs u	100110			
100111	V	X		vsmul	100111	V	X	vmulh	100111		F	vfrsub

funct6					funct6					funct6			
100111			I	vmv<nr>r									
101000	V	X	I	vsrl	101000					101000	V	F	vfmadd
101001	V	X	I	vsra	101001	V	X	vmadd		101001	V	F	vfnmadd
101010	V	X	I	vssrl	101010					101010	V	F	vfmsub
101011	V	X	I	vssra	101011	V	X	vnmsub		101011	V	F	vfnmsub
101100	V	X	I	vnsrl	101100					101100	V	F	vfmacc
101101	V	X	I	vnsra	101101	V	X	vmacc		101101	V	F	vfnmacc
101110	V	X	I	vnclipu	101110					101110	V	F	vfmsac
101111	V	X	I	vnclip	101111	V	X	vnmsac		101111	V	F	vfnmsac

funct6					funct6					funct6			
110000	V			vwredsumu	110000	V	X	vwaddu		110000	V	F	vwadd
110001	V			vwredsum	110001	V	X	vwadd		110001	V		vwredsum
110010					110010	V	X	vwsubu		110010	V	F	vfwsu
110011					110011	V	X	vwsu		110011	V		vwredo
110100					110100	V	X	vwaddu.w		110100	V	F	vwadd.w
110101					110101	V	X	vwadd.w		110101			
110110					110110	V	X	vwsu.w		110110	V	F	vfwsu.w
110111					110111	V	X	vwsu.w		110111			
111000					111000	V	X	vwmulu		111000	V	F	vfwmul
111001					111001					111001			
111010					111010	V	X	vwmulu		111010			
111011					111011	V	X	vwmul		111011			
111100					111100	V	X	vwmaccu		111100	V	F	vfwmac
111101					111101	V	X	vwmacc		111101	V	F	vfwnmac
111110					111110		X	vwmaccus		111110	V	F	vfwnsac
111111					111111	V	X	vwmaccsu		111111	V	F	vfwnmsac

表31. 16 VRXUNARY0编码空间

vs2	
00000	vmv.s.x

表31. 17 VWXUNARY0编码空间

vs1	
00000	vmv.x.s
10000	vcpop
10001	vfirst

表31. 18 VXUNARY0编码空间

vs1	
00010	vzext.vf8
00011	vsext.vf8
00100	vzext.vf4
00101	vsext.vf4
00110	vzext.vf2
00111	vsext.vf2

表31. 19 VRFUNARY0编码空间

vs2	
00000	vfmv.s.f

表31. 20 VWFUNARY0编码空间

vs1	
00000	vfmv.f.s

表31. 21 VFUNARY0编码空间

vs1	name
single-width converts	
00000	vfcvt.xu.f.v
00001	vfcvt.x.f.v
00010	vfcvt.f.xu.v
00011	vfcvt.f.x.v
00110	vfcvt.rtz.xu.f.v
00111	vfcvt.rtz.x.f.v
widening converts	
01000	vfwcvt.xu.f.v
01001	vfwcvt.x.f.v
01010	vfwcvt.f.xu.v
01011	vfwcvt.f.x.v
01100	vfwcvt.f.f.v
01110	vfwcvt.rtz.xu.f.v
01111	vfwcvt.rtz.x.f.v

vs1	name
narrowing converts	
10000	vfncvt.xu.f.w
10001	vfncvt.x.f.w
10010	vfncvt.f.xu.w
10011	vfncvt.f.x.w
10100	vfncvt.f.f.w
10101	vfncvt.rod.f.f.w
10110	vfncvt.rtz.xu.f.w
10111	vfncvt.rtz.x.f.w

表31. 22 VFUNARY1编码空间

vs1	name
00000	vfsqrt.v
00100	vfrsqrt7.v
00101	vfrec7.v
10000	vfclass.v

表31. 23 VMUNARY0编码空间

vs1	
00001	vmsbf
00010	vmsof
00011	vmsif
10000	viota
10001	vid

32 加密扩展：标量和熵源指令，V1.0.1

32.1 更新日志

版本号	变更内容
v1.0.1	修复拼写错误，明确在 RV64 中 c.srli、c.srai 和 c.slli 是 Zkt 指令
v1.0.0	初始版本

32.2 介绍

本文档描述了 RISC-V 的标量加密扩展。所有在此描述的指令使用通用的 X 寄存器，并遵循 2-读-1-写 的寄存器访问约束。这些指令设计轻量，并适用于 32 位和 64 位基础架构；从嵌入式 IoT 类核到不实现向量单元的大型应用类核。

本文档还描述了熵源的架构接口，可用于生成加密密钥。见 cryptoscalares。

它还包含一个机制，允许核实现者在 cryptoscalarzkt 中提供恒定时间执行保证。另一份配套文档第二卷：向量指令描述了基于 RISC-V 向量扩展的指令提案。向量加密扩展目前正在等待基础向量扩展稳定下来。我们预计将在 2021 年第四季度或 2022 年第一季度认真开展这项工作。

32.2.1 目标受众

加密是一个高度专业化的领域，需要不同背景的人士协同合作，才能确保其安全性和高效性。我们力求编写出易于理解的规范，但我们也清楚，动机以及算法或其他规范、标准的引用，对于非专业读者而言可能较为陌生。

预计来自不同背景的各类读者将阅读和实施本规范。我们力图概括这些背景，并简要说明我们希望读者理解的内容，以及这些内容与规范的关系。我们希望这能帮助读者明确哪些规范部分对他们尤为相关，哪些可以（在确保安全性的前提下）忽略或推荐给其他同事。

密码学家和密码软件开发人员。密码学家和密码软件开发人员是我们期望使用本规范中指令编写代码的目标人群。他们理解我们包含这些指令的动机，并熟悉我们引用的大多数算法和外部标准。

计算机架构师。我们了解架构师不一定具有密码学背景，但仍希望架构师能够检查这些指令并发现实现问题，理解指令在上下文中的使用方式，并就如何最好地满足密码学的功能需求提供建议。

数字设计工程师和微架构师。这些人将负责在核内部实现规范。同样，我们了解他们不一定具有密码学专业背景，但希望他们能够解释规范并预见任何硬件实现问题，例如高频设计考虑因素，或存在延迟/面积权衡等。具体来说，他们应该了解有关在硬件中高效实现 AES 和 SM4 SBoxes 的文献。

验证工程师。这些人负责确保在硬件中正确实现扩展。我们了解他们不一定具有密码学背景，但希望他们从规范中发现有趣的测试用例。理解其实际使用情况将有助于此。

这些绝不是唯一关心规范的人，但他们是我们在编写时最考虑的人。

32.2.2 Sail 规范

RISC-V 维护一个正式的 ISA 规范 (<https://github.com/riscv/sail-riscv>)，使用 Sail ISA 规范语言实现。请注意，Sail 指的是规范语言本身，并且有一个 RISC-V 模型，使用 Sail 编写。不正确地称之为 Sail 模型是不明确的，因为有许多使用 Sail 实现的不同 ISA 模型。我们将 RISC-V 的 Sail 实现称为 RISC-V Sail 模型。

加密扩展使用来自实际模型的内联 Sail 代码片段来提供指令功能的规范描述。每个指令都附有其在 Sail 中的表达式，并包括对支持函数的调用，这些函数过于冗长，无法直接包含于规范中。这些支持代码列在 `cryptoscalarappxsail`。推荐阅读 Sail 手册 (<https://github.com/remis-project/sail/blob/sail2/manual.pdf>) 以便最好地理解代码片段。

请注意，本文档仅包含正式模型的一个子集：请参阅正式模型 (<https://github.com/riscv/sail-riscv>) 以获取完整模型。

32.2.3 政策

在创建此提案时，我们尝试遵循以下政策：

- * 在以下选择之间：

- 支持算法的多样化实现策略

- 或

- 支持单一实现风格，该风格更高效/成本更低；密码扩展将选择更受约束但更高效的选项。这符合 RISC-V 规范其他部分的常见模式，其中给出了执行特定任务的推荐（但不要求）指令序列作为示例，以便硬件和软件实现者都可以仅针对单一用例进行优化。

- * 扩展将设计为良好地支持现有标准化的加密构造。它不会尝试支持提议的标准，或仅存在于学术界的加密构造。与 RISC-V 加密扩展标准化同时或之后确定的加密标准将通过未来的补充或版本处理。预计 NIST 轻量级加密竞赛和 NIST 后量子加密竞赛可能会以这种方式处理，具体取决于时间表。

- * 历史上，有一些讨论 关于在通用计算中支持的新操作如何启用新的加密算法基础。该标准不会试图预测可能作为未来加密结构构建块的有用低级操作。

- * 关于旁道攻击防护：在相关情况下，提议的指令必须旨在消除任何时间旁道攻击的可能性。对于基于功耗或电磁（EM）测量的旁道攻击，该扩展将不旨在支持那些在 ISA 抽象层之上的防护措施。在相关情况下，将提供如何使微架构以抗功耗/电磁旁道攻击的方式实现指令的建议。

32.3 扩展概述

标量加密指令集扩展引入的扩展组在此列出。检测单个加密扩展使用统一的软件 RISC-V 发现方法。



在撰写本文时，这些发现机制仍在进行中。

专用加密和解密指令被分为不同的功能组，因为某些用例（例如，TLS 1.3 中的 Galois/Counter 模式）不需要解密功能。

NIST 和 ShangMi 算法套件被分开，因为它们的有效在很大程度上取决于设备预期运行的国家。NIST 密码是大多数标准化互联网协议的一部分，而 ShangMi 密码在中国使用是必需的。

32.3.1 Zbkb - 用于加密的位操作指令

这些是位操作扩展 Zbb 的子集，特别适用于加密。

zbbk 中的所有指令在本文档中都有完整的规范，包括那些不在初始位操作批准包中的指令。

这是为了使当前规范作为独立文档完整。不可避免地，位操作和标量加密规范文档之间可能会有小的差异，因为它们以不同的速度移动。当这种情况发生时，假设位操作规范具有最新版本的位操作指令。这是在标量加密和位操作在公开审查前快速迭代时的一个不幸但必要的权宜之计。

32.3.2 Zbkc – 无进位乘法指令

用于 Galois/Counter 模式的恒定时间无进位乘法。这些与 zbbk 分开，因为它们具有相当大的实现开销，无法在其他指令中摊销。



这些指令在第一个位操作批准包中为 Zbc 扩展定义。zbbc 中的所有指令在本文档中都有完整的规范，包括那些不在初始位操作批准包中的指令。这是为了使当前规范作为独立文档完整。不可避免地，位操作和标量加密规范文档之间可能会有小的差异，因为它们以不同的速度移动。当这种情况发生时，假设位操作规范具有最新版本的位操作指令。这是在标量加密和位操作在公开审查前快速迭代时的一个不幸但必要的权宜之计。

32.3.3 Zbkx – 交叉开关置换指令

这些指令对于以恒定时间实现 SBoxes 并可能具有 DPA 保护非常有用。这些与 zbbk 分开，因为它们具有无法在其他指令中摊销的实现开销。



所有这些指令都缺席于第一个位操作批准包。因此，zbxk 中的所有指令在本文档中都有完整的规范。这是为了使当前规范作为独立文档完整。不可避免地，位操作和标量加密规范文档之间可能会有小的差异，因为它们以不同的速度移动。当这种情况发生时，假设位操作规范具有最新版本的位操作指令。这是在标量加密和位操作在公开审查前快速迭代时的一个不幸但必要的权宜之计。

32.3.4 Zknd – NIST 套件：AES 解密

用于加速 AES 块密码的解密和密钥调度功能的指令。

表 32.1 AES 块助记表

RV32	RV64	助记符	指令
✓		aes32dsi	高级加密标准 (AES) 最终轮解密 (RV32 架构)
✓		aes32dsmi	高级加密标准 (AES) 中间轮解密 (RV32 架构)
	✓	aes64ds	高级加密标准 (AES) 最终轮解密 (RV64 架构)
	✓	aes64dsm	高级加密标准 (AES) 中间轮解密 (RV64 架构)
	✓	aes64im	高级加密标准 (AES) 解密密钥调度列混合 (RV64 架构)
	✓	aes64ksli	高级加密标准 (AES) 密钥调度指令 1 (RV64 架构)
	✓	aes64ks2	高级加密标准 (AES) 密钥调度指令 2 (RV64 架构)



insns-aes64ksli 和 insns-aes64ks2 指令同时存在于 zknd 和 zkne 扩展中。

32.3.5 Zkne – NIST 套件：AES 加密

表32.2用于加速 AES 块密码的加密和密钥调度功能的指令

RV32	RV64	助记符	指令
√		aes32esi	高级加密标准（AES）最终轮加密（RV32 架构）
√		aes32esmi	高级加密标准（AES）中间轮加密（RV32 架构）
	√	aes64es	高级加密标准（AES）最终轮加密指令（RV64 架构）
	√	aes64esm	高级加密标准（AES）中间轮加密指令（RV64 架构）
	√	aes64ksli	高级加密标准（AES）密钥调度指令 1（RV64 架构）
	√	aes64ks2	高级加密标准（AES）密钥调度指令 2（RV64 架构）



insns-aes64ksli, aes64ksli 和 insns-aes64ks2, aes64ks2 指令同时存在于 zknd 和 zkne 扩展中。

32.3.6 Zknh – NIST 套件：哈希函数指令

表32.3用于加速 SHA2 系列加密哈希函数的指令

RV32	RV64	助记符	指令
√	√	sha256sig0	SHA2 – 256 算法的 Sigma0 指令
√	√	sha256sig1	SHA2 – 256 算法的 Sigma1 指令
√	√	sha256sum0	SHA2 – 256 算法的 Sum0 指令
√	√	sha256sum1	SHA2 – 256 算法的 Sum1 指令
√		sha512sig0h	SHA2 – 512 算法的 Sigma0 高位指令（RV32 架构）
√		sha512sig0l	SHA2 – 512 算法的 Sigma0 低位指令（RV32 架构）
√		sha512sig1h	SHA2 – 512 算法的 Sigma1 高位指令（RV32 架构）
√		sha512sig1l	SHA2 – 512 算法的 Sigma1 低位指令（RV32 架构）
√		sha512sum0r	SHA2 – 512 算法的 Sum0 指令（RV32 架构）
√		sha512sum1r	SHA2 – 512 算法的 Sum1 指令（RV32 架构）
	√	sha512sig0	SHA2 – 512 算法的 Sigma0 指令（RV64 架构）
	√	sha512sig1	SHA2 – 512 算法的 Sigma1 指令（RV64 架构）
	√	sha512sum0	SHA2 – 512 算法的 Sum0 指令（RV64 架构）
	√	sha512sum1	SHA2 – 512 算法的 Sum1 指令（RV64 架构）

32.3.7 Zksed – ShangMi 套件：SM4 块密码指令

用于加速 SM4 块密码的指令。注意，与 AES 不同，此密码使用相同的核操作进行加密和解密，因此只有一个扩展。

表32.4 用于加速 SM4 哈希函数的指令

RV32	RV64	助记符	指令
√	√	sm4ed	SM4 加密 / 解密指令

√	√	sm4ks	SM4 密钥调度指令
---	---	-------	------------

32.3.8 Zksh – ShangMi 套件：SM3 哈希函数指令

表32.5 用于加速 SM3 哈希函数的指令

RV32	RV64	助记符	指令
√	√	sm3p0	SM3 算法 P0 变换
√	√	sm3p1	SM3 算法 P1 变换

32.3.9 Zkr – 熵源扩展

熵源扩展在地址 0x015 定义了 seed CSR。此 CSR 提供最多 16 位物理 entropy 位，可用于生成加密随机位。

请参阅 cryptoscalares 以获取规范和访问控制说明。cryptoscalarappxes 包含设计原理和对实现者的进一步建议。

32.3.10 Zkn – NIST 算法套件

此扩展是以下其他扩展集的简写：

表32.6 NIST 算法套件

包含的扩展	描述
[zbkb]	用于加密的位操作指令
[zbkc]	无进位乘法指令
[zbkx]	交叉开关置换指令
Zkne	AES 加密指令
Zknd	AES 解密指令
Zknh	SHA2 哈希函数指令

实现 Zkn 的核必须实现上述所有扩展。

32.3.11 Zks – ShangMi 算法套件

表32.7 ShangMi 算法套件

包含的扩展	描述
[zbkb]	用于加密的位操作指令
[zbkc]	无进位乘法指令
[zbkx]	交叉开关置换指令
zkse	SM4 块密码指令
zksh	SM3 哈希函数指令

实现 Zks 的核必须实现上述所有扩展。

32.3.12 Zk – 标准标量加密扩展

表32.8 标准标量加密扩展

包含的扩展	描述
-------	----

zkn	NIST 算法套件扩展
zkr	熵源扩展
cryptoscalarzkt, Zkt	数据独立执行延迟扩展

实现 Zk 的核必须实现上述所有扩展。

32.3.13 Zkt – 数据独立执行延迟

此扩展允许 CPU 实现者向加密软件开发者表明，保证一部分 RISC-V 指令的执行延迟与其操作的数据值无关。此扩展的完整描述可以在 cryptoscalarzkt 中找到。

32.4 指令

32.4.1 aes32dsi

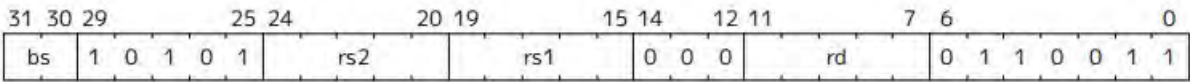
简述：

RV32 的 AES 最终轮解密指令。

助记符：

aes32dsi rd, rs1, rs2, bs

编码：



描述：

此指令根据 bs 的值，从源寄存器 rs2 中选取一个字节。然后，对这个字节应用逆向 AES S 盒（SBox）操作，之后将结果与 rs1 进行异或运算。此指令必须始终以这样的方式实现，即它的执行延迟不依赖于所处理的数据。

操作：

```
function clause execute (AES32DSI (bs,rs2,rs1,rd)) = {
  let shamt    : bits( 5) = bs @ 0b000; /* shamt = bs*8 */
  let si       : bits( 8) = (X(rs2)[31..0] >> shamt)[7..0]; /* SBox Input */
  let so       : bits(32) = 0x000000 @ aes_sbox_inv(si);
  let result   : bits(32) = X(rs1)[31..0] ^ rol32(so, unsigned(shamt));
  X(rd) = EXTS(result); RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zknd (RV32)	v1.0.0	冻结
Zkn (RV32)	v1.0.0	冻结
Zk (RV32)	v1.0.0	冻结

32.4.2 aes32dsmi

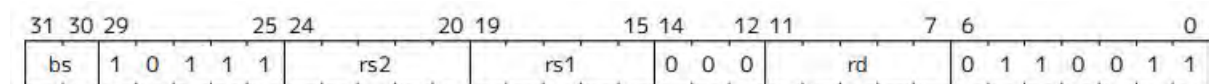
简述:

RV32 的 AES 中间轮解密指令。

助记符:

aes32dsmi rd, rs1, rs2, bs

编码:



描述:

此指令根据 bs 的值, 从源寄存器 rs2 中选取一个字节。然后, 对这个字节应用逆向 AES S 盒 (SBox) 操作和部分逆向列混淆, 之后将结果与 rs1 进行异或运算。此指令必须始终以这样的方式实现, 即它的执行延迟不依赖于所处理的数据。

操作:

```
function clause execute (AES32DSMI (bs,rs2,rs1,rd)) = {  
    let shamt    : bits( 5) = bs @ 0b000; /* shamt = bs*8 */  
    let si       : bits( 8) = (X(rs2)[31..0] >> shamt)[7..0]; /* SBox Input */  
    let so       : bits( 8) = aes_sbox_inv(si);  
    let mixed    : bits(32) = aes_mixcolumn_byte_inv(so);  
    let result   : bits(32) = X(rs1)[31..0] ^ rol32(mixed, unsigned(shamt));  
    X(rd) = EXTS(result); RETIRE_SUCCESS  
}
```

包含于

扩展	最低版本	生命周期阶段
Zknd (RV32)	v1.0.0	冻结
Zkn (RV32)	v1.0.0	冻结
Zk (RV32)	v1.0.0	冻结

32.4.3 aes32esi

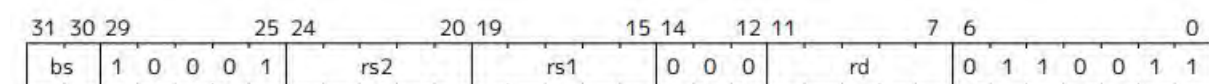
简述:

RV32 的 AES 最终轮加密指令。

助记符:

aes32esi rd, rs1, rs2, bs

编码:



描述:

此指令根据 bs 从 rs2 中获取单个字节。对此应用正向 AES SBox 操作，然后与 rs1 进行异或。此指令必须始终实现，使其执行延迟不依赖于操作的数据。
此指令根据 bs 的值，从源寄存器 rs2 中选取一个字节。然后，对这个字节应用正向 AES S 盒（SBox）操作，之后将结果与 rs1 进行异或运算。此指令必须始终以这样的方式实现，即它的执行延迟不依赖于所处理的数据。

操作:

```
function clause execute (AES32ESI (bs,rs2,rs1,rd)) = {
  let shamt  : bits( 5) = bs @ 0b000; /* shamt = bs*8 */
  let si      : bits( 8) = (X(rs2)[31..0] >> shamt)[7..0]; /* SBox Input */
  let so      : bits(32) = 0x000000 @ aes_sbox_fwd(si);
  let result  : bits(32) = X(rs1)[31..0] ^ rol32(so, unsigned(shamt));
  X(rd) = EXT5(result); RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zknd (RV32)	v1.0.0	冻结
Zkn (RV32)	v1.0.0	冻结
Zk (RV32)	v1.0.0	冻结

32.4.4 aes32esmi

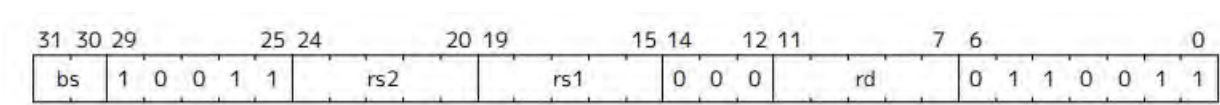
简述:

用于 RV32 的 AES 中间轮加密指令。

助记符:

```
aes32esmi rd, rs1, rs2, bs
```

编码:



描述:

此指令根据 bs 的值，从源寄存器 rs2 中选取一个字节。然后，对这个字节应用正向 AES S 盒（Sbox）操作和部分正向列混淆，之后将结果与 rs1 进行异或运算。此指令必须始终以这样的方式实现，即它的执行延迟不依赖于所处理的数据。

操作:

```
function clause execute (AES32ESMI (bs,rs2,rs1,rd)) = {
  let shamt    : bits( 5) = bs @ 0b000; /* shamt = bs*8 */
  let si       : bits( 8) = (X(rs2)[31..0] >> shamt)[7..0]; /* SBox Input */
  let so       : bits( 8) = aes_sbox_fwd(si);
  let mixed    : bits(32) = aes_mixcolumn_byte_fwd(so);
  let result   : bits(32) = X(rs1)[31..0] ^ rol32(mixed, unsigned(shamt));
  X(rd) = EXTs(result); RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zknd (RV32)	v1.0.0	冻结
Zkn (RV32)	v1.0.0	冻结
Zk (RV32)	v1.0.0	冻结

32.4.5 aes64ds

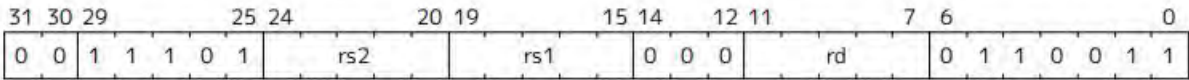
简述:

用于 RV64 的 AES 最后一轮解密指令。

助记符:

```
aes64ds rd, rs1, rs2
```

编码:



描述:

使用两个 64 位源寄存器来表示完整的 AES 状态，并产生下一轮运算输出的一半，同时应用逆行移位（Inverse ShiftRows）和字节代换（SubBytes）步骤。此指令必须始终以这样的方式实现，即它的执行延迟不依赖于所处理的数据。

. 给软件开发人员的注意事项

====

以下代码片段显示了 AES 块解密的最终轮。

t0 和 t1 保存当前轮状态。

t2 和 t3 保存下一轮状态。

```
aes64ds t2, t0, t1
aes64ds t3, t1, t0
```

注意第二条指令的寄存器顺序是反向的。

====

操作:

```
function clause execute (AES64DS(rs2, rs1, rd)) = {
  let sr : bits(64) = aes_rv64_shiftrows_inv(X(rs2)[63..0], X(rs1)[63..0]);
  let wd : bits(64) = sr[63..0];
  X(rd) = aes_apply_inv_sbox_to_each_byte(wd);
  RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zknd (RV64)	v1.0.0	冻结
Zkn (RV64)	v1.0.0	冻结
Zk (RV64)	v1.0.0	冻结

32.4.6 aes64dsm

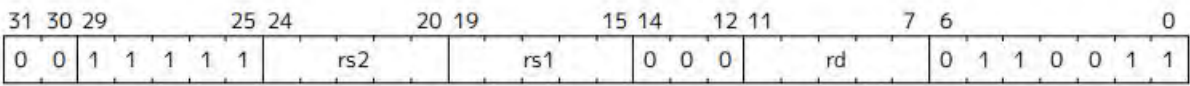
简述:

用于 RV64 的 AES 中间轮解密指令。

助记符:

```
aes64dsm rd, rs1, rs2
```

编码:



描述:

使用两个 64 位源寄存器来表示完整的 AES 状态，并产生下一轮运算输出的一半，同时应用逆行移位（Inverse ShiftRows）、字节代换（SubBytes）和列混淆（MixColumns）步骤。此指令必须始终以这样的方式实现，即它的执行延迟不依赖于所处理的数据。

. 给软件开发人员的注意事项

====

以下代码片段显示了 AES 块解密的一个中间轮。

t0 和 t1 保存当前轮状态。

t2 和 t3 保存下一轮状态。

```
aes64dsm t2, t0, t1
aes64dsm t3, t1, t0
```

注意第二条指令的寄存器顺序是反向的。

====

操作:

```
function clause execute (AES64DSM(rs2, rs1, rd)) = {
  let sr : bits(64) = aes_rv64_shiftrows_inv(X(rs2)[63..0], X(rs1)[63..0]);
  let wd : bits(64) = sr[63..0];
  let sb : bits(64) = aes_apply_inv_sbox_to_each_byte(wd);
  X(rd) = aes_mixcolumn_inv(sb[63..32]) @ aes_mixcolumn_inv(sb[31..0]);
  RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zknd (RV64)	v1.0.0	冻结
Zkn (RV64)	v1.0.0	冻结
Zk (RV64)	v1.0.0	冻结

32.4.7 aes64es

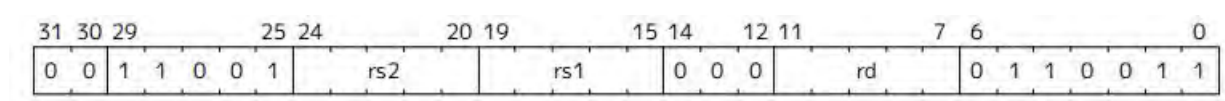
简述:

用于 RV64 的 AES 最后一轮加密指令。

助记符:

aes64es rd, rs1, rs2

编码:



描述:

使用两个 64 位源寄存器来表示完整的 AES 状态，并产生下一轮运算输出的一半，同时应用行移位（ShiftRows）和字节代换（SubBytes）步骤。此指令必须始终以这样的方式实现，即它的执行延迟不依赖于所处理的数据。

. 给软件开发人员的注意事项

[NOTE, caption=SH]

====

以下代码片段显示了 AES 块加密的最终轮。

t0 和 t1 保存当前轮状态。

t2 和 t3 保存下一轮状态。

```
aes64es t2, t0, t1
aes64es t3, t1, t0
```

注意第二条指令的寄存器顺序是反向的。

====

操作:

```
function clause execute (AES64ES(rs2, rs1, rd)) = {
  let sr : bits(64) = aes_rv64_shiftrows_fwd(X(rs2)[63..0], X(rs1)[63..0]);
  let wd : bits(64) = sr[63..0];
  X(rd) = aes_apply_fwd_sbox_to_each_byte(wd);
  RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zknd (RV64)	v1.0.0	冻结
Zkn (RV64)	v1.0.0	冻结
Zk (RV64)	v1.0.0	冻结

32.4.8 aes64esm

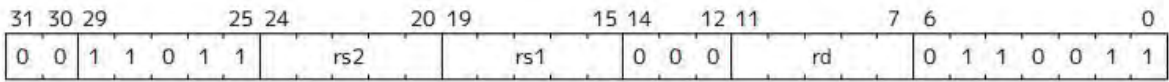
简述:

用于 RV64 的 AES 中间轮加密指令。

助记符:

```
aes64esm rd, rs1, rs2
```

编码:



描述:

使用两个 64 位源寄存器来表示完整的 AES 状态，并产生下一轮运算输出的一半，同时应用行移位（ShiftRows）、字节代换（SubBytes）和列混淆（MixColumns）步骤。此指令必须始终以这样的方式实现，即它的执行延迟不依赖于所处理的数据。

. 给软件开发人员的注意事项

====

以下代码片段显示了 AES 块加密的一个中间轮。

t0 和 t1 保存当前轮状态。

t2 和 t3 保存下一轮状态。

```
aes64esm t2, t0, t1
aes64esm t3, t1, t0
```

注意第二条指令的寄存器顺序是反向的。

====

操作:

```
function clause execute (AES64ESM(rs2, rs1, rd)) = {
  let sr : bits(64) = aes_rv64_shiftrows_fwd(X(rs2)[63..0], X(rs1)[63..0]);
  let wd : bits(64) = sr[63..0];
  let sb : bits(64) = aes_apply_fwd_sbox_to_each_byte(wd);
  X(rd) = aes_mixcolumn_fwd(sb[63..32]) @ aes_mixcolumn_fwd(sb[31..0]);
  RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zknd (RV64)	v1.0.0	冻结
Zkn (RV64)	v1.0.0	冻结
Zk (RV64)	v1.0.0	冻结

32.4.9 aes64im

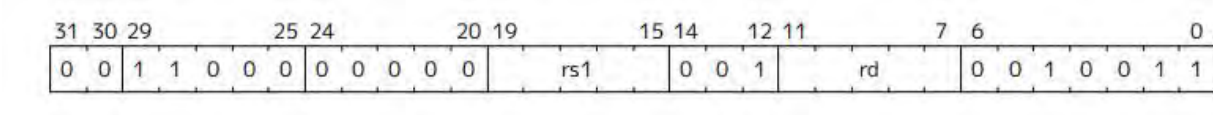
简述:

该指令加速了 AES 块密码算法中的逆 MixColumns 步骤，并用于辅助创建解密密钥调度。

助记符:

aes64im rd, rs1

编码:



描述:

该指令将逆 MixColumns 转换应用于状态数组的两列，并将其打包成一个 64 位寄存器。它用于根据等效的逆密码构造（第 23 页，第 5.3.5 节）来创建逆密码的密钥调度。该指令必须始终实现，确保其执行延迟不依赖于操作的数据。

操作:

```
function clause execute (AES64IM(rs1, rd)) = {
  let w0 : bits(32) = aes_mixcolumn_inv(X(rs1)[31.. 0]);
  let w1 : bits(32) = aes_mixcolumn_inv(X(rs1)[63..32]);
  X(rd) = w1 @ w0;
  RETIRE_SUCCESS
}
```

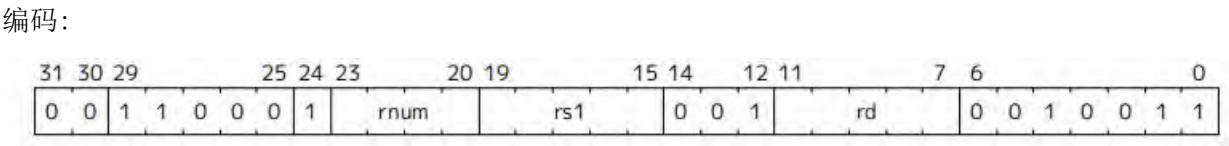
包含于

扩展	最低版本	生命周期阶段
Zknd (RV64)	v1.0.0	冻结
Zkn (RV64)	v1.0.0	冻结
Zk (RV64)	v1.0.0	冻结

32.4.10 aes64ksli

简述：
此指令实现 AES 块密码密钥调度中涉及 S 盒（SBox）操作的部分。

助记符：
aes64ksli rd, rs1, rnum



描述：
此指令实现 AES 块密码密钥调度的旋转、SubBytes 和轮常数加法步骤。此指令必须始终实现，使其执行延迟不依赖于操作的数据。注意，rnum 必须在 0x0..0xA 范围内。值 0xB..0xF 保留。

操作：

```
function clause execute (AES64KS1I(rnum, rs1, rd)) = {
  if(unsigned(rnum) > 10) then {
    handle_illegal(); RETIRE_SUCCESS
  } else {
    let tmp1 : bits(32) = X(rs1)[63..32];
    let rc   : bits(32) = aes_decode_rcon(rnum); /* round number -> round constant */
    let tmp2 : bits(32) = if (rnum == 0xA) then tmp1 else ror32(tmp1, 8);
    let tmp3 : bits(32) = aes_subword_fwd(tmp2);
    let result : bits(64) = (tmp3 ^ rc) @ (tmp3 ^ rc);
    X(rd) = EXTZ(result);
    RETIRE_SUCCESS
  }
}
```

包含于

扩展	最低版本	生命周期阶段
Zkne (RV64)	v1.0.0	冻结
Zknd (RV64)	v1.0.0	冻结
Zkn (RV64)	v1.0.0	冻结
Zk (RV64)	v1.0.0	冻结

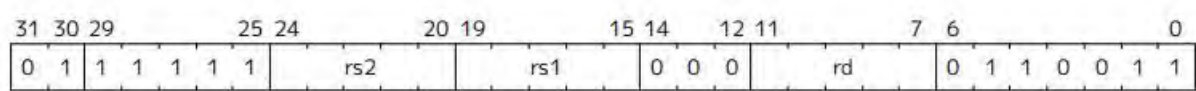
32.4.11 aes64ks2

简述：
该指令实现了 AES 块密码算法中密钥调度操作的一部分。

助记符：

aes64ks2 rd, rs1, rs2

编码:



描述:
此指令实现 AES 块密码密钥调度中密钥字的附加异或操作。此指令必须始终实现，使其执行延迟不依赖于操作的数据。

操作:

```
function clause execute (AES64KS2(rs2, rs1, rd)) = {
  let w0 : bits(32) = X(rs1)[63..32] ^ X(rs2)[31..0];
  let w1 : bits(32) = X(rs1)[63..32] ^ X(rs2)[31..0] ^ X(rs2)[63..32];
  X(rd) = w1 @ w0;
  RETIRE_SUCCESS
}
```

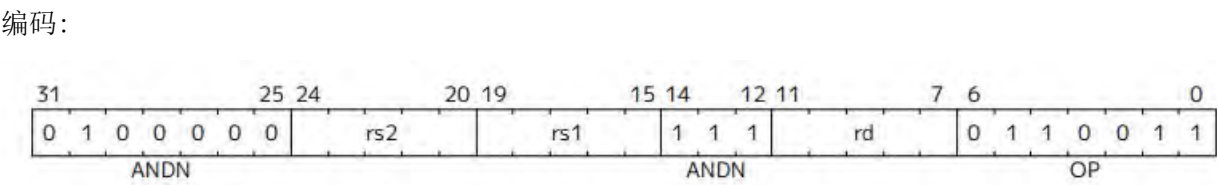
包含于

扩展	最低版本	生命周期阶段
Zkne (RV64)	v1.0.0	冻结
Zknd (RV64)	v1.0.0	冻结
Zkn (RV64)	v1.0.0	冻结
Zk (RV64)	v1.0.0	冻结

32.4.12 andn

简述:
与取反操作数的与运算

助记符:
andn rd, rs1, rs2



描述:
此指令对 rs1 和 rs2 的按位反进行按位与操作。

操作:

$$X(rd) = X(rs1) \& \sim X(rs2);$$

包含于

扩展	最低版本	生命周期阶段
Zbb	v1.0.0	冻结
Zbkb	v1.0.0-rc4	冻结

32.4.13 brev8

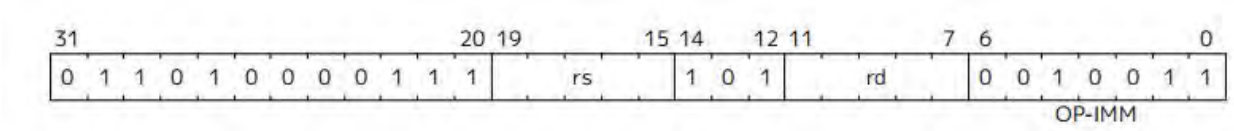
简述:

反转源寄存器中每个字节的位顺序。

助记符:

brev8, rd, rs

编码:



描述:

此指令反转寄存器中每个字节的位顺序。

注：此指令是更通用的指令的一种特定编码，该通用指令最初是作为 RISC-V 位操作扩展（grevi）的一部分提出的。最终，更通用的指令可能会被标准化。但在那之前，只有它最常见的实例，例如此指令，才会被纳入规范中。

操作:

```
result : xlenbits = EXTZ(0b0);
foreach (i from 0 to sizeof(xlen) by 8) {
    result[i+7..i] = reverse_bits_in_byte(X(rs1)[i+7..i]);
};
X(rd) = result;
```

包含于

扩展	最低版本	生命周期阶段
Zbkb	v1.0.0-rc4	冻结

32.4.14 clmul

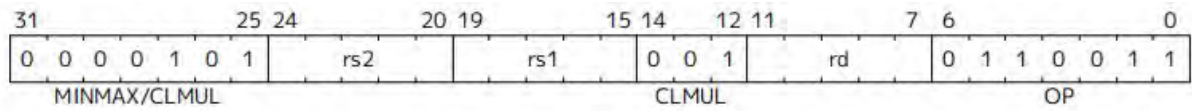
简述:

无进位乘法（低位部分）

助记符:

clmul rd, rs1, rs2

编码:



描述:

clmul 生成 $2 \cdot \text{XLEN}$ 无进位乘积的低位部分。

操作:

```
let rs1_val = X(rs1);
let rs2_val = X(rs2);
let output : xlenbits = 0;

foreach (i from 0 to (xlen - 1) by 1) {
    output = if ((rs2_val >> i) & 1)
        then output ^ (rs1_val << i);
    else output;
}

X[rd] = output
```

包含于

扩展	最低版本	生命周期阶段
Zbc	v1.0.0	冻结
Zbkc	v1.0.0-rc4	冻结

32.4.15 clmulh

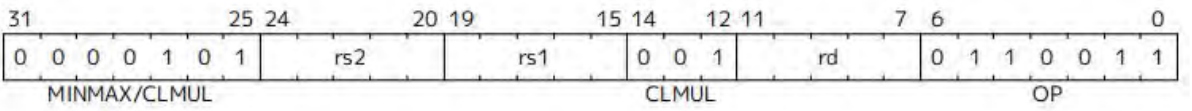
简述:

无进位乘法（高位部分）

助记符:

```
clmulh rd, rs1, rs2
```

编码:



描述:

clmulh 生成 $2 \cdot \text{XLEN}$ 无进位乘积的高位部分。

操作:

```
let rs1_val = X(rs1);
let rs2_val = X(rs2);
let output : xlenbits = 0;

foreach (i from 1 to xlen by 1) {
    output = if ((rs2_val >> i) & 1)
        then output ^ (rs1_val >> (xlen - i));
        else output;
}

X[rd] = output
```

包含于

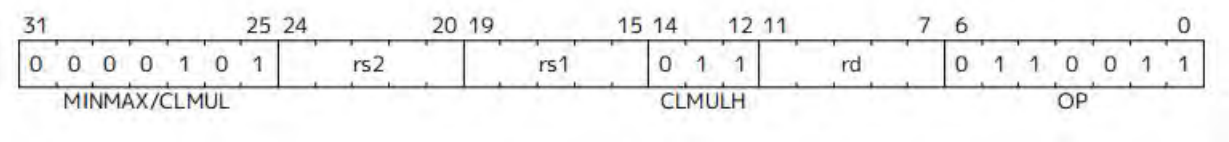
扩展	最低版本	生命周期阶段
Zbc	v1.0.0	冻结
Zbkc	v1.0.0-rc4	冻结

32.4.16 orn

简述：
或反操作数的或

助记符：
orn rd, rs1, rs2

编码：



描述：
此指令对 rs1 和 rs2 的按位反进行按位或操作。

操作：
 $X(rd) = X(rs1) \mid \sim X(rs2);$

包含于

扩展	最低版本	生命周期阶段
Zbb	v1.0.0	冻结
Zbbk	v1.0.0-rc4	冻结

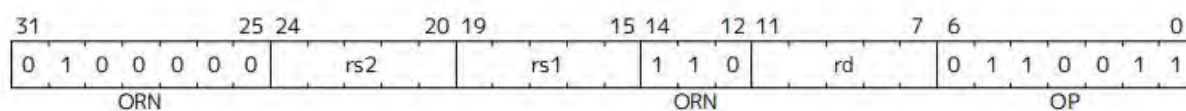
32.4.17 pack

简述：
将 rs1 和 rs2 的低半部分打包到 rd 中。

助记符:

pack rd, rs1, rs2

编码:



描述:

pack 指令将 rs1 和 rs2 的 XLEN/2 位低半部分打包到 rd 中, rs1 在低半部分, rs2 在高半部分。

操作:

```
let lo_half : bits(xlen/2) = X(rs1)[xlen/2-1..0];
let hi_half : bits(xlen/2) = X(rs2)[xlen/2-1..0];
X(rd) = EXTZ(hi_half @ lo_half);
```

包含于

扩展	最低版本	生命周期阶段
Zbkb	v1.0.0-rc4	冻结

32.4.18 packh

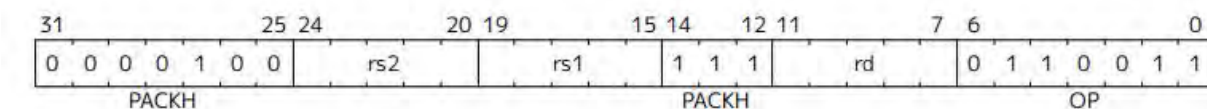
简述:

将 rs1 和 rs2 的低字节打包到 rd 中。

助记符:

packh rd, rs1, rs2

编码:



描述:

packh 指令将 rs1 和 rs2 的最低有效字节打包到 rd 的 16 个最低有效位中, 零扩展 rd 的其余部分。

操作:

```
let lo_half : bits(8) = X(rs1)[7..0];
let hi_half : bits(8) = X(rs2)[7..0];
X(rd) = EXTZ(hi_half @ lo_half);
```

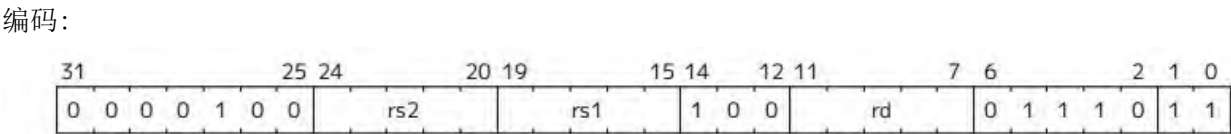
包含于

扩展	最低版本	生命周期阶段
Zbkb	v1.0.0-rc4	冻结

32.4.19 packw

简述：
在 RV64 上将 rs1 和 rs2 的低 16 位打包到 rd 中。

助记符：
packw rd, rs1, rs2



描述：
此指令将 rs1 和 rs2 的低 16 位打包到 rd 的 32 个最低有效位中，将 32 位结果符号扩展到 rd 的其余部分。此指令仅存在于基于 RV64 的系统上。

操作：

```
let lo_half : bits(16) = X(rs1)[15..0];
let hi_half : bits(16) = X(rs2)[15..0];
X(rd) = EXTs(hi_half @ lo_half);
```

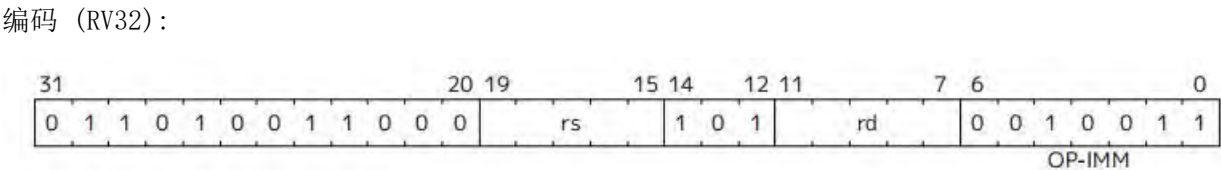
包含于

扩展	最低版本	生命周期阶段
Zbkb	v1.0.0-rc4	冻结

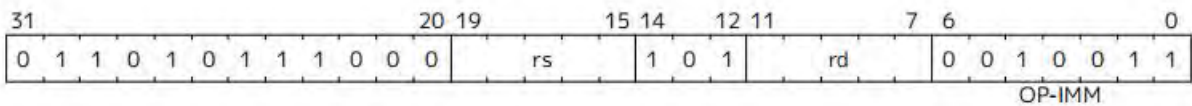
32.4.20 rev8

简述：
字节反转寄存器

助记符：
rev8 rd, rs



编码 (RV64)：



描述：
此指令反转 rs 中字节的顺序。

操作：

```

let input = X(rs);
let output : xlenbits = 0;
let j = xlen - 1;

foreach (i from 0 to (xlen - 8) by 8) {
    output[i..(i + 7)] = input[(j - 7)..j];
    j = j - 8;
}

X[rd] = output
  
```

X[rd] = output

包含于

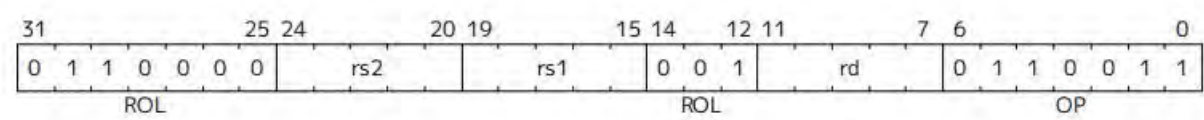
扩展	最低版本	生命周期阶段
Zbb	v1.0.0	冻结
Zbkb	v1.0.0-rc4	冻结

32.4.21 rol

简述：
左旋转（寄存器）

助记符：
rol rd, rs1, rs2

编码：



描述：
该指令将 rs1 按 rs2 中最低有效的 log2(XLEN) 位的值执行左旋转操作。

操作：


```
let shamt = if xlen == 32
    then X(rs2)[4..0]
    else X(rs2)[5..0];
let result = (X(rs1) << shamt) | (X(rs1) >> (xlen - shamt));

X(rd) = result;
```

包含于

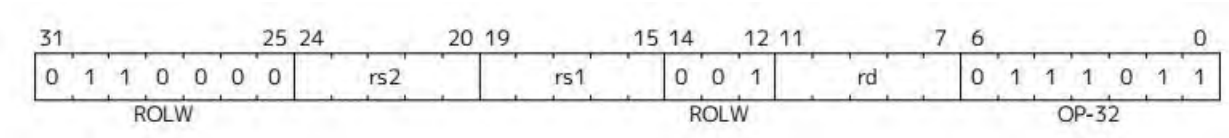
扩展	最低版本	生命周期阶段
Zbb	v1.0.0	冻结
Zbkb	v1.0.0-rc4	冻结

32.4.22 rolw

简述：
左旋转字（寄存器）

助记符：
rolw rd, rs1, rs2

编码：



描述：
此指令对 rs1 的最低有效字进行左旋转，旋转量由 rs2 的最低 5 位决定。结果字值通过将第 31 位复制到所有更高位来进行符号扩展。

操作：

```
let rs1 = EXTZ(X(rs1)[31..0])
let shamt = X(rs2)[4..0];
let result = (rs1 << shamt) | (rs1 >> (32 - shamt));
X(rd) = EXT5(result[31..0]);
```

包含于

扩展	最低版本	生命周期阶段
Zbb	v1.0.0	冻结
Zbkb	v1.0.0-rc4	冻结

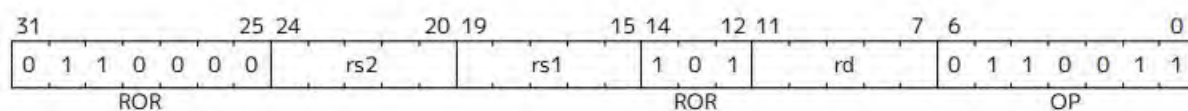
32.4.23 ror

简述：
右旋转（寄存器）

助记符:

ror rd, rs1, rs2

编码:



描述:

该指令将 rs1 按 rs2 中最低有效的 $\log_2(XLEN)$ 位的值执行右旋转操作。

操作:

```
let shamt = if xlen == 32
    then X(rs2)[4..0]
    else X(rs2)[5..0];
let result = (X(rs1) >> shamt) | (X(rs1) << (xlen - shamt));

X(rd) = result;
```

包含于

扩展	最低版本	生命周期阶段
Zbb	v1.0.0	冻结
Zbkb	v1.0.0-rc4	冻结

32.4.24 rori

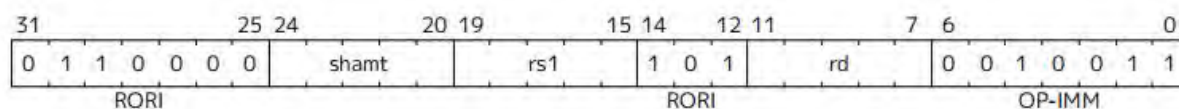
简述:

右旋转（立即数）

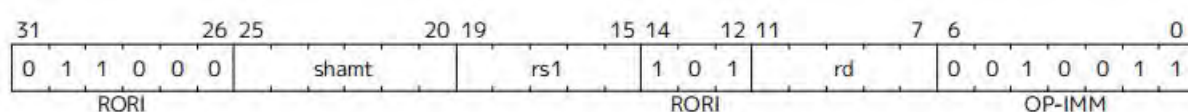
助记符:

rori rd, rs1, shamt

编码 (RV32):



编码 (RV64):



描述:

该指令将 `rs1` 按 `shamt` 中最低有效的 $\log_2(XLEN)$ 位的值执行右旋转操作。对于 RV32, `shamt[5] = 1` 对应的编码是保留的。

操作:

```
let shamt = if xlen == 32
    then shamt[4..0]
    else shamt[5..0];
let result = (X(rs1) >> shamt) | (X(rs1) << (xlen - shamt));
X(rd) = result;
```

包含于

扩展	最低版本	生命周期阶段
Zbb	v1.0.0	冻结
Zbkb	v1.0.0-rc4	冻结

32.4.25 roriw

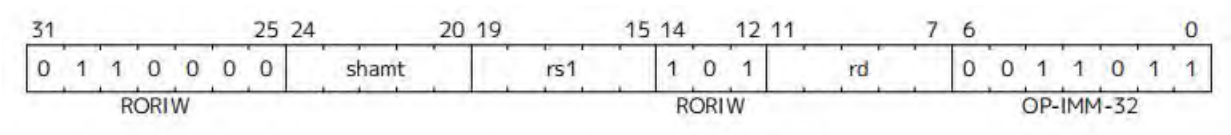
简述:

按立即数右旋转字

助记符:

```
roriw rd, rs1, shamt
```

编码:



描述:

该指令对 `rs1` 中最低有效字按 `shamt` 中最低有效的 $\log_2(XLEN)$ 位的值执行右旋转操作。结果字的值通过将第 31 位复制到所有更高位来进行符号扩展。

操作:

```
let rs1_data = EXTZ(X(rs1)[31..0]);
let result = (rs1_data >> shamt) | (rs1_data << (32 - shamt));
X(rd) = EXT5(result[31..0]);
```

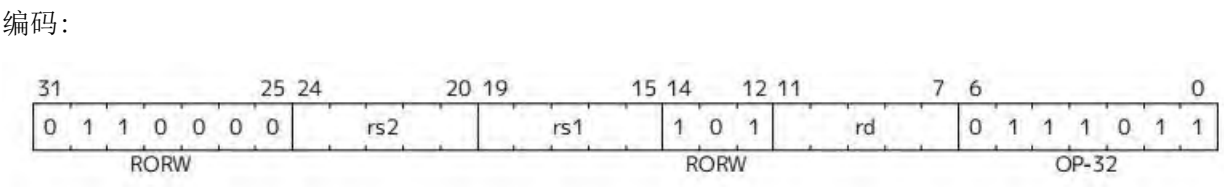
包含于

扩展	最低版本	生命周期阶段
Zbb	v1.0.0	冻结
Zbkb	v1.0.0-rc4	冻结

32.4.26 rorw

简述：
字（寄存器）右旋转

助记符：
rorw rd, rs1, rs2



描述：
该指令对 rs1 中最低有效字按 rs2 中最低有效的 5 位值执行右旋转操作。结果字被符号扩展，通过将第 31 位复制到所有更高位来实现。

操作：
let rs1 = EXTZ(X(rs1)[31..0])
let shamt = X(rs2)[4..0];
let result = (rs1 >> shamt) | (rs1 << (32 - shamt));
X(rd) = EXTZ(result);

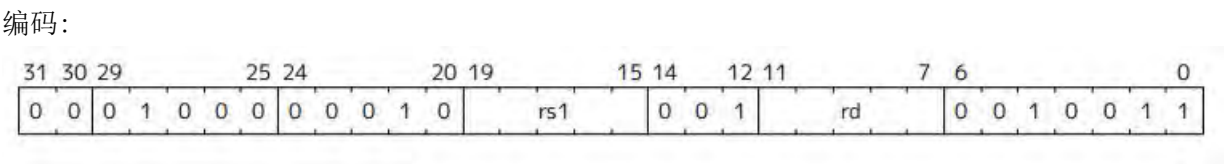
包含于

扩展	最低版本	生命周期阶段
Zbb	v1.0.0	冻结
Zbkb	v1.0.0-rc4	冻结

32.4.27 sha256sig0

简述：
实现 SHA2-256 哈希函数中使用的 Sigma0 转换函数（第 4.1.2 节）。

助记符：
sha256sig0 rd, rs1



描述：
此指令同时被 RV32 和 RV64 基础架构所支持。对于 RV32 架构，指令操作的是整个 XLEN 源寄存器。对于 RV64 架构，指令操作的是源寄存器的低 32 位，并将结果符号扩展到 XLEN 位。虽然指令名称中包含 SHA2-256，但如 中所述，该指令也适用于 SHA2-224 和 SHA2-256 参数化。此指令必须始终

以这样的方式实现，即它的执行延迟不依赖于所操作的数据。

操作：

```
function clause execute (SHA256SIG0(rs1,rd)) = {
  let inb    : bits(32) = X(rs1)[31..0];
  let result : bits(32) = ror32(inb, 7) ^ ror32(inb, 18) ^ (inb >> 3);
  X(rd)      = EXTS(result);
  RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zknh	v1.0.0	冻结
Zkn	v1.0.0	冻结
Zk	v1.0.0	冻结

32.4.28 sha256sig1

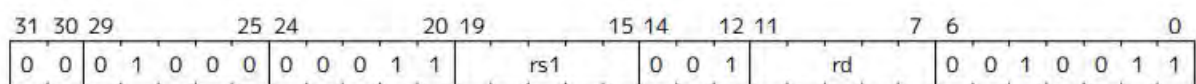
简述：

实现 SHA2-256 哈希函数中使用的 Sigma1 转换函数（第 4.1.2 节）。

助记符：

sha256sig1 rd, rs1

编码：



描述：

此指令支持 RV32 和 RV64 基础架构。对于 RV32，整个 XLEN 源寄存器都会被操作。对于 RV64，源寄存器的低 32 位会被操作，结果会被符号扩展到 XLEN 位。虽然命名为 SHA2-256，但该指令适用于 SHA2-224 和 SHA2-256 参数化。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作：

```
function clause execute (SHA256SIG1(rs1,rd)) = {
  let inb    : bits(32) = X(rs1)[31..0];
  let result : bits(32) = ror32(inb, 17) ^ ror32(inb, 19) ^ (inb >> 10);
  X(rd)      = EXTS(result);
  RETIRE_SUCCESS
}
```

包含于

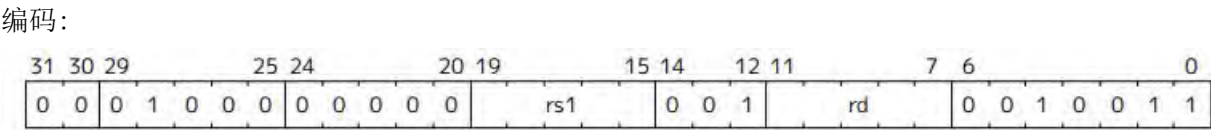
扩展	最低版本	生命周期阶段
Zknh	v1.0.0	冻结

Zkn	v1.0.0	冻结
Zk	v1.0.0	冻结

32.4.29 sha256sum0

简述：
实现 SHA2-256 哈希函数中使用的 Sum0 转换函数（第 4.1.2 节）。

助记符：
sha256sum0 rd, rs1



描述：
此指令支持 RV32 和 RV64 基础架构。对于 RV32，整个 XLEN 源寄存器都会被操作。对于 RV64，源寄存器的低 32 位会被操作，结果会被符号扩展到 XLEN 位。虽然命名为 SHA2-256，但该指令适用于 SHA2-224 和 SHA2-256 参数化。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作：

```
function clause execute (SHA256SUM0(rs1,rd)) = {
  let inb    : bits(32) = X(rs1)[31..0];
  let result : bits(32) = ror32(inb, 2) ^ ror32(inb, 13) ^ ror32(inb, 22);
  X(rd)      = EXTS(result);
  RETIRE_SUCCESS
}
```

包含于

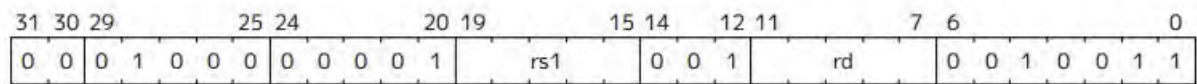
扩展	最低版本	生命周期阶段
Zknh	v1.0.0	冻结
Zkn	v1.0.0	冻结
Zk	v1.0.0	冻结

32.4.30 sha256sum1

简述：
实现 SHA2-256 哈希函数中使用的 Sum1 转换函数（第 4.1.2 节）。

助记符：
sha256sum1 rd, rs1

编码：



描述：
 此指令支持 RV32 和 RV64 基本架构。对于 RV32，将对整个 XLEN 源寄存器进行操作。对于 RV64，将对源寄存器的低 32 位进行操作，并且结果符号扩展为 XLEN 位。虽然以 SHA2-256 命名，但该指令适用于 SHA2-224 和 SHA2-256 参数化。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作：

```
function clause execute (SHA256SUM1(rs1,rd)) = {
  let inb    : bits(32) = X(rs1)[31..0];
  let result : bits(32) = ror32(inb, 6) ^ ror32(inb, 11) ^ ror32(inb, 25);
  X(rd)      = EXTS(result);
  RETIRE_SUCCESS
}
```

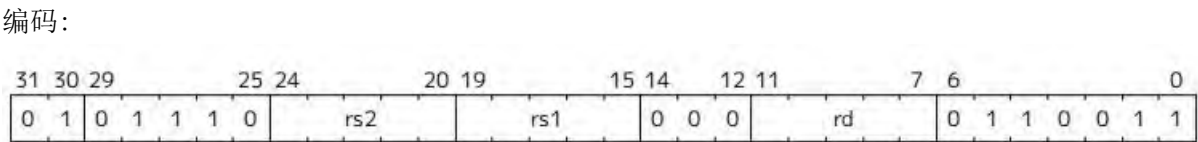
包含于

扩展	最低版本	生命周期阶段
Zknh	v1.0.0	冻结
Zkn	v1.0.0	冻结
Zk	v1.0.0	冻结

32.4.31 sha512sig0h

简述：
 实现 SHA2-512 哈希函数中使用的 Sigma0 转换的高位（第 4.1.3 节）。

助记符：
 sha512sig0h rd, rs1, rs2



描述：
 此指令仅在 RV32 上实现。用于与 insns-sha512sig0l, sha512sig0l 指令结合使用，以计算 SHA2-512 哈希函数的 Sigma0 变换。变换是一个 64 位到 64 位的函数，因此输入和输出都由两个 32 位寄存器表示。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作：


```
function clause execute (SHA512SIG0H(rs2, rs1, rd)) = {
  X(rd) = EXTSTS((X(rs1) >> 1) ^ (X(rs1) >> 7) ^ (X(rs1) >> 8) ^
    (X(rs2) << 31) ^ (X(rs2) << 24));
  RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zknh (RV32)	v1.0.0	冻结
Zkn (RV32)	v1.0.0	冻结
Zk (RV32)	v1.0.0	冻结

32.4.32 sha512sig0l

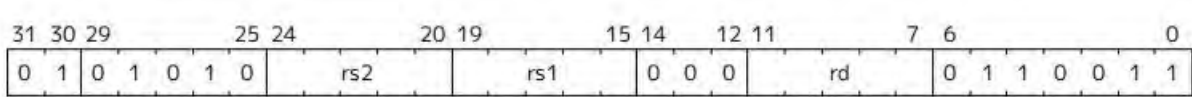
简述:

实现 SHA2-512 哈希函数中使用的 Sigma0 转换的低位（第 4.1.3 节）。

助记符:

sha512sig0l rd, rs1, rs2

编码:



描

述:

此指令仅在 RV32 上实现。用于与 insns-sha512sig0h, sha512sig0h 指令结合使用，以计算 SHA2-512 哈希函数的 Sigma0 变换。变换是一个 64 位到 64 位的函数，因此输入和输出都由两个 32 位寄存器表示。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作:

```
function clause execute (SHA512SIG0L(rs2, rs1, rd)) = {
  X(rd) = EXTSTS((X(rs1) >> 1) ^ (X(rs1) >> 7) ^ (X(rs1) >> 8) ^
    (X(rs2) << 31) ^ (X(rs2) << 25) ^ (X(rs2) << 24));
  RETIRE_SUCCESS
}
```

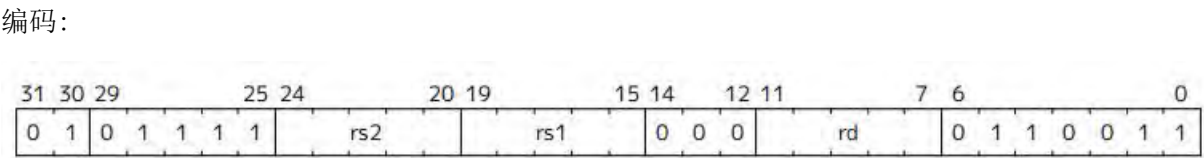
包含于

扩展	最低版本	生命周期阶段
Zknh (RV32)	v1.0.0	冻结
Zkn (RV32)	v1.0.0	冻结
Zk (RV32)	v1.0.0	冻结

32.4.33 sha512sig1h

简述：
实现 SHA2-512 哈希函数中使用的 Sigma1 转换的高位（第 4.1.3 节）。

助记符：
sha512sig1h rd, rs1, rs2



描述：
此指令仅在 RV32 上实现。用于与 insns-sha512sig1l, sha512sig1l 指令结合使用，以计算 SHA2-512 哈希函数的 Sigma1 变换。变换是一个 64 位到 64 位的函数，因此输入和输出都由两个 32 位寄存器表示。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作：

```
function clause execute (SHA512SIG1H(rs2, rs1, rd)) = {
  X(rd) = EXT5((X(rs1) << 3) ^ (X(rs1) >> 6) ^ (X(rs1) >> 19) ^
               (X(rs2) >> 29) ^ (X(rs2) << 13));
  RETIRE_SUCCESS
}
```

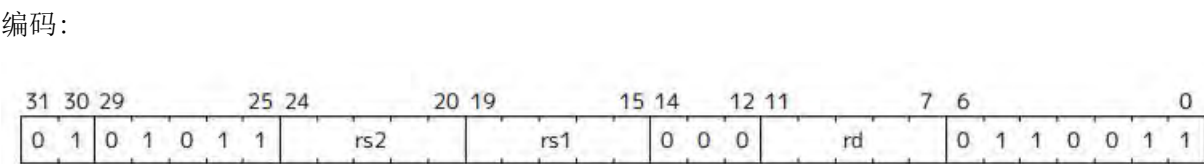
包含于

扩展	最低版本	生命周期阶段
Zknh (RV32)	v1.0.0	冻结
Zkn (RV32)	v1.0.0	冻结
Zk (RV32)	v1.0.0	冻结

32.4.34 sha512sig1l

简述：
实现 SHA2-512 哈希函数中使用的 Sigma1 转换的低位，第 4.1.3 节。

助记符：
sha512sig1l rd, rs1, rs2



描述：
此指令仅在 RV32 上实现。用于与 insns-sha512sig1h, sha512sig1h 指令结合使用，以计算

SHA2-512 哈希函数的 Sigma1 变换。变换是一个 64 位到 64 位的函数，因此输入和输出都由两个 32 位寄存器表示。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作:

```
function clause execute (SHA512SIG1L(rs2, rs1, rd)) = {
  X(rd) = EXTS((X(rs1) << 3) ^ (X(rs1) >> 6) ^ (X(rs1) >> 19) ^
               (X(rs2) >> 29) ^ (X(rs2) << 26) ^ (X(rs2) << 13) );
  RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zknh (RV32)	v1.0.0	冻结
Zkn (RV32)	v1.0.0	冻结
Zk (RV32)	v1.0.0	冻结

32.4.35 sha512sum0r

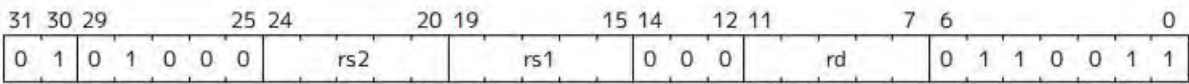
简述:

实现 SHA2-512 哈希函数中使用的 Sum0 转换，第 4.1.3 节。

助记符:

```
sha512sum0r rd, rs1, rs2
```

编码:



描述:

此指令仅在 RV32 上实现。用于计算 SHA2-512 哈希函数的 Sum0 变换。变换是一个 64 位到 64 位的函数，因此输入和输出都由两个 32 位寄存器表示。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作:

```
function clause execute (SHA512SUM0R(rs2, rs1, rd)) = {
  X(rd) = EXTS((X(rs1) << 25) ^ (X(rs1) << 30) ^ (X(rs1) >> 28) ^
               (X(rs2) >> 7) ^ (X(rs2) >> 2) ^ (X(rs2) << 4) );
  RETIRE_SUCCESS
}
```

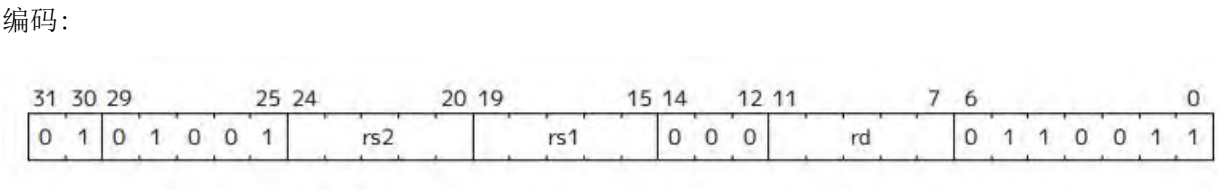
包含于

扩展	最低版本	生命周期阶段
Zknh (RV32)	v1.0.0	冻结
Zkn (RV32)	v1.0.0	冻结
Zk (RV32)	v1.0.0	冻结

32.4.36 sha512sum1r

简述：
实现了 Sum1 变换，如 SHA2-512 哈希函数 第 4.1.3 节中所用。

助记符：
sha512sum1r rd, rs1, rs2



描述：
此指令仅在 RV32 上实现。用于计算 SHA2-512 哈希函数的 Sum1 变换。变换是一个 64 位到 64 位的函数，因此输入和输出都由两个 32 位寄存器表示。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作：

```
function clause execute (SHA512SUM1R(rs2, rs1, rd)) = {  
    X(rd) = EXT5((X(rs1) << 23) ^ (X(rs1) >> 14) ^ (X(rs1) >> 18) ^  
                (X(rs2) >> 9) ^ (X(rs2) << 18) ^ (X(rs2) << 14) );  
    RETIRE_SUCCESS  
}
```

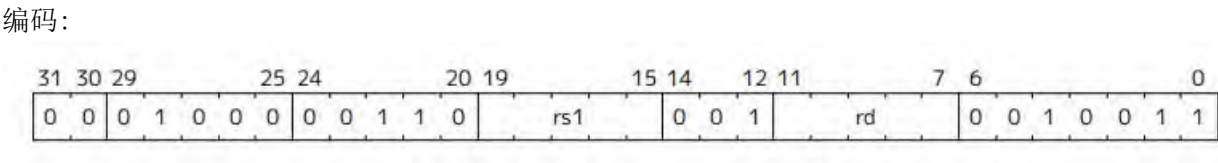
包含于

扩展	最低版本	生命周期阶段
Zknh (RV32)	v1.0.0	冻结
Zkn (RV32)	v1.0.0	冻结
Zk (RV32)	v1.0.0	冻结

32.4.37 sha512sig0

简述：
实现了 Sigma0 变换函数，如 SHA2-512 哈希函数第 4.1.3 节中所用。

助记符：
sha512sig0 rd, rs1



描述：
此指令支持 RV64 基本架构。它实现了 SHA2-512 哈希函数的 Sigma0 变换。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作：

```
function clause execute (SHA512SIG0(rs1, rd)) = {
    X(rd) = ror64(X(rs1), 1) ^ ror64(X(rs1), 8) ^ (X(rs1) >> 7);
    RETIRE_SUCCESS
}
```

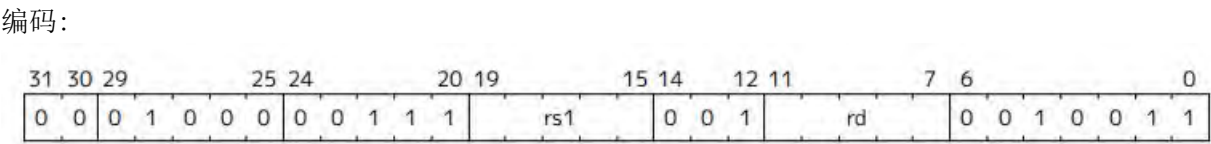
包含于

扩展	最低版本	生命周期阶段
Zknh (RV64)	v1.0.0	冻结
Zkn (RV64)	v1.0.0	冻结
Zk (RV64)	v1.0.0	冻结

32.4.38 sha512sig1

简述：
实现了 Sigma1 变换函数，如 SHA2-512 哈希函数第 4.1.3 节中所用。

助记符：
sha512sig1 rd, rs1



描述：
此指令支持 RV64 基本架构。它实现了 SHA2-512 哈希函数的 Sigma1 变换。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作：

```
function clause execute (SHA512SIG1(rs1, rd)) = {
    X(rd) = ror64(X(rs1), 19) ^ ror64(X(rs1), 61) ^ (X(rs1) >> 6);
    RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zknh (RV64)	v1.0.0	冻结
Zkn (RV64)	v1.0.0	冻结
Zk (RV64)	v1.0.0	冻结

32.4.39 sha512sum0

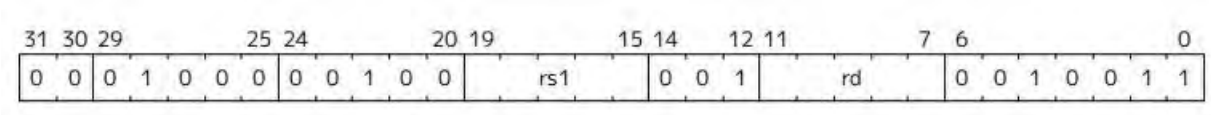
简述:

实现了 Sum0 变换函数，如 SHA2-512 哈希函数第 4.1.3 节中所用。

助记符:

sha512sum0 rd, rs1

编码:



描述:

此指令支持 RV64 基本架构。它实现了 SHA2-512 哈希函数的 Sum0 变换。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作:

```
function clause execute (SHA512SUM0(rs1, rd)) = {
  X(rd) = ror64(X(rs1), 28) ^ ror64(X(rs1), 34) ^ ror64(X(rs1), 39);
  RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zknh (RV64)	v1.0.0	冻结
Zkn (RV64)	v1.0.0	冻结
Zk (RV64)	v1.0.0	冻结

32.4.40 sha512sum1

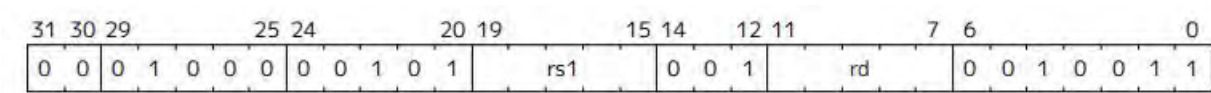
简述:

实现了 Sum1 变换函数，如 SHA2-512 哈希函数第 4.1.3 节中所用。

助记符:

sha512sum1 rd, rs1

编码:



描述:

此指令支持 RV64 基本架构。它实现了 SHA2-512 哈希函数的 Sum1 变换。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作:

```
function clause execute (SHA512SUM1(rs1, rd)) = {
  X(rd) = ror64(X(rs1), 14) ^ ror64(X(rs1), 18) ^ ror64(X(rs1), 41);
  RETIRE_SUCCESS
}
```

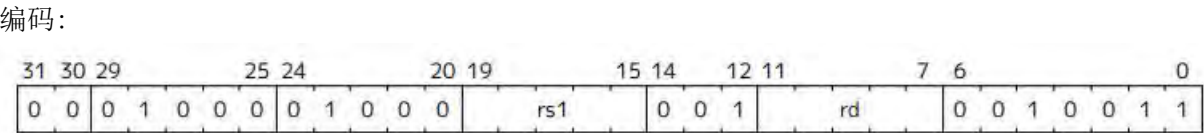
包含于

扩展	最低版本	生命周期阶段
Zknh (RV64)	v1.0.0	冻结
Zkn (RV64)	v1.0.0	冻结
Zk (RV64)	v1.0.0	冻结

32.4.41 sm3p0

简述：
实现 SM3 哈希函数中使用的 P0 转换函数。

助记符：
sm3p0 rd, rs1



描述：
此指令支持 RV32 和 RV64 基础架构。它实现了 SM3 哈希函数的 P0 转换。此指令必须始终实现，使其执行延迟不依赖于操作的数据。

操作：

```
function clause execute (SM3P0(rs1, rd)) = {
  let r1      : bits(32) = X(rs1)[31..0];
  let result : bits(32) = r1 ^ rol32(r1, 9) ^ rol32(r1, 17);
  X(rd) = EXTS(result);
  RETIRE_SUCCESS
}
```

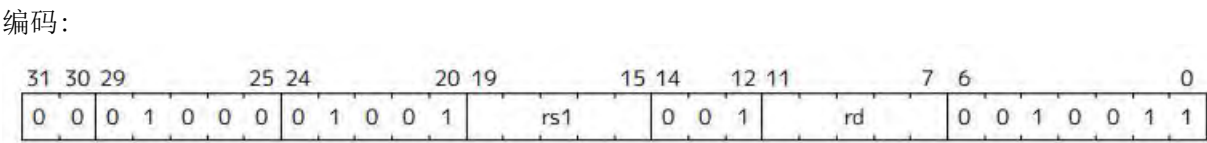
包含于

扩展	最低版本	生命周期阶段
Zksh	v1.0.0	冻结
Zks	v1.0.0	冻结

32.4.42 sm3p1

简述：
实现 SM3 哈希函数中使用的 P1 转换函数。

助记符:
sm3pl rd, rs1



描述:
此指令支持 RV32 和 RV64 基本架构。它实现了 SM3 哈希函数的 P1 变换。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作:

```
function clause execute (SM3P1(rs1, rd)) = {  
  let r1      : bits(32) = X(rs1)[31..0];  
  let result : bits(32) =  r1 ^ rol32(r1, 15) ^ rol32(r1, 23);  
  X(rd) = EXTS(result);  
  RETIRE_SUCCESS  
}
```

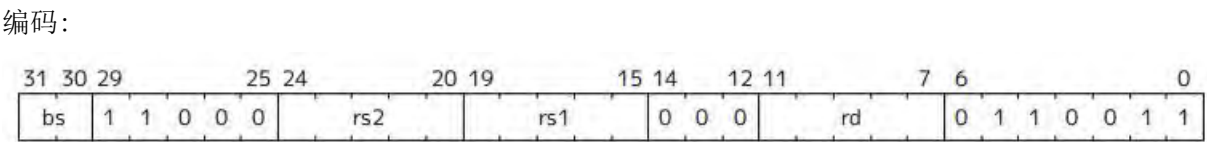
包含于

扩展	最低版本	生命周期阶段
Zksh	v1.0.0	冻结
Zks	v1.0.0	冻结

32.4.43 sm4ed

简述:
加速 SM4 块密码的块加密/解密操作。

助记符:
sm4ed rd, rs1, rs2, bs



描述:
实现了硬件 T 表风格的方法，以加速 SM4 轮函数。从 rs2 中根据 bs 提取一个字节，然后应用 SBox 和线性层变换，之后将结果与 rs1 进行异或运算，并将结果写回 rd。此指令存在于 RV32 和 RV64 基本架构上。在 RV64 上，32 位的结果会进行符号扩展至 XLEN 位。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作:

```

function clause execute (SM4ED (bs,rs2,rs1,rd)) = {
  let shamt : bits(5) = bs @ 0b000; /* shamt = bs*8 */
  let sb_in : bits(8) = (X(rs2)[31..0] >> shamt)[7..0];
  let x      : bits(32) = 0x000000 @ sm4_sbox(sb_in);
  let y      : bits(32) = x ^ (x << 8) ^ (x << 2) ^
    (x << 18) ^ ((x & 0x0000003F) << 26) ^
    ((x & 0x000000C0) << 10);
  let z      : bits(32) = rol32(y, unsigned(shamt));
  let result: bits(32) = z ^ X(rs1)[31..0];
  X(rd)
    = EXTs(result);
  RETIRE_SUCCESS
}

```

包含于

扩展	最低版本	生命周期阶段
Zksed	v1.0.0	冻结
Zks	v1.0.0	冻结

32.4.44 sm4ks

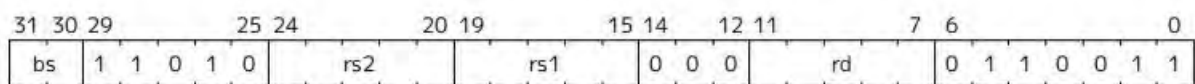
简述:

加速 SM4 块密码的密钥调度操作。

助记符:

sm4ks rd, rs1, rs2, bs

编码:



描述:

实现了硬件 T 表风格的方法，以加速 SM4 密钥调度。从 rs2 中根据 bs 提取一个字节，然后应用 SBox 和线性层变换，之后将结果与 rs1 进行异或运算，并将结果写回 rd。此指令存在于 RV32 和 RV64 基本架构上。在 RV64 上，32 位的结果会进行符号扩展至 XLEN 位。此指令的实现必须始终确保其执行延迟不依赖于正在操作的数据。

操作:

```

function clause execute (SM4KS (bs,rs2,rs1,rd)) = {
    let shamt : bits(5) = (bs @ 0b000); /* shamt = bs*8 */
    let sb_in : bits(8) = (X(rs2)[31..0] >> shamt)[7..0];
    let x      : bits(32) = 0x000000 @ sm4_sbox(sb_in);
    let y      : bits(32) = x ^ ((x & 0x00000007) << 29) ^ ((x & 0x000000FE) << 7) ^
                                ((x & 0x00000001) << 23) ^ ((x & 0x000000F8) << 13);
    let z      : bits(32) = rol32(y, unsigned(shamt));
    let result: bits(32) = z ^ X(rs1)[31..0];
    X(rd) = EXTS(result);
    RETIRE_SUCCESS
}

```

包含于

扩展	最低版本	生命周期阶段
Zksed	v1.0.0	冻结
Zks	v1.0.0	冻结

32.4.45 unzip

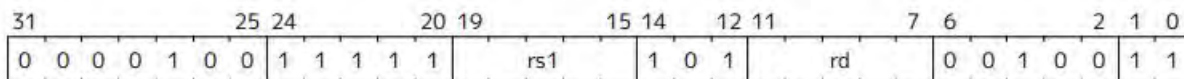
简述:

实现 zip 指令的逆操作。

助记符:

unzip rd, rs

编码:



描述:

此指令将源字的高半部分和低半部分的位收集到目标字的奇数/偶数位位置中。它是 insns-zip, zip 指令的逆指令。此指令仅在 RV32 上可用。

操作:

```

foreach (i from 0 to xlen/2-1) {
    X(rd)[i] = X(rs1)[2*i]
    X(rd)[i+xlen/2] = X(rs1)[2*i+1]
}

```

包含于

扩展	最低版本	生命周期阶段
Zbkb (RV32)	v1.0.0-rc4	冻结

32.4.46 xnor

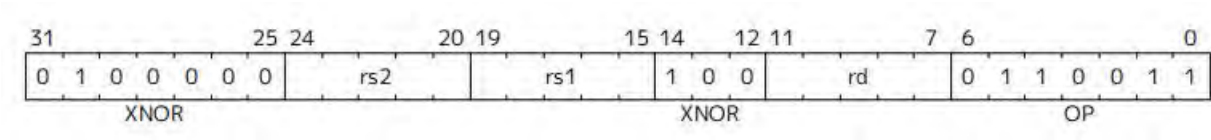
简述:

异或非

助记符:

```
xnor rd, rs1, rs2
```

编码:



描述:

此指令对 rs1 和 rs2 执行按位异或非操作。

操作:

$X(rd) = \sim(X(rs1) \wedge X(rs2));$

包含于

扩展	最低版本	生命周期阶段
Zbb	v1.0.0	冻结
Zbkb	v1.0.0-rc4	冻结

32.4.47 xperm8

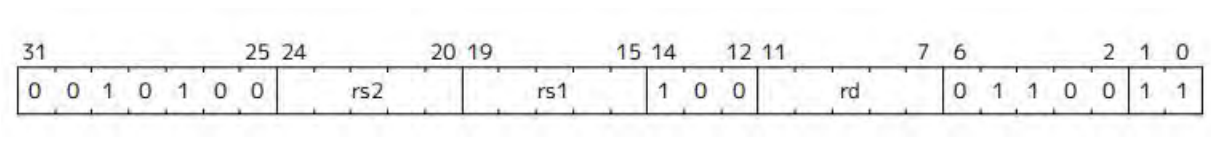
简述:

按字节查找向量中的索引。

助记符:

```
xperm8 rd, rs1, rs2
```

编码:



描述:

xperm8 指令操作的是字节。rs1 寄存器包含一个由 XLEN/8 个 8 位元素组成的向量。rs2 寄存器包含一个由 XLEN/8 个 8 位索引组成的向量。结果是将 rs2 中的每个元素替换为 rs1 中索引对应的元素；如果 rs2 中的索引超出范围，则替换为零。

操作:

```
val xperm8_lookup : (bits(8), xlenbits) -> bits(8)
function xperm8_lookup (idx, lut) = {
  (lut >> (idx @ 0b000))[7..0]
}
function clause execute ( XPERM_8 (rs2,rs1,rd)) = {
  result : xlenbits = EXTZ(0b0);
  foreach(i from 0 to xlen by 8) {
    result[i+7..i] = xperm8_lookup(X(rs2)[i+7..i], X(rs1));
  };
  X(rd) = result;
  RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zbkx	v1.0.0-rc4	冻结

32.4.48 xperm4

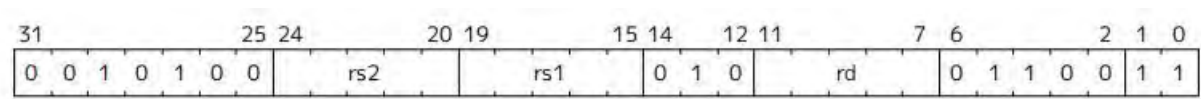
简述:

按半字节查找向量中的索引。

助记符:

xperm4 rd, rs1, rs2

编码:



描述:

xperm4 指令操作的是半字节 (nibbles)。rs1 寄存器包含一个由 XLEN/4 个 4 位元素组成的向量。rs2 寄存器包含一个由 XLEN/4 个 4 位索引组成的向量。结果是将 rs2 中的每个元素替换为 rs1 中索引对应的元素；如果 rs2 中的索引超出范围，则替换为零。

操作:

```
val xperm4_lookup : (bits(4), xlenbits) -> bits(4)
function xperm4_lookup (idx, lut) = {
  (lut >> (idx @ 0b00))[3..0]
}
function clause execute ( XPERM_4 (rs2,rs1,rd)) = {
  result : xlenbits = EXTZ(0b0);
  foreach(i from 0 to xlen by 4) {
    result[i+3..i] = xperm4_lookup(X(rs2)[i+3..i], X(rs1));
  };
  X(rd) = result;
  RETIRE_SUCCESS
}
```

包含于

扩展	最低版本	生命周期阶段
Zbkx	v1.0.0-rc4	冻结

32.4.49 zip

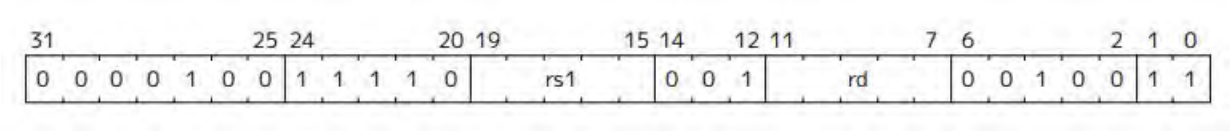
简述:

将源字的奇数位和偶数位分别收集到目标字的高半部分和低半部分。

助记符:

zip rd, rs

编码:



描述:

此指令将源字的所有奇数位和偶数位分散到目标字的高半部分和低半部分。它是 insns-unzip, unzip 指令的反向操作。此指令仅在 RV32 架构上可用。

操作:

```
foreach (i from 0 to xlen/2-1) {
    X(rd)[2*i] = X(rs1)[i]
    X(rd)[2*i+1] = X(rs1)[i+xlen/2]
}
```

包含于

扩展	最低版本	生命周期阶段
Zbkb (RV32)	v1.0.0-rc4	冻结

32.5 熵源

seed CSR 提供了一个符合 NIST SP 800-90B 或 BSI AIS-31 标准的物理熵源 (ES) 接口。

熵源本身不是一个加密安全的随机位生成器 (RBG), 但可以在对称加密的帮助下构建多种类型的标准 (和非标准) RBG。预期的用法是对熵源的输出进行调节 (通常使用 SHA-2/3), 并将其用于生成加密安全的确定性随机位生成器 (DRBG), 例如基于 AES 的 CTRDRBG。熵源、调节和 DRBG 的组合可以用来安全地生成随机位。有关认证和自我认证程序的非规范性描述、设计原理以及有关如何使用熵源输出的更详细建议, 请参见 cryptoscalarappxes。

32.5.1 种子 CSR

seed 是位于地址 0x015 的非特权 CSR。

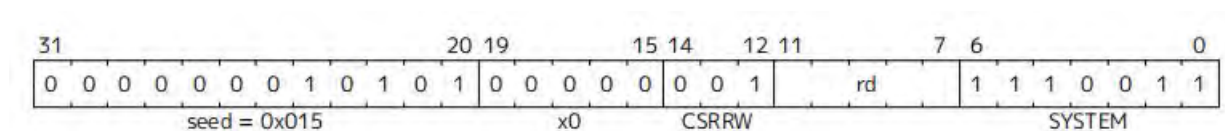
表 32.9 Seed的32位内容

位	名称	描述
31:30	OPST	状态: BIST(00), WAIT(01), ES16(10), DEAD(11)
29:24	保留	供 RISC-V 规范将来使用。
23:16	自定义	指定用于自定义和实验用途。

必须使用读写指令访问 seed CSR。使用 `rs1=x0` 的只读指令（例如 `CSRRS/CSRRC`）或使用 `uimm=0` 的只读指令（例如 `CSRRSI/CSRRCI`）将引发非法指令异常。实现必须忽略写入值（在 `rs1` 或 `uimm` 中）。写入的目的是发出轮询和刷新的信号。

指令 `csrrw rd, seed, x0` 可用于获取种子状态和熵值。它在 RV32 和 RV64 基本架构上均可用，并将 32 位字零扩展到 XLEN 位。

编码:



seed CSR 还受执行模式的访问控制，除非明确授予访问权限，否则在 M 模式之外尝试读或写访问将引发非法指令异常。有关更多详细信息，请参见 `cryptoscalaresaccess`。

状态位 `seed[31:30] = OPST` 可能是 ES16 (10)，表示轮询成功，或三种熵轮询失败状态之一 BIST (00)、WAIT (01) 或 DEAD (11)，如下所述。

每个返回的 `seed[15:0] = entropy` 值在 `OPST=ES16` (`seed[31:30] = 10`) 时表示唯一的随机数，即使其数值与先前轮询的 `entropy` 值相同。`entropy` 位的实现要求在 `cryptoscalaresreq` 中定义。当 `OPST` 不是 ES16 时，`entropy` 必须设置为 0。实现可以安全地将保留和自定义位设置为零。

出于安全原因，接口保证不会多次提供秘密 `entropy` 字。因此，轮询（读取）还必须具有清除（读取即擦除）`entropy` 内容并将状态更改为 WAIT 的副作用（除非 ES16 立即可用）。其他状态（BIST、WAIT 和 DEAD）可能不受轮询影响。

`seed[31:30]=OPST` 返回的状态位:

* 00 - BIST

表示正在执行内置自测试按需（BIST）测试。如果 `OPST` 从任何其他状态暂时返回到 BIST，这表示非致命的自测试警报，这是不可操作的，除了被记录。这样的 BIST 警报必须被锁存，直到至少轮询一次以使软件记录其发生。

* 01 - WAIT

表示尚未有足够的熵可用。这不是错误状态，可能（实际上）比 ES16 更频繁，因为物理熵源通常具有低带宽。

* 10 - ES16

表示成功；低位 `seed[15:0]` 将具有 16 位随机数（`entropy`），无论实现如何，都保证满足某些最低熵要求。

* 11 - DEAD

是不可恢复的自测试错误。这可能表示硬件故障、安全问题，或（极少见）连续测试程序中的 1 类统计假阳性。在致命故障的情况下，立即锁定也可能是专用安全设备中的适当响应。

****示例.**** `0x8000ABCD` 是有效的 ES16 状态输出，其中 `0xABCD` 是 `entropy` 值。`0xFFFFFFFF` 是无效输出（DEAD），没有 `entropy` 值。

32.5.2 熵源要求

由于较小、低功耗实现的硬件和能源限制，输出 entropy（ES16 状态下的 seed[15:0]）不一定是完全调节后的随机性。但是，定义了最低要求。主要要求是，在 256 位输入块中进行 2 比 1 的加密后处理将产生 128 位全熵输出块。熵源用户可以做出这种保守的假设，但不禁止使用相对于所需结果熵而言两倍以上种子位数。

为了被认为是安全可靠的设计，熵源的实现应至少满足以下要求集之一：

- * cryptoscalaresreq90b: 符合 NIST SP 800-90B 标准的物理熵源，评估的最小熵为每 256 输出位 192 位（最小熵率 0.75）。
- * cryptoscalaresreqptg2: 符合 AIS-31 PTG.2 标准的物理熵源，意味着平均 Shannon 熵率为 0.997。源还必须满足 NIST 800-90B 最小熵率 $192/256 = 0.75$ 。
- * cryptoscalaresreqvirt: 虚拟熵源是从物理熵源生成的 DRBG。它必须具有至少 256 位（后量子类别 5）内部安全级别。

所有实现必须按各自标准的要求发出初始化、测试模式和健康警报。这可能需要实现者以安全可靠的方式添加非标准（自定义）测试接口，示例见图 32.1。

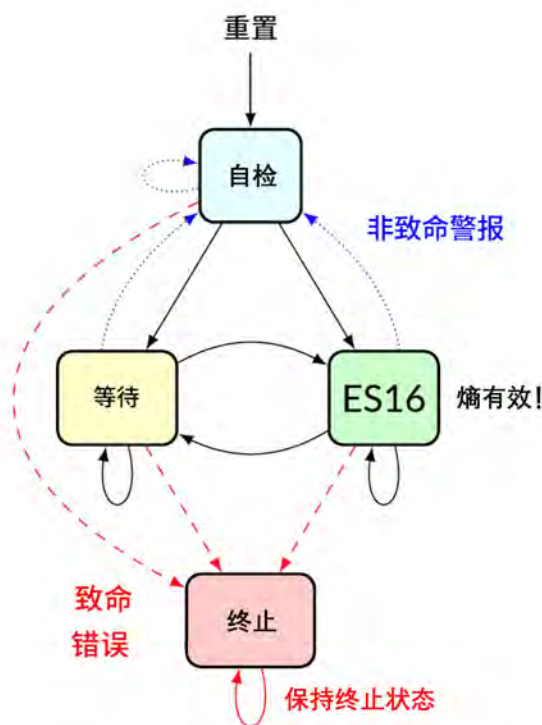


图32.1 熵源状态转移图

必须实现所有 NIST SP 800-90B 要求的组件和健康测试机制。

如果可以使用经过审查的调节算法（例如密码哈希，参见 SP 800-90B 第 3.1.5.1.1 节），从每个 256 位（16*16 位）成功但可能非连续的 entropy（ES16）输出序列中获得 128 位全熵，则满足熵要求。实际上，为此需要最小熵率达到 0.75 或更高。

请注意，在 SP 800-90B/C 评估中，128 位估计的输入最小熵并不能产生 128 位调节后的全熵。相反，其含义是，正如 SP 800-90C 中讨论的那样，每个 256 位序列应具有至少 $128+64 = 192$ 位的最小熵；即使拥有（几乎）不受约束的熵源数据和计算能力，成功猜测单个 256 位输出序列的可能性也不应高于 2^{-192} 。

与其尝试定义熵源必须满足的所有数学和架构属性，不如定义物理熵源必须足够强大和稳健，以通过 NIST SP 800-90 等效评估和认证，以便在使用 2:1 的比例和 128 位输出块进行加密调节时获得全熵。

即使要求是根据 128 位全熵块定义的，我们仍然建议 256 位安全性。这可以通过使用至少 512 个 entropy 位来初始化具有 256 位安全性的 DRBG 来实现。

BSI AIS-31 PTG. 2 / 通用标准要求

对于替代的通用准则认证（或自我认证），必须实施 AIS 31 PTG.2 级别（第 4.3 节）要求的硬件组件和机制。除了 AIS-31 PTG.2 随机性要求（该标准中评估的香农熵率为 0.997）之外，如 cryptoscalaresreq90b 中所述，整体最小熵要求仍然存在。请注意，800-90B 最小熵可能明显低于 AIS-31 香农熵。这两个指标不应等同或混淆。

虚拟源：安全要求

虚拟源不是 ISA 合规性要求。它是为 RISC-V 安全生态系统的利益而定义的，以便虚拟系统可以具有一致的安全级别。

虚拟源不是物理熵源，但提供了对隐蔽通道、耗尽攻击和操作环境中主机识别的额外保护，这些环境不能完全信任直接访问硬件资源。尽管信任有限，实现者应尽量保证即使在这样的环境中也有足够的熵可用于安全的加密操作。

虚拟源捕获对 seed CSR 的访问，模拟它，或以其他方式实现它，可能没有直接访问物理熵源。输出可以是加密安全的伪随机数，而不是实际的熵，但必须具有至少 256 位的安全性，如下定义。虚拟源特别适用于客户操作系统、沙盒、仿真器和类似用例。

作为技术定义，对输出的随机区分攻击应需要与对具有 256 位密钥的安全块密码（例如 AES 256）的穷举密钥搜索相当或更大的计算资源。这适用于经典和量子计算模型，但仅适用于经典信息流。虚拟源安全要求映射到后量子安全类别 5。任何限制安全强度的 seed CSR 实现不得将其降低到低于 256 位。如果安全级别低于 256 位，则接口必须不可用。

虚拟熵源不需要实现 WAIT 或 BIST 状态。如果主机 DRBG 或熵源失败且没有足够的种子材料用于主机 DRBG，则应失败（DEAD）。

32.5.3 seed 的访问控制

默认情况下，seed CSR 仅在 M 模式下可用，但可以通过 mseccfg.sseed 和 mseccfg.useed 访问控制位使其在其他模式下可用。sseed 是 mseccfg CSR 的第 9 位，useed 是第 8 位。如果未将相应的访问控制位设置为 1，则从 U、S 或 HS 模式尝试访问 seed 将引发非法指令异常。

VS 和 VU 模式存在于实现了 Hypervisor (H) 扩展的系统中。如果需要，虚拟机监视器可以模拟来自虚拟机的种子 CSR 访问。尝试从虚拟模式 VS 和 VU 访问 seed 总是引发异常；只读指令会引发非法指令异常，而读写指令（可能会被模拟）仅在 mseccfg.sseed=1 时引发虚拟指令异常。请注意，mseccfg.useed 对 VS 或 VU 模式的异常类型没有影响。

表 32.10 熵源访问控制

模式	Sseed	Useed	描述
M	*	*	seed CSR 在机器模式下始终可用（使用 CSR 读写指令）。尝试在没有写入的情况下读取将引发非法指令异常，无论模式和访问控制位如何。
U	*	0	任何 seed CSR 访问都会引发非法指令异常。
U	*	1	seed CSR 可正常访问。读写不会引发异常。

S/HS	0	*	任何 seed CSR 访问都会引发非法指令异常。
S/HS	1	*	seed CSR 可正常访问。读写不会引发异常。
VS/VU	0	*	任何 seed CSR 访问都会引发非法指令异常。
VS/VU	1	*	读写 seed 访问引发虚拟指令异常，而其他访问条件引发非法指令异常。

系统应实施从较低特权模式到物理熵源的仔细考虑的访问控制策略。系统可以捕获对 seed 的访问尝试，并向较低特权客户端提供虚拟熵源数据 而不是调用 SP 800-90B 或 PTG.2 物理熵源。模拟的 seed 数据生成是通过适当种子的安全软件 DRBG 进行的。有关直接访问熵源的安全考虑。

实现可以实现 mseccfg,使得 [s,u]seed 是只读常量值 0。软件可以通过将 1 写入 [s,u]seed 并读取结果来发现是否可以在 U 和 S 模式下启用对 seed CSR 的访问。

如果未实现 S 或 U 模式，则 mseccfg 的相应 [s,u]seed 位必须硬连线为零。[s,u]seed 位必须具有定义的复位值。系统不得允许它们在复位后处于 UNSPECIFIED 状态。如果实现了 Zkr，则存在 mseccfg，或者如果其他处理器功能需要它。如果未实现 Zkr，则 [s,u]seed 位必须硬连线为零。

32.6 数据独立执行延迟子集：Zkt

Zkt 扩展证明机器具有数据独立的执行时间，用于安全的指令子集。此属性通常称为恒定时间，尽管不应按字面意思理解。

所有当前提议的加密指令（标量 K 扩展）都在此列表中，以及来自 I、M、C 和 B 扩展的一组相关支持指令。



给软件开发人员的注意事项

未能防止通过直接时间通道泄漏敏感参数被视为严重的安全漏洞，通常会导致 CERT CVE 安全公告。

32.6.1 范围和目标

程序员和 RISC-V 实现之间建立了一个 ISA 契约，即 Zkt 指令不会通过执行延迟的差异泄露有关已处理的秘密数据（明文、密钥信息或其他敏感安全参数 FIPS 140-3 术语）的信息。Zkt 没有定义核中可用的一组指令；它只是限制了某些指令的行为（如果这些指令被实现）。

目前，本文档的范围仅限于标量 RV32/RV64 处理器。向量密码指令（和适当的向量支持指令）将在稍后添加，其他希望声明无泄漏执行延迟属性的安全相关功能也将如此。

排除了加载、存储、条件分支以及一组在处理秘密数据时很少需要的指令。同样排除的是那些由于其他 ISA 处理器的限制，在标准密码中间件中存在变通方法的指令。

声明的目标是，当在启用 Zkt 的 RISC-V 目标上编译和运行时，OpenSSL、BoringSSL (Android)、Linux 内核和类似的受信任软件不会有直接可观察到的定时侧信道。Zkt 扩展明确声明了加密开发者做出的许多常见延迟假设。供应商不必实现列表中的所有指令才能符合 Zkt 标准；然而，如果他们声称拥有 Zkt 并实现了任何列出的指令，则它必须具有与数据无关的延迟。

例如，许多简单的 RV32I 和 RV64I 内核（没有乘法、压缩、位操作或加密扩展）在技术上都符合 Zkt 标准。可以在它们上使用位切片技术实现恒定时间的 AES，但与使用 AES 指令的实现相比，它将极其缓慢。即使是基于位切片的密码实现（主要基于布尔逻辑指令），也不能保证在没有 Zkt 认证的内核上是安全的。

遵守 Zkt 的乱序实现仍然可以自由地融合、分解、更改甚至忽略指令序列，只要优化是确定性地应用的，而不是基于操作数数据。指导原则应该是，不应基于执行延迟泄露有关正在操作的数据的任何信息。



在具有高级乱序执行能力（使用值预测或以其他方式依赖于数据）的实现中，数据无关执行的问题有待未来的扩展或其他技术来解决。

. 给软件开发人员的注意事项



编程技术只能减轻由算术、缓存和分支直接引起的泄漏。其他 ISA 存在微架构问题，如 Spectre、Meltdown、Speculative Store Bypass、Rogue System Register Read、Lazy FP State Restore、Bounds Check Bypass Store、TLBleed 和 LITF/Foreshadow 等。参见例如 [link:https://github.com/nsacyber/Hardware-and-Firmware-Security-Guidance](https://github.com/nsacyber/Hardware-and-Firmware-Security-Guidance)。

本提案的职责范围不包括减轻这些微架构泄漏。

32.6.2 背景

- * 时间攻击比 2010 年之前意识到的更强大，这导致了当前加密代码库中的重大缓解努力。
- * 加密开发人员使用静态和动态安全测试工具来跟踪秘密信息的处理，并检测其影响分支或用于表查找的情况。
- * Zkt 的架构测试可以是务实和半正式的；通过有意识地实现（相关的迭代多周期指令或由微操作组成的指令）以避免数据依赖的延迟，通常可以实现对基本时间攻击的设计安全。
- * 实验室测试可以利用 ISO/IEC 17825 中描述的统计时间攻击泄漏分析技术。
- * 二进制可执行文件不应在指令编码中包含秘密（Kerckhoffs 原则），因此指令时间可能泄露关于立即数、输入寄存器顺序等的信息。在某些系统中，二进制加载器修改可执行文件以进行重定位，并且希望保持执行位置（PC）秘密时，可能对此有例外。这就是为什么 LUI、AUIPC 和 ADDI 等指令在列表中的原因。
- * 审计工具使用的规则相对简单易懂。简而言之；我们称明文、秘密密钥、扩展密钥、随机数和其他此类变量为秘密。一个秘密变量（算术上）修改任何其他变量/寄存器也将其变为秘密。如果一个秘密最终影响加载或存储的地址计算，这是一个违规。如果一个秘密影响分支的条件，这也是一个违规。一个秘密变量位置或寄存器通过特定的清零/清理或被声明为密文（或其他不再是秘密的信息）变为非秘密。本质上，秘密在它们是秘密时只能触及 Zkt 列表中的指令。

32.6.3 特定指令理由

- * HINT 指令形式（通常是 `rd=x0` 的编码）不受数据独立时间要求的约束。
- * 浮点（F、D、Q、L 扩展）目前不受恒定时间要求的约束，因为它们标准化加密中应用很少。如果某个特定算法（如 PQC 签名算法 Falcon）变得至关重要，我们可能会考虑将浮点加、减、乘作为某些浮点扩展的恒定时间要求。
- * 密码学家通常假设除法是可变时间的（而乘法是恒定时间的），并以此假设实现他们的 Montgomery 约简例程。
- * Zicr、Zifencei 被排除在外。
- * 一些指令在列表中只是因为我们认为将它们包括在测试范围内无害。

32.6.4 编程信息

有关安全编程模型的背景信息，请参见：

- * Thomas Pornin: 为什么恒定时间加密？（对时间假设的一个很好的介绍。）

<https://www.bearssl.org/constanttime.html>

* Jean-Philippe Aumasson: 低级加密软件指南。（一份推荐列表。）

<https://github.com/veorq/cryptocoding>

* Peter Schwabe: 时间攻击和对策。（讲义幻灯片 很好的参考资料。）

<https://summerschool-croatia.cs.ru.nl/2016/slides/PeterSchwabe.pdf>

* Adam Langley: ctgrind。（这是 2010 年的，但仍然相关。）

<https://www.imperialviolet.org/2010/04/01/ctgrind.html>

* Kris Kwiatkowski: 使用内存消毒器验证恒定时间代码。

<https://www.amongbytes.com/post/20210709-testing-constant-time/>

* 有关时间攻击漏洞的早期示例, 请参见 <https://www.kb.cert.org/vuls/id/997481> 和相关学术论文。

32.6.5 Zkt 列表

以下指令包含于 Zkt 子集中它们在此按其原始父扩展分组列出。

您不需要实现所有这些指令来实现 Zkt。相反, 核实现的每一条这些指令都必须遵守 Zkt 的要求。

仅包括基本算术和 slt*（用于进位计算）。数据独立时间要求不适用于这些指令的 HINT 编码形式。

表32.11 基本指令

RV32	RV64	助记符
✓	✓	lui rd, imm
✓	✓	auipc rd, imm
✓	✓	addi rd, rsl, imm
✓	✓	slti rd, rsl, imm
✓	✓	sltiu rd, rsl, imm
✓	✓	xori rd, rsl, imm
✓	✓	ori rd, rsl, imm
✓	✓	andi rd, rsl, imm
✓	✓	slli rd, rsl, imm
✓	✓	srli rd, rsl, imm
✓	✓	srai rd, rsl, imm
✓	✓	add rd, rsl, rs2
✓	✓	sub rd, rsl, rs2
✓	✓	sll rd, rsl, rs2
✓	✓	slt rd, rsl, rs2
✓	✓	sltu rd, rsl, rs2
✓	✓	xor rd, rsl, rs2
✓	✓	srl rd, rsl, rs2
✓	✓	sra rd, rsl, rs2
✓	✓	or rd, rsl, rs2
✓	✓	and rd, rsl, rs2
	✓	addiw rd, rsl, imm
	✓	slliw rd, rsl, imm

	✓	srlw rd, rs1, imm
	✓	sraiw rd, rs1, imm
	✓	addw rd, rs1, rs2

32.6.6 RVM（乘法）

表32.12 乘法；除法和取余被排除

RV32	RV64	助记符
✓	✓	mul rd, rs1, rs2
✓	✓	mulh rd, rs1, rs2
✓	✓	mulhsu rd, rs1, rs2
✓	✓	mulhu rd, rs1, rs2
	✓	mulw rd, rs1, rs2

32.6.7 RVC（压缩）

表32.13 RVI 中的标准

RV32	RV64	助记符
✓	✓	c.nop
✓	✓	c.addi
	✓	c.addiw
✓	✓	c.lui
✓	✓	c.srli
✓	✓	c.srai
✓	✓	c.andi
✓	✓	c.sub
✓	✓	c.xor
✓	✓	c.or
✓	✓	c.and
	✓	c.subw
	✓	c.addw
✓	✓	c.slli
✓	✓	c.mv
✓	✓	c.add

32.6.8 RVK（标量加密）

包括所有 K 特定指令。此外，seed CSR 的延迟应独立于 ES16 状态输出的 entropy 位，因为这是一个敏感的安全参数。

表32.14 按照象限组织

RV32	RV64	助记符
√		aes32dsi
√		aes32dsmi
√		aes32esi
√		aes32esmi
	√	aes64ds
	√	aes64dsm
	√	aes64es
	√	aes64esm
	√	aes64im
	√	aes64ksli
		aes64ks2
√		sha256sig0
√		sha256sigl
√		sha256sum0
√		sha256suml
√		sha512sig0h
√	√	sha512sig0l
√	√	sha512siglh
√	√	sha512sigll
√	√	sha512sum0r
√	√	sha512sumlr
	√	sha512sig0
	√	sha512sigl
	√	sha512sum0
	√	sha512suml
√	√	sm3p0

32.6.9 RVB（位操作）

包括 zbkb、zbkc 和 zbkx 扩展的全部内容。

表32.15 RVB指令

RV32	RV64	助记符
√	√	clmul
√	√	clmulh
√	√	xperm4
√	√	xperm8
√	√	ror
√	√	rol
√	√	rori
	√	rorw

	✓	rolw
	✓	roriw
	✓	andn
✓	✓	orn
✓	✓	xnor
✓	✓	pack
✓	✓	packh
✓	✓	packw
✓	✓	brev8
✓	✓	rev8
✓		zip
✓		unzip

32.7 指令设计原理

本节包含标量加密扩展中指令的各种理由、设计说明和使用建议。它还尝试记录指令设计的来源或贡献者。

32.7.1 AES 指令

32 位指令源于 Saarinen 2020 年论文中的工作，并为 RISC-V 加密扩展做出了贡献。64 位指令是由我们邮件列表上的任务组成员协作开发的。

32.7.2 SHA2 指令

这些指令基于布里斯托大学 XCrypto 项目中的学术工作开发，并贡献给 RISC-V 加密扩展。RV32 SHA2-512 指令基于此工作，并在 Marshall 等人 2019 年的论文中开发，然后以相同方式贡献。

32.7.3 SM3 和 SM4 指令

SM4 指令来源于 Saarinen 2020 年论文中的工作，因此与 RV32 AES 指令非常相似。SM3 指令受 SHA2 指令启发，并基于 Saarinen 2020 年论文中的开发工作，然后贡献给 RISC-V 加密扩展。

32.7.4 用于加密的位操作指令

对称密钥加密和加密哈希函数中使用的许多基本操作都得到了 RISC-V 位操作扩展的良好支持。



本节重复了 zbkb、zbkc 和 zbkx 中的许多信息，但包含更多理由。

我们建议标量加密扩展重用位操作扩展 Zb[abc] 的一部分指令。具体来说，这意味着实现任一标量加密扩展，或 Zb[abc]，或两者的核，都需要实现这些指令。

标准旋位操作：

RV32, RV64:

```
ror    rd, rs1, rs2
rol    rd, rs1, rs2
rori   rd, rs1, imm
```

RV64 only:

```
rorw   rd, rs1, rs2
rolw   rd, rs1, rs2
roriw  rd, rs1, imm
```

给软件开发人员的注意事项

标准按位旋转是许多分组密码和哈希函数中的基本操作；它特别出现在 ARX（加法、旋转、异或）类分组密码和流密码中。



* 使用 32 位旋转的算法：

SHA256、AES（移位行）、ChaCha20、SM3。

* 使用 64 位旋转的算法：

SHA512、SHA3。

位和字节置换：

RV32：

```
brev8    rd, rs1 // grevi rd, rs1, 7 - Reverse bits in bytes
rev8     rd, rs1 // grevi rd, rs1, 24 - Reverse bytes in 32-bit word
```

RV64：

```
brev8    rd, rs1 // grevi rd, rs1, 7 - Reverse bits in bytes
rev8     rd, rs1 // grevi rd, rs1, 56 - Reverse bytes in 64-bit word
```

标量加密扩展提供以下指令来操作数据的位和字节字节序。它们都是带立即数的广义反转（grevi）指令的参数化。标量加密扩展仅要求实现上述 grevi 实例，可以通过其伪操作调用。

grevi 指令的完整规范见 第 2.2.2 节。

给软件开发人员的注意事项



在设置输入和输出数据的标准字节序时，反转字中的字节在加密中非常常见。字节内的位反转用于实现 Galois/Counter 模式（GCM）的 GHASH 组件。

RV32：

```
zip      rd, rs1 // shfli  rd, rs1, 15 - Bit interleave
unzip    rd, rs1 // unshfli rd, rs1, 15 - Bit de-interleave
```

zip 和 unzip 伪操作是更通用的 shfli 和 unshfli 指令的特定实例。标量加密扩展仅要求实现上述 [un]shfli 实例，可以通过其伪操作调用。仅 RV32 实现需要这些指令。

shfli 指令的完整规范见 第 2.2.3 节。

给软件开发人员的注意事项



这些指令执行位交错（或反交错）操作，对于在 32 位架构上实现 SHA3 算法中的 64 位旋转非常有用。在 RV64 上，SHA3 中的相关操作可以使用旋转指令本地完成，因此不需要 zip 和 unzip。

无进位乘法

RV32, RV64:

```
clmul rd, rs1, rs2
clmulh rd, rs1, rs2
```

有关此指令的详细信息，请参见第 2.6 节。有关这些指令的其他实现要求，请参见 cryptoscalarzkt，与数据独立执行延迟相关。



给软件开发人员的注意事项

正如那里提到的，无进位乘法的明显加密用例是 Galois 计数器模式（GCM）分组密码操作。GCM 被 NIST 推荐为分组密码操作模式，并且是 TLS 1.3 协议的唯一必需模式。

带否定的逻辑

RV32, RV64:

```
andn rd, rs1, rs2
orn rd, rs1, rs2
xnor rd, rs1, rs2
```

有关这些指令的详细信息，请参见第 2.1.3 节。这些指令在哈希函数、分组密码中非常有用，并用于实现基于软件的侧信道对策，如掩码。在没有三元位操作 cmov 指令的系统中，andn 指令对于恒定时间字选择也很有用。



给软件开发人员的注意事项

在加密上下文中，这些指令有助于：SHA3/Keccak Chi 步骤，位切片函数实现，基于软件的功率/EM 侧信道对策基于掩码。

打包

RV32, RV64:

```
pack rd, rs1, rs2
packh rd, rs1, rs2
```

RV64:

```
packw rd, rs1, rs2
```

给软件开发人员的注意事项



pack* 指令对于重新排列字中的半字非常有用，并且通常在应用变换之前将数据调整到正确的形状。这对于将输入作为（可能未对齐的）字节字符串传递但可以对这些字节字符串组成的字进行操作的加密算法特别有用。例如，在将块和密钥（可能未字对齐）加载到寄存器中以执行轮函数时，这在 AES 中发生。

交叉开关置换指令

RV32, RV64:

```
xperm4 rd, rs1, rs2
xperm8 rd, rs1, rs2
```

xperm4 指令对半字节操作。

GPR[rs1] 包含一个 $XLEN/4$ 个 4 位元素的向量。

GPR[rs2] 包含一个 $XLEN/4$ 个 4 位索引的向量。

结果是 GPR[rs2] 中的每个元素被 GPR[rs1] 中的索引元素替换,如果 GPR[rs2] 中的索引超出范围,则为零。

xperm8 指令对字节操作。

GPR[rs1] 包含一个 $XLEN/8$ 个 8 位元素的向量。

GPR[rs2] 包含一个 $XLEN/8$ 个 8 位索引的向量。

结果是 GPR[rs2] 中的每个元素被 GPR[rs1] 中的索引元素替换,如果 GPR[rs2] 中的索引超出范围,则为零。

给软件开发人员的注意事项

该指令可用于实现任意位置换。对于加密,它们可以加速位切片实现、分组密码的置换层、基于掩码的对策和 SBox 操作。



使用 4 位 SBox 的轻量级分组密码包括: PRESENT, Rectangle, GIFT, Twine, Skinny, MANTIS, Midori。

使用 8 位 SBox 的国家密码包括: Camellia (日本), Aria (韩国), AES (美国, 比利时), SM4 (中国), Kuznyechik (俄罗斯)。

所有这些 SBox 都可以使用 xperm8 指令高效地、恒定时间地实现。请注意,该技术也适用于基于掩码的侧信道对策。

32.8 熵源设计理由和建议

本非规范性附录重点介绍熵源的理由、安全性、自我认证和实现方面。因此,我们还讨论了可能需要的非 ISA 系统功能,以符合加密标准和安全测试。

32.8.1 设计和自我认证检查表

加密系统的安全性基于秘密位和密钥。这些位需要是随机的,并且来自加密安全的随机位生成器(RBG)。构建安全的 RBG 需要熵源(ES)。

虽然熵源实现不必是经过认证的设计,但 RISC-V 期望它们表现兼容,并且不会给用户带来不必要的安全风险。通常需要遵循适当的安全标准进行自我评估和测试以实现这一目标。

ISA 架构测试在可能的范围内,验证本规范中的 RISC-V ISA 要求是否正确实现。这包括状态转换和访问控制,以及 seed ES16 entropy 字只能被破坏性读取。

RISC-V ISA 架构测试的范围是那些独立于物理熵源细节的行为。在设计阶段,烟雾测试 ES 模块可能会有所帮助。

熵的技术论证。这可以采取随机模型或启发式论证的形式,解释为什么噪声源输出来自随机而不是伪随机(确定性)过程,并且不容易预测或外部可观察。

不需要完整的物理模型;可以引用研究文献。例如,可以表明一个好的环形振荡器噪声从局部自发发生的 Johnson-Nyquist 热噪声中导出一定量的物理熵,因此不仅仅是看起来随机。

熵源设计审查。熵源不仅仅是噪声源,还必须具有健康测试、调节器和具有明确定义接口的安全边界等功能。可以列出 SP 800-90B、FIPS 140-3 实施指南或其他使用的标准的 SHALL 声明。官方和非官方的检查表可用: <https://github.com/usnistgov/90B-shall-statements>

实验测试。原始噪声源根据 NIST 800-90B 第 3 节中定义的熵估计进行测试。

描述的接口可用于记录数据集以用于此目的。还需要实验性地证明调节器和健康测试组件适当地工作，以满足 ES16 输出熵要求。

对于 SP 800-90B，NIST 提供了一个免费的熵估计包：
<https://github.com/usnistgov/SP800-90BEntropyAssessment>

弹性。上述物理工程步骤应考虑设备的操作环境，这可能是意外的或敌对的（积极尝试利用设计中的漏洞）。



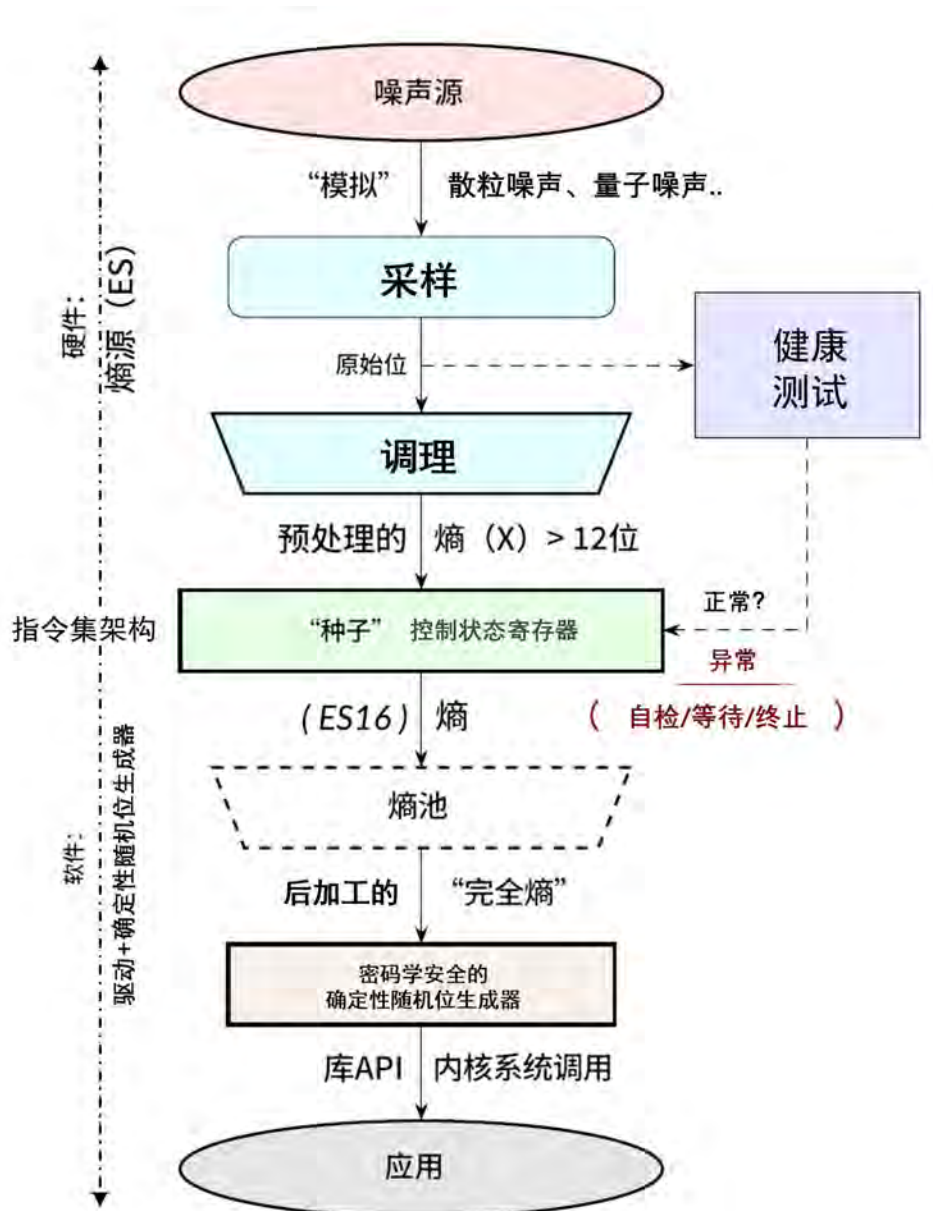
RISC-V 熵源规范的目标之一是可以从第三方许可标准 90B 熵源模块或 AIS-31 RNG IP，并与 RISC-V 处理器设计集成。与旧的（FIPS 140-2）RNG 和 DRBG 模块相比，熵源模块的面积可能相对较小（仅几千个 NAND2 门等效）。CMVP 正在引入熵源验证范围，这可能允许 90B 验证用于不同的（FIPS 140-3）模块。

32.8.2 标准和术语

作为一种基本的安全功能，随机数的生成受许多标准和技术评估方法的约束，主要是 FIPS 140-3（美国联邦使用要求）和通用标准方法（国际高安全性评估使用）。

请注意，FIPS 140-3 是一个与其前身 FIPS 140-2 相比显著更新的标准，仅在 2020 年代开始使用。这些标准设定了 RISC-V 熵源设计的许多技术要求，如果可能，我们使用它们的术语。

seed CSR 提供了一个熵源（ES）接口，而不是有状态的随机数生成器。因此，它可以支持任意安全级别。可以使用加密（AES、SHA-2/3）ISA 扩展从熵源构建高速 DRBG。



熵源 (ES)

熵源通过采样和处理来自噪声源的数据 (cryptoscalarappxesnoisesources) 构建。在这项工作中，我们只考虑真正随机的物理来源。由于这些直接基于自然现象并受环境条件（可能是对抗性的）影响，因此它们需要监控这些来源的健康和质量的功能。

物理熵源的要求在 NIST SP 800-90B (cryptoscalaresreq90b) 中规定，用于美国联邦 FIPS 140-3 评估，以及在 BSI AIS-31 (cryptoscalaresreqptg2) 中规定，用于高安全性通用标准评估。这些标准在健康测试和熵度量类型上存在一些差异，RISC-V 支持这两种替代方案。

调节：加密和非加密

原始物理随机性（噪声）源很少是统计上完美的，有些会生成大量的位，需要去偏并减少到较少的位数。这一过程称为调节。安全哈希函数是加密调节器的一个例子。重要的是要注意，即使哈希可以使任何数据看起来随机，它也不会增加其熵含量。

非加密调节器和提取器，如 von Neumann 的去偏硬币投掷 更容易高效实现，但可能比加密哈希（非常高效地混合输入熵）减少更多的熵含量（在移除的单个位中）。然而，它们不需要密码分析或计算难

度假设，因此本质上更具未来性。

伪随机数生成器 (PRNG)

伪随机数生成器 (PRNG) 使用确定性数学公式从较少的种子随机数生成大量随机数。PRNG 也分为加密和非加密的。

非加密 PRNG，如 LFSR 和许多编程库中的线性同余生成器，可能生成统计上令人满意的随机数，但绝不能用于加密密钥。这是因为它们不是为了抵抗密码分析而设计的；通常可以从一些输出中数学推导出 PRNG 的种子或内部状态。这是一个安全问题，因为了解状态允许攻击者计算未来或过去的输出。

确定性随机位生成器 (DRBG)

加密 PRNG 也称为确定性随机位生成器 (DRBG)，这是 SP 800-90A 使用的术语。使用强加密算法如 AES 或 SHA-2/3 从种子生成随机位。秘密种子材料就像加密密钥；从 DRBG 输出确定种子与破解 AES 或强哈希函数一样困难。这也说明种子/密钥需要足够长，并来自可信的熵源。DRBG 仍应频繁刷新（重新播种）以确保前向和后向安全性。

32.8.3 具体理由和考虑

接口设计简单，以便供应商和设备无关的驱动程序组件（例如，在 Linux 内核、嵌入式固件或加密库中）可以使用 seed 生成真正的随机位。

熵源不需要高带宽接口；单个 DRBG 源初始化只需要 512 位（256 位熵），DRBG 输出可以由任意数量的调用者共享。一旦启动，DRBG 只需要新的熵来减轻状态泄露的风险。

从安全角度来看，读取时刷新秘密熵位的副作用是必不可少的。因此，我们要求在此特定 CSR 上进行写操作。

阻塞指令可能更容易使用，但大多数用户应该查询 (D)RBG 而不是熵源。没有轮询机制，熵源在某些情况下可能会挂起数千个周期。wfi 或 pause 机制（至少可能）允许在 MCU 上节能睡眠，并在高端 CPU 上进行上下文切换。

特定 OPST = seed[31:0] 两位机制的原因是提供冗余。如果功能发现失败且熵源实际上不可用，则由于电气原因，故障位组合 11 (DEAD) 和 00 (BIST) 更可能出现。

16 位带宽是一个折衷，目的是在返回值中提供冗余，对潜在的功率/EM 泄漏提供一些保护，并希望编程方便性上在 RV32 和 RV64 架构上所有位都在同一个地方。

NIST SP 800-90B

SP 800-90C 规定，每个 n 位的调节块需要 $n+64$ 位输入熵才能达到完全熵。因此，NIST SP 800-90B 最小熵评估必须保证每 256 位块至少 $128 + 64 = 192$ 位输入熵。只有这样，从熵源哈希的 $16 * 16 = 256$ 位才能产生所需的 128 位完全熵。这是根据 SP 800-90C 附录 A 中包含的具体要求、威胁模型和可区分性证明得出的。隐含的最小熵率为 $192/256=12/16=0.75$ 。预期的 Shannon 熵要大得多。

在 FIPS 140-3 / SP 800-90 分类中，RBG2(P) 构造是一个具有连续访问物理熵源 (seed) 的加密安全 RBG，并由完全播种的安全 DRBG 生成输出。熵源还可以用于构建 RBG3 完全熵源。输出字的连接对应于 GetESBitstring 函数。

选择 128 位输出块大小是因为这是附录 F 中指定的 CBC-MAC 调节器的输出大小，也是我们期望在应用中看到的最小密钥大小。

如果选择 NIST SP 800-90B 认证，熵源应至少实现第 4.4 节中定义的健康测试：重复计数测试和自适应比例测试，或证明供应商定义的测试将检测到相同的缺陷。

PTG.2 是 BSI AIS 20/31 中定义的安全性和功能性类别之一。PTG.2 源要求作为其他类型 BSI 生成器（例如，DRBG 或具有适当软件后处理的 PTG.3 TRNG）的构建块。

为了验证目的，PTG.2 要求可以映射到安全控制 T1-3 和接口，如下所示：

- * P1 * [PTG.2.1] * 启动测试映射到 T1 和复位触发的（按需）BIST 测试。
- * P2 * [PTG.2.2] * 连续测试总失败映射到 T2 和 DEAD 状态。

* P3 *[PTG. 2. 3]* 在线测试是 T2 的连续测试 - 在 BIST 状态下禁止熵输出。
* P4 *[PTG. 2. 4]* 与设计有效熵源健康测试相关，我们鼓励这样做。
* P5 *[PTG. 2. 5]* 原始随机序列可以通过 GetNoise 接口 (cryptoscalesgetnoise) 检查。
* P6 *[PTG. 2. 6]* 测试程序 A (第 2. 4. 4. 1 节) 是评估过程的一部分，我们建议即使不寻求 AIS-31 认证，也使用这些测试进行自我评估。

* P7 *[PTG. 2. 7]* 内部随机位的平均 Shannon 熵超过 0.997。

注意 P7 关注的是 Shannon 熵，而不是 NIST 源的最小熵。因此，还需要说明最小熵要求。根据 AIS-31 标准构建和认证的 PTG. 2 模块也可以在 2:1 加密调节后满足完全熵条件，但不一定如此。技术验证过程有所不同。

虚拟源

所有不直接物理源 (符合 SP 800-90B 或 AIS-31 PTG. 2 要求) 的源都需要满足虚拟熵源的安全要求。假设虚拟熵源不是限制性、共享带宽资源 (而是软件 DRBG)。

DRBG 可用于馈送其他 (虚拟) DRBG，但这不会增加系统中的绝对熵量。熵源必须能够支持当前和未来的安全标准和应用。256 位要求映射到 NIST 后量子加密的类别 5 (4. A. 5 安全强度类别) 和 Suite B 以及更新的美国政府 CNSA Suite 中的绝密方案。

直接硬件访问的安全考虑

ISA 实现和系统设计必须尽量确保硬件-软件接口最小化对手信息流的途径，即使规范中没有明确禁止。

为了安全，虚拟化需要对物理熵输出进行调节和 DRBG 处理。如果单个物理熵源在多个不同的虚拟机之间共享，或者客户操作系统不受信任，建议这样做。虚拟熵源对耗尽攻击更具抵抗力，也减少了隐蔽通道的风险。

直接 msecfg.[s,u]seed 选项允许围绕组件绘制安全边界，以便与敏感安全参数 (SSP) 流相关，即使该组件不在 M 模式下。这在实现受信任的飞地时很有帮助。这样的模块可以强制整个密钥生命周期从生成 (在熵源中) 到销毁 (清零) 都在不将密钥传递给外部代码的情况下发生。

耗尽

主动轮询可能会拒绝同时运行的另一个消费者使用熵源。例如，如果需要熵来完全初始化，这可能会延迟该虚拟机的实例化。

隐蔽通道

直接访问组件 (如熵源) 可以用于跨越安全边界建立通信通道。一个消费者的主动轮询使资源对另一个 (轮询不频繁的) 不可用 (WAIT 而不是 ES16)。这种交互可以用于建立低带宽通道。

硬件指纹识别

熵源 (及其噪声源电路) 可能具有唯一可识别的硬件签名。在某些应用中，这可能是无害的甚至有用的 (因为随机源可能表现出类似物理不可克隆函数 (PUF) 的特性)，但在其他应用中 (匿名虚拟化环境和飞地) 则是非常不希望的。DRBG 掩盖了这种统计特征。

侧信道

一些对现实生活中密码系统最具破坏性的实际攻击使用了看似无关紧要的附加信息，如填充错误消息或时间信息。

我们敦促实现者不要通过状态或自定义位创建不必要的信息流，也不要允许任何其他机制禁用或影响熵源输出。所有信息流和交互机制都必须从对手的角度考虑：越少越好。

作为侧信道分析的一个例子，我们注意到熵轮询接口通常不是恒定时间的。需要分析通过时间预言

机揭示了什么样的信息；一种方法是將 seed 模型化为拒绝采样器。这样的时间预言机可以揭示有关噪声源类型和熵源使用的信息，但不能揭示随机输出 entropy 位本身的信息。如果确实如此，则需要额外的对策。

32.8.4 安全控制和健康测试

加密熵源的主要目的是生成秘密密钥材料。在几乎所有情况下，硬件熵源必须实现适当的安全控制以保证不可预测性，防止泄漏，检测攻击，并拒绝对手对熵输出或其生成机制的控制。安全测试和认证需要明确的安全控制。

内置于设备中的许多安全控制称为健康检查。健康检查可以采取完整性检查、启动测试和按需测试的形式。这些测试可以在硬件或固件中实现，通常是两者兼有。NIST SP 800-90B 等标准要求进行几项测试。适当健康测试的选择取决于认证目标、系统架构、威胁模型、熵源类型和其他因素。

健康检查不是为了硬件诊断，而是为了检测安全问题。因此，失败时的默认操作应旨在控制损害：限制进一步的输出并防止生成弱密码密钥。

我们讨论了三个特定的测试要求 T1-T3。测试要求源于熵源的定义；没有它，模块只是一个噪声源，不能安全地生成密钥材料。

T1：按需测试

通过重置、重启或启动硬件（不是 ISA 信号）调用一系列简单测试。在初始启动自测试期间，实施将返回 BIST；在任何情况下，驱动程序必须等待它们完成后才能开始密码操作。失败时，熵源将进入无输出 DEAD 状态。

理由

从软件方面与硬件自测试机制的交互应尽量减少；按需一词并不意味着最终用户或应用程序应该能够在现场调用它们（该术语是对离散、非自主密码设备与人工操作员时代的回溯）。

T2：连续检查

如果在连续测试或环境传感器中检测到错误，熵源将进入无输出状态。我们定义，如果熵源从活动状态（WAIT 或 ES16）返回到 BIST 状态，则发出非关键警报。关键故障将立即导致 DEAD 状态。基于硬件的连续测试机制不得使统计信息外部可用，并且必须定期或通过重置、启动或类似信号按需清零。

物理攻击可能在设备运行时发生。设计应避免通过揭示详细状态信息来引导此类主动攻击。检测到攻击时，默认操作应旨在控制损害 - 防止生成弱密码密钥。

某些测试的统计性质使 1 型假阳性成为可能。可能还需要发出非致命警报；AIS 31 规定了噪声警报，即使设备正常工作，也可能以非可忽略的概率触发；这些可以通过 BIST 发出信号。在无操作员的自主系统中，通常没有什么可以或应该对非致命警报条件采取措施。统计运行时健康检查（如计数器的状态可能与某些秘密密钥材料相关，因此需要清零。

T3：致命错误状态

由于大多数密码操作的安全性取决于熵源，因此对于大多数熵源故障，系统范围的默认拒绝安全策略方法是适当的。硬件测试失败至少应导致 DEAD 状态，并可能导致重置/停止。这是一个阻止因素：如果不能保证其安全操作，则不得允许熵源（或其密码客户端应用程序）运行。

这些测试可以补充其他完整性和防篡改机制（参见第 18 章的示例）。某些硬件随机生成器由于其物理结构，暴露于相对非对抗性的环境和制造问题。然而，即使是这样的无辜故障模式也可能表明故障攻击，因此应作为系统完整性故障而不是诊断问题来处理。

安全架构师将理解仅在测试阈值在整个设备生命周期内的假阳性概率可以忽略不计时，才使用永久或难以恢复的安全熔断锁定。

一些对现实生活中密码系统最具破坏性的实际攻击使用了看似无关紧要的附加信息，如填充错误消息或时间信息。在加密中，这种带外信息源称为预言机。

为了保证没有敏感数据被读取两次，并且不同的调用者不会获得相关输出，要求硬件在每次读取（成功轮询）期间在随机路径上实现读取即擦除。出于同样的原因，只能通过 entropy（seed 的 ES16 状态）提供完整且完全处理的随机字。

这也适用于原始噪声源。原始源接口已委托给可选的供应商特定测试接口。重要的是，测试接口和主接口不应同时运行。

噪声源状态应尽可能保护免受对手的了解或影响。用于此目的的方法应记录，包括（概念）安全边界在保护噪声源免受对手观察或影响中的作用。

熵源是一个单一资源，受耗尽和隐蔽通道的影响。观察熵可能与观察噪声源输出相同，因为加密调节仅作为后处理步骤是强制性的。SP 800-90B 和其他安全标准要求保护噪声位免受观察和影响。

32.8.5 实现策略

作为一般规则，RISC-V 仅指定 ISA。我们提供一些额外的建议，以便可以创建可移植的、供应商无关的中间件和内核组件。实际的硬件实现和认证留给供应商和电路设计师；本节中的讨论纯粹是信息性的。

在考虑实现选项和权衡时，必须查看整个信息流。

- 噪声源从稳定且理解良好的物理随机事件中生成私有的、不可预测的信号。
- 采样将噪声信号数字化为原始比特流。这些原始数据也需要设计保护。
- 连续健康测试确保噪声源及其环境符合其操作参数。
- 非加密调节器 去除输入噪声中的大部分偏差和相关性。
- 加密调节器生成完全熵输出，完全无法与理想随机区分。
- DRBG 将 ≥ 256 位种子熵作为密钥材料，并使用 单向 加密过程快速生成按需比特（不泄露种子/状态）。

步骤 1-4（可能还有 5）被视为熵源（ES）的一部分，并由 seed CSR 提供。添加软件端的加密步骤 5-6 和控制逻辑将其补充为真正的随机数生成器（TRNG）。

环形振荡器

我们将给出一些可以在处理器本身中实现的常见噪声源的示例（使用标准单元）。

目前生产中最常见的熵源类型基于自由运行环形振荡器及其时序抖动。这里，奇数个反相器连接成一个环，从中根据参考时钟采样噪声源比特。如果采样率适当低，采样的比特序列预计是相对不相关的（接近 IID）。然而，通常需要进行进一步处理。

AMD、ARM 和 IBM 是用于高安全性应用的环形振荡器 TRNG 的示例。还有一些相关的基于亚稳态的生成器设计，如过渡效应环形振荡器（TERO）。差分/反馈 Intel 构造略有不同，但也属于同一类基于亚稳态振荡器的类别。

环形振荡器的主要优点是：（1）它们可以使用标准单元库实现，无需外部组件——甚至可以在 FPGA 上实现（2）它们的行为有成熟的理论支持，（3）有大量先例可以在最高安全级别进行测试和认证。

环形振荡器也有众所周知的实现陷阱。它们的输出有时高度依赖于温度，这必须在测试和建模中考虑。如果环形振荡器结构是并行化的，重要的是每个链中的级数和/或反相器数量适当，以避免由于谐波惠更斯同步引起的熵减少。这种谐波也可以在频率注入攻击中恶意插入，这可能会产生灾难性结果。对策与电路设计有关；环境传感器、电气滤波器和使用差分振荡器可能有帮助。

射击噪声

一类由离散事件组成并建模为泊松过程的随机源称为射击噪声。它们有长期的认证先例；AIS 31 文档本身提供了基于噪声二极管的参考设计。射击噪声源通常比环形振荡器更能抵抗温度变化。其中一些生成器也可以完全使用标准单元实现（Rambus / Inside Secure 通用被描述为射击噪声生成器）。

其他类型的噪声

可能可以认证更奇特的噪声源和设计, 尽管它们的随机模型需要同样理解良好, 并且它们的 CPU 接口必须是安全的。

连续健康测试

健康监测需要维护一些与噪声源相关的状态信息。测试应设计为特定数量的样本保证状态刷新(无挂起状态)。我们建议刷新大小 $W \leq 1024$ 以匹配 NIST SP 800-90B 所需的测试(见第 4.4 节)。状态在系统重置时也会完全清零。

可以用最少的电路构建两个强制性测试。不需要完整的直方图, 只需简单的计数器寄存器: 重复计数、窗口计数和样本计数。每次输出样本值变化时重置重复计数; 如果计数达到某个截止限制, 则发出噪声警报(BIST)或失败(DEAD)信号。窗口计数器用于保存每第 W 个输出(通常 W 在 $\{512, 1024\}$ 中)。在接下来的窗口中计算此参考样本的频率; 标准中定义了截止值。我们看到强制性测试的结构是这样的, 如果实现良好, 没有信息会超过 W 样本的限制。

第 4.5 节明确允许额外的开发者定义测试, 并且在 FIPS 140-1 的早期版本中定义了更多测试, 然后被划掉。额外测试的选择取决于物理源的性质和实现。

特别是如果在硬件中使用非加密调节器, 可能由驱动程序软件实现在线测试。它们也可以在硬件中实现。对于某些安全配置文件, AIS 31 要求其容差设置为在正常使用下每年至少 10^{-6} 的警报概率。这样的要求在现代应用中是有问题的, 因为它们的概率对于关键系统来说太高了。

在无操作员的自主系统中, 通常没有什么可以或应该对非致命警报条件采取措施。然而, AIS 31 允许 DRBG 组件在其熵源失败的情况下继续运行, 因此我们建议重新进入临时 BIST 状态以发出非致命统计错误信号, 如果需要这种(不可操作的)信号。驱动程序和应用程序可以适当对此做出反应(或简单地记录它), 但不会直接影响 TRNG 的可用性。永久错误条件应导致 DEAD 状态。

非加密调节器

如中所述, 物理随机源通常需要一个称为调节的后处理步骤, 以满足中概述的质量要求。

此接口采用的方法是允许进行非加密和加密过滤的组合。第一阶段(硬件)仅需要能够将熵舒适地提炼到必要水平之上。

- * 可以从噪声源中获取一组比特并将它们进行异或以生成偏差较小(且更独立)的比特。

然而, 这种异或可能引入伪随机性, 使输出难以分析。

- * von Neumann 提取器 查看连续的比特对, 拒绝 00 和 11, 并分别对 01 和 10 输出 0 或 1。它将比特数量减少到原来的不到 25%, 但输出是可证明的无偏(假设独立性)。

- * Blum 提取器可用于行为类似于 N 状态马尔可夫链的源。如果其假设成立, 它还会去除依赖性, 创建一个独立同分布(IID)源。

- * 其他线性和非线性校正器, 如 Dichtl 和 Lacharme 讨论的那些。

请注意, 硬件也可以在熵源中实现完整的加密调节器, 即使软件驱动程序仍然需要加密调节器。理由:

非加密提取器的主要优点在于其能效、相对简单性和适合数学分析。如果设计良好, 可以与噪声源本身的随机模型结合进行评估。它们不需要计算难度假设。

加密调节器

为了安全使用, ISA 边界的软件端始终需要加密调节器。如果需要, 它们也可以在硬件端实现。在任何情况下, entropy ES16 输出在用作密钥材料或被视为完全熵之前, 必须始终压缩 2:1(或更多)。

加密调节器的示例包括 Linux 操作系统的随机池、安全哈希函数(SHA-2/3、SHAKE)和附录 F 中的 AES / CBC-MAC 构造, SP 800-90B。

在某些构造中, 如基于 Linux RNG 和 SHA-3/SHAKE 的生成器, 加密调节和输出(DRBG)生成由同一组件提供。

理由:

对于许多低功耗目标构造，Intel 和 AMD 使用的硬件 AES CBC-MAC 调节器类型将过于复杂和耗能，无法仅用于服务 seed CSR。另一方面，如果需要高质量的随机输出，较简单的非加密调节器可能会对输入熵过于浪费——（ARM TrustZone TRBG 在 200 MHz 下仅输出 10Kbit/sec）。因此，在硬件和软件生成之间做出了节省资源的折衷。

最终随机：DRBGs

所有到达最终用户和应用程序的随机比特必须来自加密 DRBG。这些通常由软件中的驱动程序组件实现。如果可用，应该使用 RISC-V AES 和 SHA 指令集扩展，因为它们提供了额外的安全功能，如时间攻击抵抗。

目前推荐的 DRBG 定义在 NIST SP 800-90A（修订版 1）中：CTRDRBG、HashDRBG 和 HMACDRBG。认证通常需要对对称组件和 DRBG 整体进行已知答案测试（KATs）。这些在软件中实现比在硬件中要容易得多。除了直接可认证的 SP 800-90A DRBG，还可以使用基于 ChaCha20 的 Linux 风格随机池构造，或基于 SHAKE256 的适当构造。

这些只是建议；程序员可以调整 CPU 熵源的使用以满足未来的要求。

量子与经典随机

NCSC 认为，经典 RNG 将继续满足我们对政府和军事应用的需求，在可预见的未来。量子随机数生成器（QRNG）是一种 TRNG，其随机源可以明确地识别为特定的量子现象，如量子态叠加、量子态纠缠、海森堡不确定性、量子隧穿、自发发射或放射性衰变。

直接量子熵在理论上是最好的熵类型。基于电子噪声的典型 TRNG 也主要基于量子现象，同样不可预测——区别在于，对于经典 TRNG，量子与经典物理学的相对量难以量化。

QRNG 的设计方式允许对量子起源熵的量进行建模和估计。这一区别在 QKD（量子密钥分发）安全机制使用的安全模型中很重要，可以直接使用量子力学效应保护物理层（如光纤电缆）免受拦截。这种安全模型意味着许多可用的 QRNG 设备不使用加密调节，可能无法满足加密统计要求。许多实现者可能将它们视为熵源。

关于 QRNG 实现安全性的研究相对较少，但许多 QRNG 设计可以说比经典生成器（如环形振荡器）更容易泄漏，因为它们往往使用外部组件和混合材料。例如，光子探测器信号的放大可能在功率分析中可观察到，而经典基于噪声的源设计为抵抗这种情况。

后量子加密

PQC 公钥加密标准不需要量子起源随机性，只需要足够安全的密钥材料。请记住，加密旨在保护数据本身的机密性和完整性，不对物理通信通道（如 QKD）提出任何要求。

经典的高质量 TRNG 非常适合生成 PQC 协议的秘密密钥，这些协议对量子计算机来说难以破解，但可以在经典计算机上实现。在加密中，重要的是秘密密钥具有足够的真实随机性（熵），并且它们被安全地生成和存储。

当然，必须避免基于量子计算机容易解决的问题的 DRBG，例如 Blum-Blum-Shub 生成器中的因子分解。大多数对称算法不受影响，因为最好的量子攻击仍然是指数级的密钥大小。

例如，原始 Intel RNG，其输出生成基于 AES-128，可以使用 Grover 算法进行攻击，约为平方根努力。虽然即使 64 位量子安全性也极难破解，但许多应用程序指定了更高的安全要求。将 AES-128 定义为类别 1 等效后量子安全性，而 AES-256 是类别 5（最高）。我们通过暴露对熵源的直接访问来避免这种可能的未来问题，熵源可以仅从信息理论假设中获得其安全性。

32.8.6 建议的 GetNoise 测试接口

合规性测试、熵源的特性描述和配置需要访问原始、未经调节的噪声样本。此概念测试接口在 NIST SP 800-90B 第 2.3.2 节中命名为 GetNoise。

由于这种类型的接口对于安全测试是必要的，同时也构成了密码密钥生成过程的潜在后门，我们定义了一种安全行为，使合规实现可以在测试期间暂时禁用熵源 seed CSR 接口。

为了使共享的 RISC-V 自我认证脚本（和驱动程序）能够以安全的方式适应测试接口，我们建议将其实现为自定义的、仅限 M 模式的 CSR，此处称为 `mnoise`。

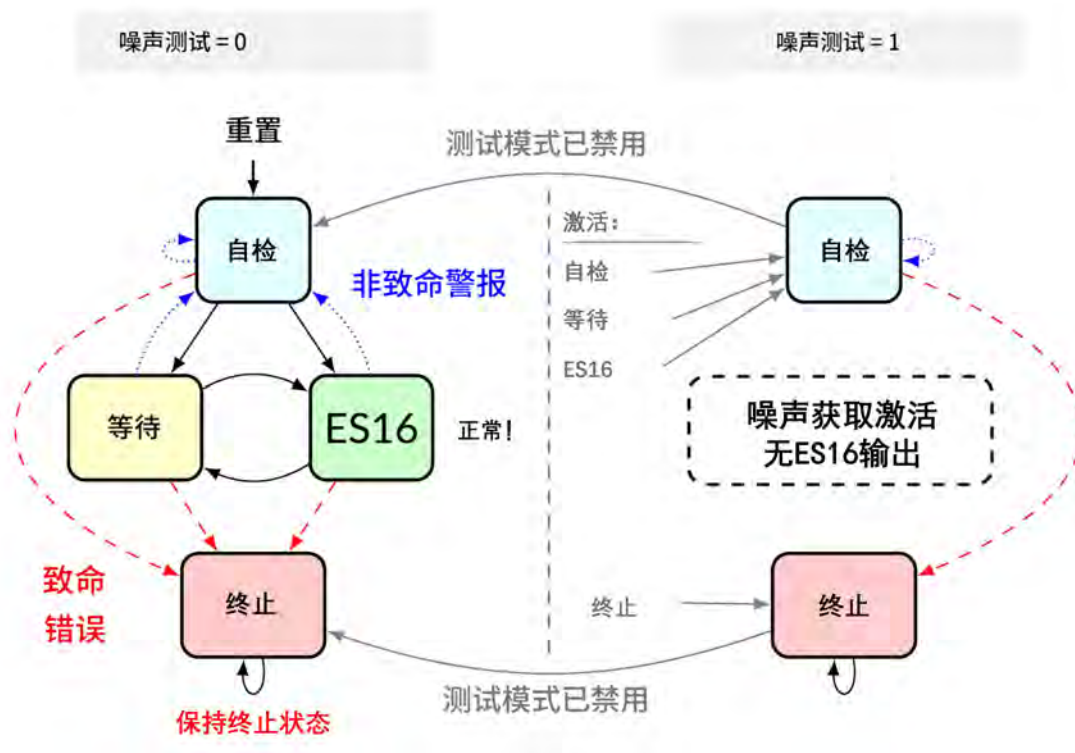


图32.2 测试模式下无法读取熵源

此非规范接口不打算用作随机性来源或其他生产用途。我们为此接口定义了一个单个位的语义，其名称为 `NOISETEST`，如果实现，将影响 `seed` 的行为。

当 `mnoise` 中的 `NOISETEST = 1` 时，`seed` CSR 不得通过 `ES16` 返回任何内容；除非源是 `DEAD`，否则它应该处于 `BIST` 状态。当 `NOISETEST` 再次禁用时，熵源应通过适当的清零和自我测试机制从 `BIST` 返回。其他输入和输出位的行为主要由供应商决定（因为它们取决于物理熵源的技术细节），自定义 `mnoise` CSR 的地址也是如此。只有在 `mvendorid`、`marchid` 和 `mimpid` CSR 标识符的上下文中才能解释 CSR 的其他内容和行为。

当未实现时（例如，在虚拟机中），`mnoise` 可以永久读取零（`0x00000000`）并忽略写入。当可用但 `NOISETEST = 0` 时，`mnoise` 可以返回一个非零常量（例如 `0x00000001`），但没有噪声样本。

在 `NOISETEST` 模式下，`WAIT` 和 `ES16` 状态不可达，并且没有输出熵。建议不要实现直接影响 `seed` CSR 接口的 `ES16` 熵输出的测试接口。此类供应商测试接口已在攻击中被利用。例如，没有足够熵的 `ECDSA` 签名过程不仅会创建不安全的签名，还可能泄露秘密签名密钥，攻击者可以使用该密钥进行身份验证伪造。因此，即使是 `entropy` 安全性的临时失效也可能具有严重的安全影响。

32.9 补充材料

虽然本文档包含 RISC-V 加密扩展的规范，但还开发了许多补充材料和示例代码。所有与 RISC-V 加密扩展相关的材料都位于 Github 存储库，地址为 <https://github.com/riscv/riscv-crypto>

32.10 支持的 Sail 代码

本节包含规范中指令描述引用的支持 Sail 代码。推荐阅读 Sail 手册（link:<https://github.com/REMS-project/sail/blob/sail2/manual.pdf>）以便最好地理解支持代

码。

```
/* 用于执行 GF 乘法的辅助函数 */
val xt2 : bits(8) -> bits(8)
function xt2(x) = {
  (x 1) ^ (if bit_to_bool(x[7]) then 0x1b else 0x00)
}

val xt3 : bits(8) -> bits(8)
function xt3(x) = x ^ xt2(x)

/* 将 8 位字段元素乘以 4 位值, 用于 AES MixCols 步骤 */
val gfmul : (bits(8), bits(4)) -> bits(8)
function gfmul( x, y) = {
  (if bit_to_bool(y[0]) then          x      else 0x00) ^
  (if bit_to_bool(y[1]) then xt2(          x      else 0x00) ^
  (if bit_to_bool(y[2]) then xt2(xt2(          x      else 0x00) ^
  (if bit_to_bool(y[3]) then xt2(xt2(xt2(x))) else 0x00)
}

/* 8 位到 32 位的部分 AES Mix Column - 正向 */
val aes_mixcolumn_byte_fwd : bits(8) -> bits(32)
function aes_mixcolumn_byte_fwd(so) = {
  gfmul(so, 0x3) @ so @ so @ gfmul(so, 0x2)
}

/* 8 位到 32 位的部分 AES Mix Column - 逆向 */
val aes_mixcolumn_byte_inv : bits(8) -> bits(32)
function aes_mixcolumn_byte_inv(so) = {
  gfmul(so, 0xb) @ gfmul(so, 0xd) @ gfmul(so, 0x9) @ gfmul(so, 0xe)
}

/* 32 位到 32 位的 AES 正向 MixColumn */
val aes_mixcolumn_fwd : bits(32) -> bits(32)
function aes_mixcolumn_fwd(x) = {
  let s0 : bits (8) = x[ 7.. 0];
  let s1 : bits (8) = x[15.. 8];
  let s2 : bits (8) = x[23..16];
  let s3 : bits (8) = x[31..24];
  let b0 : bits (8) = xt2(s0) ^ xt3(s1) ^      (s2) ^      (s3);
  let b1 : bits (8) =      (s0) ^ xt2(s1) ^ xt3(s2) ^      (s3);
  let b2 : bits (8) =      (s0) ^      (s1) ^ xt2(s2) ^ xt3(s3);
  let b3 : bits (8) = xt3(s0) ^      (s1) ^      (s2) ^ xt2(s3);
  b3 @ b2 @ b1 @ b0 /* 返回值 */
}
```



```

}

/* 32 位到 32 位的 AES 逆向 MixColumn */
val aes_mixcolumn_inv : bits(32) -> bits(32)
function aes_mixcolumn_inv(x) = {
  let s0 : bits (8) = x[ 7.. 0];
  let s1 : bits (8) = x[15.. 8];
  let s2 : bits (8) = x[23..16];
  let s3 : bits (8) = x[31..24];
  let b0 : bits (8) = gfmul(s0, 0xE) ^ gfmul(s1, 0xB) ^ gfmul(s2, 0xD) ^ gfmul(s3, 0x9);
  let b1 : bits (8) = gfmul(s0, 0x9) ^ gfmul(s1, 0xE) ^ gfmul(s2, 0xB) ^ gfmul(s3, 0xD);
  let b2 : bits (8) = gfmul(s0, 0xD) ^ gfmul(s1, 0x9) ^ gfmul(s2, 0xE) ^ gfmul(s3, 0xB);
  let b3 : bits (8) = gfmul(s0, 0xB) ^ gfmul(s1, 0xD) ^ gfmul(s2, 0x9) ^ gfmul(s3, 0xE);
  b3 @ b2 @ b1 @ b0 /* 返回值 */
}

/* 将轮数转换为 AES 的轮常数。请注意，AES64KS1I 指令定义为 r 参数始终在 0x0..0xA 范围内。
范围外的 rnum 值不会解码为 AES64KS1I 指令。0xA 情况专门用于 AES-256 密钥调度，此函数在该情
况下不会被调用。 */
val aes_decode_rcon : bits(4) -> bits(32)
function aes_decode_rcon(r) = {
  assert(r <_u 0xA);
  match r {
    0x0 => 0x00000001,
    0x1 => 0x00000002,
    0x2 => 0x00000004,
    0x3 => 0x00000008,
    0x4 => 0x00000010,
    0x5 => 0x00000020,
    0x6 => 0x00000040,
    0x7 => 0x00000080,
    0x8 => 0x0000001b,
    0x9 => 0x00000036,
    _ => internal_error(__FILE__, __LINE__, Unexpected AES r) /* 不可达 需要静音 Sail 警
告 */
  }
}

/* SM4 SBox - 只有一个 SBox 用于正向和逆向 */
let sm4_sbox_table : vector(256, bits(8)) = [
  0xD6, 0x90, 0xE9, 0xFE, 0xCC, 0xE1, 0x3D, 0xB7, 0x16, 0xB6, 0x14, 0xC2, 0x28,
  0xFB, 0x2C, 0x05, 0x2B, 0x67, 0x9A, 0x76, 0x2A, 0xBE, 0x04, 0xC3, 0xAA, 0x44,
  0x13, 0x26, 0x49, 0x86, 0x06, 0x99, 0x9C, 0x42, 0x50, 0xF4, 0x91, 0xEF, 0x98,
  0x7A, 0x33, 0x54, 0x0B, 0x43, 0xED, 0xCF, 0xAC, 0x62, 0xE4, 0xB3, 0x1C, 0xA9,

```

```

0xC9, 0x08, 0xE8, 0x95, 0x80, 0xDF, 0x94, 0xFA, 0x75, 0x8F, 0x3F, 0xA6, 0x47,
0x07, 0xA7, 0xFC, 0xF3, 0x73, 0x17, 0xBA, 0x83, 0x59, 0x3C, 0x19, 0xE6, 0x85,
0x4F, 0xA8, 0x68, 0x6B, 0x81, 0xB2, 0x71, 0x64, 0xDA, 0x8B, 0xF8, 0xEB, 0x0F,
0x4B, 0x70, 0x56, 0x9D, 0x35, 0x1E, 0x24, 0x0E, 0x5E, 0x63, 0x58, 0xD1, 0xA2,
0x25, 0x22, 0x7C, 0x3B, 0x01, 0x21, 0x78, 0x87, 0xD4, 0x00, 0x46, 0x57, 0x9F,
0xD3, 0x27, 0x52, 0x4C, 0x36, 0x02, 0xE7, 0xA0, 0xC4, 0xC8, 0x9E, 0xEA, 0xBF,
0x8A, 0xD2, 0x40, 0xC7, 0x38, 0xB5, 0xA3, 0xF7, 0xF2, 0xCE, 0xF9, 0x61, 0x15,
0xA1, 0xE0, 0xAE, 0x5D, 0xA4, 0x9B, 0x34, 0x1A, 0x55, 0xAD, 0x93, 0x32, 0x30,
0xF5, 0x8C, 0xB1, 0xE3, 0x1D, 0xF6, 0xE2, 0x2E, 0x82, 0x66, 0xCA, 0x60, 0xC0,
0x29, 0x23, 0xAB, 0x0D, 0x53, 0x4E, 0x6F, 0xD5, 0xDB, 0x37, 0x45, 0xDE, 0xFD,
0x8E, 0x2F, 0x03, 0xFF, 0x6A, 0x72, 0x6D, 0x6C, 0x5B, 0x51, 0x8D, 0x1B, 0xAF,
0x92, 0xBB, 0xDD, 0xBC, 0x7F, 0x11, 0xD9, 0x5C, 0x41, 0x1F, 0x10, 0x5A, 0xD8,
0x0A, 0xC1, 0x31, 0x88, 0xA5, 0xCD, 0x7B, 0xBD, 0x2D, 0x74, 0xD0, 0x12, 0xB8,
0xE5, 0xB4, 0xB0, 0x89, 0x69, 0x97, 0x4A, 0x0C, 0x96, 0x77, 0x7E, 0x65, 0xB9,
0xF1, 0x09, 0xC5, 0x6E, 0xC6, 0x84, 0x18, 0xF0, 0x7D, 0xEC, 0x3A, 0xDC, 0x4D,
0x20, 0x79, 0xEE, 0x5F, 0x3E, 0xD7, 0xCB, 0x39, 0x48
]

```

```

let aes_sbox_fwd_table : vector(256, bits(8)) = [
0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe,
0xd7, 0xab, 0x76, 0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4,
0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0, 0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7,
0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15, 0x04, 0xc7, 0x23, 0xc3,
0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75, 0x09,
0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3,
0x2f, 0x84, 0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe,
0x39, 0x4a, 0x4c, 0x58, 0xcf, 0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85,
0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8, 0x51, 0xa3, 0x40, 0x8f, 0x92,
0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2, 0xcd, 0x0c,
0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19,
0x73, 0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14,
0xde, 0x5e, 0x0b, 0xdb, 0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2,
0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79, 0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5,
0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08, 0xba, 0x78, 0x25,
0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86,
0xc1, 0x1d, 0x9e, 0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e,
0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf, 0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42,
0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16
]

```

```

let aes_sbox_inv_table : vector(256, bits(8)) = [
0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40, 0xa3, 0x9e, 0x81,
0xf3, 0xd7, 0xfb, 0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e,

```

```

0x43, 0x44, 0xc4, 0xde, 0xe9, 0xcb, 0x54, 0x7b, 0x94, 0x32, 0xa6, 0xc2, 0x23,
0x3d, 0xee, 0x4c, 0x95, 0x0b, 0x42, 0xfa, 0xc3, 0x4e, 0x08, 0x2e, 0xa1, 0x66,
0x28, 0xd9, 0x24, 0xb2, 0x76, 0x5b, 0xa2, 0x49, 0x6d, 0x8b, 0xd1, 0x25, 0x72,
0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xd4, 0xa4, 0x5c, 0xcc, 0x5d, 0x65,
0xb6, 0x92, 0x6c, 0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda, 0x5e, 0x15, 0x46,
0x57, 0xa7, 0x8d, 0x9d, 0x84, 0x90, 0xd8, 0xab, 0x00, 0x8c, 0xbc, 0xd3, 0x0a,
0xf7, 0xe4, 0x58, 0x05, 0xb8, 0xb3, 0x45, 0x06, 0xd0, 0x2c, 0x1e, 0x8f, 0xca,
0x3f, 0x0f, 0x02, 0xc1, 0xaf, 0xbd, 0x03, 0x01, 0x13, 0x8a, 0x6b, 0x3a, 0x91,
0x11, 0x41, 0x4f, 0x67, 0xdc, 0xea, 0x97, 0xf2, 0xcf, 0xce, 0xf0, 0xb4, 0xe6,
0x73, 0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2, 0xf9, 0x37, 0xe8,
0x1c, 0x75, 0xdf, 0x6e, 0x47, 0xf1, 0x1a, 0x71, 0x1d, 0x29, 0xc5, 0x89, 0x6f,
0xb7, 0x62, 0x0e, 0xaa, 0x18, 0xbe, 0x1b, 0xfc, 0x56, 0x3e, 0x4b, 0xc6, 0xd2,
0x79, 0x20, 0x9a, 0xdb, 0xc0, 0xfe, 0x78, 0xcd, 0x5a, 0xf4, 0x1f, 0xdd, 0xa8,
0x33, 0x88, 0x07, 0xc7, 0x31, 0xb1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xec, 0x5f,
0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0x0d, 0x2d, 0xe5, 0x7a, 0x9f, 0x93,
0xc9, 0x9c, 0xef, 0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb,
0xbb, 0x3c, 0x83, 0x53, 0x99, 0x61, 0x17, 0x2b, 0x04, 0x7e, 0xba, 0x77, 0xd6,
0x26, 0xe1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0c, 0x7d
]

```

/* 查找函数 - 获取索引和表，并检索该表的第 x 个元素。请注意，Sail 向量文字从索引 255 开始，向下到 0。 */

```

val sbbox_lookup : (bits(8), vector(256, bits(8))) -> bits(8)
function sbbox_lookup(x, table) = {
    table[255 - unsigned(x)]
}

```

/* 执行正向 AES SBox 操作的简便函数。 */

```

val aes_sbbox_fwd : bits(8) -> bits(8)
function aes_sbbox_fwd(x) = sbbox_lookup(x, aes_sbbox_fwd_table)

```

/* 执行逆向 AES SBox 操作的简便函数。 */

```

val aes_sbbox_inv : bits(8) -> bits(8)
function aes_sbbox_inv(x) = sbbox_lookup(x, aes_sbbox_inv_table)

```

/* AES SubWord 函数用于密钥扩展

* - 将正向 sbbox 应用于输入字中的每个字节。

*/

```

val aes_subword_fwd : bits(32) -> bits(32)
function aes_subword_fwd(x) = {
    aes_sbbox_fwd(x[31..24]) @
    aes_sbbox_fwd(x[23..16]) @
    aes_sbbox_fwd(x[15.. 8]) @
    aes_sbbox_fwd(x[ 7.. 0])
}

```

```

}

/* AES 逆 SubWord 函数。
 * - 将逆向 sbbox 应用于输入字中的每个字节。
 */
val aes_subword_inv : bits(32) -> bits(32)
function aes_subword_inv(x) = {
  aes_sbox_inv(x[31..24]) @
  aes_sbox_inv(x[23..16]) @
  aes_sbox_inv(x[15.. 8]) @
  aes_sbox_inv(x[ 7.. 0])
}

/* 执行 SM4 SBox 操作的简便函数。 */
val sm4_sbox : bits(8) -> bits(8)
function sm4_sbox(x) = sbbox_lookup(x, sm4_sbox_table)

val aes_get_column : (bits(128), nat) -> bits(32)
function aes_get_column(state,c) = (state (to_bits(7, 32 * c)))[31..0]

/* 64 位到 64 位函数，将 AES 正向 sbbox 应用于 64 位字中的每个字节。 */
val aes_apply_fwd_sbox_to_each_byte : bits(64) -> bits(64)
function aes_apply_fwd_sbox_to_each_byte(x) = {
  aes_sbox_fwd(x[63..56]) @
  aes_sbox_fwd(x[55..48]) @
  aes_sbox_fwd(x[47..40]) @
  aes_sbox_fwd(x[39..32]) @
  aes_sbox_fwd(x[31..24]) @
  aes_sbox_fwd(x[23..16]) @
  aes_sbox_fwd(x[15.. 8]) @
  aes_sbox_fwd(x[ 7.. 0])
}

/* 64 位到 64 位函数，将 AES 逆向 sbbox 应用于 64 位字中的每个字节。 */
val aes_apply_inv_sbox_to_each_byte : bits(64) -> bits(64)
function aes_apply_inv_sbox_to_each_byte(x) = {
  aes_sbox_inv(x[63..56]) @
  aes_sbox_inv(x[55..48]) @
  aes_sbox_inv(x[47..40]) @
  aes_sbox_inv(x[39..32]) @
  aes_sbox_inv(x[31..24]) @
  aes_sbox_inv(x[23..16]) @
  aes_sbox_inv(x[15.. 8]) @
  aes_sbox_inv(x[ 7.. 0])
}

```

```

}

/*
 * AES 全轮转换函数。
 */

val getbyte : (bits(64), int) -> bits(8)
function getbyte(x, i) = (x  to_bits(6, i * 8))[7..0]

val aes_rv64_shiftrows_fwd : (bits(64), bits(64)) -> bits(64)
function aes_rv64_shiftrows_fwd(rs2, rs1) = {
  getbyte(rs1, 3) @
  getbyte(rs2, 6) @
  getbyte(rs2, 1) @
  getbyte(rs1, 4) @
  getbyte(rs2, 7) @
  getbyte(rs2, 2) @
  getbyte(rs1, 5) @
  getbyte(rs1, 0)
}

val aes_rv64_shiftrows_inv : (bits(64), bits(64)) -> bits(64)
function aes_rv64_shiftrows_inv(rs2, rs1) = {
  getbyte(rs2, 3) @
  getbyte(rs2, 6) @
  getbyte(rs1, 1) @
  getbyte(rs1, 4) @
  getbyte(rs1, 7) @
  getbyte(rs2, 2) @
  getbyte(rs2, 5) @
  getbyte(rs1, 0)
}

/* 128 位到 128 位的 AES 正向 ShiftRows 转换的实现。
 * 状态的字节 0 是输入列 0, 位 7..0。
 * 状态的字节 5 是输入列 1, 位 15..8。
 */
val aes_shift_rows_fwd : bits(128) -> bits(128)
function aes_shift_rows_fwd(x) = {
  let ic3 : bits(32) = aes_get_column(x, 3);
  let ic2 : bits(32) = aes_get_column(x, 2);
  let ic1 : bits(32) = aes_get_column(x, 1);
  let ic0 : bits(32) = aes_get_column(x, 0);
  let oc0 : bits(32) = ic0[31..24] @ ic1[23..16] @ ic2[15.. 8] @ ic3[ 7.. 0];

```

```

    let oc1 : bits(32) = ic1[31..24] @ ic2[23..16] @ ic3[15.. 8] @ ic0[ 7.. 0];
    let oc2 : bits(32) = ic2[31..24] @ ic3[23..16] @ ic0[15.. 8] @ ic1[ 7.. 0];
    let oc3 : bits(32) = ic3[31..24] @ ic0[23..16] @ ic1[15.. 8] @ ic2[ 7.. 0];
    (oc3 @ oc2 @ oc1 @ oc0) /* 返回值 */
}

```

/* 128 位到 128 位的 AES 逆向 ShiftRows 转换的实现。

* 状态的字节 0 是输入列 0, 位 7..0。

* 状态的字节 5 是输入列 1, 位 15..8。

*/

```

val aes_shift_rows_inv : bits(128) -> bits(128)

```

```

function aes_shift_rows_inv(x) = {
    let ic3 : bits(32) = aes_get_column(x, 3); /* 输入列 3 */
    let ic2 : bits(32) = aes_get_column(x, 2);
    let ic1 : bits(32) = aes_get_column(x, 1);
    let ic0 : bits(32) = aes_get_column(x, 0);
    let oc0 : bits(32) = ic0[31..24] @ ic3[23..16] @ ic2[15.. 8] @ ic1[ 7.. 0];
    let oc1 : bits(32) = ic1[31..24] @ ic0[23..16] @ ic3[15.. 8] @ ic2[ 7.. 0];
    let oc2 : bits(32) = ic2[31..24] @ ic1[23..16] @ ic0[15.. 8] @ ic3[ 7.. 0];
    let oc3 : bits(32) = ic3[31..24] @ ic2[23..16] @ ic1[15.. 8] @ ic0[ 7.. 0];
    (oc3 @ oc2 @ oc1 @ oc0) /* 返回值 */
}

```

/* 将 AES 的正向子字节步骤应用于其状态的 128 位向量表示。 */

```

val aes_subbytes_fwd : bits(128) -> bits(128)

```

```

function aes_subbytes_fwd(x) = {
    let oc0 : bits(32) = aes_subword_fwd(aes_get_column(x, 0));
    let oc1 : bits(32) = aes_subword_fwd(aes_get_column(x, 1));
    let oc2 : bits(32) = aes_subword_fwd(aes_get_column(x, 2));
    let oc3 : bits(32) = aes_subword_fwd(aes_get_column(x, 3));
    (oc3 @ oc2 @ oc1 @ oc0) /* 返回值 */
}

```

/* 将 AES 的逆向子字节步骤应用于其状态的 128 位向量表示。 */

```

val aes_subbytes_inv : bits(128) -> bits(128)

```

```

function aes_subbytes_inv(x) = {
    let oc0 : bits(32) = aes_subword_inv(aes_get_column(x, 0));
    let oc1 : bits(32) = aes_subword_inv(aes_get_column(x, 1));
    let oc2 : bits(32) = aes_subword_inv(aes_get_column(x, 2));
    let oc3 : bits(32) = aes_subword_inv(aes_get_column(x, 3));
    (oc3 @ oc2 @ oc1 @ oc0) /* 返回值 */
}

```

/* 将 AES 的正向 MixColumns 步骤应用于其状态的 128 位向量表示。 */

```

val aes_mixcolumns_fwd : bits(128) -> bits(128)
function aes_mixcolumns_fwd(x) = {
  let oc0 : bits(32) = aes_mixcolumn_fwd(aes_get_column(x, 0));
  let oc1 : bits(32) = aes_mixcolumn_fwd(aes_get_column(x, 1));
  let oc2 : bits(32) = aes_mixcolumn_fwd(aes_get_column(x, 2));
  let oc3 : bits(32) = aes_mixcolumn_fwd(aes_get_column(x, 3));
  (oc3 @ oc2 @ oc1 @ oc0) /* 返回值 */
}

/* 将 AES 的逆向 MixColumns 步骤应用于其状态的 128 位向量表示。 */
val aes_mixcolumns_inv : bits(128) -> bits(128)
function aes_mixcolumns_inv(x) = {
  let oc0 : bits(32) = aes_mixcolumn_inv(aes_get_column(x, 0));
  let oc1 : bits(32) = aes_mixcolumn_inv(aes_get_column(x, 1));
  let oc2 : bits(32) = aes_mixcolumn_inv(aes_get_column(x, 2));
  let oc3 : bits(32) = aes_mixcolumn_inv(aes_get_column(x, 3));
  (oc3 @ oc2 @ oc1 @ oc0) /* 返回值 */
}

```

33 加密扩展：向量指令，V1.0.0

本文件介绍了 RISC-V 指令集架构的向量加密扩展。此扩展已正式批准，不允许进行任何修改。如需更改，可考虑将其纳入后续扩展的讨论范畴。需要注意的是，所有已批准的扩展都不会进行修订。

33.1 介绍

本文件介绍了 RISC-V 向量加密扩展的提案。所有提出的指令均基于向量寄存器，目的是提供高效的性能，主要针对大型应用和服务端级核。此外，还提供了配套章节第一卷：标量和熵源指令，该章节描述了面向不实现向量扩展的小型核的加密指令提案。

33.1.1 目标受众

加密是一个高度专业化的领域，需要不同背景的人士协同合作，才能确保其安全性和高效性。我们力求编写出易于理解的规范，但我们也清楚，动机以及算法或其他规范、标准的引用，对于非专业读者而言可能较为陌生。

预计来自不同背景的各类读者将阅读和实施本规范。我们力图概括这些背景，并简要说明我们希望读者理解的内容，以及这些内容与规范的关系。我们希望这能帮助读者明确哪些规范部分对他们尤为相关，哪些可以（在确保安全性的前提下）忽略或推荐给其他同事。

密码学家和密码软件开发人员。密码学家和密码软件开发人员是我们期望使用本规范中指令编写代码的目标人群。他们理解我们包含这些指令的动机，并熟悉我们引用的大多数算法和外部标准。

计算机架构师。我们了解架构师不一定具有密码学背景，但仍希望架构师能够检查这些指令并发现实现问题，理解指令在上下文中的使用方式，并就如何最好地满足密码学家功能需求提供建议。

数字设计工程师和微架构师。这些人将负责在核内部实现规范。同样，我们了解他们不一定具有密码学专业背景，但希望他们能够解释规范并预见任何硬件实现问题，例如高频设计考虑因素，或存在延迟/面积权衡等。具体来说，他们应该了解有关在硬件中高效实现 AES 和 SM4 SBoxes 的文献。

验证工程师。这些人负责确保在硬件中正确实现扩展。我们了解他们不一定具有密码学背景，但希

望他们从规范中发现有趣的测试用例。理解其实际使用情况将有助于此。

33.1.2 Sail 规范

我们原本打算在本规范中加入实际的 Sail 代码。然而，向量加密 Sail 模型依赖于向量 Sail 模型作为构建基础。由于该向量加密扩展规范在 RISC-V 向量 Sail 模型正式批准之前已经完成，因此我们无法在指令描述中提供任何 Sail 代码。因此，我们使用了类似 Sail 的伪代码。尽管我们努力遵循 Sail 语法，但为了简洁起见，我们在某些地方做了一些调整，我们相信读者能清晰地理解我们的意图。



当变量连接时，显示的顺序是它们在向量寄存器中从左到右的顺序。例如，指定为 {a, b, e, f} 的元素组将在向量寄存器中显示为 a 具有组中最高元素索引，f 具有组中最低索引。

为了简洁起见，我们的伪代码不包括掩码或尾元素的处理。我们遵循 *RISC-V V 向量扩展* 规范中描述的不受干扰和不可知策略。此外，代码没有明确处理重叠和 SEW 约束；这些在文本中明确说明。在许多情况下，伪代码包括调用支持函数，这些函数过于冗长，无法直接包含于规范中。

33.1.3 政策

在制定此提案时，我们遵循了以下方针：

- * 在选择以下两者时：
 - 1) 支持算法的多样化实现策略
 - 2) 支持单一实现风格，且该风格更高效或成本更低；

向量加密扩展将选择更具约束性但更高效的选项。这与 RISC-V 规范的其他部分的惯常做法相符，其中通常会推荐（但不强制）执行特定任务的指令序列作为示例，帮助硬件和软件实现者优化单一用例。

* 扩展将专注于支持现有的标准化加密构造，而不支持提议中的标准或仅存在于学术界的加密构造。与 RISC-V 向量加密扩展标准化同时或之后确定的加密标准，将通过未来的 RISC-V 向量加密标准扩展来处理。

* 历史上曾有关于新操作如何启用加密算法基础的讨论。标准不会试图预测哪些新低级操作可能对未来加密构造有用。

* 关于侧信道对策：在适用的情况下，提议的指令应消除任何时间侧信道的潜在风险。所有指令应实现数据独立的执行时间，即其执行延迟不应因输入数据值的变化而有所不同。

33.1.4 元素组

许多向量加密指令操作的操作数比元素宽（目前限制为 64 位宽）。通常，这些操作数为 128 位和 256 位宽。在许多情况下，这些操作数由较小的操作数组成（例如，每个 SHA-2 操作数由 4 个字组成）。然而，在其他情况下，这些操作数是单个值（例如，在 AES 轮指令中，每个操作数是 128 位块或轮密钥）。

每个操作元素组的向量加密指令明确指定它们的三个定义参数：EGW、EGS 和 EEW。

表 33.1 每条作用于元素组的向量加密指令均需显式指定其三个定义参数：EGW、EGS 和 EEW

组别	扩展	EGW	EEW	EGS
AES	Zvkned	128	32	4
SHA256	Zvknh	128	32	4

SHA512	Zvknhb	256	64	4
GCM	Zvkg	128	32	4
SM4	Zvksed	128	32	4
SM3	Zvksh	256	32	8



-元素组宽度（EGW）- 元素组中的总位数

-有效元素宽度（EEW）- 每个元素的位数

-元素组大小（EGS）- 元素组中的元素数量

对于本规范中的所有向量加密指令，EEW=SEW。



为每个加密指令选择的 SEW 是为了匹配实现目标算法时通常需要的内容。

- *向量元素组* 是一个或多个元素组的向量。
- *标量元素组* 是单个元素组。

在实现 $VLEN < EGW$ 的情况下，可以通过使用 $LMUL > 1$ 在寄存器之间形成元素组。



由于应用处理器的 *向量扩展* 要求 $VLEN$ 至少为 128，因此，最大元素组的构建需要 $LMUL=2$ 。

具有较小 $VLEN$ 的实现（如嵌入式设计）需要更大的 $LMUL$ 形成所需的元素组。需要特别注意的是，增大 $LMUL$ 会减少可用寄存器组数量，这可能导致编写高效代码来实现预期加密算法变得更加困难，甚至完全无法实现。

例如， $VLEN=32$ 的实现需要设置 $LMUL=8$ 以形成 256 位元素组来支持 SM3。这意味着系统仅有 4 个寄存器组，其中 3 个会被单个 SM3 消息扩展指令消耗。

与所有向量指令一样，处理的元素数量由向量长度 $v1$ 指定。操作的元素组数量为 $v1/EGS$ 。同样，起始元素组为 $vstart/EGS$ 。

33.1.5 指令约束

以下是特定向量加密指令的各种约束的快速参考。

$v1$ 和 $vstart$ 约束：

由于 $v1$ 和 $vstart$ 指的是元素，使用元素组的向量加密指令要求这些值是元素组大小（EGS）的整数倍。

- 违反 $v1$ 或 $vstart$ 要求的指令是保留的。

指令	EGS
Vaes	4
Vsha2	4
Vg	4

Vsm3	8
Vsm4	4

LMUL 约束:

对于元素组指令, LMUL*VLEN 必须始终至少与 EGW 一样大, 否则会引发非法指令异常, 即使 v1=0。

SEW 约束:

某些向量加密指令仅为特定 SEW 定义。在这种情况下, 所有其他 SEW 值都是保留的。

指令	SEW	EGW
Vaes	32	128
Vsha2	32	128
Vsha2	64	256
Vg	32	128
Vsm3	32	256
Vsm4	32	128

源/目标重叠约束:

某些向量加密指令具有重叠约束。违反这些约束的编码是保留的。

在本规范中定义的 .vs 指令的情况下, vs2 持有 128 位标量元素组。对于 $VLEN \geq 128$ 的实现, vs2 指的是单个寄存器。因此, vd 寄存器组不得与 vs2 寄存器重叠。然而, 在 $VLEN < 128$ 的实现中, vs2 指的是由需要容纳 128 位标量元素组的寄存器数量组成的寄存器组。在这种情况下, vd 寄存器组不得与此 vs2 寄存器组重叠。

指令	SEW
Vaes	32
Vsha2	32
Vsha2	32 OR 64

在本规范定义的 .vs 指令中, vs2 保存的是一个 128 位的标量元素组。对于 $VLEN \geq 128$ 的实现, vs2 指向单个寄存器。因此, vd 寄存器组不得与 vs2 寄存器重叠。然而, 在 $VLEN < 128$ 的实现中, vs2 指向由存储 128 位标量元素组所需的多个寄存器组成的寄存器组。在这种情况下, vd 寄存器组不得与该 vs2 寄存器组重叠。

介绍	寄存器	不可重叠
vaes*.vs	Vs2	Vd
vsm4r.vs	Vs2	Vd
vsha2c[h1]	Vs1, Vs2	Vd
vsha2ms	Vs1, Vs2	Vd
vsm3me	Vs2	Vd
vsm3c	Vs2	Vd

33.1.6 向量-标量指令

RISC-V 向量扩展定义了三种向量-标量操作的编码, 这些操作从 GPR 或 FP 寄存器获取其标量操作数:

- OPIVX: 标量 GPR x 寄存器
- OPFVF: 标量 FP f 寄存器
- OPMVX: 标量 GPR x 寄存器

然而，向量扩展包括向量归约操作，这些操作也可以被视为向量-标量操作，因为标量操作数来自向量寄存器 `vs1` 的元素 0。向量操作数在向量寄存器组 `vs2` 中提供。这些归约操作在其助记符中都使用 `.vs` 后缀。此外，归约操作在目标寄存器 `vd` 的元素 0 中生成标量结果。

向量加密扩展定义了类似于这些向量归约操作的向量-标量指令，因为它们从向量寄存器获取标量操作数。然而，不同之处在于它们从 `vs2` 获取标量元素组，并将向量结果返回到 `vd`，`vd` 也是一个源向量操作数。这些向量-标量加密指令在其助记符中也使用 `.vs` 后缀。



我们选择使用 `vs2` 作为标量操作数，`vd` 作为向量操作数，以便我们可以使用 `vs1` 说明符作为这些指令的附加编码位。这使得这些指令具有更小的编码占用空间，为未来的其他指令留出更多空间。



这些指令使得可以将单个密钥（在 `vs2` 中指定为标量元素组）应用于寄存器组 `vd` 的每个元素组。在应用处理器中，标量元素组最多占用一个寄存器。然而，在 $VLEN < 128$ 的实现中，它们将占用 2（ $VLEN=64$ ）或 4（ $VLEN=32$ ）个寄存器。通常会并行执行多个 AES 加密轮次（例如，在计数器模式中）使用相同的轮密钥。与其先将公共密钥扩展到整个向量组，不如使用这些向量-标量加密指令将轮密钥指定为标量元素组。

33.1.7 软件可移植性

以下内容包含一些指南，以确保基于向量加密的代码在具有不同 $VLEN$ 值的实现中具有可移植性。应用处理器预计将遵循 V 扩展，因此 $VLEN \geq 128$ 。

由于大多数特定于加密的指令的 $EGW=128$ ，因此这些指令无需做任何特殊处理即可支持 $VLEN=128$ 的实现，然而，SHA-512 和 SM3 指令的 $EGW=256$ 。对于 $VLEN=128$ 的实现，需要将 $LMUL$ 加倍，以便在一对寄存器中创建 256 位元素。使用这种加倍的 $LMUL$ 编写的代码不会影响 $VLEN \geq 256$ 实现的结果，因为 `v1` 控制处理的元素组数量。因此，我们建议实现 SHA-512 和 SM3 的库使用加倍的 $LMUL$ ，以确保软件能够在所有 $VLEN \geq 128$ 的实现上正常运行。

虽然这种加倍的 $LMUL$ 对于 $VLEN \geq 256$ 的实现是安全的，但可能不够优化，因为会导致不必要的寄存器压力，并可能在某些微架构中引起性能损失。因此，我们建议库在提供 SHA-512 和 SM3 的可移植代码的同时，也为 $VLEN \geq 256$ 的实现提供优化后的代码。

算法	指令	$VLEN$	$LMUL$
SHA-512	<code>vsha2</code>	64	$V1/2$
SM3	<code>Vsm3</code>	32	$V1/4$

我们建议所有应用处理器的库代码都编写为可以在任何 $VLEN \geq 128$ 的实现上运行。这样的库也鼓励为 $VLEN \geq 256$ 的实现提供 SHA-512 和 SM3 的优化代码。

嵌入式处理器：

嵌入式处理器通常具有 $VLEN < 128$ 的实现。这将需要编写具有更大 $LMUL$ 值的代码，以便形成元素组。

`.vs` 指令需要 $EGW=128$ 的标量元素组。在 $VLEN < 128$ 的实现中，这些标量元素组将跨寄存器形成。这与大多数向量指令中的标量不同，后者通常消耗单个寄存器的一部分。

33.2 扩展概述

本节介绍了向量加密指令集扩展规范中的所有扩展。

Zvknh、Zvknhb 和 zvbc 向量加密扩展——以及相应的复合扩展 Zvkn 和 Zvks ——需要 Zve64x 基础或应用（V）基础向量扩展。所有其他向量加密扩展可以建立在任何嵌入式（Zve*）或应用（V）基础向量扩展之上。所有在本向量加密规范中定义的特定于加密的指令（即，zvkned、zvknh, Zvknh[ab]、Zvkg、Zvksed 和 zvksb，但不包括 zvbb、zvkb 或 zvbc）应以数据独立的执行延迟执行。需要注意的是，向量加密指令独立于 Zkt 扩展的实现，不要求实现 Zkt。

本规范包括一个 Zvkt 扩展，当实现时，要求某些向量指令（包括 zvbb、zvkb 和 zvbc）以数据独立的执行延迟执行。

检测单个加密扩展使用统一的软件 RISC-V 发现方法。



在撰写本文时，这些发现机制仍在进行中。

33.2.1 Zvbb – 向量基本位操作

向量基本位操作指令。

助记符	指令
vandn. [vv, vx]	向量逻辑与非
vbrev. v	向量元素内位反转
vbrev8. v	向量字节内位反转
vrev8. v	向量字节反转
vclz. v	向量前导零计数
vctz. v	向量后导零计数
vcpop. v	向量置位位数统计
vrol. [vv, vx]	向量循环左移
vror. [vv, vx, vi]	向量循环右移
vwsll. [vv, vx, vi]	向量扩展逻辑左移

33.2.2 Zvbc – 向量无进位乘法

通用无进位乘法指令，通常用于加密和哈希（例如，椭圆曲线加密、GHASH、CRC）。这些指令仅为 SEW=64 定义。

助记符	指令
vclmul. [vv, vx]	向量无进位乘法
vclmulh. [vv, vx]	向量无进位乘法返回高位部分

33.2.3 Zvkb – 向量加密位操作

向量位操作指令，对于安全高效地实现常见的加密工作负载至关重要。



此 Zvkb 扩展是 Zvbb 扩展的适当子集。Zvkb 允许向量加密实现而无需承担实现 Zvbb 扩展中额外位操作指令的成本：vbrev.v、vclz.v、vctz.v、vcpop.v 和 vwsll.[vv, vx, vi]

助记符	指令
vandn. [vv, vx]	无向标
vbrev8.v	字节内位反转
vrev8.v	字节反转
vrol. [vv, vx]	字节向量左移
vror. [vv, vx, vi]	字节向量右移

33.2.4 Zvkg – 向量 GCM/GMAC

指令用于高效实现 GHASH_H，该算法用于 Galois/Counter Mode (GCM) 和 Galois Message Authentication Code (GMAC)。所有这些指令都在由四个 32 位元素组成的 128 位元素组上工作。



GCM 与块密码（例如 AES 和 SM4）结合使用，以加密消息并提供认证。GMAC 用于在不加密的情况下提供消息认证。

为了避免侧信道定时攻击，这些指令应以数据独立的时间实现。处理的元素组数量为 v1/EGS。v1 必须设置为要处理的 SEW=32 元素的数量，因此必须是 EGS=4 的倍数。

同样，vstart 必须是 EGS=4 的倍数。

SEW	EGW	助记符	指令
32	128	vghsh.vv	向量化 GHASH 乘加运算
32	128	vgmul.vv	向量化 GHASH 乘法运算

33.2.5 Zvkned – NIST 套件：向量 AES 块密码

用于加速 AES 块密码的加密、解密和密钥调度功能的指令，如联邦信息处理标准出版物 197 中定义所有这些指令都在由四个 32 位元素组成的 128 位元素组上工作。

为了获得最佳性能，建议在 VLEN≥128 的系统上实现这些指令。在 VLEN<128 的系统上，可以通过使用 LMUL=2 和 LMUL=4 分别将两个或四个寄存器的 32 位元素连接起来形成元素组。

为了帮助避免侧信道定时攻击，这些指令应以数据独立的时间实现。处理的元素组数量为 v1/EGS。v1 必须设置为要处理的 SEW=32 元素的数量，因此必须是 EGS=4 的倍数。

同样，vstart 必须是 EGS=4 的倍数。

SEW	EGW	助记符	介绍
32	128	vaesef. [vv, vs]	矢量 AES 加密最后一轮
32	128	vaesem. [vv, vs]	矢量 AES 加密中间轮
32	128	vaesdf. [vv, vs]	矢量 AES 解密最后一轮

32	128	vaesdm. [vv, vs]	矢量 AES 解密中间轮
32	128	vaeskf1.vi	向量化 AES-128 正向密钥编排
32	128	vaeskf2.vi	向量化 AES-256 正向密钥编排
32	128	vaesz. vs	向量化 AES 零循环

33.2.6 Zvknh[ab] – NIST 套件：向量 SHA-2 安全哈希

用于加速 SHA-2 的指令，如 FIPS PUB 180-4 安全哈希标准 (SHS) 中定义。SEW 区分 SHA-256 (SEW=32) 和 SHA-512 (SEW=64)。

- SHA-256：这些指令在由四个 32 位元素组成的 128 位元素组上工作。
- SHA-512：这些指令在由四个 64 位元素组成的 256 位元素组上工作。

SEW	EGW	SHA-2	扩展
32	128	SHA-256	Zvknhb
64	256	SHA-512	Zvknha

- Zvknhb 支持 SHA-256 和 SHA-512。
- Zvknha 仅支持 SHA-256。

VLEN < 128 的 SHA-256 实现需要 LMUL>1 来组合寄存器组中的 32 位元素以提供元素组的所有四个元素。

VLEN < 256 的 SHA-512 实现需要 LMUL>1 来组合寄存器组中的 64 位元素以提供元素组的所有四个元素。

为了帮助避免侧信道定时攻击，这些指令应以数据独立的时间实现。处理的元素组数量为 v1/EGS。

v1 必须设置为要处理的 SEW 元素的数量，因此必须是 EGS=4 的倍数。同样，vstart 必须是 EGS=4 的倍数。

助记符	介绍
vsha2ms.vv	矢量 SHA-2 消息调度
vsha2c[h1].vv	矢量 SHA-2 压缩

33.2.7 Zvksed – ShangMi 套件：SM4 块密码

用于加速 SM4 块密码的加密、解密和密钥调度功能的指令。SM4 块密码在 SM4 块密码算中指定。还有其他各种描述 SM4 块密码的资源，虽然不是标准的最终版本。密码套件用于 TLS 1.3]是有用且易于访问的。所有这些指令都在由四个 32 位元素组成的 128 位元素组上工作。

为了帮助避免侧信道定时攻击，这些指令应以数据独立的时间实现。处理的元素组数量为 v1/EGS。v1 必须设置为要处理的 SEW=32 元素的数量，因此必须是 EGS=4 的倍数。同样，vstart 必须是 EGS=4 的倍数。

SEW	EGW	SHA-2	扩展
32	128	Vsm4k.vi	SM4 向量扩展
64	256	Vsm4kr[vv, vs]	SM4 循环扩展

33.2.8 Zvksh – ShangMi 套件：SM3 安全哈希

用于加速 SM3 哈希函数功能的指令。SM3 安全哈希算法在 32905-2016: SM3 加密哈希算法中指定。还有其他各种描述 SM3 安全哈希的资源。虽然不是标准的最终版本，但 ShangMi (SM) 密码套件用于

TLS 1.3 提供了一个有用且易于获取的参考。

所有这些指令都在由八个 32 位元素组成的 256 位元素组上工作。VLEN < 256 的实现需要 LMUL > 1 来组合寄存器组中的 32 位元素以提供元素组的所有八个元素。为了帮助避免侧信道定时攻击，这些指令应以数据独立的时间实现。处理的元素组数量为 v1/EGS。v1 必须设置为要处理的 SEW=32 元素的数量，因此必须是 EGS=8 的倍数。+同样，vstart 必须是 EGS=8 的倍数。

SEW	EGW	助记符	指令
32	256	Vsm3me.vv	SM3 信息扩展
32	256	Vsm3c.vv	SM3 压缩

33.2.9 Zvkn – NIST 算法套件

此扩展是以下其他扩展的简写：

包括的扩展	描述
Zvkned	Zvkned
Zvknhb	Zvknhb
Zvkb	Zvkb
Zvkt	Zvkt



虽然 Zvkg 和 Zvbc 不是此扩展的一部分，但建议至少实现其中之一，以便与此扩展一起实现高效的 AES-GCM

33.2.10 Zvknc – 带无进位乘法的 NIST 算法套件

此扩展是以下其他扩展的简写：

包括的扩展	描述
Zvkn	Zvkn
Zvbc	Zvbc



此扩展将 NIST 算法套件与向量无进位乘法扩展结合，以实现 AES-GCM

33.2.11 Zvkng – 带 GCM 的 NIST 算法套件

此扩展是以下其他扩展的简写：

包括的扩展	描述
Zvkn	Zvkn
Zvkg	Zvkg



此扩展将 NIST 算法套件与 GCM/GMAC 扩展结合，以实现高性能的 AES-GCM

33.2.12 Zvks - ShangMi 算法套件

此扩展是以下其他扩展的简写：

包括的扩展	描述
Zvksed	Zvksed
Zvksh	Zvksh
Zvkb	Zvkb
Zvkt	Zvkt



虽然 Zvkg 和 Zvbc 不是此扩展的一部分，但建议至少实现其中之一，以便与此扩展一起实现高效的 SM4-GCM。

33.2.13 Zvksc - 带无进位乘法的 ShangMi 算法套件

此扩展是以下其他扩展的简写：

包括的扩展	描述
Zvks	Zvks
Zvbc	Zvbc

33.2.14 Zvksg - 带 GCM 的 ShangMi 算法套件

此扩展是以下其他扩展的简写：

包括的扩展	描述
Zvks	Zvks
Zvkg	Zvkg



此扩展将 ShangMi 算法套件与 GCM/GMAC 扩展结合，以实现高性能的 SM4-GCM。

33.2.15 Zvkt - 向量数据独立执行延迟

Zvkt 扩展要求以下列表中的所有已实现指令都必须按照 RISC-V 标量加密扩展规范中定义的数据独立执行延迟（DIEL）执行。

数据独立执行延迟（DIEL）适用于指令的所有数据操作数，即便它们不在主体中或处于非活动状态。然而，DIEL 并不适用于其他值，例如 vl、vtype 和掩码，这些值主要用于控制掩码向量指令的执行时

机。此外，DIEL 也不适用于指令编码中指定的常量值，如零寄存器（x0），以及指令的立即数字段中的值（即 imm 和 uimm）。

在某些情况下——这些情况在下面的列表中明确指定——用作控制而不是数据的操作数不受 DIEL 约束。



DIEL 有助于防止侧信道定时攻击，这些攻击通过分析执行时间推测应保密的数据值，通常包括加密密钥、明文以及部分加密文本。然而，DIEL 并不是为了保护软件及其加密算法的机密性，因为假设攻击者已经知道这些内容。因此，DIEL 不适用于指令编码中嵌入的常量值。

需要注意的是，未在主体中或被掩码掉的元素的值不会影响指令的执行延迟，尽管这些元素有时包含需要保密的数据。

所有 Zvbb 指令

- vandn.v[vx]
- vclz.v
- vcpop.v
- vctz.v
- vbrev.v
- vbrev8.v
- vrev8.v
- vro1.v[vx]
- vror.v[vxi]
- vwsll.[vv, vx, vi]



所有 Zvkb 指令也受 DIEL 约束，因为它们是 Zvbb 的适当子集。

所有 Zvbc 指令

- vclmul[h].v[vx]

加/减

- v[r]sub.v[vx]
- vadd.v[ivx]
- vsub.v[vx]
- vwadd[u].[vw][vx]
- vwsb[u].[vw][vx]

带进位的加/减

- vadc.v[ivx]m
- vmadc.v[ivx][m]
- vmsbc.v[vx]m
- vsbc.v[vx]m

比较和设置

- vmseq.v[vxi]
- vmsgt[u].v[xi]
- vmsle[u].v[xi]
- vmslt[u].v[xi]
- vmsne.v[ivx]

复制

- vmv.s.x
- vmv.v.[ivxs]
- vmv[1248]r.v

扩展

- vsext.vf[248]
- vzext.vf[248]

逻辑

- vand.v[ivx]
- vm[n]or.mm
- vmand[n].mm
- vmnand.mm
- vmorn.mm
- vmx[n]or.mm
- vor.v[ivx]
- vxor.v[ivx]

乘法

- vmul[h].v[vx]
- vmulh[s]u.v[vx]
- vwmul.v[vx]
- vwmul[s]u.v[vx]

乘加

- vmacc.v[vx]
- vmadd.v[vx]
- vnmsac.v[vx]
- vnmsub.v[vx]
- vwmacc.v[vx]
- vwmacc[s]u.v[vx]
- vwmaccus.vx

整数合并

- vmerge.v[ivx]m

置换

在 `vragather[eil6]` 指令的 `.vv` 和 `.xv` 形式中, `vs1` 和 `rs1` 中的值用作控制, 因此不受 DIEL 约束。

移位

- `vnsr[a1].w[ivx]`
- `vsll.v[ivx]`
- `vsr[a1].v[ivx]`

滑动

- `vslide1[up|down].vx`
- `vfslide1[up|down].vf`

在 `vslide[up|down].vx` 指令中, `rs1` 中的值用作控制 (即滑动量), 因此不受 DIEL 约束。

- `vslide[up|down].v[ix]`

以下指令不受 `Zvkt` 影响:

- *所有存储操作*
- *所有浮点操作*
- 饱和加/减
- * `vsadd[u].v[ivx]`
- * `vssub[u].v[vx]`
- 裁剪
- * `vnclip[u].w[ivx]`
- 压缩
- * `vcompress.vm`
- 除法
- * `vdiv[u].v[vx]`
- * `vrem[u].v[vx]`
- 平均
- * `vaadd[u].v[vx]`
- * `vasub[u].v[vx]`
- 掩码操作
- * `vcpop.m`
- * `vfirst.m`
- * `vid.v`
- * `viota.m`
- * `vms[bio]f.m`
- 最小/最大
- * `vmax[u].v[vx]`
- * `vmin[u].v[vx]`
- 乘法饱和
- * `vsmul.v[vx]`
- 归约
- * `vredsum.vs`
- * `vwredsum[u].vs`

```
* vred[and|or|xor].vs
* vred[min|max][u].vs
- 移位舍入
* vssra.v[ivx]
* vssrl.v[ivx]
- vset
* vsetivli
* vsetvl[i]
```

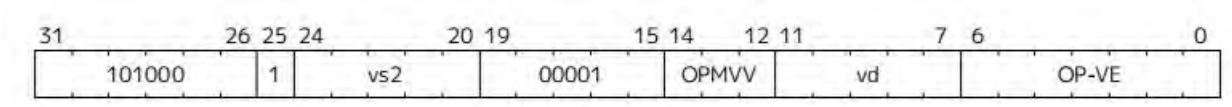
33.3 指令

33.3.1 vaesdf. [vv, vs]

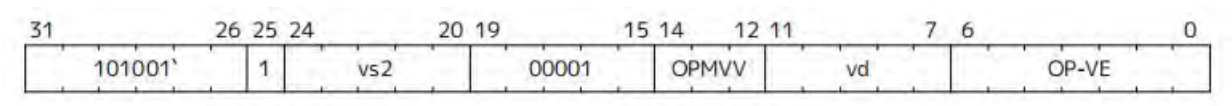
简述：
向量 AES 最终轮解密

助记符：
vaesdf.vv vd, vs2 +
vaesdf.vs vd, vs2

编码（向量-向量）：



编码（向量-标量）：



- 保留编码：
- * SEW 为 32 以外的任何值
 - * 仅适用于 .vs 形式：vd 寄存器组与 vs2 标量元素组重叠

参数：

寄存器	接口	EGW	EGS	EEW	定义
Vd	输入	128	4	32	状态
Vs2	输入	128	4	32	秘钥
Vd	输出	128	4	32	新一轮状态

描述：

执行 AES 块密码的最终轮解密。

对 vd 中的每个轮状态元素组应用 InvShiftRows 和 InvSubBytes 步骤。然后将其与 vs2 中相应元素组（向量-向量形式）或 vs2 中的标量元素组（向量-标量形式）中的轮密钥进行异或。此指令的执行延迟必须始终与操作的数据无关。

操作:

```
function clause execute (VAESDF(vs2, vd, suffix)) = {

  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {

    eg_len = (vl/EGS)
    eg_start = (vstart/EGS)

    foreach (i from eg_start to eg_len-1) {
      let keyelem = if suffix == "vv" then i else 0;
      let state : bits(128) = get_velem(vd, EGW=128, i);
      let rkey : bits(128) = get_velem(vs2, EGW=128, keyelem);
      let sr : bits(128) = aes_shift_rows_inv(state);
      let sb : bits(128) = aes_subbytes_inv(sr);
      let ark : bits(128) = sb ^ rkey;
      set_velem(vd, EGW=128, i, ark);
    }
    RETIRE_SUCCESS
  }
}
```

包含于:

Zvkn, Zvknc, Zvkned, Zvkng

33.3.2 vaesdm. [vv, vs]

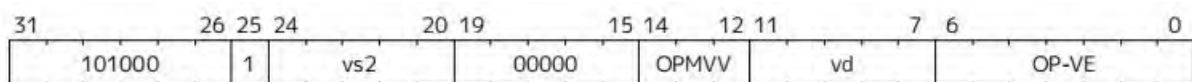
简述:

向量 AES 中间轮解密

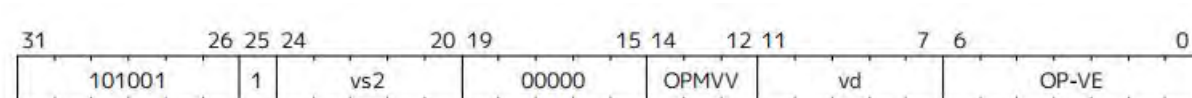
助记符:

vaesdm.vv vd, vs2 +
vaesdm.vs vd, vs2

编码 (向量-向量):



编码 (向量-标量):



保留编码:

- * SEW 为 32 以外的任何值
- * 仅适用于 .vs 形式: vd 寄存器组与 vs2 标量元素组重叠

参数:

寄存器	方向	EGW	EGS	EEW	描述
Vd	输入	128	4	32	循环状态
Vs2	输入	128	4	32	轮密钥
Vd	输出	128	4	32	新循环状态

描述:

执行 AES 块密码的中间轮解密。

对 vd 中的每个轮状态元素组应用 InvShiftRows 和 InvSubBytes 步骤。然后将其与 vs2 中相应元素组（向量-向量形式）或 vs2 中的标量元素组（向量-标量形式）中的轮密钥进行异或。结果然后应用于 InvMixColumns 步骤。此指令的执行延迟必须始终与操作的数据无关。

操作:

```
function clause execute (VAESDM(vs2, vd, suffix)) = {
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {
    eg_len = (vl/EGS)
    eg_start = (vstart/EGS)

    foreach (i from eg_start to eg_len-1) {
      let keyelem = if suffix == "vv" then i else 0;
      let state : bits(128) = get_velem(vd, EGW=128, i);
      let rkey  : bits(128) = get_velem(vs2, EGW=128, keyelem);
      let sr    : bits(128) = aes_shift_rows_inv(state);
      let sb    : bits(128) = aes_subbytes_inv(sr);
      let ark   : bits(128) = sb ^ rkey;
      let mix   : bits(128) = aes_mixcolumns_inv(ark);
      set_velem(vd, EGW=128, i, mix);
    }
    RETIRE_SUCCESS
  }
}
```

包含于:

Zvkn, Zvknc, Zvkned, Zvkng

33.3.3 vaesef. [vv, vs]

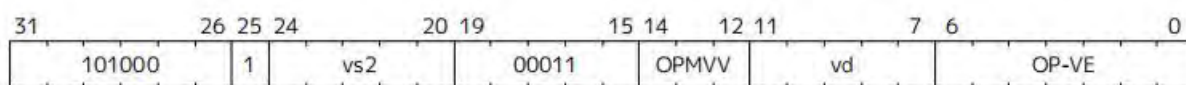
简述:

向量 AES 最终轮加密

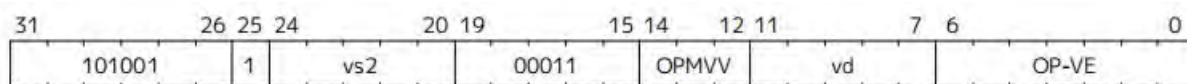
助记符:

vaesef.vv vd, vs2 +
vaesef.vs vd, vs2

编码（向量-向量）:



编码（向量-标量）:



保留编码:

- * SEW 为 32 以外的任何值
- * 仅适用于 .vs 形式: vd 寄存器组与 vs2 标量元素组重叠

参数:

寄存器	方向	EGW	EGS	EEW	描述
Vd	输入	128	4	32	循环状态
Vs2	输入	128	4	32	轮密钥
Vd	输出	128	4	32	新循环状态

描述:

执行 AES 块密码的最终轮加密。

对 vd 中的每个轮状态元素组应用 SubBytes 和 ShiftRows 步骤。然后将其与 vs2 中相应元素组（向量-向量形式）或 vs2 中的标量元素组（向量-标量形式）中的轮密钥进行异或。此指令的执行延迟必须始终与操作的数据无关。

操作:

```
function clause execute (VAESEF(vs2, vd, suffix) = {
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {

    eg_len = (vl/EGS)
    eg_start = (vstart/EGS)
```

```

foreach (i from eg_start to eg_len-1) {
    let keyelem = if suffix == "vv" then i else 0;
    let state : bits(128) = get_velem(vd, EGW=128, i);
    let rkey  : bits(128) = get_velem(vs2, EGW=128, keyelem);
    let sb    : bits(128) = aes_subbytes_fwd(state);
    let sr    : bits(128) = aes_shift_rows_fwd(sb);
    let ark   : bits(128) = sr ^ rkey;
    set_velem(vd, EGW=128, i, ark);
}
RETIRE_SUCCESS
}
}

```

包含于：

Zvkn, Zvknc, Zvkned, Zvkng

33.3.4 vaesem. [vv, vs]

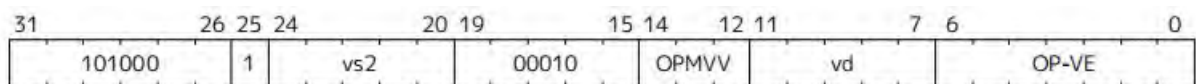
简述：

向量 AES 中间轮加密

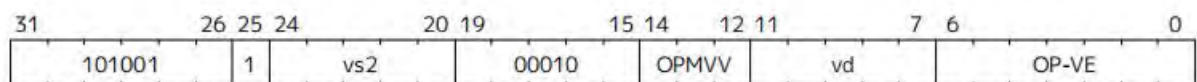
助记符：

vaesem.vv vd, vs2 +
vaesem.vs vd, vs2

编码（向量-向量）：



编码（向量-标量）：



保留编码：

- * SEW 为 32 以外的任何值
- * 仅适用于 .vs 形式：vd 寄存器组与 vs2 标量元素组重叠

参数：

寄存器	方向	EGW	EGS	EEW	描述
Vd	输入	128	4	32	循环状态
Vs2	输入	128	4	32	轮密钥
Vd	输出	128	4	32	新循环状态

描述：

执行 AES 块密码的中间轮加密。

对 vd 中的每个轮状态元素组应用 SubBytes、ShiftRows 和 MixColumns 步骤。然后将其与 vs2

中相应元素组（向量-向量形式）或 vs2 中的标量元素组（向量-标量形式）中的轮密钥进行异或。此指令的执行延迟必须始终与操作的数据无关。

操作:

```
function clause execute (VAESEM(vs2, vd, suffix)) = {
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {

    eg_len = (v1/EGS)
    eg_start = (vstart/EGS)

    foreach (i from eg_start to eg_len-1) {
      let keyelem = if suffix == "vv" then i else 0;
      let state : bits(128) = get_velem(vd, EGW=128, i);
      let rkey  : bits(128) = get_velem(vs2, EGW=128, keyelem);
      let sb    : bits(128) = aes_subbytes_fwd(state);
      let sr    : bits(128) = aes_shift_rows_fwd(sb);
      let mix   : bits(128) = aes_mixcolumns_fwd(sr);
      let ark   : bits(128) = mix ^ rkey;
      set_velem(vd, EGW=128, i, ark);
    }
    RETIRE_SUCCESS
  }
}
```

包含于:

Zvkn, Zvknc, Zvkned, Zvkng

33.3.5 vaeskf1.vi

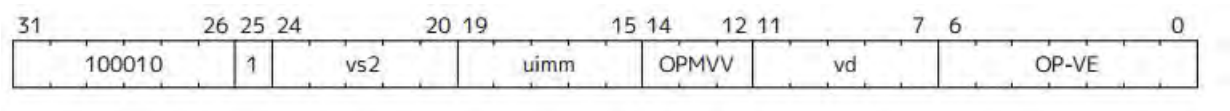
简述:

向量 AES-128 前向密钥调度生成

助记符:

vaeskf1.vi vd, vs2, uimm

编码:



保留编码:

* SEW 为 32 以外的任何值

参数:

寄存器	方向	EGW	EGS	EEW	描述
-----	----	-----	-----	-----	----

Uimm	输入	-	-	-	循环数组
Vs2	输入	128	4	32	当前轮密钥
Vd	输出	128	4	32	轮密钥

描述:

执行 AES-128 密钥调度的单轮操作。

在每个元素组内，从 vs2 中的当前轮密钥元素组和前一轮密钥的前一个字生成下一个轮密钥。最低有效字由当前轮密钥的最高有效字和由轮数选择的轮常数生成。

轮数范围为 1 到 10，取自 uimm[3:0]，uimm[4] 被忽略。对于超出范围的 uimm[3:0] 值（0 和 11-15），通过反转 uimm[3] 映射到有效范围，因此 0 映射到 8，11-15 映射到 3-7。轮数用于指定生成第一个轮密钥字的轮常数。此指令的执行延迟应与操作的数据无关。



我们选择将超出范围的轮数映射到有效范围的值，因为这样做可以确保指令的行为对于所有 uimm[4:0] 值都得到完全定义，同时仅需最少的额外逻辑。

操作:

```
function clause execute (VAESKF1(rnd, vd, vs2)) = {
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {

    if( (unsigned(rnd[3:0]) > 10) | (rnd[3:0] = 0)) then rnd[3] = ~rnd[3]

    eg_len = (vl/EGS)
    eg_start = (vstart/EGS)

    let r : bits(4) = rnd-1;

    foreach (i from eg_start to eg_len-1) {
      let CurrentRoundKey[3:0] : bits(128) = get_velem(vs2, EGW=128, i);
      let w[0] : bits(32) = aes_subword_fwd(aes_rotword(CurrentRoundKey[3])) XOR
        aes_decode_rcon(r) XOR CurrentRoundKey[0]
      let w[1] : bits(32) = w[0] XOR CurrentRoundKey[1]
      let w[2] : bits(32) = w[1] XOR CurrentRoundKey[2]
      let w[3] : bits(32) = w[2] XOR CurrentRoundKey[3]
      set_velem(vd, EGW=128, i, w[3:0]);
    }
    RETIRE_SUCCESS
  }
}
```

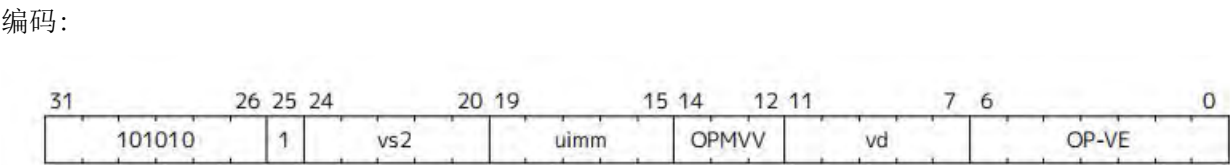
包含于:

Zvkn, Zvknc, Zvkned, Zvkng

33.3.6 vaeskf2.vi

简述：
向量 AES-256 前向密钥调度生成

助记符：
vaeskf2.vi vd, vs2, uimm



保留编码：
* SEW 为 32 以外的任何值

参数：

寄存器	方向	EGW	EGS	EEW	描述
Vd	输入	128	4	32	先前轮密钥
Vs2	输入	128	4	32	当前轮密钥
Vd	输出	128	4	32	下一个循环状态

描述：

执行 AES-256 密钥调度的单轮操作。

在每个元素组内，从 vd 中的前一轮密钥元素组和前一轮密钥的前一个字生成下一个轮密钥。最低有效的字是通过对当前轮密钥的最有效字应用一个函数，然后将结果与轮常数进行异或生成的。轮数用于选择轮常数以及函数。

轮数范围为 2 到 14，来自 uimm[3:0]；uimm[4] 被忽略。0-1 和 15 的超出范围的 uimm[3:0] 值通过反转 uimm[3] 映射到范围内的值。因此，0-1 映射到 8-9，15 映射到 7。此指令的执行延迟必须始终与操作的数据无关。

我们选择将超出范围的轮数映射到范围内的值，因为这允许指令的行为对所有 uimm[4:0] 值完全定义，并且需要最少的额外逻辑。

操作：

```
function clause execute (VAESKF2(rnd, vd, vs2)) = {
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {
```

```

if((unsigned(rnd[3:0]) < 2) | (unsigned(rnd[3:0]) > 14)) then rnd[3] = ~rnd[3]

eg_len = (vl/EGS)
eg_start = (vstart/EGS)

foreach (i from eg_start to eg_len-1) {
    let CurrentRoundKey[3:0] : bits(128) = get_velem(vs2, EGW=128, i);
    let RoundKeyB[3:0] : bits(32) = get_velem(vd, EGW=128, i); // Previous round
key
    let w[0] : bits(32) = if (rnd[0]==1) then
        aes_subword_fwd(CurrentRoundKey[3]) XOR RoundKeyB[0];
    else
        aes_subword_fwd(aes_rotword(CurrentRoundKey[3])) XOR aes_decode_rcon((rnd>>1)
- 1) XOR RoundKeyB[0];
    w[1] : bits(32) = w[0] XOR RoundKeyB[1]
    w[2] : bits(32) = w[1] XOR RoundKeyB[2]
    w[3] : bits(32) = w[2] XOR RoundKeyB[3]
    set_velem(vd, EGW=128, i, w[3:0]);
}
    RETIRE_SUCCESS
}
}

```

包含于：

Zvkn, Zvknc, Zvkned, Zvkng

33.3.7 vaesz. vs

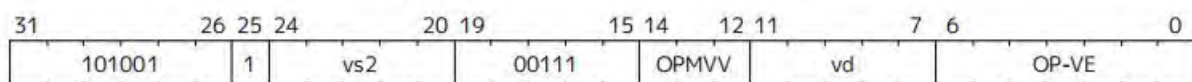
简述：

向量 AES 零轮加密/解密

助记符：

vaesz. vs vd, vs2

编码（向量-标量）：



保

留编码：

- * SEW 为 32 以外的任何值
- * vd 寄存器组与 vs2 寄存器重叠

参数：

寄存器	方向	EGW	EGS	EEW	描述
Vd	输入	128	4	32	循环状态
Vs2	输入	128	4	32	轮密钥
Vd	输出	128	4	32	新循环状态

描述:

执行 AES 块密码的零轮操作。此操作用于加密和解密。

此指令仅有 .vs 形式。Vs2 持有一个标量元素组，作为所有轮状态元素组的轮密钥。每个元素组的新轮状态输出通过将轮密钥与 vd 的每个元素组进行异或生成。此指令的执行延迟必须始终与操作的数据无关。



此指令用于避免在轮密钥对所有 128 位通道相同时需要扩展 128 位向量寄存器组。这样的扩展通常通过 vrgather 指令实现，这会在许多实现中影响性能。此指令仅存在 .vs 形式，因为 .vv 形式与 vxor.vv vd, vs2, vd 指令相同。

操作:

```
function clause execute (VAESZ(vs2, vd) = {
  if(((vstart%EGS)<>0) | (LMUL*VLEN < EGW)) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {

    eg_len = (vl/EGS)
    eg_start = (vstart/EGS)

    foreach (i from eg_start to eg_len-1) {
      let state : bits(128) = get_velem(vd, EGW=128, i);
      let rkey : bits(128) = get_velem(vs2, EGW=128, 0);
      let ark : bits(128) = state ^ rkey;
      set_velem(vd, EGW=128, i, ark);
    }
    RETIRE_SUCCESS
  }
}
```

包含于:

Zvkn, Zvknc, Zvkned, Zvkng

33.3.8 vandn. [vv, vx]

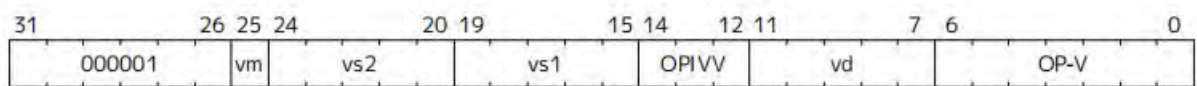
简述:

按位与非

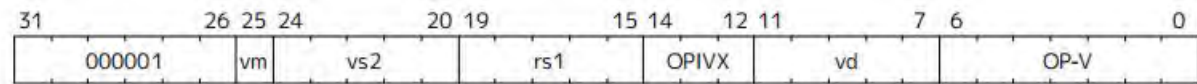
助记符:

vandn.vv vd, vs2, vs1, vm +
vandn.vx vd, vs2, rs1, vm

编码 (向量-向量):



编码（向量-标量）:



向量-向量参数:

寄存器	方向	描述
Vs1	输入	Op1
Vs2	输入	Op2
Vd	输出	结果

向量-标量参数:

寄存器	方向	描述
Vs1	输入	Op1
Vs2	输入	Op2
Vd	输出	结果

描述:

执行按位与非操作。Op1 的每个位取反并与 vs2 中的相应位进行逻辑与。在向量-标量版本中，Op1 是标量寄存器 rs1 中的符号扩展或截断值。在向量-向量版本中，Op1 是 vs1。



关于指令必要性的说明:此指令对 SHA3 的性能至关重要，特别是 FIPS 202 Keccak 置换的 Chi 步骤。通过 2 条指令模拟它预计会对性能产生重大影响。此指令的 .vv 形式是 SHA3 所需的；添加 .vx 形式是为了完整性

此指令对 SHA3 的性能至关重要，特别是 FIPS 202 Keccak 置换的 Chi 步骤。通过 2 条指令模拟它预计会对性能产生重大影响。此指令的 .vv 形式是 SHA3 所需的；添加 .vx 形式是为了完整性。



没有 .vi 版本的此指令，因为可以通过使用 vand.vi 指令的立即数值的取反来实现相同的功能。

操作:

```

function clause execute (VANDN(vs2, vs1, vd, suffix)) = {
  foreach (i from vstart to vl-1) {
    let op1 = match suffix {
      "vv" => get_velem(vs1, SEW, i),
      "vx" => sext_or_truncate_to_sew(X(vs1))
    };
    let op2 = get_velem(vs2, SEW, i);
    set_velem(vd, EEW=SEW, i, ~op1 & op2);
  }
  RETIRE_SUCCESS
}

```

包含于：

Zvkn, Zvknc, Zvkned, Zvkng

33.3.9 vbrev.v

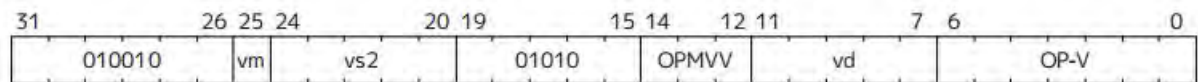
简述：

向量反转元素中的位

助记符：

vbrev.v vd, vs2, vm

编码（向量）：



参数：

寄存器	方向	描述
Vs2	输入	输入元素
Vd	输出	返回位反转元素

描述：

对每个元素的位进行反转。

操作：

```

function clause execute (VBREV(vs2)) = {
  foreach (i from vstart to vl-1) {
    let input = get_velem(vs2, SEW, i);
    let output : bits(SEW) = 0;
    foreach (i from 0 to SEW-1)
      let output[SEW-1-i] = input[i];
    set_velem(vd, SEW, i, output)
  }
  RETIRE_SUCCESS
}

```

包含于：

Zvbb, Zvkb, Zvkn, Zvknc, Zvkng, Zvks Zvksc, Zvksg

33.3.10 vbrev8.v

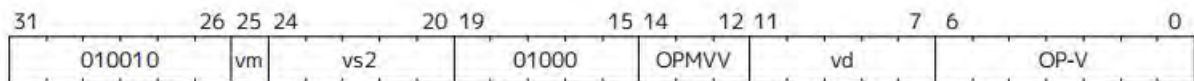
简述：

向量反转字节中的位

助记符：

vbrev8.v vd, vs2, vm

编码（向量）：



参数：

寄存器	方向	描述
Vs2	输入	输入元素
Vd	输出	每个字节位进行反转

描述：

对每个字节的位进行反转。



此指令通常用于在未实现 `zvkg` 扩展时的 GCM。此字节级指令为所有 SEW 定义，以消除在处理更宽元素时更改 SEW 的需要。

操作：

```
function clause execute (VBREV8(vs2)) = {  
    foreach (i from vstart to vl-1) {  
        let input = get_velem(vs2, SEW, i);  
        let output : bits(SEW) = 0;  
        foreach (i from 0 to SEW-8 by 8)  
            let output[i+7..i] = reverse_bits_in_byte(input[i+7..i]);  
        set_velem(vd, SEW, i, output)  
    }  
    RETIRE_SUCCESS  
}
```

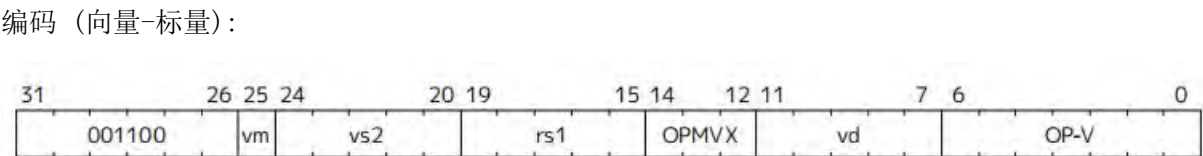
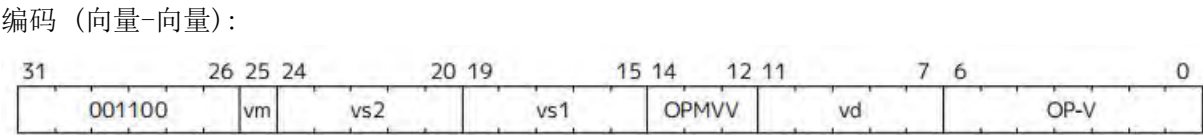
包含于：

Zvbb

33.3.11 vclmul. [vv, vx]

简述：
向量无进位乘法（返回乘积的低半部分）

助记符：
vclmul.vv vd, vs2, vs1, vm +
vclmul.vx vd, vs2, rs1, vm



保

留编码：
* SEW 为 64 以外的任何值

参数：

寄存器	方向	描述
Vs1/Rs1	输入	乘
Vs2	输入	无进位乘
Vd	输出	无进位乘积低半部分

描述：
生成 128 位无进位乘积的低半部分。
vs2 向量寄存器中的每个 64 位元素与 vs1 中的每个 64 位元素（向量-向量）或整数寄存器 rs1 中的 64 位值（向量-标量）进行无进位乘法。结果是无进位乘积的最低 64 位。

在没有zvkg扩展的情况下，64位无进位乘法指令可用于实现GCM模式。我们并未将这些指令设为独占性质，因为64位无进位乘法可以很容易地从zvkg扩展的指令中派生出来，且在其他领域也可能具有应用价值。同样地，我们将其他SEW（元素宽度）值视为保留位，以便不妨碍未来扩展使用不同元素宽度的操作码。例如，未来扩展可能会定义该指令的SEW=32版本，从而使Zve32*实现方案也能支持向量无进位乘法指令。

操作：

```
function clause execute (VCLMUL(vs2, vs1, vd, suffix)) = {  
  foreach (i from vstart to vl-1) {  
    let op1 : bits (64) = if suffix == "vv" then get_velem(vs1,i)  
                          else zext_or_truncate_to_sew(X(vs1));  
    let op2 : bits (64) = get_velem(vs2,i);
```

```

        let product : bits (64) = clmul(op1,op2,SEW);
        set_velem(vd, i, product);
    }
    RETIRE_SUCCESS
}

function clmul(x, y, width) = {
    let result : bits(width) = zeros();
    foreach (i from 0 to (width - 1)) {
        if y[i] == 1 then result = result ^ (x << i);
    }
    result
}

```

包含于：

Zvbc, Zvknc, Zvksc

33.3.12 vclmulh. [vv, vx]

简述：

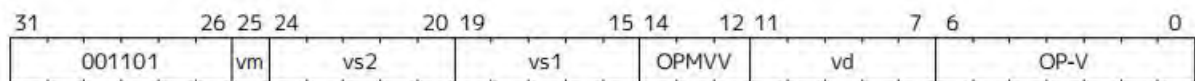
向量无进位乘法（返回乘积的高半部分）

助记符：

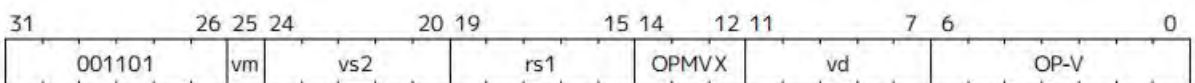
vclmulh.vv vd, vs2, vs1, vm +

vclmulh.vx vd, vs2, rs1, vm

编码（向量-向量）：



编码（向量-标量）：



保留编码：

* SEW 为 64 以外的任何值

参数：

寄存器	方向	描述
Vs1/Rs1	输入	乘
Vs2	输入	无进位乘
Vd	输出	无进位乘积高半部分

描述：

生成 128 位无进位乘积的高半部分。

vs2 向量寄存器中的每个 64 位元素与 vs1 中的每个 64 位元素（向量-向量）或整数寄存器 rs1 中的 64 位值（向量-标量）进行无进位乘法。结果是无进位乘积的最高 64 位。

操作:

```
function clause execute (VCLMULH(vs2, vs1, vd, suffix)) = {  
    foreach (i from vstart to vl-1) {  
        let op1 : bits (64) = if suffix == "vv" then get_velem(vs1,i)  
                               else zext_or_truncate_to_sew(X(vs1));  
        let op2 : bits (64) = get_velem(vs2, i);  
        let product : bits (64) = clmulh(op1, op2, SEW);  
        set_velem(vd, i, product);  
    }  
    RETIRE_SUCCESS  
}  
  
function clmulh(x, y, width) = {  
    let result : bits(width) = 0;  
    foreach (i from 1 to (width - 1)) {  
        if y[i] == 1 then result = result ^ (x >> (width - i));  
    }  
    result  
}
```

包含于:

Zvbc, Zvknc, Zvksc

33.3.13 vclz.v

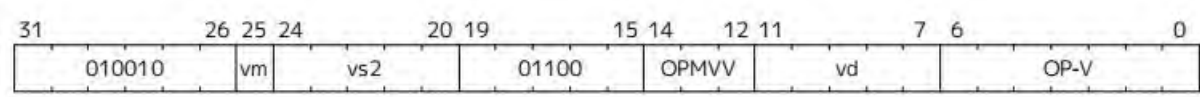
简述:

向量前导零计数

助记符:

vclz.v vd, vs2, vm

编码（向量）:



参数:

寄存器	方向	描述
Vs2	输入	输入元素
Vd	输出	每个元素执行前导零计数

描述:

对每个元素执行前导零计数。

对于零值输入，结果为 SEW 的值。

操作:

```
function clause execute (VCLZ(vs2)) = {  
    foreach (i from vstart to vl-1) {  
        let input = get_velem(vs2, SEW, i);  
        for (j = (SEW - 1); j >= 0; j--)  
            if [input[j]] == 0b1 then break;  
        set_velem(vd, SEW, i, SEW - 1 - j)  
    }  
    RETIRE_SUCCESS  
}
```

包含于:

Zvbb

33.3.14 vcpop.v

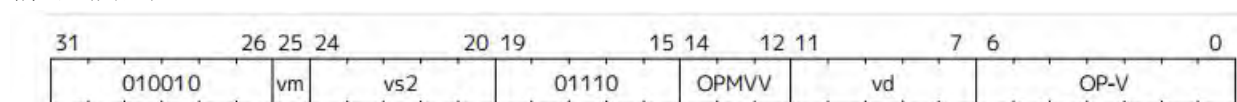
简述:

计算每个元素中设置的位数

助记符:

vcpop.v vd, vs2, vm

编码 (向量):



参数:

寄存器	方向	描述
Vs2	输入	输入元素
Vd	输出	每个元素执行计数

描述:

对每个元素执行人口计数。

操作:

```
function clause execute (VCPop(vs2)) = {
    foreach (i from vstart to vl-1) {
        let input = get_velem(vs2, SEW, i);
        let output : bits(SEW) = 0;
        for (j = 0; j < SEW; j++)
            output = output + input[j];
        set_velem(vd, SEW, i, output)
    }
    RETIRE_SUCCESS
}
```

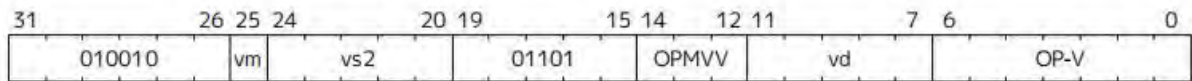
包含于：
Zvbb

33.3.15 vctz.v

简述：
向量尾随零计数

助记符：
vctz.v vd, vs2, vm

编码（向量）：



参数：

寄存器	方向	描述
Vs2	输入	输入元素
Vd	输出	每个元素执行尾随零计数

描述：
对每个元素执行尾随零计数。

操作：

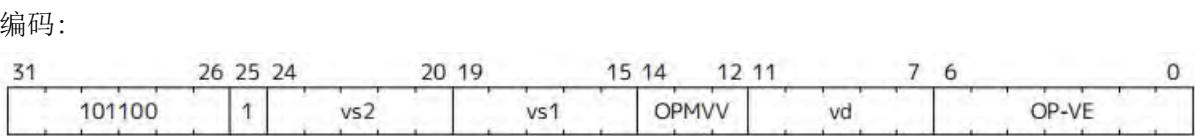
```
function clause execute (VCTZ(vs2)) = {  
    foreach (i from vstart to vl-1) {  
        let input = get_velem(vs2, SEW, i);  
        for (j = 0; j < SEW; j++)  
            if [input[j]] == 0b1 then break;  
        set_velem(vd, SEW, i, j)  
    }  
    RETIRE_SUCCESS  
}
```

包含于：
 Zvbb

33.3.16 vghsh.vv

简述：
 在 GHASH Galois-Field 上进行向量加-乘

助记符：
 vghsh.vv vd, vs2, vs1



留编码：
 * SEW 为 32 以外的任何值

参数：

寄存器	方向	EGW	EGS	EEW	描述
Vd	输入	128	4	32	部分哈希
Vs1	输入	128	4	32	密文
Vs2	输入	128	4	32	散列注册表子项
Vd	输出	128	4	32	部分哈希

描述：
 执行 GHASH^H 算法的单次迭代。
 此指令将所有输入和输出视为 128 位多项式，并在 GF[2] 上执行操作。它通过将当前部分哈希 (Yⁱ) 加到密文块 (Xⁱ)，然后将此和乘以哈希子密钥 (H) (在 GF(2¹²⁸) 上) 生成下一个部分哈希 (Yⁱ⁺¹)。在 GF(2¹²⁸) 上的乘法是两个 128 位多项式的无进位乘法，模 GHASH 的不可约多项式 (x¹²⁸ + x⁷ + x² + x + 1)。

操作可以简洁地定义为:

$$Y^{i+1} = ((Y^i \wedge X^i) \&\#183; H)$$

NIST 规范按从左到右的顺序排列系数 $x^0 \sim x^1 \sim x^2 \sim \dots \sim x^{127}$, 对于多项式 $x^0 \sim + x^1 \sim u + x^2 \sim u^2 + \dots + x^{127} \sim u^{127}$ 。这可以看作是内存中字节元素的集合, 最低系数的字节 (即 0, 1, 2, 3, 4, 5, 6, 7) 位于最低内存地址。由于字节中的位是反转的, 此指令在字节内执行位交换以将位放入标准顺序 (例如, 7, 6, 5, 4, 3, 2, 1, 0)。此指令的执行延迟必须始终与操作的数据无关。



我们正在反转输入和输出的字节位, 以便中间值与 NIST 规范一致。这些反转实现起来成本较低, 因为它们无条件地交换位位置, 因此不需要任何逻辑。

由于相同的哈希子密钥 H 通常会在给定消息上重复使用, 未来的扩展可能会定义此指令的向量-标量版本, 其中 vs2 是标量元素组。这将有助于在 $LMUL > 1$ 时减少寄存器压力。

操作:

```
function clause execute (VGSH(vs2, vs1, vd)) = {
  // operands are input with bits reversed in each byte
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {

    eg_len = (vl/EGS)
    eg_start = (vstart/EGS)

    foreach (i from eg_start to eg_len-1) {
      let Y = (get_velem(vd,EGW=128,i)); // current partial-hash
      let X = get_velem(vs1,EGW=128,i); // block cipher output
      let H = brev8(get_velem(vs2,EGW=128,i)); // Hash subkey

      let Z : bits(128) = 0;

      let S = brev8(Y ^ X);

      for (int bit = 0; bit < 128; bit++) {
        if bit_to_bool(S[bit])
          Z ^= H

        bool reduce = bit_to_bool(H[127]);
        H = H << 1; // left shift H by 1
        if (reduce)
          H ^= 0x87; // Reduce using x^7 + x^2 + x^1 + 1 polynomial
      }

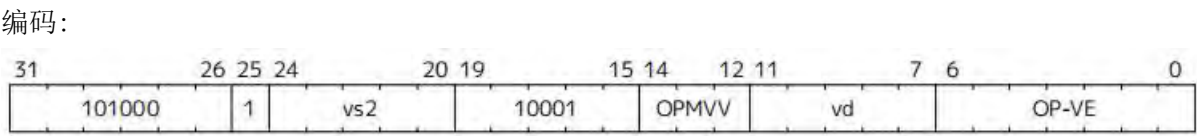
      let result = brev8(Z); // bit reverse bytes to get back to GCM standard ordering
      set_velem(vd, EGW=128, i, result);
    }
    RETIRE_SUCCESS
  }
}
```

包含于：
Zvkg, Zvkng, Zvksg

33.3.17 vgmul.vv

简述：
在 GHASH Galois-Field 上进行向量乘法

助记符：
vgmul.vv vd, vs2



保留编码：
* SEW 为 32 以外的任何值

参数：

寄存器	方向	EGW	EGS	EEW	描述
Vd	输入	128	4	32	乘
Vs2	输入	128	4	32	无进位乘
Vd	输出	128	4	32	结果

描述：
执行 GHASH^H 乘法。
此指令将所有输入和输出视为 128 位多项式，并在 $\text{GF}[2]$ 上执行操作。它生成两个 128 位输入在 $\text{GF}(2^{128})$ 上的乘积。在 $\text{GF}(2^{128})$ 上的乘法是两个 128 位多项式的无进位乘法，模 GHASH 的不可约多项式 $(x^{128} + x^7 + x^2 + x + 1)$ 。
NIST 规范按从左到右的顺序排列系数 $x^0 \sim x^1 \sim x^2 \sim \dots \sim x^{127}$ ，对于多项式 $x^0 \sim + x^1 \sim u + x^2 \sim u^2 + \dots + x^{127} \sim u^{127}$ 。这可以看作是内存中字节元素的集合，最低系数的字节（即 0, 1, 2, 3, 4, 5, 6, 7）位于最低内存地址。由于字节中的位是反转的，此指令在字节内执行位交换以将位放入标准顺序（例如，7, 6, 5, 4, 3, 2, 1, 0）。此指令的执行延迟必须始终与操作的数据无关。

我们正在反转输入和输出的字节位，以便中间值与 NIST 规范一致。这些反转实现起来成本较低，因为它们无条件地交换位位置，因此不需要任何逻辑。

由于相同的被乘数通常会在给定消息上重复使用，未来的扩展可能会定义此指令的向量-标量版本，其中 vs2 是标量元素组。这将有助于在 $\text{LMUL} > 1$ 时减少寄存器压力。

此指令与 vghsh.vv 相同，只是 vs1=0。此指令通常用于 GHASH 代码。在某些情况下，它后面会跟一个 XOR 以执行乘加。实现可能会选择融合这两个指令，以提高不使用 vghsh.vv 指令的加乘形式的 GHASH 代码的性能。

操作:

```
function clause execute (VGMUL(vs2, vs1, vd)) = {
  // operands are input with bits reversed in each byte
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {

    eg_len = (vl/EGS)
    eg_start = (vstart/EGS)

    foreach (i from eg_start to eg_len-1) {
      let Y = brev8(get_velem(vd,EGW=128,i)); // Multiplier
      let H = brev8(get_velem(vs2,EGW=128,i)); // Multiplicand
      let Z : bits(128) = 0;

      for (int bit = 0; bit < 128; bit++) {
        if bit_to_bool(Y[bit])
          Z ^= H

        bool reduce = bit_to_bool(H[127]);
        H = H << 1; // left shift H by 1
        if (reduce)
          H ^= 0x87; // Reduce using  $x^7 + x^2 + x^1 + 1$  polynomial
      }

      let result = brev8(Z);
      set_velem(vd, EGW=128, i, result);
    }
    RETIRE_SUCCESS
  }
}
```

包含于:

Zvkg, Zvkng, Zvksg

33.3.18 vrev8.v

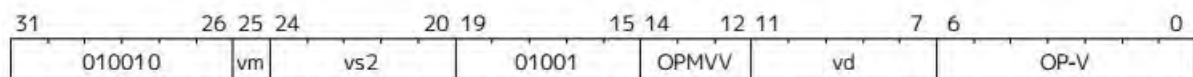
简述:

向量反转字节

助记符:

vrev8.v vd, vs2, vm

编码 (向量):



参数:

寄存器	方向	描述
Vs2	输入	输入元素
Vd	输出	有效地执行字节序交换

描述:

对 vs2 的每个元素执行字节反转，有效地执行字节序交换。



此元素级字节序交换对于包括 SHA2 和 SM3 在内的几种加密算法是必需的。

操作:

```
function clause execute (VREV8(vs2)) = {
  foreach (i from vstart to vl-1) {
    input = get_velem(vs2, SEW, i);
    let output : SEW = 0;
    let j = SEW - 1;
    foreach (k from 0 to (SEW - 8) by 8) {
      output[k..(k + 7)] = input[(j - 7)..j];
      j = j - 8;
    }
    set_velem(vd, SEW, i, output)
  }
  RETIRE_SUCCESS
}
```

包含于:

Zvbb, Zvkb, Zvkn, Zvknc, Zvkng, Zvks Zvksc, Zvksg

33.3.19 vrol. [vv, vx]

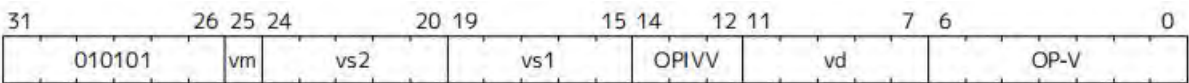
简述:

向量按向量/标量左旋

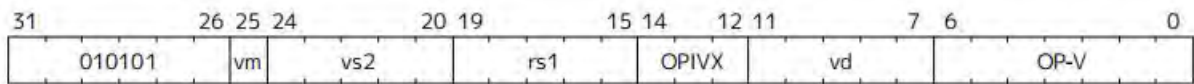
助记符:

```
vrol.vv vd, vs2, vs1, vm +
vrol.vx vd, vs2, rs1, vm +
```

编码 (向量-向量):



编码 (向量-标量):



向量-向量参数:

寄存器	方向	描述
Vs1	输入	旋转数目
Vs2	输入	数据
Vd	输出	旋转数据

向量-标量参数:

寄存器	方向	描述
Rs1	输入	旋转数目
Vs2	输入	数据
Vd	输出	旋转数据

描述:

对 vs2 的每个元素执行按位左旋。

vs2 中的元素按 vs1 (向量-向量) 或整数寄存器 rs1 (向量-标量) 指定的旋转量左旋。仅使用旋转量值的低 $\log_2(\text{SEW})$ 位, 忽略所有其他位。



此指令没有立即数形式 (即 `vrol.vi`) , 因为可以通过取反旋转量并使用右旋指令的立即数形式 (即 `vror.vi`) 实现相同的功能。

操作:

```
function clause execute (VROL_VV(vs2, vs1, vd)) = {
  foreach (i from vstart to vl - 1) {
    set_velem(vd, EEW=SEW, i,
      get_velem(vs2, i) <<< (get_velem(vs1, i) & (SEW-1))
    )
  }
  RETIRE_SUCCESS
}

function clause execute (VROL_VX(vs2, rs1, vd)) = {
  foreach (i from vstart to vl - 1) {
    set_velem(vd, EEW=SEW, i,
      get_velem(vs2, i) <<< (X(rs1) & (SEW-1))
    )
  }
  RETIRE_SUCCESS
}
```

包含于:

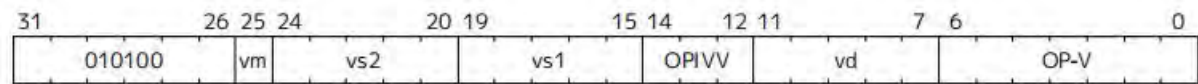
Zvbb, Zvkb, Zvkn, Zvknc, Zvkng, Zvks Zvksc, Zvksg

33.3.20 vror. [vv, vx, vi]

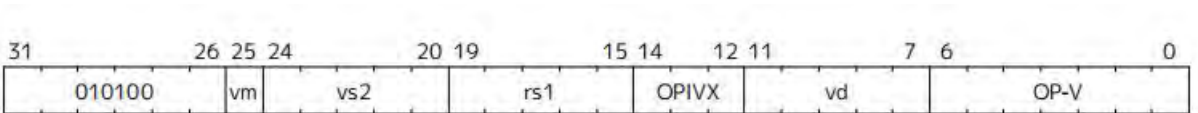
简述：
向量按向量/标量/立即数右旋

助记符：
vror.vv vd, vs2, vs1, vm +
vror.vx vd, vs2, rs1, vm +
vror.vi vd, vs2, uimm, vm

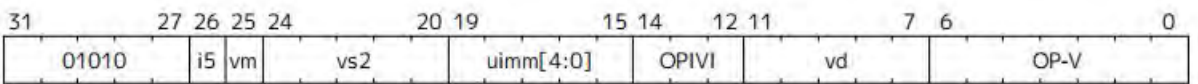
编码（向量-向量）：



编码（向量-标量）：



编码（向量-立即数）：



向量-向量参数：

寄存器	方向	描述
Vs1	输入	旋转数目
Vs2	输入	数据
Vd	输出	旋转数据

向量-标量/立即数参数：

寄存器	方向	描述
Rs1/imm	输入	旋转数目
Vs2	输入	数据
Vd	输出	旋转数据

描述：
对 vs2 的每个元素执行按位右旋。
vs2 中的元素按 vs1（向量-向量）、整数寄存器 rs1（向量-标量）或立即数值（向量-立即数）指定的旋转量右旋。仅使用旋转量值的低 log2(SEW) 位，忽略所有其他位。

操作:

```
function clause execute (VROR_VV(vs2, vs1, vd)) = {
    foreach (i from vstart to vl - 1) {
        set_velem(vd, EEW=SEW, i,
            get_velem(vs2, i) >>> (get_velem(vs1, i) & (SEW-1))
        )
    }
    RETIRE_SUCCESS
}

function clause execute (VROR_VX(vs2, rs1, vd)) = {
    foreach (i from vstart to vl - 1) {
        set_velem(vd, EEW=SEW, i,
            get_velem(vs2, i) >>> (X(rs1) & (SEW-1))
        )
    }
    RETIRE_SUCCESS
}

function clause execute (VROR_VI(vs2, imm[5:0], vd)) = {
    foreach (i from vstart to vl - 1) {
        set_velem(vd, EEW=SEW, i,
            get_velem(vs2, i) >>> (imm[5:0] & (SEW-1))
        )
    }
    RETIRE_SUCCESS
}
```

包含于:

Zvbb, Zvkb, Zvkn, Zvknc, Zvkng, Zvks Zvksc, Zvksg

33.3.21 vsha2c[h].vv

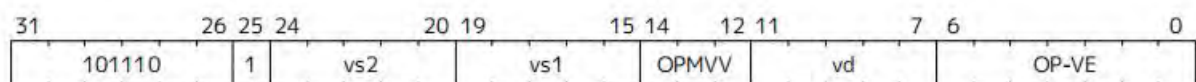
简述:

向量 SHA-2 两轮压缩

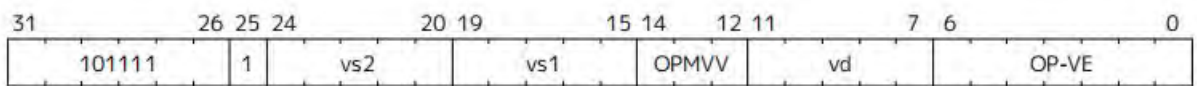
助记符:

vsha2ch.vv vd, vs2, vs1 +
vsha2cl.vv vd, vs2, vs1

编码 (向量-向量) 高部分:



编码 (向量-向量) 低部分:



- 保留编码：
- * zvknha: SEW 为 32 以外的任何值
 - * zvknhb: SEW 为 32 或 64 以外的任何值
 - * vd 寄存器组与 vs1 或 vs2 重叠

参数：

寄存器	方向	EGW	EGS	EEW	描述
Vd	输入	4×SEW	4	SEW	当前循环 {c, d, g, h}
Vs1	输入	4×SEW	4	SEW	3: 0
Vs2	输入	4×SEW	4	SEW	当前循环 {a, b, e, f}
Vd	输出	4×SEW	4	SEW	下一个循环 {a, b, e, f}

描述：

- SEW=32: 执行 2 轮 SHA-256 压缩
- SEW=64: 执行 2 轮 SHA-512 压缩

vs1 中的两个字与 vd 和 vs1 中的当前状态的 8 个字一起处理，以执行两轮哈希计算，生成下一个状态的四个字。

这些指令接收两个 SEW 字 W1 和 W0，它们是消息调度的下两个字，并加上适当的常量，以及八个 SEW 字变量：a, b, c, d, e, f, g, 和 h。输出是执行 2 轮哈希计算后的 a, b, e 和 f 的新值。新值 c, d, g, 和 h 等于输入值 a, b, e, f。

NIST 标准要求最终哈希在 SEW 大小的字内为大端字节顺序。由于此指令将所有字视为小端，软件需要在处理完所有消息块后对最终输出进行字节序交换。



vsha2ch 版本的此指令使用 vs1 中元素组的两个最高有效消息调度字，而 vsha2cl 版本使用两个最低有效消息调度字。除此之外，这些版本的指令是相同的。拥有高低版本的此指令通常在交错独立的哈希操作时提高性能（例如，同时哈希多个文件）。

防止 vd 与 vs1 或 vs2 重叠简化了 VLEN < EGW 的实现。这种限制不会对编码产生任何影响，因为算法的正确实现要求 vd、vs1 和 vs2 是不同的寄存器。

操作：

```

function clause execute (VSHA2c(vs2, vs1, vd)) = {
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {

    eg_len = (v1/EGS)
    eg_start = (vstart/EGS)

    foreach (i from eg_start to eg_len-1) {
      let {a @ b @ e @ f} : bits(4*SEW) = get_velem(vs2, 4*SEW, i);
      let {c @ d @ g @ h} : bits(4*SEW) = get_velem(vd, 4*SEW, i);
      let MessageShedPlusC[3:0] : bits(4*SEW) = get_velem(vs1, 4*SEW, i);
      let {W1, W0} == VSHA2c1 ? MessageSchedPlusC[1:0] : MessageSchedPlusC[3:2]; // 1
      vs h difference is the words selected

      let T1 : bits(SEW) = h + sum1(e) + ch(e,f,g) + W0;
      let T2 : bits(SEW) = sum0(a) + maj(a,b,c);
      h = g;
      g = f;
      f = e;
      e = d + T1;
      d = c;
      c = b;
      b = a;
      a = T1 + T2;

      T1 = h + sum1(e) + ch(e,f,g) + W1;
      T2 = sum0(a) + maj(a,b,c);
      h = g;
      g = f;
      f = e;
      e = d + T1;
      d = c;
      c = b;
      b = a;
      a = T1 + T2;
      set_velem(vd, 4*SEW, i, {a @ b @ e @ f});
    }
    RETIRE_SUCCESS
  }
}

function sum0(x) = {
  match SEW {

```

```

        32 => rotr(x,2) XOR rotr(x,13) XOR rotr(x,22),
        64 => rotr(x,28) XOR rotr(x,34) XOR rotr(x,39)
    }
}

function sum1(x) = {
    match SEW {
        32 => rotr(x,6) XOR rotr(x,11) XOR rotr(x,25),
        64 => rotr(x,14) XOR rotr(x,18) XOR rotr(x,41)
    }
}

function ch(x, y, z) = ((x & y) ^ ((~x) & z))

function maj(x, y, z) = ((x & y) ^ (x & z) ^ (y & z))

function ROTR(x,n) = (x >> n) | (x << SEW - n)

```

包含于:

Zvkn, Zvknc, Zvkng, zvknh[ab]

33.3.22 vsha2ms.vv

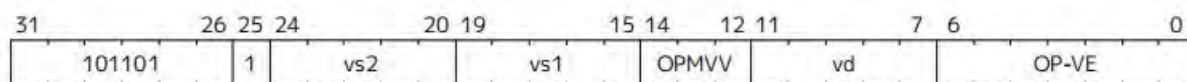
简述:

向量 SHA-2 消息调度

助记符:

vsha2ms.vv vd, vs2, vs1

编码 (向量-向量):



保留编码:

- * zvknha: SEW 为 32 以外的任何值
- * zvknhb: SEW 为 32 或 64 以外的任何值
- * vd 寄存器组与 vs1 或 vs2 重叠

参数:

寄存器	方向	EGW	EGS	EEW	描述
Vd	输入	4×SEW	4	SEW	{W[3], W[2], W[1], W[0]}
Vs1	输入	4×SEW	4	SEW	{W[11], W[10], W[9], W[4]}
Vs2	输入	4×SEW	4	SEW	{W[15], W[14],

					-, W[12]}
Vd	输出	4×SEW	4	SEW	{W[19], W[18], W[17], W[16]}

描述:

- SEW=32: 执行 4 轮 SHA-256 消息调度扩展 (zvknha 和 zvknhb)
- SEW=64: 执行 4 轮 SHA-512 消息调度扩展 (zvkhnb)

从 vd (最旧)、vs2 和 vs1 (最新) 中读取的最后 16 个 SEW 大小的消息调度字中的 11 个被处理以生成下 4 个消息调度字。消息调度的前 16 个 SEW 大小的字来自消息块，以大端字节顺序存储。由于此指令将所有字视为小端，软件需要对这些字进行字节序交换。

所有后续的消息调度字由此指令生成，因此不需要字节序交换。



软件需要将字打包成元素组，如上面的参数表所示。索引表示相对年龄，较低的索引表示较旧的字。

防止 vd 与 vs1 或 vs2 重叠简化了 VLEN < EGW 的实现。这种限制不会对编码产生任何影响，因为算法的正确实现要求 vd、vs1 和 vs2 是不同的寄存器。

操作:

```
function clause execute (VSHA2ms(vs2, vs1, vd)) = {
  // SEW32 = SHA-256
  // SEW64 = SHA-512
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {

    eg_len = (vl/EGS)
    eg_start = (vstart/EGS)

    foreach (i from eg_start to eg_len-1) {
      {W[3] @ W[2] @ W[1] @ W[0]} : bits(EGW) = get_velem(vd, EGW, i);
      {W[11] @ W[10] @ W[9] @ W[4]} : bits(EGW) = get_velem(vs2, EGW, i);
      {W[15] @ W[14] @ W[13] @ W[12]} : bits(EGW) = get_velem(vs1, EGW, i);

      W[16] = sig1(W[14]) + W[9] + sig0(W[1]) + W[0];
      W[17] = sig1(W[15]) + W[10] + sig0(W[2]) + W[1];
      W[18] = sig1(W[16]) + W[11] + sig0(W[3]) + W[2];
      W[19] = sig1(W[17]) + W[12] + sig0(W[4]) + W[3];

      set_velem(vd, EGW, i, {W[19] @ W[18] @ W[17] @ W[16]});
    }
    RETIRE_SUCCESS
  }
}
```



```

function sig0(x) = {
  match SEW {
    32 => (ROTR(x,7) XOR ROTR(x,18) XOR SHR(x,3)),
    64 => (ROTR(x,1) XOR ROTR(x,8) XOR SHR(x,7));
  }
}

function sig1(x) = {
  match SEW {
    32 => (ROTR(x,17) XOR ROTR(x,19) XOR SHR(x,10),
    64 => ROTR(x,19) XOR ROTR(x,61) XOR SHR(x,6));
  }
}

function ROTR(x,n) = (x >> n) | (x << SEW - n)
function SHR (x,n) = x >> n

```

包含于：

Zvkn, Zvknc, Zvkng, zvknh[ab]

33.3.23 vsm3c.vi

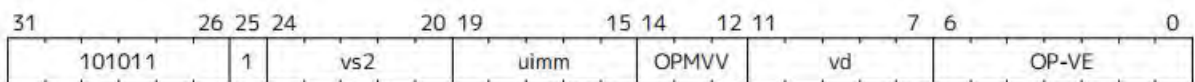
简述：

向量 SM3 压缩

助记符：

vsm3c.vi vd, vs2, uimm

编码：



保留编码：

- * SEW 为除 32 以外的任何值
- * vd 寄存器组与 vs2 寄存器组重叠

参数：

寄存器	方向	EGW	EGS	EEW	描述
Vd	输入	256	8	32	当前状态
Uimm	输入	-	-	-	循环数字
Vs2	输入	256	8	32	W[5], W[4], W[1], W[0]
Vd	输出	256	8	32	下一状态

描述:

执行两轮 SM3 压缩。

当前状态的八个 32 位字作为一个元素组从 `vd` 中读取。八个 32 位消息字作为一个元素组从 `vs2` 中读取,但只使用其中的四个。所有 32 位输入字从大端字节序转换为小端字节序。这些输入根据轮次组(由 `rnds` 指定)进行处理,并生成下一状态作为八个 32 位字的元素组。生成的八个 32 位字的下一状态从小端字节序转换为大端字节序,并返回到一个八元素组中。

轮次编号由 5 位 `rnds` 无符号立即数提供。合法值为 0–31,表示正在执行的两轮组。例如,如果 `rnds=1`,则执行第 2 和第 3 轮。

轮次编号用于常量的旋转,并告知行为在第 0–15 轮和第 16–63 轮之间的差异。



输入和输出字节的字节序交换使我们能够与 SM3 规范对齐,而无需软件执行这些交换。

防止 `vd` 和 `vs2` 重叠简化了 $VLEN < EGW$ 的实现。此限制不会对编码产生任何影响,因为算法的正确实现要求 `vd` 和 `vs2` 是不同的寄存器。

操作:

```
function clause execute (VSM3C(rnds, vs2, vd)) = {
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {

    eg_len = (v1/EGS)
    eg_start = (vstart/EGS)

    foreach (i from eg_start to eg_len-1) {

      // load state
      let {Hi @ Gi @ Fi @ Ei @ Di @ Ci @ Bi @ Ai} : bits(256) : bits(256) = (get_velem(vd,
256, i));
      //load message schedule
      let {u_w7 @ u_w6 @ w5i @ w4i @ u_w3 @ u_w2 @ w1i @ w0i} : bits(256) =
(get_velem(vs2, 256, i));
      // u_w inputs are unused

      // perform endian swap
      let H : bits(32) = rev8(Hi);
      let G : bits(32) = rev8(Gi);
      let F : bits(32) = rev8(Fi);
      let E : bits(32) = rev8(Ei);
      let D : bits(32) = rev8(Di);
      let C : bits(32) = rev8(Ci);
      let B : bits(32) = rev8(Bi);
      let A : bits(32) = rev8(Ai);
```

```

let w5 = : bits(32) rev8(w5i);
let w4 = : bits(32) rev8(w4i);
let w1 = : bits(32) rev8(w1i);
let w0 = : bits(32) rev8(w0i);

let x0 :bits(32) = w0 ^ w4; // W'[0]
let x1 :bits(32) = w1 ^ w5; // W'[1]

let j = 2 * rnds;
let ss1 : bits(32) = ROL32(ROL32(A, 12) + E + ROL32(T_j(j), j % 32), 7);
let ss2 : bits(32) = ss1 ^ ROL32(A, 12);
let tt1 : bits(32) = FF_j(A, B, C, j) + D + ss2 + x0;
let tt2 : bits(32) = GG_j(E, F, G, j) + H + ss1 + w0;
D = C;
let : bits(32) C1 = ROL32(B, 9);
B = A;
let A1 : bits(32) = tt1;
H = G;
let G1 : bits(32) = ROL32(F, 19);
F = E;
let E1 : bits(32) = P_0(tt2);

j = 2 * rnds + 1;
ss1 = ROL32(ROL32(A1, 12) + E1 + ROL32(T_j(j), j % 32), 7);
ss2 = ss1 ^ ROL32(A1, 12);

```

```

tt1 = FF_j(A1, B, C1, j) + D + ss2 + x1;
tt2 = GG_j(E1, F, G1, j) + H + ss1 + w1;
D = C1;
let C2 : bits(32) = ROL32(B, 9);
B = A1;
let A2 : bits(32) = tt1;
H = G1;
let G2 = : bits(32) ROL32(F, 19);
F = E1;
let E2 = : bits(32) P_0(tt2);

// Update the destination register - swap back to big endian
let result : bits(256) = {rev8(G1) @ rev8(G2) @ rev8(E1) @ rev8(E2) @ rev8(C1) @
rev8(C2) @ rev8(A1) @ rev8(A2)};
set_velem(vd, 256, i, result);
}

RETIRE_SUCCESS
}
}

function FF1(X, Y, Z) = ((X) ^ (Y) ^ (Z))
function FF2(X, Y, Z) = (((X) & (Y)) | ((X) & (Z)) | ((Y) & (Z)))

function FF_j(X, Y, Z, J) = (((J) <= 15) ? FF1(X, Y, Z) : FF2(X, Y, Z))

function GG1(X, Y, Z) = ((X) ^ (Y) ^ (Z))
function GG2(X, Y, Z) = (((X) & (Y)) | ((~(X)) & (Z)))
.
function GG_j(X, Y, Z, J) = (((J) <= 15) ? GG1(X, Y, Z) : GG2(X, Y, Z))

function T_j(J) = (((J) <= 15) ? (0x79CC4519) : (0x7A879D8A))

function P_0(X) = ((X) ^ ROL32((X), 9) ^ ROL32((X), 17))

```

包含于:

Zvks, Zvksc, Zvksg, Zvksh

33.3.24 vsm3me.vv

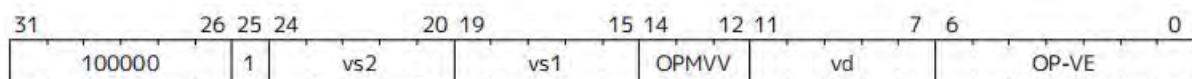
简述:

向量 SM3 消息扩展

助记符:

vsm3me.vv vd, vs2, vs1

编码:



保

留编码：

- * SEW 为除 32 以外的任何值
- * vd 寄存器组与 vs2 寄存器组重叠。

参数：

寄存器	方向	EGW	EGS	EEW	描述
Vs1	输入	256	8	32	W[7:0]
Vs2	输入	256	8	32	W[15:8]
Vd	输出	256	8	32	W[23:16]

描述：

执行八轮 SM3 消息扩展。

最近的十六个 32 位消息字作为两个八元素组从 vs1 和 vs2 中读取。每个字从大端字节序转换为小端字节序。生成的下一个八个 32 位消息字从小端字节序转换为大端字节序，并返回到一个八元素组中。



输入和输出字节的字节序交换使我们能够与 SM3 规范对齐，而无需软件执行这些交换。

防止 vd 和 vs2 重叠简化了 $VLEN < EGW$ 的实现。此限制不应应对编码产生任何影响，因为算法要求这些值保留以生成下 8 个字。

操作：

```
function clause execute (VSM3ME(vs2, vs1)) = {
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {

    eg_len = (vl/EGS)
    eg_start = (vstart/EGS)

    foreach (i from eg_start to eg_len-1) {
      let w[7:0] : bits(256) = get_velem(vs1, 256, i);
```



```

let w[15:8] : bits(256) = get_velem(vs2, 256, i);

// Byte Swap inputs from big-endian to little-endian
let w15 = rev8(w[15]);
let w14 = rev8(w[14]);
let w13 = rev8(w[13]);
let w12 = rev8(w[12]);
let w11 = rev8(w[11]);
let w10 = rev8(w[10]);
let w9 = rev8(w[9]);
let w8 = rev8(w[8]);
let w7 = rev8(w[7]);
let w6 = rev8(w[6]);
let w5 = rev8(w[5]);
let w4 = rev8(w[4]);
let w3 = rev8(w[3]);
let w2 = rev8(w[2]);
let w1 = rev8(w[1]);
let w0 = rev8(w[0]);

let w[16] = ZVKSH_W(w0 @ w7 @ w13 @ w3 @ w10);
let w[17] = ZVKSH_W(w1 @ w8 @ w14 @ w4 @ w11);
let w[18] = ZVKSH_W(w2 @ w9 @ w15 @ w5 @ w12);
let w[19] = ZVKSH_W(w3 @ w10 @ w16 @ w6 @ w13);
let w[20] = ZVKSH_W(w4 @ w11 @ w17 @ w7 @ w14);
let w[21] = ZVKSH_W(w5 @ w12 @ w18 @ w8 @ w15);
let w[22] = ZVKSH_W(w6 @ w13 @ w19 @ w9 @ w16);
let w[23] = ZVKSH_W(w7 @ w14 @ w20 @ w10 @ w17);

set_velem(vd, 256, i, {w23 @ w22 @ w21 @ w20 @ w19 @ w18 @ w17 @ w16});
}
RETIRE_SUCCESS
}
}

function P_1(X) ((X) ^ ROL32((X), 15) ^ ROL32((X), 23))

function ZVKSH_W(M16, M9, M3, M13, M6) = \
(P1( (M16) ^ (M9) ^ ROL32((M3), 15) ) ^ ROL32((M13), 7) ^ (M6))

```

包含于:

Zvks, Zvksc, Zvksg, Zvksh

33.3.25 vsm4k.vi

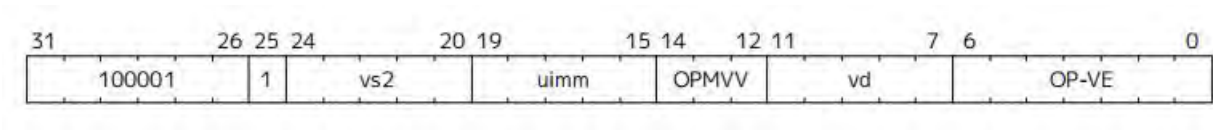
简述:

向量 SM4 密钥扩展

助记符:

vsm4k.vi vd, vs2, uimm

编码:



保留编码:

* SEW 为除 32 以外的任何值

参数:

寄存器	方向	EGW	EGS	EEW	描述
uimm	输入	-	-	-	循环状态
Vs2	输入	128	4	32	Rk[0:3]
Vd	输出	128	4	32	rX[0:3]

描述:

执行四轮 SM4 密钥扩展。

四轮密钥作为一个 4 元素组从 vs2 中读取。通过迭代地将最后三个轮密钥与一个由轮次组编号索引的常量进行异或，执行字节替换，然后在此值的旋转版本与相应的当前轮密钥之间执行异或，生成每个下一轮密钥。

轮次组编号 (rnd) 来自 uimm[2:0]；忽略 uimm[4:3] 中的位。轮次组编号范围为 0 到 7，表示正在生成的四轮密钥组。轮密钥范围为 0-31。例如，如果 rnd=1，则生成第 4、5、6 和 7 轮密钥。



软件需要生成初始轮密钥。这是通过将 128 位加密密钥与系统参数 FK[0:3] 进行异或来完成的。

实现提示:

轮常量 (CK) 可以动态生成,成本相对较低。如果常量的字节被分配一个从 0 到 127 的递增索引,则每个字节的值等于其索引乘以 7 模 256。由于结果都限制在 8 位内,模运算是免费的:

$$\begin{aligned} B[n] &= n + 2n + 4n; \\ &= 8n + \sim n + 1; \end{aligned}$$

操作:


```

function clause execute (vsm4k(uimm, vs2)) = {
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception
    RETIRE_FAIL
  } else {

    eg_len = (v1/EGS)
    eg_start = (vstart/EGS)

    let B : bits(32) = 0;
    let S : bits(32) = 0;
    let rk4 : bits(32) = 0;
    let rk5 : bits(32) = 0;
    let rk6 : bits(32) = 0;
    let rk7 : bits(32) = 0;
    let rnd : bits(3) = uimm[2:0]; // Lower 3 bits

    foreach (i from eg_start to eg_len-1) {
      let (rk3 @ rk2 @ rk1 @ rk0) : bits(128) = get_velem(vs2, 128, i);

      B = rk1 ^ rk2 ^ rk3 ^ ck(4 * rnd);
      S = sm4_subword(B);
      rk4 = ROUND_KEY(rk0, S);

      B = rk2 ^ rk3 ^ rk4 ^ ck(4 * rnd + 1);
      S = sm4_subword(B);
      rk5 = ROUND_KEY(rk1, S);

      B = rk3 ^ rk4 ^ rk5 ^ ck(4 * rnd + 2);
      S = sm4_subword(B);
      rk6 = ROUND_KEY(rk2, S);

      B = rk4 ^ rk5 ^ rk6 ^ ck(4 * rnd + 3);
      S = sm4_subword(B);
      rk7 = ROUND_KEY(rk3, S);

      // Update the destination register.
      set_velem(vd, EGW=128, i, (rk7 @ rk6 @ rk5 @ rk4));
    }
    RETIRE_SUCCESS
  }
}

```

```
let ck : list(bits(32)) = [  
    0x00070E15, 0x1C232A31, 0x383F464D, 0x545B6269,  
    0x70777E85, 0x8C939AA1, 0xA8AFB6BD, 0xC4CBD2D9,  
    0xE0E7EEF5, 0xFC030A11, 0x181F262D, 0x343B4249,  
    0x50575E65, 0x6C737A81, 0x888F969D, 0xA4ABB2B9,  
    0xC0C7CED5, 0xDCE3EAF1, 0xF8FF060D, 0x141B2229,  
    0x30373E45, 0x4C535A61, 0x686F767D, 0x848B9299,  
    0xA0A7AEB5, 0xBCC3CAD1, 0xD8DFE6ED, 0xF4FB0209,  
    0x10171E25, 0x2C333A41, 0x484F565D, 0x646B7279  
    ]  
};
```

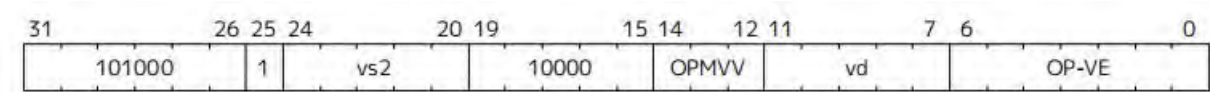
包含于：
Zvks, Zvksc, Zvk sed, Zvksg

33.3.26 vsm4r. [vv,vs]

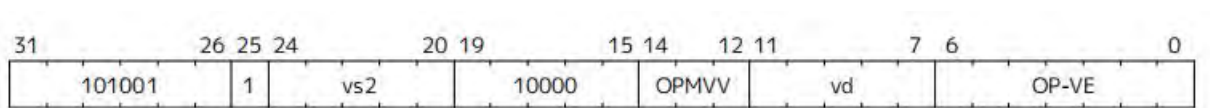
简述：
向量 SM4 轮次

助记符：
vsm4r.vv vd, vs2 +
vsm4r.vs vd, vs2

编码（向量-向量）：



编码（向量-标量）：



保

- 留编码：
- * SEW 为除 32 以外的任何值
 - * 仅适用于 .vs 形式：vd 寄存器组与 vs2 寄存器组重叠

参数：

寄存器	方向	EGW	EGS	EEW	描述
Vd	输入	128	4	32	x[0:3]
Vs2	输入	128	4	32	Rk[0:3]
Vd	输出	128	4	32	X[0:3]

描述：
执行四轮 SM4 加密/解密。

当前状态的四个字作为一个 4 元素组从 `vd` 中读取，轮密钥从 `vs2` 中的相应 4 元素组（向量-向量形式）或 `vs2` 中的标量元素组（向量-标量形式）读取。通过迭代地将状态的最后三个字与相应的轮密钥进行异或，执行字节替换，然后在此值的旋转版本与相应的当前状态之间执行异或，生成下一状态的四个字。

在 SM4 中，加密和解密是相同的，除了解密以相反的顺序消耗轮密钥。



在加密的前四轮中，当前状态是明文。

在解密的前四轮中，当前状态是密文。

在所有后续轮次中，当前状态是前四轮的下一状态。

操作：

```
function clause execute (VSM4R(vd, vs2)) = {
  if(LMUL*VLEN < EGW) then {
    handle_illegal(); // illegal instruction exception

    RETIRE_FAIL
  } else {

    eg_len = (v1/EGS)
    eg_start = (vstart/EGS)

    let B : bits(32) = 0;
    let S : bits(32) = 0;
    let rk0 : bits(32) = 0;
    let rk1 : bits(32) = 0;
    let rk2 : bits(32) = 0;
    let rk3 : bits(32) = 0;
    let x0 : bits(32) = 0;
    let x1 : bits(32) = 0;
    let x2 : bits(32) = 0;
    let x3 : bits(32) = 0;
    let x4 : bits(32) = 0;
    let x5 : bits(32) = 0;
    let x6 : bits(32) = 0;
    let x7 : bits(32) = 0;

    let keyelem : bits(32) = 0;
```

```

foreach (i from eg_start to eg_len-1) {
  keyelem = if suffix == "vv" then i else 0;
  {rk3 @ rk2 @ rk1 @ rk0} : bits(128) = get_velem(vs2, EGW=128, keyelem);
  {x3 @ x2 @ x1 @ x0} : bits(128) = get_velem(vd, EGW=128, i);

  B = x1 ^ x2 ^ x3 ^ rk0;
  S = sm4_subword(B);
  x4 = sm4_round(x0, S);

  B = x2 ^ x3 ^ x4 ^ rk1;
  S = sm4_subword(B);
  x5 = sm4_round(x1, S);

  B = x3 ^ x4 ^ x5 ^ rk2;
  S = sm4_subword(B);
  x6 = sm4_round(x2, S);

  B = x4 ^ x5 ^ x6 ^ rk3;
  S = sm4_subword(B);
  x7 = sm4_round(x3, S);

  set_velem(vd, EGW=128, i, (x7 @ x6 @ x5 @ x4));
}
RETIRE_SUCCESS
}
}

val sm4_round : bits(32) -> bits(32)
function sm4_round(X, S) = \
  ((X) ^ ((S) ^ ROL32((S), 2) ^ ROL32((S), 10) ^ ROL32((S), 18) ^ ROL32((S), 24)))

```

包含于：

Zvks, Zvksc, Zvk sed, Zvksg

33.3.27 vwsll. [vv, vx, vi]

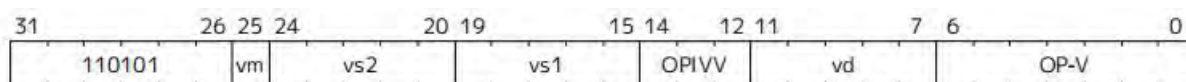
简述：

向量宽移位左逻辑（按向量/标量/立即数）。

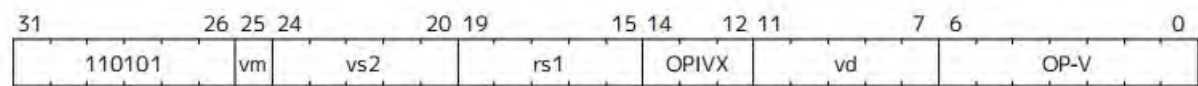
助记符：

vwsll.vv vd, vs2, vs1, vm +
 vwsll.vx vd, vs2, rs1, vm +
 vwsll.vi vd, vs2, uimm, vm

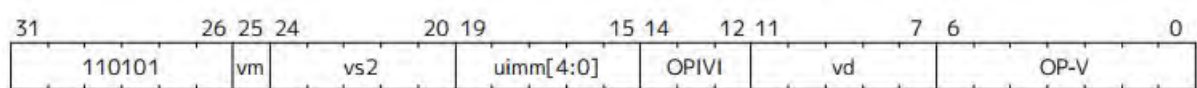
编码（向量-向量）：



编码（向量-标量）：



编码（向量-立即数）：



向量-向量参数：

寄存器	方向	描述
Vs1	输入	转移数目
Vs2	输入	数据
Vd	输出	转移数据

向量-标量/立即数参数：

寄存器	方向	描述
Rs1/imm	输入	转移数目
Vs2	输入	数据
Vd	输出	转移数据

描述：

对 vs2 的每个元素执行宽移位左逻辑。

vs2 中的元素被零扩展到 2*SEW 位，然后按由 vs1 的相应元素（向量-向量）、整数寄存器 rs1（向量-标量）或立即数值（向量-立即数）指定的移位量左移。仅使用移位量值的低 log2(2*SEW) 位，忽略所有其他位。

操作：

```
function clause execute (VWSLL_VV(vs2, vs1, vd)) = {
```

```

    foreach (i from vstart to vl - 1) {
        set_velem(vd, EEW=2*SEW, i,
            get_velem(vs2, i) << (get_velem(vs1, i) & ((2*SEW)-1))
        )
    }
    RETIRE_SUCCESS
}

function clause execute (VWSLL_VX(vs2, rs1, vd)) = {
    foreach (i from vstart to vl - 1) {
        set_velem(vd, EEW=2*SEW, i,
            get_velem(vs2, i) << (X(rs1) & ((2*SEW)-1))
        )
    }
    RETIRE_SUCCESS
}

function clause execute (VWSLL_VI(vs2, uimm[4:0], vd)) = {
    foreach (i from vstart to vl - 1) {
        set_velem(vd, EEW=2*SEW, i,
            get_velem(vs2, i) << (uimm[4:0] & ((2*SEW)-1))
        )
    }
    RETIRE_SUCCESS
}

```

包含于：

Zvbb

33.4 向量加密指令

向量加密指令，除 Zvbb 和 Zvbc

Integer					Integer					FP				
funct3					funct3					funct3				
OPIVV	V				OPMVV	V				OPFVV	V			
OPIVX		X			OPMVX		X			OPFVF		F		
OPIVI			I											

funct6					funct6					funct6				
100000					100000	V	vsm3me			100000				
100001					100001	V	vsm4k.vi			100001				
100010					100010	V	vaeskf1.vi			100010				
100011					100011					100011				
100100					100100					100100				
100101					100101					100101				
100110					100110					100110				
100111					100111					100111				
101000					101000	V	VAES.vv			101000				
101001					101001	V	VAES.vs			101001				
101010					101010	V	vaeskf2.vi			101010				
101011					101011	V	vsm3c.vi			101011				
101100					101100	V	vghsh			101100				
101101					101101	V	vsha2ms			101101				
101110					101110	V	vsha2ch			101110				
101111					101111	V	vsha2cl			101111				

33.5 向量位操作和无进位乘法指令

Zvbb*、*Zvkb* 和 *Zvbc* 向量指令 *加粗

Integer					Integer					FP				
funct3					funct3					funct3				
OPIVV	V				OPMVV	V				OPFVV	V			
OPIVX		X			OPMVX		X			OPFVF		F		
OPIVI			I											

funct6					funct6					funct6			
000000	V	X	I	vadd	000000	V		vredsum	000000	V	F	vfadd	
000001	V	X		vandn	000001	V		vredand	000001	V		vfredusum	
000010	V	X		vsub	000010	V		vredor	000010	V	F	vfsb	
000011		X	I	vrsub	000011	V		vredxor	000011	V		vfredosum	
000100	V	X		vminu	000100	V		vredminu	000100	V	F	vfmmin	
000101	V	X		vmin	000101	V		vredmin	000101	V		vfredmin	
000110	V	X		vmaxu	000110	V		vredmaxu	000110	V	F	vfmmax	
000111	V	X		vmax	000111	V		vredmax	000111	V		vfredmax	
001000					001000	V	X	vaaddu	001000	V	F	vfsgnj	
001001	V	X	I	vand	001001	V	X	vaadd	001001	V	F	vfsgnjn	
001010	V	X	I	vor	001010	V	X	vasubu	001010	V	F	vfsgnjx	
001011	V	X	I	vxor	001011	V	X	vasub	001011				
001100	V	X	I	vrgather	001100	V	X	vclmul	001100				
001101					001101	V	X	vclmulh	001101				
001110		X	I	vslideup	001110		X	vslidelup	001110		F	vfslidelup	
001111	V			vrgathereil6									
001111		X	I	vslidedown	001111		X	vslidedown	001111		F	vfslidedown	

funct6					funct6					funct6			
010011	V	X		vmsbc	010011				010011	V		VFUNARYI	
010100	V	X		vrwr	010100	V		VMUNARYO	010100				
010101	V	X		vrol	010101				010101				
01010x			I	vrwr									
010110					010110				010110				
010111	V	X	I	vmerge/vmv	010111	V		vcompress	010111		F	vfmmerge/vfmv	
011000	V	X	I	vmseq	011000	V		vmandn	011000	V	F	vmfeq	
011001	V	X	I	vmsne	011001	V		vmand	011001	V	F	vmfle	
011010	V	X		vmsltu	011010	V		vmor	011010				
011011	V	X		vmslt	011011	V		vmxor	011011	V	F	vmflt	
011100	V	X	I	vmsleu	011100	V		vmorn	011100	V	F	vmfne	
011101	V	X	I	vmsle	011101	V		vmnand	011101		F	vmfgt	
011110		X	I	vmsgtu	011110	V		vmnor	011110				
011111		X	I	vmsgt	011111	V		vmxnor	011111		F	vmfge	

funct6					funct6					funct6				
100000	V	X	I	vsaddu	100000	V	X	vdivu		100000	V	F	vfddiv	
100001	V	X	I	vsadd	100001	V	X	vdiv		100001		F	vfrdiv	
100010	V	X		vssubu	100010	V	X	vremu		100010				
100011	V	X		vssub	100011	V	X	vrem		100011				
100100					100100	V	X	vmulhu		100100	V	F	vfmul	
100101	V	X	I	vsll	100101	V	X	vmul		100101				
100110					100110	V	X	vmulhsu		100110				
100111	V	X		vsmul	100111	V	X	vmulh		100111		F	vfrsub	
			I	vmy<nr>r										
101000	V	X	I	vsrl	101000					101000	V	F	vfmadd	
101001	V	X	I	vsra	101001	V	X	vmadd		101001	V	F	vfnmadd	
101010	V	X	I	vssrl	101010					101010	V	F	vfmsub	
101011	V	X	I	vssra	101011	V	X	vnmsub		101011	V	F	vfnmsub	
101100	V	X	I	vnsrl	101100					101100	V	F	vfmacc	
101101	V	X	I	vnsra	101101	V	X	vmacc		101101	V	F	vfnmacc	
101110	V	X	I	vnclipu	101110					101110	V	F	vfmsac	
101111	V	X	I	vnclip	101111	V	X	vnmsac		101111	V	F	vfnmsac	

funct6					funct6					funct6				
110000	V			vwredsumu	110000	V	X	vwaddu		110000	V	F	vfwadd	
110001	V			vwredsum	110001	V	X	vwadd		110001	V		vfwredusum	
110010					110010	V	X	vwsubu		110010	V	F	vfwsub	
110011					110011	V	X	vwsu		110011	V		vfwredosum	
110100					110100	V	X	vwaddu.w		110100	V	F	vfwadd.w	
110101	V	X	I	vwsll	110101	V	X	vwadd.w		110101				
110110					110110	V	X	vwsu.w		110110	V	F	vfwsub.w	
110111					110111	V	X	vwsu.w		110111				

funct6					funct6					funct6				
111000					111000	V	X	vwmulu		111000	V	F	vfwmul	
111001					111001					111001				
111010					111010	V	X	vwmulsu		111010				
111011					111011	V	X	vwmul		111011				
111100					111100	V	X	vwmaccu		111100	V	F	vfwmacc	
111101					111101	V	X	vwmacc		111101	V	F	vfnwmacc	
111110					111110		X	vwmaccus		111110	V	F	vfwmsac	
111111					111111	V	X	vwmaccsu		111111	V	F	vfnwmsac	

33.6 支持的 Sail 代码

本节包含规范中指令描述所引用的支持 Sail 代码。建议阅读 Sail 手册以便更好地理解支持代码。

```

/* 执行 GF 乘法的辅助函数 */
val xt2 : bits(8) -> bits(8)
function xt2(x) = {
  (x 1) ^ (if bit_to_bool(x[7]) then 0x1b else 0x00)
}

val xt3 : bits(8) -> bits(8)
function xt3(x) = x ^ xt2(x)

/* 在 AES MixCols 步骤中将 8 位字段元素乘以 4 位值 */
val gfmul : (bits(8), bits(4)) -> bits(8)
function gfmul( x, y) = {
  (if bit_to_bool(y[0]) then          x      else 0x00) ^
  (if bit_to_bool(y[1]) then xt2(      x)      else 0x00) ^
  (if bit_to_bool(y[2]) then xt2(xt2(    x))    else 0x00) ^
  (if bit_to_bool(y[3]) then xt2(xt2(xt2(x)))   else 0x00)
}

/* 8 位到 32 位的部分 AES Mix Column - 正向 */
val aes_mixcolumn_byte_fwd : bits(8) -> bits(32)
function aes_mixcolumn_byte_fwd(so) = {
  gfmul(so, 0x3) @ so @ so @ gfmul(so, 0x2)
}

/* 8 位到 32 位的部分 AES Mix Column - 逆向 */
val aes_mixcolumn_byte_inv : bits(8) -> bits(32)
function aes_mixcolumn_byte_inv(so) = {
  gfmul(so, 0xb) @ gfmul(so, 0xd) @ gfmul(so, 0x9) @ gfmul(so, 0xe)
}

/* 32 位到 32 位的 AES 正向 MixColumn */
val aes_mixcolumn_fwd : bits(32) -> bits(32)
function aes_mixcolumn_fwd(x) = {
  let s0 : bits (8) = x[ 7.. 0];
  let s1 : bits (8) = x[15.. 8];
  let s2 : bits (8) = x[23..16];
  let s3 : bits (8) = x[31..24];
  let b0 : bits (8) = xt2(s0) ^ xt3(s1) ^      (s2) ^      (s3);
  let b1 : bits (8) =      (s0) ^ xt2(s1) ^ xt3(s2) ^      (s3);
  let b2 : bits (8) =      (s0) ^      (s1) ^ xt2(s2) ^ xt3(s3);
  let b3 : bits (8) = xt3(s0) ^      (s1) ^      (s2) ^ xt2(s3);
  b3 @ b2 @ b1 @ b0 /* 返回值 */
}

```

```

/* 32 位到 32 位的 AES 逆向 MixColumn */
val aes_mixcolumn_inv : bits(32) -> bits(32)
function aes_mixcolumn_inv(x) = {
  let s0 : bits (8) = x[ 7.. 0];
  let s1 : bits (8) = x[15.. 8];
  let s2 : bits (8) = x[23..16];
  let s3 : bits (8) = x[31..24];
  let b0 : bits (8) = gfmul(s0, 0xE) ^ gfmul(s1, 0xB) ^ gfmul(s2, 0xD) ^ gfmul(s3, 0x9);
  let b1 : bits (8) = gfmul(s0, 0x9) ^ gfmul(s1, 0xE) ^ gfmul(s2, 0xB) ^ gfmul(s3, 0xD);
  let b2 : bits (8) = gfmul(s0, 0xD) ^ gfmul(s1, 0x9) ^ gfmul(s2, 0xE) ^ gfmul(s3, 0xB);
  let b3 : bits (8) = gfmul(s0, 0xB) ^ gfmul(s1, 0xD) ^ gfmul(s2, 0x9) ^ gfmul(s3, 0xE);
  b3 @ b2 @ b1 @ b0 /* 返回值 */
}

val aes_decode_rcon : bits(4) -> bits(32)
function aes_decode_rcon(r) = {
  match r {
    0x0 => 0x00000001,
    0x1 => 0x00000002,
    0x2 => 0x00000004,
    0x3 => 0x00000008,
    0x4 => 0x00000010,
    0x5 => 0x00000020,
    0x6 => 0x00000040,
    0x7 => 0x00000080,
    0x8 => 0x0000001b,
    0x9 => 0x00000036,
    0xA => 0x00000000,
    0xB => 0x00000000,
    0xC => 0x00000000,
    0xD => 0x00000000,
    0xE => 0x00000000,
    0xF => 0x00000000
  }
}

/* SM4 SBox - 只有一个 SBox 用于正向和逆向 */
let sm4_sbox_table : list(bits(8)) = [
  0xD6, 0x90, 0xE9, 0xFE, 0xCC, 0xE1, 0x3D, 0xB7, 0x16, 0xB6, 0x14, 0xC2, 0x28,
  0xFB, 0x2C, 0x05, 0x2B, 0x67, 0x9A, 0x76, 0x2A, 0xBE, 0x04, 0xC3, 0xAA, 0x44,
  0x13, 0x26, 0x49, 0x86, 0x06, 0x99, 0x9C, 0x42, 0x50, 0xF4, 0x91, 0xEF, 0x98,
  0x7A, 0x33, 0x54, 0x0B, 0x43, 0xED, 0xCF, 0xAC, 0x62, 0xE4, 0xB3, 0x1C, 0xA9,
  0xC9, 0x08, 0xE8, 0x95, 0x80, 0xDF, 0x94, 0xFA, 0x75, 0x8F, 0x3F, 0xA6, 0x47,
  0x07, 0xA7, 0xFC, 0xF3, 0x73, 0x17, 0xBA, 0x83, 0x59, 0x3C, 0x19, 0xE6, 0x85,
  0x4F, 0xA8, 0x68, 0x6B, 0x81, 0xB2, 0x71, 0x64, 0xDA, 0x8B, 0xF8, 0xEB, 0x0F,

```

```

0x4B, 0x70, 0x56, 0x9D, 0x35, 0x1E, 0x24, 0x0E, 0x5E, 0x63, 0x58, 0xD1, 0xA2,
0x25, 0x22, 0x7C, 0x3B, 0x01, 0x21, 0x78, 0x87, 0xD4, 0x00, 0x46, 0x57, 0x9F,
0xD3, 0x27, 0x52, 0x4C, 0x36, 0x02, 0xE7, 0xA0, 0xC4, 0xC8, 0x9E, 0xEA, 0xBF,
0x8A, 0xD2, 0x40, 0xC7, 0x38, 0xB5, 0xA3, 0xF7, 0xF2, 0xCE, 0xF9, 0x61, 0x15,
0xA1, 0xE0, 0xAE, 0x5D, 0xA4, 0x9B, 0x34, 0x1A, 0x55, 0xAD, 0x93, 0x32, 0x30,
0xF5, 0x8C, 0xB1, 0xE3, 0x1D, 0xF6, 0xE2, 0x2E, 0x82, 0x66, 0xCA, 0x60, 0xC0,
0x29, 0x23, 0xAB, 0x0D, 0x53, 0x4E, 0x6F, 0xD5, 0xDB, 0x37, 0x45, 0xDE, 0xFD,
0x8E, 0x2F, 0x03, 0xFF, 0x6A, 0x72, 0x6D, 0x6C, 0x5B, 0x51, 0x8D, 0x1B, 0xAF,
0x92, 0xBB, 0xDD, 0xBC, 0x7F, 0x11, 0xD9, 0x5C, 0x41, 0x1F, 0x10, 0x5A, 0xD8,
0x0A, 0xC1, 0x31, 0x88, 0xA5, 0xCD, 0x7B, 0xBD, 0x2D, 0x74, 0xD0, 0x12, 0xB8,
0xE5, 0xB4, 0xB0, 0x89, 0x69, 0x97, 0x4A, 0x0C, 0x96, 0x77, 0x7E, 0x65, 0xB9,
0xF1, 0x09, 0xC5, 0x6E, 0xC6, 0x84, 0x18, 0xF0, 0x7D, 0xEC, 0x3A, 0xDC, 0x4D,
0x20, 0x79, 0xEE, 0x5F, 0x3E, 0xD7, 0xCB, 0x39, 0x48
[]

```

```

let aes_sbox_fwd_table : list(bits(8)) = [
0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe,
0xd7, 0xab, 0x76, 0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4,
0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0, 0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7,
0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15, 0x04, 0xc7, 0x23, 0xc3,
0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75, 0x09,
0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3,
0x2f, 0x84, 0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe,
0x39, 0x4a, 0x4c, 0x58, 0xcf, 0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85,
0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8, 0x51, 0xa3, 0x40, 0x8f, 0x92,
0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2, 0xcd, 0x0c,
0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19,
0x73, 0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14,
0xde, 0x5e, 0x0b, 0xdb, 0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2,
0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79, 0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5,
0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08, 0xba, 0x78, 0x25,
0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86,
0xc1, 0x1d, 0x9e, 0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e,
0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf, 0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42,
0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16
]

```

```

let aes_sbox_inv_table : list(bits(8)) = [
0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40, 0xa3, 0x9e, 0x81,
0xf3, 0xd7, 0xfb, 0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e,
0x43, 0x44, 0xc4, 0xde, 0xe9, 0xcb, 0x54, 0x7b, 0x94, 0x32, 0xa6, 0xc2, 0x23,
0x3d, 0xee, 0x4c, 0x95, 0x0b, 0x42, 0xfa, 0xc3, 0x4e, 0x08, 0x2e, 0xa1, 0x66,
0x28, 0xd9, 0x24, 0xb2, 0x76, 0x5b, 0xa2, 0x49, 0x6d, 0x8b, 0xd1, 0x25, 0x72,

```

```

0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xd4, 0xa4, 0x5c, 0xcc, 0x5d, 0x65,
0xb6, 0x92, 0x6c, 0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda, 0x5e, 0x15, 0x46,
0x57, 0xa7, 0x8d, 0x9d, 0x84, 0x90, 0xd8, 0xab, 0x00, 0x8c, 0xbc, 0xd3, 0x0a,
0xf7, 0xe4, 0x58, 0x05, 0xb8, 0xb3, 0x45, 0x06, 0xd0, 0x2c, 0x1e, 0x8f, 0xca,
0x3f, 0x0f, 0x02, 0xc1, 0xaf, 0xbd, 0x03, 0x01, 0x13, 0x8a, 0x6b, 0x3a, 0x91,
0x11, 0x41, 0x4f, 0x67, 0xdc, 0xea, 0x97, 0xf2, 0xcf, 0xce, 0xf0, 0xb4, 0xe6,
0x73, 0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2, 0xf9, 0x37, 0xe8,
0x1c, 0x75, 0xdf, 0x6e, 0x47, 0xf1, 0x1a, 0x71, 0x1d, 0x29, 0xc5, 0x89, 0x6f,
0xb7, 0x62, 0x0e, 0xaa, 0x18, 0xbe, 0x1b, 0xfc, 0x56, 0x3e, 0x4b, 0xc6, 0xd2,
0x79, 0x20, 0x9a, 0xdb, 0xc0, 0xfe, 0x78, 0xcd, 0x5a, 0xf4, 0x1f, 0xdd, 0xa8,
0x33, 0x88, 0x07, 0xc7, 0x31, 0xb1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xec, 0x5f,
0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0x0d, 0x2d, 0xe5, 0x7a, 0x9f, 0x93,
0xc9, 0x9c, 0xef, 0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb,
0xbb, 0x3c, 0x83, 0x53, 0x99, 0x61, 0x17, 0x2b, 0x04, 0x7e, 0xba, 0x77, 0xd6,
0x26, 0xe1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0c, 0x7d

```

```

/* 查找函数 - 接受一个索引和一个列表，并检索该列表的第 x 个元素。 */

```

```

val sbbox_lookup : (bits(8), list(bits(8))) -> bits(8)

```

```

function sbbox_lookup(x, table) = {
  match (x, table) {
    (0x00, t0:tn) => t0,
    ( y, t0:tn) => sbbox_lookup(x - 0x01, tn)
  }
}

```

```

/* 执行正向 AES SBox 操作的简便函数。 */

```

```

val aes_sbbox_fwd : bits(8) -> bits(8)

```

```

function aes_sbbox_fwd(x) = sbbox_lookup(x, aes_sbbox_fwd_table)

```

```

/* 执行逆向 AES SBox 操作的简便函数。 */

```

```

val aes_sbbox_inv : bits(8) -> bits(8)

```

```

function aes_sbbox_inv(x) = sbbox_lookup(x, aes_sbbox_inv_table)

```

```

/* AES SubWord 函数用于密钥扩展

```

```

* - 将正向 sbbox 应用于输入字中的每个字节。

```

```

*/

```

```

val aes_subword_fwd : bits(32) -> bits(32)

```

```

function aes_subword_fwd(x) = {
  aes_sbbox_fwd(x[31..24]) @
  aes_sbbox_fwd(x[23..16]) @
  aes_sbbox_fwd(x[15.. 8]) @
  aes_sbbox_fwd(x[ 7.. 0])
}

```

```

/* AES 逆向 SubWord 函数。
 * - 将逆向 sbox 应用于输入字中的每个字节。
 */
val aes_subword_inv : bits(32) -> bits(32)
function aes_subword_inv(x) = {
  aes_sbox_inv(x[31..24]) @
  aes_sbox_inv(x[23..16]) @
  aes_sbox_inv(x[15.. 8]) @
  aes_sbox_inv(x[ 7.. 0])
}

/* 执行 SM4 SBox 操作的简便函数。 */
val sm4_sbox : bits(8) -> bits(8)
function sm4_sbox(x) = sbox_lookup(x, sm4_sbox_table)

val aes_get_column : (bits(128), nat) -> bits(32)
function aes_get_column(state, c) = (state (to_bits(7, 32 * c)))[31..0]

/* 64 位到 64 位的函数，将 AES 正向 sbox 应用于 64 位字中的每个字节。 */
val aes_apply_fwd_sbox_to_each_byte : bits(64) -> bits(64)
function aes_apply_fwd_sbox_to_each_byte(x) = {
  aes_sbox_fwd(x[63..56]) @
  aes_sbox_fwd(x[55..48]) @
  aes_sbox_fwd(x[47..40]) @
  aes_sbox_fwd(x[39..32]) @
  aes_sbox_fwd(x[31..24]) @
  aes_sbox_fwd(x[23..16]) @
  aes_sbox_fwd(x[15.. 8]) @
  aes_sbox_fwd(x[ 7.. 0])
}

/* 64 位到 64 位的函数，将 AES 逆向 sbox 应用于 64 位字中的每个字节。 */
val aes_apply_inv_sbox_to_each_byte : bits(64) -> bits(64)
function aes_apply_inv_sbox_to_each_byte(x) = {
  aes_sbox_inv(x[63..56]) @
  aes_sbox_inv(x[55..48]) @
  aes_sbox_inv(x[47..40]) @
  aes_sbox_inv(x[39..32]) @
  aes_sbox_inv(x[31..24]) @
  aes_sbox_inv(x[23..16]) @
  aes_sbox_inv(x[15.. 8]) @
  aes_sbox_inv(x[ 7.. 0])
}

```



```

/*
 * AES 全轮变换函数。
 */

val getbyte : (bits(64), int) -> bits(8)
function getbyte(x, i) = (x to_bits(6, i * 8))[7..0]

val aes_rv64_shiftrows_fwd : (bits(64), bits(64)) -> bits(64)
function aes_rv64_shiftrows_fwd(rs2, rs1) = {
  getbyte(rs1, 3) @
  getbyte(rs2, 6) @
  getbyte(rs2, 1) @
  getbyte(rs1, 4) @
  getbyte(rs2, 7) @
  getbyte(rs2, 2) @
  getbyte(rs1, 5) @
  getbyte(rs1, 0)
}

val aes_rv64_shiftrows_inv : (bits(64), bits(64)) -> bits(64)
function aes_rv64_shiftrows_inv(rs2, rs1) = {
  getbyte(rs2, 3) @
  getbyte(rs2, 6) @
  getbyte(rs1, 1) @
  getbyte(rs1, 4) @
  getbyte(rs1, 7) @
  getbyte(rs2, 2) @
  getbyte(rs2, 5) @
  getbyte(rs1, 0)
}

/* 128 位到 128 位的 AES 正向 ShiftRows 变换实现。
 * 状态的字节 0 是输入列 0, 位 7..0。
 * 状态的字节 5 是输入列 1, 位 15..8。
 */
val aes_shift_rows_fwd : bits(128) -> bits(128)
function aes_shift_rows_fwd(x) = {
  let ic3 : bits(32) = aes_get_column(x, 3);
  let ic2 : bits(32) = aes_get_column(x, 2);
  let ic1 : bits(32) = aes_get_column(x, 1);
  let ic0 : bits(32) = aes_get_column(x, 0);
  let oc0 : bits(32) = ic3[31..24] @ ic2[23..16] @ ic1[15.. 8] @ ic0[ 7.. 0];
  let oc1 : bits(32) = ic0[31..24] @ ic3[23..16] @ ic2[15.. 8] @ ic1[ 7.. 0];
  let oc2 : bits(32) = ic1[31..24] @ ic0[23..16] @ ic3[15.. 8] @ ic2[ 7.. 0];

```

```

    let oc3 : bits(32) = ic2[31..24] @ ic1[23..16] @ ic0[15.. 8] @ ic3[ 7.. 0];
    (oc3 @ oc2 @ oc1 @ oc0) /* 返回值 */
}

```

/* 128 位到 128 位的 AES 逆向 ShiftRows 变换实现。

* 状态的字节 0 是输入列 0, 位 7..0。

* 状态的字节 5 是输入列 1, 位 15..8。

*/

```

val aes_shift_rows_inv : bits(128) -> bits(128)

```

```

function aes_shift_rows_inv(x) = {
    let ic3 : bits(32) = aes_get_column(x, 3); /* 在列 3 中 */
    let ic2 : bits(32) = aes_get_column(x, 2);
    let ic1 : bits(32) = aes_get_column(x, 1);
    let ic0 : bits(32) = aes_get_column(x, 0);
    let oc0 : bits(32) = ic1[31..24] @ ic2[23..16] @ ic3[15.. 8] @ ic0[ 7.. 0];
    let oc1 : bits(32) = ic2[31..24] @ ic3[23..16] @ ic0[15.. 8] @ ic1[ 7.. 0];
    let oc2 : bits(32) = ic3[31..24] @ ic0[23..16] @ ic1[15.. 8] @ ic2[ 7.. 0];
    let oc3 : bits(32) = ic0[31..24] @ ic1[23..16] @ ic2[15.. 8] @ ic3[ 7.. 0];
    (oc3 @ oc2 @ oc1 @ oc0) /* 返回值 */
}

```

/* 将 AES 的正向子字节步骤应用于 128 位向量

* 状态的表示。

*/

```

val aes_subbytes_fwd : bits(128) -> bits(128)

```

```

function aes_subbytes_fwd(x) = {
    let oc0 : bits(32) = aes_subword_fwd(aes_get_column(x, 0));
    let oc1 : bits(32) = aes_subword_fwd(aes_get_column(x, 1));
    let oc2 : bits(32) = aes_subword_fwd(aes_get_column(x, 2));
    let oc3 : bits(32) = aes_subword_fwd(aes_get_column(x, 3));
    (oc3 @ oc2 @ oc1 @ oc0) /* 返回值 */
}

```

/* 将 AES 的逆向子字节步骤应用于 128 位向量

* 状态的表示。

*/

```

val aes_subbytes_inv : bits(128) -> bits(128)

```

```

function aes_subbytes_inv(x) = {
    let oc0 : bits(32) = aes_subword_inv(aes_get_column(x, 0));
    let oc1 : bits(32) = aes_subword_inv(aes_get_column(x, 1));
    let oc2 : bits(32) = aes_subword_inv(aes_get_column(x, 2));
    let oc3 : bits(32) = aes_subword_inv(aes_get_column(x, 3));
    (oc3 @ oc2 @ oc1 @ oc0) /* 返回值 */
}

```

```

/* 将 AES 的正向 MixColumns 步骤应用于 128 位向量
 * 状态的表示。
 */
val aes_mixcolumns_fwd : bits(128) -> bits(128)
function aes_mixcolumns_fwd(x) = {
  let oc0 : bits(32) = aes_mixcolumn_fwd(aes_get_column(x, 0));
  let oc1 : bits(32) = aes_mixcolumn_fwd(aes_get_column(x, 1));
  let oc2 : bits(32) = aes_mixcolumn_fwd(aes_get_column(x, 2));
  let oc3 : bits(32) = aes_mixcolumn_fwd(aes_get_column(x, 3));
  (oc3 @ oc2 @ oc1 @ oc0) /* 返回值 */
}

```

```

/* 将 AES 的逆向 MixColumns 步骤应用于 128 位向量
 * 状态的表示。
 */
val aes_mixcolumns_inv : bits(128) -> bits(128)
function aes_mixcolumns_inv(x) = {
  let oc0 : bits(32) = aes_mixcolumn_inv(aes_get_column(x, 0));
  let oc1 : bits(32) = aes_mixcolumn_inv(aes_get_column(x, 1));
  let oc2 : bits(32) = aes_mixcolumn_inv(aes_get_column(x, 2));
  let oc3 : bits(32) = aes_mixcolumn_inv(aes_get_column(x, 3));
  (oc3 @ oc2 @ oc1 @ oc0) /* 返回值 */
}

```

```

/* 执行 AES 密钥计划的字旋转
 */
val aes_rotword : bits(32) -> bits(32)
function aes_rotword(x) = {
  let a0 : bits (8) = x[7.. 0];
  let a1 : bits (8) = x[15.. 8];
  let a2 : bits (8) = x[23..16];
  let a3 : bits (8) = x[31..24];
  (a0 @ a3 @ a2 @ a1) /* 返回值 */
}

```

```

val brev : bits(SEW) -> bits(SEW)
function brev(x) = {
  let output : bits(SEW) = 0;
  foreach (i from 0 to SEW-8 by 8)
    output[i+7..i] = reverse_bits_in_byte(input[i+7..i]);
  output /* 返回值 */
}

```

```

val reverse_bits_in_byte : bits(8) -> bits(8)
function reverse_bits_in_byte(x) = {
  let output : bits(8) = 0;
  foreach (i from 0 to 7)
    output[i] = x[7-i];
  output /* 返回值 */
}

```

```

val rev8 : bits(SEW) -> bits(SEW)
function rev8(x) = {      // 字节序交换
  let output : bits(SEW) = 0;
  let j = SEW - 1;
  foreach (k from 0 to (SEW - 8) by 8) {
    output[k..(k + 7)] = x[(j - 7)..j];
    j = j - 8;
  }
  output /* 返回值 */
}
RETIRE_SUCCESS

```

```

val rol32 : bits(32) -> bits(32)
function ROL32(x,n) = (X << N) | (X >> (32 - N))

```

```

val sm4_subword : bits(32) -> bits(32)
function sm4_subword(x) = {
  sm4_sbox(x[31..24]) @
  sm4_sbox(x[23..16]) @
  sm4_sbox(x[15.. 8]) @
  sm4_sbox(x[ 7.. 0])
}

```

34 RV32/64G 指令集列表

RISC-V 项目的目标之一是为软件开发提供一个稳定的开发环境。为实现这一目标，我们定义了基础 ISA (RV32I 或 RV64I)，并通过添加选定的标准扩展 (IMAFD、Zicsr、Zifencei)，形成了通用 ISA。我们用缩写 G 来表示 IMAFDZicsr_Zifencei 指令集扩展的组合。本章将展示了 RV32G 和 RV64G 的操作码映射及其指令集清单。

表 34.1 显示了 RVG 的主操作码映射。为了支持超过 32 位的指令，RVG 至少保留了主操作码的最低三位。标记为 reserved 的操作码应避免用于自定义指令集扩展，因为这些操作码可能会被未来的标准扩展使用。标记为 custom-0 和 custom-1 的主操作码将避免被未来的标准扩展使用，因此建议将它们用于基础 32 位指令格式中的自定义指令集扩展。标记为 custom-2/rv128 和 custom-3/rv128 的操作码，虽然为未来的 RV128 预留，但其他标准扩展会避免使用，因此它们在 RV32 和 RV64 中仍可用于自定义指令集扩展。

表 34.1 RISC-V基操作码映射, inst[1:0]=11

inst[4:2]	000	001	010	011	100	101	110	111 (>32b)
inst[6:5]								
00	LOAD	LOAD-FP	Custom-0	MISC-MEM	OP-IMM	AUIPC	OP-IMM-32	48b
01	STORE	STORE-FP	Custom-1	AMO	OP	LUI	OP-32	64b
10	MADD	MSUB	NMSUB	NMADD	OP-FP	OP-V	Custom-2	48b
11	BRANCH	JALR	Reserved	JAL	SYSTEM	OP-VE	Custom-3	>=80b

我们认为, RV32G 和 RV64G 提供了一种简洁且完整的指令集, 适用于多种通用计算需求。此外, 可以通过加入在第 26 章中定义的压缩指令集 (形成 RV32GC 和 RV64GC), 进一步提升性能、能效并减少代码大小, 但这可能会增加硬件设计的复杂性。

当我们超越 IMAFDC, 进入更复杂的指令集扩展时, 新增的指令往往具有更强的领域特定性, 主要为某些特定类型的应用 (如多媒体或安全) 提供优化。不同于大多数商业 ISA, RISC-V ISA 设计上清晰地区分了基础 ISA 和广泛适用的标准扩展, 以及这些更专业的附加扩展。Append A 详细讲解了如何向 RISC-V ISA 添加扩展。

31	27	26	25	24	20	19	15	14	12	11		7	6		0	
funct7				rs2		rs1		funct3				rd		opcode		R-type
imm[11:0]						rs1		funct3				rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3				imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3				imm[4:1 11]		opcode		B-type
imm[31:12]												rd		opcode		U-type
imm[20 10:1 11 19:12]												rd		opcode		J-type

RV32I 基本指令集

imm[31:12]				rd	0110111	LUI
imm[31:12]				rd	0010111	AUIPC
imm[20 10:1 11 19:12]				rd	1101111	JAL
imm[11:0]		rs1	000	rd	1100111	JALR
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	BEQ
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	BGE
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	BLTU
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	BGEU
imm[11:0]		rs1	000	rd	0000011	LB
imm[11:0]		rs1	001	rd	0000011	LH
imm[11:0]		rs1	010	rd	0000011	LW
imm[11:0]		rs1	100	rd	0000011	LBU
imm[11:0]		rs1	101	rd	0000011	LHU
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT

RV32M 标准扩展

0000001	rs2	rs1	000	rd	0110011	MUL
0000001	rs2	rs1	001	rd	0110011	MULH
0000001	rs2	rs1	010	rd	0110011	MULHSU
0000001	rs2	rs1	011	rd	0110011	MULHU
0000001	rs2	rs1	100	rd	0110011	DIV
0000001	rs2	rs1	101	rd	0110011	DIVU
0000001	rs2	rs1	110	rd	0110011	REM
0000001	rs2	rs1	111	rd	0110011	REMU

RV32I 标准扩展

imm[11:0]		rs1	110	rd	0000011	LWU
imm[11:0]		rs1	011	rd	0000011	LD
imm[11:5]	rs2	rs1	011	imm[4:0]	0100011	SD
000000	shamt	rs1	001	rd	0010011	SLLI
000000	shamt	rs1	101	rd	0010011	SRLI
010000	shamt	rs1	101	rd	0010011	SRAI
imm[11:0]		rs1	000	rd	0011011	ADDIW
0000000	shamt	rs1	001	rd	0011011	SLLIW
0000000	shamt	rs1	101	rd	0011011	SRLIW
0100000	shamt	rs1	101	rd	0011011	SRAIW
0000000	rs2	rs1	000	rd	0111011	ADDW
0100000	rs2	rs1	000	rd	0111011	SUBW
0000000	rs2	rs1	001	rd	0111011	SLLW
0000000	rs2	rs1	101	rd	0111011	SRLW
0100000	rs2	rs1	101	rd	0111011	SRAW

RV32/64I zicsr 标准扩展

csr	rs1	001	rd	1110011	CSRRW
csr	rs1	010	rd	1110011	CSRRS
csr	rs1	011	rd	1110011	CSRRC
csr	uimm	101	rd	1110011	CSRRWI
csr	uimm	110	rd	1110011	CSRRSI
csr	uimm	111	rd	1110011	CSRRCI

RV32M 标准扩展

0000001	rs2	rs1	000	rd	0110011	MUL
0000001	rs2	rs1	001	rd	0110011	MULH
0000001	rs2	rs1	010	rd	0110011	MULHSU
0000001	rs2	rs1	011	rd	0110011	MULHU
0000001	rs2	rs1	100	rd	0110011	DIV
0000001	rs2	rs1	101	rd	0110011	DIVU
0000001	rs2	rs1	110	rd	0110011	REM
0000001	rs2	rs1	111	rd	0110011	REMU

RV64M 标准扩展

0000001	rs2	rs1	000	rd	0111011	MULW
0000001	rs2	rs1	100	rd	0111011	DIVW
0000001	rs2	rs1	101	rd	0111011	DIVUW
0000001	rs2	rs1	110	rd	0111011	REMW
0000001	rs2	rs1	111	rd	0111011	REMUW

RV32A 标准扩展

00010	aq	rl	00000	rs1	010	rd	0101111	LR.W
00011	aq	rl	rs2	rs1	010	rd	0101111	SC.W
00001	aq	rl	rs2	rs1	010	rd	0101111	AMOSWAP.W
00000	aq	rl	rs2	rs1	010	rd	0101111	AMOADD.W
00100	aq	rl	rs2	rs1	010	rd	0101111	AMOXOR.W
01100	aq	rl	rs2	rs1	010	rd	0101111	AMOAND.W
01000	aq	rl	rs2	rs1	010	rd	0101111	AMOOR.W
10000	aq	rl	rs2	rs1	010	rd	0101111	AMOMIN.W
10100	aq	rl	rs2	rs1	010	rd	0101111	AMOMAX.W
11000	aq	rl	rs2	rs1	010	rd	0101111	AMOMINU.W
11100	aq	rl	rs2	rs1	010	rd	0101111	AMOMAXU.W

RV64A 标准扩展

00010	aq	rl	00000	rs1	011	rd	0101111	LR.D
00011	aq	rl	rs2	rs1	011	rd	0101111	SC.D
00001	aq	rl	rs2	rs1	011	rd	0101111	AMOSWAP.D
00000	aq	rl	rs2	rs1	011	rd	0101111	AMOADD.D
00100	aq	rl	rs2	rs1	011	rd	0101111	AMOXOR.D
01100	aq	rl	rs2	rs1	011	rd	0101111	AMOAND.D
01000	aq	rl	rs2	rs1	011	rd	0101111	AMOOR.D
10000	aq	rl	rs2	rs1	011	rd	0101111	AMOMIN.D
10100	aq	rl	rs2	rs1	011	rd	0101111	AMOMAX.D
11000	aq	rl	rs2	rs1	011	rd	0101111	AMOMINU.D
11100	aq	rl	rs2	rs1	011	rd	0101111	AMOMAXU.D

31	27	26	25	24	20	19	15	14	12	11	7	6	0		
funct7				rs2	rs1		funct3		rd		opcode		R-type		
rs3		funct2			rs2	rs1		funct3		rd		opcode		R4-type	
imm[11:0]						rs1		funct3		rd		opcode		I-type	
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type	

RV32F 标准扩展

imm[11:0]		rs2	rs1	010	rd	0000111	FLW
imm[11:5]		rs2	rs1	010	imm[4:0]	0100111	FSW
rs3	00	rs2	rs1	rm	rd	1000011	FMADD.S
rs3	00	rs2	rs1	rm	rd	1000111	FMSUB.S
rs3	00	rs2	rs1	rm	rd	1001011	FNMSUB.S
rs3	00	rs2	rs1	rm	rd	1001111	FNMADD.S
0000000		rs2	rs1	rm	rd	1010011	FADD.S
0000100		rs2	rs1	rm	rd	1010011	FSUB.S
0001000		rs2	rs1	rm	rd	1010011	FMUL.S
0001100		rs2	rs1	rm	rd	1010011	FDIV.S
0101100		00000	rs1	rm	rd	1010011	FSQRT.S
0010000		rs2	rs1	000	rd	1010011	FSGNJ.S
0010000		rs2	rs1	001	rd	1010011	FSGNJN.S
0010000		rs2	rs1	010	rd	1010011	FSGNJX.S
0010100		rs2	rs1	000	rd	1010011	FMIN.S
0010100		rs2	rs1	001	rd	1010011	FMAX.S
1100000		00000	rs1	rm	rd	1010011	FCVT.W.S
1100000		00001	rs1	rm	rd	1010011	FCVT.WU.S
1110000		00000	rs1	000	rd	1010011	FMV.X.W
1010000		rs2	rs1	010	rd	1010011	FEQ.S
1010000		rs2	rs1	001	rd	1010011	FLT.S
1010000		rs2	rs1	000	rd	1010011	FLE.S
1110000		00000	rs1	001	rd	1010011	FCLASS.S
1101000		00000	rs1	rm	rd	1010011	FCVT.S.W
1101000		00001	rs1	rm	rd	1010011	FCVT.S.WU
1111000		00000	rs1	000	rd	1010011	FMV.W.X

RV64F 标准扩展

1100000	00010	rs1	rm	rd	1010011	FCVT.L.S
1100000	00011	rs1	rm	rd	1010011	FCVT.LU.S
1101000	00010	rs1	rm	rd	1010011	FCVT.S.L
1101000	00011	rs1	rm	rd	1010011	FCVT.S.LU

31	27	26	25	24	20	19	15	14	12	11	7	6	0		
funct7				rs2		rs1		funct3		rd		opcode		R-type	
rs3		funct2			rs2		rs1		funct3		rd		opcode		R4-type
imm[11:0]						rs1		funct3		rd		opcode		I-type	
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type	

RV32D 标准扩展

imm[11:0]		rs2	rs1	011	rd	0000111	FLD
imm[11:5]		rs2	rs1	011	imm[4:0]	0100111	FSD
rs3	01	rs2	rs1	rm	rd	1000011	FMADD.D
rs3	01	rs2	rs1	rm	rd	1000111	FMSUB.D
rs3	01	rs2	rs1	rm	rd	1001011	FNMSUB.D
rs3	01	rs2	rs1	rm	rd	1001111	FNMADD.D
0000001		rs2	rs1	rm	rd	1010011	FADD.D
0000101		rs2	rs1	rm	rd	1010011	FSUB.D
0001001		rs2	rs1	rm	rd	1010011	FMUL.D
0001101		rs2	rs1	rm	rd	1010011	FDIV.D
0101101		00000	rs1	rm	rd	1010011	FSQRT.D
0010001		rs2	rs1	000	rd	1010011	FSGNJ.D
0010001		rs2	rs1	001	rd	1010011	FSGNJN.D
0010001		rs2	rs1	010	rd	1010011	FSGNJX.D
0010101		rs2	rs1	000	rd	1010011	FMIN.D
0010101		rs2	rs1	001	rd	1010011	FMAX.D
0100000		00001	rs1	rm	rd	1010011	FCVT.S.D
0100001		00000	rs1	rm	rd	1010011	FCVT.D.S
1010001		rs2	rs1	010	rd	1010011	FEQ.D
1010001		rs2	rs1	001	rd	1010011	FLT.D
1010001		rs2	rs1	000	rd	1010011	FLE.D
1110001		00000	rs1	001	rd	1010011	FCLASS.D
1100001		00000	rs1	rm	rd	1010011	FCVT.W.D
1100001		00001	rs1	rm	rd	1010011	FCVT.WU.D
1101001		00000	rs1	rm	rd	1010011	FCVT.D.W
1101001		00001	rs1	rm	rd	1010011	FCVT.D.WU

RV64D 标准扩展

1100001		00010	rs1	rm	rd	1010011	FCVT.L.D						
1100001		00011	rs1	rm	rd	1010011	FCVT.LU.D						
1110001		00000	rs1	000	rd	1010011	FMV.X.D						
1101001		00010	rs1	rm	rd	1010011	FCVT.D.L						
1101001		00011	rs1	rm	rd	1010011	FCVT.D.LU						
1111001		00000	rs1	000	rd	1010011	FMV.D.X						
31	27	26	25	24	20	19	15	14	12	11	7	6	0

RV32Q 标准扩展

imm[11:0]			rs1	100	rd	0000111	FLQ
imm[11:5]		rs2	rs1	100	imm[4:0]	0100111	FSQ
rs3	11	rs2	rs1	rm	rd	1000011	FMADD.Q
rs3	11	rs2	rs1	rm	rd	1000111	FMSUB.Q
rs3	11	rs2	rs1	rm	rd	1001011	FNMSUB.Q
rs3	11	rs2	rs1	rm	rd	1001111	FNMADD.Q
0000011		rs2	rs1	rm	rd	1010011	FADD.Q
0000111		rs2	rs1	rm	rd	1010011	FSUB.Q
0001011		rs2	rs1	rm	rd	1010011	FMUL.Q
0001111		rs2	rs1	rm	rd	1010011	FDIV.Q
0101111		00000	rs1	rm	rd	1010011	FSQRT.Q
0010011		rs2	rs1	000	rd	1010011	FSGNJ.Q
0010011		rs2	rs1	001	rd	1010011	FSGNJN.Q
0010011		rs2	rs1	010	rd	1010011	FSGNJX.Q
0010111		rs2	rs1	000	rd	1010011	FMIN.Q
0010111		rs2	rs1	001	rd	1010011	FMAX.Q
0100000		00011	rs1	rm	rd	1010011	FCVT.S.Q
0100011		00000	rs1	rm	rd	1010011	FCVT.Q.S
0100001		00011	rs1	rm	rd	1010011	FCVT.D.Q
0100011		00001	rs1	rm	rd	1010011	FCVT.Q.D
1010011		rs2	rs1	010	rd	1010011	FEQ.Q
1010011		rs2	rs1	001	rd	1010011	FLT.Q
1010011		rs2	rs1	000	rd	1010011	FLE.Q
1110011		00000	rs1	001	rd	1010011	FCLASS.Q
1100011		00000	rs1	rm	rd	1010011	FCVT.W.Q
1100011		00001	rs1	rm	rd	1010011	FCVT.WU.Q
1101011		00000	rs1	rm	rd	1010011	FCVT.Q.W
1101011		00001	rs1	rm	rd	1010011	FCVT.Q.WU

RV64Q 标准扩展

1100011	00010	rs1	rm	rd	1010011	FCVT.L.Q
1100011	00011	rs1	rm	rd	1010011	FCVT.LU.Q
1101011	00010	rs1	rm	rd	1010011	FCVT.Q.L
1101011	00011	rs1	rm	rd	1010011	FCVT.Q.LU

31	27	26	25	24	20	19	15	14	12	11	7	6	0		
funct7				rs2		rs1		funct3		rd		opcode		R-type	
rs3		funct2			rs2		rs1		funct3		rd		opcode		R4-type
imm[11:0]						rs1		funct3		rd		opcode		I-type	
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type	

RV64zh 标准扩展						
1100010	00011	rs1	rm	rd	1010011	FCVT.LU.H
1101010	00010	rs1	rm	rd	1010011	FCVT.H.L
1101010	00011	rs1	rm	rd	1010011	FCVT.H.LU

Zwars 标准扩展					
00000001101	00000	000	00000	1110011	WRS.NTO
00000001101	00000	000	00000	1110011	WRS.STO

35 扩展 RISC-V

除了支持标准的通用软件开发外，RISC-V 的另一个目标是为更专业的指令集扩展或定制加速器提供基础。指令编码空间和可选的可变长度指令编码的设计目的是使构建定制处理器时能够更容易地利用标准 ISA 工具链进行软件开发。例如，RISC-V 旨在继续为仅使用标准 I 基础的实现提供完整的软件支持，同时与多个非标准指令集扩展兼容。

本章介绍了扩展 RISC-V 基础 ISA 的多种方法，并讨论了独立小组开发的指令集扩展的管理方案。本卷仅涉及非特权 ISA，但第二卷中描述的监督级扩展也采用相同的方法与术语。

35.1 扩展术语

本节定义了一些描述 RISC-V 扩展的标准术语。

35.1.1 标准扩展与非标准扩展

任何 RISC-V 处理器实现必须支持一个基础整数 ISA (RV32I、RV32E、RV64I、RV64E 或 RV128I)。此外，处理器可以选择支持一个或多个扩展。我们将扩展分为两大类：标准和非标准。

* 标准扩展指那些通常有用，并且在设计上不会与其他标准扩展冲突的扩展。目前，本手册的其他章节中描述的 MAFDQLCBTPV 是已完成或计划中的标准扩展。

* 非标准扩展可能具有高度的专业性，并且可能与其他标准或非标准扩展冲突。我们预计，随着时间推移，许多非标准扩展将被开发出来，其中一些可能最终被提升为标准扩展。

35.1.2 指令编码空间和前缀

指令编码空间是指在其中存放基础 ISA 或 ISA 扩展的指令位的区域。RISC-V 支持不同的指令长度，即便在单一指令长度下，也存在多种编码空间大小。例如，基础 ISA 在 30 位编码空间内定义（32 位指令的 31-2 位），而原子扩展 A 则适合在 25 位编码空间内（32 位指令的 31-7 位）。

我们使用术语前缀来指代指令编码空间右侧的位（因为 RISC-V 中的指令获取采用小端序，右侧的位存储在较低的内存地址中，因此在指令获取顺序中形成前缀）。标准基础 ISA 编码的前缀是 32 位字中的 1-0 位中的 11 字段，而标准原子扩展 A 的前缀是 32 位字中 AMO 主操作码的 6-0 位中的 0101111 字段。编码格式的一个特点是，3 位 funct3 字段用于编码次操作码，在 32 位指令格式中与主操作码位不连续，但依然被视为 22 位指令空间前缀的一部分。

尽管指令编码空间的大小可以是任意的，采用一组较小的常见大小有助于简化将独立开发的扩展合并到单个全局编码中的过程。表 35.1 给出了 RISC-V 建议的编码空间大小。

表 35.1 建议标准 RISC-V 指令编码空间大小

尺度	用法	16-bit	32-bit	48-bit	64-bit
16-bit	压缩16位编码的象限	3			
22-bit	基本32位编码的次要操作码		2^8	2^{20}	2^{35}
25-bit	基本32位编码的主操作码		32	2^{17}	2^{32}
30-bit	基本32位编码的象限		1	2^{12}	2^{27}
32-bit	48位编码的次要操作码			2^{10}	2^{25}
37-bit	48位编码的主操作码			32	2^{20}
40-bit	象限的48位编码			4	2^{17}
45-bit	64位编码的子操作码				2^{12}
48-bit	64位编码的次要操作码				2^9
52-bit	64位编码的主操作码				32

35.1.3 绿地扩展与棕地扩展

我们使用术语绿地扩展（greenfield extension）来描述那些填充新指令编码空间的扩展，因此它们只会在前缀级别引发编码冲突。我们使用术语棕地扩展来描述在已有指令空间内进行的扩展，这些扩展围绕先前定义的编码展开。棕地扩展（brownfield extension）必然与特定的绿地父编码相关，一个绿地父编码可能关联多个棕地扩展。例如，基础 ISA 是 30 位指令空间的绿地编码，而 FDQ 浮点扩展则作为棕地扩展，添加到基础 ISA 30 位编码空间中。

请注意，标准 A 扩展被视为绿地编码，因为它在完整的 32 位基础指令编码的最左侧位置定义了一个新的、之前为空的 25 位编码空间，尽管其标准前缀使其定位于父基础 ISA 的 30 位编码空间内。只需调整其 7 位前缀，即可将 A 扩展移动到不同的 30 位编码空间，这时仅需要关注前缀级别的冲突，而不是编码空间内的冲突。

表 35.2 标准指令集扩展的二维分类

扩展类型	增加状态	无新状态
绿地扩展	RV32I (30), RV64I (30)	A (25)
棕地扩展	F (I), D (F), Q (D)	M (I)

表 35.2 显示了基础和标准扩展在简单二维分类中的位置。一个轴用于区分扩展是绿地扩展还是棕地扩展，另一个轴用于区分扩展是否增加了架构状态。绿地扩展的指令编码空间大小在括号中给出，而棕地扩展则给出其所基于的绿地或棕地扩展名称。增加用户级架构状态通常意味着需要对监督级系统或标准调用约定进行更改。需要注意的是，RV64I 并不是 RV32I 的扩展，而是一个完全独立的基础编码。

35.1.4 标准兼容的全局编码

对于实际的 RISC-V 实现，完整或全局编码必须为每个包含的指令编码空间分配一个唯一且不冲突的前缀。基础和每个标准扩展都已分配标准前缀，确保它们能够在全局编码中共存。

标准兼容的全局编码指的是基础和每个标准扩展都使用其标准前缀的编码。标准兼容的全局编码可以包含与标准扩展不冲突的非标准扩展。如果未包含相关的标准扩展，标准兼容的全局编码依然可以为非标准扩展使用标准前缀。换句话说，标准扩展必须在标准兼容的全局编码中使用其标准前缀，但在其他情况下，其前缀可以重新分配。这些约束使得通用工具链能够支持任何 RISC-V 标准兼容的全局编码的标准子集。

35.1.5 保证的非标准编码空间

为了支持专有定制扩展的开发，标准扩展必须确保永远不会使用编码空间的某些特定部分。

35.2 RISC-V 扩展设计理念

我们计划通过鼓励扩展开发者在指令编码空间内进行操作，并提供工具将这些扩展打包到标准兼容的全局编码中，从而通过分配唯一前缀来支持大量独立开发的扩展。一些扩展更自然地作为现有扩展的棕地增强，并与其父绿地扩展共享分配的前缀。标准扩展前缀有效避免了核功能编码中的虚假不兼容，同时也为更多偏门扩展提供了定制打包的灵活性。这种将 RISC-V 扩展重新打包到不同标准兼容全局编码中的能力可在多种场景下使用。

一种典型用例是开发高度专业化的定制加速器，这些加速器主要用于执行特定应用领域的核任务。这些加速器通常希望去除除基础整数 ISA 之外的所有功能，仅添加所需的扩展。基础 ISA 的设计旨在对硬件实现的要求最小，其编码仅占用 32 位指令编码空间的一小部分。

另一个用例是为新型指令集扩展构建研究原型。在这种情况下，研究人员可能不希望实现可变长度指令获取单元，因此更倾向于使用简单的 32 位固定宽度指令编码来原型化其扩展。然而，这些扩展可能过于庞大，无法与标准扩展共存于 32 位编码空间内。如果研究工作不需要所有标准扩展，标准兼容的全局编码可以移除未使用的标准扩展，并将其前缀重新分配，放置拟议扩展（proposed extension）到非标准位置，从而简化原型开发。标准工具链仍能支持现有基础和标准扩展，以减少开发时间。一旦扩展通过评估并得到改进，它可以打包进更大的可变长度编码空间，以避免与标准扩展发生冲突。

以下部分介绍了开发具有新指令集扩展实现的逐步策略，这些策略主要适用于高度定制、教育或实验架构，而不是 RISC-V ISA 开发的主流路径。

35.3 固定宽度 32 位指令格式内的扩展

本节讨论仅支持基础固定宽度 32 位指令格式的实现的扩展。



我们预计，最简单的 32 位固定宽度编码将受到许多受限加速器和研究原型的广泛应用。

35.3.1 可用的 30 位指令编码空间

在标准编码中，三个可用的 30 位指令编码空间（前缀为 00、01 和 10 的 2 位）被保留用于启用可选的压缩指令扩展。然而，如果不需要压缩指令集扩展，这三个 30 位编码空间将变为可用空间，从而使得 32 位格式中的可用编码空间增加了四倍。

35.3.2 可用的 25 位指令编码空间

25 位指令编码空间对应于基础和标准扩展编码中的主操作码。

有四个明确保留用于自定义扩展的主操作码 `opcodemap`，每个代表一个 25 位编码空间。两个主操作码保留给 RV128 基础编码使用（分别是 `OP-IMM-64` 和 `OP-64`），但在 RV32 和 RV64 的非标准扩展中也可以使用。

为 RV64 保留的两个主操作码（`OP-IMM-32` 和 `OP-32`）同样可以用于 RV32 的非标准扩展。如果实现中不需要浮点功能，可以重新使用为标准浮点扩展保留的七个主操作码（`LOAD-FP`、`STORE-FP`、`MADD`、`MSUB`、`NMSUB`、`NMADD`、`OP-FP`）。同样，如果不需要标准原子扩展，也可以重新使用 `AMO` 主操作码。如果实现不需要超过 32 位的指令，可以使用另外四个主操作码（在 `opcodemap` 中标记为灰色）。

基础 RV32I 编码使用 11 个主操作码和 3 个保留操作码，最多有 18 个主操作码可供扩展使用。基础 RV64I 编码使用 13 个主操作码和 3 个保留操作码，最多有 16 个主操作码可供扩展使用。

35.3.3 可用的 22 位指令编码空间

22 位编码空间对应于基础和标准扩展编码中的 `funct3` 次操作码空间。若干主操作码的 `funct3` 字段未完全占用，导致有多个可用的 22 位编码空间。通常，主操作码用于在指令的其他位中编码操作数格式，扩展应遵循主操作码的操作数格式，这样可以简化硬件解码过程。

35.3.4 其他空间

在某些主操作码下，编码空间较为有限，且并非所有次操作码都完全填充这些空间。

35.4 添加对齐的 64 位指令扩展

为扩展提供空间的最简单方式是添加自然对齐的 64 位指令，这些指令对于基础的 32 位固定宽度指令格式过大。实现仍然需要支持 32 位基础指令格式，但可以要求 64 位指令按照 64 位边界对齐，以简化指令获取，必要时可使用 32 位 `NOP` 指令进行对齐填充。

为了简化标准工具的使用，64 位指令应按 `instlengthcode`，Table 1 中规定的方式进行编码。然而，某些实现可以选择非标准的 64 位指令长度编码，同时保留 32 位指令的标准编码。例如，如果不需要压缩指令，可以通过在指令的前两位中设置零位来编码 64 位指令。



我们预计处理器生成器将生成能够自动处理所有支持的可变长度指令编码组合的指令获取单元。

35.5 支持 VLIW 编码

虽然 RISC-V 不是专为纯 VLIW 机器设计，但可以通过几种替代方法将 VLIW 编码作为扩展添加。在所有实现中，都必须支持基础的 32 位编码，以便能够使用标准软件工具。

35.5.1 固定大小的指令组

最简单的方式是定义一个自然对齐的大指令格式（例如 128 位），用于编码 VLIW 操作。在传统的 VLIW 设计中，这种方法可能会浪费指令内存来存储 `NOP`，但在 RISC-V 兼容实现中，必须支持基础 32 位指令，这将 VLIW 代码的大小限制在加速函数的范围内。

35.5.2 编码长度组

另一种方式是利用章节 1.4 中的标准长度编码来表示并行指令组，从而压缩 VLIW 指令中的 `NOP`。

例如，64 位指令可以包含两个 28 位操作，96 位指令可以包含三个 28 位操作等；或者，48 位指令可以包含一个 42 位操作，96 位指令可以包含两个 42 位操作等。

此方法的优点是保留了包含单一操作的基础 ISA 编码，但缺点是需要为 VLIW 指令中的操作引入新的 28 位或 42 位编码，且较大指令组的获取可能未对齐。可以采取一种简化方法，避免 VLIW 指令跨越一些微架构中的关键边界（如缓存行或虚拟内存页）。

35.5.3 固定大小的指令包

另一种方法，类似于 Itanium，采用更大且自然对齐的固定指令包（例如 128 位）来编码并行操作组。此方法简化了指令获取，但将复杂性转移到组执行引擎（group execution engine）。为了保持 RISC-V 兼容，必须支持基础 32 位指令。

35.5.4 前缀中的组结束位

上述方法未能保留 VLIW 指令中单个操作的 RISC-V 编码。另一种方法是重新利用固定宽度 32 位编码中的两个前缀位：一个用于指示组结束（end-of-group）（如果设置），另一个用于指示是否在谓词控制下执行（如果清除）。由不支持 VLIW 扩展的工具生成的标准 RISC-V 32 位指令将同时设置这两个前缀位（11），因此它们具备正确的语义——每个指令在组结束时且不受谓词控制。这种方法的缺点是，基础 ISA 并未提供激进 VLIW 系统中常见的复杂谓词支持，且在标准 30 位编码空间中，难以扩展以支持更多谓词寄存器的指定。

36 ISA 扩展命名约定

本章描述了 RISC-V ISA 扩展的命名方案，用于简洁地描述硬件实现中存在的指令集，或应用程序二进制接口（ABI）使用的指令集。



RISC-V ISA 设计支持多种实验性指令集扩展的实现。我们发现，系统化的命名方案能有效简化软件工具与文档的管理。

36.1 大小写敏感性

ISA 命名字符串不区分大小写。

36.2 基础整数 ISA

RISC-V ISA 字符串以 RV32I、RV32E、RV64I、RV64E 或 RV128I 开头，表示基础整数 ISA 支持的地址空间大小（以位为单位）。

36.3 指令集扩展名称

标准 ISA 扩展名称由单个字母组成。例如，基础整数扩展的前四个标准扩展为：M 表示整数乘法与除法，A 表示原子内存指令，F 表示单精度浮点指令，D 表示双精度浮点指令。任何 RISC-V 指令集变体可通过将基础整数前缀与其扩展名称连接，简洁地表示，如 RV64IMAFD。

此外，G 缩写代表 IMAFDZicsrZifencei 基础及扩展，代表我们的标准通用 ISA。

RISC-V ISA 的标准扩展使用其他保留字母，例如 Q 表示四倍精度浮点，C 表示 16 位压缩指令格式。

某些扩展依赖于其他扩展的存在，例如 D 依赖于 F，F 依赖于 Zicsr。这些依赖关系在 ISA 名称中已隐含：例如，RV32IF 等价于 RV32IFZicsr，RV32ID 等价于 RV32IFD 和 RV32IFDZicsr。

36.4 版本号

考虑到指令集可能会随时间变化或扩展，我们在扩展名称后面添加了版本号。版本号由主版本号和次版本号组成，二者通过 p 分隔。若次版本号为 0，则 p0 可以省略。主版本号的变化意味着不再向后兼容，而次版本号的变化则要求保持向后兼容。例如，本手册第 1.0 版中定义的初始 64 位标准 ISA 完整表示为 RV64I1p0M1p0A1p0F1p0D1p0，简化后为 RV64I1M1A1F1D1。

我们从第二版起引入了版本编号方案，并将标准扩展的默认版本设为当时的版本。例如，RV32I 即代表 RV32I2。

36.5 下划线

为提升可读性并避免歧义，ISA 扩展可以通过下划线分隔，例如 RV32I2M2A2。由于 P 扩展（表示打包 SIMD）可能与版本号中的小数点混淆，因此如果 P 紧跟在数字后面，必须加上下划线。例如，rv32i2p2 表示 RV32I 的 2.2 版，而 rv32i2p2 表示 RV32I 2.0 版以及 P 扩展 2.0 版。

36.6 其他标准扩展名称

标准扩展的命名方式还可以使用 Z 后跟字母名称，并附上可选的版本号。例如，Zifencei 代表指令获取屏障扩展；Zifencei2 和 Zifencei2p0 均为其 2.0 版。

Z 后的首字母通常表示最相关的扩展类别（如 IMAFDQCVH）。例如，Zfa 扩展表示附加浮点指令，其中字母 f 表示与 F 标准扩展相关。若命名多个 Z 扩展，应按类别排序，且在每个类别内按字母顺序排列，如 ZicsrZifenceiZam。

所有多字母扩展（multi-letter extensions），包括带有 Z 前缀的扩展，必须用下划线与其他扩展分隔，例如 RV32IMACZicsrZifencei。

36.7 监督级指令集扩展

扩展监督级虚拟内存架构的标准扩展以 Sv 为前缀，后接字母名称和可选的版本号，或者仅接无版本号的数字名称。扩展监督级架构的其他标准扩展则以 Ss 为前缀，后接字母名称和可选版本号。这些扩展的详细信息在第二卷中有所定义。

标准监督级扩展应按顺序列在标准非特权扩展之后。如果有多个监督级扩展，应按字母顺序排列。

36.8 虚拟机级指令集扩展

扩展虚拟机级架构的标准扩展以字母 Sh 开头。如果有多个虚拟机级扩展，它们应按字母顺序排列。



许多虚拟机级架构扩展更适合作为监督级扩展定义，按照上一节描述的方式处理。仅少数不涉及监督可见效果的虚拟机级扩展才使用 Sh 前缀。

36.9 机器级指令集扩展

标准机器级指令集扩展以字母 Sm 为前缀。标准机器级扩展应位于标准低特权扩展之后，多个机器级扩展应按字母顺序排列。

36.10 非标准扩展名称

非标准扩展以字母 X 开头，后跟扩展名称和可选版本号。例如，Xhwacha 表示 Hwacha 向量获取扩展，Xhwacha2 和 Xhwacha2p0 则表示同一扩展的 2.0 版本。

所有非标准扩展应排在标准扩展之后，并与其他多字母扩展用下划线分隔。例如，带有 Argle 和 Bargle 非标准扩展的 ISA 可命名为 RV64IZifenceiXargleXbargle。多个非标准扩展应按字母顺序排列。

36.11 子集命名约定

表 36.1 总结了标准化的扩展名称，并定义了这些名称在字符串中的规范顺序。表格从上到下列出的顺序，表示名称字符串中的顺序。例如，RV32IMACV 是合法的，而 RV32IMAVC 则不合法。

表 36.1 标准 ISA 扩展名称

子集	名称	含义
Base ISA		
Integer	I	
Reduced Integer	E	
Integer Multiplication and Division	M	Zmmul
Atomics	A	
Single-Precision Floating-Point	F	Zicsr
Double-Precision Floating-Point	D	F
General	G	IMAFDZicsrZifencei
Quad-Precision Floating-Point	Q	D
16-bit Compressed Instructions	C	
B Extension	B	
Packed-SIMD Extensions	P	
Vector Extension	V	D
Hypervisor Extension	H	
Additional Standard unprivileged extensions abc	Zabc	
Supervisor-level extension def	Ssdef	
Machine-level extension jkl	Smjkl	

37 历史与致谢

37.1 为什么开发新的 ISA？——来自伯克利小组的理由

RISC-V 的开发始于我们对研究和教育的需求。我们小组非常注重将研究理念付诸实际硬件实现（自本规范第一版发布以来，我们已完成了十一种不同的 RISC-V 芯片设计与制造），并为学生提供实际的硬件设计，供他们在课堂上探索（RISC-V 处理器 RTL 设计已在伯克利的多个本科和研究生课程中使用）。在目前的研究中，我们尤其关注传统晶体管缩放的终结带来的变化，特别是向专业化和异构加速器的转变。我们希望通过一个高度灵活且可扩展的基础 ISA 支持我们的研究工作。

我们经常被问到一个问题：为什么要开发新的 ISA？采用现有商业 ISA 的最大优势是能够享受广泛的软件生态系统支持，包括各种开发工具和移植的应用程序，这对研究和教学具有重要意义。其他优点包括大量现成的文件和教程。然而，根据我们的经验，使用现有商业指令集的好处在实际应用中相对有限，且远不及其潜在缺点多：

商业 ISA 是专有的。除了 SPARC V8 是一个开放的 IEEE 标准外，绝大多数商业 ISA 的所有者

都严格保护其知识产权，不欢迎自由开放的竞争性实现。对于仅依赖软件模拟器的学术研究和教学而言，这通常不是问题，但对于希望分享实际 RTL 实现的团队来说，这造成了较大障碍。对于不愿依赖少数商业 ISA 实现的机构，且被禁止自行创建净室实现的情况，这也是一个大问题。尽管我们不能保证所有 RISC-V 实现不会侵犯第三方专利，但我们可以保证不会试图起诉任何 RISC-V 实现者。

商业 ISA 仅在某些市场领域流行。在撰写本文时，ARM 架构在服务器领域支持较差，而 Intel x86 架构（或几乎所有其他架构）在移动领域的支持则较弱，尽管这两家公司都在尝试进入对方的市场。另一个例子是 ARC 和 Tensilica，它们专注于嵌入式领域，并提供可扩展的内核。这种市场细分降低了商业 ISA 的优势，因为软件生态系统往往局限于某些领域，其他领域则需要重新开发支持。

商业 ISA 迭起兴衰。以前，许多研究基础设施是围绕不再流行的商业 ISA（如 SPARC 和 MIPS）构建的，甚至一些已经停止生产（如 Alpha）。这些 ISA 失去了活跃的软件生态系统，并且由于知识产权问题，第三方无法继续为其提供支持。相比之下，开放的 ISA 即使不再流行，相关方仍然可以继续使用和发展其生态系统。

流行的商业 ISA 很复杂。当前流行的商业 ISA（如 x86 和 ARM）在硬件实现上非常复杂，主要支持常见的软件堆栈和操作系统。然而，这种复杂性大多源自早期糟糕的设计决策，而非真正提升效率的功能。

仅有商业 ISA 不足以启动应用程序。即使我们投入了大量精力实现某一商业 ISA，它依然无法直接运行现有应用程序。大多数应用程序需要完整的 ABI（应用程序二进制接口），而不仅仅依赖于用户级 ISA。大部分 ABI 依赖于特定的库，而这些库又需要操作系统的支持。为了运行现有的操作系统，我们必须实现操作系统要求的监督级 ISA 和设备接口，这通常比用户级 ISA 更为复杂且实现起来更具挑战性。

流行的商业 ISA 不是为可扩展性设计的。目前主流的商业 ISA 并未考虑可扩展性。随着指令集的扩展，指令编码变得越来越复杂。例如，Tensilica（现已被 Cadence 收购）和 ARC（现已被 Synopsys 收购）等公司围绕可扩展性设计了 ISA 和工具链，但更多专注于嵌入式应用，而非通用计算系统。

修改后的商业 ISA 是一种全新的 ISA。我们的主要目标之一是支持架构研究，特别是包括主要 ISA 扩展的研究。即使是较小的扩展，也会削弱使用标准 ISA 的优势，因为必须修改编译器，应用程序也需要从源代码重新编译以支持这些扩展。对于更大的架构扩展，还需要对操作系统进行调整。最终，修改后的商业 ISA 不再是原有的 ISA，而是变成了一个全新的 ISA，带来了基础 ISA 的所有遗留问题。

我们的立场是，ISA 是计算机系统中最为关键的接口，理应是开放的，而不应成为专有的。主流商业 ISA 基于三十多年前的指令集概念，软件开发者应当能够针对开放标准的硬件目标进行开发，商业处理器设计者则应在质量上展开竞争。

我们并非唯一考虑开放 ISA 设计的团队。在评估过其他开放 ISA 设计后，最接近我们目标的是 OpenRISC 架构。但由于以下技术原因，我们决定不采用 OpenRISC ISA：

- * OpenRISC 使用条件码和分支延迟槽，增加了高性能实现的复杂度。
- * OpenRISC 使用固定的 32 位编码和 16 位立即数，限制了指令编码密度，并影响了未来的扩展空间。
- * OpenRISC 不支持 2008 年修订的 IEEE 754 浮点标准。
- * 在我们开始设计 RISC-V 时，OpenRISC 64 位设计尚未完成。

因此，我们决定从头开始设计一个新的 ISA，以满足我们的所有目标，尽管这一过程花费了比原计划更多的精力。目前，我们已经投入了大量资源建设 RISC-V ISA 的基础设施，包括文档、编译器工具链、操作系统移植、模拟器、FPGA 实现、高效的 ASIC 实现、架构测试套件和教学材料。自本手册上一版发布以来，RISC-V 已在学术界和产业界得到广泛应用，并且我们成立了非营利性 RISC-V 基金会，以保护并推广这一标准。RISC-V 基金会网站 <https://riscv.org> 包含有关基金会成员和使用 RISC-V 的各种开源项目的最新信息。

37.2 ISA 手册 1.0 版的历史

RISC-V ISA 和指令集手册的形成源于多个早期项目的积累。手册的整体结构以及监督级机器的一些特点可以追溯到 1992 年开始的 T0 (Torrent-0) 矢量微处理器项目，该项目由加州大学伯克利分校 (UC Berkeley) 和国际计算机科学研究所 (ICSI) 共同推进。T0 基于 MIPS-II ISA, Krste Asanović 担任主要架构师和 RTL 设计师, Brian Kingsbury 和 Bertrand Irrisou 负责 VLSI 实现。ICSI 的 David Johnson 对 T0 ISA 的设计贡献巨大，特别是在监督模式设计和手册文本方面, John Hauser 也提供了大量宝贵的反馈。

2000 年, MIT 启动了 Scale (低功耗软件控制架构) 项目, 该项目在 T0 项目基础上进行扩展, 改进了监督级接口, 并通过移除分支延迟槽摆脱了 MIPS 标量 ISA 的局限。MIT Scale 矢量线程处理器的主要架构师为 Ronny Krashinsky 和 Christopher Batten, 而 Mark Hampton 负责将基于 GCC 的编译器基础设施和工具移植到 Scale。

T0 MIPS 标量处理器规范 (MIPS-6371) 经过小幅编辑后, 于 2002 年秋季学期的 MIT 6.371 VLSI 系统导论课程中用于教学, Chris Terman 和 Krste Asanović 担任讲师。Chris Terman 为该课程贡献了大部分实验材料。课程随后演变为 2005 年春季由 Arvind 和 Krste Asanović 教授的试验课程 6.884 复杂数字设计, 并最终成为常规的春季课程 6.375。6.884/6.375 课程中使用了一个基于 Scale MIPS 的简化标量 ISA, 名为 SMIPS。Christopher Batten 是这些课程的早期助教, 并开发了大量关于 SMIPS ISA 的文档和实验材料。助教 Yunsup Lee 在 2009 年秋季将这些材料进行了适应和增强, 并应用于 UC Berkeley CS250 VLSI 系统设计课程, 由 John Wawrzynek、Krste Asanović 和 John Lazzaro 教授。

Maven(可塑矢量线程引擎阵列)项目是第二代矢量线程架构, 设计由 Christopher Batten 于 2007 年夏季作为 UC Berkeley 交换学者时主导。日立的产业研究员 Hidetaka Aoki 为早期 Maven ISA 和微架构设计提供了宝贵的反馈。Maven 基础设施基于 Scale, 但与 Scale 中定义的 MIPS ISA 变体相比, Maven ISA 采用了统一的浮点和整数寄存器文件, 进一步改进了设计。Maven 旨在支持替代数据并行加速器的实验。Yunsup Lee 主要负责 Maven 矢量单元的实现, 而 Rimas Avizienis 则负责标量单元的实现。Yunsup Lee 和 Christopher Batten 将 GCC 移植到 Maven 的新 ISA。Christopher Celio 提供了 Maven 传统矢量指令集 (Flood) 变体的初始定义。

基于这些先前项目的经验,RISC-V ISA 于 2010 年夏季开始定义, 设计团队包括 Andrew Waterman、Yunsup Lee、Krste Asanović 和 David Patterson。RISC-V 32 位指令子集的初版在 2010 年秋季的 UC Berkeley CS250 VLSI 系统设计课程中首次使用, Yunsup Lee 担任助教。RISC-V 是对早期 MIPS 启发设计的一个重要突破。John Hauser 为浮点 ISA 的定义作出了贡献, 特别是在符号注入指令和浮点值内部重新编码的寄存器编码方案方面。

37.3 ISA 手册 2.0 版的历史

多款RISC-V处理器已完成实现，其中包括多次数流片验证，如表37.1。

表 37.1 已流片的 RISC-V 测试芯片列表

名称	时间	进程	ISA
Raven-1	2011. 5. 29	ST 28nm FDSOI	RV64G1Xhwacha1
EOS14	2012. 7. 01	IBM 45nm SOI	RV64G1p1Xhwacha2
EOS16	2012. 8. 17	IBM 45nm SOI	RV64G1p1Xhwacha2
Raven-2	2012. 8. 22	ST 28nm FDSOI	RV64G1p1Xhwacha2
EOS18	2013. 2. 06	IBM 45nm SOI	RV64G1p1Xhwacha2
EOS20	2013. 6. 03	IBM 45nm SOI	RV64G1p99Xhwacha2

Raven-3	2013. 9. 23	ST 28nm SOI	RV64G1p99Xhwacha2
EOS20	2014. 3. 07	IBM 45nm SOI	RV64G1p9999Xhwacha3

我

们已经完成了多个 RISC-V 处理器的实现，并进行了多次硅片制造。首批 RISC-V 处理器使用 Verilog 编写，并于 2011 年在 ST 的 FDSOI 技术中制造为 Raven-1 测试芯片。由 Yunsup Lee 和 Andrew Waterman 开发了两个内核，在 Krste Asanović 的指导下完成：1) 一个带有错误检测触发器的 RV64 标量内核，2) 一个带 64 位浮点矢量单元的 RV64 内核。第一个微架构因设计周期紧张、库不成熟，非正式地称为 TrainWreck。

随后，Andrew Waterman、Rimas Avižienis 和 Yunsup Lee 在 Krste Asanović 的指导下开发了一个清晰的顺序解耦 RV64 内核微架构，继续使用铁路主题，代号 Rocket，以纪念 George Stephenson 的蒸汽机车设计。Rocket 使用 UC Berkeley 开发的硬件设计语言 Chisel 编写。Rocket 中的 IEEE 浮点单元由 John Hauser、Andrew Waterman 和 Brian Richards 开发。此后，Rocket 进行了多次改进，并在 FDSOI 技术中制造了两次（Raven-2, Raven-3），在 IBM SOI 技术中制造了五次（EOS14, EOS16, EOS18, EOS20, EOS22）用于光子学项目。当前正在开展的工作是将 Rocket 设计转变为一个参数化的 RISC-V 处理器生成器。

EOS14-EOS22 芯片包括早期版本的 Hwacha，这是一个 64 位 IEEE 浮点矢量单元，由 Yunsup Lee、Andrew Waterman、Huy Vo、Albert Ou、Quan Nguyen 和 Stephen Twigg 开发，Krste Asanović 指导。EOS16-EOS22 芯片还包括 Henry Cook 和 Andrew Waterman 开发的双核缓存一致性协议，Krste Asanović 指导。EOS14 芯片成功在 1.25 GHz 下运行，但 EOS16 芯片受 IBM 垫库中的一个错误影响。EOS18 和 EOS20 芯片在 1.35 GHz 下成功运行。

Raven 测试芯片的贡献者有 Yunsup Lee、Andrew Waterman、Rimas Avižienis、Brian Zimmer、Jaehwa Kwak、Ruzica Jevtić、Milovan Blagojević、Alberto Puggelli、Steven Bailey、Ben Keller、Pi-Feng Chiu、Brian Richards、Borivoje Nikolić 和 Krste Asanović。

EOS 测试芯片的贡献者有 Yunsup Lee、Rimas Avižienis、Andrew Waterman、Henry Cook、Huy Vo、Daiwei Li、Chen Sun、Albert Ou、Quan Nguyen、Stephen Twigg、Vladimir Stojanović 和 Krste Asanović。Andrew Waterman 和 Yunsup Lee 开发了 C++ ISA 模拟器 Spike，它作为开发中的黄金模型，并以庆祝美国跨大陆铁路完工的金钉命名。Spike 已作为 BSD 开源项目发布。Andrew Waterman 完成了一篇硕士论文，初步设计了 RISC-V 压缩指令集。

多个 RISC-V FPGA 实现已经完成，主要作为 Par Lab 项目研究静修会的集成演示的一部分。最大的 FPGA 设计由 3 个缓存一致的 RV64IMA 处理器组成，运行研究操作系统。FPGA 实现的贡献者包括 Andrew Waterman、Yunsup Lee、Rimas Avižienis 和 Krste Asanović。

RISC-V 处理器广泛应用于 UC Berkeley 的多个课程。Rocket 作为 2011 年秋季 CS250 课程的项目基础，Brian Zimmer 担任助教。在 2012 年春季的本科 CS152 课程中，Christopher Celio 使用 Chisel 编写了一个名为 Sodor 的教育版 RV32 处理器，取名自托马斯和朋友们中的岛屿。该套件包含微码内核、未流水线内核以及 2、3 和 5 级流水线内核，并以 BSD 许可证公开。随后，Christopher Celio 更新了该套件，并于 2013 年春季和 2014 年春季继续用于 CS152 课程，分别由 Yunsup Lee 和 Eric Love 担任助教。Christopher Celio 还开发了名为 B00M (Berkeley Out-of-Order Machine) 的乱序 RV64 设计，并为该设计提供了流水线可视化工具，用于 CS152 课程。CS152 课程还使用了由 Andrew Waterman 和 Henry Cook 开发的缓存一致版本的 Rocket 内核。

2013 年夏季，RoCC (Rocket Custom Coprocessor) 接口被定义，用于简化 Rocket 内核中自定义加速器的集成。Rocket 和 RoCC 接口在 2013 年秋季的 CS250 VLSI 课程中广泛使用，多个学生加速器项目基于 RoCC 接口构建。Hwacha 矢量单元被重构为 RoCC 协处理器。两名伯克利本科生 Quan Nguyen 和 Albert Ou 在 2013 年春季成功将 Linux 移植到 RISC-V。2014 年 1 月，Colin Schmidt 完成了 RISC-V 2.0 的 LLVM 后端。2014 年 3 月，Bluespec 的 Darius Rad 为 GCC 移植贡献了软浮点

ABI 支持。John Hauser 对浮点分类指令的定义做出了贡献。此外, Tommy Thorn 和 Rishiyur Nikhil 分别提供了基于 Verilog 和 Bluespec 的 RISC-V 内核实现。

37.4 致谢

感谢 Christopher F. Batten、Preston Briggs、Christopher Celio、David Chisnall、Stefan Freudenberger、John Hauser、Ben Keller、Rishiyur Nikhil、Michael Taylor、Tommy Thorn 和 Robert Watson 对 ISA 2.0 版规范草案的评论。

37.5 ISA 手册 2.1 版的历史

自 2014 年 5 月发布冻结版 2.0 以来, RISC-V ISA 的采用速度急剧增加, 发展迅速, 诸多事件使得难以全面叙述这一短暂历史。或许最为关键的一步是, 2015 年 8 月成立了非营利性的 RISC-V 基金会。该基金会现已接管了 RISC-V ISA 标准的官方管理, riscv.org 网站成为获取相关新闻和更新的主要渠道。

37.6 致谢

感谢 Scott Beamer、Allen J. Baum、Christopher Celio、David Chisnall、Paul Clayton、Palmer Dabbelt、Jan Gray、Michael Hamburg 和 John Hauser 对 2.0 版规范的评论。

37.7 ISA 手册 2.2 版的历史

37.8 致谢

感谢 Jacob Bachmeyer、Alex Bradbury、David Horner、Stefan O'Rear 和 Joseph Myers 对 2.1 版规范的评论。

37.9 ISA 手册 2.3 版的历史

RISC-V 的采用速度继续迅猛发展。John Hauser 和 Andrew Waterman 基于 Paolo Bonzini 的提案贡献了一个虚拟机 ISA 扩展。Daniel Lustig、Arvind、Krste Asanović、Shaked Flur、Paul Loewenstein、Yatin Manerkar、Luc Maranget、Margaret Martonosi、Vijayanand Nagarajan、Rishiyur Nikhil、Jonas Oberhauser、Christopher Pulte、Jose Renau、Peter Sewell、Susmit Sarkar、Caroline Trippel、Muralidaran Vijayaraghavan、Andrew Waterman、Derek Williams、Andrew Wright 和 Sizhuo Zhang 贡献了内存一致性模型。

37.10 资金支持

RISC-V 架构及实现的研发工作得到了以下资助方的部分支持:

- 并行计算实验室 (Par Lab): 研究资金来自微软 (奖项编号#024263) 和英特尔 (奖项编号#024894), 以及加州大学探索计划 (奖项编号#DIG07-10227) 的配套资助。额外支持来自 Par Lab 合作企业: 诺基亚、英伟达、甲骨文和三星。
- Isis 项目: 美国能源部奖项 DE-SC0003624。
- ASPIRE 实验室: 美国国防高级研究计划局 (DARPA) PERFECT 计划, 奖项 HR0011-12-2-0016。DARPA POEM 计划, 奖项 HR0011-11-C-0100。未来架构研究中心 (C-FAR), 由半导体研究协会 (SRC) 资助的 STARnet 中心。以及来自 ASPIRE 产业合作伙伴英特尔, 以及成员企业谷歌、华为、诺基亚、英伟达、甲骨文和三星的额外支持。

本文内容不代表美国政府立场或政策, 亦不暗示其官方认可。

附录 A: RVWMO 解释材料, V0.1

本节通过更通俗的语言和具体的例子进一步解释了 RVWMO 内存模型, 目的是帮助读者更好地理解公理和程序顺序规则的含义与意图。本附录为注释性内容; 所有规范性材料已在 `memorymodel` 部分及 ISA 规范的其他章节中提供。已知差异列在 `discrepancies` 部分, 其他差异为无意之举。

A.1 为什么选择 RVWMO?

内存一致性模型从弱到强有着不同的表现。弱一致性模型给予硬件更多的实现自由度, 通常在性能、每瓦性能、功耗、可扩展性和硬件验证开销方面优于强一致性模型, 但代价是编程模型更为复杂。而强一致性模型则提供简化的编程模型, 但限制了流水线和内存系统中可以执行的 (非推测性) 硬件优化, 导致功耗、面积和验证开销的增加。

RISC-V 采用了 RVWMO 内存模型, 它是发布一致性的一个变体, 位于内存模型范围的中间。RVWMO 使架构师能够设计各种实现, 包括简单实现、激进实现、嵌入式实现或其他任何实现方式, 同时仍能支持高性能编程语言的内存模型。

为便于其他架构的代码移植, 一些硬件实现可能会选择实现 `Ztso` 扩展, 它提供更严格的 RVTSO 顺序语义。针对 RVWMO 编写的代码可以自动兼容 RVTSO, 但 RVTSO 编写的代码不能保证在 RVWMO 实现上正确运行。实际上, 大多数 RVWMO 实现 (且应该) 会拒绝运行仅支持 RVTSO 的二进制文件。因此, 每个实现必须决定是优先考虑与 RVTSO 代码的兼容性 (例如, 支持从 x86 移植), 还是优先考虑与其他 RVWMO 实现的 RISC-V 核的兼容性。

在 RVWMO 编写的代码中, 某些屏障和内存排序注释在 RVTSO 下可能变得多余。RVWMO 在 `Ztso` 实现中, 默认将这些屏障操作转化为无操作 (如 `FENCE R, RW` 和 `FENCE RW, W`), 从而避免增加额外开销。但如果需要兼容非 `Ztso` 实现, 这些屏障操作必须保留。

A.2 试纸测试

本章的解释采用了试纸测试, 即一段小型程序, 旨在测试或突出内存模型的某个特定方面。展示了一个包含两个硬件线程的试纸测试示例。对于该图及本章接下来的所有图, 我们假设 `s0`、`s1` 和 `s2` 在所有硬件线程中预设为相同的值, 其中 `s0` 保存地址 `x`, `s1` 保存地址 `y`, `s2` 保存地址 `z`, 且 `x`、`y` 和 `z` 是对齐到 8 字节边界的不重叠内存位置。其他寄存器和所有引用的内存位置均初始化为零。试纸测试用于分析内存模型在特定情境下的表现。例如, 在下图 A.1 中, 第一硬件线程上 `a0` 的最终值可能为 2、4 或 5, 具体取决于每个硬件线程的指令流在运行时的交错情况。然而, 在这个例子中, 第一硬件线程上 `a0` 的最终值永远不会是 1 或 3; 直观地讲, 值 1 在加载指令执行时已经不可见, 而值 3 在加载指令执行时仍然不可见。接下来, 我们将分析该测试以及其他测试。

图表位于每个试纸测试的右侧, 展示了特定执行候选的可视化表示。这些图表采用在内存模型文献中常见的符号表示法, 用于约束可能导致该执行的全局内存顺序集合。该符号表示法也是本手册中 `herd` 模型的基础, 具体符号解释可参见 `litmus-key`。在列出的关系中, `rf` 边缘、`co` 边缘、`fr` 边缘和 `ppo` 边缘直接约束全局内存顺序 (`fence`、`addr`、`data` 以及某些 `ctrl` 边缘也通过 `ppo` 约束)。其他边缘 (如硬件线程内的 `rf` 边缘) 提供信息, 但不直接约束全局内存顺序。

Hart 0	Hart 1
⋮	⋮
li t1,1	li t4,4
(a) sw t1,0(s0)	(e) sw t4,0(s0)
⋮	⋮
li t2,2	
(b) sw t2,0(s0)	
⋮	⋮
(c) lw a0,0(s0)	
⋮	⋮
li t3,3	li t5,5
(d) sw t3,0(s0)	(f) sw t5,0(s0)
⋮	⋮

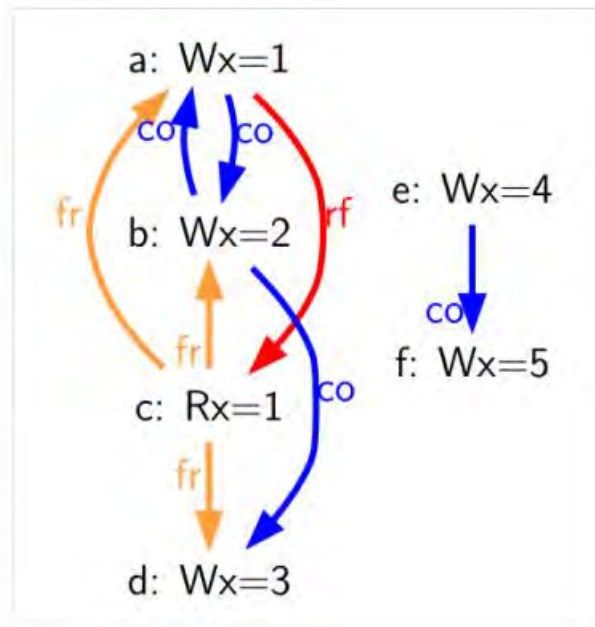


图 A. 1 Litmus测试示例与一种被禁止的执行情况 ($a0=1$)

例如，在图中， $a0=1$ 只有在以下任一条件为真时才会发生：

* (b) 出现在 (a) 之前的全局内存顺序中（并且在一致性顺序中 co）。但这违反了 RVWMO 的 PPO 规则 $ppo:-\rightarrow st$ 。从 (b) 到 (a) 的 co 边缘突出了这一矛盾。

* (a) 出现在 (b) 之前的全局内存顺序中（并且在一致性顺序中 co）。然而，这违反了加载值公理，因为 (a) 不是在程序顺序中 (c) 之前的最新匹配存储。从 (c) 到 (b) 的 fr 边缘突出了这一矛盾。

表 A. 1 试纸测试图的符号说明

简称	全名
rf	从每个存储读取返回由该存储写入的值的加载
co	一致性（存储到每个地址的总顺序）
fr	from - reads（从每次加载到加载返回值的存储的共同后继者）
ppo	保留节目令
fence	由 FENCE 指令执行的命令
addr	地址的依赖
ctrl	控制依赖
data	数据依赖

由于这两种情况都不满足 RVWMO 公理， $a0=1$ 结果被禁止的。



试纸测试库还提供了如何在 RISC-V 硬件上运行试纸测试以及如何将结果与操作模型和公理模型进行比较的说明。

未来，我们预计将这些内存模型试纸测试适配为 RISC-V 合规性测试套件的一部分。

A.3 解释 RVWMO 规则

本节提供了对所有 RVWMO 规则和公理的解释和示例。

A.3.1 保留程序顺序和全局内存顺序

保留程序顺序表示在全局内存顺序中必须遵循的程序顺序子集。具体来说，如果同一硬件线程中的事件按保留程序顺序排序，它们必须其他硬件线程和/或观察者的角度按照这一顺序出现；而如果事件不受保留程序顺序限制，它们可以在其他硬件线程和/或观察者的角度进行重新排序。

非正式地讲，全局内存顺序表示加载和存储执行的顺序。尽管当前的内存模型文献已不再以执行为基础来构建规范，但这一概念对帮助直观理解内存模型仍然至关重要。加载执行指的是加载指令的返回值已确定，而存储执行则不仅指指令已经在流水线中执行，更意味着它的值已传播到全局可见的内存中。从这一角度来看，全局内存顺序代表了内存系统和一致性协议的作用，它们将每个硬件线程发出的（可能已重新排序的）内存访问交织成一个全体硬件线程一致同意的全局顺序。

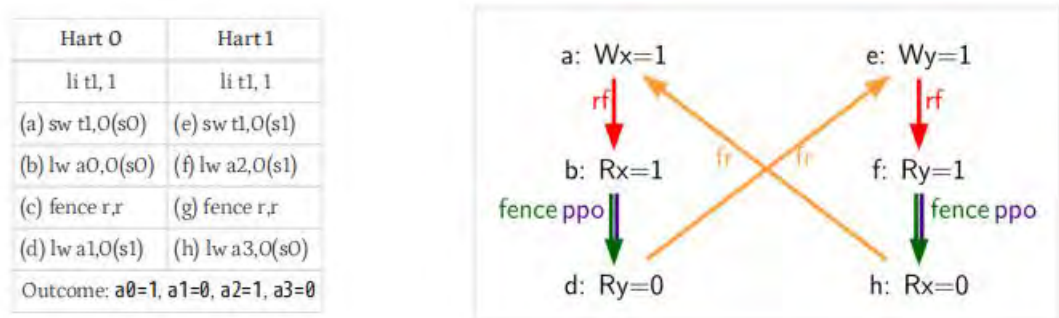
加载执行的顺序并不总是直接反映两个加载返回值的相对时间顺序。特别地，加载 b 可能在加载 a 之前执行（即 b 可能先于 a 执行，并且在全局内存顺序中 b 可能排在 a 之前），但 a 仍然可能返回比 b 更旧的值。这种不一致现象反映了（其中之一）缓冲区对核和内存交互的影响。例如，b 可能从存储缓冲区返回一个值，而 a 则忽略了这个更新的存储，反而从内存读取到更旧的值。为了考虑这种情况，每次加载执行时，返回的值是由加载值公理所决定的，而不仅仅是严格依赖于全局内存顺序中最接近的存储。

A.3.2 加载值公理

第 17.1.4 章节所述：每个加载 i 返回的每个字节的值是由以下商定的、在全局内存顺序中最新的写入该字节的存储决定的：

- 写入该字节且在全局内存顺序中先于 i 的存储
- 写入该字节且在程序顺序中先于 i 的存储

保留程序顺序不必严格遵守一个存储后跟加载的顺序，只要它们访问的是重叠地址。这种复杂性源于几乎所有实现中普遍存在存储缓冲区。非正式地说，加载可能通过从存储缓冲区转发返回值，虽然该存储仍停留在缓冲区中，并未写回全局内存。因此，其他硬件线程可能会观察到加载在存储之前执行。



图A.2 分析存储缓冲区转发测试案例

如图 A.2。在含有存储缓冲区的实现上运行该程序时，可能会得到最终结果 a0=1，a1=0，a2=1，a3=0，其执行过程如下：

存储缓冲区转发试纸测试（允许的结果）

- * (a) 执行并进入第一个硬件线程的私有存储缓冲区

- * (b) 执行并从 (a) 中通过存储缓冲区转发其返回值 1
- * (c) 执行，因为之前的所有加载（即 (b)）都已完成
- * (d) 执行并从内存中读取值 0
- * (e) 执行并进入第二个硬件线程的私有存储缓冲区
- * (f) 执行并从 (e) 中通过存储缓冲区转发其返回值 1
- * (g) 执行，因为之前的所有加载（即 (f)）都已完成
- * (h) 执行并从内存中读取值 0
- * (a) 从第一个硬件线程的存储缓冲区排出到内存
- * (e) 从第二个硬件线程的存储缓冲区排出到内存

因此，内存模型必须能够解释这种行为。换句话说，假设保留程序顺序的定义包括以下假设规则：如果内存访问 a 在程序顺序中先于内存访问 b，并且 a 和 b 访问相同的内存位置，a 是写操作，b 是读操作，那么 a 在保留程序顺序中先于 b（因此也在全局内存顺序中先于 b）。称之为规则 X。那么我们得到以下结果：

- * (a) 先于 (b)：根据规则 X
- * (b) 先于 (d)：根据规则 4

* (d) 先于 (e)：根据加载值公理。否则，如果 (e) 先于 (d)，那么 (d) 将被要求返回 1。（这是一个完全合法的执行；只是这不是我们讨论的执行）

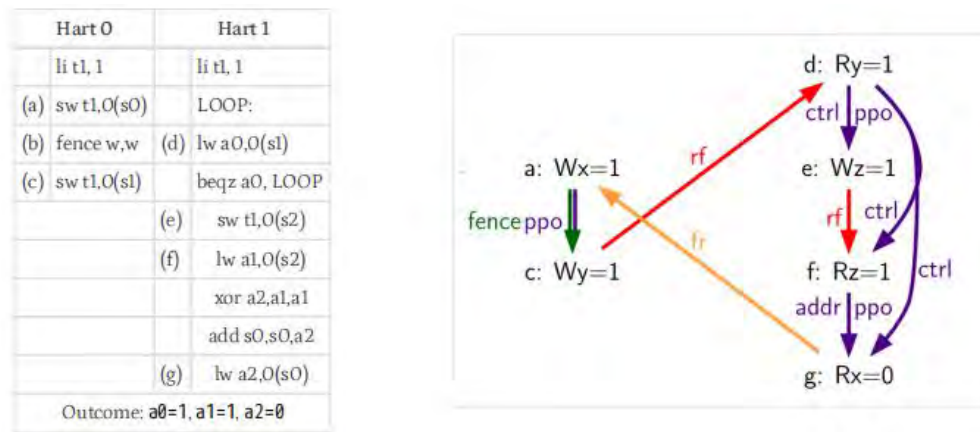
- * (e) 先于 (f)：根据规则 X
- * (f) 先于 (h)：根据规则 4
- * (h) 先于 (a)：根据加载值公理，如上所述。

全局内存顺序必须是一个全序，而不是循环的，因为循环意味着每个事件都发生在自身之前，这是不可能的。因此，提议的执行将被禁止，添加规则 X 会禁用具有存储缓冲区转发的实现，显然这是不可接受的。

尽管如此，即使在全局内存顺序中 (b) 先于 (a) 和/或 (f) 先于 (e)，在这个例子中唯一合理的情况是 (b) 返回 (a) 写入的值，同样 (f) 返回 (e) 写入的值。这种组合符合加载值公理定义中的第二种选择。即便 (b) 在全局内存顺序中先于 (a)，由于 (a) 在 (b) 执行时仍在存储缓冲区中，(a) 对 (b) 仍然可见。因此，尽管 (b) 先于 (a)，(b) 应该返回 (a) 写入的值，因为在程序顺序中 (a) 先于 (b)。类似地，这也适用于 (e) 和 (f)。

另一个用于测试存储缓冲区行为的测试如 litmusppoca 所示。在这个例子中，由于控制依赖性，(d) 在 (e) 之前排序，由于地址依赖性，(f) 在 (g) 之前排序。然而，(e) 不一定在 (f) 之前排序，即使 (f) 返回 (e) 写入的值。这可能对应于以下事件顺序：

- * (e) 推测性执行并进入第二个硬件线程的私有存储缓冲区（但不排出到内存）
- * (f) 推测性执行并从存储缓冲区中的 (e) 转发其返回值 1
- * (g) 推测性执行并从内存中读取值 0
- * (a) 执行，进入第一个硬件线程的私有存储缓冲区，并排出到内存
- * (b) 执行并退役
- * (c) 执行，进入第一个硬件线程的私有存储缓冲区，并排出到内存
- * (d) 执行并从内存中读取值 1
- * (e)、(f) 和 (g) 提交，因为推测结果是正确的
- * (e) 从存储缓冲区排出到内存



图A. 3 PPOCA存储缓冲区转发测试案例（允许的结果）

A. 3. 3 原子性公理

ax-atom 原子性公理（对齐原子操作）：如果 r 和 w 是硬件线程 h 中通过对齐的 LR 和 SC 指令生成的成对加载和存储操作，且 s 是对字节 x 的存储，且 r 返回 s 写入的值，那么 s 必须在全局内存顺序中位于 w 之前，并且 s 和 w 之间在全局内存顺序中不能存在来自 h 以外硬件线程对字节 x 的存储。

RISC-V 架构将原子性概念与排序概念分离。与 TS0 等架构不同，RISC-V 原子操作在 RVWMO 下默认不强制任何排序要求。排序语义仅由适用的 PPO 规则保证。

RISC-V 包含两种类型的原子操作：AMO 和 LR/SC 对。这两者在概念上表现不同。LR/SC 表现为旧值被带到核，修改，然后写回内存，同时对该内存位置保持保留。AMO 则表现为直接在内存中执行。因此，AMO 本质上是原子的，而 LR/SC 对在原子性方面略有不同，即在原硬件线程保持保留期间，内存位置不会被其他硬件线程修改。

表 A. 2 内存位置表

(a) lr.d a0, 0(s0)	(a) lr.d a0, 0(s0)	(a) lr.w a0, 0(s0)	(a) lr.w a0, 0(s0)
(b) sd t1, 0(s0)	(b) sw t1, 4(s0)	(b) sw t1, 4(s0)	(b) sw t1, 4(s0)
(c) sc.d t3, t2, 0(s0)	(c) sc.d t3, t2, 0(s0)	(c) sc.w t3, t2, 0(s0)	(c) addi s0, s0, 8
(d) sc.w t3, t2, 8(s0)			

原子性公理禁止其他硬件线程的存储在全局内存顺序中插入到 LR 和与该 LR 配对的 SC 之间。然而，它不禁止加载在程序顺序或全局内存顺序中插入到配对操作之间，也不禁止来自同一硬件线程的存储，或者对非重叠内存位置的存储在全局内存顺序中插入到配对操作之间。例如，在表 A. 2 中，SC 指令可能（但并不保证）成功。成功的原因是插入的非条件存储来自与配对加载和条件存储指令相同的硬件线程。因此，内存系统不会导致条件存储指令失败，即便它与保留的内存位置共享同一缓存行的其他部分。

原子性公理技术上允许 LR 和 SC 触及不同地址或使用不同的访问大小，虽然这种情况在实践中不常见。LR/SC 对之间存储重叠于 LR 或 SC 引用的内存位置的情况，也远少于存储仅落在同一缓存行上的情况。

A. 3. 4 进展公理

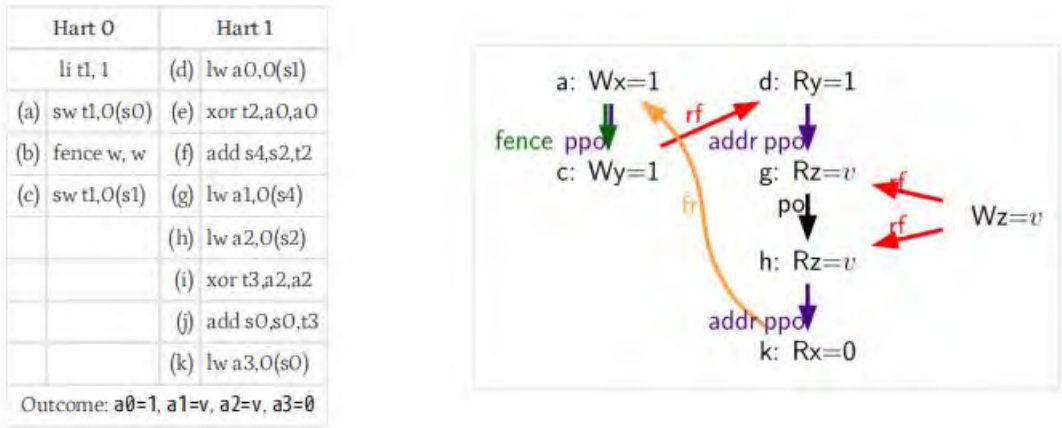
图 A. 4：在全局内存顺序中，任何内存操作之前都不能有无限序列的其他内存操作。

进展公理确保了最小的持续进展。它确保一个硬件线程的存储将在有限时间内最终对系统中的其他

硬件线程可见，并且其他硬件线程的加载最终能够读取这些值（或其后继）。没有这个规则，例如，一个自旋锁可能会无限期地在一个值上旋转，即使有另一个硬件线程的存储等待解锁自旋锁。

进展公理旨在不对 RISC-V 实现中的硬件线程施加任何其他公平性、延迟或服务质量的观念。任何更强的公平性概念由 ISA 的其余部分和/或平台和/或设备定义和实现。

在几乎所有情况下，标准缓存一致性协议将自然满足持续进展公理。具有非一致性缓存的实现可能需要提供其他机制，以确保所有存储（或其后继）最终对所有硬件线程可见。



图A. 4 RSW测试

A. 3. 5 重叠地址排序（Rules 1-3）

Rule 1: b 是存储，a 和 b 访问重叠的内存地址。

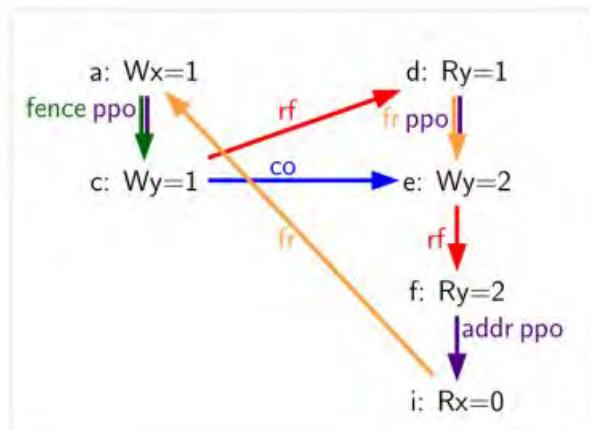
Rule 2: a 和 b 是加载，x 是 a 和 b 都读取的字节，在程序顺序中 a 和 b 之间没有对 x 的存储，并且 a 和 b 返回由不同内存操作写入的 x 的值。

Rule 3: a 是由 AMO 或 SC 指令生成的，b 是加载，并且 b 返回由 a 写入的值。

直接地址排序：加载或存储永远不能与后来的存储到重叠的内存位置重新排序。从微架构的角度来看，一般来说，如果推测是无效的，撤销推测性重新排序的存储是极为困难，因此模型不允许这种行为。另一方面，从存储到后来的加载的同地址排序不需要强制执行。如负载值公理中所述，这反映了实现从缓冲存储转发值到后续加载的可观察行为。

同地址加载-加载排序要求要微妙得多。基本要求是，较年轻的加载不得返回比同一硬件线程中较旧的加载返回的值更旧的值。这通常被称为 CoRR（加载-加载对的一致性），或作为更广泛的同一位置的顺序一致性要求的一部分。过去一些架构放宽了同地址加载-加载排序，但事后看来，这通常被认为使编程模型过于复杂，因此 RVWMO 要求强制执行 CoRR 排序。然而，由于全局内存顺序对应于加载执行的顺序，而不是返回值的顺序，因此需要一些间接方法来捕捉 CoRR 要求。

Hart 0	Hart 1
li t1, 1	li t2, 2
(a) sw t1, 0(s0)	(d) lw a0, 0(s1)
(b) fence w, w	(e) sw t2, 0(s1)
(c) sw t1, 0(s1)	(f) lw a1, 0(s1)
	(g) xor t3, a1, a1
	(h) add s0, s0, t3
	(i) lw a2, 0(s0)
Outcome: a0=1, a1=2, a2=0	



图A. 5 Litmus测试 MP+fence.w.w+fre-rfi-addr（允许的结果）

考虑 frirfi 的试纸测试，这是更一般的 fri-rfi 模式的一个特定实例。fri-rfi 一词指的是 (d)、(e)、(f) 的序列：(d) 从读取（即从早期写入读取）(e) 是同一硬件线程，并且 (f) 从 (e) 读取，它们在同一硬件线程中。

从微架构的角度来看，结果 a0=1, a1=2, a2=0 是合法的（以及其他各种不太微妙的结果）。直观地说，以下将产生所讨论的结果：

- * (d) 停顿（无论出于何种原因；可能是等待某些其他前面的指令）
- * (e) 执行并进入存储缓冲区（但尚未排出到内存）
- * (f) 执行并从存储缓冲区中的 (e) 转发
- * (g)、(h) 和 (i) 执行
- * (a) 执行并排出到内存，(b) 执行，(c) 执行并排出到内存
- * (d) 解除停顿并执行
- * (e) 从存储缓冲区排出到内存

这对应于 (f)、(i)、(a)、(c)、(d)、(e) 的全局内存顺序。注意，即使 (f) 在 (d) 之前执行，(f) 返回的值也比 (d) 返回的值更新。因此，这种执行是合法的，不违反 CoRR 要求。

同样，如果两个背靠背的加载返回由同一存储写入的值，则它们也可以在全局内存顺序中无序出现，而不会违反 CoRR。注意，这与说两个加载返回相同的值不同，因为两个不同的存储可能写入相同的值。

考虑 litmus-rsw 的试纸测试。结果 a0=1, a1=v, a2=v, a3=0（其中 v 是由另一个硬件线程写入的某个值）可以通过允许 (g) 和 (h) 重新排序来观察到。这可能是推测性完成的，并且微架构可以通过嗅探缓存失效并发现没有失效来证明这种推测是合理的，因为在 (g) 之后重放 (h) 将返回相同存储写入的值。因此，假设 a1 和 a2 最终会得到相同存储写入的值，(g) 和 (h) 可以合法地重新排序。与此执行对应的全局内存顺序将是 (h)、(k)、(a)、(c)、(d)、(g)。

在 litmus-rsw 的测试中，a1 不等于 a2 的执行确实要求 (g) 在全局内存顺序中出现在 (h) 之前。允许 (h) 在全局内存顺序中出现在 (g) 之前在这种情况下会导致违反 CoRR，因为这样 (h) 将返回比 (g) 返回的值更旧的值。因此，rule 2 禁止这种 CoRR 违规的发生。因此，rule 2 在所有情况下强制执行 CoRR 的同时，足够弱以允许在实际微架构中常见的 RSW 和 fri-rfi 模式。

还有一个重叠地址规则：rule 3 仅仅指出，在 AMO 或 SC 成功执行之前，不能将值从 AMO 或 SC 返回到后续加载。这在概念上自然地遵循 AMO 和 SC 指令旨在在内存中原子执行的观点。然而，值得注意的是，rule 3 规定硬件甚至不能非推测性地将 AMOSWAP 存储的值转发到后续加载，即使对于 AMOSWAP，该存储值实际上并不依赖于内存中的先前值，其他 AMO 也是如此。同样，即使在 SC 存储值不依赖于配对 LR 返回的值时，从 SC 存储值转发到后续加载也是如此。

上述三个 PPO 规则也适用于仅部分重叠的内存访问。例如，当使用不同大小的访问来访问同一对

象时可能会发生这种情况。还要注意，对于两个重叠的内存操作，基地址不一定相同。当使用未对齐的内存访问时，重叠地址 PPO 规则适用于每个组件内存访问。

A. 3. 6 内存屏障

Rule 4: 存在一个 FENCE 指令将 a 排在 b 之前，默认情况下，FENCE 指令确保程序顺序中栅栏之前的所有内存访问（前驱集）在全局内存顺序中出现在程序顺序中栅栏之后的内存访问（后继集）之前。然而，栅栏可以选择进一步限制前驱集和/或后继集到更小的内存访问集，以提供一些加速。具体来说，栅栏具有 PR、PW、SR 和 SW 位，这些位限制前驱集和/或后继集。前驱集仅在 PR（分别为 PW）设置时包括加载（分别为存储）。同样，后继集仅在 SR（分别为 SW）设置时包括加载（分别为存储）。

FENCE 编码目前有九种非平凡组合的四个位 PR、PW、SR 和 SW，加上一个额外的编码 FENCE.TSO，便于映射获取+释放或 RVTSO 语义。其余七种组合具有空的前驱集和/或后继集，因此是无操作的。在十种非平凡选项中，只有六种在实践中常用：

- * FENCE RW, RW
- * FENCE.TSO
- * FENCE RW, W
- * FENCE R, RW
- * FENCE R, R
- * FENCE W, W

除了 PR、PW、SR 和 SW 组合外，其他 FENCE 指令组合均为保留。我们建议程序员使用这六种组合，因为其他组合可能与内存模型产生意外或未预见的交互。

最后，由于 RISC-V 采用了多副本原子内存模型，程序员可以在每个线程的上下文中推理栅栏位。与非多副本原子内存模型中存在的栅栏累积性概念不同，这种模型不需要考虑这种复杂性。

A. 3. 7 显式同步（Rules 5-8）

Rule 5: a 具有获取注释

Rule 6: b 具有释放注释

Rule 7: a 和 b 都具有 RCsc 注释

Rule 8: a 与 b 配对

获取操作（如在关键部分开始时的操作）要求程序顺序中的所有后续内存操作在全局内存顺序中依次跟随获取操作。这保证了关键部分中的所有加载和存储都能与保护它的同步变量保持一致并保持最新。获取排序可以通过以下两种方式之一来强制：使用获取注释仅对同步变量强制排序，或者使用 FENCE R, RW 强制对所有之前的加载进行排序。

. 带有原子操作的自旋锁

```
sd          x1, (a1)      # 任意无关存储
ld          x2, (a2)      # 任意无关加载
li          t0, 1         # 初始化交换值。
again:
  amoswap.w.aq t0, t0, (a0) # 尝试获取锁。
  bnez       t0, again     # 如果被持有则重试。
  # ...
  # 关键部分。
  # ...
  amoswap.w.rl x0, x0, (a0) # 通过存储 0 释放锁。
sd          x3, (a3)      # 任意无关存储
```

```
ld          x4, (a4)      # 任意无关加载
```

因为这个例子使用了 `aq`，所以关键部分中的加载和存储在全局内存顺序中保证出现在用于获取锁的 `AMOSWAP` 之后。然而，假设 `a0`、`a1` 和 `a2` 指向不同的内存位置，关键部分中的加载和存储在全局内存顺序中可能会或可能不会出现在示例开头的任意无关加载之后。

. 带有栅栏的自旋锁

```
sd          x1, (a1)      # 任意无关存储
ld          x2, (a2)      # 任意无关加载
li          t0, 1         # 初始化交换值。
again:
  amoswap.w  t0, t0, (a0)  # 尝试获取锁。
  fence     r, rw         # 强制获取内存排序
  bnez      t0, again     # 如果被持有则重试。
  # ...
  # 关键部分。
  # ...
  fence     rw, w         # 强制释放内存排序
  amoswap.w  x0, x0, (a0)  # 通过存储 0 释放锁。
  sd        x3, (a3)      # 任意无关存储
  ld        x4, (a4)      # 任意无关加载
```

在这种情况下，即使 `AMOSWAP` 操作没有使用 `aq` 位强制排序，栅栏依然会确保 `AMOSWAP` 在全局内存顺序中出现在关键部分的所有加载和存储之前。然而，这个栅栏还会强制执行其他的排序要求：它还要求程序开头的任意无关加载在全局内存顺序中出现在关键部分的加载和存储之前（但是，这个栅栏并不要求任意无关存储在全局内存顺序中出现在关键部分的存储之前）。因此，栅栏强制的排序比 `aq` 强制的排序要宽松一些。

释放排序与获取排序完全相同，只是方向相反。释放语义要求程序顺序中所有释放操作之前的加载和存储也在全局内存顺序中出现在释放操作之前，这确保了，关键部分中的内存访问会出现在释放锁存储之前。释放语义可以通过释放注释或 `FENCE RW, W` 来强制执行。类似地，在同一个示例中，关键部分的加载和存储与代码段末尾的任意无关存储之间的排序仅由带有栅栏的自旋锁案例中的 `FENCE RW, W` 强制执行，而不是由带有原子操作的自旋锁案例中的 `rl` 强制执行。

使用 `RCpc` 注释时，存储释放到加载获取的排序不会被强制执行，这对于移植在 `TSO` 和/或 `RCpc` 内存模型下编写的代码非常有帮助。如果需要强制存储释放到加载获取的排序，代码必须使用存储释放-`RCsc` 和加载获取-`RCsc` 操作，从而使 `PP0` 规则 7 适用。`RCpc` 对 `C/C++` 中的许多应用程序来说已经足够，但对于 `C/C++`、`Java` 和 `Linux` 中的其他很多用例来说则不足够，更多细节请参见 `memoryporting`，内存移植。

`PP0` 规则 8 表明 `SC` 必须在全局内存顺序中出现在其配对的 `LR` 之后，这一点自然来自于 `LR/SC` 用法的常见模式，即执行原子读-修改-写操作，因为它们之间存在数据依赖性。然而，即便存储的值在语法上不依赖于配对的 `LR` 返回值，`PP0` 规则 8 依然适用。

最后，我们强调，像栅栏一样，程序员在分析排序注释时无需担心累积性。

A. 3. 8 句法依赖 (Rules 9-11)

Rule 9: `b` 对 `a` 有句法地址依赖

Rule 10: `b` 对 `a` 有句法数据依赖

Rule 11: b 是存储操作, 且 b 对 a 有句法控制依赖

RVWMO 内存模型遵循加载到同一硬件线程后续内存操作之间的依赖关系。Alpha 内存模型以不强制执行这些依赖关系的顺序为特征, 但现代硬件和软件内存模型认为允许重排序依赖指令是混乱且不直观的。此外, 现代代码有时故意依赖于这种依赖关系, 作为一种简便的排序强制手段。

术语定义如下: 指令会将依赖关系从其源寄存器传递到目标寄存器, 前提是目标寄存器的写入值是源寄存器值的函数。对于大部分指令, 目标寄存器携带来自所有源寄存器的依赖关系。然而, 对于内存指令, 目标寄存器的写入值来自内存系统, 而不是直接从源寄存器中获取, 因此打破了依赖关系链。对于无条件跳转指令, 目标寄存器的值来自当前的 pc, 因此内存模型不会认为 pc 是源寄存器, 因此 JALR 指令 (唯一带有源寄存器的跳转) 不会携带从 rs1 到 rd 的依赖关系。

. (c) 通过 fflags 对 (a) 和 (b) 都有句法依赖, fflags 是 (a) 和 (b) 都隐式累积到的目标寄存器

(a) fadd f3, f1, f2

(b) fadd f6, f4, f5

(c) csrrs a0, fflags, x0

累积到目标寄存器, 而不是直接写入它的行为, 反映了像 fflags 之类 CSR 的特性。尤其是, 累积到寄存器并不会覆盖先前的写入或累积。例如, 在 fflags 中, (c) 对 (a) 和 (b) 都存在句法依赖。

与其他现代内存模型一致, RVWMO 内存模型使用句法依赖而非语义依赖。换句话说, 依赖关系的定义取决于不同指令访问的寄存器的身份, 而非这些寄存器的实际内容。这意味着即便计算本身似乎可以被优化掉, 也必须强制执行地址、控制或数据依赖。这样的设计确保了 RVWMO 依然能够与那些将虚假句法依赖作为轻量级排序机制的代码保持兼容。

句法地址依赖

ld a1, 0(s0)

xor a2, a1, a1

add s1, s1, a2

ld a5, 0(s1)

例如, 从第一条指令生成的内存操作到最后一条指令生成的内存操作存在句法地址依赖, 尽管 a1 XOR a1 为零, 因此对第二次加载访问的地址没有影响。

依赖关系作为轻量级同步机制的优势在于, 排序强制仅作用于特定的两条指令, 其他不相关的指令可以由实现自由地重新排序。一个替代方案是使用加载获取, 但这会强制第一条加载与所有后续指令排序。另一种替代方案是使用 FENCE R, R, 它将包括所有之前和之后的加载, 从而使得该选项的开销更大。

. 句法控制依赖

lw x1, 0(x2)

bne x1, x0, next

sw x3, 0(x4)

next: sw x5, 0(x6)

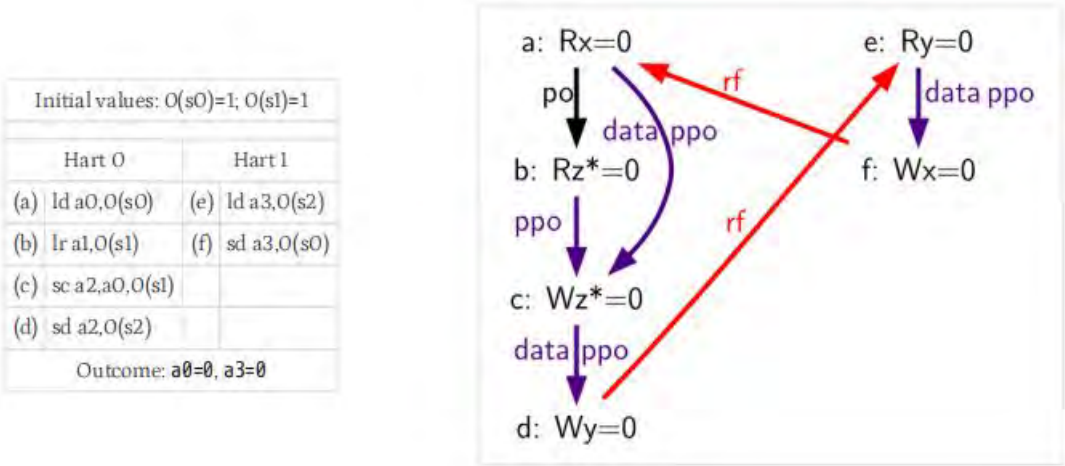
控制依赖与地址和数据依赖的行为不同, 因为控制依赖总是扩展到程序顺序中初始目标之后的所有指令。考虑 controll, next 处的指令将始终执行, 但最后一条指令生成的内存操作仍然对第一条指令生成的内存操作有控制依赖。

. 另一个句法控制依赖

```
lw x1, 0(x2)
bne x1, x0, next
next: sw x3, 0(x4)
```

同样，考虑 control2。即使两个分支结果具有相同的目标，从这个片段中的第一条指令生成的内存操作到最后一条指令生成的内存操作仍然存在控制依赖。这一定义的控制依赖比其他上下文（例如 C++）中看到的稍强，但它符合文献中控制依赖的标准定义。

值得指出的是，PPO 规则 9-11 被有意设计为遵循从成功的条件存储指令开始的依赖关系。通常，SC 指令后面会有一个条件分支，用于检查 SC 是否成功；这意味着 SC 生成的存储操作与分支后的任何内存操作之间存在控制依赖关系。PPO 规则 ppo, 11 规定，任何后续的存储操作会在全局内存顺序中出现在 SC 生成的存储操作之后。然而，由于控制、地址和数据依赖是根据内存操作定义的，且不成功的 SC 不会生成内存操作，因此不强制执行不成功的 SC 与其依赖指令之间的顺序。最后，由于 SC 仅在成功时从源寄存器将依赖关系传递到 rd，因此不成功的 SC 不会影响全局内存顺序。



图A. 6 LB测试

此外，选择尊重从条件存储指令开始的依赖关系，能够有效避免某些类似凭空出现的行为。考虑 litmusblrrsc。假设某个实现偶尔能够提前保证条件存储操作会成功。在这种情况下，(c) 可以提前返回 0 给 a2（在实际执行之前），允许序列 (d)、(e)、(f)、(a) 和 (b) 执行，然后 (c) 可能仅在那时执行（成功）。这将意味着 (c) 将其成功值写入 O(s1)！幸运的是，通过 RVWMO 内存模型尊重从成功的 SC 指令生成的存储开始的依赖关系，避免了这种情况及类似问题的出现。

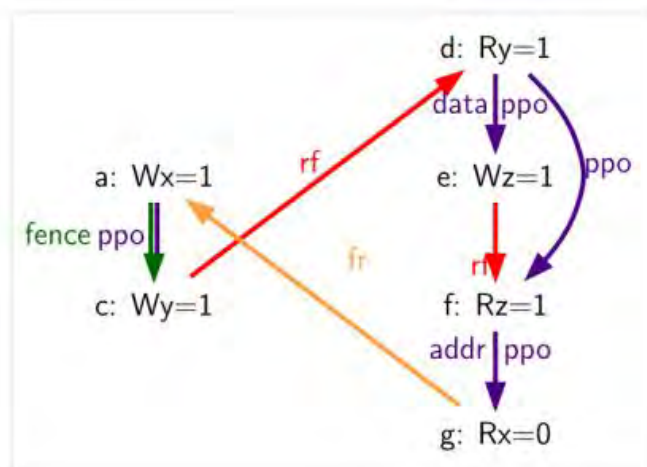
我们还要指出，指令之间的句法依赖只有在它们表现为句法地址、控制或数据依赖时才有效。例如：在 source-dest-regs 中，两个 F 指令之间的句法依赖并不意味着这两个指令必须按顺序执行。这类依赖关系仅会在稍后访问 CSR 标志时，导致从两个 F 指令到相应的 CSR 指令之间产生依赖。

A. 3. 9 流水线依赖 (Rules 12-13)

Rule 12: b 是一个加载指令，并且在程序顺序中 a 和 b 之间存在某个存储 m，m 对 a 有地址或数据依赖，并且 b 返回 m 写入的值

Rule 13: b 是一个存储指令，并且在程序顺序中 a 和 b 之间存在某个指令 m，m 对 a 有地址依赖。由于 Rule 12 和 (d) 到 (e) 的数据依赖，(d) 也必须在全局内存顺序中先于 (f)（结果禁止）

Hart 0	Hart 1
li t1, 1	(d) lw a0, 0(s1)
(a) sw t1, 0(s0)	(e) sw a0, 0(s2)
(b) fence w, w	(f) lw a1, 0(s2)
(c) sw t1, 0(s1)	xor a2, a1, a1
	add s0, s0, a2
	(g) lw a3, 0(s0)
Outcome: a0=1, a3=0	



图A. 7 由于PP0规则12以及从(d)到(e)的数据依赖性，(d)在全局内存顺序中也必须位于(f)之前（禁止结果）

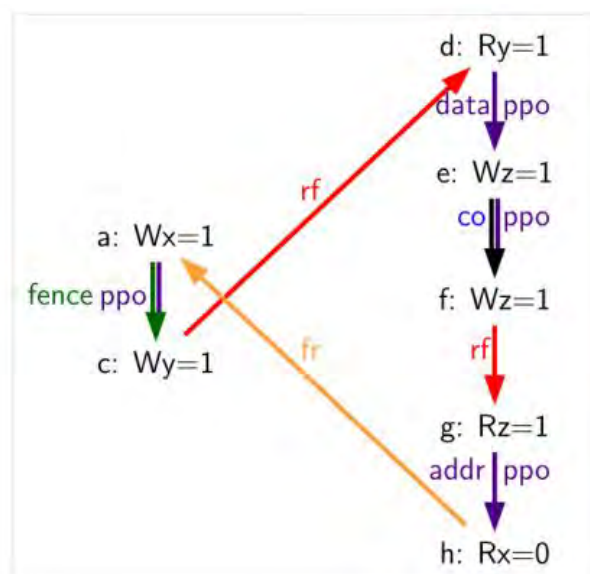
PP0 规则 12 和 13 反映了几乎所有实际处理器流水线实现的行为。规则 12 规定加载不能从存储转发，直到该存储的地址和数据已知。考虑 litmusdata rfi (f) 不能执行，直到 (e) 的数据已解析，因为 (f) 必须返回 (e) 写入的值（或全局内存顺序中更晚的值），并且在 (d) 执行之前，(e) 的写回不能覆盖旧值。因此，(f) 永远不会在 (d) 执行之前执行。

由于 (e) 和 (g) 之间的额外存储，(d) 不再需要先于 (g)（结果允许）

如果在 (e) 和 (f) 之间有另一个对相同地址的存储，如 A. 7 中所示，那么 (f) 将不再依赖于 (e) 的数据解析，因此 (f) 对 (d) 的依赖将被打破，(d) 生成 (e) 的数据。

规则 13 对前一规则做了类似的观察：存储不能在内存中执行，直到所有可能访问相同地址的先前加载都已执行。这样的加载必须在存储之前执行，但如果存储在加载有机会读取旧值之前覆盖了内存中的值，则加载不能这样做。同样，存储通常不能执行，直到知道前面的指令不会因地址解析失败而导致异常，从这个意义上说，规则 13 可以看作是规则 11 的一个特例。

Hart 0	Hart 1
li t1, 1	li t1, 1
(a) sw t1, 0(s0)	(d) lw a0, 0(s1)
(b) fence w, w	(e) sw a0, 0(s2)
(c) sw t1, 0(s1)	(f) sw t1, 0(s2)
	(g) lw a1, 0(s2)
	xor a2, a1, a1
	add s0, s0, a2
	(h) lw a3, 0(s0)
Outcome: a0=1, a3=0	



图A. 8 由于(e)和(g)之间的额外存储，(d)不再一定在(g)之前（结果允许）

考虑 A. 8 不能执行，直到 (e) 的地址解析，因为地址可能匹配；即 a1=s0。因此，在 (d) 执行并

确认地址确实重叠之前，(f) 不能发送到内存。

A. 4 超越主内存

RVWMO 目前不尝试正式描述 FENCE.I、SFENCE.VMA、I/O 栅栏和 PMA 的行为。所有这些行为将在未来的形式化中描述。与此同时，FENCE.I 的行为在 zifencei 中描述，SFENCE.VMA 的行为在 RISC-V 指令集特权架构手册中描述，I/O 栅栏和 PMA 的行为如下所述。

A. 4.1 一致性和可缓存性

RISC-V 特权 ISA 定义了物理内存属性 (PMA)，其中指定了地址空间的某些部分是否一致和/或可缓存。有关完整详细信息，请参阅 RISC-V 特权 ISA 规范。这里，我们仅讨论每个 PMA 中的各种详细信息如何与内存模型相关：

- * 主内存与 I/O 以及 I/O 内存排序 PMA：定义的内存模型适用于主内存区域。I/O 排序如下所述。
- * 支持的访问类型和原子性 PMA：内存模型仅在每个区域支持的原语之上应用。
- * 可缓存性 PMA：一般来说，可缓存性 PMA 不影响内存模型。非缓存区域的行为可能比缓存区域更严格，但无论如何，允许的行为集不会改变。然而，一些平台特定和/或设备特定的可缓存性设置可能会有所不同。
- * 一致性 PMA：标记为非一致性的内存区域的内存一致性模型目前是平台特定和/或设备特定的：加载值公理、原子性公理和进展公理都可能被非一致性内存违反。然而，一致性内存不需要硬件缓存一致性协议。RISC-V 特权 ISA 规范建议不鼓励硬件非一致性区域的主内存，但内存模型与硬件一致性、软件一致性、由于只读内存而隐含的一致性、由于只有一个代理访问而隐含的一致性或以其他方式兼容。
- * 幂等性 PMA：幂等性 PMA 用于指定加载和/或存储可能具有副作用的内存区域，这反过来用于微架构确定，例如，预取是否合法。这一区别不影响内存模型。

A. 4.2 I/O 排序

对于 I/O，加载值公理和原子性公理通常不适用，因为读取和写入可能具有设备特定的副作用，并且可能返回与最近存储到相同地址的值不同的值。然而，以下保留程序顺序规则通常仍适用于对 I/O 内存的访问：如果 a 在程序顺序中先于 b，并且以下之一成立，则 a 在全局内存顺序中先于 b：

· a 在保留程序顺序中先于 b，如 memorymodel 中定义，获取和释放排序注释仅适用于从一个内存操作到另一个内存操作以及从一个 I/O 操作到另一个 I/O 操作，但不适用于从内存操作到 I/O 操作或反之亦然

- a 和 b 是对 I/O 区域重叠地址的访问
- a 和 b 是对相同强排序 I/O 区域的访问
- a 和 b 是对 I/O 区域的访问，并且与 a 或 b 访问的 I/O 区域相关的通道是通道 1
- a 和 b 是对与相同通道（除通道 0 外）相关的 I/O 区域的访问

请注意，FENCE 指令在其前驱集和后继集中区分主内存操作和 I/O 操作。要强制 I/O 操作和主内存操作之间的排序，代码必须使用带有 PI、PO、SI 和/或 SO 以及 PR、PW、SR 和/或 SW 的 FENCE。例如，要强制主内存写入和设备寄存器的 I/O 写入之间的排序，需要 FENCE W,0 或更强的排序。

当实际使用栅栏时，实现必须假设设备可能在接收到 MMIO 信号后立即尝试访问内存，并且该设备对内存的后续内存访问必须观察到所有在该 MMIO 操作之前排序的访问的效果。换句话说，在 wo 中，假设 0(a0) 在主内存中，0(a1) 是 I/O 内存中设备寄存器的地址。如果设备在接收到 MMIO 写入后访问 0(a0)，则根据 RVWMO 内存模型的规则，该加载必须概念上出现在第一次存储到 0(a0) 之后。在某些实现中，确保这一点的唯一方法是要求第一次存储在发出 MMIO 写入之前实际完成。其他实现可能会找到更积极的方法，而其他实现可能根本不需要对 I/O 和主内存访问做任何不同的事情。然而，RVWMO 内存模型不区分这些选项；它只是提供了一种与实现无关的机制来指定必须强制执行的排序。

许多架构包括排序和完成栅栏的单独概念，特别是与 I/O（与常规主内存相对）相关。排序栅栏仅确保内存操作保持顺序，而完成栅栏确保前驱访问在任何后继可见之前都已完成。RISC-V 没有明确区分排序和完成栅栏。相反，这种区别只是从 FENCE 位的不同使用中推断出来的。

对于符合 RISC-V Unix 平台规范的实现，I/O 设备和 DMA 操作需要一致地访问内存并通过强排序 I/O 通道。因此，同时由外部设备访问的常规主内存区域的访问也可以使用标准同步机制。不符合 Unix 平台规范和/或设备不一致访问内存的实现将需要使用机制（目前是平台特定或设备特定的）来强制一致性。

地址空间中的 I/O 区域应被视为这些区域的 PMA 中的非缓存区域。如果这些区域不被任何代理缓存，则可以通过 PMA 视为一致的。

本节中的排序保证可能不适用于 RISC-V 内核和设备之间的平台特定边界之外。特别是，通过外部总线（例如 PCIe）发送的 I/O 访问可能在到达最终目的地之前重新排序。在这种情况下，必须根据这些外部设备和总线的平台特定规则强制执行排序。

A. 5 代码移植和映射指南

表 A. 3 TSO 操作到 RISC-V 操作的映射

x86/TSO 操作	RVWMO 映射
加载	<code>l {b h w d}; fence r, rw</code>
存储	<code>fence rw, w; s {b h w d}</code>
原子 RMW	<code>amo<op>. {w d}. aqrl OR loop:lr. {w d}. aq; <op>; sc. {w d}. aqrl; bnez loop</code>
栅栏	<code>fence rw, rw</code>

tsomappings 提供了 TSO 内存操作到 RISC-V 内存指令的映射。正常的 x86 加载和存储本质上都是获取-RCpc 和释放-RCpc 操作：TSO 默认强制所有加载-加载、加载-存储和存储-存储排序。因此，在 RVWMO 下，所有 TSO 加载必须映射到加载后跟 FENCE R, RW，所有 TSO 存储必须映射到 FENCE RW, W 后跟存储。TSO 原子读-修改-写和使用 LOCK 前缀的 x86 指令是完全排序的，可以通过设置 aq 和 rl 的 AMO 实现，或者通过设置 aq 的 LR、相关的算术操作、设置 aq 和 rl 的 SC 以及检查成功条件的条件分支实现。在后一种情况下，LR 上的 rl 注释实际上是多余的，可以省略。

tsomappings 的替代方案也是可行的。TSO 存储可以映射为设置 rl 的 AMOSWAP。然而，由于 RVWMO PPO 规则 3 禁止从 AMO 向后续加载转发值，使用 AMOSWAP 进行存储可能会导致性能下降。TSO 加载可以通过设置 aq 的 LR 进行映射：所有这类 LR 指令都是未配对的，但这一点并不排除使用 LR 进行加载。不过，再次强调，这种映射可能会对性能产生不利影响，特别是在它对保留机制的压力超出预期时。

表 A. 4 提供了 Power 内存操作到 RISC-V 内存指令的映射。Power ISYNC 在 RISC-V 上映射到 FENCE.I 后跟 FENCE R, R；后者的栅栏是必需的，因为 ISYNC 用于定义 RVWMO 中不存在的控制+控制栅栏依赖关系。

表 A. 4 映射关系

Power 操作	RVWMO 映射
加载	<code>l {b h w d}</code>
加载-保留	<code>lr. {w d}</code>
存储	<code>s {b h w d}</code>

存储-条件	sc. {w d}
lwsync	fence. tso
sync	fence rw, rw
isync	fence. i; fence r, r

armmappings 提供了从 ARM 内存操作到 RISC-V 内存指令的映射。由于 RISC-V 目前没有带有 `aq` 或 `rl` 注释的普通加载和存储操作码，ARM 的加载-获取和存储-释放操作应使用 `fence` 来映射。此外，为了强制存储-释放到加载-获取的顺序，在存储-释放和加载-获取之间必须有一个 `FENCE RW, RW`；armmappings 通过在每个获取操作前始终放置 `fence` 来强制执行这一点。ARM 的加载-独占和存储-独占指令同样可以映射到它们的 RISC-V LR 和 SC 等价物，但我们不在带有 `aq` 设置的 LR 前放置 `FENCE RW, RW`，而是简单地也设置 `rl`。ARM 的 ISB 在 RISC-V 上映射为 `FENCE.I`，然后是 `FENCE R, R`，类似于 Power 的 `ISYNC` 映射。

表 A.5 从 Linux 内存原语到 RISC-V 原语的映射

ARM 操作	RVWMO 映射
Load	<code>l {b h w d}</code>
Load-Acquire	<code>fence rw, rw; l {b h w d}; fence r, rw</code>
Load-Exclusive	<code>lr. {w d}</code>
Load-Acquire-Exclusive	<code>lr. {w d}. aqrl</code>
Store	<code>s {b h w d}</code>
Store-Release	<code>fence rw, w; s {b h w d}</code>
Store-Exclusive	<code>sc. {w d}</code>
Store-Release-Exclusive	<code>sc. {w d}. rl</code>
<code>dmb</code>	<code>fence rw, rw</code>
<code>dmb. ld</code>	<code>fence r, rw</code>
<code>dmb. st</code>	<code>fence w, w</code>
<code>isb</code>	<code>fence. i; fence r, r</code>

关于 `linuxmappings`，其他构造（如自旋锁）应相应遵循。具有非一致性 DMA 的平台或设备可能需要额外的同步（如缓存刷新或失效机制）；目前，任何此类额外的同步措施是设备特定的。

`linuxmappings` 提供了将 Linux 内存排序宏映射到 RISC-V 内存指令的映射方案。在支持一致性 DMA 的平台上，Linux 的 `dmarmb()` 和 `dmawmb()` 将分别映射为 `FENCE R, R` 和 `FENCE W, W`，这是因为 RISC-V Unix 平台要求一致性 DMA。然而，对于具有非一致性 DMA 的平台，这些宏将映射为 `FENCE RI, RI` 和 `FENCE WO, WO`。在这类平台上，还可能需要机制来刷新或失效缓存行，这些机制通常是设备特定的，且可能在未来的 ISA 扩展中得到标准化。

Linux 的释放操作映射可能比实际所需的更强，但这是为了确保 Linux 在需要时能提供比直观映射更强的顺序保障。特别是在撰写本文时，Linux 正在积极探讨是否要求在同一硬件线程中的关键区访问和后续由相同同步对象保护的关键区访问之间，必须提供加载-加载、加载-存储和存储-存储的顺序。这种要求并非所有 `FENCE RW, W/FENCE R, RW` 映射与 `aq/rl` 映射的组合都能满足。为了解决这个问题，存在几种解决方案：

- 始终使用 `FENCE RW, W/FENCE R, RW`，不使用 `aq/rl`。这种方案能够满足基本需求，但并不理想，因为它忽视了 `aq/rl` 修饰符的作用。
- 始终使用 `aq/rl`，不使用 `FENCE RW, W/FENCE R, RW`。由于缺少带有 `aq` 和 `rl` 修饰符的加载和存

储操作码，这种方案目前不可行。

．加强释放操作的映射，使其在任何获取映射存在时，能够强制执行所需的顺序。当前推荐的解决方案正是这一方案，也是 linuxmappings 中显示的解决方案。

RVWMO 映射：(a) lw a0, 0(s0) (b) fence.tso // vs. fence rw,w (c) sd x0,0(s1) ... loop: (d) amoswap.d.aq a1,t1,0(s1) bnez a1,loop (e) lw a2,0(s2)

例如，Linux 社区正在讨论的关键区顺序规则要求 (a) 在 lkmm11 中排在 (e) 之前。如果这是必需的，(b) 映射为 FENCE RW,W 将无法提供足够的顺序保证。因此，随着 Linux 内核内存模型的进一步发展，映射可能会有所变化。

c11mappings 提供了 C11/C++11 原子操作到 RISC-V 内存指令的映射。如果引入带有 aq 和 rl 修饰符的加载和存储操作码，那么 c11mappingshypothetical 中的映射将足够。然而请注意，只有当 atomic<op>(memory_order_seq_cst) 使用同时设置了 aq 和 rl 的 LR 进行映射时，这两种映射才能正确互操作。

更重要的是，c11mappings 中的顺序一致存储，后跟 c11mappingshypothetical 中的顺序一致加载，除非通过添加第二个 fence 或将存储映射到 amoswap.rl 来加强 c11mappings 中的存储映射，否则可以重新排序。

表 A.6 从 C/C++ 原语到 RISC-V 原语的映射

C/C++ Construct	RVWMO Mapping
Non-atomic load	l {b h w d}
atomic_load(memory_order_relaxed)	l {b h w d}
atomic_load(memory_order_acquire)	l {b h w d}.aq
atomic_load(memory_order_seq_cst)	l {b h w d}.aq
Non-atomic store	s {b h w d}
atomic_store(memory_order_relaxed)	s {b h w d}
atomic_store(memory_order_release)	s {b h w d}.rl
atomic_store(memory_order_seq_cst)	s {b h w d}.rl
atomic_thread_fence(memory_order_acquire)	fence r,rw
atomic_thread_fence(memory_order_release)	fence rw,w
atomic_thread_fence(memory_order_acq_rel)	fence.tso
atomic_thread_fence(memory_order_seq_cst)	fence rw,rw
C/C++ Construct	RVWMO AMO Mapping
atomic_<op>(memory_order_relaxed)	amo<op>. {w d}
atomic_<op>(memory_order_acquire)	amo<op>. {w d}.aq
atomic_<op>(memory_order_release)	amo<op>. {w d}.rl
atomic_<op>(memory_order_acq_rel)	amo<op>. {w d}.aqr1
atomic_<op>(memory_order_seq_cst)	amo<op>. {w d}.aqr1
C/C++ Construct	RVWMO LR/SC Mapping
atomic_<op>(memory_order_relaxed)	lr. {w d}; <op>; sc. {w d}
atomic_<op>(memory_order_acquire)	lr. {w d}.aq; <op>; sc. {w d}
atomic_<op>(memory_order_release)	lr. {w d}; <op>; sc. {w d}.rl
atomic_<op>(memory_order_acq_rel)	lr. {w d}.aq; <op>; sc. {w d}.rl
atomic_<op>(memory_order_seq_cst)	lr. {w d}.aq* <op>; sc. {w d}.rl

任何 AMO 都可以通过 LR/SC 对来模拟，但要确保从 LR 开始的 PPO 顺序会从 SC 开始，并且从

SC 结束的 PPO 顺序会在 LR 结束时完成。LR 需要遵循 AMO 的数据依赖性，因为加载操作本身并没有数据依赖的概念。同时，确保同一硬件线程中的其他 FENCE R,R 操作也对 SC 起作用，否则 SC 无法遵守该 fence。模拟器可以通过将 AMO 映射为 `lr.aq; <op>; sc.aqrl` 来实现这一点，从而与其他用于完全有序原子操作的映射方式保持一致。

这些 C11/C++11 映射要求平台为所有内存提供以下物理内存属性(如 RISC-V 特权 ISA 中定义)：

- * 主内存
- * 一致性
- * AMOArithmetic
- * RsrEventual

具有不同属性的平台可能需要不同的映射，或需要特定平台的软件（例如，内存映射 I/O）。

A. 6 实现指南

RVWMO 和 RVTSO 内存模型允许微架构使用复杂的推测技术和其他优化手段来提升性能，而不会排除这些技术的使用。同时，这些模型并不要求硬件采用特定的缓存层次结构或一致性协议。它们主要关注可以暴露给软件的行为，允许硬件在满足内存模型规则的前提下自由选择流水线设计、一致或非一致的缓存层次结构、片上互连等。为了帮助架构师和程序员理解如何实现这些规则，本节提供了相关的指导建议。

RVWMO 和 RVTSO 都实现了多副本原子性（或类似的多副本原子性）：任何存储值一旦对发出该存储的硬件线程以外的其他硬件线程可见，就必须对所有其他硬件线程也可见。换句话说，硬件线程可以在其之前存储的值对所有硬件线程可见之前，从自己的存储中转发该值，但不能提前向其他硬件线程转发。多副本原子性可以通过多种方式实现，可能是由于缓存和存储缓冲区的物理设计，也可能是通过单写入者/多读取者缓存一致性协议，或其他机制来强制执行。

虽然多副本原子性确实对微架构施加了某些限制，但它是避免内存模型变得过于复杂的关键因素之一。例如，硬件线程不能从其他硬件线程的私有存储缓冲区中转发值，除非这种操作不会导致架构中出现新的非法行为。同样，缓存一致性协议要求，在将一个值从一个硬件线程转发到另一个硬件线程之前，必须确保其他缓存中的旧副本已被失效处理。这意味着缓存的一致性操作是顺序的，不允许提前转发数据。

当然，微架构可以通过推测执行或其他优化手段来违反这些规则，只要这些不合规的行为不会被程序员看到，并且不会影响程序的正确性。

在解释 RVWMO 中 PPO 规则时，从软件角度出发，我们期望：

- * 程序员将定期和积极地使用 PPO 规则 1 和 4-8。
- * 专家程序员将使用 PPO 规则 9-11 来加速重要数据结构的关键路径。
- * 即使是专家程序员也很少或从不直接使用 PPO 规则 2-3 和 12-13。

这些规则包括在内是为了促进常见的微架构优化(规则 2)和描述的操作形式建模方法(规则 3 和 12-13) operational。它们还促进了从具有类似规则的其他架构移植代码的过程。

我们还期望从硬件角度来看：

- * PPO 规则 1 和 3-6 反映了应该对架构师几乎没有惊喜的规则。
- * PPO 规则 2 反映了一种自然且常见的硬件优化，但这种优化非常微妙，因此值得仔细检查。
- * PPO 规则 7 可能对架构师来说并不立即明显，但它是标准的内存模型要求。

* 加载值公理、原子性公理和 PPO 规则 8-13 反映了大多数硬件实现自然会强制执行的规则，除非它们包含极端优化。当然，实现仍应确保仔细检查这些规则。硬件还必须确保语法依赖性不会被优化掉。

架构可以自由地以他们选择的任何保守方式实现任何内存模型规则。例如，硬件实现可以选择执行以下任何或所有操作：

- * 将所有 fence 解释为 FENCE RW,RW (如果涉及 I/O, 则为 FENCE IORW,IORW), 无论实际设置了哪些位
- * 将所有带有 PW 和 SR 的 fence 实现为 FENCE RW,RW (如果涉及 I/O, 则为 FENCE IORW,IORW), 因为 PW 和 SR 是四种可能的主内存排序组件中最昂贵的
- * 按 memory_porting 中描述的方式模拟 aq 和 rl
- * 强制执行所有相同地址的加载-加载排序, 即使存在诸如 fri-rfi 和 RSW 的模式
- * 禁止从存储缓冲区中的存储值转发到同一地址的后续 AMO 或 LR
- * 禁止从存储缓冲区中的 AMO 或 SC 值转发到同一地址的后续加载
- * 在所有内存访问上实现 TSO, 并忽略不包括 PW 和 SR 排序的任何主内存 fence (例如, Ztso 实现将这样做)
- * 将所有原子操作实现为 RCsc 甚至完全有序, 无论注释如何实现 RVTSO 的架构可以安全地执行以下操作:
- * 忽略所有不同时具有 PW 和 SR 的 fence (除非 fence 还对 I/O 排序)
- * 忽略除规则 4 到 7 之外的所有 PPO 规则, 因为在 RVTSO 假设下其余规则与其他 PPO 规则冗余
- * 静默存储 (即, 写入与内存位置中已存在的值相同的存储) 从内存模型的角度来看与任何其他存储行为相同。同样, 实际上不改变内存中值的 AMO (例如, AMOMAX, 其中 rs2 中的值小于内存中当前的值) 在语义上仍被视为存储操作。尝试实现静默存储的微架构必须注意确保仍然遵守内存模型, 特别是在诸如 RSW mm-overlap 的情况下, 这些情况往往与静默存储不兼容。
- * 写入可以合并 (即, 对同一地址的两个连续写入可以合并) 或替代 (即, 对同一地址的两个背靠背写入中的较早一个可以省略), 只要结果行为不以其他方式违反内存模型语义。

Hart 0	Hart 1
li t1, 3	li t3, 2
li t2, 1	
(a) sw t1,0(s0)	(d) lw a0,0(s1)
(b) fence w, w	(e) sw a0,0(s0)
(c) sw t2,0(s1)	(f) sw t3,0(s0)

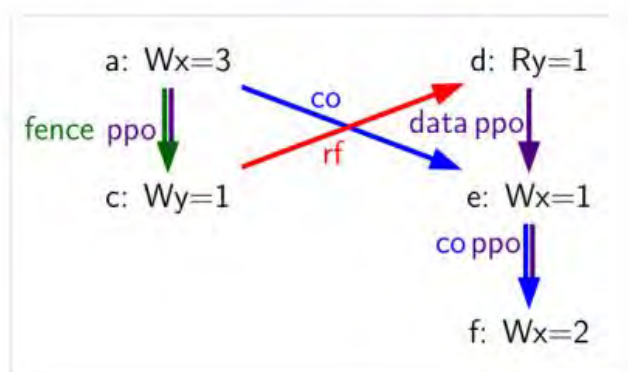


图 A.9 允许执行的指令映射

如所写, 如果加载(d)读取值 1, 则(a)必须在全局内存顺序中先于(f):

- * (a) 在全局内存顺序中先于(c), 因为规则 2
- * (c) 在全局内存顺序中先于(d), 因为加载值公理
- * (d) 在全局内存顺序中先于(e), 因为规则 7
- * (e) 在全局内存顺序中先于(f), 因为规则 1

换句话说, 地址在 s0 中的内存位置的最终值必须是 2 (由存储(f)写入的值), 而不能是 3 (由存储(a)写入的值)。

在某些情况下, 过于激进的微架构可能会错误地选择丢弃 (e), 因为 (f) 替代了它。这会导致 (d) 和 (f) 之间的依赖关系被破坏, 从而影响到 (a) 和 (f) 之间的依赖关系。这样的行为违反了内存模型规则, 因此是不被允许的。不过, 如果 (d) 和 (e) 之间没有数据依赖性, 那么写入替代的做法是合法的。

A. 6.1 可能的未来扩展

我们预计以下任何或所有可能的未来扩展都将与 RVWMO 内存模型兼容：

- * V 向量 ISA 扩展
- * J JIT 扩展
- * 带有 aq 和 rl 设置的加载和存储操作码的本地编码
- * 限制到某些地址的 fence
- * 缓存写回/刷新/失效等指令

A. 7 已知问题

Hart 0		Hart 1	
li t1, 1		li t1, 1	
(a)	lw a0,0(s0)	(d)	lw a1,0(s1)
(b)	fence rw,rw	(e)	amoswap.w.rl a2,t1,0(s2)
(c)	sw t1,0(s1)	(f)	ld a3,0(s2)
		(g)	lw a4,4(s2)
			xor a5,a4,a4
			add s0,s0,a5
		(h)	sw t1,0(s0)
Outcome: a0=1, a1=1, a2=0, a3=1, a4=0			

Hart 0		Hart 1	
li t1, 1		li t1, 1	
(a)	lw a0,0(s0)	(d)	ld a1,0(s1)
(b)	fence rw,rw	(e)	lw a2,4(s1)
(c)	sw t1,0(s1)		xor a3,a2,a2
			add s0,s0,a3
		(f)	sw t1,0(s0)
Outcome: a0=1, a1=1, a2=0			

Hart 0		Hart 1	
li t1, 1		li t1, 1	
(a)	lw a0,0(s0)	(d)	sw t1,4(s1)
(b)	fence rw,rw	(e)	ld a1,0(s1)
(c)	sw t1,0(s1)	(f)	lw a2,4(s1)
			xor a3,a2,a2
			add s0,s0,a3
		(g)	sw t1,0(s0)
Outcome: a0=1, a1=0x100000001, a2=1			

图 A.10 混合尺寸差异

在图 A.10 中显示的混合大小 RSW 变体家族中，操作规范与公理规范之间存在已知的差异。为了弥补这个差异，我们可能需要增加如下的 PPO 规则：假如内存操作 a 在程序顺序中先于内存操作 b （同时也在全局内存顺序中），则 a 必须在 b 之前执行，且 a 和 b 必须都访问常规主内存（而非 I/O 区域）， a 为加载操作， b 为存储操作，且存在一个加载操作 m 在 a 和 b 之间， a 和 m 都读取字节 x ，同时在 a 和 m 之间不会有写入 x 的存储操作，且 m 必须在 PPO 中先于 b 。换句话说，在 herd 语法中，我们可以考虑在 PPO 中加入 $(po_loc \ \& \ rsw)$ 。很多实现已经自然地强制执行这一顺序。因此，即使此规则尚未成为正式规范，我们依然建议实现者执行它，以保证未来与可能加入 RVWMO 的规则兼容。

附录 B：正式内存模型规范，V0.1

为了便于对 RVWMO 进行形式化分析，本章采用了多种工具和建模方法。任何不一致之处均为无意之举；我们期望这些模型能够准确描述相同的合法行为集。

本附录应视为注释性材料；所有规范性内容已在第 17 章以及 ISA 规范的其他主要部分中详细说明。所有已知的差异列在 A.7 章节中，任何其他差异均为无意的。

B.1 Alloy 中的正式公理规范

我们在 Alloy (<http://alloy.mit.edu>) 中提供了 RVWMO 内存模型的正式规范。该模型可在线获取，网址为 <https://github.com/daniellustig/riscv-memory-model>。

该在线材料还包含一些试纸测试，以及 Alloy 如何用于模型检查 `memory_porting` 中部分映射的示例。

```
fun ppo : Event->Event {
  po_loc :> Store
  + rdw
  + (AMO + StoreConditional) <: rfi
  + ppo_fence
  + Acquire <: ^po :> MemoryEvent
  + MemoryEvent <: ^po :> Release
  + RCsc <: ^po :> RCsc
  + pair
  + addrdep
  + datadep
  + ctrldep :> Store
  + (addrdep+datadep).rfi
  + addrdep.^po :> Store
}

fact { ppo in ^gmo }

fun candidates[r: MemoryEvent] : set MemoryEvent {
  (r.^~gmo & Store & same_addr[r]) // 在 gmo 中位于 r 之前的写操作
  + (r.^~po & Store & same_addr[r]) // 在 po 中位于 r 之前的写操作
}
```

```

fun latest_among[s: set Event] : Event { s - s.^gmo }

pred LoadValue {
  all w: Store | all r: Load |
    w->r in rf <=> w = latest_among[candidates[r]]
}

pred Atomicity {
  all r: Store.^pair | // 从 1r 开始,
  no x: Store & same_addr[r] | // 没有其他存储 x 地址相同
  x not in same_hart[r] // 且 x 不来自不同的硬件线程,
  and x in r.^rf.^gmo // 并且 x 在 gmo 中跟随 (r 读取的存储),
  and r.pair in x.^gmo // 并且 r 在 gmo 中跟随 x
}

sig Hart { // 硬件线程
  start : one Event
}

sig Address {}

abstract sig Event {
  po: lone Event // 程序顺序
}

abstract sig MemoryEvent extends Event {
  address: one Address,
  acquireRCpc: lone MemoryEvent,
  acquireRCsc: lone MemoryEvent,
  releaseRCpc: lone MemoryEvent,
  releaseRCsc: lone MemoryEvent,
  addrdep: set MemoryEvent,
  ctrldep: set Event,
  datadep: set MemoryEvent,
  gmo: set MemoryEvent, // 全局内存顺序
  rf: set MemoryEvent
}

sig LoadNormal extends MemoryEvent {} // 1{b|h|w|d}
sig LoadReserve extends MemoryEvent { // 1r
  pair: lone StoreConditional
}

sig StoreNormal extends MemoryEvent {} // s{b|h|w|d}
// 模型中的所有 StoreConditionals 都假定成功
sig StoreConditional extends MemoryEvent {} // sc
sig AMO extends MemoryEvent {} // amo
sig NOP extends Event {}

```

```

fun Load : Event { LoadNormal + LoadReserve + AMO }

```

```

fun Store : Event { StoreNormal + StoreConditional + AMO }

sig Fence extends Event {
  pr: lone Fence, // 操作码位
  pw: lone Fence, // 操作码位
  sr: lone Fence, // 操作码位
  sw: lone Fence // 操作码位
}

sig FenceTSO extends Fence {}
/* Alloy 编码细节: 操作码位要么被设置 (已编码, 例如, 作为 f.pr 在 iden 中)
 * 要么未设置 (f.pr 不在 iden 中)。这些位不能用于其他任何用途 */
fact { pr + pw + sr + sw in iden }
// 同样地, 处理顺序注释
fact { acquireRCpc + acquireRCsc + releaseRCpc + releaseRCsc in iden }
// 不要尝试通过 pr/pw/sr/sw 编码 FenceTSO; 直接按原样使用它
fact { no FenceTSO. (pr + pw + sr + sw) }

// 排序注释组
fun Acquire : MemoryEvent { MemoryEvent.acquireRCpc + MemoryEvent.acquireRCsc }
fun Release : MemoryEvent { MemoryEvent.releaseRCpc + MemoryEvent.releaseRCsc }
fun RCpc : MemoryEvent { MemoryEvent.acquireRCpc + MemoryEvent.releaseRCpc }
fun RCsc : MemoryEvent { MemoryEvent.acquireRCsc + MemoryEvent.releaseRCsc }
// 除非同时是两者, 否则不存在存储-获取或加载-释放
fact { Load & Release in Acquire }
fact { Store & Acquire in Release }
// FENCE PPO
fun FencePRSR : Fence { Fence. (pr & sr) }
fun FencePRSW : Fence { Fence. (pr & sw) }
fun FencePWSR : Fence { Fence. (pw & sr) }
fun FencePWSW : Fence { Fence. (pw & sw) }

fun ppo_fence : MemoryEvent->MemoryEvent {
  (Load <: ^po :> FencePRSR). (^po :> Load)
  + (Load <: ^po :> FencePRSW). (^po :> Store)
  + (Store <: ^po :> FencePWSR). (^po :> Load)
  + (Store <: ^po :> FencePWSW). (^po :> Store)
  + (Load <: ^po :> FenceTSO) . (^po :> MemoryEvent)
  + (Store <: ^po :> FenceTSO) . (^po :> Store)
}

// 辅助定义
fun po_loc : Event->Event { ^po & address. ~address }
fun same_hart[e: Event] : set Event { e + e. ^~po + e. ^po }
fun same_addr[e: Event] : set Event { e. address. ~address }
// 初始存储

```

```

fun NonInit : set Event { Hart.start.*po }
fun Init : set Event { Event - NonInit }
fact { Init in StoreNormal }
fact { Init->(MemoryEvent & NonInit) in ^gmo }
fact { all e: NonInit | one e.*~po.^start } // 每个事件恰好在一个硬件线程中
fact { all a: Address | one Init & a.^address } // 每个地址一个初始存储
fact { no Init <: po and no po :> Init }

// gmo
fact { total[^gmo, MemoryEvent] } // gmo 是所有 MemoryEvent 的全序
// rf
fact { rf.^rf in iden } // 每次读取仅返回一个写入的值
fact { rf in Store <: address.^address :> Load }
fun rfi : MemoryEvent->MemoryEvent { rf & (*po + *~po) }
// dep
fact { no StoreNormal <: (addrdep + ctrldep + datadep) }
fact { addrdep + ctrldep + datadep + pair in ^po }
fact { datadep in datadep :> Store }
fact { ctrldep.*po in ctrldep }
fact { no pair & (^po :> (LoadReserve + StoreConditional)).^po }
fact { StoreConditional in LoadReserve.pair } // 假设所有 SC 都成功

// rdw
fun rdw : Event->Event {
  (Load <: po_loc :> Load) // 从所有同地址的加载-加载对开始,
  - (~rf.rf) // 减去从同一存储读取的对,
  - (po_loc.rfi) // 并减去 fri-rfi 模式
}
// 过滤冗余实例与/或可视化
fact { no gmo & gmo.gmo } // 保持可视化简洁
fact { all a: Address | some a.^address }
// =可选: 操作码编码限制=
// 正式批准和定义的 fence 指令列表
fact { Fence in
  Fence.pr.sr
  + Fence.pw.sw
  + Fence.pr.pw.sw
  + Fence.pr.sr.sw
  + FenceTSO
  + Fence.pr.pw.sr.sw
}
pred restrict_to_current_encodings {
  no (LoadNormal + StoreNormal) & (Acquire + Release)
}

```

```
// =Alloy 快捷方式=
pred acyclic[rel: Event->Event] { no iden & ^rel }
pred total[rel: Event->Event, bag: Event] {
  all disj e, e: bag | e->e in rel + ^rel
  acyclic[rel]
}
....
```

B.2 Herd 中的正式公理规范

工具 herd 接收一个内存模型和一个试纸测试作为输入，并在该内存模型上模拟测试执行。内存模型用特定领域语言 Cat 编写。本节提供了两种 RVWMO 的 Cat 内存模型。第一个模型 herd2 尽可能遵循全局内存顺序，即本书第 17 章中对 RVWMO 的定义。第二个模型 herd3 是一个等效的、更高效的基于部分顺序的 RVWMO 模型。

模拟器 herd 是 diy 工具套件的一部分 — 相关软件和文档请参见 <http://diy.inria.fr>。模型和更多内容可在线访问 <http://diy.inria.fr/cats7/riscv/>。

```
let fence.r.r = [R];fencerel(Fence.r.r);[R]
let fence.r.w = [R];fencerel(Fence.r.w);[W]
let fence.r.rw = [R];fencerel(Fence.r.rw);[M]
let fence.w.r = [W];fencerel(Fence.w.r);[R]
let fence.w.w = [W];fencerel(Fence.w.w);[W]
let fence.w.rw = [W];fencerel(Fence.w.rw);[M]
let fence.rw.r = [M];fencerel(Fence.rw.r);[R]
let fence.rw.w = [M];fencerel(Fence.rw.w);[W]
let fence.rw.rw = [M];fencerel(Fence.rw.rw);[M]
let fence.tso =
  let f = fencerel(Fence.tso) in
    ([W];f;[W]) | ([R];f;[M])
let fence =
  fence.r.r | fence.r.w | fence.r.rw |
  fence.w.r | fence.w.w | fence.w.rw |
  fence.rw.r | fence.rw.w | fence.rw.rw |
  fence.tso
(* 相同地址之间没有对相同地址的写操作 *)
let po-loc-no-w = po-loc \ (po-loc?;[W];po-loc)
(* 读取相同写入 *)
let rsw = rf^-1;rf
(* 获取，或更强 *)
let AQ = Acq|AcqRel
(* 释放或更强 *)
and RL = RelAcqRel
(* 所有 RCsc *)
let RCsc = Acq|Rel|AcqRel
(* Amo 事件既是 R 也是 W，关系 rmw 关联配对的 lr/sc *)
let AMO = R & W
```

```

let StCond = range(rmw)

(*****)
(* ppo 规则 *)
(*****)

(* 重叠地址排序 *)
let r1 = [M];po-loc;[W]
and r2 = ([R];po-loc-no-w;[R]) \ rsw
and r3 = [AMO|StCond];rfi;[R]
(* 显式同步 *)
and r4 = fence
and r5 = [AQ];po;[M]
and r6 = [M];po;[RL]
and r7 = [RCsc];po;[RCsc]
and r8 = rmw
(* 句法依赖 *)
and r9 = [M];addr;[M]
and r10 = [M];data;[W]
and r11 = [M];ctrl;[W]
(* 管道依赖 *)
and r12 = [R];(addr|data);[W];rfi;[R]
and r13 = [R];addr;[M];po;[W]
let ppo = r1 | r2 | r3 | r4 | r5 | r6 | r7 | r8 | r9 | r10 | r11 | r12 | r13

(* 注意 herd 已定义其自己的 rf 关系 *)
(* 定义 ppo *)
include riscv-defs.cat
(*****)
(* 生成全局内存顺序 *)
(*****)

let gmo0 = (* 前体: 即构建 gmo 作为包含 gmo0 的全序 *)
  loc & (W\FW) * FW | # 任何写入后的最终写入到相同位置
  ppo | # ppo 兼容
  rfe | # 包括 herd 外部 rf (优化)

(* 遍历 gmo0 的所有线性扩展 *)
with gmo from linearizations(M\IW,gmo0)
(* 添加初始写入在前 — 便于计算 rfGMO *)
let gmo = gmo | loc & IW * (M\IW)
(*****)
(* 公理 *)
(*****)

```

```

(* 根据加载值公理计算 rf, 即 rfGMO *)
let WR = loc & ([W];(gmo|po);[R])
let rfGMO = WR \ (loc&([W];gmo);WR)
(* 检查 herd rf 和 rfGMO 的相等性 *)
empty (rf\rfGMO)|(rfGMO\rf) as RfCons
(* 原子性公理 *)
let infloc = (gmo & loc)^-1
let inflocext = infloc & ext
let winside = (infloc;rmw;inflocext) & (infloc;rf;rmw;inflocext) & [W]
empty winside as Atomic

```

```

(*****)
(* 定义 *)
(*****)
(* 定义 ppo *)
include riscv-defs.cat
(* 计算一致性关系 *)
include cos-opt.cat
(*****)
(* 公理 *)
(*****)
(* 每个位置的 Sc *)
acyclic co|rf|fr|po-loc as Coherence
(* 主要模型公理 *)
acyclic co|rfe|fr|ppo as Model
(* 原子性公理 *)
empty rmw & (fre;coe) as Atomic

```

B.3 操作内存模型

这是 RVWMO 内存模型操作风格的替代表述，旨在实现与公理化表述完全相同的扩展行为：对于任何给定的程序，只有当公理化表述允许时，才允许执行该程序。

公理化表述定义了完整候选执行的谓词。相比之下，操作风格表述则具备抽象的微架构特点：它被表示为一个状态机，其中的状态是硬件机器状态的抽象表示，且显式包括乱序执行和推测执行（但省略了更多与实现特定的微架构细节，如寄存器重命名、存储缓冲区、缓存层次结构、缓存协议等）。因此，操作风格表述提供了有用的直觉，并支持逐步构造执行，允许在更大规模的示例中交互式地和随机地探索行为，而公理化模型则需要完整的候选执行以检查公理。

操作风格表述还包括混合大小的执行，可能涉及不同 2 的幂字节大小的重叠内存访问。未对齐的访问会被拆分为单字节访问。

操作模型与 RISC-V ISA 语义的一个片段已集成到 rmem 探索工具中 (<https://github.com/rems-project/rmem>)。rmem 通过穷举、伪随机和交互式的方式来探索试纸测试和小型 ELF 二进制文件。在 rmem 中，ISA 语义使用 Sail 显式表示（关于 Sail 语言，请参见 <https://github.com/rems-project/sail>，关于 RISC-V ISA 模型，请参见 <https://github.com/rems-project/sail-riscv>），并且并发语义通过 Lem 表达（关于 Lem 语言，请

参见 <https://github.com/rem-s-project/lem>）。

rmem 提供命令行和网页两种界面。网页界面完全在客户端运行，并在线提供，包含一系列试纸测试库：<http://www.cl.cam.ac.uk/>。命令行界面在穷举模式下运行更快，特别是在大型数据集上。

下面是对模型状态和转换的非正式介绍，正式模型描述将在下一小节开始。

术语：与公理化表述不同，这里每个内存操作要么是加载，要么是存储。因此，原子内存操作（AMO）引发加载和存储两种内存操作。当与 instruction 结合时，术语 load 和 store 指的是触发这些操作的指令，因此 AMO 指令也包括在内。术语 acquire 指带有 acquire-RCpc 或 acquire-RCsc 注释的指令。术语 release 指带有 release-RCpc 或 release-RCsc 注释的指令（或其内存操作）。

模型状态

模型状态：一个模型状态由共享内存和一组硬件线程状态组成。

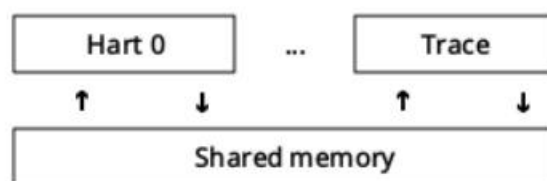


图 B.1 模型状态

共享内存状态记录了迄今为止所有已生效的内存存储操作，并按它们生效的顺序存储（虽然可以通过优化提升效率，但为了简化展示，我们保持这种方式）。

每个硬件线程的状态主要由一棵指令实例树构成，其中一些指令实例已完成，另一些尚未完成。未完成的指令实例可能会被重启，尤其是当它们依赖的乱序执行或推测执行的加载操作最终被判定为无效时。

条件分支和间接跳转指令可能在指令树中有多个后继节点。当此类指令执行完成时，任何未执行的分支路径会被丢弃。

指令树中的每个指令实例都有一个状态，表示该指令的内部指令语义的执行状态（即该指令的 ISA 伪代码）。模型使用 Sail 对内部指令语义进行形式化。指令的执行状态可以视为伪代码控制状态、伪代码调用栈和局部变量值的表示。指令实例的状态还包括该实例的内存和寄存器占用情况、寄存器的读写操作、内存操作、是否完成等信息。

模型为每个模型状态定义了一个允许的转换集合，每个转换都表示一个原子步骤，进而将模型状态转换为新的抽象机器状态。单个指令的执行通常会涉及多个转换，这些转换可能与来自其他指令的转换交织在一起。每个转换来源于单一指令实例，并会改变该实例的状态，可能依赖或改变其他硬件线程的状态和共享内存状态，但不会依赖或改变其他硬件线程的状态。接下来将介绍这些转换，并在 transitions 中定义，每个转换都有前置条件和后转换模型状态的构造。

所有指令的转换：

- * fetch，取指：这个转换表示一个新的指令实例的取指和解码，作为之前已取指指令实例（或初始取指地址）的程序顺序后继。

模型假设指令内存是固定的；它不描述自修改代码的行为。特别地，fetch，取指 转换不会生成内存加载操作，且共享内存不参与该转换。相反，模型依赖于一个外部预言机（oracle），当给定该预言机内存位置时，它将提供一个操作码。

- * reg_write，寄存器写入：这是对寄存器值的写操作。

*reg_read, 寄存器读取: 这是从最近的程序顺序前驱指令实例读取寄存器值, 该前驱指令实例向该寄存器写入数据。

*sail_interp, 伪代码内部步骤: 这涵盖了伪代码内部的计算: 算术运算、函数调用等。

*finish, 完成指令: 指令的伪代码一旦执行完毕, 指令无法重启, 内存访问无法丢弃, 所有内存效果均已生效。对于条件分支和间接跳转指令, 任何不涉及写入 pc 寄存器的地址获取的后继指令将被丢弃, 并且其下的指令实例树也将被丢弃。

特定于加载指令的转换:

*initiate_load, 启动内存加载操作: 此时加载指令的内存足迹已暂时确定 (如果之前的指令被重启, 则可能会发生变化), 并且其各个内存加载操作可以开始生效。

*sat_from_forwarding, 通过从未传播的存储中转发来满足内存加载操作: 通过从程序顺序前的内存存储操作转发, 部分或完全满足单个内存加载操作。

*sat_from_mem, 从内存中满足内存加载操作: 这完全满足了来自内存的单个内存加载操作的所有未完成部分。

*complete_loads, 完成加载操作: 此时, 指令的所有内存加载操作已被完全满足, 指令的伪代码可以继续执行。加载指令在此之前可能会被重启。但在某些条件下, 模型可能会将加载指令视为不可重启, 即使在它完成之前。

特定于存储指令的转换:

*initiate_store_footprint, 启动内存存储操作的足迹: 此时存储的内存足迹已暂时确定。

*instantiate_store_value, 实例化内存存储操作的值: 此时, 内存存储操作已具备值, 程序顺序后继的加载操作可以通过从这些存储中转发来满足。

*commit_stores, 提交存储指令: 此时, 存储操作已被确保执行 (指令不再可重启或丢弃), 并可以开始将其传播至内存。

*prop_store, 传播存储操作: 将单个内存存储操作传播到内存。

*complete_stores, 完成存储操作: 此时, 指令的所有内存存储操作已传播到内存, 指令伪代码可以继续执行。

特定于 sc 指令的转换:

*early_sc_fail, 早期 SC 失败: 导致 sc 失败, 可能是自发失败, 也可能是因为它没有与程序顺序前的 lr 配对。

*paired_sc, 配对的 SC: 此转换表示 sc 与 lr 配对, 可能会成功。

*commit_sc, 提交并传播 SC 的存储操作: 这是 commit_stores, 提交存储指令 和 prop_store, 传播存储操作 转换的原子执行, 只有在 lr 读取的存储没有被覆盖的情况下才会启用。

*late_sc_fail, 晚期 SC 失败: 导致 sc 失败, 可能是自发失败, 也可能是因为 lr 读取的存储已被覆盖。

特定于 AMO 指令的转换:

*do_amo, 使满足、提交并传播 AMO 的操作: 这是满足加载操作、执行所需算术运算, 并传播存储操作所需的所有转换的原子执行。

特定于 fence 指令的转换:

*commit_fence, 提交 fence 指令

标记为 circ 的转换可以在其前置条件满足时立即执行, 而无需排除其他行为; 而 bullet 则不可以。尽管 fetch, 取指 标记为 bullet, 但只要它没有被执行无限次, 它也可以立即执行。

一个非 AMO 加载指令的实例, 在被提取后, 通常会经历以下转换顺序:

- . reg_read, 寄存器读取
- . initiate_load, 启动内存加载操作
- . sat_by_forwarding, 通过从未传播的存储中转发来满足内存加载操作 和/或 sat_from_mem, 从内存中满足内存加载操作（根据需要满足实例的所有加载操作）
- . complete_loads, 完成加载操作
- . reg_write, 寄存器写入
- . finish, 完成指令

在上述转换之前、之间和之后，可能会出现任何数量的 `sail_interp`, 伪代码内部步骤 转换。此外，直到被执行之前，`fetch`, 取指 转换将一直可用，用于提取下一个程序位置的指令。

操作模型的非正式描述部分已结束。接下来的章节将描述正式的操作模型。

B. 3.1 指令内部伪代码执行

每个指令实例的内部指令语义通过状态机表示，主要执行指令伪代码。给定伪代码的执行状态，状态机计算并跳转到下一个状态。大部分状态表示伪代码请求的待处理内存或寄存器操作，这些操作是内存模型必须完成的。状态如下（这是一组标签联合，标签为小写字母）：

表 B. 1 指令对应

<code>Load_mem(kind, address, size, load_continuation)</code>	内存加载操作
<code>Early_sc_fail(res_continuation)</code>	允许 <code>sc</code> 提前失败
<code>Store_ea(kind, address, size, next_state)</code>	内存存储有效地址
<code>Store_memv(mem_value, store_continuation)</code>	内存存储值
<code>Read_reg(reg_name, read_continuation)</code>	寄存器读取
<code>Write_reg(reg_name, reg_value, next_state)</code>	寄存器写入
<code>Internal(next_state)</code>	伪代码内部步骤
<code>Done</code>	伪代码结束

在这里：

- * `mem_value` 和 `reg_value` 是字节列表；
- * `address` 是一个 `XLEN` 位的整数；对于加载/存储，`kind` 标识它是 `lr/sc`, `acquire-RCpc/release-RCpc`, `acquire-RCsc/release-RCsc`, `acquire-release-RCsc`；
- * 对于 `fence`, `kind` 标识它是普通的 `fence` 还是 `TSO`（Total Store Order, 总存储顺序），并且（对普通 `fence`）指定前驱和后继排序位；
- * `reg_name` 标识一个寄存器及其切片（起始和结束位索引）；继续部分描述了指令实例如何在内存模型提供的每个可能值上继续执行（`load_continuation` 和 `read_continuation` 分别获取从内存加载和从先前寄存器写入读取的值，`store_continuation` 对于失败的 `sc` 返回 `false`, 在所有其他情况下返回 `true`, `res_continuation` 在 `sc` 失败时返回 `false`, 否则返回 `true`）。



例如，给定加载指令 `lw x1, 0(x2)`, 其执行通常如下。初始执行状态将从给定操作码的伪代码中计算得出。这可以预期为 `Read_reg(x2, _read_continuation_)`。将寄存器 `x2` 的最新写入值（如果寄存器值尚不可用，指令语义将被阻塞，直到该值可用）传递给 `_read_continuation_`, 假设该值为 `0x4000`, 返回 `Load_mem(plain_load, 0x4000, 4, _load_continuation_)`。将从内存位置 `0x4000` 加载的 4 字节值（假设为 `0x42`），传递给 `_load_continuation_`, 返回 `Write_reg(x1, 0x42, Done)`。在上述状态之前和之间，可能会出现多个 `Internal(_next_state_)` 状态。

请注意，内存写入过程分为两个步骤：Store_ea 和 Store_memv。Store_ea 使得存储的内存足迹暂时可知，而 Store_memv 则负责将实际值存入内存。这两个步骤在伪代码中是配对的（Store_ea 紧跟着 Store_memv），但它们之间可能插入其他操作。



可以观察到，Store_ea 操作可以在存储值确定之前执行。例如，为了支持试纸测试 LB+fence.r.rw+data-po（与 RVWMO 行为一致），Hart 1 中的第一个存储操作必须在其值确定之前执行 Store_ea 步骤，以确保第二个存储操作看到的是写入一个不重叠内存区域，从而允许第二个存储操作在不违反一致性规则的情况下乱序提交。

每条指令的伪代码最多执行一次存储或加载，除了执行恰好一次加载和一次存储的 AMO（原子内存操作）。然后，这些内存访问随后被硬件线程语义分割成体系结构上原子化的单元，（参见 initiate_load，启动内存加载操作 和 initiate_store_footprint，启动内存存储操作的足迹）。

非正式地，寄存器读取的每一位应从程序顺序中最接近的（能够写入该位的）指令实例的寄存器写入中获取（如果没有这样的写入，则由硬件线程的初始寄存器状态提供）。因此，理解每个指令实例的寄存器写入足迹至关重要，我们在指令实例创建时计算该足迹（参见下文的 fetch，取指 操作）。我们确保每条指令在伪代码中对每个位寄存器最多执行一次写入，并避免读取刚写入的寄存器值。

模型中的数据流依赖（包括地址和数据）基于以下事实：每次寄存器读取都必须等到相应的寄存器写入完成（如上所述）。

B.3.2 指令实例状态

每个指令实例 i 的状态包括：

- * programloc，指令被提取的内存地址；
 - * instruction_kind，标识这是加载、存储、AMO、fence、分支/跳转还是一个 simple 指令（这也包括类似于伪代码执行状态中描述的 kind）；
 - * src_regs，从指令的伪代码静态确定的源寄存器集 reg_name（包括系统寄存器）；
 - * dst_regs，从指令的伪代码静态确定的目标寄存器集 reg_name（包括系统寄存器）；
 - * pseudocode_state（有时简称为 state），其中之一（这是一个标签联合；标签为小写字母）：
- +
- * reg_reads，指令实例已执行的寄存器读取操作，包括每次读取操作所读取的寄存器写入切片；
 - * reg_writes，指令实例已执行的寄存器写入操作；
 - * mem_loads，一组内存加载操作，对于每个操作，记录尚未满足的切片（即尚未满足的字节索引），以及已满足的切片所对应的存储切片（每个存储切片包括一个内存存储操作及其字节索引子集组成）。
 - * mem_stores，一组内存存储操作，对于每个操作，有一个标志指示它是否已传播（已传递到共享内存）；
 - * 记录实例是否已提交、完成等信息。

Plain(isa_state)	准备进行伪代码转换
Pending_mem_loads	请求内存加载操作
Pending_mem_stores	请求内存存储操作

每个内存加载操作包括一个内存足迹（地址和大小）。每个内存存储操作包括一个内存足迹，并且在可用时包括一个值。

对于一个具有非空 mem_loads 的加载指令实例，如果它的所有加载操作都已满足（即没有未满足的加载切片），则该指令实例被称为完全满足。

非正式地，如果加载指令提供的源寄存器的值已完成，则称指令实例具有完全确定的数据。类似地，

如果加载指令提供其内存操作地址寄存器的值已完成，则称其具有完全确定的内存足迹。正式地，我们首先定义完全确定的寄存器写入的概念：如果满足以下条件之一，则来自 `reg_writes` 的寄存器写入 令实例被认为是完全确定的：

写入的值不受所执行的内存操作影响（即从内存加载的值或 `sc` 的结果），并且对于每一个影响的每个寄存器读取，所读取的寄存器写入是完全确定的（或者从初始寄存器状态读取）。

现在，如果对于 `_reg_reads_` 中的每个寄存器读取，读取的寄存器写入是完全确定的，则称指令实例具有完全确定的数据。如果对于 `_reg_reads_` 中的每个寄存器读取，读取的寄存器写入是完全确定的，则称指令实例具有完全确定的内存足迹。



`rmem` 工具会记录每次寄存器写入操作，并为每个写入记录该指令所依赖的寄存器写入集。通过合理安排工具覆盖的指令伪代码，我们确保记录的寄存器写入集正是该写入所依赖的。

B. 3. 3 硬件线程状态

单个硬件线程的模型状态包括：

- * `hart_id`，硬件线程的唯一标识符；
- * `initial_register_state`，每个寄存器的初始寄存器值；
- * `initial_fetch_address`，初始指令获取地址；
- * `instruction_tree`，已获取（且未丢弃）的指令实例树，按程序顺序排列。

B. 3. 4 共享内存状态

共享内存的模型状态包含一个内存存储操作列表，按照它们传播到共享内存的顺序排列。

每当存储操作传播到共享内存时，它会被添加到列表末尾。加载操作从内存中获取数据时，返回与加载字节对应的最近存储切片。



在大多数情况下，视共享内存为一个数组较为简单，即将内存位置映射到最近的内存存储操作切片，其中每个位置映射到对应字节切片。然而，这种抽象不足以处理 `sc` 指令。`RVWMO` 允许来自同一硬件线程的存储操作在 `sc` 存储操作和配对的 `lr` 读取操作之间进行干预。为了实现这一点，并避免其他存储操作的干预，数组抽象需要扩展以记录更多信息。尽管我们选择使用列表作为实现方式，它既简单又直观，但更高效的实现可能需要其他数据结构。

B. 3. 5 转换

以下各段描述了一种系统转换类型。描述从当前系统状态的条件开始。只有在满足条件时，才能在当前状态下进行转换。条件之后是一个操作，当该操作在执行转变时应用于当前状态，以生成新的系统状态。

取指令

如果满足以下条件，指令实例的一个可能的程序顺序后继可以从地址 `loc` 取指令：

- 它尚未被取指，即硬件线程的 `instructiontree` 中，任何直接后继都不是来自 `loc`；
- 如果 `is` 的伪代码已经将地址写入 `pc`，则 `loc` 必须是该地址，否则 `loc` 是：
- * 对于条件跳转，后继地址或分支目标地址；
- * 对于（直接）跳转并链接指令（`jal`），目标地址；

- * 对于间接跳转指令 (jalr)，任何地址；
- * 对于任何其他指令，`i.program_loc+4`。

操作：为程序内存中位于 `loc` 处的指令构造一个新初始化的指令实例 `is`，其状态为 `Plain(isa_state)`，该状态是从指令伪代码计算得出的，包括从伪代码中获得的静态信息，如 `instruction_kind`、`src_regs` 和 `dst_regs`，并将 `is` 作为后继添加到硬件线程的 `instruction_tree` 中。

可能的下一个取指地址 (`loc`) 在获取后立即可用，模型不需要等待伪代码写入 `pc`，从而支持乱序执行，并使得条件分支和跳转指令的预测成为可能。大多数指令的地址可以直接从伪代码中得出，唯一的例外是间接跳转指令 (jalr)，其地址依赖于寄存器的值。数学模型理论上应允许在此处猜测任意地址。`rmem` 工具中的穷举搜索通过反复迭代来处理这个问题，每次扩展间接跳转的可能下一个取值地址集合。初始搜索使用空集，意味着间接跳转指令执行后，取指操作将推迟，直到该指令的伪代码写入 `pc`，然后使用该值来确定下一条指令。在每次搜索迭代开始前，我们收集前一次搜索中每个间接跳转写入 `pc` 的所有值，并将其作为可能的下一个取指地址。如果没有新地址被发现，搜索结束。

启动内存加载操作

状态为 `Plain(Load_mem(kind, address, size, load_continuation))` 的指令实例 `i` 始终可以启动相应的内存加载操作。操作如下：

- . 构造适当的内存加载操作 `mlos`：
 - * 如果 `address` 对齐到 `size`，则 `mlos` 是一个从 `address` 开始的大小为 `size` 字节的单个内存加载操作；
 - * 否则，`mlos` 是一组 `size` 个内存加载操作，每个操作作为一个字节，地址范围为 `address...address+1`。
- . 将 `mem_loads` 设置为 `mlos`；
- . 将状态更新为 `Pending_mem_loads`。



在 `rvwmo-primitives` 中提到，未对齐的内存访问可以在任何粒度下进行分解。这里我们将其分解为单字节访问，因为这种粒度包含了所有其他粒度。

通过转发未传播的存储满足内存加载操作

对于状态为 `Pending_mem_loads` 的非 AMO 加载指令实例 `i`，以及在 `i.memloads_loads` 中具有未满足切片的内存加载操作 `mlos`，如果满足以下条件，则该内存加载操作可以通过从未传播的内存存储操作中转发以部分或完全满足，这些存储操作来自在程序顺序中位于 `i` 之前的存储指令实例：

- . 所有程序顺序中的 `fence` 指令，且其 `.sr` 和 `.pw` 标志已设置，均已完成；
- . 对于每个程序顺序中的 `fence` 指令 `f`，其 `.sr` 和 `.pr` 标志已设置且 `.pw` 标志未设置，如果 `f` 尚未完成，则所有程序顺序之前的加载指令都已完全满足；
- . 对于每个程序顺序之前的 `fence.tso` 指令 `f`，如果它尚未完成，则所有在 `f` 之前的加载指令必须完全满足；
- . 如果 `i` 是一个 `load-acquire-RCsc` 指令，则所有程序顺序中的先前 `store-releases-RCsc` 指令必须已完成；
- . 如果 `i` 是一个 `load-acquire-release` 指令，则所有程序顺序中的先前指令必须已完成；
- . 所有未完成的程序顺序中的 `load-acquire` 指令必须完全满足；
- . 所有程序顺序中的先前 `store-acquire-release` 指令必须已完成；

令 `msoss` 成为所有尚未传播的内存存储操作切片集合，这些切片来自程序顺序中位于 `i` 之前的非 `sc` 存储指令实例，且这些存储指令已经计算出要存储的值。它们与 `mlo` 的未满足切片重叠，并且没有被插入的存储操作或被插入的加载操作所覆盖。最后一个条件要求，对于来自指令 `i` 的 `msoss` 中每个内存存储操作切片 `msos`：

- * 必须保证在 `i` 和 `i` 之间没有存储指令，其内存存储操作与 `msos` 重叠；
- * 必须保证在 `i` 和 `i` 之间没有加载指令，其满足条件的内存存储切片来自于不同的硬件线程的重叠内存存储操作。

操作：

- . 更新 `i.mem_loads`，表示 `mlo` 已通过 `msoss` 被满足；
- . 重启因违反一致性而需要重启的任何推测性指令，即，对于每个未完成的指令 `i`（它是 `i` 的程序顺序后继），并且对于每个内存加载操作 `mlo`（它是通过 `msoss` 满足的），如果在 `msoss` 中存在一个内存存储操作切片 `msos`，并且存在来自 `msoss` 的重叠内存存储操作切片，并且 `msos` 不是来自 `i` 的程序顺序后继的指令，则重启 `i` 及其 `restart-dependents`。

其中，指令 `j` 的 `restart-dependents` 是：

- * 程序顺序后继指令 `j`，并且它们对 `j` 的寄存器写入操作具有数据流依赖关系；
- * 程序顺序后继指令 `j`，并且它们有一个内存存储操作从 `j` 内存存储操作读取数据（通过转发）；
- * 如果 `j` 是一个 `load-acquire` 指令，则 `j` 的所有程序顺序后继；
- * 如果 `j` 是一个 `load` 指令，对于每个具有 `.sr` 和 `.pr` 标志且 `.pw` 标志未设置的 `fence` 指令 `f`，如果它是 `j` 的程序顺序后继，那么 `f` 的所有加载指令；
- * 如果 `j` 是一个 `load` 指令，对于每个 `fence.tso` 指令 `f`，如果它是 `lj` 的程序顺序后继，那么 `f` 的所有加载指令；
- * （递归地）上述所有指令实例的所有 `restart-dependents`。

内存存储操作转发给内存加载可能仅满足加载的部分切片，剩下的切片可能仍未满足。



一个程序顺序之前的存储操作，在上述转换时不可用，但当它变得可用时，可能会使 `msoss` 暂时不可靠（违反一致性）。该存储操作会阻止加载指令完成（见 `finish`，取指），并会在该存储操作传播时导致加载指令重新开始（见 `prop_store`，传播存储操作）。

上述转换条件的一个后果是，`store-release-RCsc` 内存存储操作不能转发给 `load-acquire-RCsc` 指令：`msoss` 不包括已完成存储的内存存储操作（因为那些必须是传播的内存存储操作），并且上述条件要求所有程序顺序之前的 `store-releases-RCsc` 操作在加载为 `acquire-RCsc` 时必须完成。

从内存满足内存加载操作

对于非 `AMO` 加载指令或 `AMO` 指令的指令实例 `i`，在 `AMO` 上下文的 `do_amo`，使满足、提交并传播 `AMO` 的操作转换中，任何 `i.mem_loads` 中的内存加载操作 `mlo`，如果其有未满足的片段，且满足 `sat_by_forwarding`，通过从未传播的存储中转发来满足内存加载操作的所有条件，则可以通过从内存中转发未传播的存储来满足。操作：令 `msoss` 为覆盖 `mlo` 未满足片段的内存存储操作片段，并应用 `do_amo`，通过从未传播的存储中转发来满足内存操作的操作。



请注意，do_amo，通过从未传播的存储中转发来满足内存操作 可能会留下内存加载操作的某些片段未满足，这些未满足的片段将需要通过再次进行转换，或通过执行 sat_from_mem，从内存中满足内存加载操作 来满足。另一方面，sat_from_mem，从内存中满足内存加载操作 将始终满足内存加载操作的所有未满足片段。

在状态 Pending_mem_loads(load_continuation) 下，如果所有内存加载操作 i.textit.mem_loads 都完全满足（即没有未满足的片段），则可以完成加载指令实例 i（与完成状态不同）。操作：将 i 的状态更新为 Plain(load_continuation(mem_value))，其中 mem_value 是由满足 i.textit.mem_loads 的所有内存存储操作片段组合而成。

提前 sc 失败在状态 Plain(Early_sc_fail(res_continuation)) 下，sc 指令实例 i 总是可以使其失败。操作：将 i 的状态更新为 Plain(res_continuation(false))。

配对 sc 在状态 Plain(Early_sc_fail(res_continuation)) 下，如果 i 与 lr 配对，则 sc 指令实例 i 可以继续其（可能成功的）执行。操作：将 i 的状态更新为 Plain(res_continuation(true))。

在状态 Plain(Store_ea(kind, address, size, next_state)) 下，指令实例 i 总是可以宣布其待处理的内存存储操作足迹。操作：

- 构造适当的内存存储操作 msos（不包括存储值）：

- * 如果 address 与 size 对齐，则 msos 是一个大小为 size 字节的内存存储操作，目标地址为 address；

- * 否则，msos 是一个由 size 个内存存储操作组成的集合，每个操作的大小为 1 字节，目标地址为 address...address+size-1。

- 将 i.textit.mem_loads 设置为 msos；

- 并将 i 的状态更新为 Plain(next_state)。



请注意，在执行上述转换后，内存存储操作仍然没有其值。将此转换与后续转换分开执行的目的是，它允许程序顺序中的其他存储指令在数据寄存器值可用之前观察到该指令的内存足迹，并在没有重叠的情况下尽早按乱序方式传播。

在状态 Plain(Store_memv(mem_value, store_continuation)) 下，指令实例 i 总是可以实例化内存存储操作 i.mem_stores 的值。操作：

- 将 mem_value 在内存存储操作 i.mem_stores 之间分配；

- 将 i 的状态更新为 Pending_mem_stores(store_continuation)。

在状态 Pending_mem_stores(store_continuation) 下，非 sc 存储指令或在 commit_sc 提交并传播 sc 的存储操作 转换上下文中的 sc 指令实例 i，如果满足以下条件，则可以提交（与传播不同）：

- i 具有完全确定的数据；

- 所有程序顺序之前的条件分支和间接跳转指令都已完成；

- 所有程序顺序之前设置了 .sw 的 fence 指令都已完成；

- 所有程序顺序之前的 fence.tso 指令都已完成；

- 所有程序顺序之前的 load-acquire 指令都已完成；

- 所有程序顺序之前的 store-acquire-release 指令都已完成；

- 如果 i 是一个 store-release 指令，则所有程序顺序之前的指令都已完成；

- 所有程序顺序之前的内存访问指令具有完全确定的内存足迹；

- 所有程序顺序之前的存储指令（失败的 sc 除外）都已启动，因此具有非空的 mem_stores；

- 所有程序顺序之前的加载指令都已启动，因此具有非空的 mem_loads。



请注意，如果条件 `commit_stores, 8` 得到满足，那么条件 `commit_stores, 9` 和 `commit_stores, 10` 也会得到满足，或者在执行一些急切的转换后得到满足。因此，要求这些条件并不会增强模型的严密性。通过要求它们，我们可以保证之前的内存访问指令已经经过足够的转换，使它们的内存操作在检查条件时是可见的，这也是该指令将要执行的下一个转换，从而简化了该条件。

对于处于 `Pending_mem_stores(store_continuation)` 状态的已提交指令实例 `i`，以及在 `i.textit.mem_loads` 中未传播的内存存储操作 `mso`，只有在以下情况下 `mso` 才能被传播：

- 所有与 `mso` 重叠的程序顺序中前置存储指令的内存存储操作已经传播完成；
 - 所有与 `mso` 重叠的程序顺序中前置加载指令的内存加载操作已经满足，并且（这些加载指令）是不可重启的（见下文定义）；
 - 所有通过转发 `mso` 满足的内存加载操作都已完全满足。
- 一个未完成的指令实例 `j` 是 不可重启的，当且仅当：
- 不存在存储指令 `s` 和 `s` 的未传播内存存储操作 `mso`，使得对 `mso` 执行 `prop_store`，传播存储操作 转换的操作会导致 `j` 被重启；
 - 不存在未完成的加载指令 `l` 和 `l` 的内存加载操作 `mlo`，使得对 `mlo` 执行 `sat_by_forwarding`，通过从未传播的存储中转发来满足内存加载操作/`sat_from_mem`，从内存中满足内存加载操作 转换的操作（即使 `mlo` 已经满足）会导致 `j` 被重启。

操作：

- 使用 `mso` 更新共享内存状态；
- 更新 `i.textit.mem_loads`，以表明 `mso` 已被传播；
- 重新启动任何由于此操作而违反一致性的推测指令，即，对于每个程序顺序在 `i` 之后且未完成的指令 `i`，以及每个满足自 `msoss` 的内存加载操作 `mlo`，如果存在一个与 `mso` 重叠的内存存储操作切片 `msos`，且该切片不是来自 `mso`，并且 `msos` 不是来自 `i` 的程序顺序后继指令，则重新启动 `i` 及其 `restart-dependents`（见 `sat_by_forwarding`，通过从未传播的存储中转发来满足内存加载操作）。

如果满足以下条件，则可以同时提交和传播一个未提交的 `sc` 指令实例 `i`（来自硬件线程 `h`），其状态为 `Pending_mem_stores(store_continuation)`，并且具有一个由一些存储切片 `msoss` 满足的配对 `lr` 指令实例 `i`：

- `i` 已完成；
- 每个已转发到 `i` 的内存存储操作都已传播；
- 满足 `commit_stores`，提交存储指令 的条件；
- 满足 `prop_store`，传播存储指令 的条件（注意 `sc` 指令只能有一个内存存储操作）；
- 对于来自 `msoss` 的每个存储切片 `msos`，自从 `msos` 被传播到内存以来，`msos` 在任何时刻都没有被来自非 `h` 的硬件线程的存储在共享内存中覆盖。

操作：

- 执行 `commit_stores`，提交存储指令 的操作；
- 执行 `prop_store`，传播存储指令 的操作。

延迟 `sc` 失败

在状态为 `Pending_mem_stores(store_continuation)` 的 `sc` 指令实例 `i` 中，如果其内存存储操作尚未传播，则始终可以使其失败。操作：

- 清除 `i.textit.mem_loads`；
- 将 `i` 的状态更新为 `Plain(store_continuation(false))`。



为了提高效率, rmem 工具仅在无法进行 commit_sc, 提交并传播一个 sc 的存储操作 转换时允许此转换。这不会影响允许的最终状态集, 但在交互式探索时, 如果 sc 失败, 则应使用 early_sc_fail, 早期 sc 失败 转换, 而不是等待此转换。

在状态为 Pending_mem_stores(store_continuation) 的存储指令实例 i 中, 所有内存存储操作 (即 i.textit.mem_loads) 都已传播完毕, 因此该指令实例始终可以完成 (不应与完成混淆)。操作: 将 i 的状态更新为 Plain(store_continuation(true))。

处于 Pending_mem_loads(load_continuation) 状态的 AMO 指令实例 i 可以执行其内存访问操作, 如果可以执行以下一系列转换而没有任何中间转换:

- . sat_from_mem, 从内存中满足内存加载操作
- . complete_loads, 完成加载操作
- . sail_interp, 伪代码内部步骤 (零次或多次)
- . instantiate_store_value, 实例化内存存储操作的值
- . commit_stores, 提交存储指令
- . prop_store, 传播存储操作
- . complete_stores, 完成存储操作

此外, 除不要求 i 处于 Plain(Done) 状态外, finish, 完成指令 的条件在这些转换之后依然成立。操作: 依次执行上述转换 (不包括 finish, 完成指令), 且没有任何中间转换。

请注意, 程序顺序中的前置存储操作不能被转发到 AMO 的加载操作。这是因为上述转换序列没有包括转发转换。但即使它包括转发转换, 当尝试进行 prop_store, 传播存储操作 转换时, 序列也会失败, 因为该转换要求所有程序顺序中前置的存储操作与重叠的内存足迹必须被传播, 而转发要求存储操作未传播。



此外, AMO 的存储也不能转发到程序顺序的后继加载操作。在执行上述转换之前, AMO 的存储操作没有其值, 因此不能进行转发; 在执行上述转换之后, 存储操作已经被传播, 因此也不能进行转发。

提交 fence 指令

处于 Plain(Fence(kind, next_state)) 状态的 fence 指令实例 i 可以被提交, 前提是:

- . 如果 i 是一个普通的 fence 且设置了 .pr, 则所有程序顺序中前置的加载指令都已完成;
- . 如果 i 是一个普通的 fence 且设置了 .pw, 则所有程序顺序中前置的存储指令都已完成;
- . 如果 i 是一个 fence.tso, 则所有程序顺序中前置的加载和存储指令都已完成。

操作:

记录 i 已被提交;

将 i 的状态更新为 Plain(next_state)。

处于 Plain(Read_reg(reg_name, read_cont)) 状态的指令实例 i 可以执行 reg_name 寄存器读取操作, 如果它需要读取的每个指令实例已经执行了预期的 reg_name 寄存器写入操作。

让 read_sources 包括, 对于 reg_name 的每一位, 由能够写入该位的最新 (按程序顺序) 指令实例的写操作, 如果有的话。如果没有这样的指令, 源就是来自 initial_register_state 的初始寄存器值。让 reg_value 是从 read_sources 组装得到的值。操作:

将 reg_name 添加到 i.textit.reg_read, 并与 read_sources 和 reg_value 一同记录;

将 i 的状态更新为 Plain(_read_cont(reg_value))。

处于 Plain(Write_reg(reg_name, reg_value, _next_state_)) 状态的指令实例 i 始终可以执行 reg_name 寄存器写入操作。操作:

将 `reg_name` 添加到 `i.textit.reg_writes`, 并与 `deps` 和 `reg_value` 一同记录;

将 `i` 的状态更新为 `Plain(_next_state_)`。

其中 `deps` 是一对来自 `i.textit.reg_read` 的所有 `read_sources` 的集合, 以及一个标志, 该标志在 `i` 是已完全满足的加载指令实例时为真。

处于 `Plain(Internal(_next_state_))` 状态的指令实例 `i` 始终可以执行该伪代码内部步骤。操作: 将 `i` 的状态更新为 `Plain(_next_state_)`。

一个处于 `Plain(Done)` 状态的未完成指令实例 `i` 可以被完成, 当且仅当:

如果 `i` 是一个加载指令, 则可以完成, 前提是:

所有程序顺序之前的 `load-acquire` 指令都已完成;

所有程序顺序中前置的 `.sr` 的 `fence` 指令都已完成;

对于每个未完成的程序顺序中前置的 `fence.tso` 指令 `f`, 所有在 `f` 之前的加载指令都已完成;

保证 `i` 的内存加载操作所读取的值不会引发一致性冲突, 即, 对于任何程序顺序中前置的指令实例 `i`, 令 `cfp` 为从指令 `i` 和 `i` 之间的存储指令传播的内存存储操作的组合足迹, 以及从指令 `i` 和 `i` 之间的存储指令转发到 `i` 的固定内存存储操作, 包括 `i`, 并令 `cfp` 为 `cfp` 在 `i` 的内存足迹中的补集。

如果 `cfp` 非空:

`i` 具有完全确定的内存足迹;

`i` 没有与 `cfp` 重叠的未传播内存存储操作;

如果 `i` 是一个加载指令, 并且其内存足迹与 `cfp` 重叠, 则所有与 `cfp` 重叠的内存加载操作都已满足, 并且 `i` 是不可重启的 (参见 `prop_store`, 传播存储操作 转换以确定指令是否不可重启)。

这里, 如果存储指令具有完全确定的数据, 则称内存存储操作为固定。

`i` 具有完全确定的数据; 并且

如果 `i` 不是 `fence` 指令, 所有程序顺序中前置的条件分支和间接跳转指令都已完成。

操作:

如果 `i` 是条件分支或间接跳转指令, 则丢弃任何未被采取的执行路径, 即移除所有在 `_instruction_tree_` 中通过分支/跳转未被执行到的指令实例;

并且将该指令记录为已完成, 即将 `_finished_` 设置为 `true`。

B.3.6 限制

* 该模型支持 RV64I 和 RV64A 用户级, 但不支持不对齐的原子粒度 PMA 或总存储排序扩展 Ztso。适配 RV32I/A 以及 G、Q 和 C 扩展应该相对简便, 但我们尚未进行尝试。此适配主要包括为指令编写 Sail 代码, 且对并发模型的更改极少, 甚至没有。

* 该模型仅支持常规内存访问 (不包括 I/O 访问)。

* 该模型不考虑 TLB 相关效应。

* 该模型假设指令内存是固定的。`fetch`, 取指 转换不会生成内存加载操作, 且共享内存不参与此过程。模型依赖外部预言机, 根据内存位置返回操作码。

* 该模型不涵盖异常、陷阱和中断。

附录 C: 正式内存模型规范

C.1 向量-向量加法示例

```
# vector-vector add routine of 32-bit integers
# void vvaddint32(size_t n, const int*x, const int*y, int*z)
```

```

# { for (size_t i=0; i<n; i++) { z[i]=x[i]+y[i]; } }
#
# a0 = n, a1 = x, a2 = y, a3 = z
# Non-vector instructions are indented
vvaddint32:
    vsetvli t0, a0, e32, ta, ma # Set vector length based on 32-bit vectors
    vle32.v v0, (a1) # Get first vector
    sub a0, a0, t0 # Decrement number done
    slli t0, t0, 2 # Multiply number done by 4 bytes
    add a1, a1, t0 # Bump pointer
    vle32.v v1, (a2) # Get second vector
    add a2, a2, t0 # Bump pointer
    vadd.vv v2, v0, v1 # Sum vectors
    vse32.v v2, (a3) # Store result
    add a3, a3, t0 # Bump pointer
    bnez a0, vvaddint32 # Loop back
    ret # Finished

```

C.2 混合位宽掩码与计算示例

```

# Code using one width for predicate and different width for masked
# compute.
# int8_t a[]; int32_t b[], c[];
# for (i=0; i<n; i++) { b[i] = (a[i] < 5) ? c[i] : 1; }
#
# Mixed-width code that keeps SEW/LMUL=8
loop:
    vsetvli a4, a0, e8, m1, ta, ma # Byte vector for predicate calc
    vle8.v v1, (a1) # Load a[i]
    add a1, a1, a4 # Bump pointer.
    vmslt.vi v0, v1, 5 # a[i] < 5?
    vsetvli x0, a0, e32, m4, ta, mu # Vector of 32-bit values.
    sub a0, a0, a4 # Decrement count
    vmv.v.i v4, 1 # Splat immediate to destination
    vle32.v v4, (a3), v0.t # Load requested elements of C, others undisturbed
    sll t1, a4, 2
    add a3, a3, t1 # Bump pointer.
    vse32.v v4, (a2) # Store b[i].
    add a2, a2, t1 # Bump pointer.
    bnez a0, loop # Any more?

```

C.3 内存复制 (Memcpy) 示例

```

# void *memcpy(void* dest, const void* src, size_t n)

```

```

# a0=dest, a1=src, a2=n
#
memcpy:
mv a3, a0 # Copy destination
loop:
vsetvli t0, a2, e8, m8, ta, ma # Vectors of 8b
vle8.v v0, (a1) # Load bytes
add a1, a1, t0 # Bump pointer
sub a2, a2, t0 # Decrement count
vse8.v v0, (a3) # Store bytes
add a3, a3, t0 # Bump pointer
bnez a2, loop # Any more?
ret # Return

```

C.4 条件运算示例

```

# (int16) z[i] = ((int8) x[i] < 5) ? (int16) a[i] : (int16) b[i];
#
loop:
vsetvli t0, a0, e8, m1, ta, ma # Use 8b elements.
vle8.v v0, (a1) # Get x[i]
sub a0, a0, t0 # Decrement element count
add a1, a1, t0 # x[i] Bump pointer
vmslt.vi v0, v0, 5 # Set mask in v0
vsetvli x0, x0, e16, m2, ta, mu # Use 16b elements.
slli t0, t0, 1 # Multiply by 2 bytes
vle16.v v2, (a2), v0.t # z[i] = a[i] case
vmnot.m v0, v0 # Invert v0
add a2, a2, t0 # a[i] bump pointer
vle16.v v2, (a3), v0.t # z[i] = b[i] case
add a3, a3, t0 # b[i] bump pointer
vse16.v v2, (a4) # Store z
add a4, a4, t0 # z[i] bump pointer
bnez a0, loop

```

C.5 SAXPY（线性代数运算）示例

```

# void
# saxpy(size_t n, const float a, const float *x, float *y)
# {
# size_t i;
# for (i=0; i<n; i++)
# y[i] = a * x[i] + y[i];
# }

```

```

#
# register arguments:
# a0 n
# fa0 a
# a1 x
# a2 y
saxpy:
    vsetvli a4, a0, e32, m8, ta, ma
    vle32.v v0, (a1)
    sub a0, a0, a4
    slli a4, a4, 2
    add a1, a1, a4
    vle32.v v8, (a2)
    vfmacc.vf v8, fa0, v0
    vse32.v v8, (a2)
    add a2, a2, a4
    bnez a0, saxpy
    ret

```

C.6 SGEMM（矩阵乘法）示例

```

# RV64IDV system
#
# void
# sgemm_nn(size_t n,
# size_t m,
# size_t k,
# const float*a, // m * k matrix
# size_t lda,
# const float*b, // k * n matrix
# size_t ldb,
# float*c, // m * n matrix
# size_t ldc)
#
# c += a*b (alpha=1, no transpose on input matrices)
# matrices stored in C row-major order
#define n a0
#define m a1
#define k a2
#define ap a3
#define astride a4
#define bp a5
#define bstride a6
#define cp a7

```



```

#define cstride t0
#define kt t1
#define nt t2
#define bnp t3
#define cnp t4
#define akp t5
#define bkp s0
#define nvl s1
#define ccp s2
#define amp s3
# Use args as additional temporaries
#define ft12 fa0
#define ft13 fa1
#define ft14 fa2
#define ft15 fa3
# This version holds a 16*VLMAX block of C matrix in vector registers
# in inner loop, but otherwise does not cache or TLB tiling.
sgemm_nn:
    addi sp, sp, -FRAMESIZE
    sd s0, OFFSET(sp)
    sd s1, OFFSET(sp)
    sd s2, OFFSET(sp)
    # Check for zero size matrices
    beqz n, exit
    beqz m, exit
    beqz k, exit
    # Convert elements strides to byte strides.
    ld cstride, OFFSET(sp) # Get arg from stack frame
    slli astride, astride, 2
    slli bstride, bstride, 2
    slli cstride, cstride, 2
    slti t6, m, 16
    bnez t6, end_rows
c_row_loop: # Loop across rows of C blocks
    mv nt, n # Initialize n counter for next row of C blocks
    mv bnp, bp # Initialize B n-loop pointer to start
    mv cnp, cp # Initialize C n-loop pointer
c_col_loop: # Loop across one row of C blocks
    vsetvli nvl, nt, e32, ta, ma # 32-bit vectors, LMUL=1
    mv akp, ap # reset pointer into A to beginning
    mv bkp, bnp # step to next column in B matrix
    # Initialize current C submatrix block from memory.
    vle32.v v0, (cnp); add ccp, cnp, cstride;
    vle32.v v1, (ccp); add ccp, ccp, cstride;

```

```

vle32.v v2, (ccp); add ccp, ccp, cstride;
vle32.v v3, (ccp); add ccp, ccp, cstride;
vle32.v v4, (ccp); add ccp, ccp, cstride;
vle32.v v5, (ccp); add ccp, ccp, cstride;
vle32.v v6, (ccp); add ccp, ccp, cstride;
vle32.v v7, (ccp); add ccp, ccp, cstride;
vle32.v v8, (ccp); add ccp, ccp, cstride;
vle32.v v9, (ccp); add ccp, ccp, cstride;
vle32.v v10, (ccp); add ccp, ccp, cstride;
vle32.v v11, (ccp); add ccp, ccp, cstride;
vle32.v v12, (ccp); add ccp, ccp, cstride;
vle32.v v13, (ccp); add ccp, ccp, cstride;
vle32.v v14, (ccp); add ccp, ccp, cstride;
vle32.v v15, (ccp)
mv kt, k # Initialize inner loop counter
# Inner loop scheduled assuming 4-clock occupancy of vfmacc instruction and
single-issue pipeline
# Software pipeline loads
flw ft0, (akp); add amp, akp, astride;
flw ft1, (amp); add amp, amp, astride;
flw ft2, (amp); add amp, amp, astride;
flw ft3, (amp); add amp, amp, astride;
# Get vector from B matrix
vle32.v v16, (bkp)
# Loop on inner dimension for current C block
k_loop:
vfmacc.vf v0, ft0, v16
add bkp, bkp, bstride
flw ft4, (amp)
add amp, amp, astride
vfmacc.vf v1, ft1, v16
addi kt, kt, -1 # Decrement k counter
flw ft5, (amp)
add amp, amp, astride
vfmacc.vf v2, ft2, v16
flw ft6, (amp)
add amp, amp, astride
flw ft7, (amp)
vfmacc.vf v3, ft3, v16
add amp, amp, astride
flw ft8, (amp)
add amp, amp, astride
vfmacc.vf v4, ft4, v16
flw ft9, (amp)

```

```

    add amp, amp, astride
    vmacc.vf v5, ft5, v16
    flw ft10, (amp)
    add amp, amp, astride
    vmacc.vf v6, ft6, v16
    flw ft11, (amp)
    add amp, amp, astride
    vmacc.vf v7, ft7, v16
    flw ft12, (amp)
    add amp, amp, astride
    vmacc.vf v8, ft8, v16
    flw ft13, (amp)
    add amp, amp, astride
    vmacc.vf v9, ft9, v16
    flw ft14, (amp)
    add amp, amp, astride
    vmacc.vf v10, ft10, v16
    flw ft15, (amp)
    add amp, amp, astride
    addi akp, akp, 4 # Move to next column of a
    vmacc.vf v11, ft11, v16
    beqz kt, 1f # Dont load past end of matrix
    flw ft0, (akp)
    add amp, akp, astride
1: vmacc.vf v12, ft12, v16
    beqz kt, 1f
    flw ft1, (amp)
    add amp, amp, astride
1: vmacc.vf v13, ft13, v16
    beqz kt, 1f
    flw ft2, (amp)
    add amp, amp, astride
1: vmacc.vf v14, ft14, v16
    beqz kt, 1f # Exit out of loop
    flw ft3, (amp)
    add amp, amp, astride
    vmacc.vf v15, ft15, v16
    vle32.v v16, (bkp) # Get next vector from B matrix, overlap loads with
jump stalls
    j k_loop
1: vmacc.vf v15, ft15, v16

# Save C matrix block back to memory
vse32.v v0, (cnp); add ccp, cnp, cstride;

```

```

vse32.v v1, (ccp); add ccp, ccp, cstride;
vse32.v v2, (ccp); add ccp, ccp, cstride;
vse32.v v3, (ccp); add ccp, ccp, cstride;
vse32.v v4, (ccp); add ccp, ccp, cstride;
vse32.v v5, (ccp); add ccp, ccp, cstride;
vse32.v v6, (ccp); add ccp, ccp, cstride;
vse32.v v7, (ccp); add ccp, ccp, cstride;
vse32.v v8, (ccp); add ccp, ccp, cstride;
vse32.v v9, (ccp); add ccp, ccp, cstride;
vse32.v v10, (ccp); add ccp, ccp, cstride;
vse32.v v11, (ccp); add ccp, ccp, cstride;
vse32.v v12, (ccp); add ccp, ccp, cstride;
vse32.v v13, (ccp); add ccp, ccp, cstride;
vse32.v v14, (ccp); add ccp, ccp, cstride;
vse32.v v15, (ccp)
# Following tail instructions should be scheduled earlier in free slots during C
block save.
# Leaving here for clarity.
# Bump pointers for loop across blocks in one row
slli t6, nv1, 2
add cnp, cnp, t6 # Move C block pointer over
add bnp, bnp, t6 # Move B block pointer over
sub nt, nt, nv1 # Decrement element count in n dimension
bnez nt, c_col_loop # Any more to do?
# Move to next set of rows
addi m, m, -16 # Did 16 rows above
slli t6, astride, 4 # Multiply astride by 16
add ap, ap, t6 # Move A matrix pointer down 16 rows
slli t6, cstride, 4 # Multiply cstride by 16
add cp, cp, t6 # Move C matrix pointer down 16 rows

slti t6, m, 16
beqz t6, c_row_loop
# Handle end of matrix with fewer than 16 rows.
# Can use smaller versions of above decreasing in powers-of-2 depending on code
size concerns.
end_rows:
# Not done.
exit:
ld s0, OFFSET(sp)
ld s1, OFFSET(sp)
ld s2, OFFSET(sp)
addi sp, sp, FRAME_SIZE
ret

```

C.7 除法近似计算示例

```
# v1 = v1 / v2 to almost 23 bits of precision.
vfreq7.v v3, v2 # Estimate 1/v2
    li t0, 0x40000000
vmv.v.x v4, t0 # Splat 2.0
vfnmsac.vv v4, v2, v3 # 2.0 - v2 * est(1/v2)
vfmul.vv v3, v3, v4 # Better estimate of 1/v2
vmv.v.x v4, t0 # Splat 2.0
vfnmsac.vv v4, v2, v3 # 2.0 - v2 * est(1/v2)
vfmul.vv v3, v3, v4 # Better estimate of 1/v2
vfmul.vv v1, v1, v3 # Estimate of v1/v2
```

C.8 平方根近似计算示例

```
# v1 = sqrt(v1) to almost 23 bits of precision.
    fmv.w.x ft0, x0 # Mask off zero inputs
vmfne.vf v0, v1, ft0 # to avoid div by zero
vfrsqrt7.v v2, v1, v0.t # Estimate 1/sqrt(x)
vmfne.vf v0, v2, ft0, v0.t # Additionally mask off +inf inputs
    li t0, 0x40400000
vmv.v.x v4, t0 # Splat 3.0
vfmul.vv v3, v1, v2, v0.t # x * est
vfnmsub.vv v3, v2, v4, v0.t # - x * est * est + 3
vfmul.vv v3, v3, v2, v0.t # est * (-x * est * est + 3)
    li t0, 0x3f000000
    fmv.w.x ft0, t0 # 0.5
vfmul.vf v2, v3, ft0, v0.t # Estimate to 14 bits
vfmul.vv v3, v1, v2, v0.t # x * est
vfnmsub.vv v3, v2, v4, v0.t # - x * est * est + 3
vfmul.vv v3, v3, v2, v0.t # est * (-x * est * est + 3)
vfmul.vf v2, v3, ft0, v0.t # Estimate to 23 bits
vfmul.vv v1, v2, v1, v0.t # x * 1/sqrt(x)
```

C.9 C 标准库 strcmp 函数示例

```
# int strcmp(const char *src1, const char* src2)
strcmp:
    ## Using LMUL=2, but same register names work for larger LMULs
    li t1, 0 # Initial pointer bump
loop:
    vsetvli t0, x0, e8, m2, ta, ma # Max length vectors of bytes
    add a0, a0, t1 # Bump src1 pointer
```

```

vle8ff.v v8, (a0) # Get src1 bytes
add a1, a1, t1 # Bump src2 pointer
vle8ff.v v16, (a1) # Get src2 bytes
vmseq.vi v0, v8, 0 # Flag zero bytes in src1
vmsne.vv v1, v8, v16 # Flag if src1 != src2
vmor.mm v0, v0, v1 # Combine exit conditions

vfirst.m a2, v0 # ==0 or != ?
csrr t1, v1 # Get number of bytes fetched

bltz a2, loop # Loop if all same and no zero byte
add a0, a0, a2 # Get src1 element address
lbu a3, (a0) # Get src1 byte from memory
add a1, a1, a2 # Get src2 element address
lbu a4, (a1) # Get src2 byte from memory
sub a0, a3, a4 # Return value.
ret

```

C.10 分数阶向量乘法 (Fractional Lmul) 示例

本附录提供一个非规范性示例，用以说明编译器如何有效利用分数阶LMUL (Fractional LMUL) 特性。

请看以下用C语言编写的（人为构造的）循环示例：

```

void add_ref(long N,
    signed char *restrict c_c, signed char *restrict c_a, signed char *restrict c_b,
    long *restrict l_c, long *restrict l_a, long *restrict l_b,
    long *restrict l_d, long *restrict l_e, long *restrict l_f,
    long *restrict l_g, long *restrict l_h, long *restrict l_i,
    long *restrict l_j, long *restrict l_k, long *restrict l_l,
    long *restrict l_m) {
    long i;
    for (i = 0; i < N; i++) {
        c_c[i] = c_a[i] + c_b[i]; // Note this char addition that creates a mixed type
situation
        l_c[i] = l_a[i] + l_b[i];
        l_f[i] = l_d[i] + l_e[i];
        l_i[i] = l_g[i] + l_h[i];
        l_l[i] = l_k[i] + l_j[i];
        l_m[i] += l_m[i] + l_c[i] + l_f[i] + l_i[i] + l_l[i];
    }
}

```

该示例循环由于需要处理大量输入变量和临时变量，导致寄存器压力较高。编译器识别出循环内存在两种数据类型：8位的char型和64位的long*型。若不采用分数阶LMUL机制，编译器将被迫对8位计算

使用LMUL=1,而对64位计算使用LMUL=8,以确保同一循环迭代内所有计算操作的元素数量相同。在LMUL=8模式下,寄存器分配器仅能使用4个寄存器。考虑到该循环需要处理大量64位变量和临时变量,编译器最终会生成大量溢出代码。以下代码展示了这一现象:

```
.LBB0_4: # %vector.body
    # =>This Inner Loop Header: Depth=1
    add s9, a2, s6
    vsetvli s1, zero, e8,m1,ta,mu
    vle8.v v25, (s9)
    add s1, a3, s6
    vle8.v v26, (s1)
    vadd.vv v25, v26, v25
    add s1, a1, s6
    vse8.v v25, (s1)
    add s9, a5, s10
    vsetvli s1, zero, e64,m8,ta,mu
    vle64.v v8, (s9)
    add s1, a6, s10
    vle64.v v16, (s1)
    add s1, a7, s10
    vle64.v v24, (s1)
    add s1, s3, s10
    vle64.v v0, (s1)
    sd a0, -112(s0)
    ld a0, -128(s0)
    vs8r.v v0, (a0) # Spill LMUL=8
    add s9, t6, s10
    add s11, t5, s10
    add ra, t2, s10
    add s1, t3, s10
    vle64.v v0, (s9)
    ld s9, -136(s0)
    vs8r.v v0, (s9) # Spill LMUL=8
    vle64.v v0, (s11)
    ld s9, -144(s0)
    vs8r.v v0, (s9) # Spill LMUL=8
    vle64.v v0, (ra)
    ld s9, -160(s0)
    vs8r.v v0, (s9) # Spill LMUL=8
    vle64.v v0, (s1)
    ld s1, -152(s0)
    vs8r.v v0, (s1) # Spill LMUL=8
    vadd.vv v16, v16, v8
    ld s1, -128(s0)
    vl8r.v v8, (s1) # Reload LMUL=8
```



```

vadd.vv v8, v8, v24
ld s1, -136(s0)
vl8r.v v24, (s1) # Reload LMUL=8
ld s1, -144(s0)
vl8r.v v0, (s1) # Reload LMUL=8
vadd.vv v24, v0, v24
ld s1, -128(s0)
vs8r.v v24, (s1) # Spill LMUL=8
ld s1, -152(s0)
vl8r.v v0, (s1) # Reload LMUL=8
ld s1, -160(s0)
vl8r.v v24, (s1) # Reload LMUL=8
vadd.vv v0, v0, v24
add s1, a4, s10
vse64.v v16, (s1)
add s1, s2, s10
vse64.v v8, (s1)
vadd.vv v8, v8, v16
add s1, t4, s10
ld s9, -128(s0)
vl8r.v v16, (s9) # Reload LMUL=8
vse64.v v16, (s1)
add s9, t0, s10
vadd.vv v8, v8, v16
vle64.v v16, (s9)
add s1, t1, s10
vse64.v v0, (s1)
vadd.vv v8, v8, v0
vsll.vi v16, v16, 1
vadd.vv v8, v8, v16
vse64.v v8, (s9)
add s6, s6, s7
add s10, s10, s8
bne s6, s4, .LBB0_4

```

若编译器可采用分数阶 $LMUL=1/2$ (而非 $LMUL=1$) 处理 8 位计算, 则 64 位计算可对应使用 $LMUL=4$ (注意: 64 位与 8 位元素的数量比仍保持与前一示例相同)。此时寄存器分配器可用寄存器数量增至 8 个, 从而完全消除溢出代码, 如下方循环所示:

```

.LBB0_4: # %vector.body
# =>This Inner Loop Header: Depth=1
add s9, a2, s6
vsetvli s1, zero, e8, mf2, ta, mu // LMUL=1/2 !
vle8.v v25, (s9)

```

```

add s1, a3, s6
vle8.v v26, (s1)
vadd.vv v25, v26, v25
add s1, a1, s6
vse8.v v25, (s1)
add s9, a5, s10
vsetvli s1, zero, e64,m4,ta,mu // LMUL=4
vle64.v v28, (s9)
add s1, a6, s10
vle64.v v8, (s1)
vadd.vv v28, v8, v28
add s1, a7, s10
vle64.v v8, (s1)
add s1, s3, s10
vle64.v v12, (s1)
add s1, t6, s10
vle64.v v16, (s1)
add s1, t5, s10
vle64.v v20, (s1)
add s1, a4, s10
vse64.v v28, (s1)
vadd.vv v8, v12, v8
vadd.vv v12, v20, v16
add s1, t2, s10
vle64.v v16, (s1)
add s1, t3, s10
vle64.v v20, (s1)
add s1, s2, s10
vse64.v v8, (s1)
add s9, t4, s10
vadd.vv v16, v20, v16
add s11, t0, s10
vle64.v v20, (s11)
vse64.v v12, (s9)
add s1, t1, s10
vse64.v v16, (s1)
vsll.vi v20, v20, 1
vadd.vv v28, v8, v28
vadd.vv v28, v28, v12
vadd.vv v28, v28, v16
vadd.vv v28, v28, v20
vse64.v v28, (s11)
add s6, s6, s7
add s10, s10, s8

```

```
bne s6, s4, .LBB0_4
```

附录 D：向量状态调用约定（非权威性 - 仅作占位说明）

本附录仅为占位说明，旨在帮助解释代码示例中使用的约定，尚未最终确定，也不属于正式标准化流程的一部分。官方的RISC-V psABI 文档正在扩展，以明确规定向量调用的相关规范。在 RISC-V psABI（处理器专用应用二进制接口）中，向量寄存器 `v0-v31` 均为调用者保存（`caller-saved`）。此外，`v1` 和 `vtype` 这两个控制状态寄存器（CSR）同样遵循调用者保存规则。函数调用入口可假定 `vstart` 为零。函数调用返回时亦可假定 `vstart` 为零。



应用软件通常不应显式写入 `vstart` 寄存器。任何将 `vstart` 显式设置为非零值的程序，必须在返回或调用其他程序前将 `vstart` 清零。

`vcsr` 寄存器中的 `vxrm` 和 `vxsat` 字段具有线程存储周期。执行系统调用会导致所有调用者保存的向量寄存器（`v0-v31`、`v1`、`vtype`）以及 `vstart` 的值变为未定义状态。



该机制允许引发上下文切换的系统调用无需保存及恢复向量寄存器状态。多数操作系统会选择保持这些寄存器内容不变，或将其重置为初始状态，以避免跨进程信息泄露。